

Kairos

Architecture and Micro-Architecture Reader

Print edition - generated 2026-08-01

This reader combines the current architecture chapters, product context, approved decision ledger, and every feature micro-architecture. The shared chapters in docs/arch are the operational source of truth. Feature documents may describe shipped work, a shadow, a proposal, a deferred idea, or a rejected direction; read their lifecycle language and any IMPLEMENTATION_RESULT.md before treating them as live behavior.

Reading route: overview -> agents and schedules -> providers and database -> safety -> learning -> the feature appendices that match the area you want to investigate.

Reader Map

Read Foundation and Operational Architecture in order. The Decision ledger explains why consequential choices were made. The final appendix contains every feature micro-architecture alphabetically; these are design records and may describe live work, a shadow, a proposal, a deferred item, or a retired direction.

Foundation

SYSTEM OVERVIEW

ARCHITECTURE

PRD

Operational Architecture

00-index

01-what-is-kairos

02-tech-stack

03-agents

04-database-schema

05-crons-and-scheduling

06-env-variables

07-coding-conventions

08-risk-and-safety

09-learning-loop

Decisions and Rationale

PROJECT DECISIONS

Feature Micro-Architecture Appendix

advanced learning - Feature Architecture

agent evolution - Feature Architecture

agent mind - Feature Architecture

agent source pipeline remediation - Feature Architecture

agent transparency debate - Feature Architecture

agentic quant platform - Feature Architecture

asset allocation - Feature Architecture

atr paper exit policy - Feature Architecture

automated strategy validation - Feature Architecture

benchmark alpha - Feature Architecture

briefing - Feature Architecture

capital rotation - Feature Architecture

correlation aware construction - Feature Architecture

cross sectional rank - Feature Architecture

data availability layer - Feature Architecture

data provider abstraction - Feature Architecture

data source policy - Feature Architecture

decision review - Feature Architecture

downside hedging - Feature Architecture

earnings aware risk - Feature Architecture

earnings repricing barrier - Feature Architecture

edge calibration readiness - Feature Architecture

edge factor discovery - Feature Architecture

event aware fundamentals adr - Feature Architecture

exogenous risk evidence - Feature Architecture

external research integrations - Feature Architecture

external research shadow - Feature Architecture

external skill observation - Feature Architecture

forecast calibration - Feature Architecture

health control and paper pnl - Feature Architecture

historical evidence intake - Feature Architecture

historical replay harness - Feature Architecture

holding risk daily - Feature Architecture

hybrid stop - Feature Architecture

india markets parity - Feature Architecture

international equity allocation - Feature Architecture

known anomalies - Feature Architecture

learning core - Feature Architecture

learning integrity - Feature Architecture

leveraged etf and intraday execution - Feature Architecture

live auto trading - Feature Architecture

live trading hardening - Feature Architecture

local historical replay - Feature Architecture

market local risk configuration - Feature Architecture

markets data - Feature Architecture

mentor agent - Feature Architecture

momentum factors - Feature Architecture

multi tenant - Feature Architecture

nav restructure - Feature Architecture

paper exit plan display - Feature Architecture

paper portfolio benchmarks - Feature Architecture

performance truth - Feature Architecture

pipeline data and timing - Feature Architecture

pit fundamentals - Feature Architecture

policy event ledger - Feature Architecture

relationship graph - Feature Architecture

research journal - Feature Architecture

research journal controls - Feature Architecture

research trade plan - Feature Architecture

research universe fix - Feature Architecture

risk research visibility - Feature Architecture

risk sector breach allocation - Feature Architecture

robinhood agentic evidence - Feature Architecture

robinhood mcp integration - Feature Architecture

router cutover - Feature Architecture

scoring data truth - Feature Architecture

scoring methodology - Feature Architecture

self healing agent - Feature Architecture

settings llm control - Feature Architecture

shadow registry - Feature Architecture

stock context - Feature Architecture

system health model resilience - Feature Architecture

system reference - Feature Architecture

technical factor calibration - Feature Architecture

trade behavior mirror - Feature Architecture

trading mandate - Feature Architecture

us paper fractional shares - Feature Architecture

walk forward ic folds - Feature Architecture

webull broker - Feature Architecture

webull trading api - Feature Architecture

weekend research catchup - Feature Architecture

Foundation

SYSTEM OVERVIEW

Source: *SYSTEM_OVERVIEW.md* | Authoritative system orientation

Kairos System Overview

Last updated: 2026-07-30

Audience: an engineer or operator who needs a quick orientation before reading the canonical architecture chapters.

Kairos is a personal investing research system for separate US/USD and India/INR books. It runs research and paper-trading workflows autonomously within fixed policy boundaries, records outcomes, and uses governed evidence to evaluate future changes. Live broker behavior remains separately controlled and safety-gated.

The lifecycle

```
flowchart LR
    DATA["Market and broker evidence"] --> RESEARCH["Research and deterministic scoring"]
    RESEARCH --> SIGNALS["Market-local signal ledger"]
    SIGNALS --> PAPER["PaperTrader"]
    PAPER --> MONITOR["PositionMonitor"]
    MONITOR --> OUTCOMES["Outcomes and performance truth"]
    OUTCOMES --> LEARNING["Learner and validation"]
    LEARNING --> RESEARCH
```

This is an orientation diagram, not the live topology. The rendered agent map in Dashboard -> Agents is the single source for agent-to-agent edges and history.

What keeps the system safe

- US/USD and India/INR pools remain isolated.
- LLM output is advisory and explainable; deterministic code owns financial state, eligibility, pricing, risk checks, and broker execution.
- Provider degradation, stale evidence, and ambiguous order state are treated as explicit conditions rather than silently fabricated data.
- Shadows and historical replay are observation systems until their own evidence and approval gates are satisfied.
- Live actions are subject to the active autonomy, market-control, kill-switch, mandate, and broker-account controls.

Where to go next

The complete architecture is not repeated here. Use the Architecture Portal (ARCHITECTURE.md) for the documentation hierarchy, then begin at docs/arch/00-index.md (docs/arch/00-index.md). For a particular feature, its features/<feature>/FEATURE_ARCHITECTURE.md is the micro-architecture; PROJECT_DECISIONS.md (PROJECT_DECISIONS.md) records why approved choices were made.

When this overview and a chapter disagree, the chapter wins for subsystem behavior; when a chapter and the live diagram disagree about topology, the diagram source and its history must be reconciled in the same change.

ARCHITECTURE

Source: ARCHITECTURE.md | Authoritative documentation portal

Kairos Architecture Portal

Last updated: 2026-07-31

Purpose: the stable entry point to Kairos architecture. It explains where truth

lives; it does not duplicate the implementation detail held by the documents it links.

What Kairos is

Kairos is a personal US and India investing research system. It keeps USD and INR books separate, researches instruments, executes an autonomous paper loop, observes outcomes, and permits only explicitly governed live actions. LLMs may summarize or propose; deterministic services own eligibility, pricing, risk controls, accounting, and broker execution.

Non-negotiable system properties

- The US and India markets are separate pools. Values, returns, risk, and decisions are never cross-summed across currencies.
- An LLM is never a money-path authority. It cannot create an executable order, override a safety gate, or promote a strategy.
- A missing, stale, ambiguous, or degraded required input fails closed for the decision it protects. Exits retain their independent safety path.
- Paper behavior is evidence, not proof. Research, shadow, replay, validation, and promotion are distinct lifecycle states.
- Broker credentials, account identifiers, and raw provider responses do not belong in client bundles or reference documentation.
- Daily technical scoring consumes completed regular-market sessions only, and scheduled agent slots are expressed in exchange-local time rather than assumed UTC.

Documentation hierarchy

```
| Source | Owns | Do not use it for |
| `ARCHITECTURE.md` | This portal, source-of-truth rules, cross-cutting principles | Detailed schedules,
schemas, or current provider behavior |
| `SYSTEM_OVERVIEW.md` | Short onboarding explanation and high-level lifecycle | Operational implementation
detail |
| `docs/arch/` | Definitive chapter-by-chapter operational architecture | Feature alternatives and
implementation history |
| `features/<feature>/FEATURE_ARCHITECTURE.md` | Feature micro-architecture: scope, contracts, states,
alternatives, rollout, and non-goals | A replacement for shared architecture chapters |
| `features/<feature>/IMPLEMENTATION_RESULT.md` | What was actually delivered and any approved deviations |
Future design authority |
| `PROJECT_DECISIONS.md` | Approved decisions, rationale, alternatives, and reversal cost | Unapproved
proposals |
| `public/agent-diagrams/system-map.json` | Rendered live agent-to-agent topology and its history |
Narrative safety sequences or data-model detail |
```

Start with the chapter index (docs/arch/00-index.md). Read a feature document before changing a scoped feature. Read the decision record before reopening an approved choice.

Architecture views

- System orientation: SYSTEM_OVERVIEW.md (SYSTEM_OVERVIEW.md)
- Agent responsibilities: docs/arch/03-agents.md (docs/arch/03-agents.md)
- Schema and data contracts: docs/arch/04-database-schema.md (docs/arch/04-database-schema.md)
- Schedules: docs/arch/05-crons-and-scheduling.md (docs/arch/05-crons-and-scheduling.md)
- Safety and money path: docs/arch/08-risk-and-safety.md (docs/arch/08-risk-and-safety.md)
- Learning and promotion: docs/arch/09-learning-loop.md (docs/arch/09-learning-loop.md)
- Live topology: Dashboard -> Agents -> the "Agent & Flow Architecture" section.

The topology is rendered from the JSON source above. Do not paste a second copy of its edges into a chapter. A chapter diagram is appropriate only when it explains a different altitude, such as a gate sequence, an event timeline, or a subsystem flow.

Change protocol

Every meaningful change follows this sequence:

- Record the decision or create/update the feature architecture before implementation.
- Implement only the approved scope, with tests proportional to risk.
- Update every affected architecture chapter in the same change.
- Update the live agent diagram when an agent's inputs, outputs, dependencies, or safety boundary changes. Update its history entry with the reason.
- Add an implementation result when a feature architecture is shipped; record an approved deviation rather than silently editing history.
- Verify the rendered diagram and architecture drift tests before shipping.

For a scoring-dimension change, "update the architecture" specifically means all of the following must agree with production code in the same commit:

- plain-English meaning: what question the dimension is trying to answer;
- source order, market applicability, freshness, and fallback behavior;
- exact formula, constants, thresholds, clamps, and any veto/cap;
- missing-data behavior and whether the dimension is excluded or merely displayed;
- whether the value can affect eligibility, sizing, exits, learning, or only explanation.

The definitive home for those facts is docs/arch/03-agents.md under ResearchAgent. Provider mechanics belong in chapter 02, persisted fields in chapter 04, and money-path consequences in chapter 08. A feature document may explain why a change was proposed, but it must not become the only place where the active formula is documented.

Changes to a money path additionally require the applicable risk chapter, schema chapter (when persistence changes), migration verification, and an explicit fail-closed review. A feature is not complete merely because code compiles.

In-app reference surface

Dashboard -> Agents -> System Reference exposes a curated owner-only allowlist of these records for reading or download. It is deliberately not a general repository browser and does not add a main-navigation "Docs" destination. The agent page remains the appropriate contextual home because it already owns the live topology.

PRD

Source: PRD.md | Product requirements and conventions

?? Kairos ?? Product Requirements Document

For LLMs: This file is the single source of truth for this codebase. Read it fully before writing any code. It describes what exists, what conventions are in use, what is planned, and why decisions were made. Do not invent conventions ?? follow what is documented here.

Last updated: 2026-06-01

0. Quick Context

- What: Personal Kairos for one user (Vaibhav / vterminater@gmail.com). Helps learn markets, navigate economy, and trade autonomously via AI agents.
- Stack: Next.js 15 App Router ?? Supabase ?? Anthropic Claude API ?? Tailwind v4 ?? Recharts ?? Stripe
- New capability (June 2026): Robinhood Agentic Trading MCP ?? AI agents can place real stock trades via <https://agent.robinhood.com/mcp/trading>
- Single user. No multi-tenancy concerns for agent features. Auth still required (Supabase).
- Superadmin email: vterminater@gmail.com (auto-assigned in `handle_new_user()` trigger)

1. Codebase Map

```
Kairos/
?"?"?"? app/
?"?  ?"?"?"? api/
?"?  ?"?"  ?"?"?"? ai/route.ts           ??? Claude API endpoint (POST)
?"?  ?"?"  ?"?"?"? admin/route.ts
?"?  ?"?"  ?"?"?"? stripe/checkout/route.ts
?"?  ?"?"  ?"?"?"? webhooks/stripe/route.ts
?"?  ?"?"?"?"? auth/callback/route.ts     ??? Supabase OAuth callback
?"?  ?"?"?"?"? dashboard/
?"?  ?"?"  ?"?"?"? layout.tsx             ??? wraps all dashboard pages with DashboardShell
?"?  ?"?"  ?"?"?"? page.tsx               ??? home (server component, passes data to DashboardHome)
?"?  ?"?"  ?"?"?"? admin/page.tsx
?"?  ?"?"  ?"?"?"? calendar/page.tsx
?"?  ?"?"  ?"?"?"? intelligence/page.tsx
?"?  ?"?"  ?"?"?"? markets/page.tsx
?"?  ?"?"  ?"?"?"? portfolio/page.tsx
?"?  ?"?"  ?"?"?"? settings/page.tsx
?"?  ?"?"  ?"?"?"? you/page.tsx
?"?  ?"?"?"?"? login/page.tsx
?"?  ?"?"?"?"? page.tsx                  ??? landing page
?"?  ?"?"?"?"? layout.tsx                ??? root layout
?"?"?"?"? components/
?"?  ?"?"?"?"? dashboard/
?"?  ?"?"?"?"? DashboardHome.tsx          ??? "use client", receives profile/holdings/predictions
?"?  ?"?"?"?"? DashboardShell.tsx          ??? "use client", sidebar nav + layout wrapper
?"?"?"?"? lib/
?"?  ?"?"?"?"? supabase/
?"?  ?"?"?"?"? server.ts                 ??? createClient() for server components/routes
?"?  ?"?"?"?"? client.ts                 ??? createClient() for "use client" components
?"?"?"?"? middleware.ts                 ??? protects /dashboard/* and /admin/*
?"?"?"?"? types/index.ts                 ??? all TypeScript types + TIER_LIMITS
?"?"?"?"? supabase/migrations/
?"?  ?"?"?"?"? 001_initial_schema.sql        ??? full DB schema (applied)
?"?"?"?"? next.config.ts
?"?"?"?"? package.json
?"?"?"?"? PRD.md                          ??? this file
```

2. Coding Conventions (MUST FOLLOW)

2.1 Styling

- NO Tailwind utility classes. Inline styles throughout using T color token object.

- Every file that renders UI defines T at top:

```
const T = {
  bg: "#0D0F14", surface: "#13151C", card: "#1A1D27", border: "#252836",
  text: "#ECEDEF", textSub: "#9B9EA8", muted: "#6B7280",
  accent: "#6366F1", green: "#34D399", red: "#F87171", yellow: "#FBBF24",
};
```

- All layout via inline `style={{}}` props. No CSS modules. No `className` strings.

2.2 Server vs Client Components

- Server components (no `"use client"`): fetch data from Supabase, pass as props to client components.

- Client components (`"use client"` at top): all interactivity, event handlers, `useState`, `useRouter`.

- Pattern: `page.tsx` (server) `??` fetches data `??` renders `<ComponentName data={data} />` (client).

2.3 Supabase

- Server: `import { createClient } from "@lib/supabase/server" ??` async, uses cookies

- Client: `import { createClient } from "@lib/supabase/client" ??` synchronous

- Auth check pattern in server components:

```
const supabase = await createClient();
const { data: { user } } = await supabase.auth.getUser();
```

- RLS is enabled on all tables. Policies enforce `auth.uid() = user_id`.

2.4 AI / LLM routing

- Reasoning/text generation routes through the LLM router (`lib/llm-router.ts`,

`callLLM/runAgentLoop`), which defaults to DeepSeek (`deepseek-v4-flash/pro`). Anthropic Claude is only used when a task's model explicitly resolves to a Claude model AND the anthropic key is present; otherwise it falls back to DeepSeek.

- `/api/ai/route.ts` is a small owner-gated ad-hoc reasoning endpoint (Intelligence

page) - it now uses `callLLM` (DeepSeek), NOT the Claude CLI. Request body: `{ prompt: string, systemPrompt?: string } -> Response: { text, tokensUsed, costUsd }`.

- MCP tool access (Robinhood/market reads - RH has no public read API) goes through

`execClaude` (`lib/claude-exec.ts`), which shells the Claude CLI on the local Windows host only (throws on Vercel). This is a tool bridge, not the text LLM.

- Default model: `claude-sonnet-4-20250514`

- Rate limiting enforced per tier (5/50/??? queries/day).

- Usage logged to `usage_logs` table automatically.

2.5 Types

- All types in `types/index.ts`. Import with `import type { Profile, Holding } from "@types"`.

- When adding new DB tables, add corresponding TypeScript interface to `types/index.ts`.

2.6 Navigation

- Nav items defined in `DashboardShell.tsx` as NAV array.
- To add a new page: add route to `app/dashboard/<name>/page.tsx` AND add entry to NAV array.

2.7 New API Routes

- Place under `app/api/<feature>/route.ts`.
- Always check auth first: `supabase.auth.getUser()` ?? 401 if no user.
- Return `NextResponse.json(...)`.

3. Existing Database Schema (Applied ??? 001_initial_schema.sql)

Tables

Table	Key Columns	Purpose
<code>`profiles`</code>	<code>id, email, role, subscription_tier, xp, streak_days, dna_*, ai_model</code>	User profile + gamification + DNA scores
<code>`holdings`</code>	<code>user_id, symbol, name, qty, avg_cost, current_price, sector, exchange</code>	Manual portfolio holdings
<code>`predictions`</code>	<code>user_id, asset, thesis, direction, target_price, confidence, status, score</code>	User market predictions
<code>`journal_entries`</code>	<code>user_id, asset, direction, entry/exit_price, pnl, setup, emotion, lesson</code>	Trade journal
<code>`quiz_history`</code>	<code>user_id, domain, question, correct, xp_earned</code>	Learning quiz log
<code>`brief_history`</code>	<code>user_id, content, model_used, tokens_used</code>	AI morning brief log
<code>`reading_list`</code>	<code>user_id, title, url, domain, notes, completed</code>	Curated reading
<code>`watchlist`</code>	<code>user_id, symbol, name, market, entry_price, stop_price, notes</code>	Tickers to watch
<code>`usage_logs`</code>	<code>user_id, action, tokens_used, cost_usd</code>	Rate limiting + cost tracking
<code>`announcements`</code>	<code>title, content, target_tier, active</code>	Admin broadcasts

Key Functions

- `handle_new_user()` ??? trigger: auto-creates profile on `auth.users` insert, assigns superadmin to `vterminater@gmail.com`
- `get_daily_ai_count(user_id)` ??? returns today's AI query count
- `get_ai_limit(tier)` ??? returns query limit by tier (5/50/9999)
- `daily_usage` ??? view: today's usage counts grouped by user/action

Subscription Tiers

Tier	AI queries/day	Holdings	Markets
free	5	3	US only
pro (\$29/mo)	50	unlimited	US + India
elite (\$99/mo)	unlimited	unlimited	US + India + Crypto + Macro + Global

4. Planned: Agentic Trading System

4.1 Overview

Four AI agents work in a self-improving loop to research stocks, score them, trade via Robinhood, and learn from outcomes.

```

ResearchAgent "???"??-? AnalystAgent "???"??-? TraderAgent
              "???"              "???"
              "???"??"? LearnerAgent ? - "???"
              (weekly feedback loop)

```

Self-improvement mechanism: LearnerAgent compares predictions vs actual outcomes, adjusts ONE signal weight per weekly cycle (scientific method: single variable). New weight becomes baseline. Repeats until accuracy targets met.

4.2 Agent Specifications

ResearchAgent

Input:

- watchlist symbols (from Supabase `watchlist` table)
- sources: financial news RSS, SEC EDGAR Form 4, Reddit (r/investing, r/wallstreetbets), user-uploaded PDFs/notes

Output (stored in `research\_packets` table):

```

{
  ticker: string,
  sentiment_score: number,      // -1.0 to 1.0
  key_facts: string[],         // bullet points, max 5
  source_urls: string[],
  created_at: timestamp
}

```

Schedule: every 30 minutes for watchlist tickers (Railway cron), on-demand via chat
 Tools: Puppeteer MCP (scraping), Supabase pgvector (embed + store)

AnalystAgent

Input:

- research_packets for ticker
- price/volume data (Polygon.io or Yahoo Finance)
- signal_weights from Supabase

Output (stored in `agent\_signals` table):

```

{
  ticker: string,
  score: number,                // 0-100 composite
  regime: "bull" | "bear" | "sideways",
  recommendation: "buy" | "hold" | "sell",
  confidence: number,           // 0-100
  signal_breakdown: {
    momentum: number,
    technicals: number,
    news_sentiment: number,
    insider_buying: number,
    earnings_revision: number,
    social_velocity: number,
    user_thesis: number
  }
}

```

Schedule: after each ResearchAgent cycle + on-demand

TraderAgent

Input:

- AnalystAgent output (score, recommendation, confidence)
- strategy_config from Supabase
- Robinhood account state (via MCP)

Behavior:

- Only trades if score $\geq$ strategy_config.min_analyst_score (default: 70)
- Only uses Robinhood AGENTIC account (not primary ??) this is enforced by Robinhood)
- Respects max_position_pct and max_daily_trades from strategy_config

Modes:

- approval_required (default):
 - Emits trade proposal to `trade\_queue` table
 - UI shows toast ??' modal with reasoning + risk ??' user approves/rejects
 - On approve: executes via Robinhood MCP
- auto:
 - Executes immediately within hard position limits
 - User must explicitly unlock per session

Output (stored in `trade\_log`):

- every action logged with agent reasoning snapshot

LearnerAgent

Input:

- trade_log (agent predictions)
- actual price movements (fetched from price data API)
- signal_weights (current)

Process:

1. Score prediction accuracy for completed trades
2. Identify worst-performing signal
3. Adjust that signal's weight by ??0.02 (max ??0.05/cycle)
4. Store old ??' new weight with reason in learning_log
5. New weights become baseline for AnalystAgent

Schedule: weekly (Railway cron, 3-day offset from ResearchAgent)

4.3 New Database Tables (to be migrated)

```
-- Agent watchlist (extends existing watchlist table ??" add thesis_notes column)
-- OR use existing watchlist + new thesis_notes column via ALTER

-- Research output
CREATE TABLE research_packets (
  id uuid DEFAULT uuid_generate_v4() PRIMARY KEY,
  ticker text NOT NULL,
  sentiment_score numeric NOT NULL,          -- -1.0 to 1.0
  key_facts text[] NOT NULL DEFAULT '{}',
  source_urls text[] NOT NULL DEFAULT '{}',
  raw_text text,
  created_at timestamptz DEFAULT now()
);

-- Analyst scores per ticker
CREATE TABLE agent_signals (
  id uuid DEFAULT uuid_generate_v4() PRIMARY KEY,
  ticker text NOT NULL,
  score numeric NOT NULL,                    -- 0-100
  regime text CHECK (regime IN ('bull','bear','sideways')),
  recommendation text CHECK (recommendation IN ('buy','hold','sell')),
  confidence numeric,
  signal_breakdown jsonb,                    -- { momentum, technicals, ... }
  created_at timestamptz DEFAULT now()
);

-- Current signal weights (single-row config, updated by LearnerAgent)
CREATE TABLE signal_weights (
  id uuid DEFAULT uuid_generate_v4() PRIMARY KEY,
  momentum numeric DEFAULT 0.20,
  technicals numeric DEFAULT 0.15,
  news_sentiment numeric DEFAULT 0.20,
  insider_buying numeric DEFAULT 0.15,
  earnings_revision numeric DEFAULT 0.15,
  social_velocity numeric DEFAULT 0.10,
  user_thesis numeric DEFAULT 0.05,
  updated_at timestamptz DEFAULT now()
);

-- Pending trade proposals (approval_required mode)
CREATE TABLE trade_queue (
  id uuid DEFAULT uuid_generate_v4() PRIMARY KEY,
  ticker text NOT NULL,
  action text NOT NULL CHECK (action IN ('buy','sell')),
  quantity numeric NOT NULL,
  estimated_price numeric,
  analyst_score numeric,
  signal_snapshot jsonb,                    -- full AnalystAgent output at time of proposal
  status text DEFAULT 'pending' CHECK (status IN ('pending','approved','rejected','executed','expired')),
  user_decision_at timestamptz,
  rejection_reason text,
  created_at timestamptz DEFAULT now()
);

-- Executed trades + outcomes
CREATE TABLE trade_log (
  id uuid DEFAULT uuid_generate_v4() PRIMARY KEY,
  ticker text NOT NULL,
  action text NOT NULL CHECK (action IN ('buy','sell')),
  quantity numeric NOT NULL,
  executed_price numeric,
  agent text DEFAULT 'TraderAgent',
  mode text CHECK (mode IN ('approval_required','auto')),
  approved_by_user boolean DEFAULT false,
  robinhood_order_id text,
  outcome_pnl numeric,                      -- filled after position closes
  outcome_correct boolean,                  -- did price move in predicted direction?
  resolved_at timestamptz,
  created_at timestamptz DEFAULT now()
);

-- LearnerAgent weight change history
```

```

CREATE TABLE learning_log (
  id uuid DEFAULT uuid_generate_v4() PRIMARY KEY,
  signal_adjusted text NOT NULL,
  old_weight numeric NOT NULL,
  new_weight numeric NOT NULL,
  reason text,
  accuracy_before numeric,      -- prediction % correct before change
  accuracy_after numeric,      -- filled on next cycle
  created_at timestampz DEFAULT now()
);

-- Vector memory for semantic search (requires pgvector extension)
CREATE TABLE agent_memory (
  id uuid DEFAULT uuid_generate_v4() PRIMARY KEY,
  ticker text,
  source_type text CHECK (source_type IN ('news', 'filing', 'user_note', 'pdf', 'reddit', 'twitter')),
  raw_content text NOT NULL,
  content_embedding vector(1536), -- text-embedding-3-small
  created_at timestampz DEFAULT now()
);
CREATE INDEX ON agent_memory USING ivfflat (content_embedding vector_cosine_ops);

-- Strategy config (single row, edited via settings UI)
CREATE TABLE strategy_config (
  id uuid DEFAULT uuid_generate_v4() PRIMARY KEY,
  name text DEFAULT 'Vaibhav Alpha v1',
  mode text DEFAULT 'approval_required' CHECK (mode IN ('approval_required', 'auto')),
  max_position_pct numeric DEFAULT 5,      -- % of agentic account per position
  max_daily_trades integer DEFAULT 3,
  min_analyst_score numeric DEFAULT 70,
  target_30d_accuracy numeric DEFAULT 0.60,
  max_drawdown_pct numeric DEFAULT 15,
  failure_drawdown_pct numeric DEFAULT 20,
  trading_enabled boolean DEFAULT true,    -- kill switch
  updated_at timestampz DEFAULT now()
);

-- Seed default config
INSERT INTO strategy_config DEFAULT VALUES;

```

4.4 Robinhood MCP Integration

- MCP endpoint: `https://agent.robinhood.com/mcp/trading`
- Transport: HTTP + OAuth
- Connected via: `claude mcp add robinhood-trading --transport http https://agent.robinhood.com/mcp/trading`
- Auth: OAuth flow via `mcp__robinhood-trading__authenticate` tool
- Scope: Dedicated Robinhood Agentic Account only ?? agents CANNOT touch primary account (enforced by Robinhood)
- Supported assets (June 2026): Equities only (options/crypto planned)
- Kill switch: `strategy_config.trading_enabled = false` ?? TraderAgent no-ops all trade calls

MCP Tool Capabilities (confirmed from Robinhood docs)

- Query: portfolio value, buying power, positions, order history, P&L
- Execute: market orders, limit orders, all standard order types
- Rebalance: build portfolios to match criteria
- Backtest: test strategies against historical data
- Monitor: watch for market events (analyst upgrades, etc.)

4.5 New API Routes (to build)

POST /api/agents/research	??? trigger ResearchAgent for ticker(s)
POST /api/agents/analyze	??? trigger AnalystAgent for ticker(s)
POST /api/agents/trade/propose	??? TraderAgent proposes trade (writes to trade_queue)
POST /api/agents/trade/approve	??? user approves trade_queue item ??' executes via Robinhood MCP
POST /api/agents/trade/reject	??? user rejects with optional reason
GET /api/agents/status	??? all agent run statuses
POST /api/agents/learn	??? trigger LearnerAgent cycle
GET /api/portfolio/robinhood	??? read agentic account positions from Robinhood MCP

5. UI Architecture

5.1 Existing Dashboard Nav (DashboardShell.tsx NAV array)

/dashboard	??' Home (4-panel overview)
/dashboard/portfolio	??' Holdings + P&L
/dashboard/markets	??' Watchlist + prices
/dashboard/intelligence	??' AI research feed
/dashboard/calendar	??' Economic calendar
/dashboard/you	??' Profile + DNA + learning
/dashboard/settings	??' Preferences
/dashboard/admin	??' Admin only (role check)

5.2 New Pages to Add

/dashboard/agents	??' Agent control panel (status, kill switch, run manually)
/dashboard/trading	??' Trade queue, approval UI, Robinhood agentic account
/dashboard/learning	??' Signal weight history, prediction accuracy charts

Add these to NAV array in DashboardShell.tsx:

```
{ href: "/dashboard/agents", label: "Agents", icon: "???" },  
{ href: "/dashboard/trading", label: "Trading", icon: "? - ?" },  
{ href: "/dashboard/learning", label: "Learning", icon: "? - ?" },
```

5.3 Design System

Colors defined via T token object (see Section 2.1). All styling inline. Font: Inter. No icon libraries ??' uses Unicode symbols as icons (see NAV array).

5.4 Trade Approval Flow (/dashboard/trading)

```
TraderAgent writes to trade_queue (status: 'pending')  
??"  
UI polls trade_queue every 30s (or websocket)  
??"  
Toast appears: "Buy 10 NVDA @ ~$142 ??' Score: 83/100"  
??"  
User clicks ??' Modal opens:  
- Why: top 3 signals that triggered  
- Risk: position size, % of account, estimated max loss  
- Chart: 30-day price + agent score history  
??"  
[Approve] ??' POST /api/agents/trade/approve ??' Robinhood MCP executes  
[Reject] ??' POST /api/agents/trade/reject ??' logged, agent notes rejection  
[Modify] ??' User edits qty/price ??' approves modified version
```

5.5 Dashboard Home Layout (target)

[illegible]

6. External Integrations

```
| Integration | Purpose | Status | Notes |
| Robinhood MCP | Trade execution + read portfolio | ??... Connected, auth in progress | Agentic account only |
| DeepSeek API | Agent reasoning/text (default LLM) | ? Active | Via `lib/llm-router.ts` (`callLLM`/`runAgentLoop`) |
| Anthropic Claude CLI | MCP tool bridge (Robinhood/market reads) | ? Active (local host only) | Via `execClaude`; throws on Vercel |
| Supabase | DB + auth + vector store | ??... Active | pgvector needs enabling |
| Puppeteer MCP | Web scraping (news, filings) | ??... Connected | For ResearchAgent |
| SEC EDGAR API | Form 4 insider trades, 10-K filings | ??"? To add | Free, no key required |
| Polygon.io | Price + volume + technicals | ??"? To add | $29/mo ??" preferred over Yahoo Finance |
| Reddit API | Sentiment from r/investing, r/wallstreetbets | ??"? To add | Scrape via Puppeteer or official API |
| Stripe | Subscriptions | ??... Active | Webhooks at `/api/webhooks/stripe` |
| Railway | 24/7 agent cron workers | ??"? To add | ResearchAgent (30min) + LearnerAgent (weekly) |
```

7. Success Metrics & Failure Modes

Target (define success)

- Prediction accuracy > 60% over rolling 30 days
- Sharpe ratio > 1.0
- Max drawdown < 15%
- Signal weights converge toward stable, accurate configuration over 8-12 weeks

Failure (triggers pause + alert)

- Prediction accuracy < 40% over 30 days
- Drawdown > 20%
- Any single day loss > 5% of agentic account

```
??' trading_enabled flag set to false, user alerted
```

Learning Guardrails

- LearnerAgent adjusts max 0.05 weight per cycle
- Weights must always sum to 1.0

- No weight may go below 0.02 (floor) or above 0.40 (ceiling)

8. Build Order (Phases)

Phase 1 ??" Foundation (current)

- [x] Robinhood MCP added
- [] Complete Robinhood OAuth auth
- [] Read live portfolio from Robinhood agentic account
- [] Create new DB tables (migration 002)
- [] Seed signal_weights + strategy_config
- [] /dashboard/agents page ??" status + kill switch
- [] /dashboard/trading page skeleton

Phase 2 ??" Research + Analysis

- [] ResearchAgent: news scraping via Puppeteer MCP
- [] SEC EDGAR Form 4 fetcher
- [] Polygon.io or Yahoo Finance price data
- [] AnalystAgent with static signal weights
- [] Render scored watchlist on /dashboard/markets
- [] agent_memory vector store wired up

Phase 3 ??" Trading

- [] TraderAgent in approval_required mode
- [] Trade approval UI (toast ??" modal ??" execute)
- [] Full trade logging
- [] End-to-end test: research ??" score ??" propose ??" approve ??" execute

Phase 4 ??" Learning

- [] LearnerAgent: prediction scoring + weight adjustment
- [] /dashboard/learning with weight history charts
- [] Accuracy tracking over rolling 30-day window

Phase 5 ??" Automation

- [] auto mode with hard guards
- [] Railway cron job deployment (30min research, weekly learn)
- [] Push notifications for trade proposals
- [] Mobile-optimized trade approval

9. Features Built (2026-06-29 Session)

9.1 Risk Profile System

Three preset risk profiles stored in `strategy_config`:

Profile	score_threshold	position_size_pct	stop_loss_pct	target_pct
Conservative	72	7	5	12
Balanced	60	10	7	20
Aggressive	52	15	10	35

- API: GET `/api/settings/risk-profile` returns current profile. PATCH `/api/settings/risk-profile` updates `strategy_config` row.
- UI: Settings page ?? Agents tab ?? Risk Profile card. User selects preset or edits per-field.
- ResearchAgent integration: Reads `risk_profile` from `strategy_config`, applies `PROFILE_WEIGHTS` multiplier to signal scoring, uses `score_threshold` as the minimum score for a buy signal.

9.2 Position Monitor (Dynamic Exit Price Management)

Runs daily after market close (weekdays 4:15PM). Manages trailing stops and target exits for all open `paper_positions`.

New columns added to `paper_positions` (migration 026):

- `price_target` ?? target exit price
- `stop_loss` ?? initial stop loss price
- `highest_price` ?? highest price seen since entry (trail anchor)
- `target_updated_at` ?? last time target was updated
- `exit_reason` ?? 'stop', 'target', or 'llm_exit'

Trailing stop logic: `new_stop = max(original_stop_loss, highest_price - 0.93)`

On each run:

- Fetch current prices for all open positions
- Update `highest_price` if current price > previous highest
- Recompute trailing stop
- If `current_price <= stop` ?? close position, set `exit_reason = 'stop'`
- If `current_price >= price_target` ?? close position, set `exit_reason = 'target'`
- Closed positions return cash to buying power

UI: TradingPage has PositionMonitor card with "Run Now" button and last-run timestamp.

9.3 MacroSentinel (Recession Risk Agent)

Weekly macro regime scoring agent. Runs Mondays 8AM.

Indicators fetched from Alpha Vantage (8 total):

- Yield Curve (10Y-2Y spread) ?? inverted = danger
- Sahm Rule proxy (unemployment rate delta)
- Real GDP (QoQ growth rate)
- Nonfarm Payrolls (MoM change)

- CPI (YoY inflation)
- Retail Sales (MoM change)
- Federal Funds Rate
- Durable Goods Orders (MoM)

Regime classification (weighted danger score 0-100):

- GREEN: score < 25 " expansion
- YELLOW: 25-49 " caution
- ORANGE: 50-74 " slowdown
- RED: 75-100 " recession risk

Advisory-only: MacroSentinel reports regime; it does not auto-throttle agents or halt trading. User decides how to act.

Storage (migration 028):

- macro_regime " current regime + score + timestamp
- macro_signals " per-indicator readings and contribution

UI: MarketsPage " MacroSentinel card with danger gauge + signal breakdown table. DashboardHome shows colored regime banner (hidden when GREEN).

9.4 Smart Money Trades (MarketsPage)

API: /api/markets/insider-trades/route.ts

Two data sources:

- Insiders tab: Alpha Vantage INSIDER_TRANSACTIONS " corporate insider buy/sell filings
- Congress tab: House Stock Watcher public S3 data " congressional stock trade disclosures (free, no auth required)

UI in MarketsPage with tabbed Insiders/Congress view, showing symbol, insider name, transaction type, shares, and date.

9.5 LLM Cost Monitor

API: /api/admin/llm-costs/route.ts " queries llm_call_log table.

Metrics computed:

- Total spend (last 24h, 7d, 30d)
- Burn rate (\$/hour)
- Projected daily cost
- Per-model breakdown (Claude / DeepSeek / Groq)
- 24-bar hourly cost chart (Recharts BarChart)

UI:

- Settings " Agents tab " LLM Cost Monitor card
- DashboardHome shows " banner if projected_daily > \$2

9.6 Mentor System (Judgment Score Chart)

Mentor nav link restored in DashboardShell sidebar (" icon, under "Learn" section).

API: `/api/mentor/scores/route.ts` groups `trade_journal` scores by date, returns time series.

MentorPage: Recharts LineChart showing judgment score over time with reference lines:

- 50 = Learning
- 70 = Proficient
- 90 = Expert

9.7 Visual Agent Mermaid Diagrams

Component: `components/dashboard/AgentDiagram.tsx`

- Renders clickable Mermaid v10 flowcharts per agent (v11 incompatible with webpack/es-toolkit)
- Nodes color-coded: active=green, new=blue, changed=red, removed=gray
- Click any node ??' detail drawer showing why-added and change history

Data files in `public/agent-diagrams/` (7 JSON files):

- `research-agent.json`, `learner-agent.json`, `theme-scout.json`, `deepseek-agent.json`,
`position-monitor.json`, `paper-trader.json`, `macro-sentinel.json`

9.8 TradingView CSV Watchlist Import

WatchlistPanel now has an "Import CSV" button. Modal accepts paste of TradingView export format (EXCHANGE:TICKER,Description). Parses tickers, batch POSTs to `/api/watchlist` with progress indicator.

9.9 Signal Backtest Tab (AgentsPage)

New "Backtest" tab in AgentsPage. Joins `agent_signals` to `paper_trades` by symbol + date (??3-day window). Displays:

- Hit Rate (signals that became profitable trades)
- Misses (signals with no matching trade or negative outcome)
- Open (signals still in open positions)
- Avg Return (%)

10. Open Decisions

```
| Decision | Options | Notes |
| Price data | Polygon.io ($29/mo) vs Yahoo Finance (free, rate-limited) | Polygon preferred for reliability |
| Reddit data | Official API vs Puppeteer scrape | Puppeteer simpler, already connected |
| Notifications | Browser push vs SMS (Twilio) vs Slack MCP | Slack MCP already connected in session |
| Embedding model | OpenAI text-embedding-3-small vs Claude | OpenAI cheaper for embeddings |
| Auto mode trigger | Score threshold only vs also time-gated | Requires explicit per-session unlock |
```

10. Security & Risk Controls

CRITICAL: TraderAgent ONLY operates on Robinhood agentic account. This is enforced structurally by Robinhood ??" the MCP cannot access the primary account. Do not attempt to add primary account access.

- Kill switch: `strategy_config.trading_enabled = false` ??" check this before every TraderAgent action

- approval_required mode is default and must be explicitly changed to auto
- All agent actions stored in trade_log with full reasoning snapshot (non-deletable by design)
- LearnerAgent weight changes bounded: $\pm 0.05/\text{cycle}$, weights floor 0.02, ceiling 0.40
- Robinhood ToS: user (Vaibhav) is fully responsible for all agent-executed trades
- Never log or expose Robinhood OAuth tokens anywhere in the codebase

Next immediate step: Paste Robinhood OAuth redirect URL to complete authentication, then run Phase 1 DB migration.

Operational Architecture

00-index

Source: docs\arch\00-index.md | Authoritative operational architecture

Kairos Architecture Chapter Index

Last updated: 2026-07-31

This directory is the definitive chapter-by-chapter operational architecture. Each chapter is independently updateable: a broker change does not require editing the agent chapter. Use ARCHITECTURE.md (../ARCHITECTURE.md) for the documentation hierarchy and SYSTEM_OVERVIEW.md (../SYSTEM_OVERVIEW.md) for a short orientation.

Chapters

File	What it covers	Update when
`01-what-is-kairos.md`	Core idea, principles, feature map, onboarding diagrams	Product direction or feature areas change
`02-tech-stack.md`	Runtime stack, providers, adapters, and integrations	A library, provider, adapter, or framework changes
`03-agents.md`	Agent responsibilities, inputs, outputs, and behavior	An agent changes or is added/removed
`04-database-schema.md`	Tables, persistence contracts, indexes, and triggers	Any schema migration changes a data contract
`05-crons-and-scheduling.md`	Vercel and local schedules, operational timing	A cron is added, removed, or rescheduled
`06-env-variables.md`	Required, optional, and runtime-editable variables	An environment variable changes ownership
`07-coding-conventions.md`	UI, data access, API, and agent conventions	A project-wide convention changes
`08-risk-and-safety.md`	Money-path gates, autonomy, kill switches, ledgers	Order flow, autonomy, or safety behavior changes
`09-learning-loop.md`	Evaluation, learning, validation, and promotion controls	Learning flow, genome, or promotion controls change

Source-of-truth hierarchy

Source	Owns
`ARCHITECTURE.md`	Architecture portal, principles, documentation ownership, and read paths
`SYSTEM\_OVERVIEW.md`	Concise product and system orientation
`docs/arch/`	Detailed operational architecture
`features/<feature>/FEATURE\_ARCHITECTURE.md`	Feature micro-architecture: contracts, states, alternatives, rollout, and non-goals
`features/<feature>/IMPLEMENTATION\_RESULT.md`	What actually shipped and approved deviations
`PROJECT\_DECISIONS.md`	Approved decisions, rationale, and reversal cost
`public/agent-diagrams/system-map.json`	Rendered agent-to-agent topology and topology history

When these sources disagree, reconcile the conflict in the same change. A feature document cannot silently override a shared chapter or approved decision.

Update rules

Change	Update
New or changed agent	`03-agents.md`; topology JSON/history; `05-crons-and-scheduling.md` when scheduled
Scoring dimension, formula, source, weight, applicability, missing-data rule, veto, or cap	`03-agents.md` plain-English explanation + exact formula contract; `02-tech-stack.md`, `04-database-schema.md`, and `08-risk-and-safety.md` when their ownership changes
New table, column, index, trigger, or RPC	`04-database-schema.md`
New provider, broker, or runtime integration	`02-tech-stack.md`
New or changed schedule	`05-crons-and-scheduling.md`
New environment variable or vault ownership	`06-env-variables.md`
Money-path, autonomy, or safety-gate change	`08-risk-and-safety.md` and relevant feature design
Learning, validation, strategy, or promotion change	`09-learning-loop.md` and relevant feature design
Product scope/principle change	`01-what-is-kairos.md` and `SYSTEM\_OVERVIEW.md`
Feature implementation	Its feature design and an implementation result; affected shared chapters

Diagram ownership

Agent-to-agent topology lives only in `public/agent-diagrams/system-map.json`, rendered on Dashboard -> Agents -> Agent & Flow Architecture. Do not paste the same edges into chapters. A chapter diagram is appropriate only when it documents a different view, such as a gate sequence, timeline, or contained subsystem. The drift test in `tests/arch-diagram-drift.test.ts` validates the map files and declared overlaps.

In-app access

Dashboard -> Agents -> System Reference exposes a small owner-only allowlist for reading or downloading architecture records. It is not a repository browser and does not replace this documentation structure.

01-what-is-kairos

Source: docs\arch\01-what-is-kairos.md | Authoritative operational architecture

Kairos - What Is This?

Last updated: 2026-07-10

Update this file when: product direction changes, new feature areas are added, core principles change, or the top-level pitch changes.

1. The core idea

Kairos is an automated investing research assistant that behaves like a small, careful hedge-fund team living inside one web app. It doesn't just pick stocks - it runs a loop:

Research -> Decide -> Trade (on paper first) -> Watch -> Learn -> Improve -> repeat.

The important word is **loop**. Most stock tools give you a score and stop. Kairos scores a stock, acts on it in a paper (pretend-money) portfolio, watches how that trade actually turns out, and then feeds the outcome back to make the next decision smarter. Real money is optional, always small, and never moves without you clicking "yes".

2. Three core principles (apply everywhere)

- Paper first, real money last. Every strategy proves itself on pretend money before a cent of real money is at stake.
- AI proposes, human disposes. Agents can **suggest** a new strategy or trade, but a person must approve anything that touches real money or changes the live strategy.
- Evidence before belief. A proposed improvement must beat the current one on held-out historical data before it is allowed to take effect.

3. The big picture

```
flowchart LR
    DATA[Market data\nUS + India] --> RESEARCH[ResearchAgent\nscores each stock]
    MACRO[MacroSentinel\neconomy read] --> RESEARCH
    RESEARCH --> SIGNALS[Signals\nscore per stock]
    SIGNALS --> PAPER[PaperTrader\npretend-money trades]
    SIGNALS -- approved by you --> LIVE[Live order\nreal money, tiny]
    PAPER --> MONITOR[PositionMonitor\nwatches + exits]
    MONITOR --> OUTCOMES[Closed trades\nwin/loss]
    OUTCOMES --> LEARNER[LearnerAgent\nproposes a better strategy]
    LEARNER --> VALIDATE[Validation Engine\ntest on history]
    VALIDATE -- you promote --> CHAMPION[Champion strategy]
    CHAMPION --> RESEARCH
```

Read it as a wheel: data comes in on the left, becomes decisions, becomes trades, becomes outcomes, and the outcomes teach a better Champion strategy that feeds back into ResearchAgent. That feedback arrow is the whole point.

4. Feature map

- | Feature area | What it does |
- | Screening & research | Scans US + India universes, scores candidates across 5 dimensions, tracks score trend |
- | Paper trading | Simulated portfolios per market (\$US, ?India) - realistic fills, stops, targets |
- | Live trading | Real orders via Robinhood (US) + Zerodha Kite (India), human-gated |
- | Evolution loop | Turns closed outcomes into Challenger strategies; human promotes to Champion |
- | RAG trade memory | Semantic recall of past setups at scoring time |
- | Performance Truth | Mandate-aware Sharpe/Sortino/alpha/drawdown evaluation ledger |
- | Coaching & briefings | MentorAgent coaching notes + daily email briefings |
- | System Health | Funnel of open issues -> dashboard card + brief section |
- | Multi-LLM routing | Claude / DeepSeek / Groq / Gemini; per-agent assignments; paper P&L per model |
- | India parity | Full NSE scoring, ? paper pool, Kite execution, NSE insider+options feeds |
- | Admin & DB cleanup | User management; monthly DB pruning cron |

5. What makes this different from a scanner

- | Ordinary scanner | Kairos |
- | Gives you a score today | Scores + remembers its past accuracy |
- | You decide every trade | Pretend-money paper book tests decisions first |
- | No feedback loop | Closed loop: outcomes teach future scoring |
- | Single model | Multi-LLM routing; Claude vs DeepSeek P&L compared |
- | US-only | US + India (INR ? pool separate from USD \$) |
- | No safety layers | 9+ independent money-safety gates |

6. A trade's life, end to end (worked example)

US stock "ACME"

```
flowchart TD
    A[Mon AM: ResearchAgent scores ACME 78\nwrites decision_observation + signal_score_history] --> B{score  
=> threshold AND long?}
    B -- yes --> C[PaperTrader claims signal\nbuys ACME in $ pool\nrecords expected_price + realized_slip]
    B -- no --> Z[logged only, no trade]
    C --> D[PositionMonitor watches daily\nupdates highest_price]
    D --> E{exit trigger?}
    E -- trailing stop breached --> F[Full close: realized_pnl, outcome=win/loss]
    E -- price target hit --> G[Partial: sell half, move stop to breakeven]
    E -- time stop: age > horizon_days --> F
    E -- score drops via llm_exit --> F
    F --> D
    G --> H[indexClosedTrade: embed setup, store in trade_memories]
    H --> I[Learner training set grows]
    I --> J[After 10+ trades on Fridays: LearnerAgent proposes Challenger]
```

Step by step:

- Monday morning: ResearchAgent scores ACME 78, writes agent_signals (pending), signal_score_history, decision_observations.
- 10:05 AM: PaperTrader wakes, claims the signal, checks re-entry cooldown (ACME not held recently), checks pyramid gate (not held at all), buys ACME with 10% of pool NAV. Records expected_price = 102.50, fill at 102.55 (0.05% slip).
- Daily 4:15 PM: PositionMonitor fetches live ACME price. Updates highest_price. Trailing stop = max(stop_loss, highest_price x 0.93). Checks time stop (day 10 max).
- Day 8: ACME hits the price target at \$123. PositionMonitor sells half (floor(qty/2)), moves stop_loss to avg_cost (\$102.55). The remaining half runs.

- Day 12: Time stop fires (age > 10 days). PositionMonitor closes the remainder.
- Close: paper_t trades updated with exit_price, pnl_pct, outcome. Cash returned.

indexClosedTrade() embeds the setup and stores in trade_memories.

- Friday 5 PM (after 10+ trades): LearnerAgent reads the closed cohort, proposes a Challenger with slightly higher technical weight.

- Owner reviews: Validation Engine confirms Sharpe 0.72, win_rate 58%. Owner promotes Challenger to Champion. ResearchAgent's next Monday run uses the new weights.

7. Dashboard navigation map

All pages under app/dashboard/. All require auth (Supabase middleware).

```
| Path | Page name | What's on it |
| `~/dashboard` | Home | Portfolio summary, live account, macro regime banner, System Health card, agent status |
| `~/dashboard/research` | Research | Signal list, symbol cards, score breakdown, thesis |
| `~/dashboard/learning` | Learning | LearnerAgent controls, Champion/Challenger, strategy versions, Performance Truth panel, mandate selector |
| `~/dashboard/agents` | Agents | Agent status grid, System Map diagram (Mermaid, from `system-map.json`), per-agent diagrams |
| `~/dashboard/upgrade-path` | Upgrade Path | Owner-only live inventory of shadow/paper experiments: evidence counts, provider-call accounting, benefit verdict, blockers, schedule truth, and review ETA |
| `~/dashboard/markets` | Markets | Macro sentinel gauge, sector TradingView chart, insider trades, breadth |
| `~/dashboard/smart-money` | Smart Money | Options flow, insider signals, trade queue; 4 tabs; both markets |
| `~/dashboard/india` | India | NSE score tracker, ? paper portfolio, Kite live holdings, India order form |
| `~/dashboard/live-portfolio` | Live Portfolio | All Robinhood account positions, CSV import, trade enrichment, performance chart |
| `~/dashboard/journal` | Decision Journal | Trade decision log, signal->fill->outcome linking |
| `~/dashboard/mentor` | Mentor | MentorAgent coaching insights |
| `~/dashboard/briefing` | Briefing | Latest briefing, send history |
| `~/dashboard/backtest` | Backtest | Validation Engine replay, results |
| `~/dashboard/strategies` | Strategies | Strategy Registry (Champion/Challenger list + promote/retire/reject) |
| `~/dashboard/scanner` | Scanner | US + India universe scan, NIFTY-100 live fallback |
| `~/dashboard/settings` | Settings | Risk profile, market focus, live order limits, broker connections |
| `~/dashboard/admin` | Admin | User management, role/tier updates, DB cleanup |
| `~/dashboard/admin/vault` | API Vault | Runtime API key management |
```

02-tech-stack

Source: docs\arch\02-tech-stack.md | Authoritative operational architecture

Kairos - Tech Stack

2026-07-31: Daily scoring now has a provider-independent completed-session boundary. Yahoo is the current primary US daily-candle source and the India fallback behind Upstox; after normalization, ResearchAgent removes the current market-local bar until 16:00 ET / 15:30 IST. The separate Yahoo deep-history adapter remains an offline/replay input and does not authorize strategy promotion.

2026-07-31: Reported fundamentals are event-aware and ADR-safe. ResearchAgent batch-reads earnings_calendar before provider work: a report in the prior 3 or next 14 days uses a 1-day cache; otherwise or when unknown it uses 7 days. Finnhub issuer profiles use 30 days. Theme Scout validates candidates with a quote, never a full fundamentals fetch. Reviewed exchange-listed ADRs use Yahoo ADS-basis fundamentals only; an unavailable ADR response cannot fall through to a provider that resolves the foreign ordinary share.

2026-07-28: Yahoo daily candles became market-agnostic in lib/data/yahoo-candles.ts (fetchYahooCandles + yahooRange). Measured five-year depth: AAPL 1254 bars and RELIANCE.NS 1239 bars. This is the free keyless deep-history fallback when Massive's entitlement rejects older US history. Range boundaries guarantee the returned window is not shorter than requested.

Last updated: 2026-07-22 (Yahoo Finance v8 chart promoted to primary US candle source; recency guard added to fetchUsCandles; minCandles default raised 15->60; Massive/EODHD/TwelveData demoted to fallback)

Prior: 2026-07-19 (webull_trade signed sender restored behind database-backed preflight and nine-gate one-shot permits; adapter and activation remain DISABLED; read-only Cloud MCP remains query-only)

Prior: 2026-07-16 (House Stock Watcher congressional feed retired - upstream bucket went private; no licence-clean free replacement qualified)

Update this file when: a new library is added, a provider changes, a new adapter is added, the framework is upgraded, or any layer in the table below changes.

1. Frontend + framework

```
| Layer | Technology | Notes |
| Framework | **Next.js 15** (App Router) | All pages under `app/`; both RSC and client components |
| Language | **TypeScript** (strict) | Centralized types at `@/types` |
| Styling | **Inline styles + T tokens** | No Tailwind, no CSS modules. Each file declares `const T = { bg, surface, card, border, text, ... }`. Never add external class utilities. |
| Charts | **Recharts** | Used for paper P&L, score history, correlation charts |
| Diagrams | **Mermaid v10** | v11 webpack-broken; agent flowcharts rendered by `components/dashboard/MermaidChart.tsx` |
| TradingView | **Advanced Chart widget** | Sector ETF charts on Markets page; symbol detail page |
| Fonts | JetBrains Mono (monospace for numbers/code) | Loaded as system font fallback |
```

2. Backend + data

```
| Layer | Technology | Notes |
| Database | **Supabase (PostgreSQL)** | All tables, RLS policies, migrations in `supabase/migrations/` |
| Auth | **Supabase Auth** | Email/password; `OWNER_EMAIL` gate on all sensitive API routes |
| Storage | Supabase (not used for large files) | - |
| Vector store | **Supabase pgvector** | `trade_memories` table; 1024-dim Jina/Voyage embeddings; cosine similarity |
| Server functions | **Next.js API routes** | All under `app/api/`; `force-dynamic` where needed |
| Cron (cloud) | **Vercel Crons** | Two crons in `vercel.json`; hit deployed URL |
| Cron (local) | **Windows Task Scheduler** | 8 tasks via `scripts/run-agents.ps1`; PC must be on |
| Observability | **Langfuse** | Traces `callLLM` (single-shot) + `runAgentLoop` (multi-step tool calls) |
```


3. AI / LLM

Per-flow model selection (2026-07-12). Every agent/flow's model is chosen from Settings -> Agents -> LLM Config (agent_config table), NOT hardcoded. Routes read getConfiguredModel(svc, agentName, fallback) (lib/agent-model-config.ts) and pass the result to callLLM({ model }). MODEL_ROUTING in lib/llm-router.ts is now only the DEFAULT when no model is passed. Policy: default OFF Claude - hard-reasoning flows (research/trade/evaluate/thesis) default to deepseek-v4-pro with thinking explicitly enabled; cheap flows use deepseek-v4-flash with thinking explicitly disabled. Claude is opt-in per flow from Settings, never a silent default.

Providers (each key set in Settings -> Provider API Keys, vault-first / env-fallback):

```
| Provider | Concrete models | Dispatch in llm-router |
| DeepSeek | `deepseek-v4-flash` (fast/non-thinking), `deepseek-v4-pro` (thinking) | `callDeepSeek` |
| Anthropic | `claude-haiku-4-5`, `claude-sonnet-4-6`, `claude-opus-4-8` | `callClaude` (extended thinking auto-on for research/trade/evaluate/thesis) |
| Groq | `llama-3.3-70b-versatile`, `llama-3.1-8b-instant`, ... | `callGroq` |
| Google | `gemini-2.5-flash`, `gemini-2.5-pro` | `callGemini` (added 2026-07-12) |
| xAI | `grok-4-fast`, `grok-4` | `callGrok` (added 2026-07-12) |
```

Tier aliases (TIER_MODELS): fast->deepseek-v4-flash, reasoning->deepseek-v4-pro, claude-fast->Haiku, claude-smart->Sonnet. LEGACY_ALIASES transparently rewrites the retiring deepseek-chat/deepseek-reasoner aliases to the concrete V4 IDs. SAME_TIER_FALLBACK degrades a single unavailable model to a same-tier sibling (Gemini/Grok fall back to deepseek-v4-pro) and raises a System Health alert. A missing Anthropic key falls back to deepseek-v4-flash, not a crash. No local Claude-CLI/PowerShell path exists anymore - lib/claude-exec.ts was deleted 2026-07-12; all LLM + data fetches are HTTP.

Adding / changing models

To register a new model: call registerLLMProvider() in lib/llm-router.ts and update the relevant tier alias. A System Health alert fires automatically when any model is deprecated/renamed.

4. External brokers

```
| Broker | Market | Auth | Role |
| Robinhood | US | OAuth PKCE S256 loopback; token in `api_key_vault` | Read-only account (monitoring) + agentic account (orders only) |
| Zerodha Kite | India | Daily re-login; SHA256 checksum exchange; `KITE_ACCESS_TOKEN` in vault | Real NSE/BSE portfolio read + order placement |
```

Broker adapter layer

Location: lib/brokers/adapters/, lib/brokers/registry.ts

All broker interactions go through a BrokerAdapter interface. Current adapters:

```
| Adapter | File | Market |
| `robinhood-mcp` | `lib/brokers/adapters/robinhood-mcp.ts` | US |
| `robinhood` | `lib/brokers/adapters/robinhood.ts` | US (direct REST, serverless-capable) |
| `kite` | `lib/brokers/adapters/kite.ts` | India |
| `alpaca` | `lib/brokers/adapters/alpaca.ts` | US (future) |
| `webull_trade` | `lib/brokers/webull-trade/` | US - **built, DISABLED, transport fixture-tested** |
```

The adapter registry (lib/brokers/registry.ts) resolves which adapter to use per market and account role. The order placement code never calls a broker API directly - it goes through the registry.

webull_trade - signed Webull Trading API (2026-07-18, disabled)

Webull exposes two unrelated products and they must never be combined:

```
| Surface | Registry | Role |
```

```
| Cloud MCP (`webull`) | `lib/brokers/mcp-registry.ts` | **Read-only.** Accounts/positions/balances/quotes.
No order tools, no write scopes, `orderCapable:false`. Not in the order registry. |
| Trading API (`webull_trade`) | `lib/brokers/registry.ts` | **Signed REST money path.** The only Webull
order surface. |
```

The inert MCP-based order scaffold (`lib/brokers/adapters/webull.ts`) was deleted along with its `webull order-registry` entry - there is no second Webull order adapter.

Module layout (`lib/brokers/webull-trade/`):

```
| File | Purpose |
| `signing.ts` | HMAC-SHA1 request signing, implemented directly (no vendor SDK). One auditable canonical
request; fresh nonce per request; timestamp-skew guard; constant-time verify. |
| `credentials.ts` | Vault-only app-key/app-secret/access-token accessor, provider tag `webull_trade`,
bounded cache. **Sandbox and prod use separate env-prefixed vault keys bound to separate hosts derived from
the same `env`, so a sandbox credential can never resolve a prod host.** |
| `gates.ts` | All 9 gates of the Mandatory Gate Ladder, pure and ordered. Every gate fails closed; unknown/
undefined is never "satisfied". |
| `token.ts` | Token lifecycle `PENDING` / `NORMAL` / `INVALID` (15 idle days) / `EXPIRED` (5-min verify
window). Fails closed on pending/invalid/expired/unknown/idle; alerts before the 15-day boundary; no
keepalive traffic. |
| `preflight.ts` | Parses the official `POST /openapi/auth/token/check` response into the fresh server-
confirmed token record required by gate 7. |
| `order.ts` | Normalization + boundary rejects. Scope locked to **market + limit + GTC `STOP_LOSS` in
`CORE`. `SHORT` is rejected even though the API accepts it - a capability is not a permission. Options,
trailing, OCO/OTOCO, algos, extended/overnight sessions rejected. |
| `lifecycle.ts` | place/query/cancel/reconcile on a stable `client_order_id` (<=32). A timeout, a throw
after send, or a 200 with no parseable order id -> `needs_reconcile`: never success, never a blind resubmit.
|
| `capabilities.ts` | `BrokerProtectiveCapabilities` declaration (shape per `features/hybrid-stop`; that
shared type is not yet on `main`, so it is declared locally). Claims only US equity `sell_long` via GTC
stop-market in the **regular** session. |
| `transport.ts` | Sole Webull `fetch` path. Adds signed headers plus `x-access-token`, pins host/env,
wraps timeout/HTTP errors, and consumes one-shot capability permits. Preflight reads the database flag and
cannot reach order endpoints; production order permits require all nine gates. |
```

It places no order today and fails closed on all three of: `absent strategy_config.webull_trade_orders_enabled` (migration 20260718120000_... written but NOT applied), no allowlisted `broker_accounts{broker='webull_trade',market='us',role='trading'}` row, and no `api_key_vault provider='webull_trade'` credential. Everything is verified against fixtures with injected transports and fetch stubs only - no live or sandbox Webull call has been made, and the registry adapter still refuses submit/query/cancel before constructing a transport.

Before any live dollar the owner must: confirm the Webull API entitlement; reconcile the canonical signing layout, request paths, and hosts against the current official docs; provision the vault records and the allowlist row; apply the migration; and run a manually-approved sandbox test. Design + open decisions:
[features/webull-trading-api/FEATURE_ARCHITECTURE.md](#).

5. External data providers

Provider	What it provides	Auth
Alpha Vantage (AV)	Technicals (RSI, EMA, MA), 8 macro indicators; ****last-resort**** fundamentals/insider fallback (25/day cap, usually exhausted)	`ALPHA\_VANTAGE\_API\_KEY` in vault
Finnhub	****PRIMARY** domestic-US fundamentals** - `/stock/metric` (P/E, net margin, ROE, EPS, revenue growth) + `/stock/profile2` (sector), mapped to AV-OVERVIEW shape in `lib/data/fundamentals.ts: fetchFinnhubOverview`. Metric cache is event-aware (1/7d); profile cache is 30d. Reviewed ADRs bypass Finnhub because it can resolve the foreign underlying. Free 60/min, no daily cap	`FINNHUB\_API\_KEY` in Vercel env
Massive Market Data	****FALLBACK** US candles** (was primary; demoted 2026-07-22), stock screener, options, quotes; ****PRIMARY** US insider** - `/stocks/filings/vx/form-4` open-market P/S transaction scoring (`lib/data/massive-insider.ts`)	`MASSIVE\_API\_KEY` in vault
Yahoo Finance (fundamentals)	****SECONDARY** domestic US + PRIMARY India + PRIMARY reviewed ADR fundamentals** - `quoteSummary` (crumb handshake) -> AV-OVERVIEW shape via `fetchIndiaOverview` (market-agnostic: margin, ROE, revenue growth, P/E, sector). ADRs fail unavailable if Yahoo is thin; Kairos never mixes a foreign-underlying per-share response with a USD ADS price. Unofficial - cached, paced + fail-soft	None (crumb)
SEC EDGAR (fundamentals)	****NOT WIRED**** - `lib/data/sec-fundamentals.ts` companyfacts adapter is experimental; a 2026-07-13 spot-check found its raw-tag margin/ROE unreliable (needs the `frames` API). Left for a future task	None
FMP	`screen\_stocks` screener for US momentum/value buckets; ****secondary**** fundamentals fallback only (its `stable/ratios-ttm`+`key-metrics-ttm` are premium-gated on the free plan -> usually returns nothing, so Finnhub is primary)	`FMP\_API\_KEY` in vault
FinancialDatasets	Supplemental US screener discovery and manual Scanner fundamentals - ****metered** credits; currently \$0 balance**. Entitlement/HTTP/timeout failures report a self-healing System Health warning and fall back to non-FD candidate queues; this provider is NOT used for ResearchAgent scoring fundamentals (Finnhub/Yahoo own that path; SEC remains experimental and unwired).	`FINANCIAL\_DATASETS\_API\_KEY` in vault
Jina AI	`jina-embeddings-v3` embeddings (1024-dim, free) + `jina-reranker-v2-base-multilingual` reranker (free, 1M tokens/month, no CC)	`JINA\_API\_KEY` in `.env.local`
Langfuse	LLM trace/generation observability	`LANGFUSE\_PUBLIC\_KEY`, `LANGFUSE\_SECRET\_KEY`, `LANGFUSE\_HOST`
Resend	Transactional email (briefings)	`RESEND\_API\_KEY`
Stripe	Subscription billing (Pro/Elite tiers)	`STRIPE\_SECRET\_KEY`, `STRIPE\_WEBHOOK\_SECRET`, `NEXT\_PUBLIC\_STRIPE\_PUBLISHABLE\_KEY`
Yahoo Finance	****PRIMARY** US candles** - `v8/finance/chart/{SYMBOL}?interval=1d&range=1y` (no key, no observed rate limit, 251 adjusted bars, works for US+India); also India `.NS` price+candles (free, no auth) + fundamentals (cookie+crumb). Recency-guarded in `fetchUSCandles`: stale source (newest bar > 4 calendar days old) is rejected before fallback chain runs.	None
NSE public JSON	Full equity list (`EQUITY\_L.csv`), insider trades (`corporates-pit`), option chain, ****daily FII/DII net cash flows**** (`fiidiiTradeReact`, `lib/india-macro.ts` - India macro input, verified reachable from Vercel 2026-07-12)	Cookie handshake via `lib/nse-data.ts` / `lib/india-macro.ts`
GDELT DOC 2.0	US fallback/theme discovery only. The India per-symbol scoring dependency was retired 2026-07-31 after 0/310 usable production observations and traffic-policy throttling.	None (public API)
NSE announcements + Google News RSS	India replacement ****shadow****: official company announcements plus bounded unofficial headline coverage. Stored in canonical evidence cache; no scorer or money-path reader.	None
~~House Stock Watcher~~	****DEAD** - REMOVED 2026-07-16. Was congressional stock trade disclosures. Bucket taken private: `us-east-2` -> HTTP 301 PermanentRedirect, `us-west-2` -> HTTP 403 AccessDenied; project unmaintained (sibling Senate Stock Watcher is 403 too). `/api/markets/insider-trades` no longer fetches - it returns an explicit `discontinued` state + a System Health alert (`markets-congress-source: discontinued`), and the Markets panel renders a permanent "Discontinued" note with no retry. ****No free replacement qualified**** Lambda Finance (`lambdafin.com/api/congressional/recent`) serves live data on HTTP 200 but its ToS forbid automated access ("any data mining, robots, or similar data gathering and extraction tools"; "personal, non-commercial use ... only") and it re-serves FMP data; Finnhub/FMP gate it behind a paid tier (violates free-cloud-only); the official House clerk feed publishes filing ****metadata only**** (no ticker/amount/side - trades are per-filing scanned PDFs needing OCR, explicitly out of scope). Restore only if a licence-clean free source appears.	- (retired)
SEC EDGAR	Form 4 CIK lookup + XML -> `evidence\_records`	None (public)
StockTwits	Social sentiment for US tickers	(check vault)
WandB	Experiment tracking (optional)	`WANDB\_API\_KEY` in `.env.local`
Robinhood MCP	JSON-RPC tool bridge for Robinhood agentic account	OAuth token in vault

6. Provider adapter layer (added 2026-07-10)

Swap providers by changing one env var. Each adapter implements a shared interface; callers never know which concrete provider runs.

Embedding provider

Env var: EMBEDDING_PROVIDER=jina|openai Location: lib/providers/embeddings/

Value	Provider	Model	Notes
`jina` (default)	Jina AI	`jina-embeddings-v3` (1024-dim)	Free 1M tokens/month; no CC required
`openai`	OpenAI	`text-embedding-3-small` (1536-dim)	Requires `OPENAI_API_KEY`

Rerank provider

Env var: RERANK_PROVIDER=jina|cohere Location: lib/providers/rerank/

Value	Provider	Model	Notes
`jina` (default)	Jina AI	`jina-reranker-v2-base-multilingual`	Free 1M tokens/month
`cohere`	Cohere	`rerank-english-v3.0`	Requires `COHERE_API_KEY`

Email provider

Env var: EMAIL_PROVIDER=resend|smtp Location: lib/providers/email/

Value	Provider	Notes
`resend` (default)	Resend	Requires `RESEND_API_KEY`; test-mode sends to `BRIEFING_TO`
`smtp`	Any SMTP server	Requires `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`

7. Key library files

File	Purpose
`lib/llm-router.ts`	LLM tier resolution, `callLLM()`, `runAgentLoop()`, Langfuse tracing
`lib/vault.ts`	Read/write runtime keys from `api_key_vault` table
`lib/av-cache.ts`	Alpha Vantage day-cache + daily budget guard
`lib/research-agent.ts`	Core scoring pipeline (deterministic, no LLM)
`lib/brokers/registry.ts`	Broker adapter registry
`lib/market-context.tsx`	`MarketProvider` / `useMarket()` hook
`lib/market-support.ts`	Coverage level map per route
`lib/india-data.ts`	Yahoo Finance + NSE data adapters
`lib/nse-data.ts`	NSE cookie-handshake adapter
`lib/system-health.ts`	`reportIssue()` / `resolveIssue()` helpers
`lib/autonomy.ts`	Autonomy ladder check; `AUTONOMOUS_LIVE_ENABLED = false` (constant)
`lib/validators/backtest.ts`	Validation Engine (deterministic replay)
`lib/evaluation/run-evaluation.ts`	Performance Truth Layer evaluation runner
`lib/robinhood-mcp-client.ts`	Robinhood MCP JSON-RPC client + token refresh CAS

03-agents

Source: docs\larch\03-agents.md | Authoritative operational architecture

Kairos - Agents

2026-08-01 documentation truth audit: reconciled the ResearchAgent screener,

five scoring dimensions, provider order, exact formulas, availability rules,

weight resolution, and breakdown veto against production code. Added a

plain-English report-card explanation beside the exact quantitative contract.

2026-07-31 event-aware/ADR correction: ResearchAgent batch-annotates company symbols from earnings_calendar before fetching fundamentals (1-day cache inside -3/+14 report window; otherwise/unknown 7 days; Finnhub profile 30 days). Theme Scout only proves a ticker with quote data. Reviewed US exchange ADRs persist asset_class='adr', skip structurally inapplicable Form 4 evidence, and use ADS-compatible Yahoo fundamentals without foreign-underlying fallthrough. SKHY is the reviewed Nasdaq SK hynix ADS; SKHYV, HXSCL, and HXSCLF are blocked substitutes.

2026-07-31 scoring data-truth correction: ResearchAgent is the sole authoritative

scorer; the legacy Supabase research-agent function is a 410 tombstone. Provider

sector taxonomies are explicit. Finnhub industries are crosswalked only when

unambiguous, unknown sectors omit relative-P/E scoring, and P/E outside (0, 200]

is unavailable for valuation. Fundamental, sentiment, macro, and insider inclusion

requires explicit availability. A weak close remains evidence, but only ATR-scaled

or high-volume declines can hard-veto. See

features/scoring-data-truth/FEATURE_ARCHITECTURE.md.

Last updated: 2026-07-22 (contract layer fix: macro.regime_inputs marked US-only in INTENT_CATALOG, INTENT_CLASSIFICATION, and SCORER_FIELD_CONTRACTS - contracts now match the pipeline reality that fetchMacroScore/applicableDimensions/persistObservedResearchEvidence have always enforced; shadow resolver no longer reports 0% India macro starvation; ETF analyst_score capped at 65 in ResearchAgent + archetypes; lane-comparison API added; Monte Carlo in BacktestPage; capital-rotation trigger unblocked - see history below)

Prior: 2026-07-17 (the asset allocator is now market-scoped too - India no longer inherits the US FRED regime for sleeve weights, and macro UNAVAILABLE means NO allocation rather than an untilted config echo; macro_score is US-only - India macro now honestly UNAVAILABLE instead of inheriting the US FRED regime; macro_regime reads are age-bounded (10d) + indicator-backed, fail-safe to UNAVAILABLE never to calm; SEC Form 4 URL fixed - US insider data had never resolved; insider availability now recovered from decision_observations.availability_mask at the smart-money boundary; insider symbol universe unioned across all broker accounts; ResearchAgent holdings: staleness-ordered rotation under the wall-clock budget + fail-loud holdings fetch, both markets; per-flow LLM from Settings; India GDELT news sentiment + live NSE FII/DII macro inputs; low-confidence-research quality alert; Trading Style presets govern the time-stop before a champion is promoted; India SEBI PIT evaluated as the India insider analog and REJECTED on live evidence - 0/34 India symbols clear the US open-market bar at 90d, ~70% of PIT rows are non-open-market (ESOP allotments marked "Buy"); India insider stays honestly UNAVAILABLE, pinned by tests/india-insider-not-wired.test.ts; macro_score is US-only - India macro now honestly UNAVAILABLE instead of inheriting the US FRED regime; macro_regime reads are age-bounded (10d) + indicator-backed, fail-safe to UNAVAILABLE never to calm; SEC Form 4 URL fixed - US insider data had never resolved; insider availability now recovered from decision_observations.availability_mask at the smart-money boundary; insider symbol universe unioned across all broker accounts; ResearchAgent holdings: staleness-ordered rotation under the wall-clock budget + fail-loud holdings fetch, both markets; per-flow

LLM from Settings; India GDELT news sentiment + live NSE FII/DII macro inputs; low-confidence-research quality alert; Trading Style presets govern the time-stop before a champion is promoted; Macro read (Agent Mind Phase 3) documented for the first time and scoped US-only - the India read is killed, not faked: BOTH its macro inputs (macro_regime AND the category='macro' learning_priors) are US-only and unmarket-tagged, and the priors were the live leak in prod row id=6, so gating only the regime would not have fixed it; the route now refuses market=india on both verbs before any LLM call or DB write, and MacroReadCard states the gap instead of vanishing. Also fixed the 4-day silent outage of that agent - deepseek-reasoner burned the whole maxTokens: 600 budget on chain-of-thought and returned empty content (finish_reason=length) from 2026-07-13 onward, so nothing was written while both crons reported success; 600 -> 1500. Corrected the stale "looks back up to 3 weeks" macro line that contradicted this chapter's own 10-day consumer contract; macro_score is US-only - India macro now honestly UNAVAILABLE instead of inheriting the US FRED regime; macro_regime reads are age-bounded (10d) + indicator-backed, fail-safe to UNAVAILABLE never to calm; SEC Form 4 URL fixed - US insider data had never resolved; insider availability now recovered from decision_observations.availability_mask at the smart-money boundary; insider symbol universe unioned across all broker accounts; ResearchAgent holdings: staleness-ordered rotation under the wall-clock budget + fail-loud holdings fetch, both markets; per-flow LLM from Settings; India GDELT news sentiment + live NSE FII/DII macro inputs; low-confidence-research quality alert; Trading Style presets govern the time-stop before a champion is promoted; CPI Inflation now actually loads - MacroSentinel advertised 8 indicators and shipped 7 on every run since the FRED cutover because it fetched exactly 13 CPI readings and required 13, while FRED writes "." for a missing month (2025-10) that the adapter drops; YoY now aligns on the paired MONTH via fredSeriesDated + observationMonthsBefore rather than array index, since a gap-collapsed array makes vals[12] 13 months back - a wrong number, not just an absent one)

Update this file when: a new agent is added or removed, an agent's schedule changes, an agent's inputs or outputs change, or an agent's key behavior changes.

Adding an agent: create app/api/agents/<name>/route.ts + add cron entry in vercel.json (cloud) or scripts/run-agents.ps1 (local) + update this file (prose/registry only) + update public/agent-diagrams/system-map.json (the node, its edges, and a history entry - this is where the topology change lands) + add or update the per-agent diagram public/agent-diagrams/<agent>.json (same agentId/agentLabel/diagram/nodes/history contract; nodes is an object keyed by node id, never an array).

Agent coordination model

There are zero direct agent-to-agent HTTP calls. All coordination is via shared Supabase tables. Agents write and read a common set of tables; they never invoke each other's HTTP handlers directly.

The topology diagram lives in exactly one place: public/agent-diagrams/system-map.json, rendered at /dashboard/agents -> "System Map" (the default diagram view). It is the single source of truth for which agent hands what to which, and it carries a history audit trail of every edge change. This chapter deliberately does not redraw it - a second copy of the same edges does not add safety, it just gives a stale claim a second place to hide. (It already did: both copies asserted MACRO --> RESEARCH unqualified long after macro_score became US-only.) tests/arch-diagram-drift.test.ts enforces this.

The table below is the chapter's own altitude: which tables carry the coordination, not which arrows exist.

Table-to-agent matrix

```
| Table | Written by | Read by |
| `macro_regime`, `macro_signals` | MacroSentinel | ResearchAgent (**US only**), Macro read (**US only**), Dashboard |
| `macro_interpretations` | Macro read (**US only**) | Macro read's own GET -> `MacroReadCard` on `/dashboard/markets`. **Nothing else** - no scoring/sizing/gate/order/exit path reads it |
| `agent_signals` | ResearchAgent, DeepSeekAgent | PaperTrader, TraderAgent, Dashboard |
| `signal_score_history` | ResearchAgent | ResearchAgent (trend), Dashboard charts |
| `decision_observations` | ResearchAgent | LearnerAgent, Validation Engine, PerformanceTruth |
| `paper_positions` | PaperTrader | PositionMonitor, LearnerAgent, Dashboard |
| `paper_trades` | PaperTrader, PositionMonitor | LearnerAgent, MentorAgent, PerformanceTruth, **lane-comparison API** (closed trades -> strategy lane NAV curves + Monte Carlo) |
| `strategy_versions` | LearnerAgent, User | ResearchAgent, Dashboard |
| `trade_memories` | PositionMonitor | ResearchAgent (RAG retrieval) |
| `agent_alerts` | All reporters | Health-Triage, Dashboard, Briefing |
| `mentor_insights` | MentorAgent | LearnerAgent (context), Dashboard |
| `strategy_evaluations` | PerformanceTruth/Evaluation | Dashboard |
| `benchmark_scorecard` | BenchmarkScorecard route | PerformanceTruth dashboard, future gated learner evidence |
| `rotation_events` | PaperTrader capital-rotation shadow evaluator | Dashboard/audit, future rotation validation |
| `validation_experiments`, `model_artifacts` | Validation/Calibration | Promotion gate, Dashboard |
| `broker_orders` | Execution Gateway + order sync | Reconciliation, **Trading Dashboard** (live orders surface on `/dashboard/trading` - read-only, pending/submitted/filled/error/unknown_needs_reconcile statuses, last 20 rows, market-scoped) |
| `broker_order_events` (target) | Execution Gateway + order sync | Audit/reconciliation |
| `trade_proposals` (shadow) | AutonomousShadow | Dashboard, owner audit |
| `trade_proposals` (live) | AutonomousLive | Dashboard, reconciliation |
| `broker_order_events` | AutonomousLive, Execution Gateway | Audit/reconciliation |
| `llm_call_log` | All LLM callers | Admin cost view |
| `rag_traces` | ResearchAgent (retrieval) | Debug/audit |
```

Agent registry

MacroSentinel - the economist

File: app/api/agents/macro-sentinel/route.ts Schedule: Mondays 8:00 AM ET (Windows Task Scheduler) LLM: None - fully deterministic

Inputs:

- 8 Alpha Vantage macro endpoints: Treasury yield 10Y + 2Y, unemployment, real GDP, nonfarm payrolls, CPI, retail sales, federal funds rate, durable goods orders

What it computes:

$\text{danger_score} = ? (\text{indicator_value} \times \text{weight} \times \text{direction_sign})$

Each indicator has a hardcoded `direction_sign` (+1 bad, -1 good) and weight summing to 1.0.

```
| danger_score | regime |
| 0-24 | GREEN |
| 25-49 | YELLOW |
| 50-74 | ORANGE |
| >=75 | RED |
```

Outputs:

- One `macro_regime` row (current regime + score)
- One `macro_signals` row per indicator (raw_value + contribution + direction)

Key behavior: Advisory-only. MacroSentinel never auto-throttles agents or halts trading. User sees the regime first and decides whether to act.

Scope: US ONLY. Every indicator is a US series (yield curve, Sahm rule, US GDP, nonfarm payrolls, US CPI, US retail sales, fed funds, US durables). `macro_regime` has no market column, so its verdict is US-only regardless of which symbol reads it. It must never score a non-US symbol. Consumers are responsible for market-gating their own reads - `lib/data/scores.ts` does this for the money path (India -> macro UNAVAILABLE). A real India regime would be a separate agent + a market column; `lib/india-macro.ts` (FII/DII) is thesis evidence, NOT a regime.

Consumer contract for `macro_regime` (money path - `lib/data/scores.ts`): A row may score a symbol only if all hold. Otherwise macro is UNAVAILABLE -> excluded from the weighted score, weights renormalize. It never falls back to a calm/green default.

```
| Check | Rule | Why |
| Market | symbol is US | agent is US-only; table has no `market` column |
| Verdict | `regime != 'unknown'` | an unknown row's `danger_score` is a placeholder 0, not a calm read |
| Freshness | `week_of` within `MAX_MACRO_AGE_DAYS` = **10** | weekly cadence (7d) + 3d cron slack, and strictly < 14 so a fully missed run can never be masked by riding the prior week |
| Evidence | `raw_indicators` length >= 3 | mirrors MacroSentinel's own classification floor; rejects pre-guard legacy rows that stamped `green`/danger-0 off **zero** indicators |
```

`signals_triggered = 0` is NOT treated as suspect on its own - a genuinely calm week really does trip zero signals, and rejecting those would bias the book bearish. The honest discriminator is the indicator count.

Second consumer - the asset allocator (`lib/allocation/allocator.ts` + `lib/allocation/regime.ts`, 2026-07-17). `computeAllocation(svc, market)` maps the regime -> sleeve target weights; its equity target may only ever shrink that market's gross-equity cap in PaperTrader sizing. It applies the same four checks as the table above and imports `MAX_MACRO_AGE_DAYS` from `lib/data/scores.ts` rather than re-declaring the bound, so the two money-path consumers cannot drift on what "too old to act on" means.

It previously filtered `strategy_sleeves` by market but read `macro_regime` unscoped, so the US FRED verdict tilted India's sleeves - the same contamination class as the `macro_score` defect, and latent only because `allocation_enabled` defaults off (verified false in prod). India sleeves **do** exist in prod (equity 70 / defensive 20 / cash 10), so this was live-capable, not theoretical. India now returns UNAVAILABLE without querying `macro_regime` at all.

Macro UNAVAILABLE -> NO allocation (null), not an untilted allocation. The regime is the only input that turns owner-configured sleeve rows into an allocation; emitting the base `target_pct` with `tilt=0` would be byte-identical to a real "macro says neutral" verdict - the neutral-50-and-included fake in a different hat. For India this is permanent until a real India regime is built; whether India should instead run **static** sleeve targets is a product call that is deliberately left to the owner rather than silently made. Accepted tension, stated not hidden: because the equity target only ever shrinks the gross cap, null leaves the owner's configured `max_gross_exposure_pct` in force - **looser** than any regime outcome, including `risk_off`. That is the documented default (identical to `allocation_enabled=false`, today's live state), not a fabricated one; tightening on absent evidence would be an unapproved position change justified by nothing. Callers already handle null (it is the shipped-off path). `computeAllocationDetailed()` returns the same result plus a reason naming the specific rejected row(s).

Known gap (not a regression - pre-existing, reported 2026-07-17): `normalizeRegime` matches

bull/bear/risk_on/risk_off/aggressive/defensive, but `MacroSentinel` writes

green/orange/red/unknown. Every real label therefore normalizes to neutral, so the

*regime tilt is dead in prod today. Fixing the vocabulary would **activate** tilting - a real*

behaviour change requiring its own approval - so it was deliberately left alone.

Proposal (not applied - schema change): give `macro_regime` a market column so the table can

express what it actually is, instead of every consumer having to remember to market-gate its own

read. Until then, "US-only" is a convention enforced consumer-by-consumer, and each new consumer

is a fresh chance to reintroduce this exact bug.

Macro read (Agent Mind Phase 3) - the narrator

File: app/api/agent-mind/macro-read/route.ts (+ pure logic in lib/macro-read.ts) Schedule: weekdays - kairos-macro-read-us (13:30 UTC). Cadence: US only. Writes: macro_interpretations (one cached row/day/market) Read by: its own GET -> MacroReadCard on /dashboard/markets. Nothing else.

Turns the US macro regime + the US book + the system's US macro priors into a plain-English "what this means for your book" narrative.

NARRATIVE ONLY. This is the LLM sibling of macro_score, and it is on no money path: macro_interpretations is written here and read back by this route's own GET alone. The deterministic rule stands - no LLM on any scoring/sizing/gate/order/exit path - and this route must never widen its reach.

Scope: US ONLY. The India read is killed, not faked. (2026-07-17)

Both of this read's macro inputs are US-only and neither is market-tagged:

```
| Input | Why it is US-only | Market column? |
| `macro_regime` | MacroSentinel = 8 US FRED series (see above) | **No** |
| `learning_priors` WHERE category='macro' | US beliefs: Fed funds, DXY, 2Y/10Y curve, ISM/PMI, VIX |
**No** |
```

The priors were the live leak, not just the regime. Prod macro_interpretations id=6

(2026-07-13, market=india) reads: "...no regime-based bias can be assigned to this India book.

The system's high-conviction belief (80%) that rising Fed funds rates comp[resses]..." - against

a 13-position India book, while the regime was already unknown. So market-gating only

macro_regime would not have prevented that sentence. Prior id=8 ("Rising Fed funds rate

environment...", confidence 0.80) did it.

Why killed rather than "honestly scoped": once both US-only inputs are withheld, an India read has zero macro evidence left - only the held symbols. An LLM asked for a macro read with no macro input either restates a constant ("we know nothing" - which does not need a deepseek - reasoner call every weekday) or reaches into training knowledge for RBI/rupee/FII colour, inventing exactly what the prompt forbids. So:

- The route refuses market=india on both GET and POST before any LLM call or DB write - zero spend, zero rows, and the legacy India rows (ids 2/4/6) stay unreachable.
- MacroReadCard renders a deterministic not-supported note for India instead of vanishing.
- This also resolves a self-contradiction: the India Markets view already renders a NotSupportedNote for "...macro sentinel..." while a cron wrote an India read derived from that same US-only sentinel.

Parity means both markets get an *honest* answer, not that both get an LLM call. lib/india-macro.ts (NSE FII/DII) is deliberately not wired in as a substitute regime - that is a separate build.

US path reuses the same discipline as lib/data/scores.ts (unknown-guard, 10-day age bound, raw_indicators >= 3 floor, fail-safe to UNAVAILABLE). When no row qualifies the prompt states the regime is UNAVAILABLE and interpolates no regime, danger score, summary or indicators - an absent verdict is never described as calm. MAX_MACRO_AGE_DAYS is imported from scores.ts; the indicator floor is restated in lib/macro-read.ts (it is module-private in scores.ts) and must be kept in sync.

Known-open: the kairos-macro-read-india cron (jobid 47, 30 4 * * 1-5) is still active and should be dropped - a prod DB mutation, so it is flagged rather than done. The route-level refusal makes it a harmless no-op meanwhile.

ResearchAgent - the analyst (the brain)

File: `app/api/agents/research/route.ts`, `lib/research-agent.ts` Schedule: US 09:00 + 14:00 ET and India 09:30 + 12:30 IST on trading weekdays. US slots are DST-safe paired UTC jobs admitted by exact New York local time. Closed-day catch-up is non-executable. LLM: per-flow from Settings -> AI Models (agent_config row research, read via `getConfiguredModel`), default deepseek-v4-pro with thinking enabled. Writes thesis text only - scores are deterministic. (Was hardcoded Groq Llama pre-2026-07-12.)

Inputs:

- Account-scoped holdings snapshots. Research may analyze approved holdings, but only holdings verified on the actual order account can authorize a SELL.
- Watchlist from `watchlist` table
- Screener candidates from `FinancialDatasets screen_stocks` (US) or NSE universe cache (India) - dual buckets. `FinancialDatasets` is supplemental discovery only: missing credentials, exhausted credits, HTTP failures, and timeouts open a self-healing System Health warning and return no screener candidates; holdings/watchlist/carry-forward research continues, and scoring fundamentals are unaffected:
- *US fundamental-momentum screen*: revenue growth >15%, earnings growth >10%, gross margin >25%, ROE >15%, market cap >\$2B; ranked by revenue growth. It does not use RSI or a moving average.
- *US value screen*: $0 < P/E < 18$, FCF yield >4%, debt/equity <1, market cap >\$1B; ranked by lowest P/E. It does not compare P/E with a sector median or require insider/analyst activity.
- The two US lists are interleaved before the six-name screener cap, so one bucket cannot silently crowd out the other. India candidates come from the separately scored rotating `india_screen_cache`; the US criteria above are not applied to India.
- Score trend from `signal_score_history` (last 5 rows per symbol)
- Champion weights from `strategy_versions` WHERE `is_champion = true` AND `market = ?`
- Macro regime from the most recent usable `macro_regime` row - US symbols only. A row is usable only if it has a real verdict (`regime != 'unknown'`), is within `MAX_MACRO_AGE_DAYS` (10) of `week_of`, and rests on ≥ 3 real indicators. Otherwise macro is UNAVAILABLE (excluded, never defaulted to calm). India never reads this table. (`lib/data/scores.ts`, 2026-07-16)
- RAG memory via `retrieveSimilarTrades()` (if Jina embeddings are configured - Voyage was replaced by Jina free tier 2026-07)
- India news/event replacement shadow - NSE corporate announcements + bounded Google News RSS, persisted to the canonical evidence cache but not read by scoring (2026-07-31). The old zero-output GDELT dimension is retired.
- India FII/DII net cash flows - live NSE (`lib/india-macro.ts`), injected into the India thesis prompt (2026-07-12)

Webull boundary: Webull analyst/extended research is collected by the Evidence Router shadow, not called synchronously by `processSymbol`. The old inline path could issue up to nine MCP tool calls per US symbol and repeatedly exhausted the per-symbol deadline. Authoritative research may consume Webull only after a separately approved cache-only reader passes Router parity; until then it cannot consume scoring capacity, alter scores/directions, or delay holdings research.

Current production baseline (`deterministic_v1`) - 5 dimensions:

Plain-English model: a five-subject report card

Think of each stock as receiving a report card. ResearchAgent, not the LLM, is the teacher doing the arithmetic:

```
| Subject | Child-friendly question | Important limitation |
| Fundamentals | "Is the business healthy and reasonably priced?" | Quarterly company facts move slowly. An analyst target is written in the evidence, but currently earns **zero points**. |
| Technicals | "Is the settled price trend healthy, or did the stock just break down?" | Uses completed daily bars, not today's unfinished candle. It describes price behavior; it does not know the business. |
| Sentiment | "Are enough real news/social observations leaning positive or negative?" | Tiny samples are pulled toward 50. No data means the subject is omitted, not treated as a neutral vote. |
| Macro | "Is the current US economic weather supportive?" | US only. India does not inherit the Fed/FRED result. |
| Insider | "Are US insiders spending more money buying than selling in real open-market trades?" | US domestic issuers only; sparse data and ADR/India inapplicability are omitted. |
```

Every available subject receives 0-100. A missing or structurally meaningless subject is removed and the remaining weights are rescaled. Fewer than two usable subjects means abstain. The resulting `analyst_score` is a ranking/eligibility input, not a promised return or a predicted price. The configured LLM may explain the facts, but it cannot change any subject score, weight, direction gate, or order.

```
| Dimension | Source | What it measures |
| `fundamental_score` | Finnhub -> Yahoo -> FMP -> AV reserve (domestic US); Yahoo-only ADS basis (reviewed ADR); Yahoo (India) | Sector-relative P/E when taxonomy maps safely, profit margin, ROE, EPS sign, and revenue growth YoY. Analyst target upside is persisted as `observational_only` and contributes no points. Cache is 1d near earnings, otherwise 7d. |
| `technical_score` | Yahoo -> Massive -> EODHD -> Twelve Data -> AV reserve (US); Upstox -> Yahoo (India), all normalized to completed daily candles | RSI(14) continuous curve, EMA20/50, 20d trend, volume confirmation, and an ATR/high-volume breakdown veto. A weak bottom-quartile close is warning-only. A still-forming daily bar never enters scoring. |
| `sentiment_score` | StockTwits + GDELT first; AV NEWS_SENTIMENT only when both free sources are unusable (US only) | Sample-shrunk US news/social bullishness. StockTwits requires >=5 tagged messages. When both social and news exist they blend 40%/60%. **India: structurally not applicable in the active scorer**; replacement headline/event evidence is shadow-only. |
| `macro_score` | `macro_regime.danger_score` - **US ONLY** | Macro backdrop from MacroSentinel. **India: dimension is UNAVAILABLE and excluded - weights renormalize onto the remaining dimensions** (2026-07-16). MacroSentinel is US-only by construction (8 US FRED series) and `macro_regime` has no `market` column, so scoring an India symbol from it stamped the US Fed's verdict onto Indian equities. India research still injects a factual **FII/DII net-flow line** (`lib/india-macro.ts`, live NSE) into the thesis prompt - narrative grounding only, deliberately NOT wired into `macro_score`; a real India regime is a separate build. FII/DII is null (line omitted) when NSE geo-throttles Vercel. (2026-07-12) |
| `insider_score` | **US:** Massive Form 4 -> SEC EDGAR Form 4 -> AV INSIDER_TRANSACTIONS (cascade, `resolveInsider`, first `available:true` wins). **India: none wired** - the dimension is excluded, not scored. | 90-day open-market (P/S only) buy/sell **value** ratio. India's SEBI PIT feed (`fetchNseInsider`) was evaluated as the analog on 2026-07-17 and **rejected**: 0/34 live India symbols clear the US >=3-open-market-txn bar at 90d, and only ~30% of PIT rows are open-market at all (ESOP allotments are marked "Buy"). See the India insider block below. Its `agent_signals.insider_score` default-fills `50`, but `decision_observations.availability_mask.insider` is `false`, which is the honest record - do not read the 50 as neutral evidence. (2026-07-17) |
```

Sub-score formulas are deterministic and fixed (hand-tuned priors in `lib/data/scores.ts` + `lib/data/technicals.ts`) - they are NOT agent/genome-mutable. Only the dimension weights evolve through an explicitly promoted champion. New candidate features flow through the validation/registry process, not by silently editing these formulas. Volume is scored; analyst-target upside is intentionally observational-only.

Technical calibration shadow (2026-07-21): EdgeScout now measures the exact `kairos_technical_score_v1` composite beside bounded `macd_atr_12_26_9` and `signed_adx_14` challengers. EdgeIC evaluates them independently for US and India and emits market-wide plus sufficiently broad sector diagnostic rows from the same already-fetched candles. This is measure-only: it does not add MACD/ADX to the score, create sector-specific parameters, or change any agent decision. Formula, dataset, segment, and run fingerprints preserve later recalibration history.

Calibration readiness monitor (2026-07-21): A weekly deterministic monitor collapses same-window provider revisions, counts only market-wide windows at least five calendar days apart, and publishes progress per edge/market/horizon. Six stable historical windows can request the PIT/walk-forward/cost/FDR validation build; four qualified validation windows can request shadow review. Both are review milestones, not lifecycle promotion or trading permission. The monitor makes no provider call and emits a one-time informational notice plus a warning if collection is stale >10 days.

Weighted composite (availability-masked + renormalized):

```
analyst_score = ? (dimension_score x effective_weight[dimension])
```

Missing/inapplicable dimensions are EXCLUDED and the remaining weights renormalized to sum to 1.0 (lib/scoring/weighted-score.ts); < 2 usable dimensions -> abstain (thin evidence), never a low score. Weight source: this market's promoted champion weights_snapshot; if none exists, the selected risk-profile static weights; if the profile is invalid, balanced F.30/T.25/S.20/M.15/I.10. The mandate's strategy tilt is then applied. The legacy signal_weights and learning_priors tables are not live-score fallbacks.

ETF score cap (2026-07-22): ETFs exclude fundamental + insider dimensions; after renorm, technical carries ~62% weight. Low-volatility ETFs (SGOV, VTV, SCHD) were scoring 77-80 - above single-name equities - and the capital-rotation shadow proposed selling PLTR to buy SGOV. A hard cap of ETF_SCORE_CAP = 65 is applied post-computation in both lib/research-agent.ts (main analystScore + challenger shadowScore) and lib/scoring/archetypes.ts (computeArchetypeScore for etf_trend). ETFs can still surface as SELL targets for held positions and can enter the book up to score 65; they cannot displace an equity candidate scoring >= 66.

Indicative trade plan (2026-07-20): ResearchAgent now records the latest scoring-candle close in decision_observations.price_at_decision and a versioned native-currency features.trade_plan. Eligible new longs get deterministic mandate-based approximate risk/target levels; rejected and held names do not. This adds no provider call and no LLM number. The plan is audit/presentation context only; a close older than seven calendar days is marked unavailable.

Low-confidence output (2026-07-12): ResearchAgent aggregates per-market evidence availability across a run; when >= 50% of scored symbols (min 2) were scored on < 2 of 5 dimensions, it raises a low-confidence-research:<market> System Health alert (warn/data) naming the commonly-missing dimensions, and resolves it when a run recovers. Surfaces *quality* gaps (thin data) alongside the existing *quota* alerts (provider budget).

Holdings ordering + fail-loud (2026-07-16, bug fix - both markets):

Holdings lead the batch and are exempt from the candidate cap, but they are not exempt from the cron's wall-clock budget (RESEARCH_BUDGET_MS, <=105s). Throughput is ~30 symbols/run; the US book was 56. Because holdings order was *stable* (broker capture order, later alphabetical), the budget decapitated the same tail every run, permanently - prod run a4530e8f (07-16) wrote signals for batch slots 1-30 and zero for slots 31-56, all holdings. AVGO (slot 55) went unscored 07-13 -> 07-16 while Risk Analytics advised trimming it. enqueueDeferred could not rescue them: gatherSymbols rebuilds holdings from the broker snapshot each run and addCandidate drops any symbol already in holdingSet, so a deferred holding re-entered at the same starved index forever.

- Holdings are staleness-ordered - least-recently-scored first (orderHoldingsByStaleness, lib/research/holding-symbols.ts), from this market's own agent_signals. The budget's cut now rotates, bounding worst-case staleness at ceil(nHoldings / throughput) runs instead of never. Applies to US and India identically.
- fetchHoldings fails loud - the old catch { return []; } made a broken holdings read indistinguishable from "owns nothing", so research would score new BUYS while blind to every position it might need to SELL. It now raises a research-holdings-fetch:us critical System Health issue and throws (no holdings visibility -> no run). PostgREST error is checked, not just data.
- India parity - a rejected Kite call raises research-holdings-fetch:india-kite (warn, auto-resolves) and continues on the paper book, since a token lapse is expected/recoverable; an India paper_positions DB fault aborts like US.
- Deferred holdings alert - any held symbol the budget misses emits a warn cron alert naming the symbols. This is the only signal that the book exceeds one run's throughput, since the deferral queue can't carry holdings.

*Known capacity limit (not fixed by ordering): with 56 holdings and ~30/run, *every holding every session* does not fit in one 150s maxDuration invocation. Rotation bounds staleness at ~2 runs; closing it fully needs a capacity decision (raise RESEARCH_PARALLEL, or split holdings into their own cron) - a design change, deliberately not taken here.*

Sub-score formula reference (deterministic_v1, exact values)

Each dimension outputs 0-100, clamped. Source of truth: `lib/data/scores.ts`, `lib/data/technicals.ts`.

Fundamental (`scoreFundamentals`) - base 50. At least two real fields among P/E, margin, ROE, EPS, and revenue growth are required before the dimension is marked available. The function emits a display placeholder of 55 for an ETF or missing overview, but that placeholder is excluded from the composite and is not evidence. Additive when available:

```
| Field | Bands -> points |
| P/E (sector-relative) | `ratio = pe / SECTOR_PE_NORM[sector]`. <0.7 -> +18 ? <1.0 -> +8 ? <1.4 -> -3 ? <2.
0 -> -12 ? >=2.0 -> -22 |
| Profit margin | >0.20 -> +20 ? >0.10 -> +10 ? <0 -> -20 |
| ROE (TTM) | >0.20 -> +15 ? >0.10 -> +8 ? <0 -> -10 |
| EPS | >0 -> +5 ? <=0 -> -10 |
| Rev growth YoY | >0.20 -> +15 ? >0.10 -> +8 ? <0 -> -10 |
| Analyst target upside `(target-price)/price` | **0 points.** Stored as
`analyst_target_mode='observational_only'` for audit/display only. |
```

SECTOR_PE_NORM: technology 30 ? communication 20 ? health care 25 ? consumer disc. 24 ? staples 22 ? industrials 20 ? materials 16 ? energy 12 ? financials 14 ? utilities 18 ? real estate 30. Missing/unmapped sector taxonomy omits the P/E component; it does not receive a made-up default norm. Nonpositive P/E and P/E >200 are also omitted.

Technical (`scoreTechnicals`) - base 50; <15 candles -> flat 50. Additive:

```
| Signal | Contribution |
| RSI(14) | continuous interp over anchors `(20,-20)(35,-16)(45,-5)(50,+2)(55,+12)(60,+25)(72,+25)(75,+6)(
85,-10)(100,-15)` |
| Price vs EMA50 | above +15 ? below -15 |
| Price vs EMA20 | above +10 ? below -10 |
| 20-day trend (?3% band) | up +10 ? down -10 |
| Volume vs 20d avg (direction-confirming) | in-direction >=1.5x -> ?8 ? >=1.2x -> ?4 (direction from EMA20/
trend; neutral context -> 0) |
```

Breakdown veto (`detectBreakdownVeto`, runs FIRST) - before the additive math, a deterministic crash/meme-reversal check caps the technical score at 20 when the last bar is down ≥ 2.5 ATR, or $\leq -7\%$ on $\geq 1.5x$ volume. A bottom-quartile close on a down bar is persisted as a warning only and does not veto. This fixed the case where a -12% high-volume reversal scored ~100 because RSI fell into the preferred band while price remained above its EMAs.

Sentiment (`fetchSocialSentiment` + `scoreSentiment`) - StockTwits and GDELT are fetched first. Fewer than 5 sentiment-tagged StockTwits messages is unavailable. Each available component is shrunk toward 50: social uses neutral prior $K=10$; news uses $K=5$. If both exist, $\text{sentiment} = 0.4 \times \text{social} + 0.6 \times \text{news}$; otherwise the available component wins. AV NEWS_SENTIMENT $((\text{sent}+1) \times 50)$ is called only when both free sources fail. The final value is excluded unless `has_data=true`; the label fallback exists for legacy shapes but cannot make a failed current fetch available.

Macro (`fetchMacroScore`) - US symbols only (India -> UNAVAILABLE, excluded, weights renormalize). 100 - danger_score from the newest macro_regime row that satisfies the full consumer contract above: real verdict (regime != 'unknown') and week_of within MAX_MACRO_AGE_DAYS = 10 and raw_indicators length ≥ 3 . Nothing qualifies -> UNAVAILABLE, never a calm default. (Corrected 2026-07-17: this line previously said "looks back up to 3 weeks", contradicting the 10-day bound documented in the consumer-contract table in this same chapter - the unbounded reach-back was the 2026-07-13 prod bug, not the intended behavior.)

Insider (`resolveInsider` -> Massive / EDGAR / AV) - 10 + $\text{buyRatio} \times 80$ where $\text{buyRatio} = \text{buyValue} / (\text{buyValue} + \text{sellValue})$ over 90 days, counting only open-market P/S codes (awards A, exercises M, gifts G, tax-withholding F are other and excluded - they are not conviction trades). Requires ≥ 3 transactions; <3, no data, ADRs, or fetch-fail -> available:false (excluded).

Reviewed ADR contract (2026-07-31): `lib/instruments/adrs.ts` is the explicit identity registry; suffix guessing is forbidden. ADRs run in the US/USD research and paper pools and are segmented as `adr` for evidence/learning. Because foreign private issuers generally have no Section 16 Form 4 stream, insider is not fetched and remaining applicable dimensions are renormalized. Live execution recognizes ADR as a non-fund equity but adds no permission: broker review, account allowlist, market controls, kill switches, portfolio gates, and broker acknowledgement all remain mandatory.

*Insider is expected to be sparse, and that is correct (EXPECTED_SPARSE_DIMS holds us: insider). Most Form 4 activity is compensation-related, not open-market buying, so an empty insider result is usually a real finding rather than a fault. That is exactly why a *failure* must never render as one - see below.*

India insider (SEBI PIT): evaluated 2026-07-17 -> DO NOT WIRE. India *does* have an insider analog - fetchNseInsider (lib/nse-data.ts) reads SEBI PIT (Prohibition of Insider Trading) disclosures from NSE /api/corporates-pit. It is built but deliberately not connected to insider_score. It failed the data-qualification bar on live evidence measured against the 34 real India symbols in the book:

```
| Test | Result | Bar |
| Coverage: any PIT row in 90d (the US window) | **2/34 = 6%** | - |
| Coverage: >=3 open-market txns in 90d (the US bar, `MIN_INSIDER_TRANSACTIONS=3`) | **0/34 = 0%** | fails |
| Widening to 365d | 1/10 large caps scorable | fails |
| Semantics vs US Form 4 (90d open-market P/S buy/sell **value** ratio) | only ~30% of PIT rows are open-
market (Market Purchase 9.7% + Market Sale 19.5%); the rest are ESOP allotments 29.2%, Off Market 22.1%,
Gift 5.3%, pledges, amalgamations | fails |
| Freshness / PIT lag (`intimDt` - `acqfromDt`) | p50 2d, p90 34d, max 71d; 83% disclosed after the
transaction | usable in principle |
```

A dimension available for 0% of the universe is not a usable dimension. Worse, PIT's tdpTransactionType marks an ESOP allotment as "Buy" - so a naive wiring would read routine equity compensation as insider conviction. A same-named field carrying different meaning across markets is worse than an absent one, and insider_score is a genome dimension that reaches paper buys. India's insider dimension therefore stays honestly unavailable: applicableDimensions() omits it for India, the availability mask excludes it, and the weights renormalize onto the remaining dimensions. Pinned by tests/india-insider-not-wired.test.ts. Revisit only if SEBI PIT open-market density rises materially - the 0%-at-90d measurement is the gate.

Two live defects found in the PIT read path while qualifying it (display-only, NOT on the money

path, not fixed here - no caller feeds scoring): (1) fetchNseInsider() with no symbol

returns {"data": []} unconditionally - NSE's market-wide PIT feed requires from_date/to_date,

which the function never sends. SmartMoneyPage.tsx calls /api/india/insider with no symbol, so

the India insider tab has never shown a row (same class as the 2026-07-16 EDGAR Form 4 404).

(2) The per-symbol call sends no date window either, so NSE returns the latest ~20 disclosures

regardless of age - RELIANCE's default response spans back to 24-Sep-2021 - while the

route reports available: trades.length > 0, which would present 5-year-old disclosures as

current. Both are cosmetic today precisely because nothing consumes them.

Form 4 URL contract (bug fixed 2026-07-16 - US insider data had never worked): the Form 4 XML must be resolved from filings.recent.primaryDocument in the submissions JSON, with the xslF345X0N/ prefix stripped (buildForm4XmlUrl, lib/data/edgar-insider.ts). Two traps: <accession>.xml is not a real EDGAR artifact and 404s for every filing; and primaryDocument itself points at the XSL *rendering*, which SEC serves as text/html - it returns 200 but contains no <rptOwnerName></nonDerivativeTransaction> tags, so it parses to zero transactions (a silent empty, worse than an error). The filename also varies by filer agent (form4.xml, tm2618092-2_4seq1.xml, wk-form4_1784149645.xml), so it must never be hardcoded. The archive path needs the unpadded CIK; the zero-padded form 301-redirects.

Availability is recoverable without a schema change. agent_signals stores only the score, so an unavailable 50 and a genuinely balanced 50 are identical there. The available flag is already persisted in decision_observations.availability_mask (linked by signal_id), which is what /api/markets/smart-money joins to filter highInsider. Note prod holds rows that are available:true and exactly 50 - so "treat any 50 as unavailable" is *not* a valid shortcut; it would misreport real balanced data as missing.

Known gaps (deliberately not yet added - see the hype-catch discussion / IC-gate path): relative-strength vs index, MACD/ATR, 52w-high proximity, EMA200; debt/leverage, FCF yield, EV/EBITDA, revenue *acceleration*, sector-relative margins; per-symbol macro beta; insider role/cluster weighting.

Target v2: asset/setup-specific PIT feature snapshots, comparable-universe rank, structural evidence confidence, contradiction/event gates, and deterministic action. The complete contract is `features/scoring-methodology/FEATURE_ARCHITECTURE.md`; v2 remains non-actionable until its lifecycle and validation gates pass.

LLM role: explanation, risks, catalysts, and a bounded evidence-citing veto only. It never generates score, probability, expected return, direction, weight, size, or lifecycle state.

Screener target: 3 candidates/day (not 5). With \$10k NAV and 10% sizing, max 10 positions. Daily churn of 5+ creates overtrading.

Outputs:

- `agent_signals` row per symbol (score + thesis + recommendation)
- `signal_score_history` row (append-only score history)
- `decision_observations` row (even for skipped/expired candidates)
- `rag_traces` row (if RAG ran)

DeepSeekAgent - the comparison analyst

File: `app/api/agents/deepseek/route.ts` Schedule: Weekdays 9:00 AM ET (parallel with ResearchAgent) LLM: DeepSeek deepseek-v4-flash with thinking disabled

Inputs: Same watchlist and screener pipeline as ResearchAgent.

Key behavior: Advisory comparison only. Current code asks DeepSeek for an LLM-generated `analyst_score`; it must be tagged `score_source='llm_advisory'`, `remain_status='advisory'`, and be structurally excluded from PaperTrader and TraderAgent. Future comparison should reuse the deterministic score and compare explanation/veto quality.

Outputs: `agent_signals` rows tagged `agent_label = 'deepseek'`

PaperTrader - the pretend-money trader

File: `app/api/agents/paper-trade/route.ts` Schedule: US 10:05 AM ET, India 4:35 PM IST (standalone crons, independent of research) LLM: None

Inputs:

- `agent_signals` WHERE `status = 'pending'` AND `created_at` is today (market timezone) AND `market = ?`
- the market's `trading_mandates.score_threshold` (canonical entry threshold; the legacy global threshold cannot loosen it)
- deterministic `score_source` and a strategy version currently in `paper_active` lifecycle
- `paper_portfolio` for pool cash
- `paper_positions` for existing open positions

Key behavior:

Signal freshness gate: Only fills signals created today in the market's own timezone (New York for US, Kolkata for India). Older signals are marked expired.

Claim-and-fill protocol (prevents double-fills):

- Claims a signal by stamping `claim_run_id` on the `agent_signals` row
- Opens paper position only if it still owns the claim

Position sizing:

- `position_size_pct` from champion genome (clamped to `strategy_config.position_size_pct`)
- Slippage model: 0.05% above mid
- Records `expected_price` and `realized_slip_pct` on every fill

Fill-bound risk plan: PaperTrader ignores research-time absolute levels. It binds stop and target to the fresh actual fill using the current market mandate, or the same-market/horizon MAE/MFE percentiles only after 60 eligible-long labels exist. Learned values are bounded to a 10% maximum stop and 40% maximum target; invalid or thin data falls back to the mandate. Planned-versus-bound percentages are appended to the Research Journal pipeline trail.

Risk gates (added 2026-07-09):

- Latched controls: both pause and trading-enabled controls are checked per market; a recovered kill-switch metric cannot silently re-enable entries
- Name cap: `trading_mandates.max_open_positions` per market (default 10), enforced again from the canonical DB value inside the row-locked fill RPC. It gates new names only and never liquidates an over-cap book.
- Re-entry cooldown: 5-calendar-day block after a position in a symbol closes
- Pyramid gate: New BUY only if fill price > existing `avg_cost` (no averaging down)
- Long-only for new positions: SELL signals only apply to symbols already held

Capital-rotation shadow (added 2026-07-13; trigger corrected 2026-07-22): When a candidate cannot be taken as-is - because the book is at its `max_open_names` cap or because it lacks cash - PaperTrader calls the deterministic rotation evaluator and writes one `rotation_events` row with the would-be source holding, edge, notional, and gate reasons. This is P0 measurement only: it does not sell, buy, create a proposal, or move cash. Paper rotation execution and live rotation proposals remain disabled/unbuilt behind `rotation_config`.

Until 2026-07-22 the evaluator was reachable only from the `insufficient_cash` branch. In practice the name cap binds first - the book exhausts its 10 slots long before it runs out of cash - and the cap check continued before the rotation call, so the evaluator was unreachable and `rotation_events` stayed empty for nine days with shadow enabled. The cap check now sets a flag instead of skipping; the candidate flows through the remaining gates (sector cap, re-entry cooldown, pricing, sizing) and is evaluated for rotation at the funding step. Rotation is slot-for-slot, so this cannot grow the book; a candidate with no viable rotation is still skipped with the same `max_open_names` reason.

Outputs:

- `paper_positions` row (new open position)
- `paper_trades` row (buy leg)
- `paper_order_events` row (submitted + filled events)
- Updates `paper_portfolio.cash` and `paper_portfolio.nav`
- `rotation_events` row when either `max_open_names` or `insufficient_cash` triggers a shadow rotation evaluation

PositionMonitor - the risk watcher

File: `app/api/agents/position-monitor/route.ts` Schedule: US 4:15 PM ET, India 6:35 AM ET LLM: None (exits are rule-based)

Inputs: All open `paper_positions` for the market; current prices.

What it does on each run:

- Fetch current prices for all open `paper_positions` in the market
- Update `highest_price` if today's price is a new high
- Run exit checks (in priority order):
- Time stop: age from `paper_positions.opened_at` > holding horizon -> close. The per-market Trading Mandate is authoritative unless its governance explicitly permits a promoted champion horizon within the mandate bounds.
- Trailing stop: `stop_loss = max(original_stop, highest_price x 0.93)` -> close if breached
- Price target: at target price -> partial profit-taking (sell half, move stop to

breakeven on remainder; US paper uses six-decimal fractional quantity, India remains whole-share; full close only when no valid partial leg remains)

- Score exit (immediate): a fresh same-market `deterministic_v1` score below the exit threshold with direction still long closes the position at once. The score must be no older than `trading_mandates.max_signal_age_sessions` (default 2). A stale/unavailable score can never close a position; price, stop, target, time-stop, and hedge exits remain active. Legacy score flags are revalidated and cleared when stale or no longer below the exit threshold.
- Direction-flip exit (debounced): a held-long position whose fresh signal flips to short below exit is NOT sold on the first session. It arms (staged in `paper_positions.exit_reason` as `direction_flip_armed:<session>`) and only confirms the exit once a strictly newer research session still flips; a one-session wobble disarms and the position is held. A flip on a position held fewer than `MIN_FLIP_HOLD_DAYS` (2) market days is ignored. Pure logic in `lib/trading/direction-flip.ts`. This debounce was added 2026-07-24 after 13/22 closed paper trades exited on same-week flips (min 1.3 days held).
- NAV drawdown circuit breaker: if weekly NAV return < -5%, set

`strategy_config.app_paused = true` and fire a critical System Health alert

- Benchmark sync: `upsert paper_performance.bench_nav` with today's VOO (US) / ^NSEI (India) price

On close:

- Call `execute_paper_exit`, which atomically realizes FIFO lots (including a closed slice for a partial lot), updates/deletes the position, credits the correct market pool, and writes the decision journal; any mismatch rolls the whole exit back
- Call `indexClosedTrade()` for RAG (if Voyage embeddings are configured)

TraderAgent - the live order proposer

File: `app/api/agents/trader/route.ts` Schedule: Weekdays 9:45 AM ET (after research settles) LLM: None

Inputs: eligible deterministic `agent_signals`. A numeric score alone is not live eligibility.

Current behavior: creates proposals. Manual submission is owner-gated through the hardened Execution Gateway in `app/api/broker/orders/route.ts`.

- manual (default): Creates `trade_proposals` rows with `status = 'pending_review'`.

Owner reviews and approves/rejects in the dashboard. Send invokes the deterministic Execution Gateway and broker preview/place sequence; no LLM supplies order parameters.

- auto (future L4): the old direct-submit branch is rejected. An authenticated worker may

call only the shared execution kernel, under deployment flag, expiring owner lease, atomic budget, and a `live_approved` scoring version. Auto BUY is blocked until live protective exits, partial-fill sync, and reconciliation are operational.

Current auto status: disabled. India auto is a separate architecture. Eventual L4 must support verified risk-reducing SELL before autonomous BUY.

Architecture doc: `features/live-auto-trading/FEATURE_ARCHITECTURE.md`

Outputs: `trade_proposals` rows (expire after 30 min in manual mode)

LearnerAgent - the strategy improver

File: `app/api/agents/learner/route.ts` (entry); `app/api/agents/learner-brain/route.ts` Schedule: Fridays 5:00 PM ET LLM: Claude Opus 4.8 (upgraded 2026-07-03)

Phase gate: Mutation blocked until 10+ closed trades per market exist.

Inputs (via tool-use loop - 9 tools):

- `get_closed_trades` - recent `paper_trades` with outcomes
- `get_signal_weights` - current champion weights
- `get_strategy_versions` - all challengers + their backtest results
- `get_decision_observations` - scored decisions (including skipped)
- `query_trade_decisions` - real historical enriched Robinhood trades by regime/action
- `propose_challenger` - write a new `strategy_versions` row with new weights + genome
- `run_validation` - trigger Validation Engine on the proposed challenger
- `get_mentor_insights` - recent coaching notes
- `semantic_search_decisions` - pgvector RAG over trade memories (if Voyage embeddings are configured)

What it proposes: A Challenger `strategy_versions` row containing:

- 5 dimension weights (must sum to 1.0)
- Genome: {`entry_threshold`, `exit_stop_pct`, `exit_target_pct`, `horizon_days`, `position_size_pct`, `sizing_mode`}
- Possibly: a Feature Registry entry (a new formula idea - never runs as code)

Auto-guard: Blocks mutation if last 3 runs have `win_rate` < 35%.

Governance boundary: Learner/LLM may propose hypotheses, feature specs, and bounded challengers. Deterministic fitting/optimizers produce numeric candidate parameters. Only Vaibhav may promote lifecycle state. Learner cannot activate weights, versions, thresholds, money limits, accounts, orders, or code.

Closed-loop closure (2026-07-05): When user promotes a Challenger to Champion, the promoted `weights_snapshot` is read by ResearchAgent on its next run.

Per-trade notes: 1-sentence outcome summary per closed trade written to `learning_log`.

Outputs: `strategy_versions` (Challenger row), `learning_log` entries

ThemeScout - the watchlist manager

File: `app/api/agents/theme-scout/route.ts` Schedule: Independent weekly `pg_cron` job, Sunday 8:00 PM ET during daylight time LLM: Per-flow `agent_config`; default Groq `llama-3.3-70b-versatile`

Inputs: Broad GDELT market headlines first. Alpha Vantage NEWS_SENTIMENT is a cached fallback; TOP_GAINERS_LOSERS is used only when neither news source returns usable headlines. Every LLM-suggested candidate must then pass a deterministic Yahoo US quote lookup. That lookup proves the ticker currently resolves to a positive market price; it deliberately does not fetch fundamentals, because discovery should not spend scarce fundamentals calls.

Key behavior: Identifies emerging themes (e.g. ai_infrastructure, clean_energy). Adds at most six verified US symbols to owner-scoped watchlist rows tagged by theme, with a seven-day expiry. It is discovery-only: additions enter a later normal ResearchAgent run and must pass the same deterministic scoring and eligibility gates as every other symbol. It is not awaited by ResearchAgent, so a slow scout cannot consume a research run's time or provider budget.

Outputs: New watchlist rows with source = 'llm_theme', theme, reason, auto_added = true, and expires_at; invalid tickers are quarantined.

MentorAgent - the coach

File: app/api/agents/mentor/route.ts Schedule: After position-monitor + learner runs LLM: Claude Sonnet 4.6 (claude-smart)

Inputs: Closed paper_trades + learner_insights + macro context.

Key behavior: Writes plain-English coaching insights to mentor_insights. Three types: pattern (what worked), lesson (what to change), warning (risk concentrations). Advisory only - never touches money, weights, or positions.

Mentor UI surfaces (all per-flow model from Settings -> AI Models; each response carries a meta: {agent, model, agentKind} for the AI-attribution chip, 2026-07-12):

- AI Coach (/api/agents/mentor-coach, mentor->deepseek-v4-pro) - a TRUE tool-using agent loop (runAgentLoop, <=10 steps, tools: query_behavior/learning_progress/market_context/read_principles).

- Ask the Agent (/api/mentor/ask, mentor-ask->deepseek-v4-flash) - UPGRADED from a fixed recent-rows snapshot to a tool-using retrieval agent (runAgentLoop, <=8 steps): tools lookup_symbol / recent_activity / list_open_positions / worst_and_best_trades -> answers on the EXACT data the question needs (targeted, not a generic LLM guess). SSE-streams the final answer word-by-word after the loop; emits meta as the first + last stream event.

- Judgment Coach (/api/mentor/evaluate, mentor-evaluate->deepseek-v4-pro) and Market

Thesis (/api/mentor/thesis, mentor-thesis->deepseek-v4-pro) - single grounded call LLM (no tool loop); evaluate injects the symbol's real research_packet + signal.

Outputs: mentor_insights rows

Health-Triage - the SRE

File: app/api/agents/health-triage/route.ts Schedule: Every 6h + on-demand from dashboard LLM: None - operational truth is deterministic

Read-only - can never change config, money limits, weights, orders, or code.

Inputs: Current open agent_alerts plus the latest run per agent/market in the last 48h.

Key behavior: Builds machine-readable structured_issues deterministically. An older failed run disappears from current triage once a newer successful run for the same agent/market exists. The snapshot carries its alert count and timestamp so Home can mark it stale against the live feed.

Dashboard display: SystemHealthCard on dashboard home. Green when clean. Severity-ranked. Deep-link fix hints. Tier-1 safe actions (retry, resolve info/warn) are one-click.

Outputs: Append-only health_triage snapshots and an agent_runs bookkeeping row. It never creates inferred alerts or modifies trading/configuration state.

AutonomousShadow - the execution dry-run

Files: lib/trading/execution-kernel.ts, lib/trading/autonomous-shadow.ts, app/api/agents/autonomous-shadow/run/route.ts (owner POST), app/api/agents/autonomous-shadow/cron/route.ts (CRON_SECRET POST) Schedule: Weekdays 07:30 UTC (30 min after research cron) LLM: None - fully deterministic

No broker calls in PA1. Purpose: prove the execution kernel fires, gates fire correctly, and shadow proposals accumulate evidence before live mode is ever enabled.

Inputs:

- strategy_config live_auto_* policy snapshot
- agent_signals (last 24h, score_source='deterministic_v1', direction=long, score >= threshold)
- Current broker_orders filled count (open positions)
- Today's trade_proposals with execution_mode='autonomous_shadow' count

Key behavior: For each qualifying signal, creates a trade_proposal row (execution_mode='autonomous_shadow', auto_run_id, auto_decided_at), then runs it through evaluateAutonomousExecution() - 9 ordered gates (see lib/trading/execution-kernel.ts). Updates proposal status to queued_auto (kernel approved) or manual_review_required (gate failed with named reason). Writes one decision_journal entry per run. Never touches broker APIs, never calls reserve_live_order_budget, never submits any order.

Gates (in order):

- AUTONOMOUS_LIVE_ENABLED deployment flag
- live_auto_enabled DB toggle
- Lease not expired (live_auto_enabled_until)
- Direction = long only
- Score >= score_threshold
- evidence_confidence >= live_auto_min_evidence_confidence (floor 0.6)
- Open positions < live_auto_max_open_positions
- Orders today < live_auto_max_orders_per_day
- Proposed notional <= live_auto_max_per_order_usd (skipped in PA1 - notional = 0)

In current deployment: gate 1 fires unless AUTONOMOUS_LIVE_ENABLED=true is set in Vercel env. When false, all proposals land on manual_review_required. Shadow accumulates evidence.

Outputs: trade_proposals rows (shadow), decision_journal run summary

AutonomousLive - the live submitter (PA3)

Files: `lib/trading/execution-kernel.ts`, `lib/trading/autonomous-live.ts`,
`app/api/agents/autonomous-live/cron/route.ts` (CRON_SECRET POST) Schedule: Weekdays 14:00 UTC
(10:00 AM ET, after research at 13:00 UTC) LLM: None - fully deterministic

Runs ONLY when:

- `AUTONOMOUS_LIVE_ENABLED=true` in Vercel env AND
- `strategy_config.live_auto_enabled=true` AND
- `live_auto_enabled_until` not expired AND
- `live_auto_mode_us='autonomous'` or `live_auto_mode_india='autonomous'`

Inputs:

- `strategy_config` policy + per-market mode columns
- `agent_signals` (last 24h, `score_source='deterministic_v1'`, `direction=long`, markets in autonomous mode)
- `live_account_snapshots` for NAV (account 605420660, max age 4h)
- `paper_trades` for Kelly calibration (last 100 closed)

Key behavior: Same 9-gate kernel as shadow, plus:

- Checks kill switches (`app_paused`, `security_locked`, `trading_enabled`)
- Checks `live_auto_mode_[market] = 'autonomous'` per signal's market
- Calls `computeAutonomousSizing()` for approved signals
- Calls `reserve_live_order_budget_v2` RPC (`p_execution_actor='autonomous_worker'`) - atomic
- Submits to broker:
- US: `rhPlaceMarketOrder()` via Robinhood REST API (direct, no MCP - unavailable in serverless)
- India: `placeEquityOrder()` via Kite Connect REST
- Updates `broker_orders`: `status=submitted` + `broker_order_ref`
- Appends `broker_order_events` row (`actor_kind='autonomous_live'`)
- Updates `trade_proposals`: `status=queued_auto` or `manual_review_required`

Per-market mode (migration 141):

- `off` - market skipped entirely
- `manual` - no live orders from this agent; owner clicks Approve in dashboard
- `autonomous` - live orders submitted per above

Safety: `approved_by_user=false` in `broker_orders`. Any gate failure = `manual_review_required`, no order. Budget exceeded (RPC throws) = skip, log. Broker error = `unknown_needs_reconcile`, budget stays reserved.

Outputs: `trade_proposals` (`autonomous_live`), `broker_orders`, `broker_order_events`, `decision_journal`

BriefingAgent - the daily email

File: `app/api/briefing/generate/route.ts` Schedule: Weekdays 8:00 AM (morning) + 4:30 PM (evening) ET LLM: Settings-controlled through `getConfiguredModel("briefing")`; default `deepseek-v4-flash`, for editor/outlook prose only.

Inputs: Latest signals, paper positions, NAV, macro regime, open System Health alerts, and the latest complete deterministic Holding Risk snapshots for every live account in the edition's market.

Key behavior: Generates morning and evening briefing emails. Morning: pre-market outlook. Evening: trade recap. Sends via Resend (or configured EMAIL_PROVIDER). Includes "Open Issues" band when System Health alerts are present. Its Live Holdings Risk band replays immutable per-account results (actionable/review postures first), exposes freshness/confidence/missing inputs, and never recomputes risk or combines accounts/currencies. LLM prose cannot change a HoldingRisk score or posture.

Outputs: briefings row, newsletters row (on successful Resend send)

Validation Engine

File: lib/validators/backtest.ts, app/api/agents/backtest/route.ts

Deterministic, no LLM. Replays Challenger vs Champion on the same PIT opportunity set. For v2 it must use purged/embargoed walk-forward folds, train-fold-only preprocessing/calibration, out-of-fold predictions, costs/turnover, and multiple-testing accounting. The existing five-weight replay remains a baseline, not sufficient proof for a new scoring architecture.

Eligibility gates:

- Sharpe ≥ 0.5
- Win rate $\geq 40\%$

Computes: Sharpe, Sortino, max drawdown, win rate, expectancy, alpha vs benchmark. If gates pass, sets `eligibility_passed = true` on the `experiment_runs` row. Promotion is blocked (HTTP 412) unless `eligibility_passed = true`.

Performance Truth Layer

File: lib/evaluation/run-evaluation.ts, /api/agents/evaluation/*

Mandate-aware, deterministic (no LLM), honesty-first evaluation panel on /dashboard/learning.

Evaluation metrics: Sharpe, Sortino, max drawdown, win rate, expectancy, profit factor, alpha vs benchmark, execution slip (mean realized vs 0.05% modeled).

Honesty rules:

- Fewer than 20 trades -> shows "too small" instead of a number
- Tainted trades are counted (P&L must not hide them) but labeled as tainted
- `health_label` summarizes: `insufficient_sample` -> `negative_or_zero_edge` -> `promising_but_unvalidated` -> `validation_required`

Downside Hedge Controller

File: app/api/agents/downside-hedge/route.ts, lib/trading/downside-hedge.ts

Deterministic US paper-book overlay. It combines MacroSentinel with locally computed SPY/QQQ confirmation and an audited off -> armed -> active -> exit_pending -> cooldown state machine. Generic agents block inverse ETFs; only the paper-only hedge RPC may buy unleveraged SH/PSQ when both settings flags are enabled. No LLM, live-order, broker, rotation, or cross-market path. It ships fully OFF.

P1 gate: Weekly Vercel cron counts closed evaluable trades per market. Fires a System Health info alert when ≥ 20 accumulate.

2026-07-17 Research and Exit Contract Audit

- The watchlist input is filtered by `research_enabled=true`, unexpired rows, and authoritative market; suffix inference is only a legacy fallback. US and India watchlist rows cannot enter each other's scoring pool.
- Holdings from paper and live snapshots are staleness-ordered and persist `agent_signals.is_holding=true`. PositionMonitor may only treat a holding-path signal as a reassessment input.
- Held positions receive the same deterministic evidence fetch, five-dimension scoring, availability mask, and market-local weights as candidates. They skip optional LLM narrative and RAG retrieval because those outputs cannot alter an action; this preserves wall-clock capacity for exit coverage and candidates.
- One worker from the existing concurrency pool is reserved for candidates while the remaining workers prioritize staleness-ordered holdings. Concurrency and provider quotas do not increase, but a large book cannot starve all discovery.
- Candidate overflow is carried forward with a stable original defer time and is pruned after six unsuccessful deferrals or seven days. Theme Scout rows expire after seven days, require an owner id, deduplicate per run, and cannot overwrite a durable manual row.
- Sentiment inputs are shrunk toward 50 using source sample size before blending. A five-message 100% bullish split therefore scores 67, not 100.
- Analyst targets are retained as observational evidence only. They do not alter fundamental or composite scores until a validation/promotion decision says so.
- India macro remains unavailable and excluded. It never queries or inherits the US-only `macro_regime`; effective weights renormalize over genuine India inputs.
- A stored held short below the entry threshold is not an independent exit gate. PositionMonitor still requires the lower entry threshold - hysteresis bound, a fresh score, and holding provenance. Stops and targets remain independent.
- Risk Analytics remains deliberately research-free in its risk formula. The UI may show the latest holding score and age beside the independent risk verdict; research cannot veto a concentration, drawdown, liquidity, or volatility risk.

Earnings-Aware Risk P0 (2026-07-29)

PaperTrader annotates an otherwise-eligible entry after its fill-bound stop/target plan exists and before either capital rotation or `execute_paper_fill`. TraderAgent attaches the same normalized block to `trade_proposals.risk_check_reasons`; its existing Alpha Vantage earnings blackout remains unchanged and authoritative. Policy version 1 is shadow-only: no agent reads the counterfactual verdict to change a score, quantity, stop, target, fill, rotation, or exit. PositionMonitor is deliberately not wired.

US event dates are cross-checked against the PIT calendar, Finnhub, Webull, and Robinhood. India records market-local proximity and always reports options as unavailable. Robinhood research tools are a hardcoded read allowlist disjoint from order/review/cancel tools.

Shadow Registry and Upgrade Path (2026-07-29)

`lib/shadows/registry.ts` is the descriptive inventory for every active, paper-active, armed, idle, or disabled evidence program. `/api/upgrade-path` adapts existing truth ledgers into a read-only status model; it does not create a parallel evidence table. `/dashboard/upgrade-path` refreshes that model every minute and reports market scope, schedule state, progress, collection rate, provider-call accounting, benefit evidence, blockers, and the separate activation gate.

The DashboardShell market switch is the only market selector. The API requires `market=us|india` and applies it to every market-bearing ledger query before aggregation. It never combines US and India readiness. A US-only program stays visible in the India view as `not_applicable`.

The registry has no write or activation capability. Estimated days are shown only for a declared target with a non-zero observed rate. Uninstrumented calls are labelled `unmetered` rather than `zero`. Capital rotation is labelled `paper_active`, not `shadow-only`. Any future shadow producer is incomplete until its ledger, schedule, safety boundary, and activation gate are registered.

Deep-Dive Debate (2026-07-31 correction)

`/api/agents/deep-dive` is an owner-only, on-demand LLM research surface. Both POST and cached GET use the owner gate before service-role reads. It remains advisory: only `deep_analyses` is written and no signal, score, paper position, proposal, or broker path consumes its verdict.

The route accepts canonical US class tickers and NSE/BSE `.NS/.BO` symbols. US uses Massive/cache quotes plus Alpha Vantage fundamentals; India uses Yahoo INR quotes and Yahoo India fundamentals and never borrows the US macro regime. The configured LLM provider is resolved before its key is checked. The diagram source is `public/agent-diagrams/deep-dive.json`.

04-database-schema

Source: docs\arch\04-database-schema.md | Authoritative operational architecture

Kairos - Database Schema

2026-07-28 (20260729021633_pit_snapshot_persistence.sql, applied + verified): US PIT universe snapshots are now written only through `persist_edge_pit_snapshot()`, a service-role-only SECURITY DEFINER RPC with pinned search path, per-market/date/policy advisory lock, complete-member validation, idempotent exact-match replay, and conflict refusal. PIT rows in `edge_universe_members` are append-only by trigger; legacy non-PIT EdgeScout rows keep their existing update behavior. The resolver policy is `us_pit_adv20_top400_v2`: rank uses a complete 20-session trailing point-in-time dollar-volume window, not one event day, and persists one top-400 superset so $n=200/n=400$ experiments use deterministic prefixes of the same snapshot. Verification proved insert->existing idempotency, conflict refusal, rollback cleanup, anon/authenticated EXECUTE=false, service_role=true. This remains measure-only and is read by no score or order path.

2026-07-28 (20260728120000_pit_universe_provenance.sql, applied + verified): migration extends `edge_universe_members` with point-in-time provenance rather than adding a parallel table - `is_point_in_time` (NOT NULL default false, so every pre-existing row correctly declares itself a survivorship-biased current-universe snapshot), membership_source, pit_policy_version, active_on_as_of, delisted_at, adv_value, adv_rank, snapshot_fingerprint, plus a partial unique index on (market, as_of_date, pit_policy_version, symbol) WHERE `is_point_in_time`. The promotion gate must refuse any experiment whose universe rows have `is_point_in_time = false`.

2026-07-29 (20260729132244_immutable_oos_experiment_manifest.sql, applied + verified): OOS experiment identity is committed before provider access. `backtest_experiments` now binds `edge_id`, formula version, horizon, validation mode, trial family/count, universe policy, fixed data cutoff, exact git SHA, schema-versioned validation spec, and canonical plan hash. The plan and variants are immutable after insert. Start/completion timestamps, result summary, variant count, policy binding, and realized universe/dataset/run hashes are each write-once. `oos_ic` rows cannot exist without the full manifest, and anonymous/authenticated table grants are explicitly revoked.

2026-07-28 (20260728090000_promotion_schema_repair_and_atomic_rpc.sql, applied + verified): promotion schema repair, done while both tables were empty. `strategy_policies.dsr -> t_margin_vs_trials` (it never held a Deflated Sharpe Ratio, only $t - E[\max t \text{ over trials}]$); `walk_forward_pass -> ic_stability_pass` (the legacy IC windows overlap ~98.4% and are not folds); new validation_mode NOT NULL CHECK in (purged_temporal_oos, walk_forward) so a legacy rolling-window result has no representable value and cannot be promoted; new `experiment_id` NOT NULL FK binding every policy to the frozen experiment that justifies it. The mutation trigger now compares to `jsonb(old) - 'superseded_at'` against the same for new instead of a hand-listed column set that left `dsr`, `pbo`, `walk_forward_pass`, `cost_adjusted_return`, `max_drawdown_pct`, `stability_score` and notes silently mutable; `superseded_at` and `backtest_experiments.policy_id` are both write-once. New `promote_strategy_policy()` RPC (SECURITY DEFINER, EXECUTE revoked from public/anon/authenticated, granted to service_role only) does validate -> advisory-lock -> supersede -> insert -> bind in ONE transaction, replacing a two-round-trip supersede-then-insert that could leave a segment with zero active policies.

Last updated: 2026-07-27 (migrations 20260727140000_strategy_policies.sql + 20260727141000_backtest_experiments.sql): two promotion-layer tables for the deterministic backtest pipeline. `strategy_policies` is the versioned output destination - deterministic gate writes here, LLM never does; core fields are immutable, only `superseded_at` may be updated; one active policy per (market, sector, regime, horizon) at a time via partial unique index; `promoted_by = 'deterministic_gate'` enforced by CHECK. `backtest_experiments` is the lineage table - variant_budget committed pre-run (1-20), hypothesis/author/budget/segment are immutable after insert; variants_proposed <= variant_budget and variants_run <= variants_proposed enforced by DB constraints; result_summary is structured JSONB not free text; FK to `strategy_policies` set only on promotion. Both tables: RLS enabled, service-role-only, append-only core fields with trigger guards. Zero behavior change - tables are empty; promotion gate is Phase 2 (after router cutover).

Prior: 2026-07-27 (migration 20260727133000_robinhood_mcp_capability_snapshot.sql): `broker_mcp_capability_snapshots` is an append-only, service-only US/Robinhood contract-observation ledger. It records only tools/list names/count and a schema hash; no raw MCP schema, tool result, account, position, market-data, or order payload is persisted. RLS is enabled with browser roles revoked. It is outside the Evidence Router and cannot change scoring, paper, or live execution.

Last updated: 2026-07-25 (migration 20260725130000_etf_allocation_cap.sql applied: `strategy_config.etf_allocation_cap_pct` NUMERIC DEFAULT 30 - ETF portfolio allocation cap enforced at the execution gateway for BUY orders on US ETF symbols.)

Prior: 2026-07-19 (protective-orders schema hardened via 20260718130000_harden_protective_orders_invariants.sql - applied 2026-07-19 to zero-row table: (1) protective_orders.mode locked to 'wider_disaster_floor' only (old touch_at_analytical_stop branch removed at every layer); (2) currency NOT NULL; (3) protective_orders_market_currency_check - (us,USD) or (india,INR) only; (4) protective_orders_single_broker_id_check - at most one of broker_order_id/kite_trigger_id non-null, and exactly one when status is active/triggered/filled/canceling/canceled; (5) protective_orders_floor_is_wider_check - broker_floor < analytical_stop always; (6) protective_orders_kind_price_check - stop_market <-> no limit price, stop_limit/gtt_limit <-> limit price required; (7) protective_orders_learning_provenance_check - open position has learning_scope='full', disaster-floor exit records learning_scope='risk_policy_only'; sequence EXECUTE revoked from anon/authenticated, granted to service_role; trigger functions similarly locked.)

Prior: 2026-07-18 (20260718000000_protective_orders_shadow.sql - new protective_orders table + append-only protective_order_events audit log + learning_scope column on paper_trades/broker_orders + protective_orders_enabled (false by default) on strategy_config. 20260718120000_webull_trade_orders_enabled.sql - webull_trade_orders_enabled boolean (false by default) on strategy_config.)

Prior: 2026-07-16 (mandate capacity, score freshness, and RLS optimization through 20260716013100)

Update this file when: any migration adds, removes, or modifies a table, column, index, trigger, or RLS policy.

Latest schema addition: migration 20260716013000_mandate_capacity_and_score_freshness.sql adds per-market paper capacity and score-evidence age policy and hardens the fill RPC against caller-side cap loosening.

Security (2026-07-15, 20260715120000_security_rls_and_rpc_lockdown.sql): Supabase Security Advisor flagged 16 public tables with RLS disabled (anon-key readable). Enabled RLS deny-all on 15 agent-internal tables (macro_signals, macro_regime, mentor_dimension_logs, agent_config, learner_config, learning_priors, signal_weights_history, learner_runs, india_screen_cache, observation_labels, shadow_decisions, model_artifacts, feature_registry, validation_experiments, decision_observations) - service_role bypasses, agents unaffected. newsletters got RLS + an authenticated-SELECT policy (browser-read on /dashboard/intelligence). Also REVOKE EXECUTE ... FROM PUBLIC on anon-callable SECURITY DEFINER RPCs (kairos_call_agent, rls_auto_enable, handle_new_user, activate_evidence_policy, create_evidence_policy_version, claim_provider_refresh_jobs; get_daily_ai_count kept for authenticated), and pinned search_path=public on 15 definer/trigger fns. Advisor after: 0 ERROR, 0 anon-executable definer RPCs. Remaining WARN (deferred): 7 always-true service/authenticated policies (single-owner; tighten at multi-tenant), pg_net in public, Auth leaked-password toggle, 2 pgvector RPCs' search_path.

Stock Context (2026-07-15, 20260715150000_symbol_profiles.sql): new symbol_profiles display cache - PK (symbol, market), cols company_name, one_liner, sector, industry, exchange, market_cap_tier, next_earnings_date, peers text[], source, updated_at. RLS enabled + authenticated-SELECT policy symbol_profiles_authenticated_read (anon denied, service_role writes via bypass). OFF the money path - display-only (research/symbol pages); nothing in scoring/orders reads it. Filled by /api/agents/symbol-profiles/backfill from Finnhub (US) / Yahoo (India). Applied+verified in prod.

2026-07-15 batch (all applied+verified in prod, RLS-on):

- Earnings PIT capture (20260715210000, ...210100, audit hardening ...220000): earnings_calendar gains eps_actual_first, revenue_actual_first, actual_available_at, announcement_session, eps_basis, actual_currency, actual_currency, restated_eps, restated_available_at, restated_source, market; a trigger freezes first-observed actual fields once set. New earnings_consensus_snapshots stores consensus vintages with RLS auth-read and a database UPDATE/DELETE blocker. Consensus captured on/after the US report date is excluded. Finnhub does not prove GAAP versus adjusted basis, so its basis remains null and those observations are measurement-only, not an eligible PEAD cohort. Data-capture only for future PEAD/revision feasibility - no scoring effect. The post_earnings_drift archetype was renamed pre_earnings_proximity_reweight_v1 (it was never real PEAD); pead_* reserved.

- India Markets (20260715160000 + ...162000 RLS fix): india_market_snapshot India-only display cache (separate from US price_cache; INR by contract). NOTE: the original migration shipped with RLS *disabled* (anon-readable) - corrected to RLS-on + authenticated-read; a new public table must always enable RLS.

- Backfill cron (20260715150000_symbol_profiles_backfill_cron): schedules the existing profile backfill.

Paper autonomy safety (20260716010000, corrected by 20260716011000, lot parity hardened by 20260716012000): execute_paper_fill now atomically enforces the per-market 10-alpha-name cap, no-average-down pyramid rule, and entry-time mandate

provenance. New service-role-only `execute_paper_exit` atomically realizes FIFO full/partial lots, position state, native-currency cash, and the decision journal. The correction preserves `paper_trades.total_value` as a database-generated value for fills and split lots; exits fail closed unless aggregate open lots exactly match the position. Application entry gates separately honor latched per-market pause and trading-enabled controls. No live-order path changed.

Mandate capacity/freshness (20260716013000): `trading_mandates.max_open_positions` (1-50, default 10) and `max_signal_age_sessions` (0-10, default 2) are per-market owner policy. The fill RPC reads the canonical mandate cap and treats the caller parameter as tighten-only defense in depth. Cap changes are gate-only and never mutate positions. `PositionMonitor` ignores stale score/direction evidence while continuing every price-based exit.

Mandate advisor cleanup (20260716013100): the owner-read policy uses an `init-plan` (`select auth.jwt()`) and `updated_by` has a covering partial index. Access semantics are unchanged.

Watchlist market casing (2026-07-16, 20260716202749_normalize_watchlist_market_casing.sql): `watchlist.market` held THREE casings - 'us' (POST writes), 'US' (column DEFAULT, inherited by theme-scout which omitted the field), and the 'India' the GET filter looked for but nothing ever wrote. The original CHECK (`market in ('US', 'India', 'Global', 'Crypto')`) from `001_initial_schema.sql:192` had been dropped from the live table, which is why all three coexisted silently and the US view returned 7 of 249 rows. Migration normalizes every row to lowercase, restores a CHECK `market = ANY( '{us,india}' )` (verified to actively reject 'US'), and sets DEFAULT 'us'. 'Global'/'Crypto' were queried and had zero rows, ever - legacy fiction in the filter list, now unwritable. Convention: lowercase us/india everywhere - it matches every agent/query and the rest of the DB. NOTE: this migration and its code must ship together - after the backfill, the old capitalized filter matches nothing.

Router cutover prerequisites (2026-07-16, 20260716210000_router_cutover_prerequisites.sql): 4 new tables backing the pre-cutover machinery - the frozen dual-run evaluation cohort, its parity/divergence records, and the degradation-guard event log. All RLS-on with owner-read, anon revoked, authenticated SELECT-only, service-role writes; no_mutate append-only triggers on the three evidence tables; the activation RPC is security definer with `search_path=public` and service-role-only EXECUTE, and binds approval to the exact candidate version + baseline version + evaluation ID + eval code version + strategy version + market + expiry - a stale evaluation cannot authorize a policy (probed live: unknown/stale evaluation and invalid market are both refused). A schema CHECK additionally makes it impossible to persist a guard event that `*created*` an entry (guard is subtractive-only). `router_enabled` remains false for BOTH markets - 1 policy version each, both active, zero evaluations exist, so nothing can authorize a cutover even if the RPC were called. Verified in prod.

Migrations in `supabase/migrations/`. Applied via Supabase MCP `apply_migration` or the Supabase SQL editor. Always verify with `list_migrations` before shipping schema-coupled code. A migration file existing in the repo does NOT mean it ran against production.

Append-only ledgers (NEVER DELETE)

These tables must never be hard-deleted by any agent, cron, or cleanup job:

- `paper_trades` - financial ledger
- `paper_order_events` - event sourcing log (trigger blocks UPDATE/DELETE)
- `decision_observations` - learning fuel (trigger blocks UPDATE/DELETE)
- `broker_orders` - live trade audit trail
- `strategy_evaluations` - evaluation history (trigger blocks UPDATE/DELETE)
- `evidence_records` - immutable evidence ledger
- `holding_risk_runs` - daily per-holding risk run header (trigger blocks UPDATE/DELETE)
- `holding_risk_snapshots` - daily per-holding risk snapshot (trigger blocks UPDATE/DELETE)
- `account_risk_snapshots` - daily per-account risk snapshot (trigger blocks UPDATE/DELETE)
- `rotation_events` - capital-rotation shadow/execution audit ledger (trigger blocks UPDATE/DELETE)

8.1 Core user & auth

profiles

One row per user. Extended from `auth.users`.

Column	Type	Notes
<code>`id`</code>	uuid PK	References <code>`auth.users.id`</code>
<code>`email`</code>	text	
<code>`full_name`</code>	text	
<code>`role`</code>	text	<code>`user`</code> \ <code>`admin`</code> \ <code>`superadmin`</code>
<code>`subscription_tier`</code>	text	<code>`free`</code> \ <code>`pro`</code> \ <code>`elite`</code>
<code>`xp`</code>	int	Experience points
<code>`analysis_count`</code>	int	Total AI analysis runs
<code>`market_focus`</code>	text	Comma-separated: <code>`us`</code> , <code>`india`</code> (others removed 2026-07-05)
<code>`created_at`</code>	timestampz	

api_key_vault

Runtime-editable API keys (not in code or env files).

Column	Type	Notes
<code>`id`</code>	uuid PK	
<code>`provider`</code>	text	UNIQUE; e.g. <code>`alpha_vantage`</code> , <code>`kite`</code> , <code>`robinhood_oauth`</code>
<code>`display_name`</code>	text NOT NULL	Human-readable name for UI
<code>`key_value`</code>	text	Encrypted at rest by Supabase
<code>`updated_at`</code>	timestampz	
<code>`expires_at`</code>	timestampz	Optional; shown as "expiring soon" in vault UI

8.2 Strategy & configuration

strategy_config

Single-row table: the live risk profile + trading parameters.

Column	Type	Default	Notes
<code>`id`</code>	uuid PK		
<code>`risk_profile`</code>	text	<code>`Balanced`</code>	<code>`Conservative`</code> \ <code>`Balanced`</code> \ <code>`Aggressive`</code> (sizing/thresholds)
<code>`trading_style`</code>	text	<code>`position`</code>	**Migration 167 (2026-07-12).** <code>`swing`</code> \ <code>`position`</code> \
<code>`long_term`</code>	-	Settings -> Agents preset	that sets the four knobs below + <code>`target_hold_days`</code> . Orthogonal to <code>`risk_profile`</code> (style = horizon/tempo, profile = sizing).
<code>`target_hold_days`</code>	int	null	**Migration 167.** Holding horizon the PositionMonitor time-stop prefers ONLY before a champion genome is promoted; a promoted champion's learned <code>`horizon_days`</code> always wins. null = let the genome decide.
<code>`score_threshold`</code>	numeric	60	Minimum <code>`analyst_score`</code> to open a paper position
<code>`position_size_pct`</code>	numeric	10	% of pool NAV per trade (hard cap for genome)
<code>`stop_loss_pct`</code>	numeric	7	Default stop-loss % below entry
<code>`target_pct`</code>	numeric	20	Default price target % above entry
<code>`autonomy_level`</code>	text	<code>`L3_live_manual`</code>	Master gate for live order autonomy
<code>`robinhood_mcp_enabled`</code>	bool	false	Live US order path via Robinhood MCP
<code>`kite_enabled`</code>	bool	false	Live India order path via Kite
<code>`app_paused`</code>	bool	false	NAV circuit breaker auto-sets true; manual reset
<code>`live_auto_enabled`</code>	bool	false	DB toggle for autonomous shadow path (migration 139)
<code>`live_auto_enabled_until`</code>	timestampz	null	Owner lease expiry; null = no lease active
<code>`live_auto_policy_version`</code>	int	1	Snapshot version stamped on every proposal
<code>`live_auto_daily_cap_usd`</code>	numeric	null	Max USD spend per calendar day (null = uncapped)
<code>`live_auto_max_per_order_usd`</code>	numeric	null	Per-order notional cap
<code>`live_auto_min_evidence_confidence`</code>	numeric	0.6	Floor below which proposals fail gate 6
<code>`live_auto_max_open_positions`</code>	int	null	Max open broker positions
<code>`live_auto_max_orders_per_day`</code>	int	null	Max new proposals per calendar day
<code>`etf_allocation_cap_pct`</code>	numeric	30	**Migration 20260725130000.** Max % of US portfolio NAV in regular ETFs. BUY orders breaching this are refused (fail-open on read errors). SELL always allowed. India skipped.

agent_config

Per-agent configuration rows.

Column	Type	Notes
`agent_name`	text PK	e.g. `research`, `learner`, `macro-sentinel`
`enabled`	bool	
`schedule`	text	Human-readable schedule note
`model`	text	LLM model assignment (tier alias preferred)
`params`	jsonb	Agent-specific parameters
`updated_at`	timestampz	

strategy_versions

Champion/Challenger governance table.

Column	Type	Notes
`id`	uuid PK	
`market`	text	`us` \ `india`
`is_champion`	bool	True for the one active champion per market
`weights_snapshot`	jsonb	5-dim weights: `{fundamental, technical, sentiment, macro, insider}`
`genome`	jsonb	{entry_threshold, exit_stop_pct, exit_target_pct, horizon_days, position_size_pct, sizing_mode}
`proposed_by`	text	`learner` \ `user`
`backtest_result`	jsonb	Sharpe, Sortino, win_rate, max_dd from Validation Engine
`promoted_at`	timestampz	Null until promoted
`retired_at`	timestampz	
`created_at`	timestampz	

strategy_validation_automation (migration 170)

Per-market, owner-controlled policy for automatic deterministic challenger validation + shadow routing. Fail-closed: a missing row / read error is treated as fully disabled. Owner-email SELECT RLS; writes only via the service client (Settings PATCH) - authenticated cannot write. Seeded us/india both enabled.

Column	Type	Notes
`market`	text PK	`us` \ `india`
`enabled`	bool	Master switch - off = no auto validation for this market (challengers still created + manually validatable)
`auto_shadow_enabled`	bool	When on AND validation passes, auto-route the challenger into one `shadow_paper` slot
`max_active_shadows`	int	`0`-`1` (checked) - at most one shadow strategy per market
`updated_by`	uuid FK	-> `auth.users` (owner who last changed it)
`created_at` / `updated_at`	timestampz	

RPC `activate_strategy_shadow(p_version_id)` - SECURITY DEFINER, service_role only. The single automatic lifecycle transition: atomically flips a challenger to state=`shadow\_paper` under a per-market advisory lock, only if the policy is enabled + `auto_shadow_enabled`, the linked `validation_experiments` row `passed=true`, the version is not a champion / terminal state, and the market is under its `max_active_shadows` cap. Returns a typed reason (`strategy_not_found` / `automation_disabled` / `invalid_strategy_state` / `validation_not_passed` / `shadow_capacity_reached` / `already_shadow`) and cannot promote a champion, create a fill, move cash, or place an order. Driver: `runAutomatedValidation()` in `lib/validation/automation.ts`, called in-process by `LearnerAgent` when it creates a challenger (replaced the old fire-and-forget localhost request) and by the Friday `kairos-validation-sweep` recovery cron. Settings API: `GET/PATCH /api/settings/validation-automation` (owner-gated).

market_controls (migration 171)

Per-market pause / trading-enable state. Previously `app_paused` and `trading_enabled` were GLOBAL columns on the single `strategy_config` row, so a market's own circuit breaker (India NAV drawdown, US kill switch) flipped one shared flag and halted BOTH markets - an India phantom drawdown skipped the US research run (2026-07-13). Now one row per market. A market is paused / trading-disabled if either the legacy GLOBAL master (`strategy_config.app_paused/trading_enabled`) or its own row is set - helpers `isPaused(svc, market)` / `isTradingEnabled(svc, market)` in `lib/market-controls.ts` (fail-closed on read error). Writers: kill switch -> `setMarketTrading(market, false)`; drawdown breaker -> `setMarketPaused(market, true)`. Owner-read RLS; service-role writes. Seeded from the current global flags.

Column	Type	Notes
<code>`market`</code>	text PK	<code>`us`</code> \ <code>`india`</code>
<code>`paused`</code>	bool	New-entry pause (drawdown breaker / manual) - gates research-scored entries + autonomous-live entries; exits still run
<code>`trading_enabled`</code>	bool	Kill switch - <code>`false`</code> blocks this market's orders
<code>`paused_reason`</code>	text	Why (breaker reason or manual note)
<code>`paused_at`</code>	timestampz	

investment_mandates

Named strategy contexts for attribution and evaluation.

Column	Type	Notes
<code>`id`</code>	uuid PK	
<code>`name`</code>	text	e.g. <code>`Swing US 2-20d`</code>
<code>`market`</code>	text	<code>`us`</code> \ <code>`india`</code>
<code>`benchmark_ticker`</code>	text	<code>`V00`</code> (US), <code>`^NSEI`</code> (India)
<code>`horizon_days_min`</code>	int	
<code>`horizon_days_max`</code>	int	
<code>`is_default`</code>	bool	Used when no mandate explicitly specified
<code>`created_at`</code>	timestampz	

learner_config

LearnerAgent dimension-level controls.

Column	Type	Notes
<code>`id`</code>	uuid PK	
<code>`dimension`</code>	text	<code>`fundamental`</code> \ <code>`technical`</code> \ <code>`sentiment`</code> \ <code>`macro`</code> \ <code>`insider`</code>
<code>`learn_from`</code>	bool	Whether to include this dimension in learning
<code>`allow_mutation`</code>	bool	Whether LearnerAgent can propose weight changes for this dimension
<code>`updated_at`</code>	timestampz	

learning_priors

Current signal weight priors used by ResearchAgent.

Column	Type	Notes
<code>`id`</code>	uuid PK	
<code>`dimension`</code>	text	
<code>`weight`</code>	numeric	0-1
<code>`updated_at`</code>	timestampz	

learning_priors_history

Immutable audit log of every prior weight change.

Column	Type	Notes
<code>`id`</code>	uuid PK	
<code>`dimension`</code>	text	
<code>`old_weight`</code>	numeric	
<code>`new_weight`</code>	numeric	
<code>`reason`</code>	text	
<code>`changed_by`</code>	text	<code>`learner`</code> \ <code>`user`</code> \ <code>`factory_reset`</code>


```
| `created_at` | timestampz | Pruned >365d by DB cleanup |
```

experiment_runs

Backtest / Validation Engine run records.

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `strategy_version_id` | uuid | FK -> `strategy_versions` |
| `market` | text | |
| `sharpe` | numeric | |
| `sortino` | numeric | |
| `win_rate` | numeric | |
| `max_drawdown` | numeric | |
| `alpha` | numeric | vs benchmark |
| `trade_count` | int | |
| `eligibility_passed` | bool | Sharpe >= 0.5 and win_rate >= 40% |
| `created_at` | timestampz | |
```

observation_labels ATR exit evidence

Migration 20260722110000_atr_exit_evidence_labels.sql extends the canonical future-label table with entry_atr, entry_atr_pct, ATR-normalized MAE/MFE, atr_exit_outcomes, and atr_policy_version. Entry ATR comes only from the immutable decision observation's point-in-time technical evidence; unavailable ATR remains null. Outcomes are versioned, deterministic, close-observed, measure-only candidate labels. There is no trigger, RPC, or relationship from these columns to paper positions, cash, broker orders, or live execution.

strategy_evaluations

Append-only mandate-aware evaluation snapshots (Performance Truth Layer).

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `mandate_id` | uuid | FK -> `investment_mandates` |
| `market` | text | |
| `trade_count` | int | |
| `tainted_count` | int | Trades with low data_confidence |
| `sharpe` | numeric | |
| `sortino` | numeric | |
| `max_drawdown` | numeric | |
| `win_rate` | numeric | |
| `expectancy` | numeric | |
| `profit_factor` | numeric | |
| `alpha` | numeric | |
| `exec_slip_mean` | numeric | Mean realized slip vs 0.05% modeled |
| `health_label` | text | `insufficient_sample` \| `negative_or_zero_edge` \| `promising_but_unvalidated` \|
| `validation_required` | |
| `dataset_hash` | text | Dedup reruns on same trade set |
| `created_at` | timestampz | Append-only. Trigger blocks UPDATE/DELETE. |
```

8.3 Research + signals

agent_signals

One row per symbol per research run. The "today's score" table.

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `symbol` | text | Ticker |
| `agent_label` | text | `claude` \| `deepseek` |
| `market` | text | `us` \| `india` |
| `asset_class` | text | Current authoritative values: `us_equity` \| `adr` \| `etf` \| `india` \| `metal`.
| `adr` is a reviewed US exchange-listed depository receipt and remains in the US/USD book. |
| `analyst_score` | numeric | 0-100 composite |
```

```

| `fundamental_score` | numeric | |
| `technical_score` | numeric | |
| `sentiment_score` | numeric | |
| `macro_score` | numeric | |
| `insider_score` | numeric | |
| `recommendation` | text | `BUY` \| `SELL` \| `HOLD` \| `WATCH` |
| `direction` | text | `long` \| `short` \| `neutral` |
| `signal_breakdown` | jsonb | Per-dimension evidence detail |
| `thesis` | text | Groq-generated one-paragraph thesis |
| `data_confidence` | numeric | 0-1; below 0.5 -> tainted |
| `discovery_source` | text | How symbol entered the batch |
| `mandate_id` | uuid | FK -> `investment_mandates` |
| `status` | text | Includes `pending`, `paper_traded`, `expired`, `claiming`, `rank_rejected`, and non-
executable closed-day lifecycle states `weekend_staged` (legacy name), `superseded`, `revalidated` |
| `session_validated` | boolean | Positive entry/conviction-exit eligibility proof. Weekend catch-up writes
false; a weekday re-score writes a new true row. |
| `as_of_session` | date | Market-local completed session underlying the score. |
| `staged_at` | timestampz | Set only for non-executable weekend catch-up rows. |
| `claim_run_id` | uuid | FK -> `agent_runs`; prevents double-fill |
| `rank_pct` | numeric | Within-comparable-group percentile (migration 151); null until Pass-2 rank runs.
Check `[0,1]`. |
| `rank_rejected` | bool | True when candidate cleared the absolute floor but failed the cross-sectional
rank gate (migration 151); default false. |
| `created_at` | timestampz | |

```

weekend_staged rows (weekend or verified full exchange holiday) are evidence, not orders. PaperTrader and TraderAgent require positive session validation, as do the direct recent-signal queries in AutonomousShadow and AutonomousLive. CapitalRotation ignores unvalidated scores. PositionMonitor applies the same positive validation requirement to score/direction exits while its mechanical stop, target, trailing, and time exits remain independent. A weekday re-score writes a fresh row and moves the old staged row to revalidated; the staged row is never mutated into an executable decision.

signal_score_history

Append-only per-symbol score history. Never mutated after insert.

```

| Column | Type | Notes |
| `id` | uuid PK | |
| `symbol` | text | |
| `market` | text | |
| `analyst_score` | numeric | |
| `fundamental` | numeric | |
| `technical` | numeric | |
| `sentiment` | numeric | |
| `macro` | numeric | |
| `insider` | numeric | |
| `direction` | text | |
| `source` | text | `claude` \| `deepseek` |
| `created_at` | timestampz | Index: `(symbol, created_at DESC)`. Pruned >180d by DB cleanup. |

```

decision_observations

Immutable ledger of EVERY scored candidate (even ones not traded). The learning fuel.

```

| Column | Type | Notes |
| `id` | bigint PK | |
| `ts` | timestampz | Point-in-time decision timestamp. |
| `symbol` | text | |
| `market` | text | |
| `mandate_id` | uuid | |
| `analyst_score` | numeric | |
| `fundamental_score`...`insider_score` | numeric | Deterministic dimension scores. |
| `weights_used` | jsonb | Effective weights actually applied. |
| `availability_mask` | jsonb | Point-in-time evidence availability. |
| `features` | jsonb | Versioned evidence, quality, mandate, regime and indicative `trade_plan` snapshot. |
| `evidence_confidence` | numeric | Weighted structural coverage. |
| `discovery_source` | text | |
| `direction` | text | Deterministic entry/exit direction. |
| `entry_eligible` | boolean | Research eligibility, not execution authority. |
| `action` | text | Recorded research action. |
| `score_threshold` | numeric | Threshold used for this decision. |

```



```
| `price_at_decision` | numeric | Latest scoring-candle close in native currency; null on unavailable historical rows. |
| `currency` | text | `USD` or `INR`; markets are never cross-summed. |
| `signal_id` | uuid | Links the downstream pipeline audit. |
```

An update/delete trigger makes the table append-only. Research-time features .trade_plan is indicative only; PaperTrader re-prices from its fill.

research_packets

Full research context per run (for debug + audit).

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `symbol` | text | |
| `raw_data` | jsonb | Full scoring inputs + scores |
| `created_at` | timestampz | |
```

watchlist

Tracked symbols.

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `symbol` | text | UNIQUE with `user_id` |
| `market` | text | `US` \| `India` \| `Global` \| `Crypto`, NOT NULL default `US`. **Added migration 165 (2026-07-12)** - the route code (GET filter, POST) had read/written this column for months but no migration ever created it, so GET 500'd (panel showed "0 tracked") and every manual add silently failed. |
| `source` | text | `manual` \| `llm_theme` \| `tradingview_import` \| `robinhood` \| `briefing` |
| `theme` | text | AI-Scout theme label |
| `reason` | text | why added |
| `research_enabled` / `alert_on_signal` / `alert_on_earnings` | bool | per-symbol toggles |
| `created_at` | timestampz | |
```

edge_signals

Factor/edge lab signal rows.

```
| Column | Type | Notes |
| `id` | bigint identity PK | |
| `edge_id` | text | FK to `edge_catalog.edge_id` |
| `symbol` | text | |
| `market` | text | |
| `date` | date | Signal as-of session |
| `raw_value` | numeric | Raw factor value |
| `z_value` | numeric | Cross-sectional z-score |
| `universe_id` | text | Exact sampled universe reference |
| `created_at` | timestampz | Pruned >180d by DB cleanup |
```

edge_ic_history

Information Coefficient (IC) history per factor.

```
| Column | Type | Notes |
| `id` | bigint identity PK | |
| `edge_id` | text | |
| `market` | text | |
| `window_end` / `horizon` | date / int | Independent market window and forward-return horizon |
| `segment_type` / `segment_value` | text / text | Explicit `market/all` or diagnostic `sector/<name>` identity; sector rows never drive market lifecycle |
| `formula_version` | text | Versioned formula identity; currently the immutable edge ID |
| `dataset_fingerprint` / `run_fingerprint` | text / text | Frozen OHLCV dataset identity and exact experiment identity; exact reruns deduplicate, changed data/config appends |
| `ic` / `ic_ir` / `t_stat` | numeric | Rank IC diagnostics |
| `n_obs` / `universe_size` / `as_of_dates` | int | Confidence and breadth; never inferred from row count |
| `step_days` / `history_days` | int | Reproducible run configuration |
| `net_of_fee_ic` / `turnover` | numeric | Null until cost phase; null blocks capital promotion |
| `evidence_quality` / `provider_report` | text / jsonb | Retrospective/PIT quality and provider coverage |
```

`status_after`	text	Advisory horizon status only
`created_at`	timestampz	Append-only experiment timestamp; calibration history is retained

Migration 20260721130000 made this ledger append-only and replaced the old market-window overwrite key with a run fingerprint. Historical rows are preserved as market/all with legacy_unverified dataset provenance. The technical calibration trial family and sector sample policy are defined in features/technical-factor-calibration/FEATURE_ARCHITECTURE.md.

edge_market_status

Latest advisory lifecycle state per (edge_id,market), introduced by 20260718140000. It prevents India and US evaluations from overwriting one global catalog label. Includes latest_window_end, n_obs_min, evidence_quality, and per-horizon JSON diagnostics. Service-role write; authenticated/anon have no table grant. No money-path reader exists.

edge_readiness_status

Latest deterministic readiness projection per (edge_id, market, horizon). It stores policy version, independent-window progress, stability diagnostics, future cost-validation progress, next action, gate details, and separate first-notified timestamps for validation-build and shadow-review milestones. The immutable source remains edge_ic_history; this projection is service-role only and may be updated by the weekly monitor. No score, signal, mandate, position, or order reader exists.

edge_signal_inputs records actual observed_at, provenance_mode, and an input fingerprint. Original synthetic next-session rows are retained but marked legacy_unverified; they cannot prove point-in-time availability.

macro_regime

Current macro risk regime.

Column	Type	Notes
`id`	uuid PK	
`regime`	text	`GREEN` \ `YELLOW` \ `ORANGE` \ `RED`
`danger_score`	numeric	0-100
`computed_at`	timestampz	Most recent row is the live regime

macro_signals

Per-indicator breakdown for the current regime.

Column	Type	Notes
`id`	uuid PK	
`regime_id`	uuid	FK -> `macro_regime`
`indicator`	text	`yield_curve` \ `sahm_rule` \ `real_gdp` \ `nonfarm_payroll` \ `cpi` \ `retail_sales` \ `fed_funds` \ `durables`
`raw_value`	numeric	
`contribution`	numeric	Weighted contribution to danger_score
`direction`	text	`positive` \ `negative` \ `neutral`
`computed_at`	timestampz	Pruned >90d by DB cleanup

policy_rate_events, policy_rate_expectation_snapshots, policy_event_impacts

US-only FOMC event evidence (migration 20260726123000_policy_event_ledger). policy_rate_events is the mutable official schedule/outcome record. Expectations are append-only snapshots captured before the scheduled 2:00 PM New York decision; an expectation feed is intentionally unconfigured until a licensed source is available. Impacts are append-only 1- and 5-session returns from already-frozen symbol_daily_returns, with SPY-relative excess only when price bases match. All three tables are RLS deny-by-default with no browser grants and service-role only writes. They are display/measurement evidence only: no scoring, sizing, paper, live, or exit path reads them.

india_screen_cache

Full NSE universe cache (avoids re-scoring 5000 names each run).

Column	Type	Notes
`id`	uuid PK	
`symbol`	text	`.NS` ticker
`analyst_score`	numeric	
`scores_json`	jsonb	5-dim breakdown
`updated_at`	timestampz	Pruned >7d by DB cleanup

8.4 Paper portfolio

paper_portfolio

NAV state per market (cash + positions = NAV).

Column	Type	Notes
`id`	uuid PK	
`market`	text	`us` \ `india`
`cash`	numeric	Starting: US \$10,000, India ?1,000,000
`nav`	numeric	cash + sum of open position values
`peak_nav`	numeric	All-time high NAV (for drawdown circuit breaker)
`updated_at`	timestampz	

paper_positions

Open pretend-money positions (deleted on close).

Column	Type	Notes
`id`	uuid PK	
`symbol`	text	
`market`	text	
`qty`	numeric	Shares held
`avg_cost`	numeric	Fill price
`expected_price`	numeric	Pre-slippage decision price
`realized_slip_pct`	numeric	`fill/expected - 1`; execution quality signal
`fill_status`	text	`full` \ `partial` \ `failed`
`price_target`	numeric	Exit at this price
`stop_loss`	numeric	Hard stop (trailing or original)
`highest_price`	numeric	For trailing stop computation
`exit_reason`	text	`stop` \ `target` \ `llm_exit` \ `time_stop` \ `partial_profit`
`mandate_id`	uuid	
`opened_at`	timestampz	Age used for time stop

paper_trades

Closed paper trade ledger (append-only in practice; never hard-deleted).

Column	Type	Notes
`id`	uuid PK	
`symbol`	text	
`market`	text	
`action`	text	`buy` \ `sell`
`qty`	numeric	
`price`	numeric	Fill price
`expected_price`	numeric	Decision price
`realized_slip_pct`	numeric	
`fill_status`	text	
`exit_price`	numeric	Closing price (on sell)
`realized_pnl`	numeric	
`pnl_pct`	numeric	
`outcome`	text	`win` \ `loss` \ `break_even`
`exit_reason`	text	
`data_confidence`	numeric	From the signal that opened the trade
`tainted`	bool	Low data_confidence; excluded from learner training
`excluded_from_learning`	bool	Manually flagged

```
| `mandate_id` | uuid | |
| `market_regime` | text | `GREEN` \| `YELLOW` \| `ORANGE` \| `RED` at time of trade |
| `agent_label` | text | `claude` \| `deepseek` (P&L comparison) |
| `opened_at` | timestampz | |
| `closed_at` | timestampz | |
```

paper_order_events

Immutable append-only event log for every paper order state change.

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `paper_trade_id` | uuid | FK -> `paper_trades` |
| `event_type` | text | `submitted` \| `filled` \| `partially_filled` \| `cancelled` \| `error` |
| `price` | numeric | |
| `qty` | numeric | |
| `detail` | jsonb | |
| `created_at` | timestampz | Trigger blocks UPDATE/DELETE. |
```

paper_performance

Daily paper-book truth snapshots per market. PaperTrader and PositionMonitor share the same canonical derivation contract; neither may leave derived fields at database defaults. Returns are always measured from the fixed seed for that row's market (US \$10,000; India ?10,00,000), never from the first observed row or from another market.

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `date` | date | |
| `market` | text | |
| `nav` | numeric | |
| `cash_balance` | numeric | Same-market paper pool cash |
| `positions_value` | numeric | Same-market marked open positions |
| `daily_pnl` | numeric | NAV change from the prior same-market row; first row = 0 |
| `total_pnl` | numeric | `nav` - market_seed |
| `total_pnl_pct` | numeric | Cumulative return from the same-market seed, percentage points |
| `win_count` / `loss_count` | integer | Cumulative resolved paper outcomes for this market as of `date` |
| `win_rate` | numeric | Wins / all resolved outcomes (including breakeven), fraction 0..1 |
| `bench_nav` | numeric | Benchmark (VOO/^NSEI) NAV on this date |
| `bench_return_pct` | numeric | Benchmark return from its first recorded same-market observation |
| `alpha_pct` | numeric | `total_pnl_pct` - `bench_return_pct` |
| UNIQUE | `(date, market)` | One row per day per market |
```

live_performance (migration 169, RLS tightened 20260713112754)

Daily equity curve per LIVE account - the live analogue of paper_performance, backing the per-account Live Portfolio vs VOO chart. Robinhood's MCP exposes NO account-value history (get_equity_historical is per-symbol OHLC; get_portfolio is current-only), so this cannot be backfilled - it is accrued forward: each account-snapshot refresh (/api/live-account/refresh-snapshot, robinhood_mcp source plus connected registry MCP brokers such as Webull) upserts one row/account/day with real broker equity + that day's VOO close. Until >=2 real days exist for the selected accounts, /api/live-portfolio/performance falls back to a labeled constant-holdings estimate (estimated:true) reconstructed from current holdings x Massive symbol history. Service-role writes; owner-email authenticated SELECT only.

```
| Column | Type | Notes |
| `account_id` | text | Broker account ID; PK part |
| `date` | date | Calendar day (UTC); PK part |
| `equity` | numeric | Real broker account value (USD) that day |
| `bench_nav` | numeric | VOO close the same day (null until a refresh records it) |
| PK | `(account_id, date)` | One row per account per day |
```

8.5 Live trading

broker_accounts

Known broker account references (no passwords or credentials stored here).

Column	Type	Notes
`id`	text PK	Internal label, e.g. `rh-trading`, `rh-agentic`
`broker`	text	`robinhood` \ `kite`
`account_role`	text	`trading-read-only` \ `agentic-orders-only`
`market`	text	
`enabled`	bool	
`live_account_source`	bool	Whether this is the source for `live_account_snapshots`
`notional_cap_usd`	numeric	Hard per-order cap

live_account_snapshots

One row per account (upserted, not history). Live Robinhood positions.

Column	Type	Notes
`account_id`	text PK	Robinhood account ID (not human-readable role label)
`equity`	numeric	
`buying_power`	numeric	
`positions_json`	jsonb	Array of `{symbol, qty, avg_cost, current_price}`
`captured_at`	timestampz	

broker_order_events

Append-only event log for every live broker order state transition (migration 139). Protected by `boe_block_mutation()` trigger - UPDATE and DELETE are blocked at the DB level.

Column	Type	Notes
`id`	uuid PK	
`broker_order_id`	uuid	FK -> `broker_orders`
`event_type`	text	e.g. `status_change`, `fill`, `cancel`, `reconcile`
`from_status`	text	Prior status
`to_status`	text	New status
`actor`	text	`owner` \ `cron` \ `autonomous_shadow`
`detail`	jsonb	Event-specific payload
`created_at`	timestampz	

broker_orders

Immutable live order ledger. Never deleted.

Column	Type	Notes
`id`	uuid PK	
`broker`	text	
`market`	text	
`symbol`	text	
`action`	text	`buy` \ `sell`
`qty`	numeric	
`order_id`	text	Broker-assigned order ID
`status`	text	`pending` \ `filled` \ `needs_reconcile` \ `failed`
`needs_reconcile`	bool	True when broker order ID is missing
`submitted_at`	timestampz	
`filled_at`	timestampz	

trade_proposals

TraderAgent / AutonomousShadow proposals. Auto-expire 30 minutes.

Column	Type	Notes
`id`	uuid PK	
`symbol`	text	
`market`	text	`us` \ `india`

```

| `side` | text | `buy` \| `sell` |
| `order_type` | text | `market` \| `limit` |
| `qty` | numeric | |
| `limit_price` | numeric | |
| `analyst_score` | numeric | Score at proposal time |
| `estimated_value` | numeric | Kelly-sized notional (shadow only) |
| `pct_of_nav` | numeric | Fraction of live NAV (shadow only) |
| `price_at_proposal` | numeric | Quote price used for sizing (shadow only) |
| `thesis` | text | |
| `signal_id` | bigint | FK -> `agent_signals` |
| `status` | text | `pending_review` \| `approved` \| `rejected` \| `expired` \| `queued_auto` \|
`manual_review_required` |
| `execution_mode` | text | `manual` \| `autonomous_shadow` (migration 139) |
| `policy_snapshot` | jsonb | Full `LiveAutoPolicy` + kernel + sizing result snapshot |
| `auto_run_id` | text | `runAutonomousShadow` run ID |
| `auto_decided_at` | timestamptz | When the execution kernel evaluated this proposal |
| `expires_at` | timestamptz | `created_at` + 30m |
| `created_at` | timestamptz | |

```

decision_journal

Audit log of every trade decision (live or paper).

```

| Column | Type | Notes |
| `id` | uuid PK | |
| `symbol` | text | |
| `market` | text | |
| `action` | text | |
| `rationale` | text | |
| `approved_by` | text | `owner` \| `auto` (auto not used; kept for schema compat) |
| `created_at` | timestamptz | |

```

8.6 Learning

learning_log

LearnerAgent mutation audit log (not the same as trade outcomes).

```

| Column | Type | Notes |
| `id` | uuid PK | |
| `event_type` | text | `weight_change` \| `champion_promoted` \| `mutation_blocked` |
| `market` | text | |
| `detail` | jsonb | |
| `created_at` | timestamptz | |

```

signal_weights_history

History of every signal weight change (rollback source).

```

| Column | Type | Notes |
| `id` | uuid PK | |
| `dimension` | text | |
| `old_weight` | numeric | |
| `new_weight` | numeric | |
| `changed_by` | text | |
| `created_at` | timestamptz | |

```

trade_memories

pgvector store for RAG trade memory.

```

| Column | Type | Notes |
| `id` | uuid PK | |
| `paper_trade_id` | uuid | FK -> `paper_trades` |
| `text` | text | Setup description: symbol, scores, outcome |
| `embedding` | vector(1024) | Jina `jina-embeddings-v3` embedding |

```

```
| `metadata` | jsonb | Symbol, market, outcome, exit_reason, mandate_id |
| `created_at` | timestampz | |
```

8.7 Evidence & enrichment

evidence_records

Immutable evidence ledger. payload_hash deduplicates re-imports.

```
| Column | Type | Notes | | | |
| `id` | uuid PK | |
| `symbol` | text | |
| `source` | text | `edgar_form4` \ | `av_insider` \ | `av_earnings` \ | etc. |
| `payload` | jsonb | Raw evidence data |
| `payload_hash` | text | UNIQUE; SHA-256 of payload |
| `created_at` | timestampz | Append-only. |
```

fundamental_facts

Point-in-time (PIT) fundamentals vintage ledger (migration 150). Append-only: one immutable row per (symbol, market, report_period, restatement vintage). A later restatement of the same report_period inserts a NEW row with restatement_seq = prev+1 and flips the prior row's is_latest=false - nothing is mutated in place, so "fundamentals as known on date D" is reconstructable and a restatement can never retroactively change a past as-of read. OFF by default: written by the capture-on-fetch hook in lib/research-agent.ts (fail-open), read via lib/data/pit-fundamentals.ts::getFundamentalsAsOf. Not yet wired into live scoring - scoreFundamentals is unchanged. RLS: ff_service_all (service_role ALL) + ff_owner_read (authenticated, owner email); anon REVOKEd.

The active provider cache is av_cache, shared through providerCachedFetch; it is not a second fundamental truth table. ResearchAgent derives a request freshness hint from earnings_calendar in one batch read: 1 day for report dates in the prior 3/next 14 calendar days and 7 days otherwise or when unknown. The fetched payload is still captured into fundamental_facts with source provenance. No schema migration was required for event-aware freshness or asset_class='adr' because both are application-level contracts over existing text/cache columns.

```
| Column | Type | Notes | | | |
| `id` | uuid PK | |
| `symbol` | text | |
| `market` | text | `us` \ | `india` (CHECK) |
| `metric_set` | text | Default `ttm_overview` |
| `report_period` | date | Fiscal period end the values describe (null for TTM rollups) |
| `fiscal_period` | text | Q1/Q2/Q3/Q4/FY (nullable) |
| `filing_date` | date | When it became public (nullable -> falls back to `captured_at`) |
| `values` | jsonb | OVERVIEW-shaped field map (`PERatio`, `ProfitMargin`, ...) |
| `source` | text | `fmp` \ | `alpha_vantage` \ | `yahoo` \ | `financialdatasets` |
| `restatement_seq` | int | 0 = as-first-observed; 1,2,... = later restatements |
| `is_latest` | bool | Default true; flipped false when a newer vintage of the same period lands |
| `payload_hash` | text | UNIQUE; dedup key for identical re-fetches |
| `captured_at` | timestampz | Kairos vintage clock |
```

corporate_actions

Stock splits + dividends from Alpha Vantage.

```
| Column | Type | Notes | |
| `id` | uuid PK | |
| `symbol` | text | |
| `action_type` | text | `split` \ | `dividend` |
| `ex_date` | date | |
| `detail` | jsonb | |
| `created_at` | timestampz | |
```

trade_decisions

Historical Robinhood trade CSV imports + enrichment.

Column	Type	Notes
`id`	uuid PK	
`symbol`	text	
`action`	text	`buy` \ `sell`
`qty`	numeric	
`exec_price`	numeric	
`exec_date`	date	
`price_1d_after`	numeric	AV DAILY price lookup
`price_1w_after`	numeric	
`price_1m_after`	numeric	
`price_3m_after`	numeric	
`outcome_score`	numeric	$(price_{1m_after} - exec_price) / exec_price * 100$
`macro_market_regime`	text	Hardcoded epoch table
`enrichment_status`	text	`pending` \ `enriched` \ `no_data`
UNIQUE		$(symbol, action, exec_date, exec_price, qty)$ Cross-source dedup

uploaded_trade_files

Tracks CSV upload dedup (SHA-256 hash of file content).

Column	Type	Notes
`id`	uuid PK	
`filename`	text	
`file_hash`	text	UNIQUE
`trade_count`	int	
`duplicate_count`	int	
`date_range_start`	date	
`date_range_end`	date	
`broker`	text	
`created_at`	timestampz	

8.8 Observability & ops

agent_runs

One row per agent invocation.

Column	Type	Notes
`id`	uuid PK	
`agent_name`	text	
`market`	text	
`status`	text	`running` \ `completed` \ `error`
`started_at`	timestampz	
`ended_at`	timestampz	
`summary`	text	
`error`	text	
`created_at`	timestampz	Pruned >60d by DB cleanup

agent_alerts

Open-issues funnel (System Health).

Column	Type	Notes
`id`	uuid PK	
`issue_key`	text	Stable dedup key, e.g. `model-deprecated:deepseek-reasoner`
`severity`	text	`info` \ `warn` \ `error` \ `critical`
`category`	text	`model` \ `broker` \ `budget` \ `data` \ `ops`
`title`	text	
`detail`	text	
`structured_issues`	jsonb	Machine-readable: `{issue_key, root_cause, blast_radius, suggested_fix}`
`resolved`	bool	
`resolved_at`	timestampz	
`auto_expire_at`	timestampz	Self-expiring for budget alerts


```
| `created_at` | timestampz | |
| UNIQUE | `(issue_key) WHERE resolved = false` | At most one open row per issue |
```

llm_call_log

Every LLM call: model, tokens, cost.

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `agent_name` | text | |
| `model` | text | |
| `input_tokens` | int | |
| `output_tokens` | int | |
| `cost_usd` | numeric | |
| `created_at` | timestampz | Pruned >90d by DB cleanup |
```

rag_traces

Every vector retrieval - what memory influenced a decision.

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `symbol` | text | Symbol being scored |
| `query_text` | text | |
| `retrieved_ids` | jsonb | Array of `trade_memories.id` |
| `reranked_ids` | jsonb | Post-rerank IDs |
| `summary` | text | "3/5 prior setups were wins" note passed to LLM |
| `created_at` | timestampz | Pruned >90d by DB cleanup |
```

briefings

Daily briefing records (in-app + email).

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `type` | text | `morning` \| `evening` |
| `content` | text | Full HTML/markdown content |
| `sent_at` | timestampz | |
| `created_at` | timestampz | Pruned >90d by DB cleanup |
```

newsletters

Full email send history (inserted on successful Resend call).

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `subject` | text | |
| `html_body` | text | |
| `resend_message_id` | text | |
| `nav_snapshot` | numeric | |
| `signals_count` | int | |
| `positions_count` | int | |
| `sent_at` | timestampz | Pruned >90d by DB cleanup |
```

8.9 Mentor + coaching

mentor_insights

MentorAgent's coaching notes.

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `insight_type` | text | `pattern` \| `lesson` \| `warning` |
| `content` | text | Plain-English coaching |
| `market` | text | |
```

```
| `symbols_mentioned` | text[] | |
| `created_at` | timestampz | |
```

8.10 India-specific

kite_portfolio_cache

NSE/BSE holdings snapshot from Kite API.

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `holdings` | jsonb | Raw Kite portfolio response |
| `captured_at` | timestampz | |
```

8.11 Historical replay harness (measure-only, OFF)

Local bulk experiment lineage (2026-07-29)

`backtest_experiments` also accepts `historical_replay` plans for the local bulk-evidence worker. A plan binds formula, horizon, validation mode, trial family/count, local PIT universe policy, data cutoff, exact git SHA, schema-versioned spec, unique plan fingerprint, and predeclared variants. Completion reuses the existing write-once result and universe/dataset/run fingerprints. RLS and grants remain service-role-only. Raw historical rows remain outside Supabase and are never returned to the browser.

`backtest_experiment_quality_reviews` is a one-row-per-experiment, append-only operator review. It records `accepted_diagnostic` or `invalidated`, a bounded reason/detail, and an optional replacement experiment. It never edits the immutable result. RLS is enabled; browser roles have no grants; service role can select/insert but cannot update/delete.

Four additive tables (migration 149) backing `features/historical-replay-harness`. Internal, server-side/offline analyst tooling - RLS enabled with no policy (`service_role` bypasses RLS; all other roles denied). The P0-P4 harness code (`lib/replay/*`) runs on in-memory fixtures and does not depend on these tables; they persist frozen point-in-time eligibility runs when the harness is wired to persist. `replay_packets/replay_packet_items` are write-once by convention (assembler deep-freezes in memory).

replay_packets

One row per (cohort, symbol, as-of date); immutable after write. `manifest_hash` = sha256 over the frozen item set. `UNIQUE (cohort, symbol, as_of)`.

replay_packet_items

The frozen inputs for a packet. `item_type` ? {ohlc, fundamental, news, universe, macro, corporate_action} (macro/action extension: 20260729160000). Macro vintages use the source `realtime_start` as `knowable_at`; corporate actions use the announcement timestamp or a conservative ex-date fallback. Invariant `knowable_at <= packet.as_of` (sealed accessor enforces at read; a backfill test asserts at write). FK -> `replay_packets` ON DELETE CASCADE.

replay_eligibility_runs

One row per replay execution. `packet_manifest_hash` + `code_git_sha` make a run reproducible from its frozen inputs and the gate code version. `model_kind` e.g. `pwin_logistic`.

replay_eligibility_events

Per (run, scope, as-of) gate verdict. gate ? {calibration_oos, thin_evidence, ic, validation, breakdown_veto}; passed boolean. The reporter's first_eligible_asof is MIN(as_of) WHERE passed over this table. FK -> replay_eligibility_runs ON DELETE CASCADE.

universe_snapshot_scores (columns added - migration 151)

Cross-sectional rank provenance added to the migration-137 table: rank_quality (ok | degraded | excluded_*), comparable_group_key (market:asset-type:sector; ETFs never grouped with single names), group_n (eligible names in the final group that day), rank_eligible (passed ?4.1 data-quality gates). All nullable/additive.

8.12 Benchmark Alpha + Capital Rotation Shadow

Migration 20260713143000_benchmark_alpha_rotation_shadow.sql adds the Phase-1 benchmark-alpha measurement layer and the P0 capital-rotation shadow ledger. Both are deterministic. Benchmark-alpha writes analytics only; capital rotation records shadow opportunities only.

benchmarks

Config rows for market-local benchmark definitions. One enabled primary benchmark per market is enforced by partial unique index. Seeded primaries: US V00 (USD) and India ^NSEI/NIFTY 50 (INR). Owner can read through RLS; service routes write.

benchmark_price_observations

Durable benchmark component price observations keyed by (benchmark_id, component_symbol, date). The current scorecard route fills primary benchmark observations from existing paper_performance.bench_nav / live_performance.bench_nav ledgers. Missing/unpriceable data is surfaced in scorecard rows rather than skipped.

benchmark_scorecard

Materialized multi-horizon rollup keyed by (market, currency, book, book_scope, benchmark_id, horizon, as_of). Stores common-window portfolio return, benchmark return, excess return, daily tracking error, annualized daily information ratio, sample counts, coverage, confidence, and status. It never feeds orders or learner mutation in Phase 1.

rotation_config

Per-market/per-book rotation flags and thresholds. rotation_shadow_enabled=true by default for measurement. rotation_paper_execute_enabled=false and rotation_live_proposals_enabled=false; no sell/buy/proposal execution is enabled by this migration.

rotation_events

Append-only capital-rotation audit ledger. P0 inserts planned or rejected shadow events from PaperTrader's insufficient_cash branch. Trigger rotation_events_block_mutation blocks UPDATE/DELETE. Rows include candidate/source symbols, scores, edge, notional, gate results, and explicit no_execution audit data.

live_performance provenance columns

Adds nullable/backfilled market, currency, broker, and book_scope so live scorecard rollups can aggregate only explicitly scoped same-market/same-currency rows.

8.13 Daily Per-Holding Risk Analytics (advisory, append-only)

Three additive tables (migration 154) backing features/holding-risk-daily. Owner-only SELECT (email RLS (auth.jwt() ->> 'email') = 'vterminater@gmail.com'), service_role writes, anon REVOKEd. Advisory-only: the risk score, posture, and LLM strategy note reach no order path for any account, including read-only 965848641. No cross-currency roll-up - every row carries its own market + currency; USD and INR are never summed. Populated daily post-close by /api/agents/holding-risk (cron migration 156). Publish/claim via the migration-155 RPCs.

holding_risk_runs

Claim/lifecycle header - one row per (market x account_id x captured_on x formula_version x input_hash) computation, identified by run_key (UNIQUE holding_risk_runs_run_key_uniq). Concurrent/retried crons race on that unique insert; the loser reads back the existing run. status ? {running, complete, failed, partial}; a failed run stays as evidence. Trigger holding_risk_runs_lifecycle_guard() blocks DELETE always, freezes identity/evidence columns, and lets status move forward once out of running only (never terminal->terminal). Partial index holding_risk_runs_latest_idx (market, account_id, currency, formula_version, captured_on DESC) WHERE status='complete' serves latest-complete lookups. Key cols: broker, account_label, source_captured_at, completed_at, data_confidence (0-1), missing_inputs text[], error.

holding_risk_snapshots

Append-only risk-data ledger - one row per run x holding (UNIQUE (run_id, symbol); FK -> holding_risk_runs). Trigger holding_risk_append_only() blocks both UPDATE and DELETE; a rerun is a new run_id, never a rewrite. Deterministic columns: holding_risk_score int CHECK 0-100, risk_posture ? {hold, review, trim, exit_review, insufficient_data}, risk_drivers jsonb (per-component detail), action_reason, add_capacity boolean (risk room exists - NEVER an order signal), weight_pct ? [0,1], beta, realized_vol_pct, unrealized_pnl_pct, data_confidence, missing_inputs, formula_version. strategy_note text is the LLM prose - nullable, best-effort, never blocks, and cannot change the deterministic score/posture/action. Indexes: holding_risk_snapshots_run_idx (run_id), holding_risk_snapshots_symbol_idx (account_id, symbol, captured_on DESC).

account_risk_snapshots

Append-only per-account roll-up - one row per run (UNIQUE (run_id); FK -> holding_risk_runs). Same holding_risk_append_only() UPDATE/DELETE block. metrics jsonb persists the RiskMetrics roll-up for ?-vs-prior trend; total_value (own-currency, never cross-summed), data_confidence, missing_inputs, formula_version. Index account_risk_snapshots_latest_idx (market, account_id, currency, formula_version, captured_on DESC).

Migration history summary

Return-observation evidence (2026-07-16)

```
| Table | Purpose | Mutation / access rule |
| `symbol_return_observations` | Per-symbol window summary: volatility, measured market beta, overlap,
provenance | Append-only; owner-read RLS; service-role write |
| `symbol_daily_returns` | Frozen per-session close pair and simple return for future point-in-time pair
correlation | Append-only; owner-read RLS; service-role write |
```

`symbol_daily_returns` is strictly market-local (us or india) and records the price basis as `adjusted_close` or `raw_close`. Corporate-action/provider revisions append with a new fingerprint; they never overwrite earlier evidence. No scoring, sizing, eligibility, order, or exit path reads either table in the measurement phase.

```
| Migration range | Key tables / changes |
| 001-020 | Core: profiles, auth, strategy_config, agent_signals, paper_portfolio, paper_positions,
paper_trades, watchlist |
| 021-030 | paper_positions exit columns, strategy_config risk profile, macro_regime + macro_signals |
| 031-040 | agent_config, learner_config + learning_priors, paper_order_events (trigger), evidence_records,
strategy_versions + experiment_runs, trade_proposals + decision_journal |
| 041-050 | uploaded_trade_files + trade_decisions, live_account_snapshots, agent_runs |
| 051-060 | signal_score_history (054), india_screen_cache (058), multi-market market column on
paper_portfolio/positions/trades/performance |
| 061-099 | broker_accounts, api_key_vault, llm_call_log, rag_traces, mentor_insights, edge_signals,
edge_ic_history, learning_priors_history |
| 099 | agent_alerts.issue_key + partial unique index |
| 126-128 | PaperTrader standalone cron: signal freshness gate, claim_run_id, expected_price +
realized_slip_pct + fill_status on positions + trades |
| 133-135 | investment_mandates + strategy_evaluations (append-only trigger), mandate_id column on
agent_signals + paper_trades + decision_observations |
| 136 | agent_signals: `score_source` + `scoring_version`; decision_observations: 10 summary cols + NOT
VALID range guards; strategy_versions: 3 new lifecycle states (`measure_only`, `live_review_eligible`,
`live_approved`) |
| 137 | universe_snapshots + universe_snapshot_scores tables (cross-sectional rank, measure-only) |
| 138 | shadow_decisions: `policy_version_id` drops NOT NULL; `setup_type text` col + index (archetype
shadow rows) |
| 139 | strategy_config: +8 `live_auto_*` cols; trade_proposals: +4 autonomous cols + status constraint
expanded (`queued_auto`, `manual_review_required`); broker_order_events: new append-only table +
`boe_block_mutation()` trigger blocking UPDATE/DELETE |
| 140 | `reserve_live_order_budget_v2` RPC - adds `p_execution_actor` param; `approved_by_user=(
actor='owner')`; counts `unknown_needs_reconcile`+`partially_filled` in daily budget; REVOKE public/anon/
authenticated, GRANT service_role only |
| 141 | strategy_config: `live_auto_mode_us`/`live_auto_mode_india` (off/manual/autonomous);
trade_proposals: `market` col + index |
| 142 | RLS: owner-scope the `USING(true)` authenticated SELECT policies on trade_proposals/strategy_config/
paper_trades/decision_journal/deep_analyses/mentor_insights/trade_decisions/uploaded_trade_files |
| 143 | **Restore** (idempotent) the out-of-band 139/140 objects so a clean DB rebuilds: trade_proposals
autonomous cols, broker_orders reservation cols, broker_order_events table+trigger+RLS,
`reserve_live_order_budget_v2` RPC - exact prod DDL |
| 144 | RLS: owner-scope corporate_actions/evidence_records/experiment_runs/paper_order_events/
strategy_versions/trade_decision_embeddings; enable service-only RLS on universe_snapshots/scores |
| 145 | Unique partial index `trade_proposals(signal_id,market) WHERE execution_mode='autonomous_live'` -
atomic autonomous signal claim (no double-propose/buy) |
| 146 | `symbol_blocklist` table (owner-curated tradable-universe blocklist; leveraged/inverse ETFs auto-
blocked in code) + RLS (service + owner-read) |
| 147 | Phase-1 repair (idempotent): `strategy_config` live_auto_* cols restore + REVOKE PUBLIC from
`reserve_live_order_budget_v2` (no-op on prod; clean-rebuild reproducibility) |
| 148 | Live kill-switches + sell atomicity: partial unique index `trade_proposals_active_sell_uniq (symbol,
market) WHERE sell + active` |
| 149 | Historical replay harness: 4 additive tables `replay_packets`, `replay_packet_items`,
`replay_eligibility_runs`, `replay_eligibility_events`; RLS enabled, no policy (service-only). Measure-only,
OFF |
| 150 | PIT fundamentals: `fundamental_facts` append-only vintage ledger; UNIQUE `payload_hash`; RLS
`ff_service_all` + `ff_owner_read`, anon REVOKEd. Index note: `captured_at::date` cannot go in an index
expr (not IMMUTABLE) - indexed as plain `(symbol,market,filing_date DESC,captured_at DESC)`, COALESCE
applied in-query. OFF (capture-on-fetch, fail-open) |
| `20260729160000` | Adds `macro` and `corporate_action` to the sealed `replay_packet_items.item_type`
vocabulary. Measure-only; no live consumer. |
| 151 | Cross-sectional rank: `universe_snapshot_scores` + `rank_quality`/`comparable_group_key`/`group_n`/
`rank_eligible`; `agent_signals` + `rank_pct`/`rank_rejected` (+ `rank_pct ? [0,1]` NOT VALID->validated
check); status `rank_rejected`. Additive, OFF by default (genome `entry.rank_pct_min` default 0.0) |
| 154 | **Daily Per-Holding Risk**: 3 additive append-only tables `holding_risk_runs` (lifecycle guard:
```

DELETE blocked, identity frozen, status forward-once out of `running`), `holding\_risk\_snapshots` + `account\_risk\_snapshots` (UPDATE+DELETE blocked). Owner-email SELECT RLS + service-role writes, anon REVOKed. Advisory-only, no order path; no cross-currency roll-up |

| 155 | Daily Per-Holding Risk RPCs: claim-run (unique `run\_key` insert, loser reads back) + publish-run (running->terminal transition with snapshots) - SECURITY DEFINER, service_role only |

| 165 | `watchlist.market` (text NOT NULL default `US`) - route code read/wrote it for months but the column never existed; GET 500'd (panel "0 tracked") + every manual add failed. Backfills India from `.NS/.BO` |

| 166 | LLM config data fix: rewrite invalid `deepseek-v4-flash/pro` -> `deepseek-chat`/`deepseek-reasoner` in `agent\_config`; seed `research`/`trader`/`mentor-evaluate`/`mentor-thesis`/`mentor-ask` rows (per-flow model from Settings) |

| 20260720114315 | DeepSeek V4 cutover: supersedes migration 166 after DeepSeek made V4 concrete and scheduled the legacy aliases for 2026-07-24 retirement; rewrites `agent\_config` and `api\_key\_vault.model\_id` to V4 Flash/Pro. |

| 167 | `strategy\_config` + `trading\_style` (default `position`) + `target\_hold\_days` - Trading Style presets (Swing/Position/Long-term); horizon governs the time-stop only before a champion is promoted |

| 169 | `live\_performance` (`account\_id`, `date`, `equity`, `bench\_nav`, PK(`account\_id`, date)) - real daily equity curve per live broker account for the Live-vs-V00 chart; accrued forward on each snapshot refresh (RH exposes no account-value history). Initial migration used broad authenticated SELECT; `20260713112754\_tighten\_live\_performance\_rls` tightens it to owner-email SELECT, service-role writes |

| 170 | **Automated strategy validation** `strategy\_validation\_automation` (per-market `enabled`/`auto\_shadow\_enabled`/`max\_active\_shadows` 0-1, owner-read RLS, seeded us+india enabled) + `activate\_strategy\_shadow(bigint)` SECURITY DEFINER RPC (service_role only) that atomically routes a PASSED challenger to `shadow\_paper` under a per-market advisory lock + capacity cap. Also schedules pg_cron `kairos-validation-sweep` (Fri 21:45 UTC). Cannot promote/execute - shadow only |

| 171 | `market\_controls` (`market` pk us/india, `paused`, `trading\_enabled`, `paused\_reason`, `paused\_at`) - per-market pause/kill so one market's breaker no longer halts the other. Global `strategy\_config.app\_paused`/`trading\_enabled` retained as a master-kill. Helpers in `lib/market-controls.ts`; owner-read RLS, service-role writes; seeded from current global flags |

| 172 | `research\_queue` (pk `market`, `symbol`, `priority`, `attempts`, `discovery\_source`, `deferred\_at`) - research candidate carry-forward: candidates beyond a run's cap are deferred here with raised priority (starvation-free) instead of the old silent `.slice()` drop, so a growing watchlist/screener pool rotates fairly under provider budgets. Helper `lib/research-queue.ts`; owner-read RLS, service-role writes |

| 173 | `oauth\_pkce\_state` - server-side one-time PKCE verifier store keyed by `state` + `provider` for generic MCP broker OAuth callbacks. Callback consumes by provider-scoped delete-and-return, so replay/race callbacks cannot reuse a verifier |

| 174 | Live snapshot broker framework - `live\_account\_snapshots.broker` labels rows by source broker and the registry MCP refresh path auto-adds connected accounts. Pruning is broker-scoped and only runs after a successful non-empty capture, never after an outage or empty result |

| 175 | `strategy\_sleeves` + `strategy\_config.allocation\_enabled` - deterministic asset-allocation proposal core. Shipped OFF by default; callers return null unless `allocation\_enabled=true`. Sanitizes malformed bands/targets and never routes to orders |

| 177 | `execute\_paper\_rotation(...)` RPC (service_role) - capital-rotation Phase 1 PAPER: atomic sell-weakest-holding + buy-candidate in one transaction (rolls back if the candidate can't be funded after the sale - never leaves the book in cash). Idempotent on `idempotency\_key`; writes status `paper\_executed` to `rotation\_events`. SHIPPED OFF: only invoked when `rotation\_config.rotation\_paper\_execute\_enabled=true` (default false) + guardrails (persistence/cooldown/per-run+day caps) pass. Paper book only |

| 176 | `provider\_pacing` (`provider` pk, `min\_interval\_ms`, `last\_started\_at`) + `try\_acquire\_provider\_slot(provider, min\_interval\_ms)` RPC (service_role) - serverless-safe per-provider rate-limit lease (atomic INSERT...ON CONFLICT DO UPDATE...WHERE). `providerCachedFetch` acquires a slot before a real call for HARD-limited providers (Massive 5/min, GDELT 1/5s); no slot -> serve stale cache instead of bursting past the wall. Data-fetch pacing only - no order path |

| 20260713112754 | RLS tightening for `live\_performance`: drops broad authenticated read policy and replaces it with owner-email SELECT (`select auth.jwt() -> 'email' = 'vterminater@gmail.com'`) |

| 20260713143000 | Benchmark-alpha scorecard tables (`benchmarks`, `benchmark\_price\_observations`, `benchmark\_scorecard`), `live\_performance` provenance columns, capital-rotation shadow config/events (`rotation\_config`, append-only `rotation\_events`), and pg_cron `kairos-benchmark-scorecard` |

| 20260714000000 | pg_cron `kairos-broker-keepwarm` (daily 06:00+18:00 UTC) -> `POST /api/broker-mcp/keepwarm` refreshes/rotates every connected MCP broker token so the OAuth refresh chain never lapses over weekends. Read-only, no order path |

| 20260714010000 | **Canonical Evidence Router - policy foundation** (`router\_enabled=false`, shadow-only): immutable `evidence\_policy\_versions` + mutable `active\_evidence\_policy` pointer + immutable `evidence\_policy\_rules` (per-intent auto/prefer/only/off) + `provider\_runtime\_config` (conservative-only overrides) + `provider\_capability\_status` (per provider/market/intent maturity) + append-only `evidence\_policy\_evaluations` (shadow proof). `activate\_evidence\_policy(market, version, required\_intents, actor)` SECURITY DEFINER RPC (advisory lock + required-intent check, service_role only). Append-only triggers block UPDATE/DELETE on versions/rules/evaluations. Seeded `us:v1`+`india:v1` all-Auto, disabled. Owner-read RLS, service-role writes. No scoring/order/money path |

| 20260714020000 | **Canonical Evidence Router - cache foundation** `evidence\_cache\_v2` (canonical per market/symbol/intent/provider/fingerprint; `\_\_MARKET\_\_` symbol for market-wide) + append-only `provider\_call\_ledger` (normalized status/error only, **no** tokens/URLs/headers/bodies) + durable `provider\_refresh\_jobs` queue (one active job per identity via partial unique index) + `claim\_provider\_refresh\_jobs(limit, lease)` SECURITY DEFINER RPC (`FOR UPDATE SKIP LOCKED`, service_role only). Owner-read RLS. Separate from legacy `av\_cache` |

| 20260714030000 | **Pacing repair** - idempotent reconciliation of the out-of-band live


```

`provider_pacing` + `try_acquire_provider_slot` (previously untracked "migration 176") so a fresh env
rebuilds identically. Matches live semantics exactly; no behavior change |
| 20260715130000 | **Evidence Router ACL + provenance repair**: explicitly removes `PUBLIC`/`anon`/
`authenticated` execution from policy, refresh-claim, and pacing RPCs; makes `provider_pacing` service-role-
only; adds the canonical `evidence_cache_v2.provenance` array used to preserve adapter field/source
attribution through cache hits |
| 20260715131000 | **Evidence Router table ACL hardening**: removes all `anon` grants and all authenticated
write grants from policy, runtime, capability, evaluation, cache, call-ledger, and refresh-queue tables;
authenticated remains owner-read through RLS, while `service_role` retains server-side access |
| 20260715160000 | **Downside hedge (US PAPER only, OFF)**: config, state, append-only ledger, paper
position/trade role provenance, transactional evaluation/fill/exit RPCs, owner-read RLS and service-only
writes. No live order path. |
| 20260716210000 | **Router cutover prerequisites** (shadow-only, `router_enabled` still false):
`evidence_field_baselines` (mutable - a moving reference point, not evidence), append-only
`evidence_degradation_events`, `evidence_evaluation_details`, `evidence_evaluation_reviews`, plus frozen-
cohort + activation-binding columns on `evidence_policy_evaluations`, and the
`activate_evidence_policy_bound()` RPC. RLS on with owner-read on all four; anon has no grants; writes
service-role only; RPC is `security_definer` with fixed `search_path`, executable by `service_role` only. *(
Row was missing from this table; the migration is confirmed APPLIED in production - verified via
`information_schema.columns` on 2026-07-18.)* |
| 20260719090000 | **Weekend research catch-up**: adds `agent_signals.session_validated/as_of_session/
staged_at`, the staged-row invariant + one-active-stage partial unique index, structured `agent_runs.
workload_metrics`, and per-market Saturday/Sunday catch-up crons. Weekend rows are non-executable until a
fresh session re-score. |
| 20260721130000 | **Router evidence hardening** (still shadow-only): `evidence_policy_evaluations` gains
immutable `safety_pass`, `quality_pass`, and `market_session_date`; one inactive `router_enabled=true`
candidate per market copies the current active rule set so evidence binds the exact future candidate
without moving the active pointer; the bound activation RPC requires both verdicts plus ten distinct
passing market sessions in the prior 45 days and a still-fresh selected evaluation. Adds daily cache-only
cohort crons. Production stays on disabled baselines. |
| 20260721164500 | Router cohort session-source clarification: `market_session_date` is sourced from
executable ResearchAgent rows (`session_validated=true`, `as_of_session`), not candidate candle
availability. Missing India bars therefore persist as failed coverage evidence instead of crashing the
evaluator; weekend/holiday staged rows cannot count. |
| 20260731210000 | **Market-local US schedules + India news shadow**: replaces five fixed-UTC US jobs with
paired seasonal UTC schedules carrying exact New York `local_slot` contracts; adds daily post-close `kairos-
india-news-shadow`. No new table: shadow payloads reuse `evidence_cache_v2` intents `sentiment.
news_headlines_shadow` / `event.corporate_announcement_shadow`, and exact call accounting reuses append-
only `provider_call_ledger` run IDs prefixed `india-news-shadow:`. No decision consumer. |

```

Capital rotation containment (20260722185000)

Migration 20260722185000 contains capital-rotation P1 and hardens its future claim contract. It forces rotation_paper_execute_enabled=false with a database check constraint because the cost/tax/turnover/fresh-price/post-swap gates are incomplete, while preserving shadow measurement. The replacement RPC verifies the exact claim_run_id and then returns p1_guardrails_incomplete; it contains no position, cash, trade, or order mutation. The caller also requires an independent deployment flag.

Migration 20260722203000 adds the read-only, service-role-only get_rotation_return_cohort(market, symbols, since) RPC. It returns one latest point-in-time revision per symbol/session from symbol_daily_returns, bounded to one market and at most 20 symbols. This prevents PostgREST row limits from silently truncating P0 candidate-correlation evidence. It has no write or trading authority.

Evidence evaluation tables

evidence_policy_evaluations and evidence_evaluation_details sat at 0 rows from the 20260716210000 migration until 2026-07-18: the comparator, degradation guard, and bound activation RPC all shipped, but nothing fed them. Their first and only writer is the cohort builder (lib/evidence/evaluation/cohort-builder.ts, exposed as GET/POST /api/agents/evidence-cohort?market=us|india), which resolves real recent research decisions into a frozen dual-run cohort and persists one evaluation row plus one detail row per cohort symbol - including rows where either path abstained or failed, which the spec requires to be retained rather than filtered out.

Migration 20260721130000 separates safety from quality and records the trading session represented by frozen daily bars. Rows remain append-only, owner-read, service-role-write, and carry an expires_at (default 72h) so the selected proof must be fresh. Historical passing rows can contribute only to the ten-session window; old v1 rows have neither verdict nor a

session date and cannot qualify.

These rows are measurement, not authority. `router_enabled` remains `false` for both markets, so an evaluation changes no score, signal, size, position, or order. Persisting one cannot activate anything - activation is the separate owner-gated `activate_evidence_policy_bound()` RPC, which additionally requires the evaluation's baseline to still be the active policy and every flagged divergence to carry an approving review row.

Earnings Risk Observations (P0)

`earnings_risk_observations` is a compact normalized decision-context ledger, not a new source-of-truth store. Raw provider payloads stay in the existing evidence/cache layers. Rows reference the signal/proposal when available and carry market, event/session, normalized option quote, proxy/stop ratio, policy, counterfactual verdict, and legacy-blackout parity.

The table is append-only and retry-idempotent. Owner-authenticated clients have `SELECT`; `service_role` has only `SELECT` and `INSERT`; `anon` has no grant. Database checks pin P0 to `policy_mode='shadow'` and `behavior_changed=false`. Migrations: `20260729200000_earnings_risk_observations.sql` and `20260729201000_harden_earnings_risk_observation_acl.sql`, with `20260729202000_optimize_earnings_risk_observations.sql` adding the proposal lookup index and init-plan-safe owner policy.

05-crons-and-scheduling

Source: docs\arch\05-crons-and-scheduling.md | Authoritative operational architecture

Kairos - Crons & Scheduling

Last updated: 2026-07-31 (US research, paper-entry, and close-monitor slots are DST-safe market-local contracts: paired EDT/EST UTC invocations plus an exact route guard admit only one. Added daily post-close kairos-india-news-shadow; it is evidence-only and uses no scoring API key.)

Previously: 2026-07-21 (PaperTrader is standalone-only: one in-session attempt per market, with US at 15:15 UTC and India at 04:10 UTC. Research no longer tail-calls it. The route independently refuses weekends, holidays, and outside-session execution.)

Previously: 2026-07-19 (Per-market ResearchAgent catch-up now runs on supported market-closed days: weekends plus verified full NYSE/NSE equity holidays. Daily triggers self-skip on trading days, special sessions, and unsupported calendar years. Results remain weekend_staged under the legacy status name, session_validated=false, and never chain a trader.)

Previously: 2026-07-17 (Documented the two macro-read crons, which were missing from this table entirely. macro-read-india is now a no-op - the route refuses market=india - and is flagged for removal.)

Previously: 2026-07-17 (kairos-price-cache-fill now also backfills ~400d of sector-XL daily history - one paced, resumable provider call per symbol on the existing schedule. No new cron, no schema change.)

Previously: 2026-07-15 (Codex audit: added the missing daily kairos-earnings-pit-capture at 02:10 UTC; moved kairos-india-markets-fill-retry from a colliding 10:45 slot to 10:35 UTC. India primary remains 10:15; symbol-profile backfill remains 11:40.)

Update this file when: a new cron is added or removed, a schedule changes, or a new endpoint is wired to a cron.

Adding a cron:

- Cloud: add to vercel.json (hit deployed URL)
- Local: add to scripts/run-agents.ps1 (Windows Task Scheduler; PC must be on)
- Update this file with the new entry

Vercel crons (cloud, hit deployed URL)

Defined in vercel.json. Fire against the Vercel deployment URL regardless of local machine state.

```
| Endpoint | Schedule (UTC) | What it does |
| `api/agents/evaluation/p1-gate/cron` | Sundays 02:00 UTC | Count closed evaluable trades per market; fire System Health info alert when >= 20 |
| `api/agents/autonomous-live/cron?market=us` | Weekdays 15:00 UTC (~10-11 AM ET, in US session) | Per-market run: 9-gate kernel + fresh kill-switch + session-window guard + per-market USD NAV + Kelly; submit live US orders via Robinhood REST; no-op when `AUTONOMOUS_LIVE_ENABLED=false`, market not autonomous, or session closed |
| `api/agents/autonomous-live/cron?market=india` | Weekdays 06:00 UTC (11:30 AM IST, in NSE session) | Same, India: INR NAV from Kite margins+holdings; Kite REST |
| `api/agents/live-exit-monitor/cron` | Every 30 min, 13:00-20:00 UTC weekdays (US session) | Protective exits for LIVE positions: reconstructs open live positions from filled broker_orders; SELLS via the gateway on stop (-8%) / target (+20%) / time (15d). No-op unless live-auto armed |
| `api/agents/db-cleanup` | 1st of month 03:00 UTC | Prune 15 safe tables (llm_call_log >90d, agent_runs >60d, etc.); never touches ledgers |
```

Windows Task Scheduler (local machine, ET)

All triggered by scripts/run-agents.ps1 -Agent <name>. PC must be on for these to fire.

```

| Task name | Schedule (ET) | Endpoint | Notes |
| `brief-morning` | Weekdays 8:00 AM | `/api/briefing/generate` | Morning email before market open |
| `research` | Weekdays 9:00 AM | `/api/agents/research/cron?market=us` | US signal generation (3 candidates/day) |
| `paper-trade-us` | Legacy local task; disable when pg_cron is active | `/api/agents/paper-trade?market=us` | Route is session-gated; production authority is pg_cron |
| `trader` | Weekdays 9:45 AM | `/api/agents/trader` | TraderAgent proposals; `approval_required=true` |
| `scan-india-refresh` | Weekdays 5:30 AM | `/api/scan/india/refresh` | Refresh up to 600 NSE equities oldest-first; scanner reports fresh/stale rotating coverage |
| `research-india` | Weekdays 6:15 AM | `/api/agents/research/cron?market=india` | India signal generation post-NSE-close |
| `paper-trade-india` | Legacy local task; disable when pg_cron is active | `/api/agents/paper-trade?market=india` | Route is session-gated; production authority is pg_cron |
| `position-monitor` | Weekdays 4:15 PM | `/api/agents/position-monitor?market=us` | US stop/target/time-stop/partial-profit checks |
| `position-monitor-india` | Weekdays 6:35 AM | `/api/agents/position-monitor?market=india` | India position exits |
| `brief-evening` | Weekdays 4:30 PM | `/api/briefing/generate` | Evening email recap |
| `nav-snapshot` | Weekdays 5:00 PM | `/api/agents/performance` | Daily NAV + alpha snapshot |
| `learner` | Fridays 5:00 PM | `/api/agents/learner` | Weekly weight learning; route skips non-Fridays |
| `macro-sentinel` | Mondays 8:00 AM | `/api/agents/macro-sentinel` | Weekly macro regime computation |
| `policy-events` | Weekdays 23:00 UTC | `/api/agents/policy-events` | US-only FOMC schedule/outcome sync plus record-only 1D/5D impact capture from frozen return evidence; no expectation source, scoring, or execution effect |
| `macro-read-us` | Weekdays 9:30 AM ET (13:30 UTC) | `/api/agent-mind/macro-read?market=us` | Agent Mind Phase 3: cached daily plain-English "what the macro backdrop means for your book" (US). Advisory/narrative only - never trades or sizes |
| `macro-read-india` | Weekdays 10:00 AM IST (04:30 UTC) | `/api/agent-mind/macro-read?market=india` | **NO-OP - should be dropped.** The route refuses `market=india` before any LLM call or DB write (2026-07-17): both macro inputs (`macro_regime`, `category='macro'` `learning_priors`) are US-only and unmarket-tagged, so there is no honest India read to generate. Left active pending owner removal (dropping the `cron.job` row is a prod DB mutation); the route-level refusal makes it harmless - zero LLM spend, zero rows |
| `theme-scout` | Sundays 8:00 PM ET | `/api/agents/theme-scout` | Weekly, discovery-only watchlist additions; independent of ResearchAgent |
| `stale-check` | Every 4h | `/api/alerts/stale-check` | Alert if agent runs are stale |
| `live-snapshot` | Weekdays (manual / Task Scheduler) | `scripts/sync_robin.py` | Python script - pulls Robinhood positions into `live_account_snapshots` |
| `live-account-refresh` | On demand / Task Scheduler (`robinhood_mcp` source) | `POST /api/live-account/refresh-snapshot` | Deterministic MCP capture of all Robinhood accounts -> upserts `live_account_snapshots` AND accrues one `live_performance` row/account/day (real equity + V00 close) - the forward-built source for the Live-vs-V00 chart (RH has no account-value history to backfill) |

```

Postgres pg_cron (in-database, fire against deployed URL)

Scheduled inside Supabase via `cron.schedule`, calling the deployed app through the `kairos_call_agent(endpoint, body, method, timeout_ms)` helper. Independent of local machine state.

```

| Job | Schedule (UTC) | Calls | What it does |
| `kairos-research` | Weekdays 13:00+14:00 UTC; only 09:00 ET admitted | `POST /api/agents/research/cron?market=us&local_slot=09%3A00` | Paired seasonal invocations keep the research contract at 09:00 New York time. The nonmatching invocation exits before provider/DB work. Daily technical scoring filters out the current session until 16:00 ET. |
| `kairos-paper-trade-us` | Weekdays 15:15+16:15 UTC; only 11:15 ET admitted | `POST /api/agents/paper-trade?market=us&local_slot=11%3A15` | The scheduled US paper-entry attempt stays at 11:15 ET across DST and independently enforces the NYSE session/calendar. |
| `kairos-paper-trade-india` | Weekdays 04:10 UTC (09:40 IST) | `POST /api/agents/paper-trade?market=india` | Morning India paper-entry attempt, after research starts and inside NSE hours. |
| `kairos-research-us-pm` | Weekdays 18:00+19:00 UTC; only 14:00 ET admitted | `POST /api/agents/research/cron?market=us&local_slot=14%3A00` | Afternoon rescore at a stable 14:00 ET. Daily factors still use the last completed session; live quotes remain execution inputs, not partial daily technical bars. |
| `kairos-paper-trade-us-pm` | Weekdays 19:15+20:15 UTC; only 15:15 ET admitted | `POST /api/agents/paper-trade?market=us&local_slot=15%3A15` | Afternoon US paper-entry/rotation attempt at a stable 15:15 ET. |
| `kairos-position-monitor` | Weekdays 20:15+21:15 UTC; only 16:15 ET admitted | `POST /api/agents/position-monitor?market=us&local_slot=16%3A15` | US daily score/stop/target/time checks remain after the regular close in both EDT and EST. The seasonal duplicate exits before monitor work. |
| `kairos-research-india-mid` | Weekdays 07:00 UTC (12:30 IST) | `POST /api/agents/research/cron?market=india` | Midday India research cycle - same intraday-adaptation rationale as the US PM run. |
| `kairos-paper-trade-india-mid` | Weekdays 07:45 UTC (13:15 IST) | `POST /api/agents/paper-trade?market=india` | Midday India paper-entry/rotation attempt inside NSE hours. |
| `kairos-holding-risk-us` | Weekdays 21:30 UTC (17:30 ET) | `POST /api/agents/holding-risk?market=us` | Daily Per-Holding Risk: scores every US live-account holding (deterministic score + posture, LLM prose note

```

only). Fires after the 16:00 ET close **and** after `nav-snapshot` refreshes the account book at 21:00 UTC. 290s timeout. **Advisory-only - touches no order path.** |
`kairos-holding-risk-india`	Weekdays 11:00 UTC (16:30 IST)	`POST /api/agents/holding-risk?market=india`	Same, India (Kite): fires after the 15:30 IST close. 290s timeout. Advisory-only.
`kairos-live-snapshot`	Weekdays 13:00-21:00 UTC every 2h	`POST /api/live-account/refresh-snapshot`	Refreshes the live account book for every CONNECTED cloud MCP broker (Robinhood + Webull via the registry driver `captureAccounts`) -> `live\_account\_snapshots` + `live\_performance` (US, `market='us'`, VOO bench). Cloud-native (OAuth vault token, no local machine). Auto-ADDs newly-returned accounts. Auto-pruning is broker-scoped and fail-safe: it runs only after that broker returns at least one valid account, deletes only rows for that broker not in the successful capture set, and never runs on failed/empty capture, so a broker outage cannot mass-delete another broker or wipe the kill-switch baseline. **India (Kite) accrual (`refreshKite`)** runs in the same call, fully independent + fail-soft: NAV = Kite `margins.equity.net` + `(last\_priceqty)`, bench = `^NSEI` close, written as ONE `live\_performance` row (`market='india'`, `broker='kite'`, `currency='INR'`, `account\_id`=Kite `user\_id`). It writes **only** `live\_performance`, never `live\_account\_snapshots`, so the Kite account can never leak into the US account chips or the US kill-switch NAV baseline; a stale Kite daily token just skips the day. This is the forward-built source for the India LIVE-vs-NIFTY chart (`/api/live-portfolio/performance?market=india`, which falls back to the paper India NIFTY curve until ≥ 2 live Kite days exist).
`kairos-prewarm-us`	**7-day** every 5 min in the 12:00 UTC hour (12 ticks before 13:00 US research; migration 20260714070000)	`POST /api/agents/prewarm?market=us`	**Evidence-cache prewarmer.** Warms `av\_cache` with each US symbol's fundamentals/sentiment/insider evidence AHEAD of scoring so a cold-start run gets cache hits instead of bursting Massive (5/min) / GDELT (1/5s) past their walls mid-run. BOUNDED (45s wall-clock) + RESUMABLE - each tick warms until the budget then returns `{warmed, remaining}`; the pacing lease + av_cache dedupe drain the universe across ticks (warm symbols are instant cache hits, so ticks advance to the cold tail). Warm-only: no scoring, no signal/packet writes, **no order path**.
`kairos-prewarm-india`	**7-day** every 10 min in the **03:00 UTC hour** (before the 04:00 India research)	`POST /api/agents/prewarm?market=india`	Warms active India fundamentals only. The retired GDELT India scoring path is no longer called; replacement news collection has its own post-close shadow.
`kairos-india-news-shadow`	Daily 12:15 UTC (17:45 IST)	`POST /api/agents/india-news-shadow`	Bounded holdings-first NSE corporate announcement + Google News RSS collection into `evidence\_cache\_v2` and `provider\_call\_ledger`. Runs on weekends/holidays because events are not exchange-session-bound. No score, signal, paper/live trade, learner mutation, or broker reader.
`kairos-evidence-shadow-us`	**7-day** 14:20/35/50 UTC (after US research)	`POST /api/agents/evidence-shadow?market=us`	**Canonical Evidence Router dual-run shadow** (migration 20260714060000). **Weekend self-skip** (both prewarm + shadow): on Sat/Sun the route no-ops when the per-market `research\_queue` backlog is < 10 , so idle weekend quota is used only when there's real backlog to drain; weekdays always run. Resolves the router-covered intents (fundamentals/analyst/insider/daily-bars) over the per-market universe = `watchlist` ? `research\_queue` (unioned so India - whose rotation universe lives in `research\_queue`, not the us/US-only `watchlist` - actually accumulates evidence; before this the India branch always resolved an empty universe), logging every attempt to `provider\_call\_ledger` + `evidence\_cache\_v2`. `router\_enabled=false` -> observational only, NEVER scored/traded. Bounded 45s + resumable (fresh cache short-circuits); 3 ticks drain the universe. Decoupled from the research hot path so it can't slow the 50-symbol run. Accumulates the coverage/disagreement evidence the Phase-4 cutover is gated on.
`kairos-evidence-shadow-india`	**7-day** 04:30/40/50 UTC (after India research)	`POST /api/agents/evidence-shadow?market=india`	Same, India.
`kairos-evidence-cohort-us`	Daily 15:05 UTC (after final US shadow tick)	`POST /api/agents/evidence-cohort?market=us&limit=50`	Cache-only cutover evidence. Replays frozen ResearchAgent score inputs, writes immutable safety/quality parity results, deduplicates unchanged cohort fingerprints, and consumes no external-provider quota. Measurement only; cannot activate a policy.
`kairos-evidence-cohort-india`	Daily 05:05 UTC (after final India shadow tick)	`POST /api/agents/evidence-cohort?market=india&limit=50`	Same, India/INR. Only session-validated ResearchAgent rows supply `as\_of\_session`, so weekend/holiday staged runs cannot increase the ten-session cutover count; missing Router bars are recorded as failed coverage rather than crashing the evaluator.
`kairos-closed-day-research-us`	Daily 15:10 UTC; route permits only supported weekends/full NYSE holidays	`POST /api/agents/research/cron?market=us&mode=closed\_day\_catchup`	Scores only US carry-forward queue symbols not already staged for the same completed session. Trading days, special sessions, and unsupported calendar years self-skip. Writes non-executable staged signals; leaves candidates queued for next-session revalidation; never chains a trader.
`kairos-closed-day-research-india`	Daily 05:10 UTC; route permits only supported weekends/full NSE CM holidays	`POST /api/agents/research/cron?market=india&mode=closed\_day\_catchup`	India-equivalent catch-up, independently scoped in INR/NSE symbol space. Uses the NSE Capital Market calendar, not settlement/derivatives calendars; Muhurat special sessions abstain. Same non-executable lifecycle.
`kairos-earnings-pit-capture`	**Daily** 02:10 UTC	`POST /api/calendar/earnings/refresh`	Captures changing US pre-report consensus vintages and the first provider actual observed after release. Conservative PIT rule rejects consensus captured on/after the US report date. Data-capture only; no score/order consumer.
`kairos-earnings-risk-monitor-us`	Weekdays 16:00 UTC (12:00 EDT / 11:00 EST)	`POST /api/agents/earnings-risk-monitor`	Bounded US holdings-only earnings/options shadow. Runs after PaperTrader and outside the 14:00-15:15 provider cluster, so same-day new positions are included without competing with evidence cohorts. Reads open paper positions and the latest complete per-account live-risk snapshots, merges the PIT cache with one Robinhood calendar call, requests per-symbol/options evidence only for an event inside the holding horizon, and appends `behavior\_changed=false` evidence. It never scores, sizes, enters, exits, or calls India options.
`kairos-india-markets-fill` (+ `retry`)	Weekdays 10:15 + 10:35 UTC	`POST /api/markets/india`	Post-close display snapshot for India indices, sectors, and versioned NIFTY-50 breadth. One deduplicated, paced

```

provider stream; GET is cache-only. Retry moved from 10:45 because that slot is used by `kairos-scan-india-
refresh`. |
| `kairos-broker-keepwarm` | **Daily** 06:00 + 18:00 UTC (7 days/week, migration 20260714000000) | `POST /
api/broker-mcp/keepwarm` | **MCP token keep-warm.** Iterates every connected MCP broker and calls
`getValidAccessToken`, which refreshes + CAS-rotates the refresh token when the short-lived access token (
Webull ~1 day) is expiring. Insurance so the OAuth refresh chain never lapses across weekends/holidays when
the weekday-only market crons (`broker-sync`, `research`) don't touch it. Read-only - no tool calls, **no
order path**. A refresh failure raises a critical System Health issue (`broker-token:<broker>`) for
reconnect. |
| `kairos-validation-sweep` | Fridays 21:45 UTC | `POST /api/validation/sweep` | **Automated strategy
validation (migration 170)** recovery sweep: for each market, validates up to 5 never-validated challengers
(`state='challenger'`, `validation_experiment_id` IS NULL) through the deterministic Validation Engine, and
- when the per-market `strategy_validation_automation` policy allows - auto-routes a PASSED challenger
into the single `shadow_paper` slot via the `activate_strategy_shadow` RPC. Catches challengers created
outside LearnerAgent or interrupted before in-process validation. Runs 45 min after the Friday learner.
Self-reports to **System Health** (`reportIssue`/`resolveIssue`, key `cron-failed:kairos-validation-sweep`)
on any execution error, clears on a clean run. Also writes an `agent_runs` heartbeat (
`agent_type='validation_sweep'`, market `us`) so **state-check** flags a SILENT non-run - registered as
a Friday-only job (expected 21:00 UTC, 2h grace). So both failure modes are covered: errored-when-run and
never-fired. **Cannot promote a champion or touch any paper/live execution path - `shadow_paper` is non-
executing.** |
| `kairos-price-cache-fill` (+ `retry`) | Weekdays 13:25 + 13:45 UTC (pre-market, before the 14:00
briefing; migration 20260715140000) | `POST /api/agents/price-cache-fill` | **Markets display price-cache
fill.** Pre-fills `price_cache` with the whole Markets ETF universe (regime proxies SPY/QQQ/IWM/TLT/IEF/HYG/
UUP/GLD + VIXY/DIA, the 11 sector XLs, and the leveraged sentiment pairs TQQQ/SQQQ/... ) via ONE Massive
**grouped-daily** call (all US tickers in a single request, filtered to the universe) - so the Markets
tiles (`markets/synthesis`, `markets/quotes`, `charts/sector-returns`) read a warm cache instead of each
bursting Massive's ~5/min free tier on page load.

```

markets/overview is deliberately NOT a price_cache reader (corrected 2026-07-17).

This chapter previously listed it here and asserted it read the warm cache. It never

did, and it should not - the claim was wrong, not the code. Verified against prod

(dionkikgdm1aotvtbnfr) on 2026-07-17:

1. The cache cannot cover the tile. The overview needs all 15 symbols (SPY/QQQ/DIA/VIXY

+ 11 XLs) on one session. QQQ, DIA and VIXY hold 2 bars each (07-14, 07-15) -

the daily fill only began covering them on 07-14 and the 400d backfill is sector-XL-only.

Only two fully-aligned 15/15 sessions exist in the entire table.

2. The head of the cache is ragged. On 07-17 the newest bar per symbol was 07-16 for

SPY and XLV but 07-15 for the other 13. A "latest bar per symbol" read would therefore

render SPY's 07-16 close beside XLK's 07-15 close as one snapshot - reintroducing the

cross-session mix the grouped rewrite exists to prevent.

3. The cache is a full session STALER. Requiring honest 15/15 alignment resolves to

07-15, while grouped serves 07-16. Cache-first would trade freshness for nothing.

4. The rate-limit argument no longer applies. It was written when the route fired 15

per-symbol /prev calls against a ~5/min ceiling. The route now spends 2-3 grouped

calls, each revalidate: 3600 (past sessions are immutable) behind a 5-min route memo -

~2 calls/hour, ~0.7% of budget. The premise was overtaken by the rewrite.

A cache fallback on provider error was also considered and rejected: price_cache is

filled from the **same provider's same endpoint**, so an outage that breaks the route

correlates with a stalled fill - it would fall back to a staler copy of the same failure,

while adding a DB dependency to a route that has none. The degraded payload + Retry is honest.

The tile's contract: every symbol on ONE session, labelled with sessionDate /

priorCloseDate, unresolved symbols as n/a - never 0.00%, never "today" for a prior

session. Pinned by tests/markets-overview.test.ts. Late, never wrong. The prev-session close is stable all day, so one fill/day is enough; the 13:45 tick is an idempotent no-op once 13:25 has filled (skips when the most-recent session is already cached). Falls back to sequential per-symbol /prev (lease-gated 5/min, bounded 45s, resumable) only if the grouped endpoint is unavailable. Raises a System Health warn (price-cache-fill-degraded, auto-clears at UTC midnight) only on a large shortfall. Display data only - never on the money/scoring path.
Sector history backfill (2026-07-17). The same tick also backfills ~400 calendar days of daily bars for the 11 sector XLs, because charts/sector-returns offers 1W/1M/3M/6M/1Y windows and the daily fill alone only ever accumulates one session per tick - with two cached sessions every window collapsed onto the same two bars and reported a one-day move as a "1Y return". Uses /v2/aggs/ticker/{sym}/range/1/day/{from}/{to}, which returns the FULL series in ONE request, so the whole 11-ETF backfill costs 11 provider calls total (not 11 x 400). Each call takes the shared try_acquire_provider_slot lease (12.5s = 5/min), is wall-clock bounded, and is resumable - a tick drains what fits (~1-3 symbols), skips symbols that already have depth, and later ticks finish the rest; once drained it is a permanent no-op. Runs before the per-symbol fallback deliberately: on a grouped-endpoint failure the fallback would otherwise consume the whole budget and starve the backfill indefinitely. Going first costs the daily fill nothing for these symbols - the range call spans up to the most recent session, so a backfilled sector is daily-filled by the same request. No new cron and no schema change: it rides the existing schedule. |

```
| `kairos-benchmark-scorecard` | Weekdays 22:15 UTC | `POST /api/agents/benchmark-scorecard` | **Benchmark Alpha P1 (migration 20260713143000)** rolls up paper/live scorecard rows for 1W/1M/3M/YTD/1Y after NAV + labels. Writes `benchmark_scorecard` status rows, including missing/unpriceable rows. Measurement-only: no learner mutation, no paper fills, no live orders. |
| `kairos-downside-hedge-us` | Weekdays 21:10 UTC | `POST /api/agents/downside-hedge` | Deterministic US paper hedge evaluation. Default OFF; shadow logging and paper execution have separate flags. No live path. |
| `kairos-edge-scout-us` | Weekdays 22:30 UTC | `POST /api/agents/edge-scout?market=us&maxSymbols=50` | Post-close prospective factor snapshot. Measure-only, cached/budgeted providers, market-scoped heartbeat. |
| `kairos-edge-scout-india` | Weekdays 11:30 UTC | `POST /api/agents/edge-scout?market=india&maxSymbols=50` | India post-close equivalent. Never cross-sums or updates US lifecycle state. |
| `kairos-edge-ic-us` / `india` | Mondays 02:00 / 03:00 UTC | `POST /api/agents/edge-ic?...` | Weekly bounded retrospective IC diagnostic. Explicitly current-universe/survivorship-biased; cannot promote, score, size, or trade. |
| `kairos-edge-readiness` | Daily 03:20 UTC | `POST /api/agents/edge-readiness` | Reads cached IC history only, updates the measure-only readiness projection, emits one-time review milestones, and warns when collection is stale. No provider or trading call. |
| `kairos-international-allocation-shadow` | Mondays 03:30 UTC | `POST /api/allocation/international/assess?mode=p2_weekly` | **P2A international-allocation operational shadow.** Appends one US/USD VXUS assessment per ISO week from persisted paper positions and the latest immutable policy snapshot. It records `action=none`, null costs/tax drag, coverage state, and input fingerprints while target/band remain unset. No provider call, candidate, paper/live position, order, broker, or India/INR read. |
```

The route fails closed (publishes a failed/insufficient-data run, never yesterday-as-today) when a broker snapshot is missing/stale, so cron timing is a best-effort ordering, not a correctness dependency. EDT/EST caveat: the US job is set for EDT (summer); shift it +1h at the November EST changeover. Both jobs are re-scheduled idempotently (unschedule-first) by migration 156.

Cron authentication

All cron-triggered routes verify the x-cron-secret request header using a timing-safe comparison (verifyCronSecret() in lib/auth/cron.ts). The secret is CRON_SECRET in env and Vercel environment variables.

Cron routes do NOT require an owner session - they accept the cron secret as an alternative. This means the secret must be kept private; anyone with it can invoke cron routes.

Daily schedule overview (US trading day)

```
5:30 AM ET - scan-india-refresh (NSE cache)
6:15 AM ET - research-india
6:35 AM ET - position-monitor-india
8:00 AM ET - brief-morning + macro-sentinel (Mon only)
9:00 AM ET - research (US)
9:25 AM ET - price-cache-fill (Markets ETF cache; retry 9:45)
9:45 AM ET - trader (US proposals)
10:15/11:15 AM ET - paper-trade-us (EST/EDT; fixed 15:15 UTC)
4:15 PM ET - position-monitor (US)
4:30 PM ET - brief-evening
5:00 PM ET - nav-snapshot
5:00 PM ET - learner (Fri only)
Every 4h - stale-check (cloud, Vercel)
Sun 8 PM ET - theme-scout
2:00 PM UTC - autonomous-live (cloud, Vercel, weekdays; after research+signals at 1 PM UTC)
Sun 2 AM UTC - p1-gate (cloud, Vercel)
1st of month 3 AM UTC - db-cleanup (cloud, Vercel)
11:00 AM UTC - holding-risk India (pg_cron, weekdays; after 15:30 IST close)
9:30 PM UTC - holding-risk US (pg_cron, weekdays; after 16:00 ET close + nav-snapshot)
Fri 9:45 PM UTC - validation-sweep (pg_cron; recovers never-validated challengers, after learner)
9:10 PM UTC - downside-hedge-us (pg_cron; US paper only, default OFF)
10:15 PM UTC - benchmark-scorecard (pg_cron; benchmark-alpha measurement only, after NAV + labels)
2:10 AM UTC - earnings-pit-capture (daily; data capture only)
10:15/10:35 AM UTC - India Markets full snapshot + retry (weekdays; display only)
```

Research capacity note (2026-07-17): the US and India jobs both prioritize stale holdings, but held rows no longer spend an LLM narrative call because the decision is deterministic. One existing worker is reserved for candidates while the others prioritize holdings, so discovery always advances without raising concurrency. Candidate overflow retains its original defer time and is bounded to six deferrals/seven days. Theme Scout's seven-day owner-scoped rows and `watchlist.research_enabled` prevent disabled or month-old machine discoveries from silently expanding the daily universe. Prewarm and evidence-shadow are Canonical Evidence Router jobs, not GitHub/Vibe jobs; shadow remains observational and never feeds scoring or orders, though its provider calls still obey the shared pacing layer.

Order-maintenance correction (2026-07-26): The every-30-minute maintenance trigger only sends stale-order cancellation during a US market session and processes one candidate. It independently reconciles one unknown order through a read-only broker status query; the owner can request that read-only reconciliation from Trading. The limits avoid overnight/weekend MCP churn and leave cold-start headroom.

06-env-variables

Source: docs\arch\06-env-variables.md | Authoritative operational architecture

Kairos - Environment Variables

Last updated: 2026-07-10

Update this file when: a new env var is added, an existing var is removed, a var moves to/from the vault, or default values change.

All secrets live in `.env.local` (gitignored) + Vercel environment variables for production. The `api_key_vault` Supabase table holds runtime-editable keys - see `lib/vault.ts`.

Warning about encoding: A UTF-8 BOM in `.env.local` causes the entire app to 500 at boot.

If the app won't start after editing env, check the file encoding (must be UTF-8 without BOM).

Required at startup (app won't boot without these)

<code>NEXT_PUBLIC_SUPABASE_URL=</code>	<code># Supabase project URL</code>
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY=</code>	<code># Supabase anon key (public, safe)</code>
<code>SUPABASE_SERVICE_ROLE_KEY=</code>	<code># Supabase service role (private - never expose to client)</code>
<code>ANTHROPIC_API_KEY=</code>	<code># Claude API</code>
<code>CRON_SECRET=</code>	<code># Shared secret for Vercel cron + Task Scheduler auth</code>

Optional - features degrade gracefully without them

```
# Stripe (billing)
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=

# Email
RESEND_API_KEY=
BRIEFING_TO=                # Email address that receives briefings (test mode: any address)

# LLM observability
LANGFUSE_PUBLIC_KEY=
LANGFUSE_SECRET_KEY=
LANGFUSE_HOST=              # e.g. https://cloud.langfuse.com

# Vector / RAG (off if absent - no-ops silently)
JINA_API_KEY=               # free at jina.ai (no CC), 1M tokens/month

# Experiment tracking
WANDB_API_KEY=

# Robinhood MCP (OAuth token stored in vault at runtime)
ROBINHOOD_CLIENT_ID=        # From MCP dynamic registration
ROBINHOOD_CLIENT_SECRET=    # From MCP dynamic registration

# Provider adapter overrides (default to jina/resend if absent)
EMBEDDING_PROVIDER=         # jina (default) | openai
RERANK_PROVIDER=            # jina (default) | cohere
EMAIL_PROVIDER=             # resend (default) | smtp

# OpenAI (only needed if EMBEDDING_PROVIDER=openai)
OPENAI_API_KEY=

# Cohere (only needed if RERANK_PROVIDER=cohere)
COHERE_API_KEY=

# SMTP (only needed if EMAIL_PROVIDER=smtp)
SMTP_HOST=
SMTP_PORT=
SMTP_USER=
SMTP_PASS=
```

Runtime-editable (stored in `api_key_vault` table, editable via `/dashboard/admin/vault`)

These keys are NOT in `.env.local`. They live in Supabase and are fetched at runtime via `lib/vault.ts`.

```
| Key name | Display name | Notes |
| `ALPHA_VANTAGE_API_KEY` | Alpha Vantage | Free tier: 25 calls/day; exhaustion fires System Health warn |
| `MASSIVE_API_KEY` | Massive Market Data | US candles + screener |
| `FMP_API_KEY` | FinancialDatasets | ResearchAgent screener candidates |
| `KITE_ACCESS_TOKEN` | Kite Access Token | Daily refresh required (SEBI mandate: expires 6 AM IST) |
| `ROBINHOOD_ACCESS_TOKEN` | Robinhood OAuth Token | CAS-protected refresh via `lib/robinhood-mcp-client.ts` |
| `ROBINHOOD_REFRESH_TOKEN` | Robinhood Refresh Token | Used to refresh access token before expiry |
| `WEBULL_MCP_*` (vault, `provider: webull_mcp`) | Webull Cloud MCP OAuth | `WEBULL_MCP_CLIENT_ID` / `*_ACCESS_TOKEN` / `*_REFRESH_TOKEN` / `*_TOKEN_EXPIRES`. Written on the one-time OAuth connect (`/api/broker-mcp/webull/login`->`/callback`; legacy `/api/webull/*` 307-redirects there); CAS refresh in the config-driven `lib/brokers/mcp-driver.ts` (registry entry `MCP_BROKERS.webull`). Read-only (Phase 1). NOT env vars
- vault rows |
| `STOCKTWITS_TOKEN` | StockTwits | Social sentiment for US tickers |
```


Provider adapter env vars (added 2026-07-10)

Var	Default	Options	Effect
<code>`EMBEDDING_PROVIDER`</code>	<code>`jina`</code>	<code>`jina`</code> , <code>`openai`</code>	Selects the embedding model used for RAG trade memory
<code>`RERANK_PROVIDER`</code>	<code>`jina`</code>	<code>`jina`</code> , <code>`cohere`</code>	Selects the reranker used after vector retrieval
<code>`EMAIL_PROVIDER`</code>	<code>`resend`</code>	<code>`resend`</code> , <code>`smtp`</code>	Selects the transport for briefing emails

When `EMBEDDING_PROVIDER` or `RERANK_PROVIDER` is absent or set to `jina`, the Jina AI free tier is used (1M tokens/month, no credit card required). When `JINA_API_KEY` itself is absent, the entire RAG path silently no-ops - no errors, no embeddings, no retrieval.

Where each key is used

Key	Used by
<code>`ANTHROPIC_API_KEY`</code>	<code>`lib/llm-router.ts`</code> (Claude tiers)
<code>`CRON_SECRET`</code>	<code>`lib/auth/cron.ts`</code> -> all cron routes
<code>`SUPABASE_SERVICE_ROLE_KEY`</code>	<code>`lib/supabase/service.ts`</code> -> crons + admin ops
<code>`ALPHA_VANTAGE_API_KEY`</code>	<code>`lib/av-cache.ts`</code> , ResearchAgent, MacroSentinel, ThemeScout
<code>`MASSIVE_API_KEY`</code>	<code>`lib/massive-data.ts`</code> -> US candles + screener
<code>`FMP_API_KEY`</code>	ResearchAgent screener (FinancialDatasets <code>`screen_stocks`</code>)
<code>`KITE_ACCESS_TOKEN`</code>	<code>`lib/brokers/adapters/kite.ts`</code>
<code>`ROBINHOOD_ACCESS_TOKEN`</code>	<code>`lib/robinhood-mcp-client.ts`</code>
<code>`JINA_API_KEY`</code>	<code>`lib/providers/embeddings/jina.ts`</code> , <code>`lib/providers/rerank/jina.ts`</code>
<code>`RESEND_API_KEY`</code>	<code>`lib/providers/email/resend.ts`</code>
<code>`LANGFUSE_*`</code>	<code>`lib/llm-router.ts`</code> (Langfuse tracing)
<code>`STRIPE_*`</code>	<code>`app/api/stripe/*`</code>

07-coding-conventions

Source: docs\arch\07-coding-conventions.md | Authoritative operational architecture

Kairos - Coding Conventions

Last updated: 2026-07-17

Update this file when: a project-wide convention changes, a new pattern is adopted across all files, or an existing pattern is deprecated. This chapter changes rarely.

Codified in PRD .md ?2. All agents must follow; apply consistently to every file.

1. Styling

- T token object declared at the top of every file that renders UI:

```
const T = {
  bg: "#0D0F14", surface: "#13151C", card: "#1A1D27", border: "#252836",
  text: "#ECEDEF", textSub: "#9B9EA8", muted: "#6B7280",
  accent: "#6366F1", green: "#34D399", red: "#F87171", yellow: "#FBBF24",
};
```

- All styles are inline (style={{ ... }}). No Tailwind utility classes.
- Color must come from T. * - never hard-coded hex outside the T object.
- Mobile-first by default: every page/component must be responsive at 375px+.
- No CSS modules. No styled-components. No class utilities of any kind.

2. Database access

```
| Context | Import | Notes |
| Server components / API routes | `createClient()` from `@/lib/supabase/server` | Cookie-based auth; runs server-side |
| Service role (crons, admin ops) | `createServiceClient()` from `@/lib/supabase/service` | Bypasses RLS; never use in client components |
| Client components | `createClient()` from `@/lib/supabase/client` | Browser-side; respects RLS |
```

Never import the service-role client in client components. The service role key must never reach the browser.

3. API route conventions

- All routes in app/api/. Each file exports named HTTP handlers (GET, POST, PATCH, DELETE).
- export const dynamic = "force-dynamic" on all routes that read runtime state.
- Auth gate: requireOwner() from @/lib/auth/require-owner on owner-only routes.
- Cron auth: verifyCronSecret(req) from @/lib/auth/cron (timing-safe comparison).
- Return NextResponse.json({ error }) with correct HTTP status on errors.
- Cron routes: accept x-cron-secret header (bypasses owner session check) OR require owner session.

Unavailable ? zero (display-data contract)

A data route must never express "I could not get this" as a number. A zero fallback renders as a confident, colour-coded value (a green +0.00% pill is indistinguishable from a genuinely flat sector), which is a silent lie.

Convention for display quotes (/api/markets/overview is the reference implementation; see lib/markets/daily-change.ts):

- Nullable values: price, change, changePct are number | null. There is no zero for a failure to hide behind.
- Explicit per-item state: status: "ok" | "unavailable" plus a human-readable reason when unavailable (per the "Detail Over Cryptic" rule - say what failed and why, never a bare status).
- Payload-level state: stale, unavailableCount, and degraded (a sentence, present only when the whole route is degraded, e.g. missing credentials or a provider rate limit).
- Any client field the route emits must actually be read. A stale flag the consumer's interface omits is the same outage, silently.
- Degraded payloads are not cached - they must clear the moment the provider or credential recovers.
- Provenance must be truthful: name the real provider, the real endpoint semantics (end-of-day vs intraday), and the data's own age (fetchAt from the server), never the browser clock at response time.

Daily change means close vs PRIOR close

A daily change is the latest session's close against the prior session's close. A single session's (close - open) / open is the *intraday* move - a different, usually smaller number whose sign can invert. Verified on the 2026-07-15 session: XLRE's intraday was -0.11% (red) while its true daily change was +0.18% (green). Never source a "today" figure from one session's OHLC alone.

Massive free tier: ~5 requests/minute (hard)

MASSIVE_API_KEY is rate limited at ~5 req/min, shared by the whole app. A per-symbol fan-out silently blows it - 15 parallel /prev calls leave ~10 failing, which is exactly how zero-fallbacks become a wall of +0.00%.

- Prefer grouped daily (/v2/aggs/grouped/locale/us/market/stocks/{date}): every US ticker's OHLC for one session in ONE request. Two calls (latest session + prior session) serve an entire tile universe and keep every symbol on the same session.
- Grouped for the current calendar date is refused until after midnight ET ("Attempted to request today's data before end of day"), and holidays return 200 with no results - so walk back from today, skipping weekends, and take the first dates that return bars.
- Past sessions are immutable -> cache grouped responses hard (by date).
- A single grouped call needs no pacing lease. Per-symbol fallbacks DO - try_acquire_provider_slot (12.5s = 5/min); see price-cache-fill.

Auth gate matrix

```
| Route type | Gate |
| `~/dashboard/**` pages | Supabase middleware (redirects to login) |
| `~/api/**` routes | Each route calls `requireOwner()` itself - middleware does NOT cover API routes |
| Cron routes | Accept `x-cron-secret` header OR require owner session |
| Admin routes | `requireOwner()` + role check |
```

4. Agent conventions

- All agents write to shared Supabase tables. Zero direct agent-to-agent HTTP calls.
- Every agent run logs to agent_runs (start/end/status). Log before any external call.
- LLM calls go through callLLM() in lib/llm-router.ts (Langfuse tracing + cost logging).
- Multi-step tool loops use runAgentLoop() (also Langfuse-traced).
- Scores are deterministic (no LLM). Thesis/direction text comes from fast (Groq).
- Use tier aliases (fast, claude-smart, claude-opus, etc.) - never hardcode model names in agent files.

5. TypeScript

- Strict mode always on.
- Centralized types at @/types/ - add new shared types there, not inline in route files.
- No any outside of tightly-scoped third-party type bridging.
- All API response shapes typed.

6. File structure

```
app/
  api/
    agents/<name>/route.ts - agent endpoint
    briefing/generate/route.ts
    admin/route.ts
    ...
  dashboard/<page>/page.tsx - dashboard pages
components/
  dashboard/
    ui/ - shared UI primitives
lib/
  supabase/
    server.ts - cookie client
    client.ts - browser client
    service.ts - service role client
  llm-router.ts
  vault.ts
  av-cache.ts
  research-agent.ts
  providers/
    embeddings/
    rerank/
    email/
  brokers/
    adapters/
    registry.ts
```

7. What never to do

- Never re-litigate approved decisions. If it's in `PROJECT_DECISIONS.md` as approved -> implement it, don't redesign it.
- Never invent styling conventions. Inline styles with T color tokens only.
- Never use Tailwind utility classes. The codebase does not use them.
- Never touch the primary Robinhood account. Agentic account only (see `08-risk-and-safety.md`).
- Never add features beyond the current task scope. No scope creep.
- Never commit secrets. No API keys, no tokens, no service role keys in code.
- Never modify `AGENTS.md` or `PRD.md` without Architect role + Vaibhav approval.
- Never call agent endpoints from other agent endpoints. Table-mediated coordination only.

08-risk-and-safety

Source: docs\arch\08-risk-and-safety.md | Authoritative operational architecture

Kairos - Risk & Safety

2026-08-01 documentation truth audit: the technical breakdown guard is a hard cap only for an ATR-scaled fall or a 7% high-volume fall. A bottom-quartile weak close is warning-only. This chapter now matches the exact scoring contract in chapter 03 and lib/data/technicals.ts.

2026-07-31 scoring input safety: unknown provider taxonomy cannot receive a fabricated P/E benchmark; implausible P/E is omitted; missing availability metadata cannot include a dimension; malformed macro/insider payloads remain unavailable; and ordinary weak closes cannot hard-veto without ATR or volume/move confirmation. Only the Next.js ResearchAgent writes authoritative scores. See features/scoring-data-truth/FEATURE_ARCHITECTURE.md.

Last updated: 2026-07-25 (ETF allocation cap - new strategy_config.etf_allocation_cap_pct NUMERIC DEFAULT 30 (migration 20260725130000)). Soft guardrail: executeApprovedOrder checks BUY orders for US ETF symbols (isEtfSymbol) and refuses if current ETF portfolio % + this order's notional / NAV would exceed the cap. Fail-open: any DB read error warns and allows through (unlike symbol blocking which fails closed). India market skipped. SELL always allowed. Configurable 0-100 via Settings -> Agents -> Risk Profile -> ETF Allocation Cap.)

Prior: 2026-07-20 (Guarded kill-switch reset + fail-closed risk reads. Settings now reads the real market_controls latch and can reset it only through owner-only POST /api/settings/kill-switch: explicit market/book + acknowledgement, originating-book match, current breaker recheck with alert resolution suppressed, durable decision-journal audit, then latch write and matching-alert resolution. Failed config/NAV/history reads block risk increase; the legacy unscoped paper-query retry is removed, so schema/read failure cannot mix US and India risk truth. Trip reason and issue key include paper | live; the conservative per-market latch remains shared across books.)

Last updated: 2026-07-19 (Webull Trading API transport restored - lib/brokers/webull-trade/ is now a fully wired, permit-backed broker adapter. Key safety properties: (1) Nine-gate ladder (gates.ts) - every Webull order must clear global_trading_enabled, market_control_enabled, circuit_breakers_clear, autonomy_mode_satisfied, single_allowlisted_account (account 605420606 exclusively), orders_feature_flag (webull_trade_orders_enabled=false by default), credential_and_token, risk_checks, quantity_within_mandate; (2) Permit system - two permit kinds, preflight (DB flag check only) and order (full 9-gate evaluation); each permit is single-use and expires after 30 s; the transport refuses any request whose path/method is not on the permit's allowlist; (3) Token preflight - preflight.ts calls POST /openapi/auth/token/check before gate 7 to resolve live token status (PENDING/NORMAL/INVALID/EXPIRED) and idle-age; assertTokenUsableForOrder() blocks on PENDING (now explicit in types), INVALID, EXPIRED, and idle > 15 days; (4) Single fetch locus - liveWebullTransport() is the only function in the codebase that calls fetch() to api.webull.com; (5) Signing - HMAC-SHA1 over path + sorted params + MD5 body hash, \${appSecret}&key, x-access-token header alongside HMAC headers, timestamp freshness enforced; (6) All flags remain false: webull_trade_orders_enabled=false, global_trading_enabled, and account allowlist require explicit owner activation before any order is possible. Constraint migration 20260718130000 applied: protective_orders.mode locked to 'wider_disaster_floor', currency NOT NULL, five new DB-enforced invariants on market/currency/broker-id/floor/order-kind/learning-provenance - zero rows in table at time of apply, all constraints verified safe.)

Prior: 2026-07-18 (Hybrid protective-stop SHADOW SCAFFOLD - built UP TO the placement line, no live order ever. New pure module set under lib/protective/: (1) a broker-neutral BrokerProtectiveCapabilities matrix (capabilities.ts) - protection is capability-driven per order-type/TIF/session/account, never a flat broker boolean; an adapter with no eligible MULTI-DAY order for the exact position is unprotected-by-broker, never silently protected; (2) Kite's declared capability (kite-capabilities.ts) filled from the

PROVEN GTT code - Kite protects via the WEAKER gtt_limit (LIMIT child, unfilled-trigger risk surfaced), and its DAY-only regular SL-M is declared and correctly REJECTED as a multi-day floor; (3) a pure disaster-floor calculator (disaster-floor.ts) parameterized by (mode, distance) - Q1 unanswered so mode defaults to wider_disaster_floor (outage + catastrophic-loss mitigation, NOT touch-at-analytical-stop) and the distance is a CONFIG input with no hardcoded value; monotonic ratchet so a falling high-water mark can never lower the floor; (4) a pure reconciliation loop (reconcile.ts) detecting out-of-band triggers, partial fills, cancels, expiry, broker edits, and corporate-action qty drift - unknown state is always needs_reconcile, a trigger without a confirmed fill never closes the book, a gap through a limit child is reported unprotected (not filled), expired protection is critical; (5) the state model (state.ts) - the protective_order record shape + status machine + long-only/cancel-before-replace invariants (total executable SELL never exceeds reconciled held qty; a competing SELL is blocked until cancellation is CONFIRMED). Exit provenance (Codex's correction): a disaster-floor fill records exit_reason = protective_disaster_floor and learning_scope = risk_policy_only - the loss STAYS in P&L, NAV, drawdown, mandate and risk-policy evaluation (real money); ONLY the Learner's signal-weight attribution + genome promotion exclude it, so broker capability can't contaminate weight learning (a generic excluded_from_learning=true is insufficient because the evaluation engine also filters that field). THE MONEY LINE: placement-gate.ts - a false-by-default strategy_config.protective_orders_enabled flag gates ALL placement and STAYS FALSE; planProtectivePlacement() produces the intended broker action as a plain object and NEVER calls a broker. Deterministic, NO LLM on the money path. US/India never cross (per-market capability scope). Migration 20260718000000_protective_orders_shadow.sql written as a PROPOSAL and NOT applied to prod (creates protective_orders + append-only protective_order_events, adds the flag + learning_scope columns). 32 acceptance/unit tests in tests/protective-hybrid-stop.test.ts (all 14 spec acceptance tests, each falsifiable - mutation-verified: breaking the ratchet fails AT3, breaking the cancel-guard fails AT1). Gated behind owner approval of touch semantics (Q1), floor distance, and post-fill policy before anything goes live. See "Hybrid protective-stop shadow scaffold" below + features/hybrid-stop/FEATURE_ARCHITECTURE.md.)

Prior: 2026-07-17 (Research visibility on the risk surface - a DISPLAY JOIN, coupled to nothing. GET /api/portfolio/risk-daily now attaches a nullable per-holding research block (score, direction, scored_at, sessions_since, days_since, state, scored_as_holding) joined from the latest agent_signals row per (symbol, market) - never symbol alone. Invariant R1: no field of it is read by computeHoldingRisk, sba-v1, constructPortfolio, the execution kernel, or any gate - the risk engine stays research-free BY DESIGN, because a sector is over-cap *because* research liked that sector and letting analyst_score also veto the cap double-counts the same signal. Why it exists: on 2026-07-16 AVGO was 6 days unscored while this panel said "trim", and nothing on screen said so - the AGE is the feature, not the score. Staleness is measured in market-local SESSIONS (reusing marketSessionsSince; a Friday score read Monday is 3 days but ONE session - a calendar-day rule would paint the book stale every Monday) and displayed in days. Four non-collapsed states: fresh (<= 2 sessions) ? stale (warning + day count; annotates the SCORE only, never the action) ? never (no signal ever - deliberately NOT a link) ? unavailable (abstained - never rendered as a number). Every annotated row is labelled a screener candidate score: is_holding is false in 463/463 prod rows, so a neutral there does not mean "no exit signal" - the exit question was never asked. Fail-soft: an agent_signals error still renders the risk table with an explicit "research unavailable". R1 is pinned behaviorally AND architecturally by tests/risk-research-annotation.test.ts, whose coupling detector is itself falsification-tested. Supersedes the unmerged features/risk-research-integration (research *ordering* trim absorption - not pursued). No schema change, no migration, no new cron, no LLM. See "Daily Per-Holding Risk Analytics" below + features/risk-research-visibility/FEATURE_ARCHITECTURE.md.)

Prior: 2026-07-16 (Sector-cap breach ALLOCATOR - hr-v1 -> hr-v2. Defect fixed: a sector-cap breach is a property of the SECTOR, so sectorUtil >= 1 was the identical number for every holding in the sector and hr-v1 handed EVERY Technology name the identical trim (the live AVGO advice: "Trim your position because Technology holdings exceed the 30% sector cap (at 65.6%)") without ever deciding WHICH names absorb the breach or HOW MUCH each gives up - arbitrary and unactionable. New pure module lib/risk/sector-breach.ts (sba-v1) allocates the breach deterministically by water-fill: trim the largest names in the sector down to a common level L where ? min(w?, L) = cap. Justified over pro-rata (which re-ships the same blanket verdict and leaves the name-cap breach untouched) and over "marginal contribution" (for a sector-weight cap, a name's marginal contribution IS its weight - the same rule with extra abstraction). NAV basis, because live-portfolio-gate enforces the owner's cap as value/NAV - the invested basis would make the advice ~43% wrong. Safety properties: (1) exits untouched - the exit_review branch is first and unconditional; no allocation, and no absence of one, can delay or suppress a protective-stop/thesis-break exit; (2) risk-internal - a function of weights and one owner-set cap, ZERO research/analyst_score coupling; (3) defaults to honest - a sector breach with no usable allocation yields review + missing_inputs:["sector_breach_allocation"], NOT a fallback to the old blanket trim; (4) sector-unknown degrades honestly - excluded from every sector total, never bucketed into a synthetic sector, never assumed cap-compliant; (5) LLM still prose-only - parseStrategyNotes (lib/risk/strategy-notes.ts) can only emit Map<requestedSymbol, string>, proven by test. Non-selected names now say hold with the reason they weren't selected. Read-only accounts (everything but 605420660) are labelled advisory-informational. No migration. See "Daily Per-Holding Risk Analytics" below + features/risk-sector-breach-allocation/FEATURE_ARCHITECTURE.md.)

2026-07-21 router proof hardening: cohort evaluation is cache-only and cannot lease, call, or enqueue provider work. ResearchAgent copies already-fetched deterministic score inputs into the canonical cache through an internal read-only adapter. Activation now requires separate safety_pass and quality_pass, a fresh selected proof, and ten distinct validated ResearchAgent as_of_session values in a 45-day window for the exact market/policy/code/strategy tuple. Weekend/holiday staged rows do not count. Existing rows default false and cannot authorize cutover. Router remains shadow-only, router_enabled=false both markets.

2026-07-31 ADR safety: reviewed ADR identity is explicit, never inferred. SKHY uses the Nasdaq ADS and ADS-basis Yahoo fundamentals; retired/OTC proxies (SKHYV, HXSCL, HXSFC) are rejected by the shared paper/live symbol policy. A thin ADS source becomes unavailable rather than falling through to foreign-underlying per-share data. ADR support adds no live-trading permission and does not bypass broker review or any existing market/account/risk gate.

Prior: 2026-07-16 (Runtime evidence-degradation guard - a NEW safety gate on the research entry path, shipped measure-only. Problem it solves: scoring renormalizes weights across *available* dimensions, so a dimension dropping out (provider outage OR a routing change) could push a symbol from ineligible->eligible on missing data rather than new information. The guard compares each symbol's current availability/quality mask against the last accepted market-local baseline; if a REQUIRED field degrades (fresh->stale beyond ceiling, available->missing, valid->conflict/quarantined) it abstains from any NEW long whose eligibility depends on renormalizing around that degradation. Safety properties, all enforced: (1) strictly subtractive - the only transformation is long -> neutral; short passes untouched in every mode, checked before the mode branch AND enforced by a schema CHECK, so no code path can persist a guard event that *created* an entry; (2) never suppresses an exit - existing holdings continue through PositionMonitor's normal risk/exit logic, an evidence outage cannot block a stop or mandatory exit; (3) defaults to abstain (no baseline + unusable required field ? abstain), and only clean runs re-baseline, so a persistent outage never normalizes itself; (4) fail-safe config - EVIDENCE_DEGRADATION_GUARD_MODE defaults to measure_only and an unparseable value ALSO falls back to measure_only, so a broken config can neither silently enforce nor silently stop recording; (5) one aggregated health event per run, not one alert per symbol. Modes: off | measure_only | enforce. Shipped in measure_only - it records what it *would* abstain and changes no direction. Flipping to enforce is an owner decision after observing its logged would-abstain rate. Also: analyst .consensus is classified narrative-only (US) / unsupported (India) and excluded from gated intents, so it can never block a cutover. Router itself remains shadow-only, router_enabled=false both markets.)

Last updated: 2026-07-15 (LLM-discretion exit hole CLOSED - the last place LLM output could move money. Research direction gate extracted to pure lib/signal-direction.ts (unit-tested, tests/signal-direction.test.ts): held-position exit ("short") is now DETERMINISTIC - isHeld && analystScore < mandate threshold - the LLM's direction field NEVER sets an executable direction (previously an LLM "short" on a held name became an exit signal stored as deterministic_v1, and could teach the learner from LLM-created outcomes). SELL capability on holdings preserved per locked rule, now evidence-driven; LLM opinion kept advisory-only in research_packets.raw_data._original_direction. LearnerAgent's reassess flag renamed llm_exit->score_reassess_exit (score-only trigger); PositionMonitor honors both (legacy drain). Historical contamination verified ZERO (no closed trade ever exited via llm_exit or a long->short LLM flip). Entries were already deterministic; paper/live consumers already require score_source="deterministic_v1".)

Prior: 2026-07-15 (Supabase Security Advisor remediation - 20260715120000_security_rls_and_rpc_lockdown.sql: the public anon API key could read 16 RLS-disabled public tables (incl. agent_config/learner_config) and call SECURITY DEFINER RPCs (kairos_call_agent, activate_evidence_policy, ...) because they carried the default GRANT EXECUTE TO PUBLIC. Fix: RLS deny-all on 15 agent-internal tables + authenticated-read on newsletters (service_role bypasses, so agents/crons unaffected); REVOKE EXECUTE ... FROM PUBLIC on the anon-callable definer RPCs (keeping service_role, and authenticated for the owner's get_daily_ai_count); pinned search_path=public on 15 definer/trigger fns. Verified via get_advisors: 0 ERROR, 0 anon-executable definer functions. Deferred WARNs: 7 always-true policies tighten at multi-tenant, pg_net schema move, Auth leaked-password toggle.)

Prior: 2026-07-11 (Proactive broker-token health check - checkRobinhoodTokenHealth attempts a CAS refresh on the short-lived RH access token from the status route + health-triage cron (6h), reporting broker - token: robinhood only on a genuinely failed/absent refresh; Settings badge shows "Reconnect required" only on a real dead-refresh, else "Connected - valid until <access-token TTL>". See "Proactive broker-token health check" section. Prior: Daily Per-Holding Risk Analytics advisory surface.)

Prior: 2026-07-11 (Codex Phase-B re-review remediation: (Codex#2/#3) the Kite identity gate no longer trusts allowlist *text* alone - verifyKiteTradingIdentity() (lib/kite.ts) now fetches Kite /user/profile and requires the CONNECTED token's user_id to equal strategy_config.active_account_india AND an allowlisted broker_accounts{broker=kite,market=india,role=trading} row. It is enforced at the single placeEquityOrder() choke point, so the canonical/autonomous/exit paths (via kiteAdapter.submitOrder) get the same check as the standalone route - not just the route. Fail-closed: config read err ? 500, unset/absent/view_only ? 403, profile unfetchable / user_id mismatch ? 502/403. (Codex#1) the v2 budget advisory lock DROPS broker from its key (now local_date:market:env) so it matches the market-wide cap it guards - two brokers in one market can no longer take different locks and jointly exceed the cap. (Codex#4) migration 153 rejects non-finite (Infinity/NaN) qty/notional/cap and validates side/env/broker/symbol/order_type enums+identifiers, fail-closed. Migration 153 additive over 152 (never edited).)

Prior: 2026-07-11 (Phase B residuals of 07_08_FULL_APP_REVIEW: A2 the standalone India Kite order route (app/api/kite/order) now enforces a fail-closed identity/allowlist gate - it requires strategy_config.active_account_india to match a broker_accounts{broker=kite,market=india,role=trading} row before reserving budget; unset/absent/view_only ? 403 (no silent fallback), so India live is blocked until an allowlisted Kite trading row is inserted. Canonical-path *unification* still deferred. A4 read-only NAV reconciliation report GET /api/paper/nav-reconcile (owner-gated, zero writes) re-derives nav == cash + ? qty?price per pool. A5

v2 daily-BUY budget window is market-local (America/New_York / Asia/Kolkata), not UTC; advisory lock keyed by local_date:market:broker:env. Dead edge-fn kill-switch copy deleted.)

Prior: 2026-07-11 (Phase A P0 remediation of 07_08_FULL_APP_REVIEW: A1 kill switches take explicit {book, accountId} context - mode no longer inferred from live_auto_enabled; live baseline = account's own snapshot peak not START_NAV; new sellAllowed separates risk-increase from risk-reduction so a trip blocks BUY but not a verified SELL; no_baseline/stale_snapshot fail-close BUY only. A3 durable broker ACK (bounded DB retry -> 202 needs_reconcile, never {ok:true}). A4 PositionMonitor NAV write errors now fatal. A5 budget-RPC v1/v2 EXECUTE revoked from public/anon/authenticated.)

Prior: 2026-07-10 (Phase 1 P0: L4 enforcement, conviction normalization, India currency, duplicate SELL, cancel-on-kill BUY-only; Codex P0/P1: breakdown veto, calibration OOS gate, promotion governance.)

Update when any authorization, scoring eligibility, limit, account, order, reconciliation, exit, or kill-switch behavior changes.

2026-07-26 policy-event ledger: US FOMC schedule, official target-range outcomes, and post-event return observations are display/measurement-only. Missing market expectations render unavailable; they never become a neutral or zero surprise. The ledger has no scorer, sizing, paper, live, exit, broker, or India reader. Post-event impacts use only frozen daily-return evidence and are append-only.

Overview

Manual and future autonomous orders must pass one shared Execution Gateway. Manual and auto differ only in who authorizes the proposal; they do not have separate money-safety implementations.

```
flowchart TD
    INTENT[Trade proposal] --> ACTOR[1. Owner or authorized auto lease]
    ACTOR --> VERSION[2. Deterministic live-approved scoring version]
    VERSION --> ENABLED[3. Global market broker account enabled]
    ENABLED --> KILL[4. Kill switches and critical alerts]
    KILL --> QUALITY[5. Data evidence and mandate]
    QUALITY --> ACCOUNT[6. Fresh target-account NAV positions buying power]
    ACCOUNT --> LIMITS[7. Per-order and portfolio limits]
    LIMITS --> QUOTE[8. Fresh quote spread drift]
    QUOTE --> RESERVE[9. Atomic daily budget and idempotency]
    RESERVE --> PREVIEW[10. Broker preview echo]
    PREVIEW --> SEND[11. Broker submit]
    SEND --> RECON[12. Durable lifecycle sync and protective exit]
```

Unknown, null, stale, malformed, or errored state on a live BUY fails closed. Verified risk-reducing SELL exits remain exempt only from BUY budgets/caps; they still require account, holdings, quote, idempotency, broker, and audit checks.

Completed-session and schedule boundaries (2026-07-31)

Daily ResearchAgent technical evidence is filtered after provider normalization and before scoring, return capture, trade-plan construction, or evidence persistence. A market-local current-date daily candle is admitted only after 16:00 America/New_York (US) or 15:30 Asia/Kolkata (India); malformed/future dates are rejected. Intraday quote consumers are unaffected.

US research, paper-entry, and daily monitor jobs use paired EDT/EST UTC schedules with an exact route-level local_slot contract. The seasonal duplicate exits before provider or database work. This prevents the close monitor from shifting to 15:15 ET in winter and prevents entry/research slots from silently moving by an hour.

India headline/event collection is shadow-only. Its intents have no scoring, learner-mutation, paper/live execution, position-monitor, or broker reader; unknown coverage remains unavailable rather than becoming a neutral score.

Gate details

1 - Authorization envelope

Manual: `requireOwner()` + request/CSRF guard + owner approval/Send action.

Future auto L4: all of deployment `AUTONOMOUS_LIVE_ENABLED=true`, `autonomy_level='L4_live_small_auto'`, `live_auto_enabled=true`, unexpired owner lease, authenticated cron worker, and a single-run lease. The gateway (`executeApprovedOrder`) enforces `autonomousWorkerAllowed(autonomy_level)` for non-owner actors - `L3_live_manual` is insufficient and returns 403. The autonomous path cannot use owner-only risk overrides.

Enabling an auto envelope is a human money/config change and is journaled. It is not permission for an LLM to change caps, accounts, strategy lifecycle, or code.

2 - Signal and scoring eligibility

Live BUY requires:

- deterministic `score_source`;
- strategy/scoring lifecycle `live_approved` with linked validation evidence;
- market/asset/setup allowed by the mandate;
- unexpired proposal and signal;
- long-only new-position decision;
- no unresolved taint or invalid LLM veto state.

A score threshold or `eligible_for_live_review` flag alone never grants live eligibility.

Breakdown veto (deterministic, runs before momentum math). `scoreTechnical`s (`lib/data/technical`s.ts) evaluates `detectBreakdownVeto` first. A last bar down at least 2.5 ATR, or down at least 7% on at least 1.5x normal volume, caps the technical score at 20 regardless of RSI/EMA. A bottom-quartile close on a down bar is recorded as a warning but does not trigger the cap. In plain language: an unusually large, well-confirmed fall can block a misleadingly strong momentum score; a merely weak close supplies context rather than pretending to prove a breakdown. This closes the prior bug where a 12% high-volume reversal scored about 100 because RSI had fallen into the preferred band while price was still above the EMAs. Thresholds remain v0 guardrails needing prospective validation per liquidity bucket.

Promotion governance (`app/api/strategies/versions/route`.ts). A champion can only be promoted with a PASSED validation experiment: the `force_unvalidated` bypass is hard-rejected (400). Demotion of the prior champion is always market-scoped - the former unscoped demote-all fallback is removed and now aborts the promotion on error, so promoting an India challenger never touches the US champion.

Automated validation boundary (migration 170, `activate_strategy_shadow` RPC, `lib/validation/automation`.ts). Automatic challenger validation + shadow routing (gated per-market by `strategy_validation_automation`, fail-closed) has exactly one automatic lifecycle transition: a PASSED challenger -> non-executing `shadow_paper`. It cannot promote a champion, create a paper fill, move cash, make a broker proposal, or place a live order - the RPC is `service_role`-only, holds a per-market advisory lock, caps at one shadow (`max_active_shadows` 0-1), and refuses any champion/terminal/unvalidated version. Promotion and every execution gate above stay separate and owner-only.

Paper accounting integrity (2026-07-13). Two silent-write bugs are fixed and guarded: (1) `disableTrading` (kill switch) wrote a non-existent `strategy_config.notes` column, so PostgREST rejected the whole update and the switch never actually set `trading_enabled=false` - notes removed, the switch now halts as intended. (2) An earlier `paper_portfolio` update bundled a non-existent `open_positions` column and silently dropped close-proceeds cash credits, understating NAV and tripping a PHANTOM drawdown/kill switch on India (~?197k). The lost cash was reconciled

from the trade ledger (seed - ?open-cost + ?realized), and the position-monitor now runs a ledger reconciliation guard every cycle: if cash_balance drifts from the ledger beyond 0.5% of seed it raises paper-cash-drift:<market> (warn) so drift is visible and actionable BEFORE the drawdown breaker acts on corrupted NAV.

Capital rotation P1 PAPER execution enabled (migration 20260723120000, owner-approved 2026-07-23). PaperTrader records a rotation_events audit row when a candidate is rejected for max_open_names or insufficient_cash, and - paper book only - may now execute the rotation. The gate stack, all fail-closed: CAPITAL_ROTATION_PAPER_ENABLED deployment env var, per-market rotation_config.rotation_paper_execute_enabled (true only for book_type='paper'; both live rows remain false), a fresh deterministic eligibility re-eval against the current book, persistence (>=2 distinct prior planned runs), cooldown (5d per symbol), per-run (1) and per-day (1) caps, and finally the atomic execute_paper_rotation RPC: it re-validates the signal claim, sells the source via execute_paper_exit (marked price - 5 bps), buys the candidate via execute_paper_fill (which re-runs mandate threshold, market controls, pyramid, name/sector caps, cash) in ONE transaction - a buy-leg denial rolls back the sell. The prior containment check constraint (rotation_paper_execution_p1_not_approved) was dropped by the same migration. There is NO live rotation path: no live proposal is created, and PositionMonitor remains the only owner of true exit labels.

New P0 rows now carry a fail-closed P1 readiness contract. Rotation source selection reuses the canonical paper exit-plan projection; post-swap sizing and measured correlation are evaluated over the same market only; persistence counts distinct prior runs; turnover uses filled paper-order notional against same-market NAV; and missing tax lots, economic mapping, configured turnover, correlation, or complete query results remain explicit blockers. These fields are diagnostic only and cannot relax an entry, exit, cash, position, or order gate. Agents -> Rotation exposes the market-local evidence without an enable control.

Per-market pause/kill isolation (migration 171). The pause and kill-switch state was GLOBAL (strategy_config.app_paused/trading_enabled), so one market's breaker halted BOTH - India's phantom drawdown even skipped the US research run. Now market_controls holds one row per market; the drawdown breaker calls setMarketPaused(market), the kill switch setMarketTrading(market, false), and every gate reads isPaused(svc, market) / isTradingEnabled(svc, market) (lib/market-controls.ts, fail-closed on read error). A market's trip isolates to that market; the legacy global flags are retained as a master-kill that still stops everything. Research is no longer gated by the pause at all (it is measurement - only entry paths pause). Exits keep running during a pause. Owner resumes a single market from the sidebar per-market banner (/api/settings/pause with a market).

3 - Trading/broker/account enablement

All global, per-market, broker, and account toggles must be true. Broker resolution fails closed.

US order account is exactly Robinhood agentic account 605420660. Account 965848641 is read-only for the approved research-holdings use; its NAV/positions cannot size or authorize agentic-account orders. The real implementation currently hardcodes/resolves account IDs; the documentation must not claim otherwise. Credentials/tokens remain encrypted in the vault and never enter code/logs.

4 - Kill switches

lib/kill-switches.ts checks per market for daily loss, peak drawdown, and rolling accuracy, disables trading, and creates a critical alert. Submit-time checks must rerun immediately before reserve/send.

Accuracy-gate minimum sample (MIN_ACCURACY_SAMPLE = 20, corrected 2026-07-29). Paper accuracy uses the actual realization timestamp and aggregates every pyramid lot closed for one symbol at one timestamp into one exit episode. The combined episode return is classified with the currency-neutral breakeven band; breakevens and tainted rows are excluded. The gate trips only after >=20 directional exit episodes, never 20 correlated entry lots. Live filled-order pairing is not a kill-switch input: pairing a SELL with the latest BUY is not broker-confirmed lot accounting and is wrong under pyramiding/partial fills. Live daily-loss and drawdown remain immediate from account NAV.

Guarded manual reset (2026-07-20). The per-market strategy toggle is not the breaker latch. Settings reads market_controls separately and exposes reset only while its trading_enabled is false. Reset names the originating

paper | live book, reruns that book's deterministic checks, refuses a mismatched book or active market pause, writes an unresolved audit record before enabling, and resolves it plus only the matching kill alert after the latch write succeeds. It never changes thresholds, accounts, global/security pause, credentials, or autonomy flags. Risk/config reads are strictly market-scoped and fail closed; the old pre-schema unscoped retry is forbidden.

`checkKillSwitches(svc, { market, book, accountId? })` takes an explicit book/account context (A1/P0-1). Mode is NO LONGER inferred from `live_auto_enabled` - an L3 manual-live order (`live_auto_enabled=false`) must still measure real live NAV, so the caller declares the book:

- book: "paper": reads `paper_portfolio / paper_performance` plus realized `paper_trades` exit episodes. A bare-string market arg (`checkKillSwitches(svc, "us")`) is the back-compatible paper form.
- book: "live": reads `live_account_snapshots` for the resolved account (`accountId`, else `active_account_{market}`) for daily-loss + drawdown. Accuracy remains analytics-only until a reconciled realized-lot ledger exists.

Live baseline is the account's OWN 90-day snapshot peak, never a static `START_NAV` - a real \$36 account is not measured against a \$10k paper floor.

Fail-closed for BUY (result { `safe:false`, `sellAllowed:true` }) on any of: no configured account or no snapshots (`tripped:"no_baseline"`), or newest snapshot older than `KS_LIVE_SNAPSHOT_MAX_AGE_MS` (default 6h, `tripped:"stale_snapshot"`).

Risk-increase vs risk-reduction are separated. The result is { `safe`, `sellAllowed`, `reason?`, `tripped?` }. A live daily-loss / drawdown trip sets `safe:false` (blocks BUY) but leaves `sellAllowed:true` - a risk-reducing SELL that has passed fresh exact-account held-quantity verification is not blocked. Callers gate as `ksBlocks = side=="sell" ? !sellAllowed : !safe`. A freshness fail-close likewise blocks BUY only. PaperTrader consumes `safe` only for new entries; PositionMonitor's deterministic held-position exits continue independently. `security_locked` still blocks live execution.

Atomic SELL idempotency: `trade_proposals_active_sell_uniq` partial unique index on (`symbol`, `market`) WHERE `side='sell'` AND `status` IN ('pending_review','queued_auto') enforces at most one active autonomous SELL per position at the DB level. Concurrent exit-monitor runs hitting the same position get a 23505 conflict, not a duplicate SELL.

For L4, any unresolved critical trading/data/reconciliation alert blocks new entries. Cancel-on-kill cancels only resting BUY orders - protective SELL orders are explicitly excluded (canceling an exit increases open exposure). Risk-reducing held-position exits remain allowed where state can be verified.

5 - Data quality and overrides

`data_confidence` uses structural applicable base weights:

`fresh valid applicable base weight / all structurally applicable base weight`

Inapplicable dimensions are omitted from both terms; missing/stale/failed/degraded dimensions stay in the denominator and contribute zero. Post-renormalization `applied_weights` are never the denominator.

Manual owner may use `acceptLowQuality` only with a durable reason written before the order. Auto has no quality or portfolio-risk override. `quality_status=unknown`, missing decision link, or confidence error blocks auto/live BUY.

6 - Fresh account state

NAV, positions, open orders, and buying power must come from the actual target account and meet explicit freshness bounds. There is no `FALLBACK_NAV` for live or auto sizing. If the target account cannot be read, BUY size is zero and SELL authorization fails unless current holdings can be independently verified.

India values remain INR and US values USD. No currency conversion is implicit. If a future cross-currency limit is needed, the FX observation/source/time is explicit and conservative.

7 - Limits and portfolio construction

Current per-order limits live in `strategy_config.max_order_notional_usd / max_order_notional_inr`; daily limits use `max_daily_notional_*` and `max_daily_trades`. Do not claim these are `broker_accounts.notional_cap_usd` unless the schema is actually migrated.

Final BUY size is the minimum of opportunity size, per-order cap, remaining atomic daily budget, buying power, and name/sector/gross/correlation/volatility limits. Quantity rounds down; zero means abstain. SELL that reduces a verified holding is exempt from BUY notional/daily budgets.

8 - Quote and drift

A fresh executable quote is obtained immediately before reservation. Validate positive finite price, retrieval age, spread/liquidity, and drift from proposal/approval. Use a marketable limit collar when the broker schema supports it; never guess tool parameters.

9 - Atomic budget and idempotency

All live submit paths must call the atomic budget-reservation RPC. It counts reserved/submitted/partial/unknown live BUYs and inserts `broker_orders.status='pending_submit'` in the same transaction. Unique active order per proposal is the hard duplicate backstop.

Advisory-lock scope must match the cap's query scope (Codex#1, migration 153). The daily-BUY cap counts/sums `broker_orders` filtered by `market + broker_env='live' + side='buy'` - it is market-wide, NOT per-broker. So the serializing advisory lock is keyed `hashtext(local_date:market:env)` and MUST NOT include broker: a broker in the key would let two concurrent live BUYs in one market but different brokers take *different* locks, both read the same pre-order total, and jointly exceed the market cap. Distinct markets still hash distinct and never block each other. (Migration 152 wrongly included broker; 153 narrows it back to the query scope.)

Fail-closed input validation (Codex#4, migration 153). The SECURITY DEFINER RPC rejects malformed service input rather than silently reserving: non-finite numerics (Infinity/-Infinity/NaN pass a naive `> 0` check, so `qty`, `estimated_notional`, and `max_daily_notional` are checked explicitly), a non-canonical `p_side/p_broker_env` (e.g. uppercase 'BUY' would skip the lowercased BUY-cap branch yet still insert a live row), and empty `broker/symbol/order_type` identifiers. A live BUY must additionally carry a positive finite notional.

The current RPC records `approved_by_user=true` for owner and `false` for `autonomous_worker` via `p_execution_actor (v2)`. A read/sum/check in TypeScript is forbidden because concurrent requests can exceed the cap.

10 - Broker preview

Robinhood requires `review_equity_order` before place. Preview must echo account, symbol, side, quantity, and order type. Any missing/mismatch/error blocks submission. Adapter schema is discovered from the live MCP tool list; no LLM constructs parameters.

11 - Submit outcome

- confirmed success with broker ID -> submitted;
- clean reject/error -> definitive error state;
- timeout/possible success/no broker ID -> `unknown_needs_reconcile`, budget remains reserved, retry blocked;
- every transition produces a durable event/audit record.

Email is secondary notification, never the source of truth.

12 - Fill reconciliation and exits

Order sync handles submitted, partial, filled, cancelled, rejected, and unknown states. Partial fill never triggers blind remainder resubmission and available quantity accounts for open SELL orders.

Autonomous BUY is prohibited until a deterministic live protective-exit path exists. Stops/time exits/targets use the same Gateway and verified held quantity. A protection/monitor heartbeat failure disables new autonomous entries. Tax and dividend preferences cannot delay a risk stop.

Autonomy ladder

Use only schema values from migration 124:

```
| Level | Meaning | Live placement |
| `L0_research` | research only | none |
| `L1_paper_auto` | automated paper | none |
| `L2_shadow` | shadow live recommendations | none |
| `L3_live_manual` | owner-approved live | owner action required |
| `L4_live_small_auto` | future small autonomous envelope | only after architecture phases and explicit
enablement |
| `L5_scaled_auto` | future scaled envelope | not implemented |
```

Unknown values fail closed. Documentation/UI must not use obsolete names such as `paper_only` or `live_supervised`.

Account allowlist

```
| Account | Market | Role | Allowed use |
| `605420660` | US | agentic/trading | only Robinhood account permitted for Kairos orders and order-account
sizing |
| `965848641` | US | view-only/manual | approved read-only holdings research; never order placement or
agentic sizing |
| configured Kite account | India | trading | official Kite API, INR limits, CNC delivery, separate manual
gate today |
```

Every broker/account lookup is scoped by broker, market, role, enabled state, and account ID. No silent default.

Kite verified-identity gate (`verifyKiteTradingIdentity()` in `lib/kite.ts`). The allowlist row is text - it says which account *should* be connected, not which one *is*. So before any Kite submit the gate ALSO fetches Kite `/user/profile` and requires the connected token's `user_id` to equal `strategy_config.active_account_india` (both are the same immutable Zerodha `user_id`, written together by the OAuth callback). It requires the matching `broker_accounts{broker=kite,market=india,role=trading}` allowlist row as well. Fail-closed: config read error ? 500; account unset, allowlist row absent, or `view_only` role ? 403; profile unfetchable or `user_id` mismatch ? 502/403. No silent fallback.

The check is enforced at the single `placeEquityOrder()` choke point (Codex#2/#3), which every programmatic Kite submit passes through - the standalone route (`app/api/kite/order`, which also runs the check before budget reservation so a mismatch never strands a pending row) AND the canonical/autonomous/exit paths that reach it via `kiteAdapter.submitOrder`. Gating the choke point, not just the route, closes the earlier gap where the canonical path bypassed the route-level block. Because no Kite row exists in `broker_accounts` today, all India live orders currently refuse until the owner inserts an allowlisted Kite trading account whose id matches the connected token - the intended posture until the path is unified onto the canonical `executeApprovedOrder` service.

State tables versus immutable ledgers

Do not describe every financial table as immutable; several require lifecycle updates.

Append-only / no UPDATE or DELETE:

- decision_observations;
- paper_order_events;
- strategy_evaluations;
- evidence_records (subject to its existing immutable design);
- target broker_order_events.

Mutable current-state/audited tables - never hard-delete financial history:

- paper_trades is updated when a trade closes;
- broker_orders is updated as broker lifecycle changes;
- trade_proposals changes approval/execution status;
- paper_positions represents current open state and may be removed/closed only through the transactional exit path.

Every material state transition must have an append-only event/journal record. Cleanup jobs never delete financial/audit history.

Fail behavior matrix

Failure Manual BUY Auto BUY Verified risk-reducing SELL
Auth/actor invalid block block block
Scoring version not live-approved block block allow only if independently triggered by risk exit and holding verified
Quality unknown/low block unless audited owner override block, no override do not block risk exit solely for entry-data quality
NAV/portfolio stale block or audited manual portfolio override block, no override require fresh held quantity; NAV cap exempt
Quote stale/missing block block block
Daily BUY cap full block block exempt
Broker timeout reconcile/no retry reconcile/no retry + disable new entries reconcile/no retry
Exit protection unavailable owner warned/manual decision block entry alert/escalate

Launch blockers for L4

- shared execution kernel used by all live paths;
- correct account test (605420660) and allowlist verification;
- atomic autonomous budget RPC with true actor audit;
- scoring version lifecycle enforcement;
- fresh agentic-account state with no fallback NAV;
- broker preview echo and idempotency/reconcile path;
- partial-fill/order sync and append-only broker events;
- deterministic live protective SELL path;
- duplicate-cron, timeout, stale-data, DB-failure, and kill-switch chaos tests;
- ? PA1 shadow evidence - AutonomousShadow running, execution kernel in lib/trading/execution-kernel.ts
- ? PA2 Kelly sizing - computeAutonomousSizing() in lib/trading/execution-kernel.ts; budget dry-run in shadow path; no-fallback NAV enforced (see PA2 section below)
- ? PA3 broker submit - lib/trading/autonomous-live.ts; direct Robinhood REST (lib/brokers/robinhood/rest-client.ts) + Kite REST; per-market mode (migration 141); requires

AUTONOMOUS_LIVE_ENABLED=true in Vercel env

Until all pass, AUTONOMOUS_LIVE_ENABLED remains false and L4 is descriptive only.

PA1 shadow path (implemented, deployment flag inactive)

lib/trading/execution-kernel.ts -> evaluateAutonomousExecution() is the single pure gate evaluator shared by the shadow path (PA1) and the future live path (PA2+). It takes a KernelInput + LiveAutoPolicy snapshot and returns KernelResult - no DB calls, no side effects, deterministic.

Gates evaluated in order:

```
| # | Gate | Fail label |
| 1 | `AUTONOMOUS_LIVE_ENABLED` deployment flag | `deployment_flag_inactive` |
| 2 | `live_auto_enabled` DB toggle | `db_toggle_off` |
| 3 | Lease not expired | `lease_expired` |
| 4 | Direction = long | `non_long_direction` |
| 5 | Score >= threshold | `score_below_threshold` |
| 6 | `evidence_confidence` >= floor (>= 0.6) | `confidence_below_floor` |
| 7 | Open positions < cap | `max_positions_reached` |
| 8 | Orders today < cap | `max_daily_orders_reached` |
| 9 | Notional <= per-order cap (skipped when 0) | `per_order_cap_exceeded` |
```

runAutonomousShadow() in lib/trading/autonomous-shadow.ts calls the kernel for each qualifying signal, creates a trade_proposals row with execution_mode='autonomous_shadow', and updates status to queued_auto (kernel approved) or manual_review_required (gate fired). No broker call, no budget reservation, no order submission in PA1.

PA2 shadow sizing (implemented, deployment flag inactive)

computeAutonomousSizing() in lib/trading/execution-kernel.ts computes position size for every queued_auto proposal. Rules:

```
| Condition | Outcome |
| `live_account_snapshots` row missing OR NAV <= 0 | `noSize('no_live_nav')` -> downgrade to `manual_review_required` |
| NAV age > 4h (default) | `noSize('stale_nav_Nmin')` -> downgrade |
| Quote unavailable or price <= 0 | `noSize('no_current_price')` -> downgrade |
| >= 10 closed `paper_trades` with P&L | Kelly (half-Kelly from win_rate x payoff_ratio, capped at min(10%, per-order-cap/NAV), floored at 2%) |
| < 10 closed trades | Flat `position_size_pct` from `strategy_config` |
| `floor(NAV x size_pct / price) < 1` | `noSize('qty_rounds_to_zero')` -> downgrade |
```

Per-order cap clamp: size_pct = min(size_pct, live_auto_max_per_order_usd / NAV).

queued_auto proposals that survive sizing get qty, estimated_value, pct_of_nav, and price_at_proposal populated. Budget dry-run (informational only; not the atomic reservation): reads today's broker_orders spend vs live_auto_daily_cap_usd and includes the result in ShadowRunResult.budget_dry_run. The atomic reserve_live_order_budget_v2 RPC is NOT called in the shadow path - it is called in the live-submit PA3 path.

PA3 live execution (implemented, requires AUTONOMOUS_LIVE_ENABLED=true)

lib/trading/autonomous-live.ts -> runAutonomousLive() is the live execution path. Triggered by POST /api/agents/autonomous-live/cron at 14:00 UTC weekdays (after research at 13:00 UTC).

Per-market mode (strategy_config, migration 141):

```
| `live_auto_mode_[market]` | Behavior |
| `off` | Cron skips market entirely |
| `manual` | TraderAgent creates proposals; owner clicks Approve (existing path) |
```


| `autonomous` | AutonomousLive cron submits live orders |

Additional gates (before kernel):

- app_paused=false + security_locked=false + trading_enabled=true
- live_auto_mode_[market]='autonomous' for signal's market
- Per-market view-only kill switch: a market is dropped from autonomousMarkets

when trading_enabled_[market]=false - the same per-market switch the manual gateways honor also blocks the autonomous path. Flipping a market to view-only stops auto orders for it even if its mode column is still autonomous.

- Daily-cap fail-closed: per signal, an effective daily notional ceiling is

required. US prefers live_auto_daily_cap_usd (the owner's Live-Auto \$/day guardrail), falling back to max_daily_notional_usd; India uses max_daily_notional_inr (the USD cap is not FX-converted on this path). If the effective ceiling is NULL the signal is blocked (gate_blocked, no_daily_cap_configured) rather than placed uncapped - autonomy must be bounded. The chosen ceiling is passed as p_max_daily_notional to the budget RPC.

Broker execution:

- US: rhPlaceMarketOrder() in lib/brokers/robinhood/rest-client.ts - direct Robinhood REST API using OAuth token from vault (ROBINHOOD_MCP_ACCESS_TOKEN). MCP tools unavailable in serverless.
- India: placeEquityOrder() in lib/kite.ts - existing Kite Connect REST path.

Budget reservation: reserve_live_order_budget_v2 with p_execution_actor='autonomous_worker' -> broker_orders.approved_by_user=false.

2026-07-10 audit hardening (Codex full-system audit - lib/trading/autonomous-live.ts):

- The signal query previously selected a nonexistent agent_signals.evidence_confidence column and swallowed the error -> the path silently processed zero signals every run. Fixed (real confidence column) and query errors are now fatal (throw + agent_runs error row).

- Fresh checkKillSwitches(svc, { market, book:"live", accountId }) runs per market before evaluation - the real drawdown/daily-loss/accuracy engine against live snapshots, not just cached config booleans. Fail-closed on error; a trip blocks BUY but leaves a verified SELL (sellAllowed).

- Live market-open guard (lib/trading/market-calendar.ts): layered - cheap local session/holiday/hours check, then authoritative Alpha Vantage MARKET_STATUS for BOTH US and India (one call, catches unscheduled closures, needs no yearly calendar update). Fail-closed on a confirmed CLOSED; when the status source is unreachable, falls back to the session guard + the broker-rejection and quote-freshness backstops. Static US/NSE 2026 holiday lists remain the defense/fallback layer. Autonomous cron split per market (US 15:00 UTC, India 06:00 UTC).

- Fail-closed gates: a null lease and a null per-market trading_enabled_* no longer pass; both must be explicitly valid/true.

- Per-market currency-correct NAV: US from the Robinhood USD snapshot, India from live Kite INR margins+holdings; a market with no fresh NAV source fails closed (no cross-currency sizing).

- Daily-cap fail-closed (live_auto_daily_cap_usd enforced) + net per-market open-position count.

- Idempotent claim: unique partial index trade_proposals(signal_id, market) WHERE autonomous_live (migration 145) - concurrent/repeated runs can't double-propose+buy the same signal.

- Money path UNIFIED (R13, 2026-07-10): both the manual owner gateway (app/api/broker/orders)

and the autonomous worker now call one shared server-only service, `lib/trading/execute-order.ts::executeApprovedOrder(svc, input, actor)`. It runs the full invariant set once - autonomy-level, per-market trading flags, fresh `checkKillSwitches`, G1 decision-quality, account allowlist, fresh-quote notional cap, G3 portfolio limits, price drift, held-SELL, and the atomic `reserve_live_order_budget_v2` reservation. The actor envelope distinguishes owner (may supply audited risk overrides) from `autonomous_worker` (may NOT override any gate; supplies its own `live_auto_daily_cap_usd` / orders-per-day caps). Autonomy authorization is the upstream deployment flag + DB toggle + lease + kernel + session gates, not an owner click.

- Serverless broker submit RESOLVED (2026-07-10): added a direct-REST Robinhood execution adapter

(`lib/brokers/adapters/robinhood.ts`, registry id `robinhood`) with `submit/status/cancel` over REST - works in Vercel serverless, unlike the MCP adapter. Set `strategy_config.active_broker_us='robinhood'` to route live US orders through it (both the manual gateway and the autonomous worker use it via the shared service). The account is allowlist-validated at the gateway and again in the adapter; the Robinhood live kill switch (`robinhood_mcp_enabled`) gates both the `robinhood` and `robinhood_mcp` ids.

- Remaining before first live dollar: R16 live position-monitor + protective-exit/cancel/reconcile

control plane, the J acceptance-test fixtures, shadow soak, and a capped canary. Deployment flag stays false until then.

- Schema reproducibility restored: migrations 143 (live-auto DDL + budget RPC), 144 (RLS), 145.

Outcomes per signal:

- `submitted` -> `broker_orders.status=submitted`, `broker_order_events` appended, `proposal queued_auto`
- `needs_reconcile` -> `broker_orders.status=unknown_needs_reconcile`, `proposal manual_review_required`
- `broker_error` -> order not submitted, `proposal manual_review_required`
- `budget_error` -> RPC threw (cap exceeded), no `broker_orders` row, `proposal manual_review_required`
- `gate_blocked / sizing_failed` -> no reservation, no submit

Advisory-only surfaces (read the book, never move money)

Some analytics read the live account book but are structurally severed from the order path. They must never be treated as an order signal, and they never call the Execution Gateway.

Daily Per-Holding Risk Analytics

`features/holding-risk-daily` - daily `/api/agents/holding-risk?market=us|india` (pg_cron migration 156, US 21:30 UTC / India 11:00 UTC). Scores every holding in every live account - Robinhood Trading 605420660, Robinhood read-only 965848641, and Kite India - with a deterministic 0-100 risk-control pressure index and a risk posture. Safety properties:

- Hybrid, deterministic-first. `lib/risk/holding-risk.ts` (hr-v3) computes the score and the

posture (`hold / review / trim / exit_review / insufficient_data`) with strict precedence: a verified protective-stop or thesis-break -> `exit_review`; unrealized drawdown ALONE never triggers `exit_review` (loss-chasing guard); hard name, allocated genuine equity-sector, and correlated- cluster concentration all produce review, never trim, because a global trading reference is not an account-specific sell mandate. Having an order path does not establish that mandate. A future trim posture requires an approved per-account objective/cap and executable share-quantity plan. A correlated cluster produces review until an allocator identifies which holding and quantity to reduce. A sector breach with no usable allocation -> review; data confidence < 0.5 -> review. An LLM writes only the human-readable `strategy_note` - it cannot change the score, posture, action, or allocation. `lib/risk/strategy-notes.ts` makes that provable rather than prompt-dependent: `parseStrategyNotes()` returns `Map<requestedSymbol, string>` and drops non-strings and

unrequested keys, so model output has no path to any other field. This mirrors the LLM boundary elsewhere: models may explain, never control a numeric limit, a posture, or an order.

- A sector breach is allocated to names, never applied as a blanket. A sector-cap breach is a property of the SECTOR - it carries no per-name allocation, so using it directly as a per-name verdict gives every holding in the sector the identical trim with no size (the shipped AVGO defect). `lib/risk/sector-breach.ts` (sba-v1, pure, deterministic, risk-internal - no research score, no LLM) decides which names absorb the breach and by how much: water-fill the sector's largest names down to a common level L where $\min(w_i, L) = \text{cap}$, on the NAV basis ($S - c \cdot \text{NAV}$), which is how `lib/risk/live-portfolio-gate.ts` enforces the owner's cap on the live money path. Ties need no arbitrary winner (equal weights $\rightarrow$ equal trims by construction); output order breaks on symbol ascending. Names below L are not selected: they get hold plus the reason they were not selected - never a bare verdict (CLAUDE.md "Detail Over Cryptic"). The allocator is consulted strictly BELOW the exit branch: it can never delay or suppress a risk-driven exit (the same invariant `lib/evidence/degradation-guard.ts` holds for short). Unknown-sector holdings are excluded from every sector total and reported as unknown - never bucketed into a synthetic sector, never assumed cap-compliant. Broad asset-class sleeves (Diversified/International Equity, Fixed Income, Commodities, Digital Assets) are also excluded: they are not equity sectors, so applying the sector cap to IVV/SGOV/GLD/IBIT is a category error. Design + justification: `features/risk-sector-breach-allocation/FEATURE_ARCHITECTURE.md`.

- Wired to NO order path - for ALL accounts. The `strategy_note`, `posture`, and `add_capacity` flag are advisory. `add_capacity` means "risk limits have room," not a buy signal. Nothing here reaches `executeApprovedOrder`, the gateway, or a broker. The UI labels the note "advisory" and, for read-only accounts, "advisory only - no order path." Since hr-v3, every account receives concentration review, not trim; allowlisting account 605420660 for order transport does not create an account-specific objective/cap mandate. The deterministic reason states that no trim is recommended. The `readOnlyAccount` flag controls advisory transport wording only, not concentration posture. An absent flag defaults to the read-only wording.

- Legacy snapshots are evidence, not current instructions. `lib/risk/holding-risk-history.ts` is the shared read-boundary normalizer for Risk Analytics and newsletters. It preserves the original `sourcePosture` but renders hr-v1/hr-v2 trim rows as review with the historical reason. The append-only rows are never rewritten, and `exit_review` is never downgraded.

- Research is SHOWN next to the verdict, and coupled to nothing (2026-07-17, `features/risk-research-visibility`). `GET /api/portfolio/risk-daily` now attaches a nullable research block per holding - score, direction, `scored_at`, `sessions_since`, `days_since`, `state`, `scored_as_holding` - joined from the latest `agent_signals` row per (symbol, market), never symbol alone (US and India books cannot cross-join). Invariant R1: this is a DISPLAY JOIN. No field of it is read by `computeHoldingRisk`, sba-v1, `constructPortfolio`, the execution kernel, or any gate; the score/posture/action on the same row were computed research-free and are replayed verbatim from the snapshot. That is deliberate, not an oversight: a sector is over-cap *because* research liked that sector, so letting `analyst_score` also veto the cap double-counts the same signal - the risk layer exists to be the one thing not persuaded by the score, and the owner integrates the two views. (This is why the competing `features/risk-research-integration` proposal - research *ordering* which names absorb a trim - was not pursued.) `tests/risk-research-annotation.test.ts` pins R1 both behaviorally (risk bytes identical with the block present vs absent, and across every score/direction/ staleness variant) and architecturally (the `lib/risk/*` modules are checked, comments stripped, for any *code* reference to research - the guard is itself falsification-tested, since `sector-breach.ts` mentions `analyst_score` in prose asserting the invariant and a naive grep would flag it).

- The AGE is the feature, not the score. On 2026-07-16 AVGO sat 6 days unscored while this panel told the owner to trim it, and nothing on screen said so. A 6-day-old 91 and a fresh 91 are not the same claim.

- Staleness is measured in market-local SESSIONS, displayed in DAYS. Reuses `marketSessionsSince` (`lib/trading/paper-exit-policy.ts`, built on the market-local holiday calendar in `lib/trading/market-calendar.ts`) - not a calendar-day count, which would paint the whole book stale every Monday

(a Friday score read Monday is 3 days but one session) and train the owner to ignore the warning. US and India holidays differ and are handled per market.

- Four states, not collapsed: `fresh` (`<= STALE_AFTER_SESSIONS = 2, owner-set`) ? `stale`

(warning + "not scored in N days") ? `never` (no signal ever - not a link, because a link to nothing is a lie) ? `unavailable` (research abstained on thin evidence - never rendered as a number). `stale` annotates the score only - it never dims or alters the risk action or verdict.

- A score's meaning depends on `is_holding`. `is_holding` is false in 463/463 prod rows: no

holding-path score has ever been written, so every annotated row is labelled a screener candidate score, not a holding verdict. This matters because the direction gate can only emit short (a deterministic exit) when `isHeld` is true - a neutral from the screener path does not mean "no exit signal"; it means the exit question was never asked.

- Fail-soft. If the `agent_signals` read errors, the risk table still renders and the column says

"research unavailable" explicitly - never a silent blank, never a fake score. Risk is the product; research is the annotation. A failed read (`research: null`) is kept distinct from a successful read that found nothing (`state: 'never'`), so the UI never claims a symbol was never scored when it merely failed to look.

- Every account gets the annotation, including read-only 965848641 (informational there, as the

strategy note already is). Additive only: no schema change, no migration, no new cron, no provider call, no LLM. Rows deep-link to `/dashboard/research-journal?symbol=&market=`.

- Fails closed. A missing/stale broker snapshot publishes a `failed/insufficient-data` run - never

yesterday-as-today. Structural-gate failures (missing qty/price/market-value, non-finite inputs, stale quote, non-USD/INR currency) yield `insufficient_data` with a null score, not a fabricated one.

- No cross-currency roll-up. Each run/snapshot carries its own market + currency; USD and INR

are never summed. `?-vs-yesterday` only compares runs of the same `formula_version`.

- Append-only evidence. `holding_risk_runs` (lifecycle-guarded: DELETE blocked, identity frozen,

status forward-once out of running) + `holding_risk_snapshots` / `account_risk_snapshots` (UPDATE+DELETE blocked). Owner-email SELECT RLS; service-role writes; anon REVOKEd. See [docs/arch/04-database-schema.md#812-daily-per-holding-risk-analytics-advisory-append-only](#).

Proactive broker-token health check

`checkRobinhoodTokenHealth(svc)` in `lib/robinhood-mcp.ts` is a proactive token-age check. Robinhood access tokens are short-lived (~days), so a naive expiry read would false-alarm on every routine rollover. Instead it mirrors `getValidAccessToken`: when the access token is past (or within 60s of) expiry and a refresh token exists, it attempts the CAS refresh (keeping the token warm on the 6h cadence) and reports the critical `broker-token: robinhood` issue only when there is no refresh token or the refresh actually fails - a genuine reconnect-required state, not a short-TTL rollover. On a valid or successfully-refreshed token it resolves the issue. It runs in two places:

- GET `/api/robinhood-mcp/status` (Settings -> Robinhood card) - so the connection badge cannot lie.

A genuinely-dead token renders "`? Reconnect required - token expired`" (red); a healthy token shows "`? Connected - valid until <expiry>`" (green), where `<expiry>` is the access-token TTL (renewed automatically, not a connection deadline). The route returns `stale` / `expires_at` / `has_refresh`.

- POST `/api/agents/health-triage` (`kairos-health-triage, 0 */6 * * *`) - runs the check **before**

reading `agent_alerts`, so every 6h it both keeps the token refreshed and surfaces a genuine dead-refresh state even when no order/snapshot path ran to trigger the lazy check in `getValidAccessToken`.

Downside hedge boundary

The downside hedge is a US paper-book overlay, not a new alpha strategy or live authority. Ordinary agents block SH, PSQ, DOG, and RWM. Only `execute_paper_hedge_fill` admits SH/PSQ, after dedicated flags, fresh audited evaluation, state, one-position, cash, and NAV checks. Hedge trades are excluded from learning, cash-funded, and bounded by stop, five-session hold, hysteresis, and cooldown. There is no true short, option, leveraged inverse ETF, India, LLM decision, broker call, or live control.

Why it exists: a dead RH refresh grant makes `fetchRobinhoodBrokerAccounts()` return an "unknown" account id, which silently drops all Robinhood accounts out of holding-risk and freezes `live_account_snapshots`. The lazy `getValidAccessToken` reporter only fired when something tried to use the token; this proactive 6h check + honest badge close that gap. The only human-required action (owner reconnects OAuth via the localhost loopback) is triggered solely on a failed refresh - the check is advisory + performs the same CAS refresh the order path uses, but reaches no credential-write beyond token renewal and no order path.

Kite GTT and Explicit SELL Safety (2026-07-17)

Kite GTT child orders are LIMIT, including the stop-side child. A trigger is not a guaranteed fill: a gap through the limit can leave protection unfilled. Kairos must report that semantics honestly and may not label the child SL-M.

For the standalone human-confirmed Kite route, an explicit SELL now:

- reads every recorded non-null GTT id for the India symbol;
- cancels each at Kite and requires positive confirmation;
- clears each id in `broker_orders`;
- only then submits the explicit SELL.

When a resting GTT exists, the standalone route permits only a full-quantity exit. A partial SELL would cancel protection for the unsold remainder and is therefore rejected until a reviewed residual-protection workflow exists.

Any read, cancel, or ledger-clear uncertainty fails closed before SELL. If the SELL fails after confirmed cancellation, a critical issue states that the still-held position may now lack broker-side protection. GTT placement/persistence failure after a confirmed BUY is also critical. The broader hybrid protective-stop feature remains a draft and is not enabled by this correction.

Webull Cloud MCP remains query-only and advertises no order scopes or tools. A future Webull Trading API adapter is a separate, signed, sandbox-first money-path feature and is currently unapproved and disabled.

Hybrid protective-stop control plane (2026-07-30)

The earlier shadow scaffold is now reconciled with production schema truth: `protective_orders`, `protective_order_events`, and the false-by-default `protective_orders_enabled` flag are applied. Direct production verification on 2026-07-30 found the flag false and zero protection rows. P1 adds read-only full-quantity coverage evaluation plus an autonomous-live entry interlock. A configuration flag alone cannot permit a BUY: the source-level placement worker is false until a separately reviewed broker-specific lifecycle exists. P1 neither places nor modifies a broker order, and it does not change manual live, paper, or the synthetic live-exit monitor. The older prose below is historical scaffolding; references to the migration as unapplied are superseded.

Coverage requires exact equality between reconstructed held quantity, reconciled held quantity, and broker-protected quantity. Under-protection leaves risk uncovered; over-protection can create an accidental short. Both fail closed. The aggregate coverage verdict must be explicitly true; false/unknown state cannot pass merely because a failing reader returned no per-position findings. P2 must also define an atomic or bounded-compensation workflow for protecting a newly filled BUY; changing the source availability constant is not sufficient.

Built UP TO the placement line and STOPPED. No live broker order is ever placed by this scaffold - not a \$1 test. It is a set of pure, deterministic modules under lib/protective/ plus an UNAPPLIED migration proposal. Nothing here is wired into an order path; the whole thing is inert until the owner approves touch semantics (Q1), the floor distance, and the post-fill protection policy.

Design honesty - protection is MITIGATION, not a guarantee. A stop-market is designed to TRIGGER a market order but neither execution nor price is guaranteed in a gap/halt/outage. A stop-limit / GTT limit child controls price but MAY NEVER FILL on a gap - the position stays unprotected and that is REPORTED, not called filled. Stops generally cannot fire while the session is closed. The honest label is outage + catastrophic-loss mitigation.

```
| Module | What it is | Money-path posture |
| `capabilities.ts` | Broker-neutral `BrokerProtectiveCapabilities` matrix + `evaluateProtection()` -
  capability-driven by order-type/TIF/session/account, NOT a flat broker boolean. GTC ? triggers-in-every-
  session (session eligibility is separate). No eligible multi-day order -> `unprotected-by-broker`, never
  silent. | pure, no broker calls |
| `kite-capabilities.ts` | Kite's declared capability from the PROVEN GTT code: protects via the WEAKER
  `gtt_limit` (LIMIT child); the DAY-only regular SL-M is declared and REJECTED as a multi-day floor. |
  declaration only |
| `disaster-floor.ts` | Pure `computeDisasterFloor({mode, distance, ...})`. Default `mode =
  wider_disaster_floor` (Q1 unanswered). Distance is a config input - no hardcoded value. Monotonic ratchet:
  a falling high-water mark can never lower the floor; a hard max-loss bound can only raise it. | pure, not
  wired to placement |
| `reconcile.ts` | Pure `reconcileProtectiveOrder()` - detects out-of-band triggers, partial fills,
  cancels, expiry, broker edits, and corporate-action qty drift. Unknown state -> `needs_reconcile`; trigger-
  without-fill never closes the book; limit-child gap -> unprotected; expiry -> critical; residual protection
  clamped to held qty. | pure, no broker calls |
| `state.ts` | The `protective_order` record shape, status machine (`placing`->...->`needs_reconcile`),
  exit provenance, and long-only / cancel-before-replace invariants. | pure helpers |
| `placement-gate.ts` | **THE MONEY LINE.** False-by-default `protective_orders_enabled` flag gates ALL
  placement and stays false. `planProtectivePlacement()` returns the intended action as a plain object and
  NEVER calls a broker. | inert - no placement |
```

Exit provenance (Codex's correction). A disaster-floor fill closes the position with `exit_reason = protective_disaster_floor` and `learning_scope = risk_policy_only`. The loss is REAL money and STAYS in P&L, NAV, drawdown, mandate evaluation, and risk-policy evaluation. ONLY the Learner's signal-weight attribution and genome promotion exclude it (`state.ts includedInSignalWeightLearning()` returns false; `includedInPnlAndRisk()` returns true). A generic `excluded_from_learning = true` is insufficient because the evaluation engine also filters that field - which would wrongly drop the loss from risk evaluation too. So `learning_scope` is explicit.

Long-only / cancel-before-replace. Total executable SELL (resting protective SELL + any competing SELL) can never exceed the reconciled held quantity. `canSubmitCompetingSell()` FAILS CLOSED: a competing SELL is refused while a resting protective order's cancellation is unconfirmed (else an accidental short), and even after a confirmed cancel a request above held qty is refused. This is the same invariant the shipped standalone Kite route already enforces (cancel-before-sell); a future shared implementation reuses the protocol.

Migration is a PROPOSAL, not applied.

`supabase/migrations/20260718000000_protective_orders_shadow.sql` creates `protective_orders` (state table, one active per position+broker via a partial unique index, a `protected_qty <= reconciled_held_qty` CHECK) and append-only `protective_order_events`, and adds `strategy_config.protective_orders_enabled` (default false) plus `learning_scope` on `paper_trades` / `broker_orders`. It is NOT applied to prod - see `docs/arch/04-database-schema.md` is intentionally not updated until the migration actually runs.

Acceptance tests. `tests/protective-hybrid-stop.test.ts` implements all 14 spec acceptance tests, each falsifiable (mutation-verified: breaking the ratchet fails AT3; breaking the cancel-guard fails AT1). Prod verified before building: no `protective_orders` table exists; `broker_orders` already has `kite_gtt_id`; no `learning_scope` column exists; `paper_trades` exit reasons in use are `direction_flip` / `score_exit` (no `protective_disaster_floor`).

What remains for the owner before anything goes live: approve Q1 (broker-hosted touch execution at all), the disaster-floor distance (recommendation: mandate-ATR with a hard max-loss bound), the minimum ratchet step/cadence, and the build sequence (Kite-first vs Webull-first). Then apply the migration, wire the plan to a reviewed live path, flip

protective_orders_enabled to true, and run a single owner-approved small live test per market.

Order-maintenance and ETF-cap correction (2026-07-26): A stale US-order cancel ACK is not a cancellation record. It becomes unknown_needs_reconcile; only a later read-only broker observation may set terminal state. The maintenance cron skips closed US sessions, bounds outbound cancellation and reconciliation to one item each, and the Trading dashboard offers an owner-triggered read-only refresh. A confirmed fill writes filled_qty, avg_fill_price, terminal time, and its proposal result without overwriting requested quantity. ETF allocation now fails closed on unavailable configuration, stale or unpriceable current marks, stale live snapshots, invalid NAV, or missing order price; it uses current marks, not cost basis. India remains outside the US ETF sleeve and SELLS bypass it.

Earnings Event Risk (P0 Shadow)

Earnings proximity and the expiry-bounded ATM straddle proxy are risk annotations, never directional alpha. Exact-expiry selection requires a common call/put strike, valid non-crossed mids, liquidity, bounded spreads, and fresh timestamps. Unknown, conflict, stale, illiquid, or incomplete pagination remains unavailable and cannot be interpreted as no event.

P0 is mechanically behavior-inert: the database permits only shadow rows and requires behavior_changed=false; paper/live hooks only log normalized context; the direct live Alpha Vantage blackout remains in force; PositionMonitor and live execution import no earnings-risk decision function. The Risk page exposes warnings and evidence counts without raw broker chains.

The scheduled US holdings monitor is also fail-loud on canonical state reads: errors reading holding-risk runs/snapshots, paper positions, or the PIT earnings calendar mark the agent run error instead of reporting a successful zero-coverage sample. A paper stop at or above spot is stale/malformed and is recorded with no stop-distance comparison; it is never converted with an absolute value into a plausible risk annotation.

Post-Report Daily-Price Repricing Barrier (2026-07-30)

Options risk context remains measure-only, but daily technical direction cannot be trusted immediately after a reported result. When the point-in-time earnings calendar has a recent event that has occurred but the daily candle series does not yet contain its reaction, ResearchAgent writes a current, session-validated neutral signal. Before-open events may use the completed report-date bar; after-close/unknown events require a later bar. A late actual feed does not reopen stale scoring for a known past event. This is a subtractive data-freshness guard: it blocks new paper/live entries and score/direction exits from that stale daily score, while mechanical stop, target, trailing-stop, and time exits continue. The existing calendar is read only; no provider or options call is added, and no earnings beat/miss is inferred as directional alpha. A calendar read error is an unknown control-plane state and writes a current neutral abstention rather than authorizing stale direction. The latest prior event remains checked until a qualifying post-event bar exists; it does not silently expire after seven calendar days. All earnings collectors share one real ISO-date validator, so impossible provider dates cannot normalize into another session. Normal scoring resumes on the first post-report daily bar.

09-learning-loop

Source: docs\arch\09-learning-loop.md | Authoritative operational architecture

Kairos - Learning Loop

2026-08-01 weight-resolution correction: live ResearchAgent scoring reads

the market champion, then the static risk-profile baseline. learning_priors

and global signal_weights are learner configuration/proposal inputs, not

hidden live-score fallbacks; challengers remain inert until validation and

explicit promotion.

2026-07-28 correction - PROMOTION REMAINS DORMANT, NOT PERMANENTLY CLOSED. The h5 study measured pooled IC sigma ~ 0.27 and found no useful mom_12_1 signal at h5 (mean IC 0.0089, t_HAC 0.32). It did not identify an effective breadth of 17: observed IC variance mixes sampling noise, changing point-in-time membership/coverage, and genuine time variation in factor returns. Inverting it with $1/\sqrt{n-3}$ cannot separate those causes, and an h5 estimate cannot set h20 requirements. The n=400 and sector-neutral tests therefore remain legitimate measure-only experiments rather than rejected escape routes. POST /api/agents/backtest/promote still fails closed; no policy can consume these diagnostics.

2026-07-28 US PIT step-4 hardening: us_pit_adv20_top400_v2 ranks membership on a complete 20-session trailing dollar-volume window, shares cached session reads across overlapping as-of dates, excludes partial-window names, and persists one top-400 superset so matched n=200/n=400 tests use the same ranking. Report schema v2 retains successful-date cross-section and complete universe provenance. persist_edge_pit_snapshot() atomically writes exact snapshots to append-only edge_universe_members; it is service-role-only and conflicts fail closed. India PIT membership remains unavailable. These changes improve measure-only evidence and do not enable promotion.

PROMOTION IS DORMANT (2026-07-27). POST /api/agents/backtest/promote fails closed with promotion_evidence_not_oos (503) before any write. Adversarial review found one P0 and three P1 issues that each independently disqualify the current path: promotion is non-atomic (supersede-then-insert can leave a segment with no active policy); the evidence is not out-of-sample ($\sim 98.4\%$ window overlap AND a current-liquid universe replayed through past dates - survivorship bias that more weekly runs cannot fix); dsr_z was not a Deflated Sharpe Ratio and is renamed t_margin_vs_trials, with the dsr column now written NULL; and experiment lineage is optional and unbound to the edge/market/horizon/segment it justifies. Re-enable only after features/walk-forward-ic-folds/FEATURE_ARCHITECTURE.md is approved and shipped: frozen experiment lineage -> PIT universe/inputs -> purged market-session OOS folds -> aggregate HAC IC -> multiple-testing + cost-adjusted validation -> atomic promotion RPC.

Last updated: 2026-07-27. The deterministic promotion route is implemented

but is governance scaffolding only: production has zero policies, its rolling

IC windows are not OOS, its current-universe history is not PIT, its

trial-adjusted t margin is not DSR, and supersede/insert is not atomic. The

revised features/walk-forward-ic-folds/FEATURE_ARCHITECTURE.md is a blocking

prerequisite before policy promotion or consumption.

Update this file when: the learning flow changes, new guardrails are added to weight mutation, genome parameters change, Phase 1 unlocks, the RAG pipeline changes, or Performance Truth Layer evaluation logic changes.

The big picture

```
flowchart LR
    LEARNER[LearnerAgent\nproposes] --> CHALLENGER[Challenger\na tweaked strategy]
    CHALLENGER --> SHADOW[Shadow test\nscore-only replay, no money]
    CHALLENGER --> VALIDATE[Validation Engine\nreplay on held-out folds]
    VALIDATE -- passes gates --> PROMOTE{You promote?}
    PROMOTE -- yes --> CHAMPION[Champion\nthe live strategy]
    PROMOTE -- no --> ARCHIVE[stays a proposal]
    CHAMPION --> RESEARCH[ResearchAgent uses it]
    RESEARCH --> OUTCOMES[new closed trades] --> LEARNER
```

The loop is:

- ResearchAgent scores stocks using the current Champion strategy's weights + genome.
- PaperTrader acts on high-score signals. PositionMonitor watches and exits positions.
- Closed trades become training data for LearnerAgent.
- LearnerAgent proposes a Challenger strategy with adjusted weights + genome.
- The Validation Engine replays the Challenger on held-out data.
- If the Challenger passes, you promote it to Champion. ResearchAgent uses the new weights

on its very next run.

Plan calibration and the LLM boundary (2026-07-30)

Every new ResearchAgent decision stores an indicative stop/objective/horizon in `decision_observations.features.trade_plan`. Nightly label maturation already stores immutable 2/5/10/20-session forward return, benchmark-neutral return, MFE, and MAE in `observation_labels`.

`lib/learning/plan-calibration.ts` joins those two truths only when the label horizon exactly matches the original plan horizon. It reports objective reach, stop breach, and realized path statistics. The objective is an exit-policy level, not a predicted terminal price; MAE/MFE cannot establish which level was touched first.

Decision Review shows per-symbol paths as illustrative. LearnerAgent receives only same-market aggregate summaries through `query_plan_calibration`. The LLM may explain the cohort and write a hypothesis, but it has no tool that changes stop, target, or horizon. The existing deterministic MAE/MFE policy remains the only automatic risk-parameter adjustment path and requires at least 60 same-market, exact-horizon eligible-long labels. US and India never share a cohort.

Benchmark-alpha scorecard boundary (2026-07-13)

`/api/agents/benchmark-scorecard` writes Phase-1 measurement rows to `benchmark_scorecard` for paper/live, US/India, and 1W/1M/3M/YTD/1Y horizons. The math uses common-window rebasing and annualized daily information ratio. These rows are displayed in Performance Truth and may be read as evidence, but they do not change LearnerAgent objectives, promotion gates, posture, sizing, paper fills, or live orders in Phase 1. Phase 2 learner/promotion wiring still requires a separate owner-approved implementation.

Champion and Challenger

strategy_versions table (governance)

One row per strategy version. At most one `is_champion = true` per market at any time.

```
| Field | Meaning |
| `market` | `us` or `india` - markets evolve independently |
| `is_champion` | True for the one active strategy per market |
```

```
| `weights_snapshot` | 5-dim weights that ResearchAgent reads |
| `genome` | Trading parameters (see below) |
| `proposed_by` | `learner` or `user` |
| `backtest_result` | Sharpe, Sortino, win_rate, max_dd from Validation Engine |
| `promoted_at` | Set when you click "Promote to Champion" |
```

The genome

What a Challenger can evolve:

```
| Parameter | Range / options | Notes |
| `entry_threshold` | 50-90 | `analyst_score` cutoff to open a position |
| `exit_stop_pct` | 3-15% | Stop-loss percentage below entry |
| `exit_target_pct` | 10-40% | Price target percentage above entry |
| `horizon_days` | 3-30 | Time-stop maximum hold days |
| `position_size_pct` | Up to `strategy_config.position_size_pct` | Can only size DOWN, never above the
owner-set cap |
| `sizing_mode` | `fixed` \| `kelly` \| `confidence_scaled` | How position size is calculated |
| `entry.rank_pct_min` | 0.0-0.95 | **Default 0.0 = OFF (feature no-op).** Minimum within-comparable-group
percentile for a NEW long. Hybrid gate: entry requires `analyst_score >= score_threshold` **AND** `rank_pct
>= rank_pct_min`. `0.0` reproduces current selection byte-for-byte. Rank gates cross-group *admission* only;
intra-group ordering stays by `analyst_score` (rank is monotonic in score within a group), so the CLAUDE.
md "top-3 by analyst_score win" rule is preserved. Becomes active only via a validated, owner-promoted
challenger. |
| 5 dimension weights | Sum must = 1.0 | `{fundamental, technical, sentiment, macro, insider}` |
```

Per-market independence

strategy_versions has a market column. LearnerAgent analyzes one market's cohort per run and proposes challengers only for that market. A bad India run cannot shift US scoring weights. India starts on a clone of the US champion as a prior and diverges once it clears the same 10+ closed-trade phase gate.

LearnerAgent details

File: app/api/agents/learner/route.ts, app/api/agents/learner-brain/route.ts Schedule: Fridays 5:00 PM ET LLM: Claude Opus 4.8 (claude-opus)

Phase gate

Weight mutation is blocked entirely until 10+ closed trades per market exist (Phase 0). LearnerAgent runs but only writes a "mutation blocked: insufficient trades" note to learning_log. Phase 1 (mutation unlocked) requires the owner to verify the trade set and acknowledge the gate has been cleared.

Run-accounting convention: A LearnerAgent run may reconcile dangling orphan trade rows as maintenance. Its run summary reports that count separately from the market-local total closed-trade corpus (Reconciled ... | Total closed: ...); zero reconciliations never means zero learning data.

OPEN ITEM (2026-07-16): India macro contamination in the historical trade set - NOT yet remediated

Until 2026-07-16, India signals were scored with the US macro regime (macro_regime has no market column) and, on 2026-07-13, with a fortnight-stale green verdict that was really a failed MacroSentinel run (raw_indicators = []). lib/data/scores.ts now excludes macro for India and age-bounds the row - but the historical rows already written are still contaminated.

Measured in prod on 2026-07-16 (entry signal joined to signal_score_history):

```
| India paper_trades | Count |
| Closed, entered on a macro-contaminated score | **4** (2 via stale-green `macro_score=100`, 2 via US-
orange `macro_score=60`) |
```

```
| Closed, clean (macro already excluded) | 3 |  
| Open, entered on a macro-contaminated score | **13** (3 stale-green, 10 US-orange) |  
| Open, clean | 2 |
```

Why this is not yet urgent: the mutation gate is 10+ closed trades per market; India has 7 closed. The learner is still Phase 0 for India, so no contaminated trade has driven a weight mutation yet. The 13 contaminated open positions will close into this set, though - so the record should be corrected *before* India reaches 10.

Proposal (owner decision - deliberately NOT applied, no rows were mutated): flag rather than delete. The taint vehicle already exists (`paper_trades.tainted`, `taint_reason`, `excluded_from_learning`; filter in `lib/learning/taint-filter.ts`), so no migration is needed - set `tainted = true`, `taint_reason = 'macro_regime_market_leak'` on the affected rows. `applyLearningTaintFilter` then drops them from learning and from the RAG memory corpus, while `run-evaluation.ts` still counts them in P&L (the book really moved - see "Tainted trades" under Performance Truth). Note the flag is coarse: it removes the whole trade from learning, not just its macro dimension. Given India's small *n*, the honest alternative - accept a smaller-but-clean India trade set - is preferable to learning from a US-Fed-scored Indian book.

Tool-use loop (9 tools)

LearnerAgent runs a multi-step tool-calling loop (via `runAgentLoop()`). The 9 tools it has:

- `get_closed_trades` - recent `paper_trades` with outcomes
- `get_signal_weights` - current champion weights
- `get_strategy_versions` - all challengers + their backtest results
- `get_decision_observations` - scored decisions (including skipped)
- `query_trade_decisions` - real historical enriched Robinhood trades by regime/action
- `propose_challenger` - write a new `strategy_versions` row with new weights + genome
- `run_validation` - trigger Validation Engine on the proposed challenger
- `get_mentor_insights` - recent coaching notes from MentorAgent
- `semantic_search_decisions` - pgvector RAG over trade memories (if Jina key present)

Automatic validation & shadow routing (migration 170)

When LearnerAgent creates a challenger it runs `runAutomatedValidation()` (`lib/validation/automation.ts`) in-process - this replaced the earlier fire-and-forget localhost request, which was unreliable on cloud cron. The flow, gated by the per-market `strategy_validation_automation` policy (fail-closed: missing row / read error = fully disabled):

- Policy enabled=`false` -> return immediately, no validation (challenger still

exists and can be validated manually).

- Else run the deterministic Validation Engine (`validateChallenger`), which

writes a `validation_experiments` row (pass or fail) exactly as the on-demand Validate button does.

- If it passed AND `auto_shadow_enabled=true` -> call the

`activate_strategy_shadow(p_version_id)` RPC, which atomically routes the challenger to `state='shadow_paper'` under a per-market advisory lock and a `max_active_shadows` (0-1) capacity cap.

The only automatic lifecycle transition is -> `shadow_paper` (non-executing: records what it *would* decide, no fills/cash - see the Shadow section). It can never promote a champion, create a paper fill, move cash, make a broker proposal, or place a live order. Champion promotion stays the separate owner-only path and remains fail-closed on a PASSED `validation_experiments` row.

A Friday cloud recovery cron - kairo-validation-sweep, 21:45 UTC (POST /api/validation/sweep) - validates up to 5 never-validated challengers per market (state='challenger', validation_experiment_id IS NULL) through the same path, catching challengers created outside LearnerAgent or interrupted before in-process validation completed. Owner controls both switches per market in Settings -> Automatic Strategy Validation (GET/PATCH /api/settings/validation-automation); disabling preserves every challenger, experiment, and shadow. Feature spec: features/automated-strategy-validation/.

Auto-guard

In addition to the phase gate, LearnerAgent blocks mutation if the last 3 runs have win_rate < 35%. This prevents a losing streak from producing an overfit Challenger.

Per-trade notes

After each trade closes, LearnerAgent writes a 1-sentence outcome summary to learning_log. This is separate from weight mutation - it runs regardless of the phase gate.

Validation Engine

File: lib/validators/backtest.ts, app/api/agents/backtest/route.ts

Deterministic, no LLM. Replays a Challenger vs the current Champion on walk-forward held-out slices of the decision_observations ledger.

ATR exit-policy evidence (measure-only)

Nightly label maturation also records point-in-time entry ATR, ATR-normalized MAE/MFE, and three predeclared close_observed_v1 exit-policy outcomes. The simulation uses completed daily closes and the standard 10 bps haircut, so it does not infer intraday fills from daily OHLC ranges. The authenticated /api/analytics/atr-exit-evidence report is strictly market- and horizon-local. Its reviewable_evidence status requires at least 60 labels and 12 effective observations, but is not a promotion gate and cannot change PaperTrader, PositionMonitor, live orders, or broker protection. Purged walk-forward and paper shadow approval remain mandatory before any execution-policy proposal.

Eligibility gates (same for US and India)

```
| Gate | Threshold |
| Sharpe | >= 0.5 |
| Win rate | >= 40% |
```

If both gates pass, eligibility_passed = true is set on the experiment_runs row.

Promotion is blocked (HTTP 412) unless eligibility_passed = true. The owner cannot click "Promote" without the Validation Engine having run and passed.

Metrics computed

- Sharpe ratio
- Sortino ratio
- Maximum drawdown
- Win rate
- Expectancy

- Alpha vs benchmark

Walk-forward design

The Validation Engine splits historical data by time, scoring the Challenger only on data it could not have seen when it was proposed. This prevents in-sample overfitting from looking like genuine improvement.

Calibration OOS acceptance gate

`lib/validation/calibration.ts` fits the `pwin_logistic` artifact from walk-forward folds (chronological, purged/embargoed). `acceptCalibrationOOS` computes the Expected Calibration Error (ECE) over the out-of-sample holdout only and is fail-closed: a model is rejected (`accepted: false`) when the holdout has <30 usable rows, is degenerate (all-win/all-loss or non-finite ECE), or $ECE > 0.1$. `fitAndStoreCalibration` gates the artifact upsert on this verdict - a miscalibrated model is never written to the live `pwin_logistic` artifact, so the money path never reads a bad `P(win)`. `MIN_OOS_SAMPLES=30` and `MAX_OOS_ECE=0.1` are v0 thresholds needing prospective tuning.

Shadow decisions

A Challenger can be set to "shadow" real runs: it records what it *would have done* on every stock with no fills and no cash. This is a free dress rehearsal - the Challenger accrues a simulated track record before any promotion decision. Off by default. Activated per Challenger in the Strategy Registry.

Feature Registry

`LearnerAgent` can also propose a new formula idea - a new factor to incorporate into scoring. This is written as a human-readable spec with a falsification test. The formula is never run as arbitrary code - it is only interpreted through a locked, whitelisted math grammar. AI cannot write executable scoring code directly.

Cross-sectional rank (measure-only -> gate, OFF by default)

Files: `lib/scoring/rank.ts` (`computeComparableRank`, `isRankRejected`),
`app/api/agents/research/cron/route.ts` (Pass 2), `lib/validation/genome.ts` (`entry.rank_pct_min`).
Spec: `features/cross-sectional-rank/FEATURE_ARCHITECTURE.md`.

A deterministic (no-LLM) second pass in the research cron, after all symbols score and before `PaperTrader`. It partitions the eligible pool into comparable groups (market x asset-type x sector), computes each name's within-group empirical percentile (`rank_pct`), and - only when the champion genome's `entry.rank_pct_min > 0` - flips rank-losing NEW candidates in `agent_signals` to `status='rank_rejected'`. Data-quality gates run before ranking (thin evidence, abstain, evidence-confidence floor, held-position exclusion); groups below their min-sample threshold (US equity 20, India equity 15, ETF 20) fall back to a fixed degraded transform - the "three finalists are not a universe" guard. ETFs are never grouped with single-name equities. Provenance is written to `universe_snapshot_scores` (`rank_quality`, `comparable_group_key`, `group_n`, `rank_eligible`).

Resolved (2026-07-11): the CLAUDE.md "top-3 by analyst_score win" rule is preserved - rank gates *cross-group admission* only; `rank_pct` is monotonic in `analyst_score` within a group so intra-group ordering is unchanged, and under today's single-group degraded case rank and raw-score selection are identical. Ships OFF (`rank_pct_min` default 0.0 -> byte-stable selection); actionable only through a validated, owner-promoted challenger. Rank is not fed into the live `P(win)/sizing` model - logged as a feature for IC measurement only.

PIT fundamentals ledger (OFF by default)

Files: `lib/data/pit-fundamentals.ts` (`getFundamentalsAsOf`, `captureFundamentalsFact`), capture hook in `lib/research-agent.ts`. Table: `fundamental_facts` (migration 150). Spec: `features/pit-fundamentals/FEATURE_ARCHITECTURE.md`.

Restatement-safe append-only vintage archive: fundamentals are captured on fetch (fire-and-forget, fail-open, dedup by `payload_hash`) so "fundamentals as known on date D" is reconstructable and a later restatement can never retroactively change a past as-of read. Not yet wired into live scoring - `scoreFundamentals` is unchanged and default scores are byte-identical. Purpose: give the Validation Engine and any future walk-forward replay a leak-free fundamentals source.

Historical replay harness (measure-only, OFF)

Files: `lib/replay/*` (sealed accessor + `FutureDataLeakError`, packet assembler, gates, reporter). Tables: `replay_packets`, `replay_packet_items`, `replay_eligibility_runs`, `replay_eligibility_events` (migration 149). Spec: `features/historical-replay-harness/FEATURE_ARCHITECTURE.md`.

An offline harness that freezes point-in-time input packets (`knowable_at <= as_of` enforced by a sealed data accessor that throws on any post-cursor datum) and replays the eligibility gates (`calibration_oos`, `thin_evidence`, `ic`, `validation`, `breakdown_veto`) to answer "on what date would this strategy first have been eligible?" (`first_eligible_asof = MIN(as_of) WHERE passed`). Reuses the live gate code (`fitCalibration -> walkForwardFolds -> acceptCalibrationOOS`, `computeWeightedAnalystScore`, `isThinEvidence`) unchanged so a replay grades on the identical rule that would run live. Runs on in-memory fixtures today; the migration-149 tables persist runs when wired.

Local bulk-evidence worker (measure-only)

Spec: `features/local-historical-replay/FEATURE_ARCHITECTURE.md`.

Large official datasets remain outside Supabase under the local Kairos evidence store. The worker verifies every manifest/file hash, predeclares the plan in the existing immutable `backtest_experiments` ledger, runs without provider access, and writes only compact results and fingerprints. Initial scope is an India NSE price-only OOS IC diagnostic. Backtest reads those results through an owner-only API; the browser cannot start the worker. These records are not promotion, scoring, or trading inputs.

Performance Truth Layer

File: `lib/evaluation/run-evaluation.ts`, `/api/agents/evaluation/*` Panel: `/dashboard/learning`

Mandate-aware, deterministic (no LLM), honesty-first evaluation.

Investment mandates

Named strategy contexts. Default mandates:

- "Swing US 2-20d" (benchmark: VOO)
- "Swing India 2-20d" (benchmark: ^NSEI)

Every `agent_signals`, `paper_trades`, and `decision_observations` row gets a `mandate_id` stamped at creation time.

Future `decision_observations` also carry the actual scoring-candle close and a deterministic indicative trade-plan snapshot. This improves point-in-time label truth and lets Research Journal explain approximate entry/risk/target context. Execution does not consume stale absolute research prices: `PaperTrader` resolves the current bounded policy and anchors prices to the fill, while `PositionMonitor` continues to own the stored position's exits.

Evaluation metrics

```
| Metric | Notes |
| Sharpe | Risk-adjusted return |
| Sortino | Downside deviation only |
| Max drawdown | Worst peak-to-trough |
| Win rate | % of trades that closed positive |
| Expectancy | Average expected profit per trade |
| Profit factor | Gross profit / gross loss |
| Alpha | vs benchmark (V00/ANSEI) |
| Execution slip | Mean realized vs 0.05% modeled slippage |
```

Honesty rules

- Fewer than 20 trades -> shows `insufficient_sample` instead of a number. No fabricated precision on tiny samples.
- Tainted trades (low `data_confidence`) are counted here - the book moved, so P&L must not hide them. They are labeled as tainted but included in the totals.
- `health_label` summarizes the overall picture:
- `insufficient_sample` -> not enough data to say anything
- `negative_or_zero_edge` -> strategy has no positive edge
- `promising_but_unvalidated` -> positive metrics but not yet through Validation Engine
- `validation_required` -> ready for formal promotion decision

P1 gate

A weekly Vercel cron counts closed evaluable trades per market. When ≥ 20 accumulate, it fires a System Health info alert: `p1-gate-ready:<market>`. This is the signal to build opportunity-level IC metrics (`decision_observations x observation_labels`).

P0 is book-truth only. The `opp_*` columns in `strategy_evaluations` are null until P1.

RAG trade memory pipeline

```
flowchart LR
    CLOSE[Trade closes\nPositionMonitor] --> INDEX[indexClosedTrade:\nwrite setup as text\nembed with Jina
    1024-dim\nstore in trade_memories]
    INDEX --> STORE[(pgvector\ncosine similarity)]
    NEW[New candidate\nResearchAgent] --> RETR[retrieveSimilarTrades:\nembed live setup\nmatch nearest\
    nrerank top-5 with jina-reranker-v2]
    STORE --> RETR
    RETR --> NOTE[prior similar setups\n3/5 were wins]
    NOTE --> THESIS[injected into thesis prompt\nLLM sees its own track record]
```

Write side - `indexClosedTrade()`

- Triggered on every trade close (PositionMonitor + LearnerAgent exits)
- Builds a short text: symbol, market, 5 dimension scores, outcome, exit reason, mandate
- Embeds via Jina AI `jina-embeddings-v3` (1024-dim)
- Stores in `trade_memories` (pgvector table in Supabase)
- Tainted / excluded trades are skipped - bad-data history cannot poison memory

Read side - retrieveSimilarTrades()

- Called by ResearchAgent before scoring each candidate
- Fingerprints the live setup as text, embeds with Jina
- Queries pgvector nearest-neighbor (cosine, IVFFlat index) for top-K candidates
- Reranks with Jina `jina-reranker-v2-base-multilingual` to pick genuinely similar past setups
- Returns a one-line summary: `"prior similar setups: 3/5 were wins"`
- Writes a `rag_traces` row for audit

Guardrails

- Ticker filter: a retrieved chunk that doesn't mention the candidate symbol is dropped
- Whole path is off when `JINA_API_KEY` is absent - no key -> silent no-op
- Does NOT move money or change weights. Advisory context only.

Signal weights

Current weights (live scoring)

ResearchAgent reads the newest promoted `strategy_versions.weights_snapshot` for the symbol's market. If no promoted champion exists, it uses the selected static risk-profile baseline (conservative, balanced, or aggressive), then applies the market mandate's strategy tilt. Missing dimensions are removed and the remaining weights are renormalized. In plain language: only an explicitly promoted market strategy, or the documented profile baseline, can set the live score mix.

`learning_priors` and the global `signal_weights` row are not live-scoring fallbacks. `learning_priors` controls which dimensions LearnerAgent may study and mutate. `signal_weights` remains a legacy/display row and is used only as a proposal baseline when LearnerAgent has no market champion; a proposal still creates an inactive challenger and cannot affect scoring until validation and promotion succeed.

Weight change audit

Legacy prior/global-weight changes are logged to `learning_priors_history` and `signal_weights_history`. The active strategy audit trail is the immutable `strategy_versions` challenger/champion lifecycle. These records are not purged by the cleanup job.

Learner config

`learner_config` table controls per-dimension mutation:

- `learn_from = false` -> exclude this dimension from LearnerAgent analysis
- `allow_mutation = false` -> LearnerAgent cannot propose weight changes for this dimension

These are toggled via `/api/agents/learner-controls`.

Promotion gate (deterministic)

The path from "this edge measures well" to "this edge is policy" runs through `lib/gates/promotion-gate.ts` and `POST /api/agents/backtest/promote`. It is the only writer to `strategy_policies`, and there is no LLM anywhere on it - the LLM's role ends at proposing a bounded hypothesis into `backtest_experiments`.

`strategy_policies.promoted_by` is CHECK-constrained to `'deterministic_gate'`, so that boundary is enforced by the database rather than by convention.

<!-- Superseded 2026-07-27 promotion-gate description retained in git history only.

Inputs

- `edge_ic_history` rows for one `edge_id` + market, scoped to the requested

segment (sector -> `segment_type='sector'`, regime -> `segment_type='regime'`, neither -> market-wide rows where `segment_type` is null) and to the horizon band [`horizon_days_min`, `horizon_days_max`].

- `backtest_experiments.variants_run` when an `experiment_id` is supplied. Absent

one, the gate assumes a single trial - the least punitive assumption, so the other gates have to carry the decision.

The four gates

```
| Gate | Rule | Why |
| Window count | >= 3 IC windows | Below this there is no walk-forward to speak of; returns
`insufficient_windows` without evaluating anything else. |
| IC floor | latest window IC >= 0.02 | Matches `classifyEdgeIC` in `lib/edges/ic.ts`. A decayed edge does
not get promoted on its history. |
| t-stat | **latest window's** Newey-West t >= 2.0 | Newey-West because overlapping forward-return windows
make naive IID t-stats overstate significance. Latest - not max - see "Which t-stat" below. |
| DSR | `t_best - E[max t over S trials] > 0` | Bailey 2014: `E[max t] ? ???(1 - 1/(2S))`. Testing more
variants must not buy significance - this is why `variant_budget` is locked before the engine runs. |
| IC stability | latest IC > 0 and >= 50% of earliest IC | An edge whose estimate halves as data is
appended is not stable. **This is NOT walk-forward** - the windows overlap ~98%, see below. |
```

Note the interaction worth knowing: `variant_budget` is capped at 20 by the schema, and $E[\max t]$ at 20 trials is ≈ 1.96 - just under the 2.0 t-hurdle. So within the allowed budget range the DSR gate never binds on its own; it tightens the margin rather than rejecting outright. The rejection case is covered by test and only triggers above the schema ceiling.

Outcomes

- Pass - supersedes the incumbent active policy for that segment (the partial

unique index allows exactly one non-superseded row per segment), then appends a new row. `verdict` is baseline when the segment had no incumbent and variant when one was superseded. `model_id` is the latest window's `formula_version`, falling back to `run_fingerprint`.

- Fail - HTTP 200 with `promoted: false` and machine-readable reason codes.

A rejected promotion is a normal outcome, not an error, and nothing is written.

- When `experiment_id` is supplied, the resulting `policy_id` and `completed_at`

are written back to `backtest_experiments` to close the lineage loop.

Auth is cron secret or an authenticated user; both land on the identical deterministic path, so there is no privileged variant. Tests live in `lib/gates/promotion-gate.test.ts`.

Which t-stat (measured against prod, 2026-07-27)

Three candidate statistics were computed on real edge_ic_history rows. Two are unusable and the choice is not cosmetic - it decides every promotion:

```
| Statistic | Result on prod data | Verdict |
| `max(t_stat)` across windows | `dma_trend_slope@20d` = **2.83** | Rejected - cherry-picks the luckiest
of N windows. Same edge's latest window reads **0.55**. This is the exact order-statistic bias the DSR gate
exists to remove, so using it made the gate self-defeating. |
| `mean(IC) / SE(IC)` pooled | An India edge with 3 windows scored **t = 13.77** | Rejected - pooling
assumes independent windows. These are ROLLING and overlapping, so the IC series is autocorrelated and the
SE collapses. |
| **latest window's `t_stat`** | best US = **1.73**, best India = **1.04** | **Adopted.** Unbiased, no
cherry-pick, uses one window of data. Underpowered, which fails closed. |
```

Correctly pooling overlapping windows (a Newey-West correction *across* windows, not within one) is the real fix and is deferred until there is enough non-overlapping history to justify the machinery.

First real run - zero promotions, and that is the correct answer

Evaluated against every market-wide edge/horizon pair with ≥ 3 windows:

```
| Market | Pairs evaluated | Pass IC floor | Pass t >= 2.0 | Pass walk-forward | **Promotable** | Best
latest t |
| US | 33 | 18 | **0** | 15 | **0** | 1.73 |
| India | 24 | 1 | **0** | 3 | **0** | 1.04 |
```

Nothing promotes in either market. The binding constraint is the t-stat hurdle, and the cause is history length: US IC history starts 2026-07-08, so 6 "windows" sit on ~3 weeks of heavily overlapping data. strategy_policies is expected to stay empty until there is materially more non-overlapping history. An empty policy table is the gate working, not the gate broken.

The windows are NOT walk-forward folds

Measured in prod 2026-07-27. Every edge_ic_history row is an IC over history_days = 1000, and the six US windows span 16 calendar days end to end - so the first and last windows share ~98.4% of their data. They are the same backtest re-run weekly with the end date nudged, not out-of-sample folds.

Consequences, and what was done:

- The cross-window check is renamed ic_stability_pass (failure code ic_stability_failed). It measures whether the IC estimate holds as data is appended - not out-of-sample decay. The gate no longer claims a property it does not have.

- strategy_policies.walk_forward_pass (the DB column) keeps its name; renaming it needs a migration on an append-only governance table. It stores stability.

- The route now dedupes by window_end, newest run wins. US edges had 6 rows

across only 4 distinct window_end values from 6 distinct run_fingerprints, with universe_size drifting 31 -> 32 -> 40. Undeduped, a same-day re-run reads as fresh evidence and can lift sample_n over MIN_WINDOWS by itself.

- A span-based guard was considered and rejected: with 1000-day windows,

genuinely disjoint history needs ~8 years. Waiting does not help either - four more weeks moves overlap from 98.4% to ~95.6%.

What is *not* affected: the Newey-West t_stat within a single window (~96 as-of dates). That is real evidence and remains the binding constraint.

The real fix is disjoint folds emitted by the IC engine, proposed in

features/walk-forward-ic-folds/FEATURE_ARCHITECTURE.md - draft, not approved, not implemented. Its own

risk section notes it would likely make promotion *harder* (fewer as-of dates per fold), so the recommendation there is to build it once an edge is close to clearing the hurdle, not before. -->

Current implementation and audit status

- The route reads one exact evidence horizon. Market-wide evidence is `segment_type='market', segment_value='all'`; sector evidence is explicitly sector-scoped. The production `edge_ic_history` constraint does not permit regime rows, so regime promotion has no evidence and fails closed.

- Duplicate `window_end` rows are collapsed newest-run-wins before evaluation.

- Interactive access is confirmed-owner-only; cron access requires the cron

secret. There is no LLM on the path.

- Current gates are at least three distinct rolling windows, latest IC ≥ 0.02 ,

latest-window Newey-West $t \geq 2.0$, a trial-count-adjusted t margin above zero, and positive/non-halving endpoint IC stability.

- The expected-max- t subtraction is not the Bailey/Lopez de Prado Deflated

Sharpe Ratio. It omits strategy-return sample length, skewness, and kurtosis. Existing `dsr` names are legacy/misnamed and must be repaired before promotion becomes active.

No edge promotes. Production has zero `strategy_policies` and zero `backtest_experiments`. The best latest US market-wide t -stat is 1.73. India's best latest observation is 1.57 in a Financials sector slice with only two distinct windows, so it fails the evidence-count gate.

Blocking governance defects

The current route is scaffolding, not a production-grade promotion boundary:

- Rolling windows share about 98.4% of their 1,000-day history.

- The historical universe is a current-liquid snapshot, not point-in-time.

- Missing experiment lineage assumes one trial, and experiments are not bound tightly enough to edge/formula/horizon/segment.

- Supersede then insert is not transactional. The unique index prevents two active policies but cannot prevent zero active policies after insert failure.

- `strategy_policies.walk_forward_pass` stores stability, and the mutation trigger does not protect every field described as immutable.

No consumer may treat a policy row as approved evidence until the revised `features/walk-forward-ic-folds/FEATURE_ARCHITECTURE.md` is approved and built. That design requires a frozen experiment, PIT universe and inputs, purged market-session OOS folds, aggregate OOS IC evidence, proper multiple-testing control, and one atomic service-role promotion RPC. Waiting for more overlapping weekly windows does not cure these defects.

Decisions and Rationale

PROJECT DECISIONS

Source: PROJECT_DECISIONS.md | Decision and rationale ledger

?# Project Decisions ??" Kairos

Purpose

Records approved product, architecture, UX, technical, and business decisions.

Only approved decisions belong here. Proposed decisions live in PROJECT_BUILD_LOG.md until approved.

Decision Template

```
### Decision <N>: <Title>

Date:
Status: Approved
Category: Product / UX / Architecture / Technical / Business / Data / Security

Context:
Decision:
Reason:
Alternatives considered:
Impact:
Files/features affected:
Reversal cost: Low / Medium / High
```

Decision 1: Governed Multi-Agent Quant Platform

Date: 2026-06-27 Status: Approved Category: Product / Architecture / Security

Context: Kairos needs to research continuously, run experiments, explain decisions, and eventually execute through the Robinhood agentic account without allowing probabilistic AI reasoning to bypass financial controls. Decision: Separate Data, Research, Analyst, Validation, Strategy Registry, Paper Execution, Learner, Risk/Tax, Explainer, and Live Trade Gateway responsibilities. LLMs may propose and explain; deterministic services calculate prices, P&L, validation, risk, tax flags, and execution state. Reason: Reproducibility, fault isolation, auditability, and safety are required before paper evidence or live execution can be trusted. Alternatives considered: Single adaptive super-agent; batch-only quant laboratory. Impact: Replaces the current prompt-driven loop with governed contracts and lifecycle states. Files/features affected: features/agentic-quant-platform/FEATURE_ARCHITECTURE.md and future phased implementation files. Reversal cost: High

Decision 2: Initial Trading Scope and Live Authority

Date: 2026-06-27 Status: Approved Category: Product / Security / Business

Context: The initial system must align paper behavior with the Robinhood agentic cash account while preserving a safe path to future automation. Decision: Use long-only 2-20 market-day swing strategies over Robinhood-supported US equities and ETFs that pass quality filters. Every initial live order requires Vaibhav's explicit approval. Future auto-live requires a separate strategy-specific, capital-limited, time-bounded manual unlock and cannot be enabled by an agent. Reason: Aligns research with executable reality and prevents silent expansion of trading authority. Alternatives considered: Long/short paper strategies; unrestricted Robinhood universe; immediate auto-live. Impact: Shorts, options, leverage, crypto, intraday trading, and non-agentic accounts are excluded. Files/features affected: Strategy Registry, paper execution, Risk Engine, Live Trade Gateway, and trading UI. Reversal cost: Medium

Decision 47: Automated validation routes passing challengers only to bounded shadow evidence

Date: 2026-07-12 Status: Approved Category: Product / Architecture / Security

Context: Manual Backtest remains useful for exploration, but Vaibhav should not need to initiate validation every time LearnerAgent proposes a strategy change. The existing fire-and-forget validation request could be lost, and a passing challenger remained inert until manual action. Decision: Store a reversible US/India automation policy. LearnerAgent validates a new challenger synchronously through the deterministic Validation Engine. A passed version may atomically enter at most one non-executing shadow_paper slot per market. A cloud sweep retries only challengers that have no experiment. The owner may disable validation or shadow routing independently by market. Champion promotion, paper fills, broker proposals, and live orders remain outside this automation. Reason: This converts validation into the normal governed workflow while preserving evidence, market isolation, and the owner-only capital authority boundary. Alternatives considered: Automatically promote a passing challenger (rejected: historical evidence is not sufficient for capital authority); rely on TradingView/Pine as an unattended API (rejected: it is not Kairos' reproducible validation backend); retain fire-and-forget local HTTP (rejected: not durable). Impact: New challengers receive automatic evidence and bounded forward shadow measurement. Disabling returns immediately to manual handling without deleting history. Files/features affected: features/automated-strategy-validation/FEATURE_ARCHITECTURE.md, migration 170, Validation Engine automation helper, LearnerAgent, validation sweep, Settings, and cloud scheduler. Reversal cost: Low

Decision 3: Evidence, Data, and Online Research Policy

Date: 2026-06-27 Status: Approved Category: Data / Architecture

Context: Current prototype routes ask an LLM for prices and unsourced scores, which invalidates P&L and experimental evidence. Decision: Use free-first replaceable data adapters, point-in-time append-only evidence, source provenance, and abstention when required data is missing or contradictory. Primary and curated sources establish evidence; broad web/social sources may generate hypotheses only. TradingView is a manual analysis/import tool, not an unattended API. Reason: Market evidence must be deterministic, timestamped, reproducible, and auditable. Alternatives considered: LLM-mediated data retrieval; a hardwired single provider; unrestricted web evidence. Impact: LLM-generated authoritative prices are prohibited and current paper results require stabilization before being trusted. Files/features affected: Data Hub, evidence store, provider adapters, current agent routes, and paper accounting. Reversal cost: High

Decision 4: Controlled Self-Improvement and Strategy Promotion

Date: 2026-06-27 Status: Approved Category: Architecture / Data / Security

Context: Direct weight changes from individual trades chase noise and allow a LearnerAgent to alter production behavior without sufficient evidence. Decision: LearnerAgent creates immutable challenger versions. A deterministic Python Validation Engine applies a dynamic statistical evidence gate. Risk may veto deployment. Vaibhav alone promotes an eligible challenger to champion. Reason: Self-improvement must follow a falsifiable, reproducible experiment lifecycle and preserve failed evidence. Alternatives considered: Fixed time/trade thresholds; direct weekly weight mutation; fully manual evidence review with no system gate. Impact: Introduces Strategy Registry, experiment artifacts, eligibility reports, and champion/challenger governance. Files/features affected: Agent learning, validation worker, strategy persistence, paper engine, and review UI. Reversal cost: High

Decision 6: MacroSentinel ??" Advisory-Only Regime

Date: 2026-06-29 Status: Approved Category: Product / Architecture

Context: MacroSentinel computes a recession danger score and regime (GREEN/YELLOW/ORANGE/RED) from 8 macro indicators. The question was whether to auto-throttle agents or halt trading when regime worsens. Decision: Advisory-only. MacroSentinel reports regime and shows it on the dashboard. It does NOT auto-throttle agents, reduce position sizes, or halt

PaperTrader. Reason: Auto-throttle without the user first observing a live run creates surprising behavior. Vaibhav reviews the regime card and decides whether to act ??” e.g., manually tightening the risk profile or pausing trading. Alternatives considered: Auto-throttle on ORANGE/RED; auto-reduce position_size_pct when score > 50. Impact: MarketsPage gauge + DashboardHome banner are display-only. No agent behavior changes automatically based on macro regime. Files/features affected: /api/agents/macro-sentinel, macro_regime, macro_signals, MarketsPage, DashboardHome Reversal cost: Low (adding auto-throttle later is additive)

Decision 7: Mermaid v10 (not v11)

Date: 2026-06-29 Status: Approved Category: Technical

Context: AgentDiagram.tsx uses Mermaid to render flowcharts. Mermaid v11 was installed initially. Decision: Pin mermaid to v10. Reason: Mermaid v11 depends on es-toolkit, which is ESM-only and incompatible with Next.js webpack bundler. Build failed with module resolution errors. v10 does not have this dependency and builds cleanly. Alternatives considered: Dynamic import workaround for v11 (fragile, not worth it). Impact: Agent diagrams render correctly. No feature difference between v10 and v11 for our use case. Files/features affected: package.json, components/dashboard/AgentDiagram.tsx Reversal cost: Low

Decision 8: Insider Scoring as LLM Context Injection (Not Score Override)

Date: 2026-06-29 Status: Approved Category: Architecture

Context: ResearchAgent needed to incorporate insider transaction signals (Form 4-equivalent via Alpha Vantage INSIDER_TRANSACTIONS). Decision: scoreInsider() computes a 90-day buy/sell ratio from insider transactions and injects the result as pre-fetched context text in the LLM prompt. The LLM weighs it alongside other signals. It is NOT added as a hardcoded numeric score component. Reason: LLM can weigh insider data contextually (e.g., a CEO buy during market panic is more meaningful than a routine RSU sale). Hardcoding it as a fixed weight component removes that nuance. Alternatives considered: Add insider_buying as a fixed-weight signal in the signal_breakdown JSON. Impact: Insider data informs every ResearchAgent run without locking a specific weight. Files/features affected: lib/research-agent.ts Reversal cost: Low

Decision 9: Risk Profile Presets (Conservative / Balanced / Aggressive)

Date: 2026-06-29 Status: Approved Category: Product / Data

Context: Strategy config needed user-selectable risk modes to control score thresholds, position sizing, stops, and targets. Decision: Three named presets stored in strategy_config:

- Conservative: score_threshold=72, position_size_pct=7, stop_loss_pct=5, target_pct=12
- Balanced: score_threshold=60, position_size_pct=10, stop_loss_pct=7, target_pct=20
- Aggressive: score_threshold=52, position_size_pct=15, stop_loss_pct=10, target_pct=35

User can select a preset or edit fields individually. Reason: Named presets make risk configuration intuitive. Per-field override preserves flexibility. Alternatives considered: Single numeric "risk tolerance" slider; no presets (all manual). Impact: ResearchAgent reads score_threshold from strategy_config; PaperTrader reads position_size_pct and stop_loss_pct. Files/features affected: migration 027, /api/settings/risk-profile, Settings page, lib/research-agent.ts, PaperTrader Reversal cost: Low

Decision 10: House Stock Watcher for Congressional Trades (not Quiver Quant)

Date: 2026-06-29 Status: Approved Category: Data / Technical

Context: Smart Money Trades feature needed congressional stock disclosure data. Decision: Use House Stock Watcher public S3 endpoint (free, no auth, public domain data). Reason: Free and publicly maintained. Quiver Quant charges for the same underlying public disclosure data. No auth token management needed. Alternatives considered: Quiver Quant API (paid), SEC EDGAR EDGAR-Online (complex parsing). Impact: Congressional trades tab in MarketsPage powered by House Stock Watcher JSON feed. Files/features affected: `/api/markets/insider-trades/route.ts` Reversal cost: Low (drop-in replacement if Quiver Quant becomes preferable)

Decision 5: Explainability and Evolving Rule Governance

Date: 2026-06-27 Status: Approved Category: Product / UX / Security

Context: Vaibhav wants to understand what the system thinks, plans, and learns while market, dividend, tax, regulatory, and broker rules evolve. Decision: Provide a layered decision journal and daily briefing. Automatically update validated market observations and corporate events. Tax, regulatory, broker, risk, and behavior-changing rule updates are versioned, impact-assessed, and require governed review; risk and execution policy never change silently. Reason: Continuous learning must remain understandable and must not silently broaden financial authority. Alternatives considered: Chat-only explanations; static reports; unrestricted automatic rule updates. Impact: Adds evidence labels, rule-version review, tax/dividend flags, and auditable user decisions. Files/features affected: Explainer, knowledge store, daily briefing, decision journal, Risk/Tax Engine, and rule monitoring. Reversal cost: Medium

Decision 11: RAG Memory for LearnerAgent (Voyage embeddings + pgvector)

Date: 2026-07-03 Status: Approved Category: Architecture / Data

Context: LearnerAgent reasons over historical trade decisions but keyword/filter queries miss semantically-similar situations across different symbols and regimes (e.g. "rate-hike sell-off", "earnings-miss hold"). Decision: Store one Voyage voyage-finance-2 (1024-dim) embedding per enriched trade_decisions row in trade_decision_embeddings (migration 045, HNSW cosine index). Expose semantic_search_decisions as a LearnerAgent tool backed by the semantic_search_trade_decisions RPC. Enrichment is treated as final, so a decision is embedded once and never re-embedded (content_hash retained for audit only). Reason: Vector recall lets the learner find shared failure modes across similar past trades regardless of ticker/wording. Enrichment-final avoids a re-embed loop and keeps the pipeline idempotent. Alternatives considered: LLM re-reading all history each run (token-expensive, lossy); keyword search only (misses semantic matches); re-embed on any content change (needless churn given enrichment is final). Impact: Adds an embeddings table, an embed route, two RPCs, and a VOYAGE_API_KEY dependency (route 503s without it). Files/features affected: `supabase/migrations/045_*, 047_*`; `app/api/live-portfolio/embed/route.ts`; `app/api/agents/learner/route.ts`. Reversal cost: Low

Decision 12: Auto-embed cron before the weekly learner run

Date: 2026-07-03 Status: Approved Category: Architecture

Context: Semantic search is only useful if new enriched decisions get embedded before the learner needs them. Decision: Run the embed route on a weekday 4:50 PM scheduled task (Kairos\embed), ahead of the Friday 5:00 PM learner. Candidate selection uses a unembedded_trade_decisions NOT-EXISTS RPC (bounded, deterministic) rather than loading all embedded ids into app memory. The learner tolerates missing embeddings (tool returns an error, agent continues). Reason: Keeps the vector store fresh with no manual step; NOT-EXISTS scales past the in-memory-id approach; the 10-minute gap plus small daily volume makes overlap a non-issue. Alternatives considered: Embed inline during enrichment (couples two concerns); load-all-ids-then-filter-in-JS (unbounded memory as history grows); no automation (stale vectors). Impact: Adds an embed agent to `run-agents.ps1` and a scheduled task; `run-agents.ps1` now resolves

CRON_SECRET at runtime from .env.local (secret never committed). Files/features affected: scripts/run-agents.ps1, scripts/register-tasks.ps1, supabase/migrations/047_*. Reversal cost: Low

Decision 13: Challenger lifecycle for learner weight changes

Date: 2026-07-03 Status: Approved Category: Architecture / Security

Context: LearnerAgent previously mutated signal_weights directly, letting probabilistic reasoning silently overwrite the live champion strategy - violating the governance principle in Decision 4. Decision: update_signal_weight no longer touches signal_weights. It inserts an immutable strategy_versions row with state='challenger' and a weights_snapshot (only the five weight columns, no metadata). Weights take effect only when a human promotes the challenger to champion via the Strategy Registry. Every Supabase error is checked; success is reported only after a confirmed insert; versions carry a date+HHMMSS suffix to avoid same-day collisions. Reason: Human-in-the-loop gate before any strategy change goes live; auditable proposal trail; no silent authority expansion. Alternatives considered: Direct weight mutation (unsafe); auto-promote above a confidence threshold (removes human gate). Impact: Learner output is advisory until promoted; adds challenger rows to strategy_versions. Files/features affected: app/api/agents/learner/route.ts, Strategy Registry UI. Reversal cost: Low

Decision 14: LearnerAgent runs on DeepSeek-reasoner, not Opus (no ANTHROPIC_API_KEY)

Date: 2026-07-04 Status: Approved Category: Technical / Architecture

Context: LearnerAgent was configured for claude-opus-4-8, but the app has no raw ANTHROPIC_API_KEY - all Claude usage falls back to the claude CLI subprocess (execClaude), which cannot do tool-calling. Every Friday the learner's tool-loop threw "Could not resolve authentication method" and recorded a 0-token error run. Decision: Run the learner on deepseek-reasoner (works via DeepSeek's native tool-calling, ~\$0.043/run). runAgentLoop dispatches by model prefix: Claude -> Anthropic SDK tool loop, everything else -> DeepSeek. The Claude branch stays in code but requires a real ANTHROPIC_API_KEY to ever run. Reason: The learner must actually run; DeepSeek-reasoner gives budget-friendly step-by-step reasoning today. Model is user-changeable per agent via /dashboard/agents -> LLM Config (agent_config.model). Alternatives considered: Keep Opus + add ANTHROPIC_API_KEY (best reasoning, real API cost \$5/\$25 per Mtok - deferred until the user adds a key); deepseek-chat (cheaper, weaker reasoning). Supersedes the earlier "Opus for learner" intent. Impact: Learner reasoning quality is DeepSeek-reasoner tier, not Opus, until an Anthropic key is added. Files/features affected: agent_config.model (learner), lib/llm-router.ts. Reversal cost: Low

Decision 15: Langfuse observability + full agent-loop cost logging

Date: 2026-07-04 Status: Approved Category: Technical / Data

Context: Direct callLLM calls were logged to llm_call_log, but agent tool-loops (learner/research/mentor/theme-scout/macro-sentinel via runAgentLoop) bypassed it - so the cost dashboard undercounted exactly where the expensive tokens live. There was also no external trace view of LLM calls. Decision: (a) runAgentLoop now calls logCall on every invocation (model, tokens, cost, duration, success, agent_label, run_id), so all agent-loop usage appears at /dashboard/admin/llm-history. (b) callLLM wraps each call in a Langfuse trace/generation, gated on LANGFUSE_SECRET_KEY/LANGFUSE_PUBLIC_KEY and no-op when absent. Reason: Accurate per-agent/per-task cost visibility is required to make model-tier decisions with data; Langfuse adds zero-risk external tracing when keys are present. Alternatives considered: Leave agent loops untracked (blind spot); build a custom trace UI (reinventing Langfuse). Impact: Complete cost ledger; optional Langfuse tracing. Files/features affected: lib/llm-router.ts, llm_call_log, /dashboard/admin/llm-history. Reversal cost: Low

Decision 16: Deep-Dive Debate - adversarial per-symbol analysis

Date: 2026-07-04 Status: Approved Category: Product / Architecture

Context: Competitor trading-agents.ai produces an argued per-symbol verdict via multi-agent debate; Kairos only produced a numeric analyst_score. Decision: On-demand per-symbol Deep-Dive: 4 parallel analysts -> Bull vs Bear advocate -> Research Evaluator -> Risk desk -> Portfolio Manager verdict (BUY/HOLD/SELL/PASS + conviction). Reasons over data we already have (quote + our signal scores + macro + 1 Alpha Vantage OVERVIEW for fundamentals) to stay in AV's 25/day budget. DeepSeek models (no ANTHROPIC_API_KEY). Long-only enforced (unheld -> BUY/PASS). Advisory only - risk gate still governs. Stored in deep_analyses. Reason: Closes the one gap where the competitor was ahead (argued verdict + transparency) without giving up governance. Alternatives considered: Full history re-read (expensive); replace analyst_score (loses the deterministic pipeline). Impact: New per-symbol "Deep Dive" tab; ~\$0.004/run. Files/features affected: app/api/agents/deep-dive/, components/dashboard/DeepDivePanel.tsx, migration 048. Reversal cost: Low

Decision 17: MentorAgent is a true AI coaching agent

Date: 2026-07-04 Status: Approved Category: Product / Architecture

Context: The old mentor just scored a single thesis. The user wants a coach that reasons over their behavior + market + learning progress. Decision: MentorAgent is a deepseek-reasoner tool-use agent (query_behavior, query_learning_progress, query_market_context, read_principles -> finish). Returns grade + confidence, strengths, focus areas, one market-tailored lesson, next milestone. Grounded; if data is thin it coaches on process. Advisory/educational only. Stored in mentor_insights, surfaced on the Mentor "AI Coach" tab + the briefing. Weekly cron (Fri 5:15 PM). Reason: A data-grounded personal trading coach is a differentiator nobody combines (behavior + regime + curriculum). Alternatives considered: Keep the one-shot scorer; a human-written static tip. Impact: New coaching surface; ~\$0.006/run. Files/features affected: app/api/agents/mentor-coach/, components/dashboard/MentorCoachPanel.tsx, migration 050. Reversal cost: Low

Decision 18: Briefing = structured data blocks + short LLM note (newsletter-grade)

Date: 2026-07-04 Status: Approved Category: Product / UX

Context: The briefing was a wall of LLM prose. The user wants newsletter-grade structure with mandatory explanations on every metric. Decision: Render the data deterministically as designed HTML blocks (accurate); the LLM writes only a short editor's note + a grounded 3-part outlook with confidence. v2 adds a lighter theme, per-metric explanations, market/positions/future outlooks, a 7-day agent-activity recap, and a Mentor block. Email delivers via Resend; email_sent reflects the real result; recipient/sender overridable via BRIEFING_TO/BRIEFING_FROM. onboarding@resend.dev only delivers to the Resend account owner until a domain is verified. Reason: Numbers must be accurate (code-built) and every figure must carry a what/why (locked user preference - details mandatory). Alternatives considered: All-LLM prose (hallucination + wall of text); all-static (no human voice). Impact: Redesigned morning/evening emails. Files/features affected: app/api/briefing/generate/route.ts. Reversal cost: Low

Decision 19: Markets synthesis from ETF regime proxies (no AV budget)

Date: 2026-07-04 Status: Approved Category: Architecture / Data

Context: The user wants a synthesis above the heatmap answering "where are markets and heading" from more than just VIX. Decision: A Market Synthesis card reads ETF regime proxies via Massive (breadth SPY/QQQ/IWM, credit HYG/IEF, rates TLT/IEF, dollar UUP, gold GLD, vol VIXY), computes risk-on/neutral/risk-off, and a DeepSeek synthesis grounded only in those numbers, with a what/why note on every tile. Sector Breadth gains a 1W-1Y period selector. Uses Massive (not Alpha Vantage) to avoid the 25/day cap. Reason: A combined-signal read guides the user and feeds regime context; ETF proxies via Massive are budget-free. Alternatives considered: Alpha Vantage macro series (blows the 25/day budget). Impact: New synthesis section on Markets; cached to briefings (session='synthesis', migration 038). Files/features affected: app/api/markets/synthesis/, components/dashboard/MarketsPage.tsx, SectorBreadth.tsx. Reversal cost: Low

Decision 20: Automation schedule source-of-truth + nav consolidation

Date: 2026-07-04 Status: Approved Category: Architecture / UX

Context: The pipeline widget was hardcoded/wrong; there was no single view of what runs when/where. Nav had duplicated/overlapping items. Decision: `lib/schedule.ts` is the single source-of-truth for all 12 scheduled jobs (name, time, days, runner, description, handoff). A read-only Settings -> Automation page shows times, runner (Windows Task Scheduler), last/next run. Consolidated nav: removed the duplicate Intelligence "brief" tab (#11), unified Strategies + Algo Library into one tabbed section and dropped the standalone Library nav item (#18/#19). All crons run via Windows Task Scheduler (cloud edge-function crons decommissioned); edit by re-running `scripts/register-tasks.ps1`. Reason: One accurate schedule surface; less nav clutter; honest about what's editable where. Alternatives considered: Keep hardcoded widget; in-app schedule editing (needs OS/admin - deferred). Impact: New Automation page; leaner nav. Files/features affected: `lib/schedule.ts`, `app/dashboard/settings/automation/`, `app/api/automation/schedule/`, `DashboardShell.tsx`, `scripts/*.ps1`. Reversal cost: Low

Decision 21: Sector chart - real TradingView widget, not a custom Massive-backed chart

Date: 2026-07-04 Status: Approved Category: Architecture / Data

Context: The Sector Correlation chart on Markets was custom-built against Massive candle data. Root-caused a missing pagination bug (`next_url` wasn't followed, so 1Y+ periods silently returned the same truncated data as 3M - commit `bc07c7a`). Fixing pagination then exposed a real provider limit, not a code bug: Massive's free tier hard-caps every aggs response at ~500 bars with no pagination beyond that (confirmed via direct API call requesting 2016-2026 and getting back only the most recent ~500 bars). Decision: Replace the custom Massive-backed sector chart entirely with TradingView's real widget. First tried the free "Symbol Overview" embed (`77f371f`) - it did not hydrate reliably in real testing (empty section on the Markets page). Landed on reusing the same `TradingViewChart` component (real `tv.js` Advanced Chart widget - full toolbar, indicators, real period buttons) already proven on the symbol detail page, with a tab switcher across the 11 sector ETFs, since TradingView's free tier has no combined multi-symbol overlay/correlation chart (`9f81fd5`). Reason: Massive's free tier cannot support long-lookback sector correlation no matter how the code is written; TradingView's real widget gives accurate, full-history charting per sector today without waiting on a paid Massive plan. Alternatives considered: Upgrade Massive plan (cost, deferred); keep the custom chart capped at ~2yr lookback for 3Y/5Y/10Y (misleading - those periods would silently show the same window); free "Symbol Overview" embed (tried first, didn't render reliably). Impact: Sector Correlation section on Markets is now single-symbol-at-a-time (tab per sector ETF) rather than a combined overlay chart, in exchange for accurate, full-toolbar real-time charting per sector. Files/features affected: `components/charts/SectorTradingViewOverview.tsx` (new); removed `components/charts/SectorLineChart.tsx` and `app/api/charts/sector-history/route.ts`. Reversal cost: Low (additive - a combined overlay could be reintroduced later on a paid Massive tier or a different data provider)

Decision 22: Privacy Mode - mask live-account dollar figures by default

Date: 2026-07-04 Status: Approved Category: Product / UX

Context: Live Robinhood account figures (equity, buying power, position values, P&L) render in plaintext on Dashboard home and Live Portfolio - a problem for screen-sharing, screenshots, or anyone glancing at the screen. Decision: An eye-icon toggle on both surfaces masks these figures by default; clicking reveals them. The reveal state is plain `useState`, not persisted, so it resets to hidden on every navigation away and back - no extra logic needed to "re-hide." A master on/off switch lives in Settings -> Preferences, persisted to `localStorage` (`components/dashboard/PrivacyMask.tsx`). Reason: Default-hidden is the safer default for a dashboard that's frequently open during screen-shares; per-mount reset means the user never has to remember to re-hide; a master switch lets users who don't need this (private single-user desktop) turn it off entirely. Alternatives considered: Persist the reveal state across navigation (risk of numbers staying visible after the user forgets); no default masking (status quo, rejected as the motivating problem); server-side masking (unnecessary - this is a display-only concern with no security boundary implication). Impact: New shared component; two

consuming surfaces (Dashboard home "Live Robinhood" panel, Live Portfolio page). Files/features affected: components/dashboard/PrivacyMask.tsx (new), components/dashboard/DashboardHome.tsx, components/dashboard/LivePortfolioPage.tsx, app/dashboard/settings/page.tsx. Reversal cost: Low

Decision 23: execClaude/MCP tool-calling gap - OPEN, decision needed, not yet resolved

Date: 2026-07-04 Status: Open - not resolved. Requires explicit user sign-off before any fix. Category: Architecture / Security

Context: An audit this session found that execClaude (lib/claude-exec.ts) runs the Claude Code CLI as a plain text-completion subprocess - no ANTHROPIC_API_KEY, no MCP server config attached anywhere. It structurally cannot call any MCP tool (Robinhood, FinancialDatasets, etc.) no matter what its prompt asks for; the pattern in every call site is "ask the model to call a tool it can't reach, trust whatever text comes back." Confirmed call sites: lib/research-agent.ts (fetchAndStoreAccountSnapshot and runScreener - meaning the CLAUDE.md-mandated dual-bucket momentum/value screener has likely never produced real candidates via this path), app/api/mentor/evaluate/route.ts (worst case: could silently write hallucinated "verified" fundamental data into trade_journal as fact), app/api/portfolio/live-holdings/route.ts, lib/market-data.ts, app/api/portfolio/robinhood/route.ts, lib/chart-data.ts (two functions), and - highest severity - app/api/agents/trader/route.ts and app/api/agents/trade/approve/route.ts, the real-money order-execution paths for account 605420660, which gate "order submitted to Robinhood" entirely on a success: true JSON flag execClaude cannot authentically produce. It currently fails toward success: false in practice rather than fabricating a fill, but this is not a code guarantee - there is no independent verification step. Decision: Not resolved. This entry exists to flag the risk and lock in why trading_mode = disabled must stay in place (per CLAUDE.md) until a real fix ships. No code change has been made against this finding. Reason: This is exactly the class of risk Decision 3 (Evidence, Data, and Online Research Policy) was written to prevent - LLM-mediated "tool calls" that aren't real must not be trusted as evidence or as execution confirmation, especially on the order-placement path. Alternatives considered (proposed, none implemented yet): (a) Rebuild these call sites as direct, typed API calls with no LLM asked to "call" a tool in the loop - most consistent with Decision 3; (b) add a real ANTHROPIC_API_KEY so execClaude's replacement can use genuine MCP tool-calling; (c) leave as-is and rely on trading_mode = disabled - acceptable short-term only, not a fix. Impact: Until resolved, do not enable live trading; do not trust research-agent.ts's screener output as verified against real MCP data; treat mentor/evaluate "verified" fundamentals with suspicion. Files/features affected: lib/claude-exec.ts, lib/research-agent.ts, app/api/mentor/evaluate/route.ts, app/api/portfolio/live-holdings/route.ts, lib/market-data.ts, app/api/portfolio/robinhood/route.ts, lib/chart-data.ts, app/api/agents/trader/route.ts, app/api/agents/trade/approve/route.ts. Reversal cost: N/A (nothing reversed - decision on the fix approach is pending)

Decision 24: Close the learning loop - ResearchAgent consumes promoted champion weights

Date: 2026-07-05 Status: Approved Category: Architecture / Data

Context: Decision 13 established the challenger->champion governance path: LearnerAgent proposes weight challengers into strategy_versions, and a human promotes one to champion (is_champion=true, weights_snapshot jsonb). But the loop was OPEN - nothing downstream ever read the promoted weights. ResearchAgent scored every symbol with a hardcoded PROFILE_WEIGHTS table keyed only on risk_profile, and signal_weights was dead code. Approved learning had zero effect on scoring. Decision: lib/research-agent.ts's processSymbol reads the promoted champion's weights_snapshot first, normalizing both key formats (seed row short keys {fundamental:0.3, ...}; LearnerAgent challenger suffixed keys {fundamental_weight:0.3, ...}). It falls back to the static PROFILE_WEIGHTS table (then signal_weights) only when no champion is promoted. research_packets.raw_data records _using_champion_weights: true/false per signal for auditability. Reason: A learning system that can't affect scoring is decorative. This is the hop that makes the human-gated challenger path from Decision 13 actually change production

behavior - while keeping the human promotion gate intact. Alternatives considered: Auto-apply challenger weights without promotion (removes the Decision 13 human gate - rejected); keep scoring on PROFILE_WEIGHTS and treat learning as advisory-only forever (defeats the learner's purpose). Impact: Promoting a challenger now changes ResearchAgent's next-run scoring. No effect until a champion is actually promoted (falls back to profile weights). Files/features affected: lib/research-agent.ts (processSymbol), strategy_versions, research_packets.raw_data. Reversal cost: Low (revert to always reading PROFILE_WEIGHTS)

Decision 25: Durable per-symbol score history (signal_score_history) + ScoreTrajectory feature

Date: 2026-07-05 Status: Approved Category: Data / Product

Context: PRD.md specced a "30-day price + agent score history" chart, but the data model never existed. agent_signals rows get status-mutated and filtered, and the table barely accumulates, so the existing ScoreTrajectory chart UI was starved of data. There was no durable record of how a symbol's score evolved over time. Decision: New append-only table signal_score_history (migration 054): symbol, analyst_score + 5 dimension scores (fundamental/technical/sentiment/macro/insider), direction, source, created_at; RLS service_role-all + authenticated-read; index on (symbol, created_at desc). Every score computation in lib/research-agent.ts appends a row (best-effort - won't fail the research run, no-ops until migration applied). Rows are never touched after insert. Consumed by (1) a SCORE TREND note injected into the thesis prompt so the agent reasons about conviction momentum, and (2) a new GET /api/charts/score-history?symbol=X route feeding the symbol-detail ScoreTrajectory chart. Reason: Score trajectory is signal - a rising score carries different conviction than a cold snapshot at the same value. An append-only history is the honest data model (unlike the mutated agent_signals) and finally delivers the PRD chart. Alternatives considered: Reconstruct history from agent_signals (unreliable - rows are mutated/filtered); store history inside research_packets.raw_data (not queryable per-symbol over time). Impact: New table + route; thesis prompt gains a trend line; symbol detail page chart now shows real durable history. Best-effort write means research runs are unaffected if the insert fails. Files/features affected: supabase/migrations/054_signal_score_history.sql, lib/research-agent.ts, app/api/charts/score-history/route.ts, symbol detail ScoreTrajectory chart. Reversal cost: Low

Decision 26: LearnerAgent Phase B is a last-resort backstop, deferring to the Phase A smart-exit path

Date: 2026-07-05 Status: Approved Category: Architecture

Context: The LearnerAgent had two exit mechanisms that raced. Phase A (smart) re-scores a position's current signal and flags exit_reason="llm_exit", which position-monitor executes with a live price + trailing-stop logic. Phase B (blunt) unconditionally closed any paper_trades row >7 days old on a crude pnl>\$0.50 win/loss threshold. The same position could be closed by both in one run, and the crude outcome usually won - overriding the smart exit. Decision: Phase B now (1) skips any trade whose position already carries the llm_exit flag - deferring to Phase A / position-monitor - and (2) its time cutoff moves from 7 -> 14 days, making it a true last-resort backstop (nothing sits open forever) rather than a primary exit competing with the smart path. Reason: The smart, live-priced exit should always win; the blunt time-based rule exists only to guarantee nothing is stranded open indefinitely. De-conflicting them prevents the crude outcome from silently clobbering the intended exit. Alternatives considered: Remove Phase B entirely (loses the guarantee that stale positions eventually close); keep both racing (status quo - crude path wins incorrectly). Impact: Phase A drives real exits; Phase B only touches positions with no llm_exit flag that are >14 days old. Files/features affected: app/api/agents/learner/route.ts (Phase A / Phase B close logic). Reversal cost: Low

Decision 27: Langfuse tracing extended to the agent tool-loop (runAgentLoop)

Date: 2026-07-05 Status: Approved Category: Technical / Data

Context: Decision 15 wrapped single-shot `callLLM` completions (thesis, screening, chat) in Langfuse traces. But the multi-step tool-calling `runAgentLoop` (LearnerAgent, MentorAgent) was invisible in Langfuse - it only wrote to the internal `llm_call_log` table. The most complex, multi-turn LLM usage had no external trace view. Decision: Wrap `runAgentLoop` in a Langfuse trace/generation span capturing system prompt in, final text out, total tokens, cost, and the tool-call trail as metadata. Gated on the Langfuse keys and no-op when absent, consistent with Decision 15. Reason: The tool-loops are exactly where multi-turn debugging and cost attribution matter most; leaving them out of Langfuse was the biggest remaining observability gap. Alternatives considered: Leave agent loops traced only in `llm_call_log` (no span/tool-call view); adopt LangChain/LangGraph for built-in tracing (rejected - the loop stays hand-rolled against the Anthropic/DeepSeek SDKs; not introducing a framework just for tracing). Impact: Agent tool-loops appear as Langfuse traces with their tool-call trail; no change to agent logic. LangChain/LangGraph remain unused. Files/features affected: `runAgentLoop` (agent tool-calling loop), Langfuse integration in the LLM layer. Reversal cost: Low

Decision 28: India market data via free Yahoo Finance (not a paid feed)

Date: 2026-07-05 Status: Approved Category: Data / Architecture

Context: Kairos added India as a second market. Execution runs through Zerodha Kite, but the Kite Personal tier provides execution + portfolio only, with no market data. Indian NSE stocks still need price, candles, and fundamentals to run the same 5-dimension scoring pipeline as US stocks. Decision: Source all India market data from free Yahoo Finance (.NS symbols). `lib/india-data.ts` uses the Yahoo chart endpoint for price + candles (unauthenticated) and a cookie+crumb `quoteSummary` call for fundamentals (P/E, ROE, margins), remapping the result into the exact AV-OVERVIEW shape the existing scorer already consumes. Candidates come from a static NIFTY-50 list (`lib/india-universe.ts`) rather than a paid screener. US-only inputs (social sentiment, options, insider) are unavailable for India, so those dimensions use a neutral baseline, flagged honestly in the score-detail. Reason: Kite Personal tier has no data feed; paying for one (or upgrading Kite) is unjustified when Yahoo covers price, candles, and core fundamentals for free. Mapping into the AV-OVERVIEW shape lets India reuse the whole scoring pipeline with no scorer changes. Alternatives considered: Pay for a Kite data subscription or a third-party Indian data vendor (cost, deferred); a paid screener for candidate selection (static NIFTY-50 is sufficient for now); skip India fundamentals entirely (loses the fundamental dimension). Impact: India scores on free data with no per-call cost; the three US-only dimensions are neutral for Indian names by design. No direct non-US equity coverage exists beyond India. Files/features affected: `lib/india-data.ts`, `lib/india-universe.ts`, `lib/research-agent.ts`, paper-trade route (India exclusion - see Decision 29). Reversal cost: Low (data adapter is replaceable; the AV-OVERVIEW mapping isolates the scorer from the source)

Decision 29: India is scored + tracked but NOT paper-traded (currency/USD-pool)

Date: 2026-07-05 Status: Approved Category: Product / Data / Architecture

Context: US signals flow into a single `paper_portfolio` USD pool for paper P&L before any real money is involved. India was added as a second market, and the question was whether Indian signals should also open paper positions. Decision: India is scored and tracked (Score Tracker, `/dashboard/india`) but NOT paper-traded. `PaperTrader` excludes `asset_class = "india"`. Indian stocks are acted on via real Zerodha Kite orders instead (see Decision 30). Reason: The `paper_portfolio` is a single USD pool; Indian stocks are INR-priced. Mixing currencies into one NAV pool would corrupt paper P&L and NAV/alpha accounting. Keeping India out of the paper pool preserves the integrity of the USD paper track while still surfacing India's scores. Alternatives considered: Convert INR fills to USD at a daily FX rate for the paper pool (adds an FX-rate dependency and silent conversion error into every India paper P&L - rejected); a separate INR paper pool (new parallel accounting surface, deferred - India already executes for real via Kite so a paper stage adds little); score India but hide it (loses the Score Tracker value). Impact: India appears in scoring/tracking surfaces but never in paper positions or paper NAV. The US "paper-first" path and the India "real-only via Kite" path are deliberately asymmetric. Files/features affected: `lib/research-agent.ts`, paper-trade route, `/dashboard/india`, Score Tracker. Reversal cost: Low (a separate INR paper pool could be added later without touching the USD pool)

Decision 30: Zerodha Kite for India execution - human-confirm + daily one-click token

Date: 2026-07-05 Status: Approved Category: Security / Architecture / Product

Context: India needs real order placement and holdings (there is no paper stage - Decision 29). Zerodha Kite Connect v3 is the broker. Kite access tokens expire at 6 AM the next day by SEBI rule and cannot be silently refreshed without storing broker credentials. Decision: Real Kite execution (app/api/kite/order - POST /orders/regular, product CNC) and holdings read (app/api/kite/portfolio) with a strict human-in-the-loop model: authenticated-user-only (never cron/agent), requires explicit confirm: true, writes a decision_journal audit row, and the /dashboard/india UI uses a two-step confirm with a prominent "REAL MONEY" warning that never fires on first click. Auth is a daily one-click Kite Connect v3 login (lib/kite.ts, app/api/kite/login|callback|status): login -> request_token -> SHA256-checksum exchange -> access_token stored in api_key_vault as KITE_ACCESS_TOKEN, treated as expired if not generated today. Reads and orders degrade to a "reconnect" state when the token is stale. Reason: India orders are real money with no paper buffer, so the confirm gate must be stricter than the US path - never agent-invoked, always explicit two-step confirm, always audited. The daily re-login is a SEBI regulatory constraint; automating it would require storing broker credentials, which is deliberately not done. Alternatives considered: Allow cron/agent-placed India orders (rejected - real money, no paper stage, unacceptable without a human); store broker credentials to auto-refresh the token (rejected - deliberately not storing credentials); a paper stage before real Kite orders (Decision 29 - INR/USD pool conflict). Impact: India execution requires a fresh one-click login each trading morning and an explicit two-step "REAL MONEY" confirm per order. No automated India trading path exists. Files/features affected: lib/kite.ts, app/api/kite/login, app/api/kite/callback, app/api/kite/status, app/api/kite/portfolio, app/api/kite/order, app/dashboard/india, api_key_vault (KITE_ACCESS_TOKEN), decision_journal, Settings -> Agents connection card. Reversal cost: Medium (execution path is broker-specific; auth/token model is Kite/SEBI-specific)

Decision 31: Multi-market via per-currency paper pools + per-market champion weights (market as a tag, not a fork)

Date: 2026-07-05 Status: Approved Category: Architecture / Data / Product

Context: India was scored + tracked but NOT paper-traded (Decision 29) because the single USD paper_portfolio pool could not absorb INR fills without corrupting NAV. The cost of that decision: India produced zero closed paper outcomes, so the Learner/Mentor never learned it - the learning loop was open for India. Meanwhile "market" was drifting toward a fork (US-only paths vs India-only paths), and market_focus still offered Europe/Asia/Crypto/Global regions that were never real markets. Decision: Make market a tag/dimension (us | india), not a fork - one app, panels filter by market, currencies are NEVER summed into one number. (a) Separate per-currency paper pools: each market gets its own pool in its own currency - US = existing USD pool (\$10k), India = new ? pool (?1,000,000 starting cash). paper_portfolio/paper_positions/paper_trades/paper_performance/agent_signals/signal_score_history all gain a market column; paper_performance unique key moves from (date) to (date, market) so each market keeps its own NAV curve. PaperTrader fills each signal into its market's pool in native currency off the right price source (US via getQuote; India via free Yahoo .NS), sizing on THAT pool's cash. PositionMonitor monitors/exits per market in native currency, crediting each close back to its own pool. This closes the learning loop for India (it now produces closed outcomes). (b) Per-market champion weights: strategy_versions gains a market column; LearnerAgent analyzes ONE market's cohort per run and proposes challengers ONLY for that market's champion, so a bad India run can never shift US scoring. India starts on a CLONE of the US champion as a prior (seeded by 057) and diverges once it clears the same 10+ closed-trade phase gate. ResearchAgent reads the market-matched champion. (c) market_focus trimmed to US + India only (Europe/Asia/Crypto/Global removed as noise) and is a non-destructive gate: turning India ON starts NIFTY scoring + ? paper fills + India learning cohort; turning it OFF stops NEW India research/fills but KEEPS open India positions monitored-to-close plus all history/weights (re-enable resumes). Real Kite holdings/execution are unaffected by the toggle (real money, independent of a preference). (d) Guarded/resilient rollout: pre-057 (no market column, single pool) every path behaves byte-for-byte as the old US-only app; India activates automatically once 057 is applied and the pool row exists. Reason: Per-currency pools let India paper-trade without ever blending INR into the USD NAV - which fixes the exact

corruption Decision 29 avoided, but now WITH a closed learning loop. Per-market champions keep cross-country learning from contaminating each other. Market-as-tag keeps it one app instead of a maintenance-doubling fork. A non-destructive gate means toggling a market is a preference, not a data-loss event. Supersedes Decision 29 (India IS now paper-traded, in its own INR pool). Alternatives considered: Keep India score-only forever (Decision 29 - leaves the India learning loop permanently open, rejected); one shared pool with daily FX conversion of INR fills to USD (silent conversion error in every India P&L - rejected, same reason as Decision 29); a single global champion across markets (a bad India cohort would shift US scoring - rejected); fork the app into US and India builds (doubles maintenance surface - rejected). Impact: India now paper-trades in ? and feeds the Learner/Mentor; US and India each keep an independent NAV curve and champion. No cross-market contamination. No change to the US path until India is enabled. Migration 057 could NOT be auto-applied this session (Supabase MCP permission-denied, no `psql/DATABASE_URL`) - the user must apply `057_multi_market.sql` manually in the Supabase SQL editor before India activates. Guarded code means the app runs unchanged until then. Files/features affected: `supabase/migrations/057_multi_market.sql`, `app/api/agents/paper-trade/route.ts`, `app/api/agents/position-monitor/route.ts`, `app/api/agents/learner/route.ts`, `lib/research-agent.ts`, Settings `market_focus`, portfolio UI market selector, `strategy_versions/paper_*/agent_signals/signal_score_history` (new market column), `public/agent-diagrams/system-map.json`. Reversal cost: Medium (schema-coupled - a market column across six tables and a changed unique key; guarded code de-risks rollback)

Decision 32: Full India feature parity via a market-scoped panel architecture + direct NSE feeds (with graceful Yahoo/NIFTY-100 fallback)

Date: 2026-07-05 Status: Approved Category: Architecture / Product / Data

Context: Decision 31 made India a paper-trading, self-learning market with its own ? pool and champion, but India was still second-class across the UI - most panels were US-only, and the two free-data ceilings from Phase 2 remained: the India scanner could only see NIFTY-100 (no full-market scan) and insider/option data was US-only. The question was how to bring India to parity across every dashboard panel without forking the app, and how to lift those two ceilings without paying for a data feed. Decision: Ship "India parity" as a market-scoped panel architecture plus direct NSE feeds. (a) Market-as-context switcher: `lib/market-context.tsx` (`MarketProvider/useMarket()`) holds the selected market (`us | india`), persisted to `localStorage` + an `mkt` cookie; rendered in the `DashboardShell` header but hidden unless `market_focus` includes India. Client panels scope to the selected market; server pages read the `mkt` cookie. (b) Honest per-page support badges: `lib/market-support.ts` is the single source of truth mapping each route -> `{ level: full | partial | us-only | india-only, note }`, rendered as a footer badge on every dashboard page - flip a level there as a panel gains coverage. Panels wired for India (US path unchanged, India via free data): Markets (NIFTY/SENSEX/BankNifty/India-VIX via Yahoo), Risk Analytics (per-? book, VaR vs NIFTY; beta-vs-NIFTY still coming soon), Backtest (Yahoo .NS candles, alpha vs NIFTY), Scanner (full NSE via nightly cache, NIFTY-100 fallback), Strategies (market-scoped fit scores; India classification still US-only), Earnings (India per-symbol dates via Yahoo, tracked names only), Smart Money (signals + trade queue both markets; India insider + option PCR/OI live from NSE). (c) NSE-direct to lift the free-data ceilings: `lib/nse-data.ts` is a cookie-handshake adapter for NSE's free JSON - full equity list (`EQUITY_L.csv`), insider (`corporates-pit`), option chain (`option-chain-indices/option-chain-equities`) - which lifted the two ceilings (full-market scan + India insider/options). It fails soft: callers fall back to Yahoo / NIFTY-100 with an honest note. (d) Cache for the full-market scan: migration `058_india_screen_cache.sql` (new table) + nightly cron `POST /api/scan/india/refresh` scores the full NSE list in 600-name oldest-first slices; the Scanner reads the cache and falls back to live NIFTY-100. (e) Per-market crons: `research/paper-trade/position-monitor` accept `?market=us | india`; the US 9 AM tasks are pinned to `?market=us` so they no longer double-process India, and India runs its own Task Scheduler tasks (PC clock = ET, all post-NSE-close): `scan-india-refresh` 5:30 AM, `research-india` 6:15 AM, `position-monitor-india` 6:35 AM. Reason: A market-scoped panel architecture brings India to full parity while keeping one app (no fork, no maintenance doubling); the support registry keeps the UI honest about exactly where India is full vs partial vs US-only. NSE-direct feeds lift the two remaining ceilings for free, and failing soft to Yahoo/NIFTY-100 means a geo-blocked NSE never breaks a page. The cache makes a full-market NSE scan affordable to run nightly. Per-market crons stop the US schedule from double-processing India. Alternatives considered: Keep India panels US-only / partial forever (leaves India

second-class - rejected); fork the app into US and India builds (doubles maintenance - rejected, consistent with Decision 31's market-as-tag choice); pay for an Indian data vendor for full-market + insider + options (unjustified when NSE's public JSON is free - rejected); hit NSE live on every scanner load (slow + geo-throttle risk - rejected in favor of the nightly cache); one shared cron set for both markets (double-processes India - rejected in favor of ?market= scoping). Impact: India reaches parity across the dashboard behind a global switcher; every page shows an honest coverage badge. NSE-direct data powers the full-market scan and India insider/options, degrading gracefully to Yahoo/NIFTY-100 when NSE is geo-blocked (likely from a US IP). New nightly India scan cron + three ET-scheduled India tasks. No change to the US path. Files/features affected: `lib/market-context.tsx`, `lib/market-support.ts`, `lib/nse-data.ts`, `components/dashboard/DashboardShell.tsx`, `Markets/Risk/Backtest/Scanner/Strategies/Earnings/Smart Money` panels, `app/api/scan/india/refresh`, `app/api/agents/research/cron` + `paper-trade` + `position-monitor` (?market=), `supabase/migrations/058_india_screen_cache.sql`, `scripts/run-agents.ps1`, `scripts/register-tasks.ps1`, `public/agent-diagrams/system-map.json`. Reversal cost: Medium (schema-coupled - migration 058 + a new cache table and cron; the switcher/support-registry are additive and low-cost to remove, but NSE-direct + per-market crons touch several panels and the schedule) Update (2026-07-05): The four remaining partial India panels were brought to full parity - Markets (real sector heatmap + NIFTY-50 breadth via `fetchIndiaSectors`), Risk Analytics (real beta-vs-NIFTY regression), Strategies (real India fit-scores from `signal_score_history`), Earnings (market-wide NSE results calendar via `fetchNseEarnings`, Yahoo per-symbol fallback); `market-support.ts` flipped all four to full. Remaining honest US-only bits: Markets TradingView/macro-sentinel tiles, Strategies Algo Library, and NSE feeds may geo-block from a US IP (graceful fallback).

Decision 33: Point-in-time decision ledger + matured horizon labels (Phase 1 learning-core)

Date: 2026-07-06 Status: Approved Category: Architecture / Data / Learning

Context: An independent architecture review (Codex, `CODX_AGENT_REVIEW_RESULT.md`) found the learning plane statistically untrustworthy: `LearnerAgent's query_signals_with_outcomes/query_score_correlation` joined signals to closed trades by symbol (last-write-wins on repeats), the label was policy P&L on filled longs only (selection bias - no signal from rejected candidates), and it mixed alpha with market beta, holding time, and exit policy. Any apparent "learning" could be noise, symbol-collision, or leakage rather than a real edge. This is the single biggest blocker to a genuinely self-evolving agent (see the full roadmap in `features/learning-core/FEATURE_ARCHITECTURE.md`). Decision: Build an immutable, point-in-time decision ledger as the learning ground truth, ahead of any statistical-method work (Phase 2). (a) `decision_observations` (migration 059, append-only via trigger): one row for every candidate `ResearchAgent` scores - filled OR rejected - with the point-in-time raw features (`computeScores()`.evidence, free - no scorer refactor), a per-dimension data-availability mask, the weights actually used, the score, and the decision/action. (b) `observation_labels` (migration 060): forward-horizon (2/5/10/20 trading days) outcomes computed only after horizon maturity by a nightly cron (`app/api/agents/label-maturation`) - `fwd_return` (cost-haircut adjusted), `benchmark_neutral_return` (SPY for US, NIFTY for India - alpha, not raw P&L), and MAE/MFE (the raw material for future risk-reward sizing). (c) `lib/learning/dataset.ts`: a pure, unit-tested walk-forward dataset builder with purge (drop train rows whose label window overlaps the test window) + embargo (skip a horizon's worth of time after each test window) - no leakage across folds. (d) `LearnerAgent's query_score_correlation` now reads the ledger FIRST (join by `signal_id`, not symbol - a second, standalone bug fixed in the same pass) and falls back to the legacy paper-trades path only while the ledger is thin (<10 rows); results are tagged source + an explicit caveat that this remains an INTERIM univariate method until Phase 2's validation engine replaces it. (e) The 10-year personal Robinhood trade history tools (`query_trade_decisions`, `semantic_search_decisions`) are explicitly quarantined - tagged role: "behavioral_evidence_only" in their return payload and in the system prompt - they may inspire hypotheses but can never satisfy `n_trades` or justify `update_signal_weight`. Reason: Every later phase (validation engine, calibrated sizing, genome, shadow A/B) optimizes against whatever dataset exists - fixing the dataset first is the only way those phases inherit a trustworthy foundation instead of compounding the existing bias. Capturing the FULL evidence blob (not just the 5 scores) costs nothing today and future-proofs Phase 3 feature discovery. No backfill: point-in-time features can't be honestly reconstructed for past signals, so the ledger accrues fresh from deploy - `signal_score_history` is untouched and keeps

powering the Score Tracker chart. Alternatives considered: Fix only the symbol->signal_id join and leave the rest (cheaper, but leaves selection bias and policy-P&L-as-alpha unaddressed - rejected, doesn't reach "statistically trustworthy"); backfill historical signals into the ledger (rejected - can't reconstruct honest point-in-time features after the fact, would silently mix real and reconstructed data); skip the walk-forward purge/embargo (rejected - overlapping swing-return windows would leak future information across folds, defeating the point of the ledger). Impact: Fully additive and guarded - no change to live trading behavior; the ledger simply accrues (~30-40 rows/day across both markets) until enough matured, horizon-aligned data exists for Phase 2's validation engine. LearnerAgent's correlation tool is immediately more honest (ledger-first, signal_id-joined, benchmark-neutral) even before Phase 2 lands. Migrations 059/060 must be applied manually (Supabase MCP permission-denied this session) before the ledger activates; until then, everything falls back to the pre-existing legacy path unchanged. Files/features affected: supabase/migrations/059_decision_observations.sql, supabase/migrations/060_observation_labels.sql, lib/research-agent.ts (observation write), app/api/agents/label-maturation/route.ts (new), lib/learning/dataset.ts (new), app/api/agents/learner/route.ts (ledger-first correlation + personal-history quarantine), scripts/run-agents.ps1, scripts/register-tasks.ps1, public/agent-diagrams/system-map.json (LEDGER node). Reversal cost: Low (two new additive tables + one new cron + a read-path preference in the learner; nothing else depends on the ledger yet)

Decision 34: Fix long-standing paper_order_events.signal_id type bug (bigint -> uuid) - every fill event had been silently failing since migration 034

Date: 2026-07-06 Status: Approved Category: Bug fix / Data integrity

Context: Reconnecting the Supabase MCP (after an earlier session's connector pointed at the wrong account) allowed direct schema introspection for the first time in several sessions. information_schema.columns showed paper_order_events.signal_id as bigint (as originally created in migration 034), while agent_signals.id and paper_trades.signal_id are both uuid. PaperTrader has always inserted signal_id: signal.id (a uuid string) into that bigint column - a type mismatch Postgres rejects outright. Live data confirmed the damage: paper_order_events had zero rows, ever, despite paper_trades having historical fills. Every fill's order-event insert was failing, and the paper-trade route's error handling (correctly, per Decision from the earlier Codex-review pass) treated it as a real failure - reverting the signal to pending and skipping the fill rather than recording a mis-tagged event. This predates all of this session's work; it is a bug in the original 2026-06 schema, not something introduced by the multi-market/India/learning-core changes. Decision: Apply alter table paper_order_events alter column signal_id type uuid using signal_id::text::uuid directly (migration 070_fix_paper_order_events_signal_id.sql), applied live via the reconnected Supabase MCP - lossless since the column had zero rows. Also applied migrations 060_observation_labels.sql and 069_portfolio_limits.sql (previously blocked by SQL-editor issues) in the same MCP session, and verified the fix end-to-end: restarted the stale next_start production server (it had been serving a build compiled before several of this session's commits), re-triggered ResearchAgent, and confirmed decision_observations now receives real rows with full 5-dimension features blobs per candidate. Reason: A schema-level type bug silently blocking every paper-trade audit-event write for the entire life of the project is a data-integrity issue, not a design question - no alternative considered beyond fixing the type. Doing it via direct MCP access (once available) rather than another round of manual-SQL-editor copy/paste avoided further exposure to the session's earlier Redis/RLS-dialog friction. Alternatives considered: Leave the column as bigint and stop inserting signal_id into paper_order_events (rejected - silently drops real audit-trail linkage instead of fixing the root cause); leave it for a future migration batch (rejected - it was actively causing every single paper fill's event log to be empty, worth fixing immediately since the fix was zero-risk with the table empty). Impact: paper_order_events will now actually populate on future fills - the audit trail this table exists for finally works. No behavior change to fills themselves (the JS-side resilience already handled the failure gracefully by reverting to pending; this just makes fills succeed instead of silently retrying forever). Confirms migrations 059/060/069/070 are all live on the FinanceOS Supabase project (dionkikgdm1aotvtbnfr). Files/features affected: supabase/migrations/070_fix_paper_order_events_signal_id.sql (new), paper_order_events table (live schema change). Reversal cost: Very low (single column type change on an empty table; trivially revertible)

Decision 35: Fail-closed Validation Engine + calibrated conviction sizing (Phase 2 learning-core)

Date: 2026-07-06 Status: Approved Category: Architecture / Learning / Risk

Context: Decision 33 (Phase 1) built the ground-truth decision ledger, but promotion of a LearnerAgent challenger to champion still required only human approval - no objective evidence that the challenger actually outperforms the champion out-of-sample. Similarly, sizing was still a flat `position_size_pct` and R:R a fixed profile percentage, regardless of a signal's actual conviction - the exact "flat bet size on every trade" ceiling identified when auditing risk-reward optimization earlier this session. Decision: (a) Validation Engine (`lib/validation/engine.ts`, migration 061 `validation_experiments`): deterministic, NO LLM. Replays champion vs challenger weights as a scoring-REPLAY against purged/embargoed walk-forward folds from the Phase 1 ledger; a challenger passes only if a 1000-draw moving-block bootstrap (fixed seed 42, reproducible) shows `p_improvement >= 0.80`, the paired-diff confidence interval floor is above a small epsilon, the overlap-adjusted effective sample size is `>= 12`, and the challenger wins `>= 3` of 5 folds. Every run - pass or fail - writes a `validation_experiments` row. (b) Fail-closed promotion gate: `promote_champion` (`app/api/strategies/versions/route.ts`) now returns HTTP 412 unless the challenger has a PASSED validation experiment attached; a `force_unvalidated: true` override exists but is journaled as a `governance_override` decision-journal entry, not the default path. (c) Auto-fire: LearnerAgent's `update_signal_weight` fires `/api/validation/run` (fire-and-forget) the moment it creates a challenger, so evidence is usually ready before a human even looks at the Strategy Registry; a manual "Validate" button exists on the Agents page too. (d) Calibrated conviction sizing (`lib/validation/calibration.ts`, migration 062 `model_artifacts`): a logistic-regression P(win) model (plain gradient descent, no new deps, standardized features, walk-forward calibration curve) replaces the raw uncalibrated `analyst_score` as PaperTrader's sizing input - half-Kelly (`lib/risk/sizing.ts`, built earlier this session) scaled by the ledger's MFE/|MAE| payoff ratio, capped at the existing `position_size_pct` as a ceiling. (e) Dynamic R:R (`lib/risk/percentiles.ts`): stop/target now come from the ledger's actual MAE/MFE percentile distribution (p25/p75) instead of the fixed 7%/20% profile constants - a signal's own `price_target/stop_loss` always wins. All of (d)/(e) are dormant (fall back to the pre-existing flat/fixed behavior) until 60+ matured observations exist per market, refit weekly (Fridays, before the learner). Reason: Human approval is valuable governance but is not evidence - without a locked evaluation protocol, a persuasive-but-statistically-wrong challenger could become the live scoring policy. Conviction-scaled sizing is what lets the system actually "bet bigger where the edge is real and confident" instead of the same flat size on a barely-qualifying and a max-conviction trade - directly answering the risk-reward-optimization question raised earlier this session. Everything here is deterministic and auditable (fixed seed, stored experiment rows, stored model coefficients) - no LLM makes a sizing or promotion decision. Alternatives considered: Trust human approval alone (rejected - Codex review's core finding: not evidence); use full Kelly instead of half-Kelly (rejected - ruin risk on a mis-calibrated model); let the Validation Engine auto-promote on pass (rejected - human stays the final live-capital gate per the project's locked design; validation adds evidence, doesn't remove the human); refit calibration continuously instead of weekly (rejected - unnecessary compute for a slow-moving model, weekly matches the learner's cadence). Impact: A challenger cannot reach champion status without walk-forward evidence (enforced server-side, not just by convention). Paper-trade sizing and stops/targets become conviction- and outcome-distribution-aware once enough ledger data exists - fully dormant and risk-free until then (falls back to today's exact flat/fixed behavior). New weekly cron fit-calibration (Fridays 4:45 PM ET, before the 5:00 PM learner). Files/features affected: `supabase/migrations/061_validation_experiments.sql`, `062_model_artifacts.sql`, `lib/validation/engine.ts`, `lib/validation/calibration.ts`, `lib/risk/percentiles.ts`, `app/api/validation/run/route.ts`, `app/api/validation/fit-calibration/route.ts`, `app/api/strategies/versions/route.ts` (fail-closed gate), `app/api/agents/learner/route.ts` (auto-fire), `app/api/agents/paper-trade/route.ts` (Kelly sizing + dynamic R:R), `components/dashboard/AgentsPage.tsx` (Validate button + pass/fail badge), `scripts/run-agents.ps1` / `register-tasks.ps1`, `public/agent-diagrams/system-map.json` (VALIDATE node, PROMOTE/TRADER updated). Reversal cost: Low-medium (three new additive tables + a gate on one existing route; the gate can be disabled by reverting the `promote_champion` check without touching the engine itself)

Decision 36: Execution Gateway (Alpaca, paper-stage) + fix of a discovered pre-existing Trade Queue bug

Date: 2026-07-06 Status: Approved Category: Architecture / Execution / Bug fix

Context: Robinhood live trading has always been manual by design (Decision - see REALORDER in system-map: no public order API, so an approved proposal generates a paste-into-Claude-MCP command). This was flagged earlier this session as the one remaining honest gap toward a real "algo execution" platform. While wiring the paper-stage gateway's UI (a "Send to Alpaca" button on Smart Money's Trade Queue tab), a pre-existing bug was discovered: the Trade Queue UI (`app/dashboard/smart-money/page.tsx`) read from a `trade_queue` table (`uuid` ids, status `pending_approval`), while `/api/agents/trader's Approve/Reject` actions - and the only table `trade_proposals` inserts real proposal rows into - use `trade_proposals` (`bigint` ids, status `pending_review`). `handleApprove's parseInt(tradeId, 10)` on a `uuid` silently produced `NaN/null`, so Approve never actually worked against real data. Both tables were confirmed empty in production, so no real approval has ever been lost - this is a latent bug, not a live-data incident. Decision: (a) Build the Execution Gateway paper-stage: `lib/brokers/alpaca-orders.ts` (`submit/get/cancel` against Alpaca's REST API, reusing the existing vault-key pattern), migration `068_broker_orders` (typed order lifecycle: `pending_submit` -> `submitted/partially_filled` -> `filled/canceled/rejected/error`), `app/api/broker/orders` (POST - human-click only, never cron; live env additionally gated on `strategy_config.trading_enabled`), `app/api/broker/orders/sync` (cron, every 30 min market hours - polls Alpaca + reconciles positions, alerts on mismatch). A "Send to Alpaca (paper)" button appears only for `status='approved'` proposals in the Trade Queue history table. (b) Fix the discovered bug in the same change: repointed `app/dashboard/smart-money/page.tsx's` query from `trade_queue` to `trade_proposals` (aliased columns so the existing UI code needs no further changes), and corrected the `pending/decided` split in `SmartMoneyPage.tsx` to check `pending_review` (the real status) instead of `pending_approval`. Reason: The Gateway's UI needed a working approval flow to attach to - building it on top of the already-broken `trade_queue` read would have shipped a second broken feature. Fixing the underlying table mismatch was the only way to make Approve/Reject AND the new Alpaca button actually functional. This is exactly the kind of thing this session's "if you notice something worth fixing that would bloat the current change, flag it" guidance is for - except it directly blocked correct completion of the task at hand, so it was fixed inline rather than deferred. Alternatives considered: Build the Alpaca button against the broken `trade_queue` table to match existing (broken) UI code (rejected - would ship a feature that provably cannot work); leave the bug and flag it as a separate follow-up (rejected - the Gateway UI literally cannot be verified without this fix, and both tables being empty made the fix zero-risk); keep Robinhood on the manual path with no typed alternative at all (rejected - this was the explicitly flagged gap toward "algo execution platform"). Impact: Approve/Reject on the Trade Queue tab now actually operate on real data going forward. Paper Alpaca orders can be sent from an approved proposal with one click, tracked through their full lifecycle, and reconciled against Alpaca's own reported positions every 30 min. Live-env orders remain fully gated (`trading_enabled` + explicit env param) and were NOT tested this session - only the paper path was built and is intended for use; live requires its own explicit go-ahead later. Robinhood's manual path is unchanged (no public API exists to replace it). Files/features affected: `supabase/migrations/068_broker_orders.sql`, `lib/brokers/alpaca-orders.ts`, `app/api/broker/orders/route.ts`, `app/api/broker/orders/sync/route.ts`, `app/dashboard/smart-money/page.tsx` (table fix), `components/dashboard/SmartMoneyPage.tsx` (status fix + Alpaca button), `scripts/run-agents.ps1` / `register-tasks.ps1`, `public/agent-diagrams/system-map.json` (GATEWAY node, REALORDER updated). Reversal cost: Low (one new additive table + two new routes; the UI fix is a net bug fix with no downside to reverting since it restores intended-but-never-working behavior)

Decision 37: Controlled evolution - strategy genome, feature registry, shadow decisions, regime features, governance rewiring (Phase 3 learning-core)

Date: 2026-07-06 Status: Approved Category: Architecture / Learning

Context: Phases 1-2 gave the learning loop a trustworthy dataset and a fail-closed evidence gate, but the LEARNABLE SURFACE was still only the 5 top-level scoring weights - the exact "reweighting 5 fixed dials can only remix existing assumptions" ceiling the Codex review identified. Nothing could discover a new feature, evolve the entry

threshold/horizon/exit policy, or observe a challenger against live opportunities before risking paper capital. Decision: (a) Typed strategy genome (migration 063, `strategy_versions.genome.jsonb` + `agent_signals.genome_hash`): a canonical, bounded manifest (entry threshold, horizon, exit family - reusing Phase 2's ledger-percentile stop/target math, sizing mode, universe, regime router) with hard search-domain bounds enforced in code (`lib/validation/genome.ts`), not just documented. Genome-less rows score exactly as before. (b) Feature registry (migration 064, `feature_registry`): a new `propose_feature` learner tool stores a machine-readable spec; the formula is NEVER executed as code - only interpreted by a from-scratch whitelisted-grammar tokenizer/parser/evaluator (`lib/validation/feature-compiler.ts`: + - * / and log/abs/min/max/lag only, no `eval()`, no dynamic code path). A weekly job (`app/api/validation/feature-check`) computes out-of-sample Spearman rank-IC (Fisher-z significance) across walk-forward folds and promotes proposed->quarantined->active on $|IC| \geq 0.03$ & $p < 0.1$ across 2+ of 3 folds, auto-retiring actives whose rolling IC decays below 0.01 for 3 checks. (c) Shadow decisions (migration 065, `shadow_decisions`; extends `strategy_versions.state` to allow `shadow_paper`): ResearchAgent records what up to 3 shadow-state versions would decide on every candidate - pure scoring replay, no fills/cash - alongside the champion's real decision. Off by default; a bounded `strategy_config.exploration_enabled` flag (default false) is reserved for future auto-promotion of at most one challenger/week. (d) Regime features (`lib/validation/regime.ts`): point-in-time trend (50d-vs-200d MA) and realized-vol tercile vs the market benchmark (SPY/^NSEI), appended to every observation's `features.regime`. No hard bull/bear switch - these are just numbers available for future interaction terms in the calibration fit. (e) Governance rewiring: the learner's auto-guard now trips on CHAMPION HEALTH (>15% drawdown from a 90-day NAV peak, OR calibration-decile drift >0.25, OR data-availability <60% over the last 10 observations) instead of a raw 3-run win-rate streak - win-rate ignores payoff asymmetry and doesn't reflect real risk. Confirmed the guard only ever gated `update_signal_weight`; hypothesis-writing, validation, and shadow research were already unaffected. Reason: Each piece targets a specific, named ceiling from the original review: the genome answers "can the system evolve more than 5 weights," the feature registry answers "can it discover new signals," shadow decisions answer "can a challenger be observed live before risking capital," regime features answer "can scoring be regime-aware without a brittle switch," and the governance rewire answers "is the safety mechanism measuring the right risk." All five are additive and default-inert - nothing here changes today's live scoring/sizing/promotion behavior until a human deliberately proposes a genome change, a feature clears IC promotion, or a version is explicitly put into `shadow_paper`. Alternatives considered: Let the LLM write and run arbitrary feature code (rejected outright - no code-execution surface, ever, per this project's LLM/deterministic-boundary rule); auto-promote validated challengers into shadow without human action (rejected - exploration must stay bounded and opt-in, matching the project's human-in-the-loop mandate); keep the win-rate auto-guard (rejected - Codex review + this session's own audit found it measured the wrong thing). Impact: The learnable surface now includes entry/horizon/exit/sizing/universe/regime, not just 5 weights. Feature discovery has a real (if narrow, whitelisted) path from LLM hypothesis to validated, IC-promoted signal. Shadow decisions accrue silently and cost nothing until a version is placed in `shadow_paper`. Auto-guard now reflects realized risk, not a noisy short-run win-rate. Nothing here is live/active by default. Files/features affected: `supabase/migrations/063_strategy_genome.sql`, `064_feature_registry.sql`, `065_shadow_decisions.sql`, `lib/validation/genome.ts`, `feature-compiler.ts`, `feature-check.ts`, `regime.ts`, `app/api/validation/feature-check/route.ts`, `lib/research-agent.ts` (shadow recording + regime features), `app/api/agents/learner/route.ts` (`propose_feature` tool + governance rewire), `scripts/run-agents.ps1` / `register-tasks.ps1`, `public/agent-diagrams/system-map.json` (GENOME/REGISTRY/SHADOW nodes). Reversal cost: Low (five additive schema pieces; genome/registry/shadow are all opt-in and read-gated, so removing them affects nothing already in production use)

Decision 38: Risk posture & goal tracker - goals are measured, postures are applied; return targets are never agent parameters

Date: 2026-07-06 Status: Approved Category: Architecture / Risk

Context: The posture/goals spec was queued to close two gaps: (1) the conservative/balanced/aggressive profiles didn't scale kill-switch thresholds or exit hysteresis, so an aggressive book with default -5%/20%/40% thresholds tripped by design; (2) there was no way to time-box a more aggressive posture with an automatic revert, and no way to track a stated return goal without that goal leaking into agent sizing/thresholds. Decision: (a) Profile rollup: `strategy_config` gains

ks_daily_loss_pct, ks_drawdown_pct, ks_accuracy_pct, exit_hysteresis, populated per-profile in both the PROFILES map (app/api/settings/risk-profile/route.ts) and checkKillSwitches/position-monitor reads (resilient - absent/null falls back to the original hardcoded -5/20/40/15 defaults). Settings shows a muted note when a promoted champion overrides scoring weights. (b) Time-bound postures: strategy_config.posture/posture_expires_at/base_risk_profile - applying a posture saves the current profile as the revert target, applies that posture's dials with an expiry, and journals to decision_journal. The research cron checks and auto-reverts expired postures before each run (resilient no-op pre-migration). (c) Goal tracker: new trading_goals table (explicitly commented "READ BY UI ONLY - never an agent input"), /api/goals computes required-vs-realized daily-return feasibility as a pure function (lib/goals/feasibility.ts) and auto-flips status to achieved/missed. A dashboard GoalCard shows progress, an on-track marker, and an honest feasibility sentence. A return target is never wired into sizing/threshold - cranking dials toward a target only adds variance (gambler's ruin) and would teach the learner that luck is skill. Reason: Kill-switches and exit hysteresis that don't scale with the chosen risk profile fight the profile's own intent. Postures need a safety valve (auto-revert) so a temporary aggressive stance can't be forgotten and left permanent. Goals are valuable as an honest progress readout but dangerous as a control input - locking that boundary in code (a commented, agent-unread table) prevents future drift toward "goal-seeking" agent behavior. Alternatives considered: Wire goal targets into position sizing directly (rejected - locked design rule, see spec); hardcoded bull/bear regime switching for postures (rejected - already covered by Phase 2 Kelly sizing + Phase 3 regime router, no duplicate hardcoded switch). Impact: Aggressive profiles no longer trip on thresholds sized for balanced. A posture can be applied and will self-revert without manual follow-up. Users get a feasibility-checked goal readout without any risk of it silently becoming an agent input. Files/features affected: lib/kill-switches.ts, app/api/agents/position-monitor/route.ts, app/api/settings/risk-profile/route.ts, app/api/agents/research/cron/route.ts (posture auto-revert), app/dashboard/settings/page.tsx (posture UI + champion note), lib/goals/feasibility.ts, app/api/goals/route.ts, components/dashboard/GoalCard.tsx, components/dashboard/DashboardHome.tsx. Reversal cost: Low (all additive columns/tables, resilient fallbacks throughout; removing the goal card or posture UI has zero effect on trading behavior since neither ever fed an agent)

Decision 39: 30-day agent-run calendar, broker adapter registry (swap/multi-broker), fortnightly model-freshness checker

Date: 2026-07-06 Status: Approved Category: Architecture / Ops

Context: Three visibility/flexibility gaps closed together: (1) no at-a-glance view of which agents ran/failed/were missing over the last month; (2) the Execution Gateway called lib/brokers/alpaca-orders.ts directly - swapping to Kite/E*TRADE/any other broker meant editing routes, and broker_orders.broker had a column nothing routed on; (3) no visibility into whether a newer or soon-to-be-deprecated LLM exists for an already-integrated provider. Decision: (a) Agent calendar (/api/agents/calendar + AgentCalendar.tsx, rendered at the top of the dashboard): aggregates agent_runs over 30 days, classifies each (date, agent) cell ok/error/partial/skipped/missing. The expected-agent-set is derived from what's actually registered in pg_cron (no strategy_config.market_focus column exists - the spec's assumption was stale; used real schedule state instead). (b) Broker adapter registry: lib/brokers/adapter-types.ts (BrokerAdapter interface - distinct from the pre-existing types.ts, which is holdings-aggregation types, not the order-execution contract) + lib/brokers/registry.ts (getBroker/listBrokers/getActiveBroker, resilient fallback to alpaca/kite) + adapters wrapping the existing alpaca-orders.ts and lib/kite.ts functions (added kiteDelete for cancellation). strategy_config.active_broker_us/active_broker_india (default alpaca/kite) drive routing; /api/broker/orders and /api/broker/orders/sync now go through the registry only - the sync loop iterates distinct brokers present in open orders, so multiple brokers can have in-flight orders simultaneously. Settings gained a per-market broker dropdown with a configured/not-configured badge. (c) Model freshness (/api/models/check, weekly pg_cron): diffs agent_config's assignments against each provider's live model list (Anthropic/Groq/DeepSeek, each fail-soft), flags newer-available and possibly-deprecated models into model_check_results + an info/warn alert. Never auto-switches - a dashboard card just surfaces findings; the human changes the assignment in the existing agent-config picker. Reason: Silent scheduling failures are worse than loud ones - the calendar makes "PC off/asleep at trigger time" visible at a glance instead of requiring a manual agent_runs query (exactly what this session's investigation had to do

manually). The broker registry removes a permanent one-broker lock-in with zero added complexity to the safety gates (which sit above the adapter layer, untouched). Model freshness is informational-only by design - auto-switching models is a bigger decision than a background job should make. Alternatives considered: Gate the calendar's expected-set on `strategy_config.market_focus` per the spec's original text (rejected - that column doesn't exist; used the actual `pg_cron` schedule as ground truth instead); auto-switch to a newer model when found (rejected - model choice affects cost/quality/behavior and needs a human in the loop, matching the project's approval-gate philosophy). Impact: Missed/failed cron runs are now visible without a manual DB query. Adding a future broker (E*TRADE, etc.) is one file + one registry line, zero route changes. Deprecated or newer models surface proactively instead of being discovered when an agent silently starts failing. Files/features affected: `app/api/agents/calendar/route.ts`, `components/dashboard/AgentCalendar.tsx`, `components/dashboard/DashboardHome.tsx`; `lib/brokers/adapters/{alpaca,kite}.ts`, `lib/kite.ts` (added `kiteDelete`), `app/api/broker/orders/route.ts`, `app/api/broker/orders/sync/route.ts`, `app/api/brokers/route.ts`, `app/dashboard/settings/page.tsx` (broker dropdown); `app/api/models/check/route.ts`, `components/dashboard/ModelFreshnessCard.tsx`, `components/dashboard/AgentsPage.tsx`; migrations `model_check_results`, `broker_registry_config`; `pg_cron` job `kairos-model-check`. Reversal cost: Low (additive schema + a routing indirection; removing the registry would mean reverting the two Gateway routes to direct `alpaca-orders` calls, a small diff)

Decision 40: Research Journal - daily per-symbol funnel trail + learning-evolution view

Date: 2026-07-06 Status: Approved Category: Architecture / Observability

Context: "Why did the agent do/not do X today" required manually cross-referencing 4+ tables by hand - no single place showed the full chain (score -> screener bucket -> research pass/fail -> Portfolio Constructor decision -> fill) for a symbol on a given day, and nothing showed whether the learning loop is actually improving over weeks. Also discovered while building this: `decision_observations.signal_id` had been hardcoded null on every insert since Phase 1 shipped (comment read "agent_signals insert doesn't return id today - Phase 2 wires it") - the join key the whole ledger was designed around was never actually wired. Decision: (a) Fixed the `agent_signals` insert in `lib/research-agent.ts` to capture its returned id and thread it into `decision_observations.signal_id` (previously always null). (b) `runScreener` now returns each symbol's bucket (momentum|value) instead of a flattened list, recorded into `decision_observations.features.screener` with the fixed criteria set for that bucket. (c) New append-only `pipeline_stage_events` table (`signal_id`, `stage`, `outcome`, `reason`, `detail`) - written by `ResearchAgent` (`stage='research'`), Portfolio Constructor's decision point and the fill/reject paths in `paper-trade/route.ts` (`stage='portfolio_constructor'`, `stage='execution'`), all fail-soft (never blocks the decision they describe). (d) New `feature_registry_history` table logging every status transition (written from `feature-check` and the learner's `propose_feature` tool). (e) Two new routes (`/api/agents/research-journal`, `.../evolution`) and one new page (`/dashboard/research-journal`, Daily Funnel + Evolution tabs) - the Evolution tab explicitly refuses to draw a trend from fewer than 3 data points rather than imply false precision on thin history. Reason: Instrumenting at the source (one small insert at each existing decision point) is the only way to get an honest trail - reconstructing "why" after the fact from final-state tables is lossy (a Portfolio-Constructor rejection before a `trade_proposals` row ever existed left zero trace before this). The dormant `signal_id` bug would have silently undermined this and any other future join through `decision_observations` had it not been caught while scoping this feature. Alternatives considered: Build the journal as a pure read-only aggregation over existing tables (rejected - the join key didn't work and downstream rejection reasons weren't captured anywhere); auto-alert on funnel anomalies as part of this feature (rejected - out of scope, that's the existing stale-run/0-signal alerting, this is for understanding why, not re-alerting). Impact: Every symbol scored going forward has a queryable, honest stage-by-stage trail. `decision_observations.signal_id` now actually joins to `agent_signals/paper_trades` for the first time. Feature-registry promotions/retirements now have a timeline, not just a snapshot. Files/features affected: `lib/research-agent.ts`, `app/api/agents/paper-trade/route.ts`, `app/api/validation/feature-check/route.ts`, `app/api/agents/learner/route.ts` (`propose_feature`),

app/api/agents/research-journal/route.ts, app/api/agents/research-journal/evolution/route.ts, app/dashboard/research-journal/page.tsx, components/dashboard/DashboardShell.tsx (nav); migrations pipeline_stage_events, feature_registry_history; features/research-journal/FEATURE_ARCHITECTURE.md. Reversal cost: Low (additive tables/fields, fail-soft writes throughout; the signal_id fix is a pure improvement with no reversal case)

Decision 41: Renormalize scoring weights across applicable+available dimensions instead of always applying the fixed 5-way split

Date: 2026-07-06 Status: Approved Category: Architecture / Scoring methodology

Context: Reviewing the Research Journal surfaced that ETFs (IAU, GDX, etc.) were being scored using the same fixed 30/25/20/15/10 (fundamental/technical/sentiment/macro/insider) weighting as individual stocks, even though fundamental and insider are structurally meaningless for an ETF (no company financials, no insiders - scoreFundamentals/normalizeInsiderScore already returned a flat neutral baseline for these cases, not a real signal). Separately, macro was frequently unavailable (Alpha Vantage rate-limited) and still counted at its full fixed weight against a neutral-50 default. Also found and fixed in the same pass: scoreSentiment() checked for field names (bullish_pct/bull_pct) that never matched the real StockTwits/AV data shape (stocktwits_bullish_pct, av_news_sentiment) - sentiment was silently always neutral-50 regardless of real (sometimes strongly bullish) data. Decision: lib/research-agent.ts's weighted score now classifies each of the 5 dimensions as included or excluded before computing analystScore: inapplicable (fundamental/insider for ETFs - structural, not data-dependent) or unavailable (macro rate-limited, no sentiment data this run - scores.dataQuality flags). When 2-4 dimensions are included, weights are renormalized to sum to 1.0 across only those; below 2 included dimensions, falls back to the original fixed-weight behavior (renormalizing a score to 100% of one thin signal is riskier than the old diluted approach). The applied weights and included-dimension list are recorded into decision_observations.features.weighting and weights_used (now the actually-applied weights, not the base profile split), surfaced in the Research Journal funnel view. Reason: A fixed weight applied against a fabricated neutral-50 default silently penalizes candidates for a data gap that has nothing to do with their real quality - an ETF was never going to have real insider data, and that shouldn't cost it 10% of its score every single run. Renormalizing across what's actually knowable is more honest than diluting with placeholders. Alternatives considered: A per-instrument-type weight profile (e.g. a dedicated ETF weight set) - rejected as a partial fix that doesn't handle the *unavailable* case (macro rate-limiting affects stocks too) and adds another hardcoded table to maintain; leaving the fixed weights and only fixing the sentiment bug - rejected, the ETF fundamental/insider exclusion is a distinct, real issue independent of the sentiment bug. Impact: ETF and thin-data scores should shift upward (no longer penalized by inapplicable/unavailable dimensions) and become more honest about what evidence actually drove them - visible per-symbol in the Research Journal. Files/features affected: lib/research-agent.ts (renormalization logic + weighting in features), lib/data/scores.ts (scoreSentiment field-name fix), app/api/agents/research-journal/route.ts + app/dashboard/research-journal/page.tsx (surface weighting). Reversal cost: Low (renormalization only changes weight distribution when dimensions are excluded; degenerate case falls back to prior behavior exactly)

Decision 42: Ultra-review fix pass - same-day kill-switch bug, screener bucket bias, Alpaca status regression, currency-blind outcome classification, missing migration files

Date: 2026-07-06 Status: Approved Category: Bug fix batch / Process

Context: Ran a 10-angle multi-agent adversarial review (line-by-line, removed-behavior, cross-file, language-pitfall, wrapper-correctness, reuse, simplification/efficiency, altitude, conventions) plus a deeper follow-up pass over the full session's diff, specifically because the density of silent defects found earlier in the session (dormant signal_id null, sentiment field-name mismatch, missing profiles.market_focus) raised the question of whether the core scoring/learning loop could be trusted at all. Decision: Fixed, in priority order: (1) checkKillSwitches's daily-loss check

compared todayPerf - a row only written at the END of the same paper - trade run that calls it - against yesterday, so the same-day circuit breaker could never actually fire same-day; now compares live current NAV (already in scope) against yesterday's close. (2) runScreener's momentum-then-value insertion order let momentum silently crowd out every value-bucket candidate whenever momentum alone returned 6+ hits; now interleaves both buckets round-robin before the 6-slot cap - a pre-existing bias, not introduced this session, but now visible via the Research Journal. (3) getAlpacaOrder's unmapped-status fallback was accidentally changed from passing through the raw status string to undefined during tonight's type-error fix, silently hiding any order in one of Alpaca's ~8 unmapped statuses from the sync job - extended the map to cover all of them. (4) Posture Object.assign ran after per-field overrides in the same PATCH request, silently clobbering an explicit override; reordered so explicit fields always win, and a cancel-when-nothing-active request now returns no_op:true instead of a silent 200. (5) getActiveBroker swallowed all errors (not just missing-column) and now logs loudly on any non-schema failure. (6) Broker sync's reconciliation only ever checked Alpaca holdings - Kite/India fills had zero reconciliation path; added the missing check. (7) None of ~10 schema changes applied live via the Supabase MCP tool this session had a committed migration file - backfilled as supabase/migrations/072-080. (8) PROFILES/PROFILE_DIALS/kill-switch fallback constants were three independent hand-typed copies of the same numbers - consolidated into lib/risk-profiles.ts. (9) Win/loss/breakeven classification used a fixed absolute ?\$0.5 band regardless of currency, meaningless for India (?) - switched to a relative ?0.1% band via a shared lib/trade-outcome.ts helper. (10) Several smaller fixes: weight-renormalization all-zero-weight edge case now falls back to an equal split instead of silently zeroing every score; a manual-close qty-mismatch check now logs loudly instead of silently miscrediting cash; lib/schedule.ts's model-check entry had contradictory days/time metadata; Research Journal's evolution "agreement %" was renamed to wouldEnterPct since no actual champion-agreement comparison is computed; PROJECT_BUILD_LOG.md hadn't been updated since 2026-06-29 despite 4 major decisions shipping. Reason: A safety mechanism that structurally cannot fire when it's supposed to, and a screener that structurally excludes half its candidate pool, are both worse than a slow feature - they look like they work (return 200, log nothing) while silently not doing their job. Fixing the process gap (no committed migrations for live schema changes) matters as much as the code bugs, since it's the same root-cause class: state that's real but invisible. Alternatives considered: Leaving lotOutcome() in lib/paper/lot-math.ts alone rather than also converting it to a relative band - it's dead code (only referenced by its own test, no production caller), so it was left as-is rather than risk touching a passing test suite for a non-live code path; noted as a follow-up cleanup, not urgent. Impact: The daily-loss kill switch can now actually trip same-day. The screener no longer structurally favors momentum over value. Kite/India orders are now reconciled. All of tonight's schema has a committed, reproducible migration file. Files/features affected: lib/kill-switches.ts, lib/research-agent.ts, lib/brokers/alpaca-orders.ts, lib/brokers/registry.ts, lib/brokers/adapters/kite.ts, app/api/broker/orders/sync/route.ts, app/api/settings/risk-profile/route.ts, app/api/agents/research/cron/route.ts, lib/risk-profiles.ts (new), lib/trade-outcome.ts (new), app/api/paper-positions/close/route.ts, app/api/agents/position-monitor/route.ts, app/api/goals/route.ts, components/dashboard/GoalCard.tsx, lib/schedule.ts, app/api/agents/research-journal/{route, evolution/route}.ts, supabase/migrations/072-080, PROJECT_BUILD_LOG.md. Reversal cost: Low - every change either restores previously-intended behavior (kill switch, Alpaca status, posture override) or is a pure additive safety net (qty-mismatch logging, zero-weight fallback).

Decision 43: Codex adversarial review fix pass - evidence-availability semantics, shared scoring contract, Theme Scout ticker validation, Kelly no-edge sizing, LearnerAgent evidence-binding

Date: 2026-07-06 Status: Approved Category: Bug fix batch / Architecture rule

Context: User asked for an independent LLM (Codex, run outside this session) adversarial deep-dive of the core scoring->paper-trade->learning loop, explicitly worried that missing/rate-limited external data (Alpha Vantage, Yahoo, Massive) could silently corrupt research and learning signals - "if those are wrong everything is wrong in the app." Codex's review (CODEX_AGENT_SYSTEM_REVIEW.md) scored Theme Scout 25/100, ResearchAgent scoring 50/100, PaperTrader 70/100, LearnerAgent 58/100, and flagged its own limitation: its Supabase MCP connection pointed at a different project, so

live-schema claims in its report were unverified. Every finding was independently re-verified against current code and the real live schema (via the project's own Supabase MCP) before being acted on; several of Codex's specific claims were stale (referencing pre-Decision-41 code) or factually wrong against the live schema and were rejected or reframed rather than blindly fixed. Decision: Fixed, in the user's specified priority order: (1) Evidence availability - `fetchSocialSentiment()/scoreInsider()` never return null even on total fetch failure, so `!!socialResult/!!insiderResult` always read "available"; added `real_has_data/available` flags. India's Yahoo-mapped fundamentals always set `Symbol` even when every real field is empty; availability now requires 2+ of 5 real fields present, both markets. (2) Macro scoring queried `macro_signals` (per-indicator rows, no `danger_score/regime` columns) instead of `macro_regime` (migration 028) - fixed to query the right table; this had silently zeroed out macro's contribution to every score since Decision 41 shipped renormalization. (3) Shared scoring contract - new `lib/scoring/weighted-score.ts` (`computeWeightedAnalystScore`) used by both `lib/research-agent.ts` and `lib/validation/engine.ts` (fed by a new `availability_mask` select in `lib/learning/dataset.ts`), replacing the Validation Engine's independent fixed-weight/coalesce-to-50 replica; also fixed a drift bug where the persisted `availability_mask` was a second, disagreeing computation from the one that actually drove live weighting (disagreed for ETFs). Added forced-abstain (`direction: "neutral"`) when fewer than 2 dimensions are included or the LLM thesis parse fails - except never overriding a held position's real exit ("short") signal, per the locked CLAUDE.md SELL-capability rule. (4) Theme Scout - backfilled migration 081 for watchlist columns that were live but never committed and relaxed `user_id` to nullable (matching live schema; Codex's not-null claim was stale); Theme Scout now sets `user_id` to the app's profile owner; added a deterministic `tickerExists()` Alpha Vantage OVERVIEW check before ANY candidate (LLM-suggested or AV-theme-confirmed) enters the watchlist, quarantining failures instead of trusting the LLM's ticker list; upsert errors now logged + alerted. (5) `lib/risk/sizing.ts`'s `positionSizePct()` clamped a no-edge/negative Kelly result to the floor size instead of zero - fixed to return 0, so a calibrated "no edge" verdict can no longer still open a real position; added explicit `Number.isFinite()` guards on `sizedPct/fillPrice/maxSpend/qty/totalCost` in the paper-trade route before the fill RPC (a NaN previously passed every `<=<` guard silently); added the 5 missing portfolio-limit columns to the route's config select (previously never fetched, silently defaulting); fail-closed on the legacy non-transactional fallback path in production if `execute_paper_fill` is ever missing there (confirmed present live today - a defensive hardening, not an active bug). (6) `LearnerAgent` - `update_signal_weight` previously gated on the LLM's own claimed `n_trades/confidence` with nothing binding them to an actual correlation result; extracted `computeScoreCorrelation()` as a shared helper and made `update_signal_weight` recompute it server-side for the target dimension, refusing to mutate at all when only the weaker `paper_trades_fallback` (not the observation ledger) is available, gating `N>=10` and an effect-size floor on the server's own number, and requiring the correlation's sign to agree with the direction of the proposed change. Reason: The app's entire premise is that research/learning are grounded in real fetched evidence, not LLM-plausible defaults. A neutral-50 fallback that's indistinguishable from real balanced evidence, a macro dimension that's been silently inert since the last major scoring change, a validation gate that doesn't replay the same math as production, and a learner tool that trusted its own caller's evidence claims are all instances of the same failure mode: state that looks correct (200 OK, no errors) while not doing its job. Fixing the shared contract once (weighted-score module) is more durable than re-patching each of the two call sites whenever the renormalization rule changes again. Alternatives considered: A full theme-relevance re-confirmation (require AV news-sentiment mention, not just existence) for Theme Scout candidates was considered but rejected as disproportionate - it would gut the LLM-driven theme-discovery step CLAUDE.md's design explicitly wants, when the real risk (hallucinated/non-existent tickers) is fully addressed by a cheap existence check. Requiring `update_signal_weight`'s evidence to come from a cryptographically-signed token tied to a specific `query_score_correlation` call (Codex's suggested mechanism) was rejected in favor of simple server-side recomputation - same trust guarantee, no new schema/infra. Impact: Missing/failed external data can no longer masquerade as neutral real evidence in scoring. MacroSentinel's weekly regime assessment actually reaches the analyst score again. A promoted challenger is now validated against the exact live scoring rule. Theme Scout can no longer seed the research universe with a nonexistent ticker. A "no edge" calibrated sizing result can no longer still open a position. `LearnerAgent` weight proposals are now bound to a server-verified correlation, not a self-reported one. Files/features affected: `lib/social-sentiment.ts`, `lib/research-agent.ts`, `lib/data/scores.ts`, `lib/scoring/weighted-score.ts` (new), `lib/validation/engine.ts`, `lib/learning/dataset.ts`, `app/api/agents/theme-scout/route.ts`, `lib/risk/sizing.ts`, `app/api/agents/paper-trade/route.ts`, `app/api/agents/learner/route.ts`, `tests/sizing.test.ts`, `tests/walk-forward.test.ts`,

supabase/migrations/081_watchlist_theme_scout_columns.sql (new). Reversal cost: Low-Medium - the shared scoring module and abstain-on-thin-evidence logic change live scoring behavior (a symbol that previously scored via 5 diluted-neutral dimensions may now abstain instead), which is the intended fix but worth watching for a few days of live signals; everything else is additive (flags, guards, logging) or restores intended behavior.

Decision 44: Market-switcher wiring gap, nav clarity badges, India cron realignment, Windows Scheduler decommission

Date: 2026-07-06 Status: Approved Category: UI/UX rule / Technical implementation rule / Bug fix

Context: User asked why it's hard to tell from the UI which left-nav pages actually respect the global US/India switcher, and separately asked whether India's agents fire at the right time relative to NSE market hours. Also flagged that migrating scheduling to Supabase pg_cron earlier this session might have left the old Windows Task Scheduler jobs still running in parallel. Decision: (1) Windows Scheduler - confirmed all 15 \Kairos\ jobs were still Ready, pointed at localhost:3000 (the same routes pg_cron now hits in the cloud); disabled all 15 with user approval - pg_cron is now the sole scheduler. (2) Market-switcher audit - surveyed all ~20 dashboard routes; only 9 actually read useMarket() despite the switcher's own tooltip claiming "across the app." Wired Watchlist, Score Tracker, and Agent History to the global switcher (API ?market=filters + component changes); corrected lib/market-support.ts's registry, which was overclaiming "full" country support on Agents, Decision Journal, and Morning Briefing - none of which actually filter by market today. (3) Nav clarity - added a market-scope badge directly in the left-nav sidebar (DashboardShell.tsx), reading the same market-support.ts registry that previously only surfaced at the bottom of each page's content (which is exactly why it was hard to tell from the UI). Distinguishes by-design single-market pages (???? flag) from known gaps (? US only). (4) India NSE status pill - added a second market-status pill (top bar, next to the existing US one) showing NSE open/pre-open/closed on real IST hours (9:15 AM-3:30 PM) plus the same fixed-date holiday list the research cron already uses; both pills now explicitly flag-labeled. Shown only when India is enabled. (5) India cron timing - found kairos-research-india fired at 16:30 IST, an hour AFTER NSE's 15:30 close, meaning India's research (scores off the latest candle) and its internally-chained paper-trade fill (fetches a "live" quote) both landed on the exact same closing print with no realistic decision-to-fill gap - unlike the US, which fires pre-open (scores off yesterday's close) and fills on a genuinely live post-open quote. Realigned India to fire at 9:30 AM IST (15min after the 9:15 open) instead, via migration 082 (cron.alter_job); position-monitor-india was already correctly timed post-close and left unchanged. Reason: A market-support badge that overclaims "full" coverage is worse than no badge - it actively hides the exact gap the user is trying to find. A live-trading cron that scores and fills on the identical closing tick isn't technically broken (no error, no crash) but defeats the purpose of having a fill step at all - there's no independent price discovery between the two, which is the same "looks fine, silently wrong" failure class as Decision 43's bugs. Alternatives considered: Moving India's research pre-open (mirroring the US exactly) was considered and rejected - free Yahoo NSE data doesn't reliably carry real pre-market price movement the way US extended-hours data does, so a pre-open fire would still land the chained paper-trade fill on a stale/frozen quote. Firing just after the 9:15 open instead gives the same "score off yesterday's close" property while ensuring the chained fill gets a genuinely live intraday print. Impact: Windows Scheduler can no longer duplicate pg_cron's API/LLM spend. Watchlist, Score Tracker, and Agent History now genuinely follow the header switcher. The left nav honestly shows which pages do and don't respect it. India's market-status is visible without navigating anywhere. India's paper-trade fills no longer happen on the exact same tick as the scoring decision that triggered them. Files/features affected: components/dashboard/DashboardShell.tsx, lib/market-support.ts, components/dashboard/WatchlistPanel.tsx, app/api/watchlist/route.ts, app/dashboard/scores/page.tsx, app/dashboard/agents/history/page.tsx, app/api/agents/history/route.ts, lib/schedule.ts, supabase/migrations/082_india_research_cron_realign.sql (new), 15 Windows Task Scheduler jobs under \Kairos\ (disabled, not deleted). Reversal cost: Low - nav badges and the India status pill are additive UI; the market filters degrade gracefully (unfiltered behavior is still reachable by omitting ?market=); the cron time change is a single cron.alter_job call, trivially reversible; Windows tasks were disabled, not deleted, so re-enabling is a one-line PowerShell call if ever needed.

Decision 45: Two critical dormant bugs found while finishing market-switcher wiring - kairos_call_agent overload ambiguity, decision_journal type mismatch

Date: 2026-07-06 Status: Approved Category: Bug fix (critical) / Technical implementation rule

Context: User asked why the scheduled evening briefing hadn't generated, and asked to finish wiring the remaining pages (Agents, Decision Journal, the Research Journal mislabel from Decision 44). Decision: (1) Investigating the missing briefing found kairos_call_agent had TWO Postgres overloads live simultaneously - an old 3-arg version (no timeout param, silently falling back to pg_net's dangerous 5s default) and the fixed 4-arg version from earlier this session. Any pg_cron call that didn't pass all 4 arguments explicitly failed with "function is not unique," since Postgres couldn't disambiguate which overload's defaults to fill. Confirmed via cron.job_run_details: kairos-proposal-reminder, kairos-broker-sync, kairos-position-monitor, kairos-nav-snapshot, kairos-rescore, and kairos-brief-evening were all failing on nearly every scheduled fire that day - only the 3 jobs that happened to pass all 4 args explicitly (research, research-india, scan-india-refresh) were unaffected. Fixed by dropping the redundant 3-arg overload (migration 083); every existing call site now resolves unambiguously to the single remaining definition. (2) While adding decision_journal.market (Decision 44's remaining wiring), found signal_id/paper_trade_id were typed bigint while agent_signals.id/paper_trades.id are uuid - every paper_fill/paper_exit insert (which pass the real uuid) had been silently failing since the table's creation; confirmed 0 of 7 ever-created rows had either column populated. Fixed both column types to uuid (migration 084, safe - no valid data existed to lose) and backfilled market via a join through them. Reason: Both bugs share the exact failure signature this whole session has been hunting: no crash, no user-visible error, console.error/silent catch swallowing the failure, "looks like it's working" while a core piece of the pipeline (scheduled agent runs; the audit trail's evidence links) has been dead on arrival. A Postgres function-overload ambiguity is a particularly sharp version of this - the SQL literally never executes, so there's nothing downstream to even log. Alternatives considered: None substantive - both are unambiguous schema/definition bugs with one correct fix (remove the ambiguous overload; fix the column type to match its actual foreign data). Impact: Scheduled agent runs (proposal reminders, broker sync, position monitoring, NAV snapshots, rescoring, evening briefing) now actually fire instead of erroring before their SQL body ever executes. Decision Journal's paper_fill/paper_exit entries - the audit trail for every real paper trade - will populate correctly going forward instead of being silently dropped. Files/features affected: supabase/migrations/083_fix_kairos_call_agent_overload_ambiguity.sql (new), supabase/migrations/084_decision_journal_market_column.sql (new, extended), app/api/agents/paper-trade/route.ts, app/api/agents/position-monitor/route.ts, app/api/paper-positions/close/route.ts, app/api/strategies/versions/route.ts, app/api/kite/order/route.ts, app/api/journal/route.ts, app/dashboard/journal/page.tsx, app/dashboard/agents/page.tsx, lib/market-support.ts. Reversal cost: Very low - both are corrective fixes to bugs that were already 100% non-functional; there is no working prior behavior to regress from.

Decision 46: Per-market Trading Mandates separate horizon from strategy preference

Date: 2026-07-12 Status: Approved Category: Product / Architecture / Data / Security

Context: The first Trading Style control labeled a 20-day strategy "Long-term" and changed only threshold, stop, target, and a horizon that a champion could silently override. It did not implement momentum or value/quality behavior and applied one global setting to US and India. Decision: Replace that concept with independent US and India Trading Mandates. Separate horizon (short_swing, swing, position) from strategy preference (adaptive, momentum, balanced, value_quality). User-controlled horizon wins by default; agent optimization requires explicit opt-in and remains bounded. Existing positions are grandfathered by default using their entry-time mandate snapshot. Day trading and genuine long-term investing remain separate future architectures. Reason: Visible settings must truthfully govern behavior across research, paper execution, monitoring, learning, validation, explanation, and live proposals without silently broadening authority or mixing markets. Alternatives considered: Keep the cosmetic three-preset control; let champions always override the user; implement day trading or months-long investing in the daily swing pipeline. Impact: Adds a per-market mandate contract, bounded factor tilts, provenance, market-day horizon resolution, and cross-agent audit context. Hard risk and live approval

gates remain unchanged. Files/features affected: features/trading-mandate/FEATURE_ARCHITECTURE.md, Settings, ResearchAgent, PaperTrader, PositionMonitor, LearnerAgent, validation/backtests, decision journal, and strategy configuration schema. Reversal cost: Medium

Decision 47: Downside hedging is a bounded paper-only overlay

Date: 2026-07-15 Status: Approved Category: Product / Architecture / Money-path safety

Decision: Add a separate deterministic US paper hedge controller using only unleveraged SH and PSQ, macro plus market confirmation, persistence/hysteresis, a 5% target/8% hard NAV cap, one hedge, five-session maximum hold, cash-only funding, and cooldown. Ship shadow and paper execution OFF independently, exclude hedge outcomes from alpha learning, and add no live path. Reason: This adds measurable portfolio insurance without broadening generic long-only agents or violating per-market accounting, explicit authority, and no-LLM-on-money-path rules. Reversal cost: Low while OFF; close any open paper hedge before disabling the controller.

Decision 48: Paper autonomy is mandate-driven, capacity-bounded, and transactionally closed

Date: 2026-07-16 Status: Approved Category: Product / Architecture / Money-path safety / Operational truth

Context: A live-state review proved the paper loop runs without approval, but found four contradictions: PaperTrader ignored latched trading_enabled controls, queried the legacy global threshold 52 while both market mandates required 60, had no enforceable 10-name cap, and PositionMonitor committed lot/position/cash/journal exit writes separately. Home repeated stale threshold/horizon/live-status copy, while LLM health triage could describe a recovered run as a current failure.

Decision: The US/India trading mandate is the canonical paper threshold and risk preset. New paper entries require both per-market pause and trading-enabled controls, with a 10-distinct-alpha-name cap enforced in TypeScript and the row-locked fill RPC. The RPC also enforces no averaging down and writes mandate provenance atomically. All full and partial paper exits use a service-role-only FIFO transaction covering lots, position, native-currency cash, and journal. Home reads the effective mandate/live posture. System Health classification is deterministic; stale/unavailable health data can never render green. Reason: A hands-off loop is safe only when operator stops remain latched, policy is singular, capacity is bounded under concurrency, financial truth cannot partially commit, and the dashboard reports current policy rather than legacy constants. Impact: Existing 11-US/13-India books remain intact but cannot add a new alpha name until each market falls below 10; exits continue automatically and atomically. Live approval authority is unchanged (L3_live_manual). Files/features affected: PaperTrader, PositionMonitor, Home, SystemHealthCard, deterministic health triage, provider-budget alert reconciliation, migrations 20260716010000_paper_autonomy_safety.sql, 20260716011000_fix_paper_rpc_generated_column.sql, and 20260716012000_enforce_paper_exit_lot_parity.sql. Reversal cost: Medium - rollback requires restoring the prior fill signature and JS exit sequence; existing trade data remains compatible.

Decision 49: Per-market capacity and score freshness are mandate policy; measured correlation requires PIT shadow evidence

Date: 2026-07-16 Status: Approved / Partially implemented Category: Product / Architecture / Money-path safety / Research priority

Decision: Store max_open_positions and max_signal_age_sessions on each market's Trading Mandate. Capacity remains an entry-only gate; changing it never opens or closes a position. Research always prioritizes current paper and live holdings, independent of the new-candidate cap. PositionMonitor accepts score/direction exits only from a fresh same-market deterministic_v1 signal. Do not activate correlation-aware P0/P1 from Holding Risk summaries: immutable point-in-time return observations and a shadow parity ledger must exist first. Reason: Capacity and freshness are per-market policy, while managing owned names outranks discovery. The proposed correlation wiring assumed candidate-to-book pair data and measured per-symbol beta that are not persisted today; using cluster summaries or sector

beta as if measured would create false precision on the money path. Impact: US and India remain capped at 10 by default, with 11/13 existing names preserved to drain naturally. Held names bypass discovery caps and receive deterministic reassessment. Stale scores cannot close positions. Correlation construction remains on its conservative existing behavior pending evidence-safe prerequisites. Files/features affected: Trading Mandates API/Settings, ResearchAgent holdings gathering, PaperTrader, PositionMonitor, features/correlation-aware-construction/FEATURE_ARCHITECTURE.md, migration 20260716013000_mandate_capacity_and_score_freshness.sql. Reversal cost: Low for policy/UI; no position or trade data migration.

Decision 50: Research prices are indicative; execution prices remain fill-bound

Date: 2026-07-20 Status: Approved Category: Product / Architecture / Money-path safety / Audit truth

Decision: Record the actual latest scoring-candle close and a deterministic, versioned mandate-based indicative entry/risk/target plan in each future decision observation. Research Journal displays the plan in native currency with its source/date and an explicit non-order disclosure. PaperTrader independently re-prices from a fresh actual fill and the current bounded mandate/validated MAE-MFE policy; PositionMonitor remains the sole owner of executable paper exits. Rejected, neutral, unavailable-price, and held-position states never display a new research scenario as an executable instruction.

Reason: The user needs actionable price context, but a fixed predicted price from an LLM or a stale research close would create false precision and could conflict with existing fill-time and exit controls. The shared deterministic contract adds clarity and auditability without broadening trade authority or API usage.

Impact: Future outcome labels gain a real point-in-time entry basis, Journal gains approximate native-currency levels, and the previously loaded-but-unused MAE/MFE exit policy is applied at fill time within its approved bounds. No schema, broker, approval, scoring-weight, threshold, or provider-quota change.

Files/features affected: features/research-trade-plan/FEATURE_ARCHITECTURE.md, lib/trading/trade-plan.ts, ResearchAgent, PaperTrader, Research Journal API/UI, learning-loop/agent architecture docs, and focused tests.

Reversal cost: Low; all persistence is additive in existing columns/JSONB and research-time rendering is independent of executable position state.

Decision 51: Technical calibration is a reusable, measure-only market and sector experiment

Date: 2026-07-21 Status: Approved Category: Research / Validation / Architecture

Decision: Reuse the existing Edge evidence lab to evaluate the exact current technical composite, market-relative strength, ATR-normalized MACD(12,26,9), and signed ADX(14) independently for US and India. Persist explicit market-wide and sector diagnostic segments with versioned formula identities and complete trial history. Sector specialization is not authorized; thin sector results remain measure-only. Pin Vibe-Trading commit a5eb30fd00d6ee71cd5099d15f57de5ae47010ff as a later independent comparator, but do not install its broad runtime.

Reason: The current technical values are hand-tuned priors. Kairos needs evidence that can be rerun later without losing old trials, while avoiding indicator-zoo and sector overfitting. Existing EdgeScout already freezes candles once per symbol and can evaluate more pure factors without additional provider calls.

Impact: Adds measurement and segmented evidence only. Production scores, weights, directions, thresholds, eligibility, positions, agents, and broker behavior are unchanged.

Files/features affected: features/technical-factor-calibration/, lib/edges/, EdgeIC route, edge_ic_history, tests, and architecture documentation.

Reversal cost: Low; stop future observations while retaining immutable experiment history. No trading rollback exists because the feature has no trading influence.

Decision 52: Edge Calibration Readiness Is A Review Milestone, Never Trade Authority

Date: 2026-07-21 Status: Approved Category: Product / Architecture / Data / Safety

- Why: Weekly factor evidence needs an automatic, durable answer to "what is next?" without relying on memory or overreacting to one favorable IC snapshot.
- Who: The single Kairos owner reviewing US and India research quality.
- ROI: Prevents premature indicator changes, makes evidence collection visible, and surfaces stalled calibration without adding providers or trading risk.
- Decision: Build deterministic per-factor/market/horizon progress with six-window historical stability and a separate four-window PIT/walk-forward/cost/FDR gate. Emit one-time informational milestones and collection-stall warnings. Neither milestone changes scoring, lifecycle, paper trading, or live trading.
- Shipped at: 2026-07-21.

2026-07-28 - Single-factor IC promotion is CLOSED; the edge lab is permanently diagnostic

Status: Superseded later on 2026-07-28. The measurement remains part of the audit record, but the effective-breadth inference and permanent closure below were invalid. See the correction immediately after this entry.

- Context: features/walk-forward-ic-folds built the full purged out-of-sample stack (PIT universe, disjoint folds, no-lookahead runner, HAC aggregation) and measured what it was built to measure.
- Measurement (Annex K): cross-sectional rank-IC sigma is stable at ~ 0.27 across four independent periods (spread 1.36x, sd 0.038) - $3.77\times$ the theoretical 0.0712 at $n=200$. Inverting $\sigma = 1/\sqrt{n_{\text{eff}} - 3}$ gives an effective breadth of ~ 17 independent names from a 200-name cross-section. Correlation binds, not name count.
- Why the escape routes fail:
- *More names:* sigma is set by n_{eff} , not n . Adding names adds correlated names. The $n=400$ experiment was cancelled on this basis.
- *Sector-neutral IC:* reaching $\sigma \leq 0.10$ needs $n_{\text{eff}} \geq 103$, a $6.1\times$ breadth increase. Even an optimistic $3\times$ gain leaves ~ 52 required as-of dates against ~ 25 available. Quantified and rejected.
- *More history:* ~ 180 as-of dates at the measured sigma is roughly 7 years of non-overlapping 20-session evidence. The liquidity entitlement covers 2.
- Decision: Do not re-derive the 1C floors and do not build build-order steps 5-10. `POST /api/agents/backtest/promote` stays permanently disabled. `strategy_policies` and `backtest_experiments` remain as empty governance scaffolding; nothing writes to them.
- What continues unchanged: EdgeScout and EdgeIC keep running as advisory diagnostics. `edge_ic_history` remains a useful monitoring surface. None of it may be cited as promotion evidence.

- What would reopen this: a materially different statistic (portfolio-level cost-adjusted returns rather than single-factor IC), or a data source giving 5+ years of whole-market liquidity history. Not a parameter change.
- Honest framing: the machinery is correct and the refusal is correct. The finding is that single-factor IC promotion is not reachable at this scale - which is a real result, not a failure of the build.

2026-07-28 - Correction: IC promotion remains dormant and measure-only

- Error corrected: The observed h5 time-series standard deviation of daily Spearman IC cannot be inverted with $1/\sqrt{t(n-3)}$ to estimate independent breadth. It combines cross-sectional sampling error, true IC variation over time, changing membership, and data coverage. The artifact did not retain successful-date cross-section counts, so those components are not identified.
- Horizon safety: An h5 dispersion estimate cannot determine h20 power or sample requirements. mom_12_1 showing no h5 signal does not resolve its h20 behavior.
- Decision: Reverse permanent closure. Keep policy promotion fail-closed and scoring unchanged. Permit bounded measure-only work: matched-date n=200 versus n=400, point-in-time sector-neutral IC, and matched-horizon evidence.
- Evidence contract: Future OOS artifacts must retain per-date usable cross-section and provenance. Experiment lineage, PIT inputs, costs, dependence adjustment, and matched edge/formula/market/horizon bindings remain prerequisites to any promotion.
- Operational effect: None. This correction cannot buy, sell, promote a strategy, or change a production score.

Decision 53: Historical Bulk Evidence Is Local, Immutable, and Replay-Only

Date: 2026-07-29 Status: Approved Category: Data / Architecture / Security

Context: Provider quotas and short entitlements constrain point-in-time validation, while arbitrary CSVs and current-universe history would introduce provenance, revision, and survivorship errors. Decision: Acquire official SEC, NSE, and FRED/ALFRED bulk history into a local hash-addressed evidence store outside the repository and OneDrive. Community or manual datasets are pinned and diagnostic-only by default. Only validated, manifest-bound records may enter the existing sealed replay assembler; no live scorer, agent cron, paper path, or broker path may read the store. Reason: This reuses free bulk sources without spending daily provider quotas or creating a parallel truth layer, while keeping every historical experiment reproducible and fail-closed. Alternatives considered: Upload all raw rows to Supabase (rejected: free-tier cost and unnecessary network dependency); dynamically execute GitHub projects (rejected: supply-chain and reproducibility risk); accept arbitrary user CSVs directly (rejected: silent schema and PIT failures). Impact: India price/universe, SEC fundamentals, and macro-vintage diagnostics can run offline. Older broad US promotion-grade price history remains blocked pending licensed survivor-safe data. Files/features affected: features/historical-evidence-intake/, local acquisition CLI, replay adapters, and experiment manifests. Reversal cost: Low; delete the local store and stop using its fingerprints. No trading rollback exists because the store is unreachable from trading.

Decision 54: Local Historical Runs Extend Immutable Experiment Lineage

Date: 2026-07-29 Status: Approved Category: Architecture / Data / Security

Context: The verified bulk evidence is too large for Supabase and Vercel, but results must be visible in Kairos without creating a third provenance system. Decision: Run manifest-bound historical diagnostics locally. Predeclare each plan in

backtest_experiments before reading normalized rows, then complete that same write-once row with compact results and fingerprints. Keep raw data local. Expose only bounded results through a confirmed-owner read API on Backtest. Reason: This turns downloaded evidence into reproducible app-visible research while preserving the existing lineage, free-tier budget, and money-path isolation. Alternatives considered: Upload raw bulk rows (rejected for cost); start desktop jobs from Vercel/browser (rejected as an unsafe and unavailable runtime boundary); new replay-result tables (rejected as parallel truth); let results alter scoring or promotion (rejected until separate promotion-grade evidence gates pass). Impact: India official-price diagnostics can run now without provider quotas. US price replay remains blocked pending survivor-safe adjusted prices. No score, strategy, agent, paper/live trade, order, or broker behavior changes. Files/features affected: features/local-historical-replay/, local replay CLI, backtest_experiments, Backtest result API/UI. Reversal cost: Low; stop the CLI and hide the read-only panel. Existing immutable research rows remain as audit history.

Decision 55: Earnings Options Data Is Risk Context, Not Alpha

Date: 2026-07-29 Status: Approved for P0 measure-only Category: Trading Risk / Data / Governance

Decision: Record market-session earnings proximity and a validated same-expiry/same-strike ATM straddle move proxy as risk context. Keep production policy pinned to shadow; do not add options to analyst_score, and do not change entry eligibility, size, stops, targets, or exits. Reason: Short-dated option premiums price event magnitude more directly than direction. Treating unusual flow or the move proxy as bullish alpha would add an unvalidated factor, while comparing planned stop distance with priced event variance closes a real risk-observability gap. Safety: The existing live Alpha Vantage blackout remains authoritative. Robinhood calls use a read-only allowlist disjoint from every execution tool. The append-only ledger rejects non-shadow policy and any behavior_changed=true row at the database boundary. India never borrows US options evidence. Activation gate: Minimum 60 otherwise-eligible US entries and 20 distinct events, plus date coverage, quote quality, calibration, and counterfactual review. A separate owner decision is required for any block or size reduction. Files/features affected: features/earnings-aware-risk/, lib/risk/earnings-risk.ts, paper/live annotation points, Portfolio Risk, and earnings_risk_observations. Reversal cost: Low; remove the display and annotation calls. Existing rows remain immutable audit evidence and have no trading effect.

Decision 56: One Read-Only Registry Governs All Shadow and Upgrade Programs

Date: 2026-07-29 Status: Approved Category: Architecture / Governance / Operations

Decision: Maintain a typed code registry for every shadow, counterfactual, paper experiment, armed validator, and dormant upgrade path. Build an owner-only Upgrade Path page over existing authoritative ledgers and service-only pg_cron schedule truth. Do not create another evidence table or activation API. Reason: The owner needs one current answer for what each program provides, what it has collected, its provider cost, whether it helped, and the exact remaining gate. A code registry makes future additions reviewable while ledger adapters avoid parallel truth. Safety: The page is read-only. It cannot score, promote, trade, toggle a feature, or call a provider. Market-local evidence remains separate. Unknown call cost is labelled unmetered, and unknown completion time has no fabricated ETA. Operational change: Remove the zero-output kairos-shadow-us and kairos-shadow-india jobs. Keep their routes for a future declared campaign. All productive Router, degradation, setup, technical, earnings, allocation, rotation, and validation flows retain their existing behavior. Files/features affected: features/shadow-registry/, lib/shadows/, /api/upgrade-path, /dashboard/upgrade-path, and migration 20260729210000_shadow_registry_cron_status.sql. Reversal cost: Low; restore the two jobs and remove the read-only surface. No evidence ledger or money-path rollback is required.

Decision 57: Broker Protection Is a Hard Prerequisite for Autonomous Live Entry

Date: 2026-07-30 Status: Approved and implemented as P1 control plane Category: Trading Risk / Execution Safety

Decision: New autonomous live BUYS require an enabled protection policy, a broker-specific placement and reconciliation worker, and full active protection coverage for every managed live position. P1 is read-only and source-default-deny; it cannot create, change, cancel, or query a broker order.

Reason: A database feature flag and a planned order ledger cannot protect a held position. Requiring reconciled broker proof prevents an accidental configuration change from creating a growing set of positions that only a scheduled synthetic exit monitor can protect.

Scope: P1 does not affect paper trading, manual live orders, scoring, learning, or current synthetic exits. P2 needs separate broker capability proof, sandbox or manual canary, lifecycle reconciliation, and owner approval. Robinhood stop-order support is not verified; Kite GTT remains weaker limit-child protection; Webull is disabled.

Reversal cost: Low. Disable the control-plane interlock only by a reviewed source change; no broker state is created by P1.

Decision 58: Post-Earnings Daily Scores Must Reprice Before Directional Use

Date: 2026-07-30 Status: Approved corrective safety change, implemented Category: Trading Risk / Data Freshness

Decision: When a recent point-in-time calendar event has occurred, no daily-score direction may authorize a new entry or score/direction exit until the candle series contains the event reaction. A completed report-date bar is sufficient for before-open events; after-close or unknown-session events require a later bar. A known past event activates the barrier even when the actual feed is late. ResearchAgent records a current, session-validated neutral signal with the barrier provenance. Price stops, targets, trailing stops, and time exits remain active.

Reason: A daily technical score calculated from the pre-event close is not a post-earnings assessment. The MSFT July 30, 2026 stale short exposed that this could otherwise generate a misleading held-position trim/exit recommendation.

Non-goals: This does not predict earnings direction, score options flow, add a provider request, change score weights, or create an order. The existing earnings-options policy remains P0 shadow-only.

Reversal cost: Low. Remove the read-only guard; existing signals retain their recorded provenance. No broker or position state is changed.

Decision 59: Calibrate Trade Plans Deterministically; Keep Options Out of Alpha

Date: 2026-07-30 Status: Approved and implemented Category: Learning / Earnings Risk / Governance

Decision: Join each original indicative stop/objective/horizon with immutable same-market realized return/MFE/MAE labels. Show per-symbol paths as illustrative and expose only aggregate calibration to LearnerAgent. The LLM may explain and write hypotheses but cannot change risk parameters. Keep the existing deterministic 60-label exact-horizon MAE/MFE gate as the only automatic stop/objective adjustment path.

Options remain magnitude/risk evidence, not a score. Add a bounded weekday US holdings shadow over open paper positions and latest complete live-risk snapshots. It merges the point-in-time cache with one bounded Robinhood calendar read, calls per-symbol earnings/options sources only when the discovered event falls inside the holding horizon, and always records `behavior_changed=false`. India has no options leg.

Reason: This provides the retrospective and LLM interpretation the owner expects without mistaking a take-profit level for a terminal-price forecast, letting one dramatic event tune a strategy, or spending option calls across the entire research universe.

Reversal cost: Low. Remove the read-only display/tool and unschedule the holding shadow. Immutable historical evidence remains; no score, position, or broker state requires rollback.

Decision 60: Enforce Provider Data Truth Before Scoring

Date: 2026-07-31 Status: Approved corrective safety change, implemented Category: Research Scoring / Data Safety

Decision: Stamp provider sector taxonomy, crosswalk only explicit Finnhub industry labels, omit relative-P/E scoring for unknown sectors and P/E outside $(0, 200]$, require explicit dimension availability, and restrict the breakdown hard veto to ATR-scaled or high-volume declines. Retire the duplicate Supabase Edge Function scorer; the Next.js ResearchAgent is

authoritative.

Reason: Production SQL proved that 68% of Finnhub rows silently received the wrong P/E norm, provider outliers reached 8,827, and ordinary weak closes forced score 20. Static review had validated formulas without validating real provider distributions.

Non-goals: No insider formula, market threshold, PEAD dimension, India exit policy, or technical-ceiling change is approved by this decision. Those remain measured proposals. No historical signal or trade is rewritten.

Reversal cost: Low in code, but reversal would restore known silent data corruption and is not recommended.

Decision 61: Completed Sessions, Market-Local Schedules, and India News Shadow

Date: 2026-07-31 Status: Approved corrective safety change, implemented 2026-07-31 Category: Research Data / Scheduling / Evidence Governance

Decision: Filter all daily scoring inputs to completed regular-market sessions; represent US agent schedules as New York local-time contracts admitted through paired seasonal UTC crons; remove the proven-zero India GDELT sentiment dimension from active applicability; and collect NSE corporate announcements plus bounded Google News RSS only as behavior-neutral canonical shadow evidence.

Reason: Intraday daily bars can mutate technical scores before settlement, fixed UTC jobs shift by an hour at DST boundaries, and India sentiment was unavailable for 0 of the latest 310 decisions while the GDELT access pattern was rate-limited. Unknown data must remain unavailable rather than consume repeated calls or imply neutral evidence.

Non-goals: No score weight, threshold, historical signal, position, cash, paper or live order changes. The India shadow has no directional classifier and cannot be read by a decision or money path. Historical replay remains diagnostic-only.

Reversal cost: Low. The local-time and completed-session guards are pure boundary checks. The shadow can be unscheduled without changing decision state; its append-only provenance can remain for audit.

Decision 62: Event-Aware Fundamentals and Exchange-Listed ADRs

Date: 2026-07-31 Status: Implemented and production-verified Category: Research Data / Instrument Governance / Trading Safety

Decision: Share one earnings-aware reported-fundamental freshness policy across agents, stop ThemeScout from using full fundamentals as a ticker-existence probe, and introduce a reviewed adr asset class for US exchange-listed depository receipts. Add Nasdaq-listed SK hynix (SKHY) to that registry; explicitly reject legacy OTC proxies as substitutes. ADRs may enter the US paper path. Live compatibility remains behind all existing controls and mandatory broker review.

Reason: Daily polling wastes calls between reports, but blanket long TTLs miss event changes and stale valuation. ADR providers can return foreign-underlying units under a US ticker; a distinct source/applicability contract prevents unit mixing and false insider-data starvation.

Non-goals: No OTC automation, foreign-exchange execution, live enablement, threshold/weight change, ADR-specific alpha claim, or broker-support promise.

Files/features affected: features/event-aware-fundamentals-adr/, ResearchAgent, fundamentals/Yahoo adapters, ThemeScout, instrument registry, paper/live asset classification, architecture chapters, and agent diagrams.

Reversal cost: Low. Remove registry entries and restore cache constants; no historical decision or trade is rewritten.

Feature Micro-Architecture Appendix

These documents are intentionally preserved as design history. They are not all active product behavior. Use the lifecycle statement inside each document, its implementation result when present, and the shared architecture chapters to determine what is actually live.

advanced learning - Feature Architecture

Source: `features\advanced-learning\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Advanced Learning - Deflated Sharpe + PBO, Regime-Conditioning, Meta-Labeling

Status: *DRAFT (design only, unapproved). No code, no migration, no deployment.*

Last updated: 2026-07-15

Update this file when: the DSR/PBO promotion-gate math, the regime-conditioning

weighting method, or the meta-labeling gate contract changes; keep it aligned with

`docs/arch/09-learning-loop.md` (Validation Engine, promotion, Performance Truth,

`P1 gate`), `lib/validation/engine.ts`, `lib/validation/calibration.ts`,

`features/learning-core/FEATURE_ARCHITECTURE.md`, and

`features/decision-review/FEATURE_ARCHITECTURE.md`.

1. One-line intent

Upgrade the learning/promotion brain with the canonical Lopez de Prado (*Advances in Financial ML*) + academic safeguards we are missing, in priority order:

- Deflated Sharpe Ratio (DSR) + Probability of Backtest Overfitting (PBO) as a

hard, deterministic layer in the promotion gate - the #1 overfitting guard.

- Regime-conditioning - a weight *adjustment* conditioned on the current

`macro_regime` state, learned per regime cohort with its own sample floor. Evidence-conditioned, not a hardcoded bull/bear switch (respects the locked `CLAUDE.md` pushback rule).

- Meta-labeling - a secondary deterministic model that decides *whether to ACT*

on a primary signal (a precision filter), trained on `decision_observations` x `observation_labels`. Measure-only -> shadow -> gated.

All three extend the existing deterministic Validation Engine / calibration / learner path. None introduces an LLM on the money path, and none creates a parallel truth layer. Performance Truth remains the NAV authority; the `LearnerAgent` + owner promotion remain the only things that change champion weights.

2. Hard boundaries (non-negotiable)

- Deterministic, no LLM sets any number. DSR, PBO, regime weight-deltas, and the

meta-label model are pure arithmetic / gradient-descent over the ledger - same posture as `lib/validation/engine.ts` (moving-block bootstrap, seeded PRNG) and `lib/validation/calibration.ts` (batch GD logistic, fixed iterations). An LLM may *narrate* the numbers in prose, never produce them.

- Extends, does not replace. DSR/PBO are added terms on the existing

validateChallenger path and are recorded on the existing validation_experiments / strategy_evaluations rows. Regime-conditioning is a bounded delta on top of the champion genome weights, not a second weight store. Meta-labeling is a *gate flag*, never a score.

- No parallel truth layer. Every number is derived from the append-only

decision_observations (059) x observation_labels (060) ledger and the existing macro_regime (028) table - the same records LearnerAgent, the Validation Engine, and Performance Truth already read. Performance Truth (lib/evaluation/run-evaluation.ts) stays the book-truth / NAV authority; nothing here recomputes NAV.

- Per-market / per-currency, never mixed. US cohorts use USD prices + SPY-neutral

returns; India cohorts use INR + ^NSEI-neutral. A regime cohort, a DSR trial count, or a meta-label training set is always scoped to a single market. India evolves on its own gate exactly as today (docs/arch/09-learning-loop.md "Per-market independence").

- Measure-only -> shadow -> gated rollout. Every method ships OFF, computing and

recording its verdict without affecting selection/sizing/promotion. Each is activated by a separate owner switch, and only after its own sample floor is cleared. The riskiest of the three (meta-labeling gating entries) is the last to ever gate.

- The champion promotion path stays owner-only and fail-closed. DSR/PBO can only

make promotion *stricter* (add a required PASS), never auto-promote. Meta-labeling can only *withhold* an entry that the deterministic score already permitted - it can never *create* an entry. Regime-conditioning can only nudge a weight within a bounded band; it can never override the entry threshold or the "sum-to-1.0" invariant.

- Respects the locked "no regime switching" rule. Regime-conditioning is a

smooth, evidence-weighted blend (shrink-to-champion by cohort sample size), not an if regime == 'red' then bearMode branch.

3. What already exists (verified against code) - the substrate

```
| Building block | Where | What we reuse |
| Walk-forward folds (purged + embargoed, anchored) | `lib/learning/dataset.ts` `walkForwardFolds()` | The
**exact** leak-free splitter. PBO's combinatorial split reuses the same purge/embargo constants. |
| Labeled dataset join | `lib/learning/dataset.ts` `loadLabeledDataset()` | `decision_observations` x
`observation_labels`, benchmark-neutral returns at 2/5/10/20d. Meta-label training data + regime cohorting
both read this. |
| Challenger replay objective | `lib/validation/engine.ts` `objectiveTerm()` / `scoreRow()` | Per-row
benchmark-neutral log-growth under a weight set. DSR consumes the **per-fold return series** this already
produces. |
| Bootstrap + seeded PRNG | `lib/validation/engine.ts` `blockBootstrap()`, `mulberry32()` | Determinism
pattern; DSR's variance/skew/kurtosis of the return stream and PBO's rank logic reuse the same seed
discipline. |
| Existing promotion gates | `lib/validation/engine.ts` (p_improvement >= 0.80, CI-low, n_effective >= 12,
>=3/5 folds) + `activate_strategy_shadow` (170) + atomic promotion (161) | DSR/PBO are **added** as further
AND-conditions; the fail-closed PASS row + owner-only promote path are unchanged. |
| Calibrated P(win) logistic + OOS/ECE fail-closed gate | `lib/validation/calibration.ts` | Meta-labeling
is architecturally a *sibling* of this model: a second logistic, same GD, same walk-forward OOS acceptance
discipline (`acceptCalibrationOOS`), different target (act-vs-skip precision). |
| Regime state | `macro_regime` (028): `week_of`, `regime` ? {green,yellow,orange,red}, `danger_score`,
read via macro-sentinel | Regime label per observation date. **Exists but weights are NOT conditioned on it
today** - this feature is the first consumer for conditioning. |
| Experiment ledger | `validation_experiments` (via `recordExperiment`), `strategy_evaluations` (134,
append-only) | DSR/PBO verdicts are new columns on these rows. No new provenance store. |
| EdgeIC statistical toolkit | `lib/edges/ic.ts` (Spearman, Newey-West SE, t-hurdle) | Reference
implementation for the significance discipline; PBO logit and DSR t-stats live beside it, not on the money
path. |
| Aggregate/per-decision measurement surfaces | `features/edge-factor-discovery`, `features/decision-
review` | Sequencing peers - DSR/PBO/meta-label verdicts surface through the same Performance Truth /
Decision-Review panels, not a new dashboard. |
```

Key implication: we are not inventing infrastructure - DSR/PBO/meta-labeling are *additional deterministic computations* over data and splitters that already exist. The build is a math + gating layer, not a new pipeline.

4. Method 1 - Deflated Sharpe Ratio + PBO in the promotion gate (priority #1)

4.1 Why

The current gate (`validateChallenger`) tests whether a challenger beats the champion on held-out folds with bootstrap significance. It does not correct for selection bias: `LearnerAgent` proposes many challengers over time, and the best of N trials will look good by luck alone. DSR and PBO are the canonical L^{pez de Prado} corrections for exactly this.

4.2 Deflated Sharpe Ratio (DSR)

DSR adjusts an observed Sharpe for (a) the number of trials that were run to find it, (b) the non-normality (skew/kurtosis) of the return stream, and (c) the track length. It returns a probability that the true Sharpe > 0 given the search.

Deterministic recipe (all inputs already available or trivially derivable):

- Return stream. For the challenger, take the per-test-row objective series already produced in `validateChallenger` (the challenger arm of `pairedDiffs`, i.e. `objectiveTerm(challengerWeights, row)` across concatenated OOS test rows). This is the leak-free, walk-forward return series - no new replay needed.
- Observed Sharpe $SR_{\hat{}}$ = mean/std of that series (annualization factor is recorded but DSR works on the raw per-observation SR; we store both).
- Skew γ_3 and kurtosis γ_4 of the same series (deterministic moments).
- Number of trials N = count of challengers proposed for this market in the current learning episode / trial family (see 4.4 - this is the load-bearing input and the biggest measurement decision).
- Expected max Sharpe under the null SR_0 = the L^{pez de Prado} closed form for the expected maximum of N independent standard-normal Sharpe estimates (uses the Euler-Mascheroni / Gaussian-quantile approximation - pure arithmetic).
- $DSR = \Phi \left(\frac{(SR_{\hat{}} - SR_0) \sqrt{T-1}}{\sqrt{1 - \gamma_3^2 SR_{\hat{}}^2 + ((\gamma_4 - 1)/4) SR_{\hat{}}^2}} \right)$ where T = track length (number of OOS observations), Φ = standard normal CDF.
- Gate: require $DSR \geq DSR_MIN$ (v0 default 0.95, i.e. 95% confidence the deflated Sharpe is positive) - a tunable const beside `MAX_OOS_ECE` in the calibration module, documented as needing prospective tuning.

4.3 Probability of Backtest Overfitting (PBO)

PBO (Bailey-Borwein-L^{pez de Prado}, CSCV - Combinatorially Symmetric Cross-Validation) estimates the probability that a configuration selected as best in-sample ranks below median out-of-sample - the direct probability that our selection is overfit.

Deterministic recipe reusing the existing splitter:

- Partition the OOS observation matrix into S contiguous, purged/embargoed sub-blocks using the *same* purge (`horizonDays`) and embargo the walk-forward splitter already applies (`lib/learning/dataset.ts`). S even (v0 S = 8 if sample allows, else the largest even S meeting the per-block floor).

- For each of the $C(S, S/2)$ combinations, designate half the blocks IS and half OOS.

- Score the candidate family (champion + all challengers in the trial family, ?4.4) on IS; pick the IS-best.

- Record that config's OOS rank among the family; compute its logit

$\phi = \ln(\text{rank}/(M+1 - \text{rank}))$.

- PBO = fraction of combinations where the IS-best has $\phi \leq 0$ (below-median OOS). Require $\text{PBO} \leq \text{PBO_MAX}$ (v0 default 0.20).

Because CSCV is combinatorial, cap S so $C(S, S/2)$ stays bounded ($S \leq 10 \rightarrow \leq 252$ combinations) - deterministic and cheap.

4.4 The single load-bearing input: trial count N / the candidate family M

DSR needs N (how many strategies were tried) and PBO needs the family of configurations scored together. Both quantify selection bias, so they must count the same search. Design decision (v0):

- The trial family = all `strategy_versions` rows for this market with

`proposed_by='learner'` since the current champion was promoted (i.e. every challenger the learner floated in this episode), plus the champion itself.

- N = size of that family. This is directly queryable - no new state - and it is a

conservative (never-underestimated) count, which is the safe direction for an overfitting guard.

- Recorded on the experiment row (`n_trials`, `trial_family_ids`) so the verdict is reproducible and auditable.

This is the single riskiest assumption - see ?9. If N is undercounted, DSR is

too permissive; the conservative "count every learner challenger this episode"

choice deliberately errs strict.

4.5 Where it plugs in (deterministic, additive)

`validateChallenger` today returns PASS iff the four existing conditions hold. Add two AND-conditions, evaluated only when the sample floor is met:

```
passed = existing_gates_pass
AND (n_effective >= DSR_PBO_MIN_SAMPLE ? (DSR >= DSR_MIN AND PBO <= PBO_MAX) : true*)
```

* When the sample floor is not met, DSR/PBO are computed and recorded but not enforced (measure-only) - the harness runs from day one, the gate activates only when data matures. The floor mirrors the existing discipline: the engine already refuses <60 observations and `n_effective < 12`; DSR/PBO enforcement adds a higher floor (v0 `n_effective >= 20`, matching the Performance Truth 20-trade honesty rule and the P1 gate). Owner flips enforcement per market via a `strategy_validation_automation-style` switch (`dsr_pbo_enforced`).

New fields on `validation_experiments` (additive migration, not in this doc): `dsr`, `sr_hat`, `sr0_expected_max`, `n_trials`, `pbo`, `cscv_blocks`, `trial_family_ids`, `dsr_pbo_enforced`, `dsr_pbo_pass`. These summarize; they do not replace `passed/p_improvement`.

Boundary: DSR/PBO can only *block* a promotion the old gate would have allowed. They never promote, never size, never touch cash. The atomic owner-only promotion RPC (161) still requires a PASS row and an owner click.

5. Method 2 - Regime-conditioning (priority #2)

5.1 Intent & the locked-rule constraint

Learn that (e.g.) *technical* weight should be shrunk and *macro* weight lifted when `macro_regime='red'`, without a brittle bull/bear mode switch. The CLAUDE.md rule is explicit: no explicit regime-detection *switching*. So regime-conditioning is a smooth, evidence-weighted, bounded delta on the champion genome weights - it degrades gracefully to "just use the champion weights" whenever the regime cohort is thin.

5.2 Cohorting

- Stamp each labeled observation with the `macro_regime.regime` in effect on its `ts` (join `decision_observations.ts` -> `macro_regime.week_of`, market-local). This is a read - `macro_regime` already exists; nothing new is written to the ledger.
- Group the labeled dataset into per-market regime cohorts (`{green, yellow, orange, red}`). To avoid 4x sparsity, `v0` collapses to two evidence bands - calm (`green|yellow`) and stress (`orange|red`) - a data-driven coarsening, not a hand-tuned bull/bear branch; the band boundary is a documented const, and the design keeps the full 4-level cohort available for when volume supports it.

5.3 The conditioned weight = champion ? shrunk delta

For each cohort `c` and market, fit the same calibration logistic (`lib/validation/calibration.ts` machinery) on the cohort's rows to get the dimension-importance implied by that regime. Convert to a candidate weight vector `w_c`, then shrink toward the champion by cohort evidence:

```
w_effective(c) = normalize( (1 - ?_c) ? w_champion + ?_c ? w_c )
?_c = min(?_max, n_c / (n_c + K))           # James-Stein-style shrinkage
```

- `n_c` = cohort sample size; `K` = shrinkage constant (`v0` `K = 50`); `?_max` caps the maximum deviation from champion (`v0` `0.25` - a regime can move a weight by at most a quarter of the way to its cohort-implied value).

- Sample floor: when `n_c < REGIME_MIN_SAMPLE` (`v0` `30`, matching the calibration `MIN_OOS_SAMPLES`), `?_c = 0` -> exactly the champion weights. Thin regimes are a no-op, not a guess.

- The sum-to-1.0 genome invariant and the `position_size_pct` cap are re-imposed after the blend; the entry threshold is never conditioned (only dimension weights).

This composes with the champion genome as a pure function of (champion weights, current regime, cohort evidence). There is no second weight store and no mode flag - the champion remains the single governance object; regime-conditioning is a deterministic transform applied at scoring time, recorded for audit.

5.4 Validation & rollout

- A regime-conditioned challenger is validated by the same `validateChallenger` path (including the new DSR/PBO layer) - regime-conditioning is just a different way of producing `weights_snapshot`-equivalent scoring, so it must clear the identical gate before it can ever affect live selection.
- Ships measure-only: the conditioned weights are computed and logged to a new

regime_weight_deltas provenance row (per market x cohort x fit date) but scoring uses champion weights until the owner enables regime_conditioning_enabled per market - and only after the cohort floors are met.

- India and US condition on their own macro_regime history independently.

Boundary: the delta is bounded (?_max), evidence-gated (?_c -> 0 when thin), and re-normalized - it can nudge, never flip. No if regime branch selects a strategy; the champion is always the base.

6. Method 3 - Meta-labeling (priority #3)

6.1 Intent

Primary model = the existing deterministic analyst_score + entry threshold (decides side / direction). Meta-labeling adds a secondary deterministic model that decides whether to act on a primary BUY - a precision filter that suppresses low-precision entries. It never overrides the deterministic score; it only gates *entry eligibility*. (Lopez de Prado, Ch. 3.)

6.2 Model

- Training data: decision_observations x observation_labels where the primary signal was a candidate entry (entry_eligible = true / score >= threshold). Label = 1 if that entry was a win (benchmark-neutral return > 0 at the mandate horizon), 0 otherwise. This is exactly the loadLabeledDataset output filtered to would-have-acted rows.

- Features: the 5 dimension scores + deterministic context already on the row

(availability mask, score-vs-threshold margin, regime band from ?5, rank_pct from cross-sectional rank if present). No look-ahead - same walk-forward discipline.

- Fit: the *same* batch-GD logistic + standardization as

lib/validation/calibration.ts, producing P(win | acted) - a precision estimate, distinct from the sizing P(win) (different training population: acted-only, not all decisions).

- OOS acceptance: reuse acceptCalibrationOOS verbatim (fail-closed: <30 OOS

rows, degenerate class, or ECE > 0.1 -> rejected, model not stored). Stored (when accepted) as a new model_artifacts.kind='meta_label_logistic' row - same table, same upsert discipline as pwin_logistic.

6.3 The gate (three-stage rollout, the last method to ever act)

- Measure-only: compute P_meta for every candidate, record it on the decision

observation / Decision-Review surface, and report what it *would* have suppressed and the counterfactual precision lift. Changes nothing.

- Shadow: a state='shadow_paper' strategy variant applies the meta-gate and

records its would-be book vs the champion's - a free dress rehearsal (exactly the existing shadow_paper mechanism, docs/arch/09-learning-loop.md "Shadow decisions"). No fills, no cash.

- Gated rollout (owner-approved): only after shadow shows a real precision lift

over a sufficient sample, and only behind an owner switch (meta_label_gate_enabled per market), the meta-gate may withhold an entry the primary model permitted: act = primary_entry_eligible AND (P_meta >= META_MIN). META_MIN is a tunable const with a min-sample floor (v0 >= 50 acted observations in the training population before the gate can enforce).

Boundary: meta-labeling is strictly subtractive - it can only turn an eligible entry OFF, never turn an ineligible one ON, never change direction, never change size, never write a weight. It is the highest-risk method (it directly withholds trades), so it is sequenced last and gated hardest.

7. C4 sketch

Context

```
flowchart TB
  OWNER[Owner\npromotes / flips switches] --> GATE
  LEARNER[LearnerAgent\nweekly, proposes challengers] --> GATE
  subgraph BRAIN[Advanced Learning brain - deterministic, no LLM]
    GATE[Promotion Gate\n+ DSR + PBO]
    REGIME[Regime-conditioner\nchampion + bounded delta]
    META[Meta-label filter\nact / skip precision]
  end
  end
  LEDGER[(decision_observations\nx observation_labels)] --> BRAIN
  MACRO[(macro_regime)] --> REGIME
  GATE --> EXPR[(validation_experiments\n+ dsr/pbo cols)]
  BRAIN --> PT[Performance Truth\nNAV authority - unchanged]
  GATE -. owner-only promote .-> CHAMPION[strategy_versions\nchampion]
```

Container / component

```
flowchart LR
  DS[lib/learning/dataset.ts\nwalkForwardFolds - REUSED] --> DSR[lib/validation/deflated-sharpe.ts\nDSR + expected-max-Sharpe]
  DS --> PBO[lib/validation/pbo.ts\nCSCV logit]
  DSR --> ENG[lib/validation/engine.ts\ninvalidateChallenger - EXTENDED]
  PBO --> ENG
  CAL[lib/validation/calibration.ts\nGD logistic + acceptCalibrationOOS - REUSED] --> REG[lib/learning/ regime-weights.ts\ncohort fit + shrinkage]
  CAL --> ML[lib/learning/meta-label.ts\nact/skip precision model]
  MACRO[(macro_regime)] --> REG
  ENG --> EXPR[(validation_experiments)]
  REG --> RWD[(regime_weight_deltas - new, provenance)]
  ML --> MA[(model_artifacts kind=meta_label_logistic)]
  ENG -. records .-> SE[(strategy_evaluations 134 - append-only)]
```

New code (all deterministic, unit-testable pure functions like the existing walkForwardFolds / blockBootstrap): lib/validation/deflated-sharpe.ts, lib/validation/pbo.ts, lib/learning/regime-weights.ts, lib/learning/meta-label.ts. Extended: lib/validation/engine.ts (two added AND-gates, all new fields recorded).

8. Phased rollout (each gate flips separately, measure-only first)

```
| Phase | Ships | Enforcement | Sample floor | Owner switch |
| **A. DSR/PBO harness** | Compute DSR, PBO, n_trials on every `validateChallenger` run; record on experiment rows; surface in Performance Truth | **Measure-only** | - | none |
| **B. DSR/PBO gate** | DSR >= 0.95 AND PBO <= 0.20 become required promotion AND-conditions | Enforced | n_effective >= 20 per market | `dsr_pbo_enforced[market]` |
| **C. Regime measure-only** | Cohort fit + shrunk deltas computed, logged to `regime_weight_deltas`; scoring still uses champion | Measure-only | - | none |
| **D. Regime conditioning** | Conditioned weights used in scoring (still must clear the gate as a challenger) | Enforced | n_c >= 30 per cohort (else ?=0) | `regime_conditioning_enabled[market]` |
| **E. Meta-label measure-only** | `P_meta` computed + recorded; counterfactual suppression reported | Measure-only | - | none |
| **F. Meta-label shadow** | `shadow_paper` variant applies gate, records would-be book | Shadow (no cash) | OOS-accepted model | none |
| **G. Meta-label gate** | Meta-gate may withhold eligible entries | Enforced | >= 50 acted obs per market | `meta_label_gate_enabled[market]` |
```

Phases A/C/E (all measure-only) can be built now; the enforcement phases (B/D/G) stay dark until the sample floors are cleared - consistent with the existing Phase-0/Phase-1 discipline and the "when data matures" reality of a fresh book.

9. Single riskiest assumption

Sample size. At this stage the book has very few closed trades (the 10-trade Phase-0 gate has barely opened, and Performance Truth still shows `insufficient_sample` below 20). DSR needs a meaningful track length T , PBO needs enough purged blocks to form $C(S, S/2)$ splits, regime cohorts fragment the already- small sample by 2-4x, and meta-labeling needs enough `*acted*` rows to fit a second logistic. Building the harness now is correct and safe (measure-only); activating any enforcement gate before the per-method sample floors are met would either block all promotions or fit noise. The design mitigates this by (a) computing everything measure-only from day one, (b) per-method floors that mirror the existing 20/30-row disciplines, (c) shrink-to-champion / no-op fallbacks whenever a cohort is thin, and (d) conservative trial-count N that errs strict.

The one open decision for the owner (?4.4): `*how to count N (the trial family) for DSR/PBO.*` The proposed "every learner-proposed challenger for this market since the current champion was promoted, plus the champion" is conservative and auditable, but it couples the deflation strength to `LearnerAgent's` proposal cadence - a chattier learner inflates N and makes promotion harder. The alternative (count only `*validated*` challengers, or a fixed rolling window) is less strict but arguably closer to "trials that could have been selected." This choice sets how aggressively the #1 overfitting guard bites, and should be locked before Phase B enforcement.

10. Acceptance tests (deterministic, no live money)

- DSR determinism: same dataset + same $N \rightarrow$ identical DSR/PBO to full float precision (seeded, like `blockBootstrap`). Snapshot test.

- DSR deflation direction: holding `SR_hat` fixed, increasing N strictly lowers DSR; increasing negative skew lowers DSR. Property test.

- PBO sanity: a deliberately overfit config (fit-to-noise weights) yields PBO $\rightarrow$ high (> 0.5) on synthetic data; a genuinely predictive config yields PBO low. Fixture test.

- Gate is additive, never looser: any challenger that FAILS the old gate still FAILS with DSR/PBO enabled; DSR/PBO can only flip PASS $\rightarrow$ FAIL. Differential test against current `validateChallenger`.

- Measure-only inertness: with all switches OFF, champion weights, selection, sizing, promotions, and NAV are byte-identical to pre-feature (golden-master over a replay window) - mirrors the cross-sectional-rank / PIT-fundamentals "OFF = no-op" proof.

- Regime no-op when thin: cohort with `n_c < 30` $\rightarrow$ `?_c = 0` $\rightarrow$ conditioned weights exactly equal champion weights (bit-exact).

- Regime bounded: for all cohorts, `?w_effective - w_champion? <= the ?_max` bound and `? w_effective = 1.0`.

- Meta-gate subtractive only: for every observation, `act ? primary_eligible` (the gate never enables a primary-ineligible entry); with the gate OFF, `act == primary_eligible`.

- OOS fail-closed inheritance: a meta-label model with < 30 OOS rows or $ECE > 0.1$

is not stored to `model_artifacts` (reuse `acceptCalibration00S` test vectors).

- Per-market isolation: a synthetic India cohort/trial family never alters any US DSR, PBO, regime delta, or meta-model, and vice-versa.

11. Sequencing vs router + Decision-Review + experiment-lineage

- Post-router. This is a *promotion-brain* upgrade; it assumes the scoring/router path and the Validation Engine are stable. It does not touch routing, data provider abstraction, or order execution. Build after the router work lands.

- Complements Decision-Review (`features/decision-review/FEATURE_ARCHITECTURE.md`):

Decision-Review is the per-symbol post-mortem UI over the same `decision_observations` x `observation_labels` ledger. Meta-labeling's measure-only `P_meta` and the DSR/PBO verdicts surface through Decision-Review / Performance Truth panels - no new dashboard. Decision-Review answers "was this one decision right?"; Advanced Learning answers "should this *strategy* be promoted, and should we *act* on its signals?" Same evidence tables, different altitude.

- Complements experiment-lineage: DSR/PBO records (`n_trials`, `trial_family_ids`, `dataset_hash`) are exactly the lineage/provenance experiment-lineage tracks - this feature *produces* lineage facts (which trials, which family, which verdict) that experiment-lineage *organizes*. Build the DSR/PBO recording columns in a shape experiment-lineage can consume.

- Ordering within this feature: #1 DSR/PBO (highest value, smallest surface, pure add to an existing gate) -> #2 regime-conditioning (needs cohort volume) -> #3 meta-labeling (highest risk, needs the most acted data, gated last). Each measure- only phase is independently shippable and independently reversible.

12. Open items to confirm at approval

- Trial-count N definition (?4.4/?9) - the single biggest decision.
- DSR_MIN (0.95), PBO_MAX (0.20), S (8), ?_max (0.25), K (50), META_MIN, and all sample floors are v0 defaults needing prospective tuning - confirm the starting values.

- Regime cohorting granularity: 2-band (calm/stress) v0 vs full 4-level - confirm the v0 coarsening.

- Whether regime-conditioning is expressed as a runtime scoring transform vs a materialized per-regime challenger set (this doc proposes runtime transform + audit row; a materialized variant is possible but adds governance objects).

agent evolution - Feature Architecture

Source: `features\agent-evolution\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Feature Architecture: Agent Evolution (source-attributed discovery, expanded genome, US/India parity)

Status

Architecture status: Draft Architecture approved: No Approved scope: None Approved date: None Implementation allowed: No

Prompted by a second Codex adversarial review pass (2026-07-06), specifically scoped to whether ResearchAgent's parameters, discovery sources, and LearnerAgent's evolution surface are actually appropriate for a world-class long-only swing-trading agent across US and India - as opposed to whether the current code has bugs (that pass is Decision 43, already fixed).

Feature Purpose

Three related gaps, all downstream of the same root cause - the system has no memory of **which discovery source** or **which parameter** a decision came from, so it can't learn which sources/parameters are worth trusting:

- Symbol discovery is unattributed. `gatherSymbols()` in

`lib/research-agent.ts` merges 7 sources (holdings, watchlist, Theme Scout, momentum bucket, value bucket, metals basket, region ETFs, India NIFTY) into one flat list. Once merged, a `decision_observations` row only knows `source`: "holding" | "screener" - Theme Scout picks, momentum-bucket picks, and value-bucket picks all collapse into the same "screener" label. There is no way to ask "did Theme Scout's picks actually make money over the last 90 days?" without hand-joining `watchlist.theme/features.screener.bucket` against `observation_labels` - a query nobody runs today.

- LearnerAgent only evolves 5 numbers. `update_signal_weight` (Decision

43 hardened it, but its scope is unchanged) can only move `fundamental_weight` / `technical_weight` / `sentiment_weight` / `macro_weight` / `insider_weight`. Everything else that defines the strategy - score threshold, position sizing curve, stop/target logic, which discovery sources get research budget, India-specific gates - is either a fixed constant in code or a value the user sets by hand in Settings. The "evolution" the product promises is five numbers wide.

- India is US scoring wearing a currency label. `isIndia(symbol)` swaps

Alpha Vantage for Yahoo and skips social/options/insider (honest), but reuses the exact same `score_threshold`, RSI/EMA thresholds (`lib/data/technicals.ts`), and benchmark logic path as the US. There is no India-specific liquidity filter, no NSE holiday-aware freshness check on candle data, and India's `computeRegimeFeatures` benchmark is `^NSEI` only because someone hardcoded it in - not because it went through the same validation gate a US weight change does.

User/System Questions This Feature Answers

- Which discovery source (Theme Scout, momentum, value, watchlist, held,

India NIFTY, metals, region ETF) actually produces profitable signals, and should the system research more/fewer symbols from it?

- If LearnerAgent wants to change something other than the 5 score weights

(e.g. "raise the score threshold" or "shorten the holding horizon"), is there a governed, validated path to do that, or does it require a manual code change?

- Is India's scoring gated on India-appropriate evidence, or silently reusing

US assumptions that were never checked against India data?

Scope

This feature includes:

- A primary `discovery_source` field recorded on every new

`decision_observations` row (`held_position` | `manual_watchlist` | `theme_scout` | `momentum_bucket` | `value_bucket` | `india_nifty` | `india_screen_cache` | `metals` | `region_etf` | `unknown_legacy`) plus `discovery_sources_all/discovery_metadata` for multi-source provenance. This is additive. It does not remove or reinterpret the current `agent_signals.source`: "holding" | "screener" field in v1, because that field is already used by existing UI/reporting paths.

- A read-only Discovery Source Scorecard (API + dashboard card) showing,

per source: candidates researched, entry-eligible rate, and - once `observation_labels` mature - mean `benchmark_neutral_return`. Advisory only in v1; see Non-Goals.

- An extended strategy genome schema (`strategy_versions.genome`, already

typed per Phase 3 - see `lib/validation/genome.ts`) covering the fields listed under "Proposed genome fields" below, each with an explicit bounded range and a validation-gate requirement before promotion - no new field becomes live without going through the exact same challenger -> validate -> promote path `update_signal_weight` already uses.

- An India evidence contract: a documented, code-enforced minimum

(candle freshness vs NSE trading calendar, minimum liquidity proxy, minimum fundamental-field count) that must hold before a India candidate is eligible to enter `Long`, independent of and stricter than the general thin-evidence abstain rule from Decision 43.

- A written decision on whether India gets its own `score_threshold/RSI`

bands or continues sharing the US ones, backed by whatever India-labeled ledger data exists at the time (may conclude "not enough India data yet to diverge - keep shared, revisit after N India closed trades").

Non-Goals

This feature does not include:

- Any automatic reallocation of research budget by source. The Discovery

Source Scorecard is read-only/advisory in v1. Automatically researching more Theme Scout candidates because they scored well recently is a feedback loop that needs its own validation story (a source that looks good on 20 trades can regress) - out of scope until the scorecard has run long enough to have an opinion on its own reliability.

- Auto-promotion of any genome field. Every genome-field change flows

through the existing challenger/validate/promote lifecycle. Nothing here makes a `strategy_versions` row live without a human promoting it via the Strategy Registry, same as today.

- Increasing screener candidates above 3/day. CLAUDE.md locks this at 3;

this feature does not touch that number regardless of what the Discovery Source Scorecard shows.

- Explicit bull/bear regime switching. CLAUDE.md explicitly wants scoring

to adapt via weights, not a hardcoded regime branch. Nothing in the genome expansion below introduces an if/else on regime; regime stays an observed feature (`features.regime.*`, Phase 3) available to future weight/threshold learning, not a switch.

- Loosening trade approval. No genome field, however it evolves,

changes `approval_required` gating on `TraderAgent/live` orders. Paper trading and shadow evaluation remain the only channels a genome change can reach before a human promotes it.

- Building India-specific thresholds speculatively. This feature defines

the `*contract*` (what evidence India needs) and instruments the ledger to measure whether shared vs India-specific thresholds perform differently. It does not invent India-specific RSI/threshold numbers without validated India trade history to back them - CLAUDE.md's "don't run ahead of evidence" principle applies here as much as to weight changes.

Codex Review Amendments Required Before Build

These amendments close ambiguity in the draft architecture and should be treated as part of the target design:

- Do not collapse multi-source provenance into one lossy label. A symbol

can be a held position, a manual watchlist item, a Theme Scout candidate, and a screener hit on the same day. Persist one primary `discovery_source` for grouping, but also persist `discovery_sources_all` and `discovery_metadata` so source performance can be audited without losing assisted-source credit.

- Keep legacy `agent_signals.source` stable. The new source fields belong

on the decision/research ledger. Do not rename or overload `agent_signals.source` in this PR; existing UI expects the coarse holding | screener meaning.

- Scorecard must be statistically honest. It must show sample size,

label coverage, benchmark-relative win rate, confidence interval, and insufficient-sample status. A raw average return by source is too easy to overread and will invite bad allocation decisions.

- No source-budget feedback loop in v1. Source performance can be shown to

the user, but cannot change `gatherSymbols()` allocation until a separate exploration policy exists with a minimum exploration floor and shadow validation.

- Separate provisioned genome fields from live-consumed fields.

`score_threshold` may be added to the genome schema and validation engine in this PR, but `ResearchAgent` must continue using the current approved runtime threshold path until a later approved promotion/consumption change defines precedence against `strategy_config`.

- India gates must fail closed for entry eligibility. If the NSE calendar,

candle freshness, liquidity proxy, or required fundamentals are unknown, the system may still record an observation, but it must not mark the India candidate entry-eligible.

Current Behavior

- `gatherSymbols()` returns a flat `SymbolEntry[]` with only `isHeld`,

`isEtf`, `assetClass`, and an optional `screenerBucket` (momentum/value - added Decision 41, screener-only). Theme Scout candidates arrive via the `watchlist` table read and are indistinguishable from a manually-added watchlist symbol once merged.

- `processSymbol()` sets `source: isHeld ? "holding" : "screener"` on

agent_signals, and copies the same 2-value source onto decision_observations implicitly via the market/symbol join - there is no persisted "this candidate came from Theme Scout" flag past the features.screener block (which only exists for the momentum/value screener bucket, not Theme Scout/watchlist/held/metals/region-ETF/India).

- update_signal_weight (LearnerAgent) is the only live weight-mutation

tool.strategy_versions.genome exists (Phase 3, migration 063) and is validated (lib/validation/genome.ts) but nothing currently proposes genome mutations beyond the 5 weights - the genome column is provisioned infrastructure, not yet a used surface.

- India shares score_threshold, stop_loss_pct, target_pct from the same

strategy_config row as US (no market-scoped strategy_config columns exist yet - only strategy_versions is market-scoped, per Phase 4). India's NSE holiday gate (Decision 39) only covers 2 fixed-date holidays (Republic Day, Independence Day, Gandhi Jayanti) - floating festivals (Holi, Diwali) are explicitly unmodeled, a known, documented gap.

Proposed Behavior

1. Discovery source attribution

- Extend SymbolEntry (lib/research-agent.ts) with a required

discoverySource field, set at the point each source contributes a symbol in gatherSymbols(): held_position, manual_watchlist (manual, non-theme), theme_scout (watchlist rows with source: 'llm_theme'), momentum_bucket / value_bucket (already tagged via screenerBucket, just renamed into the same enum), metals, region_etf, india_nifty, and india_screen_cache if the broader India scanner cache is later wired into ResearchAgent.

- Extend SymbolEntry with discoverySourcesAll and optional

discoveryMetadata. When a symbol appears from multiple sources in one run, keep all sources. Pick the primary source by causal priority: held_position > manual_watchlist > theme_scout > momentum_bucket / value_bucket > india_screen_cache > india_nifty > metals > region_etf. The scorecard should support both primary-source metrics and assisted-source metrics once enough data exists.

- Persist discovery_source on decision_observations (new column, migration) alongside the existing features.screener block - additive, does not replace the momentum/value bucket detail already recorded.

- Persist discovery_sources_all and discovery_metadata next to

discovery_source. discovery_sources_all is required for new observations and should include the primary source; discovery_metadata may include contributing watchlist row IDs, screener bucket/rank, theme name, or scanner run ID when available.

- New read-only endpoint /api/agents/discovery-scorecard: per

discovery_source x market, count of candidates researched (last 90d), entry-eligible rate, and mean benchmark_neutral_return where matured labels exist (reuses lib/learning/dataset.ts's join, grouped by source instead of by fold). Surfaced as a card on the Research Journal's Evolution tab - extends existing UI, no new page.

- The scorecard must report source/market/horizon rows across at least 30d and

90d windows, including nResearched, nLabeled, labelCoveragePct, entryEligibleRate, abstainRate, avgDataAvailability, meanBenchmarkNeutralReturn, medianBenchmarkNeutralReturn, winRateVsBenchmark, bootstrapCiLow, bootstrapCiHigh, and sampleStatus. If a row lacks enough matured labels, show sampleStatus: "insufficient_data" instead of a performance verdict.

2. Genome expansion (validated, shadow-first)

Proposed additional genome fields (each independently bounded, independently gated by the SAME validation path - walk-forward replay via `lib/validation/engine.ts`'s shared `computeWeightedAnalystScore`, extended per field type below):

```
| Field | Bounds | Validation approach |
| `score_threshold` | 50-75 | Replay: what would entry-eligible rate + benchmark-neutral return have been at this threshold, using labeled history - same walk-forward/bootstrap machinery as weights. |
| `holding_horizon_days` | 2-20 | Already have 2/5/10/20-day labels (`observation_labels`); replay picks the horizon whose labeled return the strategy is actually evaluated against. |
| `stop_loss_pct` / `target_pct` | bounded % | Already partially dynamic via MAE/MFE percentiles (`lib/risk/percentiles.ts`); genome field would let a challenger propose a *different percentile* (e.g. p20/p80 instead of p25/p75), validated the same way. |
| `discovery_source_weights` | 0-1 per source, sum-normalized | Read-only in v1 (see Non-Goals) - recorded in genome as a *proposed* field once the Discovery Source Scorecard has enough history to inform it, not wired to actually change `gatherSymbols()`'s candidate mix until a future, separately-approved iteration. |
| `sizing_curve_params` | half-Kelly cap, floor | Currently hardcoded in `lib/risk/sizing.ts` defaults; genome field would let a challenger propose different cap/floor, validated against the ledger's realized Kelly-implied returns. |
```

None of these are proposed as an immediate `LearnerAgent` tool. This document recommends building ONE new field (`score_threshold` - cheapest to validate, reuses 100% of existing walk-forward machinery, most directly answerable question: "is 60 the right bar?") end-to-end as a proof of the pattern before expanding to the rest, rather than shipping all five/six new tunable fields in one PR. Each subsequent field follows once `score_threshold` has round-tripped through challenger -> validate -> promote at least once, on real data.

Important runtime boundary: this PR may provision and validate `genome.entry.score_threshold`, but it must not silently make `ResearchAgent` consume that field. Runtime threshold precedence must be approved separately because today `strategy_config.score_threshold` acts as a user-visible manual control. Until that follow-up is approved, the genome threshold is a shadow candidate parameter only.

3. India evidence contract

Documented (and code-enforced in `lib/data/scores.ts/lib/india-data.ts`) minimum evidence before an India candidate is `entry_eligible`:

- Candle data freshness checked against an NSE trading-calendar-aware

cutoff, not a flat day count (extends Decision 39's fixed-holiday gate to also flag stale-but-not-holiday data, e.g. Yahoo feed lag).

- Minimum liquidity proxy (e.g. average daily traded value over N days,

sourced from Yahoo's `averageVolume/price.regularMarketVolume` - needs a concrete threshold, to be set from observed India ledger data once ~60 India observations exist, not guessed today).

- The `hasMinFundamentalFields` gate from Decision 43 already applies to

India; this section formalizes it as a *documented* India-specific contract rather than an incidental side effect of a US-shaped check.

- Fail-closed rule: if NSE calendar freshness, latest candle freshness,

liquidity proxy, or required fundamentals cannot be determined, the system may still save the observation with an explicit reason code, but it must set `entry_eligible = false` for that India candidate.

- `ResearchAgent` v1 currently uses the static NIFTY candidate path; the broader

India scanner/cache path should not be assumed live for `ResearchAgent` unless wired explicitly. If that cache is later used as a source, tag it as `india_screen_cache` rather than mixing it into `india_nifty`.

- Explicit open question, not resolved by this document: does India get its

own `score_threshold` once enough India-labeled data exists? Recommendation is to instrument (via the Discovery Source Scorecard's market dimension, already present) and revisit once India has 60+ matured observations (the same bar Phase 2 validation already uses for US) - not decide now from zero data.

User Journey / System Flow

- Research cron runs `gatherSymbols()` -> every returned `SymbolEntry` now carries a `discoverySource`.
- `processSymbol()` writes `discovery_source` onto `decision_observations` alongside the existing `score/availability/weighting` fields (Decision 43).
- Nightly/weekly, the Discovery Source Scorecard endpoint aggregates matured observations by source x market - visible on Research Journal.
- `LearnerAgent` (future PR, not this one) gains a `propose_genome_field` tool scoped initially to `score_threshold` only, mirroring `update_signal_weight`'s evidence-binding: server recomputes the replay itself, refuses on insufficient data, creates a challenger, never mutates live config directly.
- Validation Engine's existing walk-forward/bootstrap gate (`lib/validation/engine.ts`) evaluates the challenger; promotion is manual via Strategy Registry, same as weight challengers today.
- India's evidence contract runs inline in `computeScores/processSymbol` for every India candidate, independent of and prior to the shared thin-evidence abstain check from Decision 43.

Screen / Page / Module Inventory

- `lib/research-agent.ts` - `SymbolEntry.discoverySource`, `gatherSymbols()`, `processSymbol()` (persist `discovery_source`).
- New migration - `decision_observations.discovery_source` column.
- New `app/api/agents/discovery-scorecard/route.ts`.
- `app/dashboard/research-journal/page.tsx` - new "Discovery Sources" card on the Evolution tab.
- `lib/validation/genome.ts` - extend bounded-field schema for `score_threshold` (first field only, per Proposed Behavior ?2).
- `app/api/agents/learner/route.ts` - future `propose_genome_field` tool (explicitly deferred to a follow-up PR, not built in this pass).
- `lib/india-data.ts`, `lib/data/scores.ts` - India evidence contract checks.
- `lib/learning/dataset.ts` - include `source/provenance` fields in the labeled observation dataset used by scorecards and later validation.
- Tests for source attribution precedence, legacy unknown-source handling, scorecard insufficient-sample behavior, genome threshold bounds, and India fail-closed gates.

System Architecture

Modules

- Discovery attribution lives entirely in `lib/research-agent.ts` - no new service, extends the existing `SymbolEntry/processSymbol` types.

- Genome validation reuses `lib/validation/engine.ts` and

`lib/scoring/weighted-score.ts` (Decision 43) unchanged in mechanism; `objectiveTerm`'s per-row scoring already supports whatever weights a challenger proposes - extending it to also vary `score_threshold` per challenger is a parameter, not a new code path.

API Contracts

- GET `/api/agents/discovery-scorecard?market=us|india&days=90` -> `{

`sources: [{ source, market, n, entryEligibleRate, meanBenchmarkNeutralReturn | null }] }`. Read-only, no side effects.

Required scorecard response shape: GET

`/api/agents/discovery-scorecard?market=us|india&days=30|90&horizon=2|5|10|20` returns {
`sources: [{ source, market, horizonDays, windowDays, nResearched, nLabeled, labelCoveragePct, entryEligibleRate, abstainRate, avgDataAvailability, meanBenchmarkNeutralReturn, medianBenchmarkNeutralReturn, winRateVsBenchmark, bootstrapCiLow, bootstrapCiHigh, sampleStatus }] }`. Read-only, no side effects. `sampleStatus` must distinguish `sufficient`, `insufficient_labels`, and `legacy_unknown_source`.

Data Models

- `decision_observations.discovery_source` text (new, nullable - existing

rows have no source recorded, must degrade gracefully, not backfilled).

- `strategy_versions.genome` - already jsonb (migration 063); this feature

adds `score_threshold` as a recognized, bounded key, validated by `lib/validation/genome.ts`'s existing bounds-checking pattern.

- `decision_observations.discovery_source` is nullable in DB only for legacy

rows; new code paths must write one of the approved source values. Use a check constraint or shared enum list for: `held_position`, `manual_watchlist`, `theme_scout`, `momentum_bucket`, `value_bucket`, `india_nifty`, `india_screen_cache`, `metals`, `region_etf`, `unknown_legacy`.

- `decision_observations.discovery_sources_all` jsonb not null default '[]'

records all contributing sources and must include the primary source on new writes.

- `decision_observations.discovery_metadata` jsonb records optional provenance:

watchlist row ID, theme, screener bucket/rank, scanner run ID, and stale/gate reason codes.

- Add indexes for scorecard reads: (`market`, `discovery_source`, `ts desc`) and,

if query plans require it, a GIN index on `discovery_sources_all`.

Error Handling

- Discovery Source Scorecard degrades to "insufficient data" per source

below some minimum `n` (mirrors Validation Engine's `nobservations < 60` gate) rather than showing a misleading small-sample average.

Data Architecture

- Required data: `decision_observations.discovery_source` (new), `observation_labels` (existing, joined by `symbol+market+horizon`).
- Optional data: benchmark-neutral return only where labels have matured; scorecard rows show `n` researched even before any label matures.
- Mock vs real vs derived: fully derived from real ledger rows, no synthetic/seed data.
- Persistence: additive column, fail-soft insert (same pattern as `availability_mask/weighting` in Decision 43 - a missing column must never fail a research run).
- Validation: genome field validation reuses Phase 2's walk-forward + block-bootstrap statistical bar unchanged (`p_improvement >= 0.80`, CI floor, `n_effective >= 12`, `>=3/5` folds won).

Files Likely To Change

- `lib/research-agent.ts`
- `supabase/migrations/0NN_decision_observations_discovery_source.sql` (new)
- `app/api/agents/discovery-scorecard/route.ts` (new)
- `app/dashboard/research-journal/page.tsx`
- `lib/validation/genome.ts`
- `lib/india-data.ts`, `lib/data/scores.ts`
- `public/agent-diagrams/system-map.json` - LEARNER/RESEARCH node descriptions need updating once genome fields beyond weights exist (CLAUDE.md's system-map rule).

Files / Behavior That Must Not Change

- `TraderAgent/live` order `approval_required` gating - no genome field reaches live execution without a human approving the order, same as today.
- Screener candidates/day cap (3) - CLAUDE.md-locked, not touched by `discovery-source` attribution or scorecard.
- No explicit regime branch introduced anywhere in this feature - regime remains an observed feature, not a switch.
- `update_signal_weight`'s existing evidence-binding (Decision 43) - the proposed `propose_genome_field` tool must follow the identical fail-closed-on-fallback-data pattern, not a weaker one.

Acceptance Criteria

- Every new `decision_observations` row has a non-null `discovery_source`

matching one of the approved source values, and `discovery_sources_all` includes the primary source. Legacy rows are surfaced as `unknown_legacy`, never silently grouped into a real source.

- Discovery Source Scorecard returns real, non-fabricated aggregates and clearly marks sources with insufficient sample size rather than showing a misleading number.

- Discovery Source Scorecard exposes label coverage, abstain rate, benchmark-relative win rate, median return, and confidence interval, not only mean return.

- Source scorecard output does not alter research allocation, candidate count, paper trading, or live-trading eligibility in v1.

- `score_threshold` is provisioned in the genome schema and validated by the existing walk-forward engine, but is NOT proposable by `LearnerAgent` until a separately-approved follow-up implements `propose_genome_field` end-to-end (this PR builds the plumbing/contract, not the LLM-facing tool).

- `ResearchAgent` does not consume `genome.entry.score_threshold` until a separate approved runtime-precedence decision says how it interacts with `strategy_config.score_threshold`.

- India evidence contract checks are documented in this file and enforced in code; no India-specific `score_threshold` divergence is introduced without a documented ledger-backed reason.

- India candidates with unknown calendar/freshness/liquidity/fundamental evidence are saved as observations but are not marked entry-eligible.

- Tests cover source attribution precedence, multi-source provenance, scorecard insufficient-sample behavior, legacy rows, genome threshold bounds, and India fail-closed gates.

Approval

Architecture approved: No Approved scope: None Implementation allowed: No

agent mind - Feature Architecture

Source: features\agent-mind\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature Architecture: Agent Mind - surfacing what the agents believe, how it evolves, and what macro data means for the book

Status

Architecture status: Implemented (all 3 phases), 2026-07-07 Architecture approved: Yes (user, 2026-07-07) Approved scope: All three phases Approved date: 2026-07-07 Implementation allowed: Yes

Implementation notes (2026-07-07)

- Migration 096: learning_priors_history, unique(category, principle) on learning_priors, macro_interpretations. Migration 097: daily macro-read crons (us/india). All new tables service-role-only.
- strategy_versions already had parent_version_id - no schema add needed.
- Phase 1: /api/agent-mind/priors (GET/PATCH, owner-only) + Intelligence "Beliefs" tab (view/toggle/add; every change writes history + decision_journal).
- Phase 2: /api/agent-mind/brain (GET, owner-only) + Intelligence "Brain" tab (champion weights + why, belief drift, learner log, self-invented features, regime posture, track record, evidence-context banner). Pure DB reads.
- Phase 3: /api/agent-mind/macro-read (GET owner / POST owner+cron) + Markets "What this means for your book" card. One cheap cached LLM call/day/market.
- No LLM writes any belief; nothing here trades or sizes.

Why this feature exists

Kairos already *forms a view of the market* - it just keeps that view scattered across tables no human reads directly. As the system runs:

- learning_priors confidences are meant to drift as predictions resolve (background beliefs like "12-1 momentum works", "widening credit spreads lead equity weakness").
- strategy_versions accumulates challenger weight-sets; one per market is the promoted is_champion - the app's current working theory of how much each scoring dimension should matter.
- signal_weights holds the live per-dimension weights the ResearchAgent actually scores with.
- learning_log records the LearnerAgent's weekly hypotheses and outcomes.
- feature_registry holds machine-proposed new features and their IC-gated promotion state.
- macro_regime / macro_signals hold the current macro posture and the raw

economic prints behind it.

- `decision_observations` + `paper_trades` are the ground-truth outcomes that earn or destroy confidence in all of the above.

The user cannot see any of this as a coherent, evolving "mind". This feature surfaces it - read-only, advisory - so the user can (a) see and curate what the agents believe, (b) watch beliefs strengthen/weaken over time and learn the market alongside the system, and (c) understand what a fresh economic print means for the actual book.

This is a VIEWING/curation layer. It never places trades, never changes sizing, and never lets an LLM mutate a belief - belief changes stay owned by the evidence-bound LearnerAgent (weekly, statistics-gated). Any LLM used here writes display text only.

Non-Goals

- No new autonomous behavior. Nothing here trades, sizes, or approves.
- No LLM-driven mutation of `learning_priors`, `signal_weights`, or `strategy_versions`. Only the existing LearnerAgent (evidence-bound, weekly) changes beliefs. This feature can let the USER manually toggle a prior's enabled flag or edit its text - a human action, logged - but the LLM never does.
- No new market-data spend on the hot path. Macro interpretation (Phase 3) runs only when macro data refreshes (already daily) or on an explicit user click, on a cheap model (DeepSeek/Groq), never per-page-load.
- No change to how confidence is computed. This feature reads the confidence the LearnerAgent already maintains; it does not invent a parallel scoring.

Phased scope

The three pieces share data sources and a UI home, but are independently approvable and shippable. Recommended order: Phase 1 (foundation the others read) -> Phase 2 (Brain) -> Phase 3 (macro interpretation).

Phase 1 - Agent Beliefs panel (surface + curate `learning_priors`)

Purpose

A readable, editable table of every prior the agents reason from, grouped by category, showing confidence, source, enabled state, and - once history exists - how the confidence has moved.

Data sources (all existing)

- `learning_priors` (id, category, principle, confidence, source, enabled, notes, created_at).
- Optional new `learning_priors_history` (see below) to chart confidence drift.

Proposed behavior

- New tab under `/dashboard/intelligence` (or a card on Settings -> Agents):

"Agent Beliefs". Groups priors by category (fundamental / technical / macro / insider / general), each row showing the principle text, a confidence bar, source tag, and an enabled toggle.

- Owner-only. Read via a new GET /api/agent-mind/priors (service client, requireOwner). Toggle/edit via PATCH /api/agent-mind/priors (owner-only, logs the change to decision_journal as a human action).

- A small "add prior" form (category, principle, starting confidence, source)

so the user can feed a distilled principle from a book/article directly - exactly one testable claim per row, matching the ingestion discipline.

- Sorting by confidence; filter by category; search.

New schema (Phase 1)

- learning_priors_history (prior_id, confidence, changed_at, changed_by

('learner' | 'user'), reason) - append-only, written whenever a prior's confidence or enabled state changes. Lets the UI chart drift and the Brain view (Phase 2) show "strengthening / weakening". The LearnerAgent's existing prior-update path writes a row here; the user-edit path writes one too.

- Migration also de-duplicates and adds a unique(category, principle)

constraint to learning_priors (a bug this session left it with every row doubled - already cleaned in data, but the constraint prevents recurrence).

Acceptance criteria

- Every prior visible, grouped, with confidence + source.
- Owner can toggle enabled / edit text / add a prior; each change logged.
- No LLM writes to learning_priors.
- unique(category, principle) prevents duplicate priors.

Phase 2 - Kairos Brain (the unified, evolving belief view)

Purpose

One page that answers "what does the system believe right now, how sure is it, and how has that changed?" - stitched from the separate tables into a single human-readable, evolving narrative. This is the piece that lets the user learn the market alongside the agents.

Data sources (all existing unless noted)

- learning_priors + learning_priors_history (Phase 1) - background beliefs and their drift.

- strategy_versions where is_champion per market - the current working

theory: the promoted weight-set, its notes (why it was proposed), its validation stats, and its parent lineage (how the champion has changed over generations).

- signal_weights - the live per-dimension weights actually scoring today.

- learning_log - recent hypotheses the LearnerAgent formed and whether they were confirmed, rejected, or still open.

- feature_registry (+ _history) - features the system proposed for itself and their IC-gated promotion state (the app inventing new signals).
- macro_regime - current regime posture and danger score.
- decision_observations + paper_performance - the outcome ledger that earns all of the above its confidence, plus realized alpha.

Proposed behavior - sections on /dashboard/intelligence "Brain" tab

- Current conviction - the champion weight-set per market as a readable bar ("fundamental 32%, technical 24%, macro 18%, sentiment 14%, insider 12%") with a one-line "why this is the current theory" from the champion's notes, and how it differs from the previous champion (lineage diff).
- Strengthening vs. weakening beliefs - priors whose confidence rose or fell most over the last N weeks (from learning_priors_history), each with the direction and magnitude. This is the "the system is becoming more/less sure of X" view.
- Open hypotheses - what the LearnerAgent is currently testing (from learning_log), with status (open / confirmed / rejected) and the evidence count behind each.
- Self-invented features - anything in feature_registry the system proposed and where it sits in the IC-gate pipeline (proposed -> testing -> promoted / killed).
- Regime posture - current macro_regime with the danger score and the threshold adjustment it's currently imposing on new trades.
- Track record honesty - realized alpha, win rate, and the count of resolved predictions, so the confidence is contextualized by how much evidence actually exists (avoid false authority when N is small).

Read path

- GET /api/agent-mind/brain (owner-only, service client) - one aggregated payload assembled server-side from the tables above. No LLM required for the structured view. An OPTIONAL "explain in plain English" button calls a cheap model to narrate the current state (advisory text only, never persisted as a belief).
- Purely read-only. No writes.

New schema (Phase 2)

- None required beyond Phase 1's learning_priors_history. Everything else already exists. (If champion lineage isn't already queryable via a parent_id on strategy_versions, add one - to confirm against live schema first.)

Acceptance criteria

- Single page shows current champion weights + why, belief drift, open hypotheses, self-invented features, regime posture, and track-record context.
- Every number traces to a real table; nothing is LLM-invented.

- Small-N guardrail: confidence is always shown next to the evidence count so a belief with 3 resolved trades never reads as authoritative.

Phase 3 - Macro-to-holdings interpretation (Markets page)

Purpose

Turn the macro data the app already ingests into a plain-language read of what a fresh print means, combined with the other macro signals AND the current book - the missing "so what does this mean for me" layer.

Data sources (all existing)

- `macro_signals / macro_regime` - the economic prints (GDP, CPI, unemployment, payrolls, retail sales, Fed funds, 2Y/10Y yields) and the regime.
- `paper_positions` (+ live snapshot) - the current book, its sector/beta mix.
- `learning_priors` (macro category) - the durable macro principles to reason with (e.g. "rising real yields headwind long-duration growth").

Proposed behavior

- A "What this means for your book" card on the Markets page, generated when macro data refreshes (daily, already scheduled) or on an explicit user click - NEVER on every page load.
- Deterministic inputs assembled server-side (latest prints + regime + book composition + relevant macro priors), handed to a cheap model (DeepSeek/Groq) to produce a short, grounded read: "CPI hotter than expected + real yields rising + you hold 3 long-duration growth names (NVDA, ...) -> near-term headwind for that cluster; the macro priors flag this at confidence 0.66." Advisory only.
- Cached per macro-refresh so it costs at most one cheap LLM call/day.

Guardrails

- The model receives DATA, never instructions, and its output is display text only - it cannot change sizing, thresholds, or any belief. It is explicitly told the numbers are ground truth and it may not invent figures.
- Account numbers and any secrets are never in the prompt.
- If the model is unavailable, the card degrades to the raw macro data + regime (which the Markets page already shows) - no hard failure.

New schema (Phase 3)

- Optional `macro_interpretations` (date, market, content, model, created_at) to cache the daily read. Or reuse the existing briefings/newsletter pattern.

Acceptance criteria

- Card appears only after a macro refresh or explicit click; at most one cheap LLM call/day.
- Output ties the print to the regime, the macro priors, and the actual book.
- No LLM output changes any trading behavior; degrades gracefully if the model is down.

Cost summary

- Phase 1 & 2: zero LLM on the structured views (pure DB reads); optional "explain" buttons are on-demand cheap-model calls only.
- Phase 3: at most one cheap-model call per macro refresh (daily), cached.
- No new market-data API spend on any hot path.

Files (indicative, to confirm against live code before building)

- app/api/agent-mind/priors/route.ts (new - GET/PATCH, owner-only)
- app/api/agent-mind/brain/route.ts (new - GET, owner-only)
- app/api/agent-mind/macro-read/route.ts (new - Phase 3, owner-only + cron)
- components/dashboard/IntelligencePage.tsx (modify - Beliefs + Brain tabs)
- components/dashboard/MarketsPage.tsx (modify - Phase 3 card)
- supabase/migrations/0NN_agent_mind.sql (new - learning_priors_history, unique(category,principle), optional macro_interpretations, optional strategy_versions.parent_id if absent)
- LearnerAgent prior-update path (modify - also append to learning_priors_history when it changes a confidence)

Approval

Architecture approved: No Approved scope: None Implementation allowed: No

agent source pipeline remediation - Feature Architecture

Source: features\agent-source-pipeline-remediation\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Agent Source Pipeline Remediation

Status: APPROVED for implementation by owner on 2026-07-20.

Problem

The US and India pipelines are correctly separated at their principal scoring and paper-accounting boundaries, but an adversarial source audit found active quote coverage, quota, provenance, confidence, and advisory-semantics defects. The most material defect is PositionMonitor's burst of one unpaced Massive request per US holding: an unavailable response silently suppresses every mechanical and conviction exit for that position for the session.

Scope

- Make paper-position monitoring price-complete per market and visibly report every position that could not be evaluated.
- Make decision-quality applicability agree with production scoring: India has fundamental, technical, and sentiment dimensions today, not US macro.
- Scope discovery before external work so one market never spends the other market's provider budget or broker session.
- Remove Theme Scout from ResearchAgent's wall-clock budget and make capped Alpha Vantage data reserve-only.
- Record the provider that actually served each evidence item.
- Stop routine option-chain calls until an approved measure-only experiment has a consumer and prospective acceptance test.
- Keep Holding Risk's deterministic risk posture intact while adding a separate, read-only alpha context from already-persisted ResearchAgent output.
- Repair dormant autonomous-live cross-market assumptions while keeping every activation flag unchanged and false.
- Keep India Learner runs from consuming the US-only macro table and make valid green MacroSentinel runs idempotent.
- State India scanner coverage from measurable freshness instead of claiming the rotating cache is simultaneously fresh across the full NSE universe.

Non-Goals

- No Router or Vibe cutover.
- No autonomous-live activation and no broker/network order test.
- No score-weight, threshold, mandate, kill-switch, or account change.

- No India macro score. That requires a separately reviewed architecture.
- No conversion of Holding Risk into an execution agent.

Safety Invariants

- No LLM output may set a score, direction, posture, quantity, price, or order.
- A missing quote cannot be treated as a hold; it is an explicit unevaluated state.
- A quote failure for one position cannot suppress exits for positions with valid prices.
- US/USD and India/INR data, outcomes, NAV, mandates, and calibration samples never mix.
- Mechanical exits remain independent of ResearchAgent availability.
- Holding Risk reads persisted alpha evidence only and remains advisory.
- Autonomous-live, Router, Vibe, allocation, and protective-order flags are unchanged.
- Database migrations are forward-only and production definitions are verified after apply.

Acceptance Criteria

- PositionMonitor resolves US prices through one batched request plus existing cache/reserve fallbacks and reports missing symbols in its run result and Health.
- India quality rows no longer count macro as applicable or missing.
- US research does not call Kite/India screening; India research does not call the US FinancialDatasets screener or US holdings path.
- ResearchAgent no longer invokes Theme Scout inline and no longer fetches Yahoo option chains during routine scoring.
- Evidence records carry actual source and timestamp; unavailable evidence does not claim a successful provider.
- Holding Risk publishes alpha score/direction/age/holding-path provenance as separate context without changing its risk posture or calling ResearchAgent/providers.
- Dormant India autonomous-live uses India pricing, India mandate thresholds, and India-only paper calibration; tests prove cross-market records cannot affect it.
- India Learner reports macro unavailable; a healthy green US macro run is reused.
- Scanner responses expose eligible/fresh counts and do not claim all NSE rows are fresh.
- Focused tests, full Vitest, TypeScript, Next production build, secret scan, migration verification, and read-only production smoke checks pass.

agent transparency debate - Feature Architecture

Source: `features\agent-transparency-debate\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Feature: Agent Transparency + Deep-Dive Debate

Status: DRAFT (Phase 1 in progress) ? Owner: Vaibhav ? Started 2026-07-04

Motivation

Competitor trading-agents.ai (TradingAgents framework) produces an **argued** per-symbol verdict via an adversarial multi-agent debate. Kairos currently produces only a numeric `analyst_score`. This feature closes that gap while keeping Kairos's edge (loop closure, governance, automation).

Phases

- Deep-Dive Debate engine + per-symbol panel <- current
- Agent history/transparency page (#12)
- Score trajectory + re-scoring (#13)
- Visual Agents audit + AI-agent vs Flow rename (#14)

Phase 1 - Deep-Dive Debate

Pipeline (on-demand, per symbol)

```
Data bundle (quote, fundamentals, technicals, macro regime, prior signals)
?
?
Analyst layer (parallel): Market/Technical ? Sentiment/News ? Fundamentals ? Macro
? (4 short reports)
?
Research debate: Bull advocate vs Bear advocate -> Research Evaluator (lean)
?
?
Risk debate: Risky / Neutral / Safe -> risk summary
?
?
Portfolio Manager -> VERDICT (BUY/HOLD/SELL/PASS) + conviction + rationale
```

Model routing (no ANTHROPIC_API_KEY available)

- Analysts + advocates: deepseek-chat (cheap, parallel)
- Research Evaluator + Portfolio Manager: deepseek-reasoner (better reasoning)
- Cost ~\$0.05-0.15 per run. On-demand only (button) - never auto/cron.

Governance

- Long-only aware: for a non-held symbol the verdict is BUY or PASS (never SELL).
- Advisory only: verdict may flag high conviction into the pipeline but the deterministic risk gate + approval-required order flow remain authoritative.

Storage - table deep_analyses

id uuid pk ? symbol ? verdict ? conviction int ? summary ? reports jsonb (per-agent) ? model ? tokens_in ? tokens_out ? cost_usd ? created_at RLS: service_role all; authenticated read.

API

- POST /api/agents/deep-dive { symbol } - runs the pipeline synchronously, stores a deep_analyses row, returns the full result. Auth: user or cron.
- GET /api/agents/deep-dive?symbol=X - latest stored analysis for the symbol.

UI - DeepDivePanel on the symbol page

- "Run Deep Analysis" button.
- Agent pipeline list with status; each agent row expands to its report.
- Final verdict card (verdict, conviction, rationale).
- Shows model + token/cost footer.

Non-goals (Phase 1)

- Streaming progress (synchronous run; loading state in UI).
- Multi-market (US first; India/EU rides with the watchlist workstream).
- Auto-feeding analyst_score (advisory display first; wiring in a later phase).

agentic quant platform - Feature Architecture

Source: features\agentic-quant-platform\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature Architecture: Governed Agentic Quant Platform

Status

Architecture status: Approved Architecture approved: Yes Approved scope: Long-only, 2-20 market-day swing-trading research, validation, paper execution, explainability, approval-required Robinhood execution, and a future separately unlocked limited auto-live mode Approved date: 2026-06-27 Implementation allowed: Yes (phased ??" see Delivery Phases) Phase 0 complete: 2026-07-01 Phase 1: Not started

Feature Purpose

Kairos will become a governed multi-agent research platform that continuously gathers evidence, develops and validates trading hypotheses, runs realistic paper experiments, explains its reasoning to Vaibhav, and can eventually execute approved trades through the Robinhood agentic account.

The platform is designed to test whether strategies can outperform appropriate benchmarks after costs, taxes, and risk. It does not assume or promise market outperformance. Survival, reproducibility, and auditability take precedence over return.

Approved Product Decisions

- Trading horizon: swing trading, normally 2-20 market days.
- Universe: Robinhood-supported US equities and ETFs that pass dynamic liquidity, price, trading-status, and data-quality filters.
- Direction: long-only in paper and live pathways.
- Data budget: free-first. TradingView is a manual analysis and export tool, not an unattended data API.
- Strategy eligibility: dynamic statistical evidence gate followed by Vaibhav's explicit manual approval.
- Explanation: layered decision journal plus a daily briefing.
- Online research: primary and curated sources establish evidence; unrestricted web and social sources may generate hypotheses but cannot independently justify trades.
- Initial live mode: every live order requires explicit user approval.
- Future live mode: limited auto-live may be enabled only through a separate, strategy-specific, time-bounded manual unlock.
- Tax, dividend, broker, and regulatory knowledge is monitored for change, but behavior-changing rules require governed review.

User/System Questions This Feature Answers

- What does the system currently believe about the market, and what evidence supports that belief?
- What is the agent planning to research, test, paper trade, or propose for live execution?
- Why is a proposed action preferable to waiting or choosing another security?
- Which statements are verified facts, calculations, estimates, interpretations, or unverified hypotheses?
- How did a strategy perform out of sample, after realistic costs, and relative to a benchmark?

- What changed between a champion strategy and its challenger?
- What risks, correlated shocks, dividend events, and tax consequences apply?
- What did the platform learn after a prediction or trade resolved?
- Is the platform healthy, current, and authorized to act?

Scope

This feature includes:

- Point-in-time evidence ingestion and provenance.
- Source-quality hierarchy, conflict detection, quarantine, and abstention.
- Sourced research packets and reproducible hypotheses.
- Deterministic feature computation and a Python validation worker.
- Versioned strategy registry with champion/challenger governance.
- Historical replay, robustness tests, and dynamic eligibility reports.
- Realistic long-only paper execution and portfolio accounting.
- Deterministic portfolio risk and tax-aware decision support.
- Dividend and corporate-action awareness.
- Daily briefing and layered decision journal.
- Approval-required live execution through a single Robinhood gateway.
- A future limited auto-live state with explicit manual unlock.
- Scheduling, monitoring, audit trails, replay, and fail-closed behavior.
- Periodic governed updates to tax, broker, regulatory, and market knowledge.

Non-Goals

This feature does not include:

- Options, crypto, futures, short selling, leverage, or margin borrowing.
- Intraday, high-frequency, or latency-sensitive trading.
- Access to any Robinhood account except agentic account 605420660.
- Guaranteed returns or an assertion that Kairos will beat the market.
- Autonomous promotion of a strategy to live trading.
- Silent changes to risk policy, tax rules, execution permissions, or champion strategies.
- LLM-generated authoritative prices, P&L, fills, tax calculations, or risk calculations.
- Treating social posts, unsourced articles, or LLM opinions as verified evidence.
- Tax preparation or personalized legal/tax advice. Broker tax documents and professional advice remain authoritative.
- A big-bang rewrite of the existing Next.js application.

Current Behavior and Known Gaps

The repository already contains research, paper-trade, learner, and trade-proposal routes, a paper-trading migration, and agent dashboards. These are prototypes, not conforming implementations of this architecture.

Phase 0 resolved (2026-07-01):

- ??... ResearchAgent processSymbol now computes all 5 scores deterministically from real fetched data (lib/data/scores.ts). LLM only writes thesis + direction (no scores, no prices).
- ??... PaperTrader uses lib/data/quotes.ts (AV GLOBAL_QUOTE ??' price_cache). No MCP/LLM for prices.
- ??... Fill price = ask + 0.05% slippage via computeFillPrice() ??" not LLM-estimated.
- ??... Every fill logged to immutable paper_order_events with price source, bid/ask at fill, spread applied.
- ??... LearnerAgent weight mutation blocked by phase gate (requires 10+ closed trades) + auto-guard (3 consecutive bad runs).
- ??... Long-only enforcement: short direction overridden to neutral for non-held positions.
- ??... Paper NAV refresh uses deterministic batch quotes (no MCP cold-start).

Remaining gaps (Phase 1+):

- Evidence store with source hierarchy, quality rules, and append-only evidence records does not exist.
- Corporate actions (dividends, splits, symbol changes) are not tracked.
- Macro data uses single macro_signals row ??" no FRED/ALFRED vintage history.
- Strategy versioning, champion/challenger governance, and eligibility reports do not exist.
- Python Validation Engine does not exist ??" no deterministic backtesting or historical replay.
- Provider adapters are not behind a replaceable contract interface.

Live trade execution remains approval-required. Paper results are not yet statistically valid evidence (no out-of-sample validation or benchmark attribution).

Proposed Behavior

The platform separates probabilistic reasoning from deterministic financial controls.

```
flowchart LR
    DH["Data Hub"] --> RA["Research Agent"]
    RA --> AA["Hypothesis and Analyst Agent"]
    AA --> VE["Validation Engine"]
    VE -->|pass| SR["Strategy Registry"]
    VE -->|fail| RJ["Rejected Experiment Record"]
    RJ --> AA
    SR --> PE["Paper Execution Engine"]
    PE --> LA["Learner Agent"]
    LA -->|challenger proposal| AA
    SR --> RT["Risk and Tax Engine"]
    RT -->|pass| LTG["Live Trade Gateway"]
    RT -->|reject| NT["No-Trade Decision Record"]
    NT --> LA
    LTG --> UA["User Approval"]
    UA -->|approve| RH["Robinhood Agentic Account"]
    UA -->|reject or expire| NT
    RH --> RC["Order and Fill Reconciliation"]
    RC -->|failure| SP["Safety Paused"]
    DH --> EA["Explainer Agent"]
    RA --> EA
    AA --> EA
    VE --> EA
    PE --> EA
    RT --> EA
```

RC --> EA
EA --> DJ["Daily Brief and Decision Journal"]

Authority Boundaries

- Data Hub: acquires, validates, timestamps, versions, and stores evidence. It does not reason or trade.
- Research Agent: produces sourced research and labels unverified claims. It cannot create authoritative market facts.
- Hypothesis and Analyst Agent: defines hypotheses and candidate strategy versions. It cannot change a champion or production policy.
- Validation Engine: deterministically computes features, backtests, robustness tests, and eligibility reports.
- Strategy Registry: stores immutable versions and enforces lifecycle transitions.
- Paper Execution Engine: simulates approved paper strategies autonomously with realistic market mechanics.
- Learner Agent: reviews outcomes and proposes challengers. It cannot mutate the running champion.
- Risk and Tax Engine: deterministically calculates sizing, exposure, risk vetoes, dividend implications, and tax flags.
- Live Trade Gateway: the only module authorized to invoke Robinhood order review or submission tools.
- User Approval: Vaibhav is the only authority that may promote a live strategy or approve an initial live order.
- Explainer Agent: summarizes stored evidence, calculations, assumptions, and decisions. It cannot invent missing evidence.

Strategy Lifecycle and Promotion

Allowed states:

- draft: hypothesis is defined but not validated.
- testing: immutable version is undergoing historical and robustness tests.
- rejected: evidence failed; reason and artifacts are retained.
- paper_candidate: historical evidence passed and paper deployment is allowed.
- paper_active: frozen version is generating shadow decisions and fills.
- paper_paused: data, risk, or operational issue prevents valid continuation.
- eligible: deterministic evidence report passed the dynamic gate.
- approved_live: Vaibhav promoted the version for approval-required live proposals.
- live_paused: live use is suspended without deleting history.
- retired: version can no longer create new decisions.

eligible is not based on a fixed duration or trade count. The Validation Engine must produce a passing evidence report that assesses:

- Positive out-of-sample expectancy after modeled costs.
- Acceptable drawdown, downside risk, and recovery behavior.
- Effective independent sample size and confidence intervals.
- Market-regime coverage and regime-specific failure behavior.
- Benchmark-relative performance and unintended beta/factor exposure.
- Parameter stability and sensitivity.
- Multiple-testing, selection-bias, and overfitting penalties.

- Liquidity, turnover, and capacity.
- Correlation with active strategies.
- Paper-versus-backtest degradation.
- Tail scenarios and stress tests.

Champion/challenger governance:

- Each strategy family has at most one champion for a given universe and horizon.
- LearnerAgent creates a new immutable challenger version.
- Validation Engine determines statistical eligibility.
- Risk Engine can veto deployment but cannot promote it.
- Vaibhav alone promotes an eligible challenger to champion.
- The existing champion remains active until promotion is complete.
- Rejection returns the challenger to paper study or retires it; it does not modify the champion.

Research and Learning Flow

- Observe a timestamped event or market condition.
- Produce a sourced research packet with contradictions and missing evidence.
- Define a falsifiable hypothesis: universe, horizon, entry, exit, expected direction, invalidation, and economic rationale.
- Define deterministic features computable without an LLM.
- Freeze a strategy version and dataset manifest.
- Run point-in-time historical validation.
- Run transaction-cost, slippage, delay, missing-data, parameter-perturbation, regime, and stress tests.
- Compare against SPY, an appropriate sector/style benchmark, and a no-trade baseline.
- Deploy passing versions to paper execution.
- Evaluate resolved outcomes and create versioned challenger proposals.
- Run the dynamic evidence gate.
- Ask Vaibhav to approve or reject any live promotion.

Learning constraints:

- No weight change from one trade or one short observation window.
- No reuse of held-out evidence for tuning.
- One documented experimental change per challenger where practical.
- Failed and rejected experiments are never deleted.
- Safety may pause or retire a strategy automatically; promotion is never automatic.
- Research discoveries may affect future hypotheses but never rewrite historical evidence.
- The system distinguishes learned market evidence from LLM interpretation.

Data Architecture

Evidence Store

Evidence records are append-only and include observed time, effective time, retrieval time, source, source tier, revision, quality state, and payload hash.

Evidence categories:

- Daily OHLCV and corporate-action-adjusted prices.
- Quotes used for paper or live decisions.
- Dividend declaration, ex-dividend, record, and payment dates.
- Splits, mergers, symbol changes, halts, and delistings.
- SEC filings, XBRL facts, and Form 4 transactions.
- Earnings dates, results, estimates, and guidance.
- FRED/ALFRED macro observations with historical vintages.
- Regime inputs such as volatility, breadth, rates, and credit proxies.
- Robinhood account, position, order-review, order, and fill observations.
- External articles and social observations with verification state.

New source data never silently overwrites old evidence. Corrections create new versions.

Knowledge Store

Versioned knowledge records include:

- Market-mechanics rules.
- Approved risk and execution policy.
- Candidate, validated, rejected, decaying, and retired signals.
- Strategy definitions and assumptions.
- Experiment and eligibility reports.
- Regime-specific observations.
- Prediction and trade outcomes.
- Agent lessons tied to evidence.
- User decisions, approvals, rejections, and overrides.
- Tax, dividend, regulatory, and broker rules with effective dates.

A knowledge claim requires citations, confidence classification, effective date, and review state before it can influence a trade.

Immutable Events and Mutable Projections

- Evidence, decisions, strategy versions, experiment results, approval actions, order reviews, orders, fills, and learning events are immutable.
- Current portfolio state, current strategy state, latest quote cache, and dashboard summaries are mutable projections.

- Every projection must be rebuildable from immutable events.
- Corrections are compensating events, not destructive updates.

Source Hierarchy

- Government, regulator, exchange, issuer, and broker sources.
- Established structured-data providers.
- Reputable financial reporting.
- General web sources.
- Social media and forums.

Levels 4 and 5 may generate hypotheses. A trade-affecting fact must be corroborated by a higher-tier source. Contradictions are quarantined. Missing or stale required data results in `insufficient_data` and abstention.

Free-First Provider Policy

- Robinhood MCP supplies the executable universe, live account state, and executable-decision quotes where supported.
- SEC EDGAR supplies filings, XBRL data, and insider transactions.
- FRED/ALFRED supplies macro data and historical vintages.
- Issuer investor-relations sources supply dividend and earnings evidence.
- TradingView supports manual analysis, alerts, Pine prototypes, and explicit CSV imports only.
- Historical market data is accessed through a replaceable adapter selected after accuracy, adjustment, coverage, rate-limit, and licensing tests.
- An Alpha Vantage MCP mentioned in external review is not considered available until it is added to `knowledge/CONNECTIONS.md` and its tools pass a contract test.

Validation Engine

The Validation Engine is a deterministic Python worker. Claude or another LLM may propose hypotheses and explain reports but cannot calculate authoritative results.

Responsibilities:

- Build point-in-time dataset manifests.
- Compute deterministic features and labels.
- Run historical replay with purging/embargo where labels overlap.
- Model spread, slippage, latency, dividends, splits, and costs.
- Run robustness and stress tests.
- Calculate performance, risk, uncertainty, capacity, and benchmark metrics.
- Produce signed/versioned experiment and eligibility artifacts.
- Reproduce a run from code version, strategy version, dataset manifest, configuration, and random seed.

Next.js orchestrates user requests and displays results. Supabase persists metadata, immutable events, and artifact references. The Python worker runs behind a job contract so hosting can change without changing strategy definitions.

Paper Execution

Paper and live execution consume the same order-intent, strategy, risk, and sizing contracts. Only the execution adapter differs.

Paper behavior includes:

- Long-only equities and ETFs.
- Market-calendar-aware decisions.
- Bid/ask spread, configurable slippage, delayed fills, rejections, and unfilled orders.
- Cash and settlement constraints matching the intended Robinhood behavior.
- Deterministic daily mark-to-market.
- Dividends, splits, symbol changes, halts, and delistings.
- Strategy-specific 2-20 market-day holding periods.
- Benchmark, after-cost, and after-estimated-tax attribution.
- Append-only order intents, simulated orders, fills, cash movements, corporate actions, and valuation events.

Live Execution and Approval

Allowed execution states:

- disabled
- approval_required
- auto_live_unlocked
- safety_paused

Initial behavior is always approval_required. A proposal must display quote, quantity, estimated value, thesis, invalidation, holding period, evidence quality, risk checks, tax flags, portfolio impact, and expiry.

Before submission, the Live Trade Gateway must:

- Fetch current Robinhood account state and quote.
- Verify account number 605420660.
- Re-run cash, risk, tax, event, liquidity, and freshness checks.
- Call Robinhood review_equity_order.
- Show any values that changed since proposal creation.
- Require an unexpired explicit approval.
- Submit once with an idempotency key.
- Reconcile order state and fills.

User rejection or expiry creates a no-trade event and does not alter the strategy. A reconciliation failure enters safety_paused.

Future auto_live_unlocked requires a separate explicit authorization containing strategy version, maximum capital, allowed symbols or universe, risk policy version, start time, expiry time, and revocation state. No agent can create, extend, or broaden this authorization.

Risk Architecture

Hard controls apply regardless of model confidence:

- Volatility-targeted sizing.
- Maximum single-position, sector, factor, and total exposure.
- Daily loss and drawdown circuit breakers.
- Liquidity, spread, and execution-size limits.
- Earnings and corporate-event restrictions.
- Portfolio-correlation and crowding checks.
- LTCM correlated-shock, survival, and liquidity checks.
- Maximum daily order count.
- Stale, missing, or conflicting data veto.
- No averaging down unless explicitly specified and validated.
- No leverage, options, shorting, or non-agentic account access.

Risk failure creates a no-trade event with machine-readable reasons. It cannot be overridden by an LLM.

Dividend and Tax Architecture

Every relevant decision evaluates total return, not dividend yield alone.

Dividend inputs:

- Declaration, ex-dividend, record, and payment dates.
- Amount, frequency, currency, and special-dividend status.
- Expected mechanical ex-date adjustment.
- Payout coverage and sustainability evidence.
- Corporate-action revision history.

Tax-aware outputs:

- Tax-lot cost basis.
- Realized and unrealized gains.
- Short-term versus long-term status.
- Qualified-dividend holding-period estimate.
- Ordinary versus qualified dividend classification.
- Wash-sale exposure across all known relevant lots/accounts where data is available.
- Estimated after-tax expected return.
- Year-to-date realized tax exposure.
- `review_required` when treatment is uncertain.

Tax estimates are advisory. Robinhood tax documents and professional tax guidance are authoritative.

Evolving Rules and Knowledge Governance

Update authority depends on knowledge class:

- Market observations update automatically after data-quality checks.
- Corporate events update automatically after source validation.
- Research findings enter as unverified hypotheses until tested.
- Broker capability changes require contract validation before activation.
- Tax and regulatory changes must come from authoritative sources and create proposed rule versions.
- Risk and execution policy never changes automatically.

Rule-update flow:

- Detect a source or rule change.
- Store old and new versions with effective dates.
- Corroborate when possible.
- Produce a plain-language difference and impact report.
- Identify affected strategies, holdings, lots, and pending decisions.
- Re-run applicable calculations and tests.
- Require Vaibhav's approval for behavior-changing rules.
- Activate prospectively without rewriting history.

Ex-dividend data refreshes daily and immediately before any dividend-sensitive proposal or order. Tax sources are checked periodically and when authoritative updates are detected. Uncertainty produces `review_required` and can block tax-sensitive action.

Explainability and User Learning

Daily Briefing

The pre-market brief includes:

- Regime and confidence.
- Portfolio exposure and risk state.
- Earnings, macro, and ex-dividend events.
- Active paper strategies and experiments.
- Candidate opportunities ranked by evidence quality.
- Actions under consideration but not executed.
- Prior predictions versus outcomes.
- New lessons, failed assumptions, and uncertainty.
- Required user decisions.

Material changes can produce an updated brief. Every claim links to evidence or an experiment artifact.

Layered Decision Journal

Each decision exposes:

- Plain-language summary.
- Evidence, sources, timestamps, contradictions, and missing data.
- Signals, calculations, regime, expected return, uncertainty, and benchmark.
- Strategy version, governance state, risk checks, tax flags, approvals, order review, fills, and resolved outcome.

Every proposal answers why now, why this security, why this size, why act instead of wait, what invalidates the thesis, the strongest opposing case, the correlated-shock risk, and what similar historical experiments showed.

Content labels are explicit: `verified_fact`, `calculated_metric`, `model_estimate`, `agent_interpretation`, `unverified_hypothesis`, and `user_decision`.

User Journey / System Flow

- Vaibhav opens the daily briefing and sees regime, risks, events, experiments, and pending decisions.
- ResearchAgent continuously collects sourced observations and proposes hypotheses.
- Validation Engine tests frozen candidate versions.
- Passing candidates enter autonomous paper trading.
- LearnerAgent reviews resolved evidence and proposes challengers.
- Dynamic evidence gate may mark a challenger eligible.
- Vaibhav reviews the complete evidence and promotes, rejects, or continues paper study.
- An approved-live strategy may create a live proposal.
- Risk and Tax Engine may veto it.
- Vaibhav reviews the Robinhood order review and approves or rejects.
- Live Trade Gateway submits and reconciles the approved order.
- The journal later connects outcomes and lessons to the original decision.

Screen / Page / Module Inventory

Existing screens to evolve

- `/dashboard/agents`: system health, runs, experiments, champion/challenger status, and safety state.
- `/dashboard/trading`: paper/live portfolios, proposals, approvals, orders, fills, and kill switch.
- `/dashboard/intelligence`: sourced research feed and hypothesis inbox.
- `/dashboard/markets`: eligible universe, evidence quality, regime, events, and signals.
- `/dashboard/portfolio`: account, tax lots, exposure, dividends, and risk.

New product surfaces

- Daily briefing view.
- Decision journal detail.

- Strategy registry and version comparison.
- Experiment report and eligibility review.
- Data-quality and source-health view.
- Rule-change review.
- Auto-live authorization control, hidden until the future phase is approved.

UI Architecture

Layout

Use the existing dashboard shell and current approved design system. No independent visual redesign is included.

Required Components

- System safety state.
- Evidence-quality badge.
- Fact/estimate/interpretation labels.
- Strategy lifecycle badge.
- Champion/challenger comparison.
- Experiment metric and regime panels.
- Risk and tax check list.
- Source citations and freshness.
- Proposal expiry and changed-value warning.
- Approval/rejection controls.

Empty State

Explain which prerequisite is missing: no source data, no hypothesis, no validated strategy, no paper history, or no proposal. Never display fabricated sample results as real.

Loading State

Show the specific run or data source being awaited and retain the prior timestamped result as stale, not current.

Error State

Show the failed dependency, affected strategies, whether execution is blocked, retry state, and audit identifier.

Success State

Show persisted run/result identifiers and the next lifecycle action. A successful research run is not described as a successful strategy.

System Architecture

Modules

- TypeScript/Next.js orchestration and authenticated API layer.
- Supabase immutable events, evidence metadata, projections, auth, and RLS.
- Python validation worker behind a job contract.
- Replaceable market, filings, macro, issuer, and broker adapters.
- Deterministic paper execution, risk, tax, and live gateway modules.
- LLM research, hypothesis, learner, and explainer modules with structured outputs.
- Scheduler and run monitor suitable for a future Railway worker deployment.

Core Contracts

- EvidenceRecord: source, tier, observed/effective/retrieved times, revision, quality, hash, payload reference.
- ResearchPacket: symbol/universe, claims, citations, contradictions, missing evidence, interpretation, confidence.
- Hypothesis: universe, horizon, direction, entry, exit, invalidation, features, rationale, expected failure modes.
- StrategyVersion: immutable definition, parent, change summary, code/data/config versions, lifecycle state.
- ExperimentRun: strategy version, dataset manifest, costs, metrics, regimes, artifacts, reproducibility metadata.
- EligibilityReport: gate results, uncertainty, failures, risk vetoes, recommendation, generated time.
- OrderIntent: strategy, account, symbol, side, quantity logic, limit logic, expiry, thesis, evidence references.
- RiskDecision: pass/reject, policy version, sizing, exposure, stress results, reasons.
- TaxDecision: lots, holding periods, dividend status, wash-sale flags, after-tax estimate, uncertainty.
- ApprovalRecord: user, exact reviewed payload hash, decision, time, expiry.
- ExecutionEvent: review, submit, broker order, status transition, fill, reconciliation.
- DecisionJournalEntry: linked evidence, calculations, interpretations, governance, and resolved outcome.

Exact database columns and endpoint payload schemas belong in the implementation plan and migrations, derived from these contracts without changing their semantics.

Auth and Permissions

- All user-facing and agent-triggering routes require Supabase authentication.
- Only the superadmin user may approve live strategies, orders, rule changes, or auto-live authorizations.
- Service jobs use a narrowly scoped server credential and cannot call the live gateway unless the authorization contract permits it.
- Robinhood tokens are never persisted in application tables or logs.
- Account 605420660 is enforced at the gateway and verified against the broker response.

Failure Handling

- Missing/stale price: no decision or order.

- Conflicting sources: quarantine and abstain.
- Invalid LLM structured output: discard; never partially persist as evidence.
- Provider outage: pause dependent strategies.
- Missed scheduled run: do not backfill trades using future data.
- Duplicate request: idempotent replay of the prior result.
- Partial fill: reconcile before further dependent orders.
- Account mismatch: immediate safety pause.
- Risk calculation unavailable: no trade.
- Tax uncertainty: warn or block based on policy severity.
- Repeated job failure: bounded retry, dependency pause, and alert.

Scheduling and Operations

- Overnight: data reconciliation and corporate actions.
- Pre-market: evidence refresh, regime calculation, event calendar, and daily brief.
- Intraday: material event and source-health monitoring appropriate for swing trading.
- Post-close: valuation, paper fills, outcome checks, and experiment metrics.
- Weekly: strategy, challenger, decay, and data-quality review.
- Periodic: authoritative tax, regulatory, broker-capability, and source-policy review.

Every run stores inputs, outputs, code/config versions, timing, status, errors, freshness, and retry history.

Testing Architecture

- Unit tests for calculations, lifecycle transitions, sizing, dividends, taxes, and authority boundaries.
- Provider contract tests for freshness, adjustments, rate limits, schema, and error behavior.
- Historical replay tests using point-in-time fixtures.
- Explicit leakage, survivorship, and corporate-action tests.
- Deterministic paper-order and accounting tests.
- Failure injection for stale data, conflict, outage, duplicate order, partial fill, and reconciliation failure.
- Security tests proving only Live Trade Gateway can review or submit Robinhood orders.
- Tests proving initial live orders require an exact, unexpired approval payload.
- Audit reconstruction proving a decision can be reproduced from stored inputs and versions.
- Golden report tests for eligibility and decision-journal outputs.

Delivery Phases

Phase 0: Stabilize Current Agent Loop

- Disable LLM-derived entry and exit prices.
- Disable direct automatic weight mutation.

- Enforce long-only behavior.
- Add a deterministic quote adapter with freshness and provenance.
- Correct paper accounting, valuation, and idempotency.
- Introduce append-only paper events and rebuildable projections.
- Reconcile migration and documentation state.
- Keep all live behavior approval-required.

Phase 1: Data Foundation

- Evidence store, provider adapters, source hierarchy, quality rules, corporate actions, dividends, and macro vintages.

Phase 2: Research Laboratory

- Sourced research packets, deterministic hypotheses/features, Python Validation Engine, experiment artifacts, and Strategy Registry.

Phase 3: Paper Engine

- Realistic shadow execution, benchmarks, attribution, dynamic eligibility, and champion/challenger experiments.

Phase 4: Explainability

- Daily briefing, layered decision journal, rule-change reports, and learning experience.

Phase 5: Approval-Required Live

- Robinhood gateway, exact-payload approval, risk/tax checks, order review, idempotent submission, and reconciliation.

Phase 6: Limited Auto-Live

- Available only under a separately approved architecture amendment after approval-required live execution has been verified.

Files Likely To Change

The implementation plan will narrow each phase. Expected areas include:

- app/api/agents/
- app/api/portfolio/
- app/dashboard/agents/
- app/dashboard/trading/
- app/dashboard/intelligence/
- app/dashboard/markets/
- app/dashboard/portfolio/
- components/dashboard/
- lib/agents/
- lib/data/

- lib/execution/
- lib/risk/
- lib/tax/
- types/index.ts
- supabase/migrations/
- A focused Python validation-worker directory selected in the implementation plan
- knowledge/

Files / Behavior That Must Not Change

- Do not modify AGENTS.md or PRD.md without Architect role and explicit Vaibhav approval.
- Do not access any Robinhood account except 605420660.
- Do not enable live execution without the approved gateway and exact approval workflow.
- Do not add options, shorting, leverage, crypto, or intraday scope.
- Do not use an LLM as an authoritative market-data, P&L, fill, tax, or risk source.
- Do not silently replace existing approved visual direction.
- Do not add a data provider or package without the required architecture and user approval.
- Preserve existing useful UI and integrations unless a phase-specific plan explicitly replaces them.

Acceptance Criteria

Architecture-wide

- Every decision is attributable to immutable evidence, deterministic calculations, strategy version, policy versions, and authority actions.
- Every calculation that affects money is reproducible without relying on LLM memory or prose.
- Missing, stale, or contradictory required evidence results in abstention.
- No agent can promote itself, approve its own live strategy, or execute outside the Live Trade Gateway.

Stabilization phase

- No production route asks an LLM for an authoritative security price.
- Paper execution is long-only and accounting identities reconcile from immutable events.
- Paper NAV uses deterministic, timestamped prices and includes modeled execution costs.
- LearnerAgent cannot directly mutate champion weights.
- Existing live proposals remain approval-required.

Research and validation

- A frozen strategy can be replayed from its versioned dataset and configuration.
- Eligibility reports expose every passing and failing gate with uncertainty.
- Failed experiments and rejected challengers remain queryable.

- Benchmarks, costs, regime coverage, and paper/backtest degradation are included.

Live execution

- Only account 605420660 can pass gateway validation.
- An order cannot submit without current data, passing risk/tax checks, Robinhood review, and an exact unexpired user approval.
- Duplicate submission does not create a duplicate order.
- Partial fills and broker discrepancies reconcile or trigger safety_paused.
- Auto-live remains unavailable until a separately approved later phase.

Explainability

- Daily briefing and decision journal distinguish fact, calculation, estimate, interpretation, hypothesis, and user decision.
- Every displayed claim links to evidence or a stored experiment.
- Every trade proposal shows invalidation, counterargument, correlated-shock risk, tax/dividend flags, and why waiting was rejected.

Implementation Log

Phase 0 - Complete (2026-07-01)

See ARCHITECTURE.md Session 2026-07-01 section for full detail.

All Phase 0 acceptance criteria met:

- All 5 ResearchAgent scores deterministic (no LLM score generation)
- LLM role: thesis + direction only
- PaperTrader uses deterministic quote adapter (price_cache + AV GLOBAL_QUOTE)
- Fill price = ask + 0.05% slippage via computeFillPrice()
- Immutable paper_order_events with UPDATE/DELETE blocked
- LearnerAgent phase gate (≥ 10 closed trades before weight mutation)
- Auto-guard (3 consecutive bad runs blocks mutation)
- Long-only enforcement enforced in code

Phase 0 Addendum - 2026-07-03 Session

Live Portfolio + Trade History Layer added outside original Phase 0 scope (does not block Phase 1 gate).

All-Accounts ResearchAgent Fix

- lib/research-agent.ts::fetchHoldings() - uses only the latest snapshot per live account, then unions open paper alpha positions; older snapshots cannot keep closed names in the universe
- Deterministic held-position reassessment is available across live and paper books; holdings bypass discovery caps

Trade Decision Enrichment Pipeline

- Migration 043 applied: trade_decisions + uploaded_trade_files
- /api/live-portfolio/enrich - batch enrichment with AV TIME_SERIES_DAILY_ADJUSTED; computes outcome_score, macro_market_regime, pattern_tags
- /api/live-portfolio/import-csv - SHA-256 dedup; Robinhood CSV parsing
- LivePortfolioPage.tsx - CSV import UI, enrichment trigger, decisions table

LearnerAgent Enhancements

- Tool query_trade_decisions added (tool #7) - queries real enriched trades by action/regime/outcome range
- Model upgraded: deepseek-chat -> claude-opus-4-8 (89.08% finance accuracy, AIMultiple benchmark)

Phase 1 - Planned [REVIEW PENDING - ChatGPT]

Status: Not started. Gate: Phase 0 complete ?. Can begin.

Key Phase 1 items from original spec + new additions:

- Evidence store (Evidence categories defined in Data Architecture section above)
- Corporate actions (dividends, splits, symbol changes) automated tracking
- Macro data: FRED/ALFRED vintage history (currently: single macro_signals row)
- Provider adapter contracts (replaceable interface in front of AV/FinancialDatasets/Robinhood)
- RAG pipeline [REVIEW PENDING - ChatGPT]
- Voyage-3.5 embeddings (finance-tuned) for trade_decisions
- Supabase pgvector table: trade_decision_embeddings(decision_id, embedding vector(1536))
- gte-reranker-modernbert-base post-retrieval reranker
- New LearnerAgent tool: semantic_search_decisions
- Migration 044 needed
- Langfuse observability [REVIEW PENDING - ChatGPT]
- Wrap lib/llm-router.ts with Langfuse SDK
- Each agent run = trace; each LLM call = generation span
- Firecrawl ResearchAgent integration [REVIEW PENDING - ChatGPT]
- Firecrawl MCP active (needs new Claude session)
- Adds crawl_url tool to ResearchAgent; max 3 calls/run
- Source quality: Firecrawl output = unverified hypothesis (level 4 in source hierarchy)
- ResearchAgent model upgrade [REVIEW PENDING - ChatGPT]
- Current: Groq 70B for thesis
- Proposed: Claude Fable 5 (90.34% finance accuracy) - pending cost gate (\$2/day budget)

Approval

Architecture approved: Yes Approved scope: Governed long-only swing-trading research, paper experimentation, explainability, tax/dividend awareness, and initial approval-required live trading as specified above Implementation allowed: Yes (phased) Phase 0 complete: 2026-07-01

Next gate: Vaibhav approves Phase 1 implementation plan (Evidence store, provider adapters, source hierarchy, corporate actions, macro vintages).

Audit-Driven Remediation & Adaptive-Quant Target (2026-07-10)

Consolidates the full-system Codex audit (CODEX_FULL_SYSTEM_AUDIT_RESULT.md) and its A-J target spec into one governed roadmap. Owner directive: work every item until done.

Verified current-state truth (spot-checked against live DB + code, not docs)

- Autonomous-live never places an order. `lib/trading/autonomous-live.ts` selected `agent_signals.evidence_confidence`, a column that does not exist (real column is `agent_signals.confidence`; `evidence_confidence` lives on `decision_observations`). The query error was swallowed -> zero signals every run. DB-confirmed.
- Autonomous path bypasses the hardened Execution Gateway - direct broker-client calls, omitting `checkKillSwitches`, account allowlist, G1/G3, quote-drift, preview, held-SELL.
- India autonomous sizing uses the USD Robinhood NAV account for INR orders - dimensionally invalid.
- Migrations 139/140 + `reserve_live_order_budget_v2` are applied in prod but have no files - clean DB cannot rebuild. (RPC verified atomic in prod; the gap is reproducibility.)
- Calibration leakage - logistic fit on all rows, then "OOS" fold predictions use those same full-data coefficients. Learned $P(\text{win})$ untrustworthy until fold-local.
- Paper learning loop IS partly real - owner-promoted champion weights + genome do drive next-run scoring and paper sizing/exits. Live path and feature-registry/edge-lab loops are inert/broken. `AUTONOMOUS_LIVE_ENABLED` stays false until the money-path unification + acceptance tests pass.

Target architecture (A-J, condensed - the north star)

- A. Separate prediction / portfolio-construction / execution into independently testable layers.
- B. Small mixture of validated setup experts (quality-momentum, price-trend/RS, PEAD, value inflection, ETF rotation, India quality-momentum) - not one ever-growing composite; not a 6th weight.
- C. Real learnable genome (features, weights, routing, entry/exit/sizing/universe/costs) - bounded, hashed, immutably linked to dataset/labels/code/validation. Money ceilings, accounts, kill switches, autonomy mode, secrets, ledger are not genes.
- D. Research tournament: hypothesis -> measure-only -> purged walk-forward -> shadow -> paper A/B -> owner live-review -> capped canary -> scale/retire. Deflated-Sharpe/FDR, lockbox, no "any-horizon-wins".
- E. Point-in-time, market-specific data plane (event/available/retrieved times, dated universe + delistings, corporate-action-consistent prices, US+NSE calendars, separate USD/INR NAV).

- F. Learn from the broad decision population (every considered candidate incl. rejects), not fills only.
- G. Performance-truth + attribution per market (TWR/MWR, alpha/beta, Sharpe/Sortino/Calmar, DD, turnover, net-of-cost, calibration curves).
- H. Autonomy as one governed control system - shared execution service, market/account/currency pinned, atomic claim+idempotency, fresh kill-switch, reconciliation, protective exits, short lease, tiny capital, instant disable.
- I. Honest success metric - beat an investable benchmark net of costs within DD/tail limits; degrade safely; adapt faster than edge decay; full reproducibility. NOT "beat everyone in every regime".
- J. 16 acceptance tests (PIT reproducibility, no future leakage, no US/India cross-contamination, no research->money jump, no duplicate orders, one shared gateway, kill cancels resting orders, RLS matrix, clean-DB replay). Feature is not "done" until these pass.

Remediation checklist (work top-to-bottom; update status inline)

Legend: [] todo ? [~] in progress ? [x] done

Track 1 - correctness / safety / reproducibility (no new architecture; unambiguous fixes)

- [x] R1 #4 autonomous-live signal query: select real confidence, capture error, FAIL run + alert on error
- [x] R2 #5 add checkKillSwitches(svc, market) to the autonomous path (fresh, per market, fail closed)
- [x] R3 #1 restore migrations 139/140 + reserve_live_order_budget_v2 + broker_order_events DDL from prod -> files (143_restore_live_auto_ddl.sql)
- [x] R4 #9 autonomous fail-closed: require valid future lease + per-market trading_enabled_*===true (no null-passes)
- [x] R5 #14 calibration fold-local fitting (fit scaler+logistic inside each train fold; refit-all only for deploy artifact)
- [x] R6 #19 positive-allowlist score_source='deterministic_v1' on paper+trader eligibility (was fail-open negative filter)
- [x] R7 #22 evidence_confidence = weighted structural coverage over the 5 scored dims (not dimension count)
- [x] R8 #33 weighted-score: explicit abstain flag for <2 dims (no meaningless partial number mistaken for a score)
- [~] R9 #16/#31 owner-gate: DONE smart-money, theme-scout GET, mentor evaluate/journal/scores, import-csv. TODO metered public data (options/chain, social/sentiment, charts/*)
- [x] R10 #17/#32 RLS: owner-scoped corporate_actions/evidence_records/experiment_runs/paper_order_events/strategy_versions/trade_decision_embeddings + service-only universe_snapshots* (migration 144)
- [x] R11 #18 vault: PIN stored as salted script hash (lib/vault-pin.ts), legacy-plaintext compat verify (no lockout). Key-value envelope-encryption still TODO (separate, larger).
- [x] R12 #34 redacted literal cron secret from migration 022 text (dead - job unscheduled by 052, CRON_SECRET already rotated). NOTE: git history still contains it (rotation, not rewrite, is the real fix - done).

R13 design note - shared execution service (REVIEW BEFORE CODING)

Problem: the manual gateway (`app/api/broker/orders/route.ts`) is the hardened path (owner+CSRF, approved/unexpired proposal, dup check, symbol/qty validation, active-broker resolve, autonomy-level + trading + broker enablement, fresh `checkKillSwitches`, G1 decision quality, account allowlist, fresh-quote + currency notional cap, portfolio gate, held-SELL, drift/preview, atomic reservation). `autonomous-live.ts` reimplements a WEAKER subset and calls brokers directly. Two implementations = drift + the autonomous path missing controls.

Target: one server-only `lib/trading/execute-order.ts::executeApprovedOrder(svc, input, actor)`:

- actor: { kind: 'owner' | 'autonomous_worker', runId?, csrfOk?, ownerSessionOk? }

- Runs the FULL invariant list once, in order, for both callers. Autonomous may NOT supply any

owner-only override (accept-low-quality, force flags); those are rejected when `kind !== 'owner'`.

- The ONLY broker-write call site in the codebase. Returns a typed result

(submitted | rejected | needs_reconcile + broker ref + the durable reservation id).

Safe migration order (each step build+verify before the next):

- Extract the invariant block from `broker/orders/route.ts` into `executeApprovedOrder` with an

owner actor - the manual route delegates to it and behaves identically (diff = pure move).

- Verify the manual owner-click path is byte-for-byte equivalent (same gates, same responses).

- Repoint `autonomous-live.ts` to call `executeApprovedOrder` with `autonomous_worker` - deleting

its direct `rhPlaceMarketOrder/placeEquityOrder` calls and its weaker inline checks.

- Kite manual route (`app/api/kite/order`) folded in the same way.

Risk control: step 1-2 must not change manual behavior at all. If any manual gate/response differs, stop. Autonomy stays false throughout. No new capability - only consolidation.

Depends on: R14 (per-market NAV - done), R15 (idempotency - done). Blocks R16 (exit plane).

STATUS: awaiting owner review of this approach before implementation (it modifies the working

manual order path).

Track 2 - money-path unification (architecture-gated; build after Track 1)

- [x] R13 #2/#5/#10/#11 shared `lib/trading/execute-order.ts::executeApprovedOrder(svc, input, actor)` - the SINGLE hardened invariant set. Manual gateway delegates (owner actor); autonomous-live delegates (autonomous_worker actor, its own daily caps, no overrides). Autonomy now inherits account allowlist, fresh kill-switch, G1/G3, notional cap, drift, held-SELL, atomic v2 reservation. Serverless caveat RESOLVED: added a direct-REST Robinhood adapter (`lib/brokers/adapters/robinhood.ts, id robinhood`) - submit/status/cancel over REST, works in Vercel serverless. Owner sets `strategy_config.active_broker_us='robinhood'` to route live US orders through it (both manual + autonomous). Flag stays false until R16 + acceptance tests.
- [x] R14 #3/#8 per-market currency-correct NAV (US=RH USD snapshot, India=Kite INR margins+holdings, fail-closed) + per-market NET open-position count (was global filled count)
- [x] R15 #7 atomic signal claim - unique partial index on `trade_proposals(signal_id,market)` WHERE `autonomous_live` (migration 145) + 23505 idempotent skip. (Broker-level client-order-key deferred - RH REST lacks native idempotency.)
- [x] R16 #6 control plane COMPLETE: reconcile (broker-sync polls `getOrder` -> fill/partial/status write-back + mismatch alert, uses the robinhood REST adapter) + cancel-on-kill (sync cron cancels resting live orders when a kill switch trips) + LIVE protective exits (`lib/trading/live-exit-monitor.ts` + cron every 30min in-session: reconstructs open live positions from filled `broker_orders`, SELLS via the gateway on stop -8% / target +20% / time 15d; SELLS are held-only,

cap-exempt, journaled). Market-HOLIDAY + authoritative AV MARKET_STATUS gate in market-calendar.ts.

- [x] R17 #12/#28 per-market cron (?market=us 15:00 UTC / ?market=india 06:00 UTC) + isMarketSessionOpen session-window guard (no market orders when exchange closed, DST-correct). Full managed-scheduler consolidation still TODO.
- [~] R18 #13 learner baseline now from market champion snapshot + full-vector renormalize to 1.0. TODO #15: transactional per-market promote + force_unvalidated->paper-only (strategies/versions route).

Track 3 - statistical + evolution integrity (after Track 2)

- [] R19 #21 label maturation: exchange-calendar sessions + immutable executable decision price (no future candle)
- [] R20 #23/#24 edge lab: dated PIT universe + delistings, predeclared horizon + FDR, Newey-West lag = overlap
- [] R21 #26 rename feature-registry "active"->"research_validated"; wire only via shadow->paper->promote
- [] R22 #27 autonomous sizing: per market/setup/horizon calibrated artifact; unknown edge -> size 0
- [] R23 #20 held-exit direction deterministic (LLM advisory-only schema: veto/rationale/risks)
- [] R24 #29/#30 past-trade: RFC-4180 parse, immutable raw rows, transaction taxonomy, position/lot episodes, session/CA-consistent enrichment

Track 4 - target architecture (A-J; phased, data-gated)

- [] R25 skeleton: three-layer contract (alpha model -> portfolio constructor -> execution policy) interfaces + provenance columns
- [] R26 setup-expert framework (B) with one real expert in shadow
- [] R27 J acceptance-test fixtures; clean-DB replay in CI
- [] R28 performance-truth attribution extension (G)

Docs (docs/arch/*, system-map.json, AGENTS.md, PRD.md) reconciled per ?7 drift register as each item lands.

asset allocation - Feature Architecture

Source: `features\asset-allocation\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Feature Architecture - Asset-Class Allocation

Status: Deterministic CORE built + SHIPPED OFF (migration 175). Genome

evolution, the slow rebalancer, sizing wiring, and UI are the validated

follow-up. Owner defaults chosen (below) - change any and say so.

Last updated: 2026-07-13

Built (2026-07-13, ships OFF)

- Migration 175: ``strategy_sleeves(market, sleeve, target_pct, min_pct, max_pct, instruments, enabled) seeded per market + strategy_config.allocation_enabled`` (default false -> allocator inert; zero behaviour change today).
- `lib/allocation/allocator.ts`: deterministic `allocate(sleeves, regime)` - tilts targets by macro regime WITHIN hard bands, drops disabled sleeves, renormalizes to 100%. `computeAllocation(svc, market)` returns null unless enabled. NO LLM. Unit-tested (`tests/allocator.test.ts`).
- Default decisions used (owner-tunable in `strategy_sleeves`):
 - US: equity 70% (0-90), defensive_etf 20% (0-50, SHY/IEF/TLT/GLD), cash 10% (5-100), leveraged OFF (0-15).
 - India: equity 70%, defensive_etf 20% (LIQUIDBEES.NS/GOLDBEES.NS), cash 10%.
 - Leveraged sleeve disabled; rebalance cadence weekly / 5% deadband (documented; rebalancer not built yet).

Follow-up (NOT built - money path + evolution)

- Wire the equity-sleeve target into paper-trade sizing (only when enabled).
- Slow rebalancer for defensive/cash sleeves (weekly cron, deadband).
- Genome allocation block + LearnerAgent proposal + Validation gate.
- Allocation UI (current vs target per sleeve).

Turn it on with `allocation_enabled=true` only after 1-3 land + are validated.

Original proposal (retained)

Status: PROPOSAL - not built. Needs owner approval before implementation.

Last updated: 2026-07-13

Problem

Today the book is long-only single-name equities + a few ETFs (metals basket, region ETFs) with cash as the residual. There is no allocation across asset classes - no bonds, no defensive/leveraged sleeves, no target weights, no "hold more cash / bonds when the regime turns risk-off." Sizing is per-position (Kelly / `position_size_pct`); the learner/genome evolves scoring weights + entry/exit/horizon/sizing, NOT the asset mix. So the portfolio cannot shift its *shape* to defend or press an edge as markets change - it only picks stocks.

The ask: can the agents figure out a balance across bonds / US ETF / cash / stocks / leveraged ETF for max gain in any market, and keep evolving it?

Design (proposed)

A thin allocation layer ABOVE the existing stock picker - it never replaces scoring; it sets how much capital each sleeve gets, and the picker fills the equity sleeve as it does now.

1. Sleeves (per market / currency - never cross-summed)

Define named sleeves with target bands, e.g. (US):

```
| Sleeve | Instruments | Band |
| `equity` | scored single names (current pipeline) | 0-80% |
| `defensive_etf` | bond/treasury ETFs (e.g. SHY/IEF/TLT), gold | 0-50% |
| `cash` | residual | 5-100% |
| `leveraged` | leveraged/inverse ETFs - **OFF by default, hard-capped, opt-in** | 0-15% |
```

India mirrors with its own instruments (liquid-bond ETFs, GOLDBEES, cash).

2. Deterministic allocator (no LLM on the money path)

A pure function `allocate(regime, riskProfile, genome) -> targetWeights` maps the existing `macro_regime` signal + risk profile to sleeve targets within the bands. Risk-off -> more cash/defensive; risk-on -> more equity. Deterministic, auditable, bounded by the bands. (Consistent with the CLAUDE.md rule against fragile explicit bull/bear *scoring* switches - this governs *allocation*, not per-name scoring, and stays inside hard bands.)

3. Learner evolves the mapping (bounded, validated)

Extend the genome with an `allocation` block: the regime->sleeve-target mapping + band edges. The `LearnerAgent` proposes challenger allocations; they go through the same champion/challenger + Validation Engine gate (walk-forward, fail-closed) before a human promotes. So it "figures it out and mutates" - but only within hard bands, only via validated, owner-promoted challengers. Never unbounded, never LLM-driven on the money path.

4. Execution

The trader/paper-trader sizes new entries against the equity sleeve's current target, not the whole NAV; defensive/cash sleeves are held as ETF/cash positions rebalanced on a slow cadence (e.g. weekly, with a no-churn deadband) to avoid overtrading. Kill-switch / drawdown / per-market controls all still apply.

Safety

- Leveraged sleeve OFF by default, hard-capped, explicit opt-in.
- All targets clamped to bands in code (not just config).
- Rebalance deadband to prevent churn.
- Paper-first; live only after the same gates as today.

- No cross-currency summing; each market allocates its own pool.

Scope (build order, if approved)

- Migration: `strategy_sleeves` (targets/bands per market) + genome allocation block.
- `lib/allocation/allocator.ts` - deterministic `allocate()` + band clamp + tests.
- Wire equity-sleeve sizing into paper-trade/trader.
- Slow rebalancer for defensive/cash sleeves (cron, deadband).
- Genome extension + LearnerAgent proposal + Validation gate + backtest.
- UI: allocation panel (current vs target per sleeve); docs (arch-08/09 + system-map).

Open questions for the owner

- Which bond/defensive instruments per market? (US: SHY/IEF/TLT/GLD? India: liquid-bond ETF + GOLDBEES?)
- Enable the leveraged sleeve at all, or leave it off?
- Rebalance cadence + deadband tolerance.
- Paper-only first (recommended) before any live allocation.

atr paper exit policy - Feature Architecture

Source: `features\atr-paper-exit-policy\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

ATR Paper Exit Policy

Status: MEASURE-ONLY PHASE APPROVED AND IMPLEMENTED

Date: 2026-07-22

Decision

Do not implement fixed 1.5 ATR partial exits, 2.5 ATR full exits, or a 15-point below entry score hard exit in the current paper loop. Those values are hypotheses, not validated policy parameters, and would create a second exit engine beside the existing transactional PositionMonitor path.

Kairos will first measure ATR-normalized exit outcomes in the existing evidence and validation substrate. A later, explicitly approved version may replace the current risk/reward policy only after it clears market-local, out-of-sample validation and paper shadow evidence.

Why The Handoff Design Cannot Ship As Written

- `paper_positions`, not `paper_trades`, is the open-state authority.

`paper_trades` is an immutable lot/realized-outcome ledger. PositionMonitor already invokes `execute_paper_exit`, which atomically splits partial lots, adjusts the remaining position, credits only the same market pool, and preserves FIFO parity. Storing `partial_exit_qty`, `peak_price`, or trailing state only on `paper_trades` can re-trigger a partial exit on the same open position and break lot/position reconciliation.

- A 15-point drop from entry is not the current deterministic thesis rule.

It would exit an 90-score entry at 74 even while it remains comfortably above the market mandate's validated exit threshold. Current score exits use fresh, same-market evidence and threshold-minus-hysteresis; this avoids treating a still-strong score as a broken thesis.

- ATR parameters must not be tuned from intuition. Kairos already records MAE,

MFE, horizons, and point-in-time observations for the specific purpose of learning a future volatility/pattern-conditioned risk-reward policy. Fixed values would pre-empt that governed surface and contaminate paper outcomes.

- A once-daily monitor cannot honestly simulate intraday touch-triggered ATR

barriers. It may evaluate close-based paper exits only unless an approved, point-in-time intraday fill model is added. It must not call a daily close a fill at a level that may never have been executable.

Existing Policy To Preserve

- PaperTrader binds every fill to an entry-time stop and target through `resolveExecutionRiskReward` and `bindTradePrices`.

- The source is the market-local mandate until at least 60 eligible MAE/MFE labels support a learned alternative.

- PositionMonitor is the only paper exit executor. It runs time, fresh

score/direction, stop, target, and existing one-time half-target logic in a defined order, then calls the service-only `execute_paper_exit` RPC.

- The RPC owns FIFO lot closure, remaining quantity, cash, native currency, and decision journal writes. No new client or route may bypass it.

- US and India stay separate for prices, ATR units, data sources, policy evidence, positions, cash, and evaluation.

Measure-Only Phase (Implemented)

- Extend immutable `decision_observations` / `observation_labels`, not the

execution tables, with derived ATR-normalized MAE/MFE diagnostics where the entry candle history is adequate. Record null rather than inventing an ATR.

- Evaluate the versioned `close_observed_v1` family of three stop/partial/

target/trail candidates against the same frozen entry opportunities. The simulation observes completed daily closes, matching PositionMonitor's cadence, and applies the existing 10 bps round-trip haircut. It never claims an intraday barrier fill from a daily candle.

- Expose a read-only, market-and-horizon-local evidence report. A candidate is

`insufficient_sample` until it has at least 60 labels and 12 effective horizon-adjusted observations. This status is descriptive only: it cannot promote or activate an exit policy.

- Write only matured label evidence and health telemetry. No change to

`agent_signals`, `PaperTrader`, `PositionMonitor`, `positions`, `cash`, `broker proposals`, or `live orders`.

Implemented contracts

- Entry ATR is read from `immutable decision_observations.features.technical.atr14`.

Missing or invalid ATR remains null.

- `observation_labels` owns `entry_atr`, ATR-normalized MAE/MFE, the versioned

candidate outcomes, and the policy version. Existing labels are reprocessed only when the version is missing or stale.

- `GET /api/analytics/atr-exit-evidence?market=us|india&horizon=2|5|10|20`

returns coverage and candidate aggregates for authenticated owner review.

- No execution module imports the candidate family. The database migration has

no trigger or RPC capable of changing positions, cash, signals, or orders.

Still required before paper shadow

After enough prospective labels accrue, add the predeclared purged walk-forward, locked-holdout, drawdown/turnover, and trial-family correction evaluation. A `reviewable_evidence` aggregate is not approval and cannot skip that gate.

Later Paper Shadow Gate

After a candidate is statistically eligible, run it as a market-local, non-executing shadow beside the incumbent exit policy. It must use the same entry price, quote source, close convention, slippage model, and horizon. It may become a paper policy only after non-inferiority on drawdown and net return, no material turnover regression, reproducible records, and explicit owner approval.

Explicit Non-Goals

- No broker-hosted stops, protective orders, live execution, Webull enablement, router enablement, or change to `protective_orders_enabled`.
- No re-entry exception that bypasses the existing name cap, cooldown, deterministic score, market-control, or kill-switch gates.
- No hardcoded ATR multiplier or score-velocity constant in the money path.
- No `paper_trades` mutation that represents live position state.

Acceptance Criteria For A Future Implementation

- One versioned exit policy owns all partial/full paper exits.
- State lives on `paper_positions` and all transitions use `execute_paper_exit` in a single transaction.
- Every exit is reproducible from versioned evidence and a documented fill convention.
- The policy is separately validated and market-local; it does not borrow US results to configure India or vice versa.
- Existing exits, cash accounting, FIFO parity, long-only constraints, kill switches, and fresh-score protections remain intact.

Evidence Basis

- Kaminski and Lo show that whether a stop-loss adds value depends on the return process and regime rather than a universal parameter. Kairos must therefore validate its own market-local policy out of sample, net of costs. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=968338
- Hsu, Taylor, and Wang found that strategies selected in sample can fail out of sample even when stop-loss variants are considered. This is why the candidate family, holdout, and trial-history record are mandatory rather than optional. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3158101

automated strategy validation - Feature Architecture

Source: features\automated-strategy-validation\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature Architecture: Automated Strategy Validation and Shadow Routing

Status

Architecture status: Approved by owner Approved date: 2026-07-12 Roles: Codex Architect + Builder Implementation allowed: Yes

Purpose

Run evidence validation whenever Kairos creates a strategy challenger, without requiring an owner to press a Backtest button. A passing challenger may enter bounded, non-executing shadow evaluation automatically. It must never replace a champion, alter a user mandate, activate paper fills, or enable live trading.

Product Contract

- LearnerAgent creates an immutable challenger as it does today.
- If automation is enabled for that market, the same server action runs the deterministic Validation Engine before reporting the learner result.
- A passing challenger is atomically moved to shadow_paper only when the market has no other automated shadow challenger.
- ResearchAgent records shadow decisions only; shadow state cannot create orders, paper fills, cash movement, or live proposals.
- A scheduled sweep retries challengers that have no validation record, so a restart or transient local HTTP failure cannot leave a challenger untested.
- The owner can disable automation independently for US and India. Disabling stops new automatic validation and shadow activation immediately; it does not delete challengers, experiments, or historical shadow evidence.

Authority

hard safety/risk gates -> user trading mandate -> champion -> automated validation -> shadow evidence -> owner promotion -> paper/live.

Validation is deterministic and LLM-free. The manual Backtest page remains an exploratory tool and cannot promote a strategy. Passing validation establishes only statistical eligibility for shadow evidence; the existing owner-only, fail-closed champion-promotion RPC remains unchanged.

Data Model

strategy_validation_automation, one row per market:

- market (us | india) primary key;

- enabled: permits automatic deterministic validation;
- auto_shadow_enabled: permits a passing candidate to enter shadow state;
- max_active_shadows: hard bounded at one for the initial release;
- audit timestamps and updated_by.

The schema seeds both markets enabled because this feature is explicitly approved. Missing-table or missing-row behavior is fail-closed: existing manual validation remains available but no automatic state changes occur.

activate_strategy_shadow(version_id) is a service-role-only RPC. It locks the market, confirms an enabled policy and a passed experiment belonging to the challenger, rejects terminal/champion states, enforces the shadow cap, then transitions only that challenger to shadow_paper.

Flows

```

flowchart LR
    L[LearnerAgent] --> C[Immutable challenger]
    C --> V[Deterministic Validation Engine]
    V -->|fail or insufficient evidence| R[Recorded evidence; challenger remains inactive]
    V -->|pass and policy permits| S[Atomic shadow activation]
    S --> D[ResearchAgent shadow decisions]
    D --> O[Owner-only champion promotion]
    O --> P[Existing paper/live gates]

```

The validation sweep considers only challengers with no prior validation experiment. It is bounded per market. A failed experiment is preserved rather than overwritten; a future evidence-refresh design must use an explicit dataset-version rule rather than silently rerunning failed experiments.

TradingView Boundary

TradingView/Pine remains optional, manual corroboration and webhook-based forward-test input. No TradingView UI automation, scraping, or assumed general backtest-results API is part of this architecture. Kairos authority evidence comes from its stored, reproducible US/India data snapshots.

Reversibility

- Set enabled=false to stop automatic validation for one market.
- Set auto_shadow_enabled=false to retain automatic evidence generation but stop automatic shadow routing.
- Existing shadow strategies can be manually paused, rejected, or retired with the existing registry controls; no history is deleted.
- Removing the scheduled sweep only removes retries. No schema or data rollback is required to return to manual validation.

Files

- supabase/migrations/170_strategy_validation_automation.sql
- lib/validation/automation.ts
- app/api/agents/learner/route.ts
- app/api/validation/sweep/route.ts
- app/api/settings/validation-automation/route.ts

- components/dashboard/ValidationAutomationPanel.tsx
- app/dashboard/settings/page.tsx
- scripts/run-agents.ps1
- scripts/register-tasks.ps1

Non-Goals

- A second historical backtesting engine.
- Automatic champion promotion, paper fills, broker proposals, or live orders.
- TradingView credential/API integration or browser automation.
- Rewriting any existing validation result or shadow decision.

Acceptance Gates

- Per-market disable blocks new automatic validation and shadow activation.
- A new challenger validates synchronously through the server action, with no fire-and-forget local HTTP dependency.
- A passing challenger can create at most one active automated shadow per market.
- Failed/insufficient validation leaves a challenger inactive and is auditable.
- The sweep retries only unvalidated challengers.
- Owner promotion remains mandatory and fail-closed on passed evidence.
- Typecheck, tests, production build, and dependency audit pass.

benchmark alpha - Feature Architecture

Source: `features\benchmark-alpha\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Feature Architecture - Benchmark Alpha Scorecard

Status: P1 BUILT 2026-07-13. Phase 1 measurement is implemented; Phase 2 learner/promotion wiring remains unbuilt.

Scope: deterministic measurement first; no order path, no paper fills, no learner mutation in Phase 1.

Build gate: Phase 2 learner/promotion wiring requires a separate owner approval after Phase 1 has accumulated evidence.

Update when built: `docs/arch/04-database-schema.md`, `docs/arch/05-crons-and-scheduling.md`, `docs/arch/09-learning-loop.md`, and `public/agent-diagrams/system-map.json`.

Purpose

Kairos already stores some benchmark information:

- `paper_performance.bench_nav`, `bench_return_pct`, `alpha_pct` - a single per-market paper benchmark series.
- `live_performance.account_id`, `date`, `equity`, `bench_nav` - a forward-built live account equity curve currently tied to VOO.
- `investment_mandates.benchmark_symbol` and `strategy_evaluations.book_alpha_pct` - mandate/evaluation context.

What is missing is a trustworthy multi-horizon, per-market, per-book benchmark scorecard that answers:

- Are paper and live beating the configured benchmark over 1W, 1M, 3M, YTD, and 1Y?
- Is the result statistically usable or just short-window noise?
- Which benchmark is primary for the learning objective, and which are comparison-only?
- Can a benchmark be added without hardcoding VOO/NIFTY everywhere?

This feature belongs inside the existing Performance Truth Layer, not beside it. It is a materialized measurement surface and later an input to governed learner/promotion decisions.

Non-Negotiable Design Rules

- No cross-currency roll-up. US USD books and India INR books are never summed, compared, or converted implicitly.
- Benchmark currency must match book currency. US books benchmark against USD-traded benchmarks such as V00; India books benchmark against INR/index series such as ^NSEI/NIFTY source symbols.
- Common-window rebasing only. Portfolio and benchmark start from the same first date where both have valid levels inside the requested horizon.
- No action from noisy windows. Phase 1 is measurement-only. Phase 2 can only use longer, confidence-qualified windows.
- Deterministic only. No LLM computes, fills, overrides, or gates alpha.
- Existing series are source ledgers. `paper_performance` and `live_performance` remain the source series; `benchmark_scorecard` stores rollups so dashboards/agents read one stable, audited number.

Reviewed Design Issues And Fixes

1. Live aggregation was underspecified

Failure scenario: `live_performance` is keyed only by (`account_id`, `date`) and has VOO in `bench_nav`. A future rollup that sums all rows by date can mix Webull + Robinhood, trading + read-only, or even future India accounts. The resulting "live alpha" would be an accidental account set, not a market book.

Fix: Add explicit live series provenance before live scorecarding:

- Add nullable `market`, `currency`, `broker`, and `book_scope` columns to `live_performance`, backfilled from `live_account_snapshots/broker` registry where possible.
- Phase 1 scorecard uses canonical scopes only:
- `paper`: one market-level paper pool from `paper_performance`.
- `live`: one market-level live book per (`market`, `currency`, `book_scope`='all_live_accounts'), summing only accounts whose rows match that market/currency/scope.
- If provenance is missing for any live row in the window, the live scorecard row is `status='insufficient_data'`, not estimated.

2. Info-ratio math was ambiguous

Failure scenario: The proposal said "excess / stdev of daily excess" but did not specify units. If cumulative 1M excess return is divided by daily tracking error without annualization/window scaling, the ratio is not comparable across 1W, 3M, and 1Y.

Fix: Persist both components:

- `excess_return_pct` = `portfolio_window_return_pct` - `benchmark_window_return_pct`.
- `tracking_error_daily_pct` = `sample stdev(daily_portfolio_return_pct - daily_benchmark_return_pct)`.
- `info_ratio` = `mean(daily_excess_return_pct) / stdev(daily_excess_return_pct) * sqrt(252)`.
- If tracking error is zero, non-finite, or sample size is too small, `info_ratio`=null and `status='insufficient_data'`.

The dashboard may show cumulative excess for intuitive reading, but learner/promotion logic may only consume the annualized daily information ratio.

3. YTD/1Y edge cases were not guarded

Failure scenario: On January 3, a YTD row might have 1-2 observations and produce a huge positive/negative alpha; on a newly created live account, 1Y might silently use two days and look complete.

Fix: Each row stores `window_start`, `window_end`, `n_observations`, `n_return_days`, and `coverage_pct`.

Confidence gates:

```
| Horizon | Minimum return days for display | Minimum for Phase 2 use |
| `1W`   | 4 | never used for gating |
| `1M`   | 15 | never used for promotion |
| `3M`   | 45 | 60 |
| `YTD`  | min(20, elapsed trading days - 1) | max(60, 70% of elapsed trading days) |
| `1Y`   | 120 | 180 |
```

Rows below the display floor are stored with `status='insufficient_data'`. Rows below the Phase 2 floor are display-only.

4. Benchmark priceability was too optimistic

Failure scenario: A user adds ^NSEBANK or a sector ETF that the current provider cannot price. If the rollup simply skips it with a log line, the UI and agents can mistake absence for neutrality.

Fix: Add a small benchmark observation ledger and explicit row status:

- benchmark_price_observations: one row per (benchmark_id, date, component_symbol) with close, currency, provider, and source_status.
- The rollup writes a benchmark_scorecard row for every enabled benchmark/horizon/book even when the benchmark is unpriceable, with status='benchmark_unpriceable' and missing_reason.
- UI shows the failure state; agents treat it as unavailable.

5. Primary benchmark uniqueness must be scoped

Failure scenario: A single partial unique index on (market) where is_primary works for one benchmark per market, but blend/single representations and future inactive rows can accidentally leave two "primary-like" configs or none.

Fix: Use a normalized benchmarks table with stable IDs and an explicit partial unique index:

```
unique (market) where is_primary = true and enabled = true
```

The Settings/API update that flips primary must be transactional: clear old primary and set new primary in one service-role operation. If no enabled primary exists for a market, Phase 2 is disabled for that market and Phase 1 rows are comparison-only.

6. Phase 2 could overfit the learner to beta/regime

Failure scenario: A high beta strategy can beat VOO in a risk-on month and get promoted even if it is simply more volatile. Conversely, a defensive strategy can lag in a melt-up and be rejected despite doing its job.

Fix: Phase 2 cannot replace the existing validation gate. It is an additional guard:

- Challenger still must pass walk-forward validation (validation_experiments.passed=true) and existing sample/evidence gates.
- Primary-benchmark info ratio is one metric in the Performance Truth row, not the only objective.
- Promotion requires either:
 - positive primary info ratio over the mandate horizon with sufficient sample, or
 - explicit owner override/journaled reason when the mandate intentionally differs from the benchmark.
- Learner objective uses a blended score: validation pass/fold stability first, then primary benchmark IR, then drawdown/cost/slip. No direct mutation from scorecard rows.

7. Posture feedback was too action-like

Failure scenario: A negative 1W alpha row triggers posture tightening, which reduces entries after random weekly noise - the same failure class as the prior phantom drawdown cascade.

Fix: Posture feedback is advisory and slow:

- Only 3M, YTD, or 1Y primary rows may create trailing-benchmark:* alerts.
- Requires two consecutive completed rollups below threshold.
- Alert severity is warn, not critical.
- It never toggles market_controls, risk profile, order size, or live-auto settings. It recommends review only.

Data Model - Phase 1

benchmarks

Config rows. One enabled primary per market.

```
| Column | Type | Notes |
| `id` | uuid PK | Stable key used by scorecard rows |
| `market` | text | `us` or `india` |
| `label` | text | Human label: `V00`, `NIFTY 50`, `QQQ`, `60/40` |
| `kind` | text | `single` or `blend` |
| `symbol` | text nullable | Single-symbol benchmark display/source symbol |
| `provider_symbol` | text nullable | Provider-specific ticker, e.g. `V00`, `^NSEI`, `NIFTY50.NS` |
| `currency` | text | `USD` or `INR`; must match market/book |
| `price_provider` | text | `massive`, `yahoo`, `kite_yahoo`, `manual_import` |
| `is_primary` | boolean | One enabled primary per market |
| `enabled` | boolean | Disabled rows retained for history |
| `weights` | jsonb nullable | Blend components: `[benchmark_id, weight]`; weights must sum to 1.0 |
| `created_at` / `updated_at` | timestamptz | |
```

Seed:

- US: V00, USD, provider massive, primary.
- India: NIFTY 50, INR, provider symbol used by the existing quote path (^NSEI or NIFTY50.NS, whichever the implementation proves priceable), primary.

benchmark_price_observations

Small durable price ledger for benchmark components. This avoids refetching every horizon and makes missing data auditable.

```
| Column | Type | Notes |
| `benchmark_id` | uuid FK |
| `component_symbol` | text |
| `date` | date |
| `close` | numeric |
| `currency` | text |
| `provider` | text |
| `source_status` | text | `ok`, `missing`, `provider_error`, `unpriceable` |
| `error` | text nullable | Short provider error; no secrets |
| PK | `(benchmark_id, component_symbol, date)` | |
```

benchmark_scorecard

Materialized rollup. This is the always-on number.

```
| Column | Type | Notes |
| `market` | text | `us` or `india` |
| `currency` | text | Must match book currency |
| `book` | text | `paper` or `live` |
| `book_scope` | text | `market_paper_pool`, `all_live_accounts`; future account subsets require explicit scopes |
| `benchmark_id` | uuid FK |
| `benchmark_symbol` | text | Snapshot display value |
| `is_primary_snapshot` | boolean | Snapshot at compute time |
| `horizon` | text | `1W`, `1M`, `3M`, `YTD`, `1Y` |
| `as_of` | date | Rollup date |
| `window_start` / `window_end` | date | Actual common window used |
| `portfolio_return_pct` | numeric | Rebased over common window |
| `bench_return_pct` | numeric | Rebased over common window |
| `excess_return_pct` | numeric | Portfolio minus benchmark |
| `daily_excess_mean_pct` | numeric | Mean daily excess return |
| `tracking_error_daily_pct` | numeric | Sample stdev of daily excess |
| `info_ratio` | numeric | Annualized daily excess / tracking error |
| `n_observations` | int | Common level observations |
| `n_return_days` | int | Common daily return pairs |
| `coverage_pct` | numeric | Common valid observations / expected trading observations |
```

```
| `confidence` | text | `insufficient`, `low`, `medium`, `high` |
| `status` | text | `ok`, `insufficient_data`, `benchmark_unpriceable`, `book_series_missing`,
`currency_mismatch`, `stale_series` |
| `missing_reason` | text nullable | Explain unavailable rows |
| PK | `(market, currency, book, book_scope, benchmark_id, horizon, as_of)` | |
```

RLS: owner-read, service-role writes. No public/anon policy. Add indexes on (market, book, book_scope, as_of desc) and (benchmark_id, as_of desc).

Computation Contract

Series loading

Paper:

- Source: paper_performance(date, market, nav).
- Currency: USD for us, INR for india.
- Existing bench_nav/alpha_pct remains for the legacy single-benchmark chart; the scorecard computes its own benchmark series for config-driven rows.

Live:

- Source: live_performance rows with explicit market, currency, and book_scope.
- Aggregation: sum equity by date only within the same (market, currency, book_scope).
- No constant-holdings estimate may feed benchmark_scorecard; estimated live charts stay visually labeled and are not learner inputs.

Benchmark:

- Source: benchmark_price_observations, filled by the rollup from provider adapters.
- For single: one close series.
- For blend: combine component daily returns with fixed weights, rebalanced daily for measurement. Blend weights are normalized only if the stored sum is within a small tolerance; otherwise row status is benchmark_unpriceable.

Windowing

- Determine target horizon start:
- 1W: as_of minus 7 calendar days.
- 1M: as_of minus 30 calendar days.
- 3M: as_of minus 90 calendar days.
- YTD: January 1 of as_of year.
- 1Y: as_of minus 365 calendar days.
- Filter portfolio and benchmark levels to dates >= target start and <= as_of.
- Inner join by date. Do not forward-fill for scorecard math.
- Use the first joined row as the base for both portfolio and benchmark.
- Compute simple daily returns from adjacent joined rows.
- If common rows or return pairs fail the horizon confidence floor, write status='insufficient_data'.

Math

Use fractions internally, persist percentages for UI consistency.

```
portfolio_return_pct = (portfolio_last / portfolio_first - 1) * 100
bench_return_pct     = (bench_last / bench_first - 1) * 100
excess_return_pct    = portfolio_return_pct - bench_return_pct

daily_excess_pct[i]   = portfolio_daily_return_pct[i] - benchmark_daily_return_pct[i]
tracking_error_daily_pct = sample_stdev(daily_excess_pct)
daily_excess_mean_pct = mean(daily_excess_pct)
info_ratio            = daily_excess_mean_pct / tracking_error_daily_pct * sqrt(252)
```

Never divide cumulative excess by daily tracking error.

Surfacing - Phase 1

- Dashboard Alpha Scorecard:
- Market switch: US/India.
- Book tabs: paper/live.
- Rows: horizons.
- Columns: enabled benchmarks.
- Cell: cumulative excess return, annualized info ratio, confidence/status badge.
- Missing/unpriceable rows are visible, not omitted.
- Agent helper:
- `getAlpha(svc, {market, book, bookScope, horizon, benchmarkId?})`
- Returns only the latest scorecard row with status/confidence.
- Agents must treat non-ok or low-confidence rows as unavailable.

Objective And Feedback - Phase 2

Phase 2 is not part of the Phase 1 build. It needs a separate architecture approval after Phase 1 data exists.

Allowed Phase 2 wiring:

- Add primary benchmark scorecard fields to `strategy_evaluations` snapshots:
- `primary_benchmark_id`
- `primary_benchmark_symbol`
- `primary_horizon`
- `primary_excess_return_pct`
- `primary_info_ratio`
- `primary_alpha_confidence`
- LearnerAgent may read these fields as evidence but may not mutate weights from them directly.
- Promotion gate remains: existing walk-forward validation pass first, then benchmark-alpha guard second.
- Benchmark guard consumes only confidence-qualified 3M, YTD, or 1Y primary rows, never 1W/1M.
- Alerts are advisory:
- `trailing-benchmark:<market>:<book>` only after two consecutive qualified underperformance rows.

- Warn-only, clears after recovery.
- No automatic pause, kill switch, order-size change, risk profile change, or live-auto change.

Extensibility

Adding a benchmark should be possible by inserting a config row, but only if the provider can price it:

- Insert benchmarks row as disabled.
- Run a priceability check for at least 30 recent observations.
- If priceable and currency matches market, enable it.
- To change primary, call a service-route transaction that ensures exactly one enabled primary per market.

Blend benchmarks ship after single-symbol benchmarks:

- Component benchmarks must already be enabled and priceable.
- Weights must sum to 1.0.
- Blend currency must match every component and the target market.

Build Order

- P1A - series/provenance migration
- Add benchmark config/observation/scorecard tables.
- Add live performance provenance columns or an explicit live-account-to-market mapping needed for safe aggregation.
- P1B - deterministic math library
- Pure functions for common-window rebasing, daily excess, tracking error, info ratio, confidence/status.
- Unit tests for missing dates, zero tracking error, January YTD, short 1W/1M windows, USD/INR mismatch, and live multi-account aggregation.
- P1C - rollup route + cron
- Owner/cron-gated route.
- Writes every enabled benchmark/book/horizon row, including unavailable statuses.
- No learner/order side effects.
- P1D - dashboard + helper
- Alpha Scorecard UI.
- getAlpha read helper.
- Docs/system-map updates.
- P2 - gated objective wiring
- Only after P1 evidence and owner approval.
- Add Performance Truth snapshot fields and learner/promotion read-only consumption.
- Add warn-only posture reporter.

Files - Phase 1

- supabase/migrations/20260713143000_benchmark_alpha_rotation_shadow.sql

- lib/analytics/benchmark-alpha.ts
- app/api/agents/benchmark-scorecard/route.ts
- components/dashboard/AlphaScorecard.tsx
- tests/benchmark-alpha.test.ts
- docs/arch/04-database-schema.md
- docs/arch/05-crons-and-scheduling.md
- docs/arch/09-learning-loop.md
- public/agent-diagrams/system-map.json

What Not To Build In Phase 1

- Do not change LearnerAgent objectives.
- Do not change promotion gates.
- Do not change risk posture automatically.
- Do not add live-order, paper-fill, sizing, or broker behavior.
- Do not use estimated live backcasts in scorecard rows.
- Do not silently skip unpriceable benchmarks.

briefing - Feature Architecture

Source: features\briefing\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature: Daily Briefing

Status: v1 shipped (2026-07-04). v2 partially shipped; live holding-risk integration shipped 2026-07-21.

v1 (done)

Newsletter-grade email: structured HTML data blocks (accurate, code-built) + short LLM editor's note. Sections: header -> editor's note -> portfolio health score -> market snapshot + regime -> agent signals -> positions -> earnings -> learning/phase -> footer deep-links. Subject formula with preheader substance. Route: `app/api/briefing/generate/route.ts`.

v2 - REQUIREMENTS (user, 2026-07-04) - NOT yet built

The guiding rule from the user: "the more details and explanations, the more I like it. Top bullets OK, but explanation and details are MANDATORY."

- Lighter theme - not an all-dark background. Mixed/light layout.
- Explain every metric - the Portfolio Health score and each Market

Snapshot number must carry a short "what this is + why it matters" line, not just the number. No unexplained figures.

- Outlook sections with confidence - add:
- Market outlook (today + forward) with a stated confidence level.
- Positions-I-hold outlook - per holding, where it's headed and why + confidence.
- Future market & position outlook - near-term expectation + confidence.
- Agent activity recap - what the Research agent and Learner agent actually

did (a) on the day of the newsletter and (b) over the past 7 days. Concrete: runs, signals, hypotheses, weight challengers, rescore flags.

- Mentor block - what the Mentor agent has for the user to learn, based on

current market + the user's behavior + how they're doing on the learning curve. (Depends on the Mentor AI agent - see features/mentor-agent.)

- Structure - top bullets for scannability, THEN mandatory explanation/

detail beneath each. Details are the point, not an afterthought.

Design note for v2: keep the accurate code-built data blocks, but pair each with an LLM-written explanation/outlook (grounded, with confidence), and pull the 7-day agent recap from `agent_runs` + `learner_runs` + `rescore flags`.

Live Holdings Risk addition (approved and shipped 2026-07-21)

- Each US/India edition reads the latest complete `holding_risk_runs` row per

live broker account for that market, then replays its immutable `holding_risk_snapshots` and `account_risk_snapshots` rows.

- The briefing never recomputes risk and never calls a broker or market-data

provider for this section. The LLM receives the deterministic postures only as grounded advisory context and cannot alter them.

- Accounts, markets, and currencies remain separate. Every row must match the selected run's `run_id + market + currency + account_id`; mismatches are discarded instead of cross-summed or relabelled. The internal paper-book adapter is excluded from this explicitly live section, and account identifiers embedded in labels are masked to their last four characters.

- The email shows every non-hold posture plus the three highest-risk ordinary holds per account. It states how many lower-priority holds were omitted and links to the complete Risk Analytics page.

- Snapshot date, session age, formula version, confidence, missing inputs, current price, NAV weight, unrealized P&L, score, posture, and deterministic action reason are displayed. Missing values stay unavailable, never zero.

- Every displayed live holding also carries the latest canonical ResearchAgent annotation for the same `(symbol, market)`: score, direction, exact researched timestamp, market-session age/freshness, and holding re-score versus candidate score. Only `deterministic_v1 + session_validated=true` rows count; a failed read, abstention, stale result, and never-researched state remain distinct. This is display-only and never changes Holding Risk.

- The email explains that research conviction and portfolio risk answer different questions. Since hr-v3, concentration is REVIEW, not TRIM, for every account; a high research score never erases measured exposure, but a global trading reference is never treated as an account-specific sell mandate.

- Failed reads and missing complete runs are visible states. A stale or missing snapshot must never be described as safe/current.

- Legacy hr-v1/hr-v2 stored trims are immutable evidence, not current advice.

The shared history normalizer preserves `sourcePosture=trim` for audit and displays REVIEW in both newsletters and Risk Analytics.

capital rotation - Feature Architecture

Source: features\capital-rotation\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature Architecture - Capital Rotation

Status: P0 SHADOW BUILT 2026-07-13. Paper/live execution remains disabled and unbuilt.

Scope: deterministic opportunity-cost reallocation for PAPER first, LIVE approval proposals later.

Last updated: 2026-07-13

Update when built: docs/arch/03-agents.md, docs/arch/04-database-schema.md, docs/arch/08-risk-and-safety.md, docs/arch/09-learning-loop.md, public/agent-diagrams/system-map.json.

One-line decision

Build capital rotation as a deterministic evaluator inside the existing entry flows, not as a new autonomous reallocation agent. It may only propose or execute a rotation after the candidate has passed every normal entry gate except cash. PositionMonitor still owns exits. Live rotation is approval-required and two-leg reconciled.

Problem

When a market-local book is fully invested and a materially better candidate appears, the current paper/live flows skip the buy with `insufficient_cash`. Existing exits are absolute risk or thesis exits: score-below-exit, stop, target, time-stop, and partial-profit. The missing behavior is opportunity-cost reallocation: sell the weakest still-held, still-valid position only when the new candidate is materially better after cost, tax, turnover, and safety gates.

Non-negotiable rules

- No LLM on the money path. LLM output may explain a signal upstream, but rotation selection, thresholds, tax/cost math, and execution decisions are deterministic.
- Per-market and per-currency only. US/USD and India/INR books are separate. No cross-market cash, NAV, alpha, turnover, or holdings aggregation.
- Existing gates remain authoritative. Rotation cannot bypass owner auth, app pause, per-market pause, kill switch, drawdown breaker, market controls, account allowlists, live autonomy level, approval-required live mode, name/sector/gross caps, quality gates, correlation gates, volatility gates, re-entry cooldown, or position-count caps.
- PositionMonitor exits win. If a holding would exit today for stop, target, time-stop, score-below-exit, direction flip, thesis break, or partial-profit workflow, rotation must not treat that holding as a discretionary funding source.
- Paper execution must be atomic. A paper rotation is one transaction-like sell-and-buy operation. The system must not sell W and then leave the book idle because the buy failed.
- Live execution is two-leg and reconciled. The live path creates an approval-required rotation proposal. The sell leg must confirm before the buy leg is allowed. Partial fills or unknown status stop the workflow in `needs_reconcile`; no automatic buy follows.
- Default off. Shadow measurement is first. Paper auto-rotation and live proposals have separate flags and are both false by default.

Prioritized design issues found and required fixes

P0 - Sell-then-buy can create unintended de-risking

Failure scenario: PaperTrader sells the weakest holding, then the candidate buy fails because price moved, cap checks changed, the fill RPC rejects, or the daily notional cap is exhausted. The portfolio is now unintentionally in cash even though the rotation did not complete.

Fix: P1 paper execution must be a single atomic operation, preferably a Supabase RPC that validates the same snapshot, inserts both trade legs, updates/deletes the sold position, inserts the new position, adjusts `paper_portfolio.cash`, writes `rotation_events`, and aborts all writes on any failed invariant. If the existing paper fill RPC cannot support this, add a dedicated `execute_paper_rotation` RPC before enabling paper auto-rotation.

P0 - Live rotation cannot be a one-click opaque sell-and-buy

Failure scenario: The app submits a live sell, assumes proceeds are available, then submits a buy while the sell is partially filled, rejected, delayed, or in unknown status. This can overspend, bypass budget truth, or buy before the account is reconciled.

Fix: Live rotation emits a `trade_proposals` rotation record with explicit sell and buy legs, `approval_required`, market/account/currency, idempotency key, and full reason JSON. Approval submits only the sell leg through the canonical execution gateway. The buy leg is eligible only after broker-confirmed sell fill and refreshed account/budget data. Any partial fill, unknown state, broker error, or stale account snapshot moves the proposal to `needs_reconcile` and blocks the buy.

P0 - Rotation can fight PositionMonitor

Failure scenario: A holding is near a stop or has a score-below-exit signal today, but rotation sells it as "weakest" and records the action as an opportunity-cost rotation. That hides the true exit reason and corrupts learning labels.

Fix: Rotation source eligibility must run after PositionMonitor predicates or reuse a shared deterministic `evaluateExitEligibility` helper. Holdings with any current exit, partial-profit, near-stop, near-target, stale quote, stale score, or pending order state are excluded. If a holding has an exit reason, PositionMonitor owns the exit and the learning label.

P0 - Entry gates must be replayed after the hypothetical swap

Failure scenario: Candidate C passed standalone checks before the sell, but after replacing W with C the book breaches sector exposure, symbol/name cap, gross exposure, correlation, volatility, max positions, or daily notional limits.

Fix: The evaluator must apply a hypothetical portfolio state: remove W, add C at proposed size, then run the same portfolio-construction gates used by paper/live entry. Rotation is allowed only if the post-rotation book passes every gate.

P1 - Score-only edge is too noisy for auto-rotation

Failure scenario: A one-day score fluctuation makes C beat W by the margin, causing a buy-then-rotate-out whipsaw. This is the same risk class as a phantom weekly drawdown cascade, but now it sells holdings.

Fix: Paper auto-rotation requires persistence: C must beat W by the configured margin in at least two consecutive eligible research runs, or the latest run plus a prior run inside a configured freshness window. Shadow mode may log single-run opportunities, but auto paper cannot act on them. Live proposals require persistence and confidence-qualified benchmark-alpha data when the alpha term is enabled.

P1 - Cost/tax hurdle is underspecified

Failure scenario: `edge_value` is not mapped to dollars/rupees, so a taxable gain or spread/slippage cost can be smaller than a score gap but larger than the actual expected economic benefit.

Fix: Define expected value deterministically:

```
expected_edge_value =  
    target_notional *  
    clamp(expected_excess_return_C - expected_excess_return_W, -cap, cap) *  
    confidence_discount
```

Costs subtract estimated sell spread, buy spread, commissions/fees if any, market-impact/slippage buffer, and tax drag. For paper, tax drag is modeled and logged. For live, tax drag must use lot-level cost basis when available. If cost basis or holding period is unavailable and `tax_sensitivity` is medium/high, fail closed.

P1 - Turnover budget needs a ledger definition

Failure scenario: The system treats only the sell notional as turnover in one place and sell+buy in another, allowing more churn than the mandate intended.

Fix: Define rotation turnover as `sell_notional + buy_notional`, measured against same-market book NAV for the current calendar month. Budget consumption is checked before execution and logged to `rotation_events`. If `investment_mandates.turnover_budget_monthly` is null, rotation budget is zero unless an explicit rotation override exists.

P1 - Benchmark-alpha dependency can silently degrade

Failure scenario: Benchmark-alpha is unavailable, so rotation falls back to score-only without changing risk level. Live proposals then optimize on a weaker edge measure than expected.

Fix: P0/P1 shadow may write `alpha_status = unavailable` and use score-only as measurement. Paper auto-rotation may run score-only only if `rotation_allow_score_only_paper = true` and the edge margin is widened. Live rotation proposals cannot use score-only fallback; they require confidence-qualified primary-benchmark alpha or remain blocked.

P1 - Correlation and duplicate exposure need post-swap evaluation

Failure scenario: W is sold and C is bought, but C is highly correlated with another retained holding, creating self-competition and no real portfolio improvement.

Fix: Correlation checks must compare C against the whole post-swap portfolio, not only W. Reject same issuer, duplicate ETF exposure, blocked symbols, leveraged/inverse products, and high-correlation substitutes unless an explicit allowlist exists.

P2 - Audit trail needs lifecycle and idempotency

Failure scenario: A rotation decision is logged after execution with a partial reason. A retry or cron duplicate creates duplicate sell attempts or impossible-to-reconstruct learning labels.

Fix: `rotation_events` is append-only and lifecycle-based: `evaluated`, `rejected`, `planned`, `paper_executed`, `live_sell_submitted`, `live_sell_confirmed`, `live_buy_submitted`, `completed`, `aborted`, `needs_reconcile`. Every row includes idempotency key, market, currency, book/account, candidate, source holding, scores, alpha inputs/status, cost/tax model, turnover budget, gate outcomes, snapshot timestamps, and actor.

Target architecture

Components

CapitalRotationEvaluator lives in `lib/trading/capital-rotation.ts` and is pure/deterministic. It returns a plan, not side effects.

```
type RotationDecision =
  | { action: 'reject'; reason: string; audit: RotationAudit }
  | { action: 'shadow'; plan: RotationPlan; audit: RotationAudit }
  | { action: 'paper_execute'; plan: RotationPlan; audit: RotationAudit }
  | { action: 'live_propose'; plan: RotationPlan; audit: RotationAudit };
```

Call sites:

- `app/api/agents/paper-trade/route.ts`: after C has passed every normal entry gate except cash, call evaluator at the `insufficient_cash` branch.
- `app/api/agents/trader/route.ts`: proposal generation only. It may produce a rotation proposal, never submit an order directly.
- PositionMonitor is not a call site for discretionary rotation. It owns exits and may expose reusable exit predicates.

Evaluator inputs

- Market-local candidate signal C: symbol, market, currency, score, direction, confidence, horizon, expected alpha if available, quote, target notional.
- Market-local open positions with fresh quote, fresh score, stop/target state, holding age, lot/cost basis if available, pending order state.
- Same-market book state: cash, NAV, exposure by name/sector, position count, monthly turnover used, daily notional used.
- Mandate/config: `turnover_budget_monthly`, `min_holding_days`, `max_holding_days`, `tax_sensitivity`, `max_position_pct`, score threshold, exit threshold, rotation flags.
- Safety state: app pause, market controls, kill switch, drawdown breaker, live autonomy mode, account allowlist, approval-required state.
- Benchmark-alpha state: primary benchmark excess return, confidence, status, window metadata.

Source holding eligibility

A holding W is sellable only if all are true:

- Same market and currency as candidate C.
- Still held, no pending order, no stale quote/score, no account/book mismatch.
- Not below exit threshold, not direction-flipped, not stop/target/time-stop/partial-profit eligible today.
- Not near stop or near target inside configured buffers.
- Holding age is at least `min_holding_days`.
- Selling W would not violate tax/cost guardrails.
- Selling W plus buying C improves the post-swap portfolio after all gates replay.

Decision rule

Rotation is allowed only if all are true:

- C is eligible and blocked only by cash.
- W is the weakest sellable holding by fresh deterministic score, with alpha tie-break when available.
- $\text{score}(C) - \text{score}(W) \geq \text{rotation_margin}$.
- Edge persists across the configured run count/window.
- Expected edge value after confidence discount is positive after all costs and tax drag.
- Monthly turnover budget remains available after $\text{sell_notional} + \text{buy_notional}$.
- Post-swap book passes every entry and portfolio gate.
- C is not a duplicate/correlated substitute for retained holdings.
- Per-run and per-day rotation caps are not exhausted.
- The feature flag for the current phase/action is enabled.

Data model

rotation_events

Append-only audit ledger. No UPDATE or DELETE except service migration repair.

Required columns:

- id, created_at, owner_user_id
- market, currency, book_type (paper or live), account_id nullable for paper
- idempotency_key unique per candidate/run/book
- status
- candidate_symbol, source_symbol
- candidate_signal_id, source_position_id
- candidate_score, source_score, score_edge
- benchmark_id, alpha_status, candidate_alpha, source_alpha, alpha_edge
- sell_notional, buy_notional, turnover_consumed
- cost_model_json, tax_model_json, gate_results_json, audit_json
- trade_proposal_id nullable, paper_trade_ids nullable

RLS: owner read only; service role writes. Public, anon, and authenticated direct writes revoked.

Config

Prefer a small rotation_config table keyed by owner/market/book type, or explicit strategy_config columns if the codebase already centralizes strategy flags there. Required flags:

- rotation_shadow_enabled default true for measurement only once built.
- rotation_paper_execute_enabled default false.
- rotation_live_proposals_enabled default false.
- rotation_allow_score_only_paper default false.

- rotation_margin_score, rotation_persistence_runs, rotation_cooldown_days, max_rotations_per_run, max_rotations_per_day.

Implementation must not overload global trading_enabled; rotation has its own flags in addition to all existing money-path flags.

Phasing

P0 - Shadow-only measurement

Built 2026-07-13. The evaluator and append-only audit rows are implemented. On insufficient_cash, PaperTrader records whether a rotation would have been eligible and why. No sell, no buy, no proposal. This validates frequency, churn pressure, score persistence, priceability, and missing tax/lot data.

P0 readiness completion (2026-07-22): New shadow rows reuse the canonical paper exit-plan projection so a holding already due for a time, score, stop, or target exit cannot be mislabeled as a rotation source. They also record distinct-run persistence, current-month paper turnover, mandate budget/tax sensitivity, the existing 5 bps-per-leg paper slippage floor, post-swap constructor replay, and measured candidate-to-remaining-book correlation from frozen symbol_daily_returns. Missing or truncated evidence fails closed and is recorded in p1_blockers; no value is guessed as zero. Agents -> Rotation exposes these blockers per market. This is measurement and visibility only: it does not grant P1 permission.

P1 - Paper auto-rotation, still default off

After P0 evidence is reviewed, enable market-by-market paper execution only. Requires atomic paper RPC, persistence, post-swap gate replay, turnover budget, cost/tax model, and complete audit rows.

Current enforcement (2026-07-22): P1 is not approved. A production audit found the US paper flag enabled while several required gates above were still absent. It was reset to false, and migration 20260722185000 adds a database constraint that prevents either market from enabling paper rotation. The old money-moving RPC is replaced by a claim-verifying stub that always returns p1_guardrails_incomplete and performs no ledger mutation. P0 shadow measurement is the only reachable phase until a later migration removes that constraint and introduces a fully reviewed P1 transaction.

P2 - Live approval proposals

After P1 shows low churn and net-positive behavior, add live proposal generation. Owner approval is required. Sell and buy legs are reconciled separately through the canonical gateway. No autonomous live rotation in this phase.

P3 - Optional owner-approved automation

Out of scope for this architecture. Would need a separate architecture review, broker canary evidence, live reconciliation evidence, and explicit owner approval.

What is sound in the original design

- The core product idea is useful: fully invested books need an opportunity-cost path, otherwise the system holds mediocre but not-yet-exit positions while better candidates are skipped.
- Keeping rotation deterministic and per-market is correct.
- Using mandate fields such as turnover budget, min holding days, tax sensitivity, and max position percent is the right fit.
- Putting live behind approval-required proposals is correct.
- Using benchmark-alpha as an edge tie-break is appropriate once benchmark-alpha has confidence-qualified data.

Recommended build order

- [Done] Add P0 shadow evaluator and `rotation_events` audit table.
- [Done] Reuse the canonical paper exit-plan projection so rotation cannot relabel true exits.
- [Done for P0 measurement] Replay the post-swap paper constructor and measured candidate correlation; live remains out of scope.
- [Done for P0 measurement] Add persistence, turnover, friction, tax-evidence, and truncation tests.
- [Blocked] Validate a score-to-forward-return mapping, configure an owner-approved turnover budget, and collect independent-session evidence before designing a new atomic P1 RPC.
- [Done] Add owner-visible market-local rotation readiness reporting.
- Add live proposal generation only after P1 paper evidence is reviewed.

Single riskiest assumption to validate first

The riskiest assumption is that the current deterministic score plus benchmark-alpha has enough signal stability to justify selling a still-valid holding. Validate this in P0 shadow mode before allowing any paper or live rotation.

Explicit non-goals

- No autonomous live sell-to-buy workflow.
- No rotation across US and India.
- No score-only live fallback.
- No new standalone reallocation agent that initiates orders independently.
- No LLM override or LLM-generated tax/cost/edge math.
- No implementation code until this architecture is approved.

correlation aware construction - Feature Architecture

Source: features\correlation-aware-construction\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Correlation-Aware Portfolio Construction - FEATURE ARCHITECTURE

Status: Reviewed / measurement prerequisite required / P0-P1 activation not approved. Design only.

Last updated: 2026-07-16.

Per-market applicability: both - correlation gating/sizing is strictly per-market (US/USD and India/INR pools are separate; never cross-summed). Cross-market correlation is display-only awareness, never a gate.

Sequencing: folds in after the in-flight Codex paper-autonomy fixes (they own paper-trade/position-monitor right now) and alongside the holdings-priority research fix.

Update this file when: the risk inputs, estimator, selection rule, or the gating/display boundary change.

0. 2026-07-16 adversarial correction (authoritative over the earlier draft)

The earlier draft overstates what is reusable from Holding Risk. Do not wire P0/P1 as originally written:

- Holding Risk computes pairwise correlations in memory only among already-held names. It persists per-name cluster summaries, not the pairwise matrix needed to estimate a new candidate's marginal risk.
- A new candidate is not part of that Holding Risk run, so there is no measured candidate-to-book correlation to consume at entry.
- The persisted per-holding beta is currently derived from sector beta in the account enrichment path; it is not a uniformly measured symbol-to-benchmark beta contract. India can replace the account-level aggregate beta, but that does not make every persisted holding beta measured.
- PaperTrader already estimates candidate daily volatility, but the India implementation fetches candles synchronously on the entry path and the US path depends on whatever history exists in price_cache. This does not satisfy the proposed no-provider-call, point-in-time evidence contract.
- P1 intentionally reorders already-eligible candidates. Therefore "no previously rejected name becomes eligible" can apply only to safety-gate rejection, not selection rank; otherwise P1's stated purpose is impossible.

Required prerequisite before activation

- During ResearchAgent's existing candle fetch, append a compact, immutable per-symbol return observation with market, symbol, as_of, available_at, provider/source, return dates, daily vol, benchmark beta when measurable, overlap count, and input fingerprint. Do not make another provider call for this record.
- Build candidate-to-held pair estimates from observations where available_at <= decision_at, with at least 60 shared sessions, deterministic shrinkage, and sector-proxy fallback explicitly labeled low_confidence.
- Run the measured penalty in shadow only and persist baseline size/rank, shadow size/rank, evidence IDs, fallback reason, and every eligibility flip.
- Activate P0 only after a frozen-cohort test proves no order is larger and no baseline safety rejection becomes eligible. Activate P1 separately after shadow evidence shows the reordering improves diversification without weakening score, mandate, liquidity, capacity, or execution gates.

Until those prerequisites exist, the production constructor remains on its current conservative volatility/sector-proxy behavior. Faking candidate correlations from held-name cluster averages is prohibited.

Implementation notes - ?0 item 1 SHIPPED (2026-07-16), items 2-4 NOT started

Item 1 (the per-symbol return-observation contract) is built. It activates nothing. Items 2, 3 and 4 remain open; the constructor is untouched and still runs on beta: null, dailyVol: null + the sector-proxy constants.

```
| Piece | Where |
| Observation builder + capture (pure, deterministic, no LLM) | `lib/data/return-observations.ts` |
| Shared run-level benchmark series | `lib/data/benchmark-series.ts` |
| Capture hook (concurrent, fail-soft, joined before return) | `lib/research-agent.ts` -> `processSymbol`,
immediately after the existing candle `Promise.all` |
| Tables (append-only, owner-read RLS) | `symbol_return_observations` summaries + `symbol_daily_returns`
frozen per-session series |
| Coverage report (owner-gated) | `GET /api/analytics/return-observation-coverage?minShared=60` |
| Unit tests | `lib/data/return-observations.test.ts` |
```

Hook point. The capture piggybacks on the candles processSymbol already fetches for scoring - it makes no provider call of its own. The candle *source* (previously discarded by .then(r => r.candles)) is now carried alongside the bars so the observation records its provenance. The benchmark series was hoisted, not added: getRegimeFeatures previously fetched SPY/^NSEI inline; both it and the capture now share one promise-deduped 30-min cache, so the run still makes exactly one benchmark load per market. US benchmark closes come from price_cache (a DB read, not a provider call).

Added per-run cost. Provider calls: +0. The same already-fetched candles produce one summary row plus idempotent per-session rows. Unchanged sessions deduplicate by symbol/market/date/source/basis/input fingerprint; provider revisions append instead of rewriting history.

PIT enforcement is doubled up. The builder drops any bar dated after available_at before computing anything (symbol *and* benchmark series), and the table carries check (as_of <= (available_at at time zone 'utc')::date) so a lookahead row is not storable even if the builder were bypassed.

Beta honesty. benchmark_beta is written only when genuinely measured vs the market's own benchmark (us->SPY, india->^NSEI) over >= 60 shared sessions with non-zero benchmark variance. Otherwise it is null and beta_unmeasurable_reason records why. A DB check constraint makes beta and the reason mutually exclusive, so a sector proxy cannot be written into the beta column - the failure mode ?0 called out.

Per-session series (Option A). symbol_daily_returns stores each frozen simple return with both session dates, source, available_at, explicit adjusted_close/raw_close basis, closes, and an immutable input fingerprint. Item 2 can align exact shared sessions point-in-time without refetching candles. Revisions remain separately auditable; an estimator must select the latest row available at its decision cutoff and must not compare incompatible price bases silently.

1. Verified current state (not assumed - read from code)

The machinery is ~80% built and fed fake inputs:

```
| Piece | Reality |
| Sector cap (`max_positions_per_sector`, default 3, market-local) | **Enforced** ? |
| Portfolio Constructor (`lib/portfolio/constructor.ts`) - name/sector/gross/**vol/correlation** rules at
entry | **Runs**, and is correctly **subtractive** ("never increases a size, never force-sells the book") ?
|
| `beta`, `dailyVol` inputs | **`paper-trade` passes `beta: null, dailyVol: null`** -> constructor falls
back to a **flat `0.02` (2%/day) vol for every name** ? |
| Correlation used in the vol budget + Rule-5 haircut | **Proxied from sector**: `corr = sameSector ?
CORR_SAME_SECTOR : CORR_CROSS_SECTOR` - two constants, **not measured co-movement** ? |
| `max_avg_pairwise_corr` (0.7) | Explicitly **informational bound; not solved for directly** ? |
| Real measured beta + aligned-return correlation | **Computed daily** by holding-risk (`lib/risk/holding-
risk.ts`, `lib/risk/correlation.ts` -> `computeCorrelationClusters`) - but powers the **Risk Analytics
display only**, never the entry decision ? |
```

Consequence: two instruments in different nominal "sectors" that move identically (e.g. a semi ETF and a semi name; two USD-revenue India IT names) pass the entry gate as if diversifying. The knobs exist; the numbers exist; the wiring between them does not.

2. Why this matters more than raising the position cap

The research budget already caps position count (a US run scores ~30 of ~102; 72 deferred). Once holdings are re-scored daily, 10/market ? 20 holdings ? most of a run's capacity, leaving ~10 discovery slots. 15-20/market would starve discovery or holdings-freshness.

So the lever is quality of the 10, not quantity: 10 genuinely uncorrelated names beat 15 correlated ones. Do this before revisiting the cap.

3. Non-negotiable boundaries

- Deterministic - no LLM anywhere in estimation, selection, or sizing.
- Subtractive only - preserve the constructor's invariant: rules may only shrink a size or reject a new entry. Never increase a size, never force-sell an existing position because a correlation estimate moved.
- Per-market gating only. Correlation/vol constraints are computed and applied within a market's pool. Cross-market correlation is informational and must never size, gate, or reject.
- Point-in-time. Correlation/beta/vol must be computed from returns available at the decision timestamp. No lookahead; a decision cannot use a correlation estimated with later data.
- Fail-safe, not fail-open. Missing/insufficient risk inputs must degrade to the current conservative behavior (sector proxy), never to "no constraint".
- No new provider calls on the entry path. Reuse the existing daily holding-risk computation + price_cache; entry must not burst providers.

4. Statistical rigor (why the naive version is wrong)

At 10-24 names with limited history, sample correlation matrices are notoriously unstable. Naive implementation will reject good trades on noise.

- Shrinkage (Ledoit-Wolf style) toward a structured target (sector-average or identity) - never raw sample correlation.
- Minimum overlapping history (~60 sessions) per pair; below it, fall back to the sector proxy and label the estimate low_confidence.
- Penalty, not hard gate. Correlation reduces a candidate's effective attractiveness/size; it does not hard-reject at $\text{corr} > 0.7$. A hard threshold on a noisy estimate is a coin-flip rejector.
- Beta ? correlation - keep both, they answer different questions:
- Beta (to market benchmark) -> *how much market exposure does the book carry?* (10 names all at ? 1.5 = a 1.5x levered market bet, even if uncorrelated to each other.)
- Pairwise correlation / clusters -> *how redundant are these names with each other?*

Both are needed; conflating them hides one of the two risks.

- Estimate provenance: every risk input stored with as_of, source, and confidence so a decision can be audited and a low-confidence input can be treated conservatively.

5. Phases

P0 - Wire the real inputs (highest value, cheapest)

Feed measured beta / dailyVol / pairwise correlation - from the existing daily holding-risk computation - into the constructor at entry, replacing beta: null, dailyVol: null and the sector-proxy constants. Apply shrinkage + min-history fallback to the sector proxy. Turns existing dead knobs live with no new data source and no new provider calls.

P1 - Correlation-penalized selection

Rank candidates by score per unit of *marginal* risk added to the current book, not raw score. Greedy: for each candidate, compute its marginal contribution to book variance given current holdings; penalize accordingly.

*This is the owner's stated intent: *a 70-score name uncorrelated to the book should be able to beat an 85-score name that is a clone of what we already own* - and should be able to displace it as a second/third choice.*

Deterministic + auditable: persist the penalty and the marginal-risk term in the decision rationale so "why did it pick #4 over #2" is answerable.

P2 - "These move together" view (owner-facing)

Surface correlation clusters (reuse computeCorrelationClusters) on the Risk/Portfolio surface: which stocks/ETFs actually co-move, with coverage + confidence labels. Display-only.

P3 - Cross-market awareness panel (display-only, never gates)

Show US<->India co-movement at the owner/total level - the blind spot per-market views structurally cannot see (e.g. India IT is largely USD/US-client revenue -> correlates with US tech; a US tech drawdown can hit both pools at once).

Hard requirement: NSE closes before NYSE opens - naive same-day cross-market correlation is an artifact. Must use lag-aligned returns (India day T vs US day T-1, or a documented overlapping-window convention) and treat INR/USD as its own factor. Label explicitly: *informational; not used for sizing or eligibility.*

6. Acceptance / rollback

- P0 must be a no-op-or-safer change: with real inputs the constructor may only shrink or reject relative to today's behavior on the same cohort; prove on a frozen cohort that no position gets *larger* and no previously-rejected name becomes eligible.
- Missing inputs ? identical behavior to today (sector proxy).
- Rollback = a config flag reverting to the sector-proxy constants; no schema loss.
- Tests: shrinkage estimator determinism; min-history fallback; PIT correctness (no lookahead); subtractive invariant (never upsizes / never force-sells); per-market isolation (US inputs never enter India's book).

7. Open decisions (owner / Codex)

- Estimator: Ledoit-Wolf shrinkage vs simpler constant-correlation shrinkage? (Recommend constant-correlation target first - fewer moving parts at N?10-24.)
- Penalty strength: how hard should correlation discount score in P1? Must be a config value, not hardcoded, and ideally learnable later - but fixed until the learner has a validated edge.
- Beta budget: should the book carry an explicit portfolio-beta cap (distinct from vol/correlation), so 10 uncorrelated-but-all-high-beta names still get constrained? (Recommend yes, config, default off until measured.)

- Cross-market lag convention: India T vs US T-1, or overlapping-window? Must be fixed + documented before P3 renders a number.
- Do NOT raise the position cap until P0+P1 ship - confirm this sequencing.

cross sectional rank - Feature Architecture

Source: features\cross-sectional-rank\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Cross-Sectional Ranking - Feature Architecture

STATUS: IMPLEMENTED (P0-P2), OFF by default (entry.rank_pct_min = 0.0). Migration 151 applied 2026-07-11.

Last updated: 2026-07-11 Author: Claude (P0-P2 shipped: lib/scoring/rank.ts, cron Pass 2, genome param, migration 151. P3-P4 validation-replay/learner-proposal still pending.)

Open flag RESOLVED (2026-07-11) - "top-3 by analyst_score win" (?9 borderline item): the rank gate governs cross-group admission only; final ordering of admitted candidates stays by analyst_score (PaperTrader.order("analyst_score", desc) unchanged). Within a comparable group rank_pct is monotonic in analyst_score, so intra-group top ordering is byte-identical, and under today's single-/small-group degraded case (the common case) rank selection and raw-score selection are identical. The divergence only appears once the universe is large enough for multiple valid sector groups - precisely when cross-sectional rank has value. This is behavior-neutral today because the feature ships OFF (rank_pct_min default 0.0) and can only activate via a validated, owner-promoted challenger. No conflict with the locked CLAUDE.md decision.

Scope: US equities, US ETFs, India NSE equities; long-only new positions; 2-20 trading-day swing horizon. Parent doctrine: features/scoring-methodology/FEATURE_ARCHITECTURE.md ?5 ("Universe and comparable groups") and ?3 ("comparable-universe rank + uncertainty"). This document is the concrete, buildable slice of that named P1 gap.

1. Problem statement (the named P1 gap)

Today a candidate becomes a long entry only through an absolute per-stock gate:

- lib/research-agent.ts (processSymbol, ~line 1180):

```
signalDirection = analystScore >= (scoreThreshold ?? 60) ? "long" : "neutral";
```

scoreThreshold comes from the champion genome (genome.entry.score_threshold, bounded [50,75]) -> strategy_config.score_threshold -> min_analyst_score -> 60.

- app/api/agents/paper-trade/route.ts (~line 150) re-applies the same absolute floor:

```
.eq("status", "pending").eq("direction", "long")
.gte("analyst_score", scoreThreshold)
.order("analyst_score", { ascending: false }).limit(10)
```

An absolute 0-100 composite is a calibration-fragile entry rule. analyst_score is a renormalized weighted sum (lib/scoring/weighted-score.ts), not a probability, so a fixed cutoff of 60 silently means "trade more" on strong tapes and "trade nothing" on weak tapes even when the *relative* best names are unchanged. It cannot answer the question a swing book actually needs: "Is this the strongest candidate available in its comparable group today?"

What already exists (do NOT rebuild)

The measure-only half of cross-sectional rank is already shipped:

- Migration 137 (supabase/migrations/137_universe_snapshots.sql) created universe_snapshots and universe_snapshot_scores(universe_snapshot_id, symbol, analyst_score, rank_pct, decision_observation_id).
- Migration 136 added decision_observations.universe_snapshot_id.
- The research cron (app/api/agents/research/cron/route.ts, ~lines 152-215) already:
- inserts one universe_snapshots row per run (source: "mixed"),

- after all symbols score, computes `rank_pct = (# symbols scoring below s) / (n-1)` over the whole run, and
- writes `universe_snapshot_scores`.
- `processSymbol` accepts `universeSnapshotId` and stamps it on `decision_observations`.

Gaps this feature closes:

- The existing `rank_pct` is never consumed by any selection path - it is logged and forgotten.
- It is computed over a mixed universe (US equities + ETFs + metals + region ETFs + India names all pooled), violating scoring-methodology ?5 ("never rank ETFs against companies", per-market/sector comparable groups, minimum sample sizes). It also ranks all scored symbols including thin-evidence / vetoed ones, violating "rank composites after data-quality gates".
- Rank is computed after the per-symbol entry decision is already written, so it structurally cannot gate entry today.

2. Design goals & non-negotiables

Goal: make cross-sectional rank an actionable, correct input to the new-entry decision - layered on top of (not instead of) a conservative absolute floor - while leaving sizing, execution, and the SELL-for-holdings path untouched.

Respect these locked decisions (from CLAUDE.md and scoring-methodology ?2):

- Rank is a scoring refinement, not a regime switch. No explicit bull/bear mode is introduced (?3).
- Dual-bucket momentum/value screener is unchanged; rank sits downstream of scoring, not inside discovery (?3).
- ≤ 3 screener candidates/day; rank can only *reduce* the actionable set, never expand it (?4).
- Long-only for new positions; the held-position exit branch (`isExitSignal`) is not touched (?4).
- An LLM never generates rank, score, direction, or eligibility (scoring-methodology ?2).
- Comparable-universe transforms use a recorded reference universe for the same market/date; the three daily finalists are not a valid reference universe (scoring-methodology ?5 / ?2).

3. Where rank must live in the pipeline

Rank is intrinsically a whole-universe operation: a symbol's percentile is undefined until every peer in its comparable group has scored. The current pipeline decides `direction/entry_eligible` per symbol, streaming, before peers finish. Therefore rank cannot be a drop-in inside `processSymbol`'s existing gate.

Chosen shape: a deterministic second pass in the cron, after scoring, before `PaperTrader`.

```
Pass 1 (unchanged): processSymbol x N -> writes analyst_score, per-dim scores,
                    decision_observations, agent_signals(direction, status='pending')
                    Data-quality state already computed here:
                    thinEvidence, includedDims, evidence_confidence,
                    breakdown veto (baked into technical_score), abstain
```

```
Pass 2 (NEW, deterministic, no LLM): rankAndGate(runId, universeSnapshotId)
  a. Build the ELIGIBLE ranking pool = scored symbols that passed data-quality gates
  b. Partition pool into comparable groups (market x asset-type x sector/bucket)
  c. Compute rank_pct within each group (empirical percentile)
  d. Persist rank_pct + rank_quality to universe_snapshot_scores (replaces the
    current naive mixed-pool computation)
  e. Apply the rank+floor entry policy -> update agent_signals.direction/status
    for NEW candidates only (holdings & exits untouched)
```

```
PaperTrader (minimally changed): consumes the gated signals as today
```

Why a second pass and not gate inside `processSymbol`:

- `decision_observations` is append-only (mutation-blocking trigger, per scoring-methodology ?9) - we must not rewrite the PIT observation. Rank is logged as an **additional** `universe_snapshot_scores` row keyed to the observation, exactly what migration 137 was built for.
- `agent_signals` is mutable (PaperTrader already flips status there) - it is the correct surface to flip a rank-rejected candidate from `direction='long' -> neutral/status='rank_rejected'`.

Two placement options for Pass 2, both acceptable; recommend (A) for auditability:

- (A) Cron reconciliation (recommended). Extend the existing block in `research/cron/route.ts` (~line 193) that already computes `rank_pct`. It runs once, sees the whole universe, writes correct grouped ranks, then applies the gate to `agent_signals` before it chains PaperTrader (~line 249). Fully deterministic, easy to unit-test, one write site.
- (B) PaperTrader join. Leave `agent_signals` alone; have PaperTrader join `universe_snapshot_scores` and apply the rank gate at selection. Less invasive to research, but scatters the entry policy across two agents and re-derives the pool at fill time. Rejected as primary.

CLAUDE.md coupling note: `decision_observations.universe_snapshot_id` and the whole `universe_snapshot_scores` table must be confirmed present in the target DB before Pass 2 ships (they are migrations 136/137). Per the global "verify migration before merging" rule, a `list_migrations / information_schema` check is required at build time; the new columns in ?? add migration ~145.

4. Comparable groups & the eligible ranking pool

4.1 Data-quality gates run BEFORE ranking (the "rank composites after gates" requirement)

A symbol enters its comparable group's ranking pool only if all of the following hold (all already computed in Pass 1):

```
| Gate | Source | Rejected symbols |
| Not thin evidence | `isThinEvidence(includedDims)` (`weighted-score.ts`) - >=2 usable dims | excluded from pool, `direction=neutral` (already) |
| Not abstained | `signalDirection !== "neutral"` from Pass 1's mechanical gate | excluded |
| Breakdown veto not fired | already folded into `technical_score` cap 20 (`detectBreakdownVeto`) | stays in pool but scores low naturally |
| Evidence confidence floor | `decision_observations.evidence_confidence >= RANK_MIN_CONF` (default 0.60, mirrors scoring-methodology ?7) | excluded from pool |
| Is a **new** candidate | `isHeld === false` | holdings never enter the new-entry rank pool |
```

Excluded symbols still get a `universe_snapshot_scores` row with `rank_pct = null`, `rank_quality = 'excluded_<reason>` for auditability, but are not rank-eligible for entry. This is the operational meaning of "rank composites after data-quality gates".

4.2 Comparable groups (scoring-methodology ?5, made concrete)

Partition the eligible pool by:

- US equity / ADR: `market='us' x sector` (`decision_observations.features.fundamental.sector`) when the sector has $\geq$ `RANK_MIN_GROUP_EQUITY` (20) eligible names that day; else fall back to `market='us' x asset-type=equity` (all US equities pooled).
- US ETF: `market='us' x ETF category` (broad / sector / thematic / leveraged-excluded). Never ranked against single companies. Given daily ETF counts are tiny (metals basket + region ETFs, typically < 5), ETFs almost always fall to the degraded path (?4.3).
- India equity: `market='india' x NSE sector` when $\geq$ `RANK_MIN_GROUP_INDIA` (15); else `market='india' x large/mid liquidity bucket`.

Groups are keyed off fields already present in `decision_observations.features` and entry metadata - no new fetch. Sector for US comes from AV OVERVIEW.Sector (already stored in fundamental evidence); India sector from the screen

cache / Yahoo mapper.

4.3 Small-group degraded path (the "three finalists are not a universe" guard)

If a comparable group has fewer than its minimum-sample threshold eligible names:

- Do NOT invent a percentile from 3-5 names.
- Mark `rank_quality = 'degraded'` and fall back to a pre-registered fixed transform: `rank_pct := clamp01((analyst_score - RANK_FLOOR_LO) / (RANK_FLOOR_HI - RANK_FLOOR_LO))` with fixed constants (e.g. 45 -> 0, 80 -> 1). This makes the rank gate collapse to a slightly-softened absolute gate exactly when there is no valid cross-section - the honest degenerate behavior.
- A degraded rank may drive paper selection but is flagged so the learner and any future live path can treat it as lower-quality evidence.

Because the realistic daily universe today is small (~6 screener + watchlist + holdings, one market per cron), most runs will start on the degraded path. That is acceptable and honest: the value of true cross-sectional rank grows as the PIT universe widens (scoring-methodology ?5/?12). The architecture is correct now and improves automatically as universe breadth increases.

5. How rank composes with the genome threshold (the entry policy)

Hybrid gate = absolute floor AND relative rank. Both must pass for a NEW long entry:

```
entry_long ? (analyst_score >= score_threshold)      # existing genome floor, unchanged
              AND (rank_pct >= rank_pct_min)          # NEW cross-sectional gate
              AND passed ?4.1 data-quality gates
              AND not thin evidence
```

Rationale for AND, not replace:

- Floor alone (today) trades the weak-tape problem described in ?1.
- Rank alone would trade "the best of a bad universe" on a day when *every* candidate is weak - a real failure mode the absolute floor exists to prevent. Keeping the floor means: on a day where the top-ranked name still can't clear an absolute conviction bar, we trade nothing. This is the correct humility prior (doctrine ?3).
- AND = "trade only names that are both objectively decent (floor) and the relative best available (rank)." This is what a disciplined swing book does and it is the tightest, safest composition.

Genome extension

Add one evolvable parameter to the genome (`strategy_versions.genome`, documented in `docs/arch/09-learning-loop.md`):

Parameter	Range	Default	Notes
<code>`entry.rank_pct_min`</code>	0.0 - 1.0	<code>**0.0**</code> (feature OFF)	Minimum within-group percentile for a NEW long. <code>`0.0`</code> = today's behavior exactly (rank never rejects). Champion promotion bounds it, e.g. <code>[0.5, 0.9]</code> .

Backward-compatibility is exact: a champion with no `rank_pct_min` (or `0.0`) reproduces current selection byte-for-byte. This makes the whole feature a no-op until a challenger carrying `rank_pct_min > 0` is validated and promoted - satisfying "no scoring change affects live until owner promotion" (scoring-methodology ?15) and the CLAUDE.md pushback mandate.

Top-3/day preservation

Rank is a filter that only removes candidates. It never lifts the screener's `<=3/day` target, PaperTrader's `limit`, position caps, or cash constraints. When `rank_pct_min > 0`, the actionable set is a subset of today's absolute-floor survivors, so daily churn can only decrease. No conflict with the top-3 rule.

Ordering note (CLAUDE.md "top 3 by analyst_score win"): within a single comparable group, rank_pct is a monotonic transform of analyst_score - so the *ordering* of the top-3 within a group is identical to ordering by raw score. Rank changes which names clear the bar (relative-to-peers), not the intra-group ranking of those that do. PaperTrader's .order("analyst_score", desc) can stay. No locked-decision conflict.

6. Rank vs the calibrated P(win) model - feature, not gate (yet)

lib/validation/calibration.ts fits a logistic P(win) over the 5 dimension scores (DIMS) and gates the pwin_logistic artifact on walk-forward ECE. Two candidate roles for rank:

- As a GATE (selection): covered in ?5 - this is the actionable use now, and it is deterministic and testable.
- As a FEATURE (calibration/sizing): cross_sectional_rank_pct is a genuinely different axis from raw dimension scores (it encodes *relative* strength, which the absolute dims cannot). It is a strong candidate to become a 6th feature in the P(win) logistic.

Recommendation: log rank as a feature now; do NOT feed it into the live P(win) model until it earns an IC track record. This mirrors exactly how analyst consensus and days_to_earnings are already handled in processSymbol (logged into decision_observations.features, graded by the learner, promoted through the IC-gated Feature Registry - never wired straight into live sizing). Concretely:

- Pass 2 writes features.cross_sectional.rank_pct, rank_quality, comparable_group_key, group_n into the observation's companion evidence (via universe_snapshot_scores, since decision_observations is append-only and already written in Pass 1).
- The learner's IC tooling (lib/edges/ic.ts) can then measure Spearman IC of rank_pct vs forward returns per market/setup/horizon.
- Only after rank clears the same OOF/IC bar the calibration ?10 rules demand does adding it to DIMS (a deterministic_v2 concern) get proposed. Not in this feature's scope.

This keeps the money path (sizing off calibrated P(win)) unchanged while rank proves itself.

7. Persistence & migrations (additive only)

Reuse migration 137 tables; add columns, do not create a parallel truth store (scoring-methodology ?15).

Migration ~145 (verify next number at build time - repo is at 144):

```
-- universe_snapshot_scores: richer rank provenance (all nullable, additive)
alter table public.universe_snapshot_scores
  add column if not exists rank_quality      text,          -- 'ok' | 'degraded' | 'excluded_thin' |
  'excluded_conf' | 'excluded_held'
  add column if not exists comparable_group_key text,        -- e.g. 'us:equity:technology'
  add column if not exists group_n          int,            -- eligible names in the group that day
  add column if not exists rank_eligible     boolean;        -- passed ?4.1 gates -> counted in the group

-- agent_signals: record why a candidate was rank-rejected (audit; PaperTrader already
-- reads status). No new features blob - canonical PIT stays in decision_observations.
alter table public.agent_signals
  add column if not exists rank_pct          numeric,
  add column if not exists rank_rejected     boolean default false;

- decision_observations gets no new columns - universe_snapshot_id (136) already links it; rank lives in the
  companion universe_snapshot_scores row (keyed by decision_observation_id). This respects the
  append-only invariant.

- New status value: agent_signals.status = 'rank_rejected' for candidates that cleared the floor but failed the
  rank gate (distinct from expired/neutral so the Research Journal can explain "scored well but wasn't the best available
  today").
```

- Range check rank_pct ? [0,1] already exists on universe_snapshot_scores; mirror as NOT_VALID on agent_signals then validate.

8. Learning-loop / validation evaluation of a rank challenger (no regime logic)

A challenger that sets rank_pct_min > 0 must be replayable by the Validation Engine on held-out folds without any regime branch.

- Reference universe reconstruction: the validation replay already reads decision_observations per PIT day. To evaluate the rank gate, it joins universe_snapshot_scores (or recomputes grouped rank_pct from the day's eligible observations using the same ?4 grouping function - the function must be shared between live Pass 2 and validation, exactly as computeWeightedAnalystScore is shared between research-agent.ts and the engine). This guarantees the challenger is graded on the identical rank rule that would run live.
- Walk-forward integrity: rank is computed within each PIT day from data available that day - it introduces no look-ahead (a day's percentile only uses that day's cross-section). Purge/embargo rules are unchanged.
- Metrics: top-K precision / top-minus-median spread (scoring-methodology ?10) are the natural fit - a rank gate should *improve* precision@K if it has edge. Champion-vs-challenger is paired on the same opportunity set.
- Eligibility gates unchanged: Sharpe >= 0.5, win rate >= 40% (docs/arch/09). No new gate needed; rank is just another genome dimension the engine already knows how to replay.
- No regime detection: the challenger contains a single scalar rank_pct_min. Its adaptivity is emergent (on a weak day fewer names clear the paired floor+rank bar), exactly the "scoring naturally adapts" principle the CLAUDE.md pushback mandate protects. There is no bull/bear state, no regime table read for the gate.

9. Explicit non-conflict checklist (CLAUDE.md locked decisions)

```
| Locked decision | How this design respects it |
| No explicit bull/bear regime switching | Rank is a within-day percentile scalar; no regime state, no mode switch. ?2, ?8. |
| Dual-bucket momentum/value screener | Untouched - rank is downstream of scoring, screener discovery unchanged. |
| <= 3 screener candidates/day | Rank only removes candidates from the actionable set; churn can only fall. ?5. |
| Top-3 by analyst_score win | Within a comparable group, `rank_pct` is monotonic in `analyst_score`; intra-group top ordering is identical. ?5. |
| Long-only for NEW positions | Gate applies only to `isHeld === false` candidates; the `isExitSignal` SELL path is not read or modified. ?4.1, ?5. |
| SELL capability for existing holdings | Holdings never enter the rank pool; their exit signals bypass the gate entirely. ?4.1. |
| No agent complexity before the weekly learner has run | Ships **OFF** (`rank_pct_min` default 0.0); becomes active only via a validated, owner-promoted challenger. ?5. |
| LLM never generates score/direction/eligibility | Pass 2 is fully deterministic, no LLM call. ?3. |
```

Only borderline item: "top 3 by analyst_score win" *could* be read as "selection must be by raw absolute score." This design keeps raw-score ordering within a comparable group (monotonic) and only changes the *cross-group admission* rule. If the owner intends the phrase to mean "never let a lower-raw-score name beat a higher one anywhere," note that with a single comparable group (the common degraded/small-universe case today) the two are identical - so the divergence only appears once the universe is large enough to have multiple valid sector groups, which is precisely when cross-sectional rank has value. Flag for explicit owner sign-off.

10. Phased build plan & effort

```
| Phase | Deliverable | Gate to advance | Effort |
| **P0 - measure-only correctness** | Replace the cron's naive mixed-pool `rank_pct` (research/cron ~L193) with the ?4 grouped, gated computation + migration ~145 columns. Rank still **not** consumed for entry. Shared `computeComparableRank()` in `lib/scoring/`. | Unit tests: grouping, small-group degraded path, exclusion reasons, monotonicity within group. Migration verified applied. | **~1.5 days** |
| **P1 - genome plumbing (still off)** | Add `entry.rank_pct_min` to genome type + `DEFAULT_GENOME` (0.0), bounds at promotion, docs/arch/09 update. No behavior change. | Byte-stable selection with default genome (regression test). | **~0.5 day** |
| **P2 - Pass 2 entry gate + shadow** | `rankAndGate()` in cron after scoring: flip `agent_signals` for rank-rejected NEW candidates; write `rank_rejected`/'status='rank_rejected'; Research Journal reason. Runs **only** when champion `rank_pct_min > 0`. Add shadow scoring so a shadow challenger's rank decisions are recorded (mirrors existing shadow_decisions). | Acceptance tests ?11; shadow evidence accrues; no live/paper change while champion default. | **~2 days** |
| **P3 - validation replay** | Shared rank function wired into Validation Engine; challenger with `rank_pct_min` replayable on held-out folds with top-K precision metric. | Paired champion/challenger replay reproduces live rank decisions; walk-forward no-leakage test. | **~2 days** |
| **P4 - learner proposes rank challengers** | Learner's `propose_challenger` may set `rank_pct_min`; IC of `rank_pct` feature reported. Owner promotes if validated. | Same Sharpe>=0.5 / win-rate>=40% gates; owner promotion. | **~1 day** |
| **P5 - rank as P(win) feature** | Out of scope here. Only after rank clears the IC/00F bar (?6). | scoring-methodology ?10 calibration gates. | - |
```

Total to actionable-but-off (P0-P2): ~4 days. To validated & promotable (P0-P4): ~7 days.

11. Acceptance tests

- Same universe snapshot + same genome -> byte-stable rank_pct, rank_quality, and entry decisions.
- ETFs never share a comparable group with single-name equities; an ETF-only group with < min sample is degraded, never a fabricated percentile.
- A comparable group with < min eligible names uses the pre-registered fixed transform and is flagged degraded - the "three finalists are not a universe" test.
- Thin-evidence, abstained, low-confidence, and held symbols are excluded from the ranking pool (row written with rank_eligible=false, rank_pct=null).
- With rank_pct_min = 0.0, selection is identical to pre-feature behavior (regression).
- With rank_pct_min > 0, the actionable set is always a subset of the absolute-floor survivors (never larger).
- Held-position SELL/exit signals are unaffected by any rank_pct_min value.
- Within one comparable group, ordering by rank_pct equals ordering by analyst_score.
- Validation replay of a rank_pct_min challenger reproduces the live Pass-2 decisions on the same PIT day (shared function).
- No regime table is read anywhere in the rank path.

12. Risks & open questions

- Small daily universe (biggest risk). Realistic per-market daily eligible counts (~6 screener + watchlist + a few holdings) are below the min-sample thresholds, so most runs use the degraded fixed-transform path - meaning true cross-sectional rank rarely engages until the PIT universe widens. *Mitigation:* the degraded path is honest (collapses to a softened absolute gate); the feature's payoff scales with universe breadth work already on the scoring-methodology roadmap. Open question: should we widen the daily screener pull (currently RESEARCH_SCREENERS_MAX=6) specifically to feed ranking? That trades against the <=3-candidate/overtrading rule - the pull can widen for *ranking universe* while the *actionable* cap stays <=3. Needs owner decision.

- Sector attribution quality. Grouping by OVERVIEW. Sector depends on that field being populated; missing sector -> falls to the marketxasset-type group. Acceptable but coarsens groups. India sector coverage is thinner still.
- Two-pass latency. Pass 2 runs inside the 150s cron maxDuration after scoring. It is a pure in-memory computation over already-fetched results + a couple of writes - negligible, but must not re-fetch anything.
- Interaction with RESEARCH_CANDIDATE_CAP. gatherSymbols already caps the candidate map at 10 before scoring; ranking operates on whatever scored, so the cap upstream still governs how many names *reach* the pool. Confirm this ordering is intended (rank ranks the post-cap survivors).
- analyst_score monotonicity assumption. The "ordering preserved within group" claim holds because rank_pct is the empirical percentile of analyst_score within the group. If a future deterministic_v2 ranks on a rankScore distinct from analyst_score, revisit the PaperTrader ordering note (?5).
- Degraded-rank live eligibility. Should a degraded rank ever be allowed to authorize an autonomous-live order? Recommend no - degraded rank is paper/shadow-only evidence; live auto keeps requiring evidence_confidence and live_approved regardless. Confirm.

13. What this does NOT do

- Does not add or read any market-regime / bull-bear detection (CLAUDE.md locked).
- Does not change the dual-bucket screener, its criteria, or its candidate count.
- Does not raise the ≤ 3 actionable candidates/day cap or any position/cash cap - it can only tighten.
- Does not remove or alter SELL-for-holdings; the exit path is untouched.
- Does not feed rank into the live P(win)/sizing model (logged as a feature only, IC-gated - ?6).
- Does not let an LLM produce rank, score, direction, or eligibility.
- Does not rewrite append-only decision_observations rows (rank lives in universe_snapshot_scores).
- Does not change any live behavior on ship - default rank_pct_min = 0.0 reproduces today exactly until a validated challenger is owner-promoted.
- Does not relabel the scorer deterministic_v2 (scoring-methodology ?1); this is an additive gate on deterministic_v1.

data availability layer - Feature Architecture

Source: features\data-availability-layer\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature Architecture - Data Availability Layer

2026-07-31 correction: the India GDELT sentiment assumptions in this historical design did not survive production evidence (0/310 usable observations). Active India scoring no longer calls or applies that dimension. The approved replacement is the shadow-only NSE-announcement + Google News RSS plan in features/pipeline-data-and-timing/FEATURE_ARCHITECTURE.md; this document must not be used to re-enable GDELT.

Status: PROPOSAL - draft, Codex-reviewed, not built. Needs owner sign-off. NOTHING here is wired into the running app yet - the shipped US fundamentals chain is still Finnhub -> FMP -> AV; SEC + US-Yahoo fundamentals are proposal-only. "Endpoint returns data" ? "integrated + correct metric".

Scope: guarantee no scoring dimension / agent flow is ever starved of input data on a max-symbol day, using only free + already-keyed sources.

Last updated: 2026-07-13

Update when built: docs/arch/02-tech-stack.md, docs/arch/03-agents.md, docs/arch/05-crons-and-scheduling.md, docs/arch/09-learning-loop.md, public/agent-diagrams/system-map.json.

One-line decision

Put a Data Availability Layer in front of ResearchAgent: a paced, cache-first refresh cron fills Supabase evidence caches ahead of scoring; ResearchAgent scores from cache snapshots and only makes bounded, last-resort live calls. Resilience for thin dimensions (fundamentals) comes from cache TTL + stale-serve + pacing, NOT from fallback breadth - because on the free tier the fallbacks mostly do not exist. (Design from the Codex review, corrected to only live-validated sources.)

Why not "just add more fallbacks inside processSymbol()"

- Vercel serverless: no shared in-memory rate state across invocations, and a cron can't spin-wait 17 min for a 5/min provider. Synchronous per-symbol fallback chains blow both the cron maxDuration and the provider limits.
- On a max day (measured: US 42 symbols, India 13), synchronous scoring would need ~84 Massive calls (5/min) and 84 Finnhub calls (60/min) in one burst.

LIVE-VALIDATED provider truth (tested 2026-07-13 - the correction to Codex)

Every entry below was hit with the real key this session. A provider may only enter a chain after it is live-validated; "documented as free" is not enough (FMP, FinancialDatasets, TwelveData, EODHD all *looked* usable and were not).

```
| Dimension | Market | Provider | Result | In chain? | |
| Fundamental | US | Finnhub | `stock/metric`+`/profile2` | ? works (PE, margin, ROE, EPS, rev-growth, sector) | **primary** |
| Fundamental | US | Yahoo | `quoteSummary` (crumb) | ? endpoint returns data (profitMargins, returnOnEquity, revenueGrowth, **forward**PE, sector; fraction-scaled). **Unofficial** - no published quota, discretionary throttling, ToS restricts automated use. | **opportunistic (forward-looking fields)** |
| Fundamental | US | SEC EDGAR | `xbrl/companyfacts` | ? endpoint returns data (Revenues, NetIncomeLoss, EPS, StockholdersEquity). Free/no-key; **fair-access ~10 req/s, no published daily quota** (NOT unlimited). Needs CIK map + price for P/E, and careful **period selection** (see Metric Equivalence). | **primary reported/accounting** |
| Fundamental | US | FMP | `stable/ratios-ttm` | ? premium-gated on free | no |
| Fundamental | US | FinancialDatasets | snapshot | ? $0 credit balance | no |
| Fundamental | US | TwelveData | `statistics` | ? premium-only (403) | no |
```

```

| Fundamental | US | EODHD `/fundamentals` | ? free = EOD prices only | no |
| Fundamental | US | Massive `/financials/v1/ratios` | ? plan NOT_AUTHORIZED | no |
| Fundamental | India | Yahoo `quoteSummary` (crumb) | ? endpoint returns data for `.NS` (RELIANCE.NS:
margin 0.076, ROE 0.091, rev-growth 0.125, sector Energy). **Unofficial, single source - resilience NOT
solved, only coverage.** Crumb flow already exists in `lib/india-data.ts`. | **monitored single-source
primary** |
| Fundamental | India | Finnhub (NSE) | ? "no access to this resource" | no |
| Fundamental | India | TwelveData / EODHD (NSE) | ? premium / symbol-not-found | no |
| Sentiment | US+India | GDELT `tonechart` | ? works for any company name, free, no daily cap |
**backbone** |
| Sentiment | US | StockTwits | ?? unreliable (frequent 403) | opportunistic first |
| Sentiment | US | AV `NEWS_SENTIMENT` | ? but 25/day HARD cap | reserve only |
| Insider | US | Massive `/filings/vx/form-4` | ? works (P/S transaction detail) | **primary** |
| Insider | US | SEC EDGAR Form 4 | ? free/official (was returning unavailable in practice - re-verify) |
secondary |
| Insider | India | NSE corporate PIT disclosures | ?? exists in `lib/nse-data.ts` - UNVALIDATED for
scoring | validate before claiming |
| Technical | US | Massive aggregates | ? works | primary |
| Technical | US | TwelveData `/time_series` | ?? on free tier (8/min, 800/day) - validate | secondary |
| Technical | US | AV daily | ? capped | reserve |
| Technical | India | Yahoo chart | ? works (`lib/india-data.ts`) | primary |
| Technical | India | Upstox candles | ?? `UPSTOX_ACCESS_TOKEN` present - validate | secondary |

```

Honest verdicts (corrected after Codex review 2026-07-13)

"Endpoint validated" (the URL returns the fields with our key/handshake) is NOT the same as "integrated + reliable." The three US sources are not interchangeable - they carry different metric bases and must be used in DISTINCT roles, not treated as equivalent fallbacks:

- SEC EDGAR = primary reported/accounting fundamentals (TTM margin, ROE, revenue growth from official XBRL). Per-field coverage varies: sampled AAPL/MSFT/NVDA/JPM covered; TSM is IFRS (different taxonomy), BRK.B lacked the standard diluted-EPS tag. So coverage is measured per field, not "universal third provider."
 - Finnhub = primary profile + computed-ratio fields (sector, TTM P/E, EPS).
 - Yahoo = opportunistic forward-looking fields (forwardPE) + the SOLE India source. Unofficial - pace + fail-soft + monitor; never a guaranteed backbone.
 - AV OVERVIEW = capped emergency reserve.
 - US fundamentals: genuine multi-source, but role-split, not "3 equal fallbacks."
 - India fundamentals: coverage exists (Yahoo) but resilience does NOT - single unofficial source. Keep the explicit single-source risk; on failure -> provider_unpriceable + renormalize (like ETFs), never faked neutral 50.
 - ETFs legitimately have no fundamentals/insider - "not applicable," never starvation.
 - Reality gap: these are PROPOSAL-only for US. Current shipped US chain is still
- Finnhub -> FMP -> AV (lib/data/fundamentals.ts); SEC + US-Yahoo are NOT wired yet, so they do not protect the running app until built.

Corrected provider chain matrix (endpoint-validated; role-split, not equal fallbacks)

```
| Dimension | Market | Ordered chain |
| Fundamental | US | **reported** SEC EDGAR (TTM) + **profile/ratios** Finnhub + **forward** Yahoo (
opportunistic) -> AV `OVERVIEW` (reserve) -> stale-cache-serve. Each field records which source+basis
served it. |
| Fundamental | India | Yahoo quoteSummary (crumb, monitored single-source) -> stale-cache-serve -> else
`provider_unpriceable` |
| Technical | US | Massive -> TwelveData `/time_series` *(if validated)* -> AV daily (reserve) |
| Technical | India | Yahoo chart -> Upstox *(if validated)* |
| Sentiment | US | StockTwits (opportunistic) -> GDELT tonechart -> AV NEWS (reserve) |
| Sentiment | India | GDELT tonechart -> NSE announcements *(if validated)* |
| Insider | US | Massive Form 4 -> SEC EDGAR -> AV (reserve) |
| Insider | India | NSE PIT *(if validated)* -> else not-enough-data |
```

(Twelve Data /time_series, Upstox, NSE PIT, Yahoo-crumb each carry a validation gate - a live probe that must pass before the source is enabled in config. This gate is the durable fix for the "looked wired, wasn't" failure mode.)

Metric equivalence + provenance (the hard part - Codex correction)

Different providers report the SAME dimension on DIFFERENT bases. Swapping a provider must never silently change what a score MEANS. So every fundamental field is stored with provenance, and derived metrics use consistent periods:

- Per-field provenance: store metric_basis (e.g. ttm | forward | annual | quarterly), period_end, source, taxonomy (us-gaap | ifrs), and fetched_at for EACH field - not one label for the whole overview.
- SEC period selection (must not take "latest" blindly): companyfacts mixes annual, quarterly, and YTD observations. The adapter must:
 - TTM margin from matching-period TTM revenue and TTM net income (sum of 4 consecutive quarters, same units), not latest annual ? latest quarterly.
 - ROE on average equity ((begin+end)/2), not the latest ending equity.
 - Revenue growth from equivalent periods (TTM vs prior TTM, or FYvsFY).
 - Handle US-GAAP and IFRS tags (TSM sampled as IFRS); fall through when the standardized tag is absent (BRK.B lacked the standard diluted-EPS tag).
 - Keep TTM P/E (SEC EPS + price) distinct from Yahoo/Finnhub forward P/E - never blend the two into one field.
 - Per-field coverage, not per-provider: SEC covers major facts for most large caps but NOT all fields for all issuers, so availability is tracked at the field level. A field with no consistent-period value is genuine_no_data, and the fundamental score renormalizes over the fields it DOES have.
- The scorer already renormalizes over available dims; extend that to available fields within the fundamental dimension so partial SEC coverage still scores.

Architecture

1. Cache-first evidence store

Per (symbol, market, dimension): evidence_cache(symbol, market, dimension, payload, source_used, fetched_at, ttl_days, stale_ok). ResearchAgent reads this, never the provider directly.

2. Per-dimension freshness TTL (the real load-killer)

On a 42-symbol day, most symbols already have fresh evidence, so few live calls fire.

- Fundamentals: 14 days (financials change quarterly).
- Insider: 1-3 days.
- Sentiment: 1 day.
- Technicals: current trading day / latest EOD.

3. Priority classes (whom to freshen first, not "trim to 3")

- Held live/paper positions - always freshen.
- Open candidates likely to cross threshold - second.
- Watchlist - third.
- Broad screener names - last; may score on stale fundamentals.

Score ALL applicable symbols; only *freshen* expensive dims for priority 1-2 daily.

4. Supabase-backed pacing (not in-memory - serverless-safe)

- `provider_limits(provider, min_interval_ms, daily_budget, burst, enabled)`
- `provider_call_ledger(provider, started_at, cache_key, status, http_status, throttle_reason)`
- `provider_refresh_jobs(provider, symbol, dimension, market, priority, status, next_attempt_at)`
- `providerCachedFetch()`: fresh-cache-hit -> return; else acquire a Supabase lease

via RPC/advisory lock enforcing `last_started_at + min_interval_ms <= now()`; no slot -> return stale cache + enqueue a refresh job. Never spin-wait in Vercel.

- Min gaps: Massive 12.5s, GDELT 5.5s, TwelveData 7.8s, AV 15s (+ hard daily reserve),

Finnhub 1.2s, SEC 250-500ms, Upstox/Yahoo/NSE 1-2s cache-first.

5. Refresh cron separate from research cron - BOUNDED + resumable

A prewarm/refresh cron drains `provider_refresh_jobs` at each provider's safe pace BEFORE the research cron runs. Research scores from warm cache; live calls become bounded exceptions. A single Vercel function cannot run ~17 min (Hobby caps ~60s; even Pro maxDuration is bounded) - so the prewarm CANNOT be one long drain. Instead: each invocation processes only as many jobs as fit in a hard wall-clock budget (e.g. <=45s), leases per-provider slots, marks jobs `next_attempt_at`, and returns - leaving the rest queued. The job is scheduled to fire repeatedly (pg_cron every few minutes) so the queue drains across MANY short invocations, never one long one. State lives in Supabase (`provider_refresh_jobs` + `provider_call_ledger`), so progress survives across invocations. Cold-start of a full 42-symbol universe simply takes several prewarm ticks, not one function call.

6. Starvation invariant + telemetry (first-class)

Invariant: *No applicable scoring dimension may be unavailable solely because a provider budget/rate/burst limit was exhausted. It may be unavailable only because every configured source returned genuine no-data, the symbol is structurally inapplicable, or no validated free source exists for that market/dimension.*

- Per symbol/dimension record: ``applicable, available, source_used,`

source_chain_attempted, unavailable_reason`.

- Reasons: `not_applicable | genuine_no_data | provider_rate_limited |
provider_daily_budget | provider_auth_missing | provider_unpriceable | stale_cache_used | chain_exhausted`.

- System Health (extends the existing low-confidence-research alert):

data-availability:<market>:<dimension> - warn < 85% availability, critical < 70%, detail names the throttled/exhausted provider.

7. Evidence provenance labels

Fix stale "Alpha Vantage" labels in the journal to show the real source_used (Finnhub / Yahoo / GDELT / Massive / SEC).

Capacity proof (max day: US 42, India 13) - after TTL + prewarm

With 14-day fundamental TTL and priority-class freshening, a steady-state day freshens only priority-1-2 fundamentals (~5-10), all sentiment (1-day TTL) via GDELT, insider (1-3d) via Massive. Worst-case COLD start (empty cache) is absorbed by the prewarm cron over its window, never the scoring cron:

- AV: target 0, <=10 reserve; never 42 -> under 25/day. ?
- Finnhub: <= (priority fundamentals x 2) paced at 1.2s -> seconds. ?
- Massive: prewarmed at 12.5s gaps -> ~17 min cold, off the scoring path. ?
- GDELT: <= (US+India sentiment) at 5.5s -> prewarm window. ?

Any chain that still can't cover 42 (e.g. India fundamentals) is reported as provider_unpriceable, not silently starved.

Build order - SHIPPED status (2026-07-13)

- ? DONE - US sentiment -> StockTwits -> GDELT tonechart -> AV reserve (commit 504cf44).
- ? DONE (adapted) - Supabase-backed pacing: provider_pacing + try_acquire_provider_slot

RPC (migration 176) in providerCachedFetch(), for the HARD-limited providers (Massive/GDELT). Serve-stale on no-slot. (Full provider_limits/ledger/refresh-jobs deferred with the prewarm cron.)

- ? PARTIAL - source chains are code-ordered (Finnhub->Yahoo->SEC->FMP->AV; Massive->EDGAR->AV; StockTwits->GDELT->AV). Not yet a config registry with per-source validation gates.

- ? DONE - data-availability:<market>:<dim> telemetry (warn<85%/crit<70% among applicable).
- ? DONE (core) - bounded+resumable prewarm cron (/api/agents/prewarm, prewarmSymbol,

pg_cron kairos-prewarm-us/-india) warms av_cache (the evidence cache) ahead of scoring, paced so it never bursts. ? Explicit priority classes (holdings-first) + a dedicated provider_refresh_jobs queue not built - the bounded tick + 14d-TTL cache-hit skip approximate it.

- ? NOT BUILT - evidence provenance labels in the journal (still may show generic sources).

Also deferred: SEC EDGAR fundamentals (spot-check failed - needs the frames API).

- ? DONE (fundamentals adapters) - SEC companyfacts adapter (lib/data/sec-fundamentals.ts,

consistent latest-annual basis, avg-equity ROE, FY-vs-FY growth, US-GAAP+IFRS, symbol->CIK map, per-field coverage) + US Yahoo quoteSummary (reuses fetchIndiaOverview, crumb cache + fail-soft) wired into the US chain. ?

Still-unvalidated ?? sources (Upstox, TwelveData /time_series, NSE PIT/announcements) remain behind a future live probe. Insider adapter (Massive Form 4) shipped earlier (commit 3d742c4). Note: "endpoint returns data" ? "correct metric" - a live spot-check of the SEC-derived margin/ROE against >=5 names is still worth doing before fully trusting the SEC tier.

Riskiest assumption

Fundamentals redundancy now rests on two unofficial sources (Yahoo quoteSummary, and Finnhub's free tier) plus one official (SEC EDGAR, US-only). The residual risks:

- Yahoo is unofficial - the crumb/cookie flow or field shapes can change without notice; it is the SOLE India fundamentals source. Mitigation: cache aggressively (14-day TTL), fail soft to provider_unpriceable, and health-alert if Yahoo's India availability drops. A second India source (e.g. Screener.in/Tickertape scrape, or NSE fundamentals) remains an open follow-up - nice-to-have, not blocking.

- EDGAR needs a maintained symbol->CIK map + a price to derive P/E; margin/ROE/growth come straight from XBRL. Low risk (official, stable) but more mapping code. The earlier "no free India fundamentals source" blocker is RESOLVED (Yahoo .NS live-validated), so this is no longer a launch blocker - just a monitoring concern.

Non-negotiables

- Free cloud only - never a paid tier/proxy/VPS. Every source free + keyed in vault/Vercel.
- Deterministic - no LLM on the scoring/fallback path.
- Forward-only - no backfill of historical signals.
- Per-market / per-currency isolation; US and India chains independent.
- A source enters a chain ONLY after a live validation probe passes.

data provider abstraction - Feature Architecture

Source: `features\data-provider-abstraction\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Data Provider Abstraction + Per-Dimension Routing - FEATURE_ARCHITECTURE (DRAFT)

Status: DRAFT - awaiting approval. No code written yet. Date: 2026-07-07 Trigger: `av_budget` hit 183 vs the 25/day free cap -> ~70% of symbols get no real technical/fundamental data every run (observed as technical=50 / fundamental=55 across GLD/IBIT/XAR). Root cause: every heavy per-symbol dimension is forced through Alpha Vantage's 25/day free tier.

All provider capabilities below were verified live (2026) against official provider docs, not assumed. Sources in `MARKET_DATA_API_INVENTORY.md` + this session's research.

1. Problem (verified)

Per US research symbol, current code spends ~4 AV calls, all through `avCachedFetch` which increments the single shared `av_budget` counter:

```
| Dimension | AV call | File |
| Technicals (candles) | `TIME_SERIES_DAILY_ADJUSTED` | `lib/research-agent.ts:608` |
| Fundamentals | `OVERVIEW` | `lib/research-agent.ts:622` |
| Insider | `INSIDER_TRANSACTIONS` | `lib/research-agent.ts:49` |
| News sentiment | `NEWS_SENTIMENT` | `lib/social-sentiment.ts:125` |
```

Plus two confirmed waste bugs:

- FD eats AV budget - `app/api/agents/research/scan/route.ts:80` routes

FinancialDatasets `FD_SNAPSHOT` through `avCachedFetch`, incrementing `av_budget`. FD calls burn AV's 25 slots.

- Scanner double-fetches - `research/scan/route.ts:53, 64, 65` calls AV RSI,

`GLOBAL_QUOTE`, EMA50 per symbol, redundant with research's own candle fetch that already computes RSI/EMA locally.

24 symbols x ~4 calls = ~96 AV calls per run; two runs/day = ~190. Cap is 25.

2. Design - provider abstraction

Replace the single AV-specific `avCachedFetch` with a generic cached fetcher keyed by provider, each with its own daily budget counter. NO trading-execution code changes.

2a. `lib/data/providers.ts` (new)

```
type ProviderId = "alpha_vantage" | "massive" | "finnhub" | "fmp"
  | "alpaca" | "upstox" | "marketaux" | "gdelt" | "yahoo"
  | "sec_edgar" | "fred" | "financialdatasets";

interface ProviderSpec {
  id: ProviderId;
  dailyBudget: number | null; // null = no daily cap (rate-limited only)
  budgetKey: string;          // av_budget-style counter row key
  keyEnv?: string;            // env var holding the key (absent = keyless)
  expiresAt?: string | null;  // ISO date the credential expires; null = never.
                                // For JWT tokens (Upstox), auto-derived by decoding
                                // the token's `exp` claim rather than hardcoding.
}
```


Credential expiry tracking (user-requested)

Most keys never expire (Finnhub/FMP/FRED/Massive = indefinite until revoked). Token-based providers do: Upstox (1yr JWT), Kite (daily), Robinhood (OAuth refresh). The registry carries an `expiresAt` per provider:

- For JWT tokens (Upstox), `expiresAt` is derived by base64-decoding the token's `exp` claim at read time - no manual date entry, stays correct if the token is rotated. (Verified: current Upstox token `exp` = 2027-07-08.)
- For static keys, `expiresAt` = `null` ("No expiry").
- A daily provider-credential-check reporter (reuses the existing `lib/system-health.ts` issue funnel) raises a `WARN` at 30 days before expiry and `CRITICAL` at 7 days / expired, auto-resolving when the credential is refreshed. Same pattern already used for `broker-token:kite`.

2b. `providerCachedFetch(provider, cacheKey, url, opts)` (generalizes `avCachedFetch`)

Same `reserve-before-spend` + `day-cache` + `stale-fallback` logic already in `av-cache.ts`, but:

- increments a per-provider counter (`provider_budget` table, keyed by `(provider, date)`), so FD no longer touches AV's budget;
- the 7-day stale-cache cap already added this session carries over;
- `avCachedFetch` becomes a thin wrapper: `providerCachedFetch("alpha_vantage", ...)`.

2c. Migration `NNN_provider_budget.sql`

- New `provider_budget(provider text, cache_date date, calls int, primary key(provider, cache_date))`.
- New `provider_budget_increment(p_provider text, p_date date)` RPC (mirrors `av_budget_increment`).
- Keep `av_budget` for backward-compat during rollout; migrate reads after.
- `av_cache` table reused as-is (already provider-agnostic by `cache_key`).

3. Per-dimension routing table (the core decision)

Primary -> fallback chain per dimension. Fallback only fires when primary is unavailable/over-budget. Availability-mask semantics (this session's fixes) still apply: a dimension with no live data is `EXCLUDED` from weighting, never scored as `fake-neutral`.

US

```
| Dimension | Primary (new) | Fallback | Was | Provider notes (verified) |
| Candles/technicals | **Massive** (`/v2/aggs`) | Alpaca -> AV | AV | Massive=Polygon rebrand, key already
configured, no daily cap |
| Quotes | **Massive** (already) | Alpaca IEX -> AV | Massive/AV | unchanged primary |
| Fundamentals | **FMP free** (250/day) | FD credits -> AV | AV `OVERVIEW` | FMP 150+ endpoints, statements/
ratios free |
| Insider | **SEC EDGAR** (free, unlimited) | AV | AV | EDGAR route already exists (`app/api/markets/edgar-
insiders`) |
| News sentiment | **Finnhub free** (60/min) | Marketaux -> GDELT | AV `NEWS_SENTIMENT` | per-ticker scores,
free |
| Analyst ratings/revisions | **Finnhub free** | FMP | (none today) | NEW dimension - recommendation
trends + price targets free |
| Earnings calendar | **Finnhub free** | AV | AV | free |
| Macro/regime | **FRED** (key added) | AV | AV | official, keyed, drafted next |
| Options (advisory) | Yahoo | - | Yahoo | unchanged |
```

India

```
| Dimension | Primary (new) | Fallback | Was |
| Candles/technicals | **Upstox** (analytics token, free, hist to 2000) | Yahoo | Yahoo |
| Quotes | **Upstox** LTP | Yahoo | Yahoo |
| Fundamentals | **Upstox** (ISIN-keyed statements/ratios) | Yahoo quoteSummary | Yahoo |
| Corporate actions | **Upstox** (splits/div/bonus) | NSE/Yahoo | (weak today) |
| Insider/PIT | NSE `corporates-pit` | - | NSE (unchanged) |
| Execution | Kite (unchanged) | - | Kite |
```

Upstox verified: Analytics token = 1-yr, read-only GET, no static IP for data, covers all four India data needs.

4. Keys required (user creates; Claude wires)

```
| Provider | Key/env | Free? | Needed for |
| Massive | `MASSIVE_API_KEY` (have) | yes | US candles/fundamentals/news |
| Finnhub | `FINNHUB_API_KEY` | yes | US news/analyst/earnings |
| FMP | `FMP_API_KEY` | yes (250/day) | US fundamentals |
| Upstox | `UPSTOX_ACCESS_TOKEN` | yes (1-yr) | India everything |
| FRED | `FRED_API_KEY` (added) | yes | US macro |
| Alpaca | `ALPACA_KEY_ID`/`ALPACA_SECRET` | yes | US fallback (optional) |
| Marketaux | `MARKETAUX_API_KEY` | yes | news fallback (optional) |
```

GDELT + SEC EDGAR = keyless.

Minimum to end the AV-limit problem: Massive (have) + Finnhub + FMP + Upstox. Alpaca/Marketaux are fallback-only, deferrable.

5. Rollout (safety-first, no trading-behavior change)

- Provider abstraction + provider_budget migration (no routing change yet).
- Split FD off AV budget (fdCachedFetch) - immediate AV relief, zero new keys.
- Kill scanner's redundant AV RSI/EMA/QUOTE calls.
- Route US candles -> Massive (key already present).
- Wire Finnhub/FMP as keys land; each dimension behind a per-provider adapter with the existing availability-mask fallback (missing -> excluded, not faked).

- Route India -> Upstox as token lands.
- FRED macro adapter (drafted, enabled last).
- Settings -> Data Providers capacity dashboard (user-requested - "always

know where the ceiling is"). Per provider, per market:

- Limit: daily cap (FMP 250) or per-minute rate (Massive 5/min, Upstox 500/min) or "no cap".

- 7-day rolling average real calls/day, from provider_budget history

(the counter logs one row per provider per day, so the average is a plain query). Also today's live count.

- Headroom / projected ceiling: limit - avg, and for capped providers a

"days/x of growth until you hit the cap" figure (e.g. "FMP: 60/250 avg - room for ~4x your current universe").

- Cache hit rate (real calls ? requested) so you see caching working.

- Credential expiry date + days-remaining countdown (green >30d/no-expiry, amber <30d, red <7d/expired), driven by registry expiresAt.

- Bottleneck flag: highlights whichever provider is closest to its ceiling

so the single limiting dimension is always visible at a glance.

5b. Redundancy - every dimension has ≥ 2 sources (never starve)

Each dimension routes primary -> fallback(s). Fallbacks fire automatically when the primary is over-budget/rate-limited/erroring. Critically, EVERY dimension already has a keyless or already-owned fallback, so agents never starve even with zero extra signups:

All keys below are ADDED + live-tested unless marked. Fallback chains (primary -> ... -> keyless) mean no dimension can fully starve.

Dimension	Primary	Fallback 1	Fallback 2	Fallback 3 (keyless)
US candles/EOD	Massive	EODHD ?	Twelve Data ?	Alpha Vantage (have)
US fundamentals	FMP ?	EODHD ?	Massive financials	Alpha Vantage
US corporate actions	EODHD ?	AV	-	-
US news	Finnhub ?	Marketaux*	-	GDELT (keyless)
US insider	SEC EDGAR (keyless)	Alpha Vantage	-	-
US analyst	Finnhub ?	FMP ?	-	-
India candles/quotes	Upstox ?	-	-	Yahoo (keyless)
India fundamentals	Upstox ?	-	-	Yahoo (keyless)
India corp actions	Upstox ?	-	-	NSE/Yahoo (keyless)
Macro	FRED ?	Alpha Vantage	-	-

* = optional, not added. GDELT/Yahoo/SEC-EDGAR keyless -> every row survives a provider outage.

LIVE-TEST CORRECTION (2026-07-08)

The prior research claim that EODHD/Twelve Data free tiers cover India NSE is FALSE - verified by hitting the APIs:

- EODHD free RELIANCE .NSE -> "Ticker Not Found" (NSE = paid tier).

- Twelve Data free RELIANCE/NSE -> HTTP 404 "available starting with the Grow plan" (NSE = paid).

So EODHD + Twelve Data add US redundancy only on their free tiers. India redundancy stays Upstox (primary, has fundamentals+corp-actions) -> Yahoo (keyless) - which is fine; Upstox is free and complete. Adding paid India coverage (EODHD ~\$60/mo or Twelve Data Grow) is deferred until/unless the free Upstox->Yahoo chain proves insufficient. US now has 4-deep candle redundancy and gains clean corporate-actions data (EODHD) that was previously weak.

Capacity at expanded 20 US + 20 India fresh/day (+ holdings ? 40 US / 30 India)

Provider	Limit	Load at 40 US/30 India	Verdict
Finnhub	60/min, no daily cap	~160 calls (~3 min)	fine
FMP	250/day	~80/day (40x2)	fine - still 3x headroom
Massive	5/min, no daily cap	~40 candle calls	fine w/ throttle
Upstox	500/min, no daily cap	30 India	fine

20/20 expansion is comfortably within free tiers. FMP (the only daily cap) only becomes the bottleneck past ~100 fresh US symbols/day - at which point Twelve Data or Massive-financials absorb the overflow, or FMP \$22/mo.

Each step is independently shippable and reversible. Steps 2-4 need no new keys and alone likely get US under the 25/day AV cap.

6. Out of scope (explicitly not this feature)

- TradingView integration (no data API - signal-webhook feature, separate track).
- Any paid tier (EODHD/Twelve Data/Massive Starter) - only if free proves short.
- Order-execution / broker changes - untouched.
- Sector-median P/E (needs a sector-median data source - separate).

7. Open questions for approval

- Approve the routing table as-is, or adjust any primary/fallback?
- Build all 8 rollout steps, or start with the no-new-key relief (steps 1-4)

and pause for keys before 5-7?

- Data Providers panel (step 8) now, or defer as its own UI task?

data source policy - Feature Architecture

Source: features\data-source-policy\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Data Source Policy + Canonical Evidence Router - Feature Architecture

2026-07-31 current-state correction: India per-symbol GDELT sentiment is retired from active scoring after zero usable production coverage. Replacement NSE/Google headline evidence is a separate shadow and is not an active sentiment.news provider policy. See features/pipeline-data-and-timing/FEATURE_ARCHITECTURE.md.

Status: DRAFT FOR CLAUDE + OWNER REVIEW. DESIGN ONLY.

Date: 2026-07-13

Owner request: stop rewriting agent code and quota logic whenever a provider or

MCP tool changes. Give the owner simple market/evidence controls while Kairos

automatically handles provider selection, freshness, pacing, fallback, and audit.

Implementation gate: no code, migration, provider activation, scoring change, or

deployment until this architecture is reviewed and Vaibhav says Proceed/Code it.

1. Decision

Build one deterministic Canonical Evidence Router between agents and external data providers. Agents request an evidence intent, never a provider or tool:

```
await evidenceRouter.resolve({
  market: "us",
  symbol: "AAPL",
  intent: "fundamentals.reported",
  asOf: decisionTs,
  runId,
});
```

Settings exposes Auto, Prefer <provider>, <Provider> only, and Off per market + evidence intent. Auto is the default and recommended mode. The router owns the provider chain, durable cache, contract validation, quota/pacing, stale policy, source health, and provenance.

Raw MCP tool names and provider URLs are never user-configurable. Adding or changing a provider requires one allowlisted adapter, not edits across agents.

2. Why this is needed now

Kairos currently has several overlapping abstractions:

- lib/data/provider-interface.ts exposes provider-first methods, but only AV/FMP are substantially implemented and its result types do not carry field-level basis, period, currency, or provenance.
- lib/data/provider-fetch.ts centralizes HTTP caching/budgets but cannot carry MCP calls, treats dailyBudget=null as both rate-only and unknown, and does not express capability/market/entitlement.
- lib/data/fundamentals.ts hardcodes a provider chain and maps everything into an Alpha Vantage-shaped string object, losing per-field source and metric basis.

- `lib/data/webull-data.ts` separately opens MCP sessions and uses `process-local` caching, bypassing the durable provider budget/pacing/cache path.

- Settings -> Data is observational only. It cannot safely alter routing.

- `lib/data/evidence.ts` has a closed source-name union that omits current sources, while `computeScores()` writes inaccurate generic source labels.

This feature consolidates those paths. It does not add a second parallel data system and does not replace the immutable evidence ledger.

Relationship to older feature documents

After owner approval, this document supersedes the provider-routing, cache-control, quota-model, and Settings-control sections of:

- `features/data-provider-abstraction/FEATURE_ARCHITECTURE.md`
- `features/data-availability-layer/FEATURE_ARCHITECTURE.md`

Those documents remain historical evidence of why the current adapters/pacing were built. Their already-shipped behavior remains the legacy fallback until this router passes rollout gates. Any provider capability claim that conflicts with this document must be re-probed rather than copied forward.

3. Confirmed defects to repair in the implementation

The following were verified against the live read-only Webull MCP contract on 2026-07-13. They are acceptance-test requirements, not optional cleanup.

P0 - Webull calls silently fail

`get_analyst_rating`, `get_analyst_target_price`, `get_stock_forecast_eps`, and `get_financial_indicators` require:

```
{ "symbol": "AAPL", "category": "US_STOCK" }
```

The current adapter sends only `{ symbol }`. Because it is fail-soft, all calls return `null` without making the failure visible to the research run.

Fix: the typed Webull adapter owns constant category: `"US_STOCK"`; callers cannot omit or override it.

P0 - Webull payload parsing is wrong

- Forecast EPS is an array of fiscal periods with fields including `est`, `actual`, and `reported`; the current parser unwraps the first object and does not recognize `est`.

- Financial indicators are `{ currency, values: { metric: [{ fiscal_year, fiscal_period, value } ] } }`; the current numeric flattener treats arrays as scalar values and returns no usable data.

- Analyst ratings use `strong_buy`, `buy`, `hold`, `under_perform`, `sell`, and `number`. The current parser ignores `under_perform`, biasing consensus upward.

Fix: use explicit schema validators and period selection. Do not use generic first-record unwrapping for financial data.

P1 - Cache is not serverless-durable

The Webull six-hour Map exists only inside one Vercel process. Cold starts and parallel instances repeat calls, while a negative result can remain hidden in a warm instance.

Fix: all provider responses use Supabase-backed `evidence_cache_v2`. A small process cache may remain only as a non-authoritative micro-cache.

P1 - Provenance is inaccurate

`computeScores()` records fundamentals/ohlc evidence as Alpha Vantage even when Finnhub, Yahoo, Massive, Upstox, or another provider served it.

Fix: the resolver returns canonical field-level provenance; scoring writes the actual evidence references and policy version.

P1 - Unknown quota is displayed as unlimited

Settings currently renders `dailyBudget=null` as `no cap (rate-limited)`. For Yahoo/Webull and other providers, no published daily limit may mean unknown, not unlimited.

Fix: model daily-limit knowledge, rate-limit knowledge, and subscription entitlement as separate fields. UI must display Unknown and use conservative pacing whenever a limit is unknown.

P2 - Diagnostic surface is broader than necessary

`POST /api/broker-mcp/[broker]/tools` exists only so a cron helper can enumerate tools. Tool discovery is now complete and the route exposes provider descriptions to anyone holding `CRON_SECRET`.

Fix: after contract fixtures are captured, remove the POST alias. Keep an owner-only GET diagnostics route only if Settings needs it; otherwise remove the route. It must never call tools or return credential/token data.

4. Scope

In scope

- US and India evidence routing.
- Fundamentals, analyst consensus, daily bars, quotes, sentiment, insider, earnings/calendar, corporate actions, and macro evidence intents.
- Code-owned provider capabilities and typed adapters.
- Versioned owner policy with `Auto/preferred/only/off` modes.
- Durable cache, pacing, refresh queue, call ledger, health, and diagnostics.
- Settings product controls and provider capacity visibility.
- Compatibility adapter for existing AV-shaped scoring inputs during migration.
- Webull research adapter fixes and shadow validation.

Explicitly out of scope

- Any broker/order routing or order scopes.
- Enabling Webull orders or changing its `orderCapable=false` gate.

- New signal weights or making Webull analyst consensus a scored dimension.
- Letting an LLM select providers, quotas, fallbacks, or data values.
- Arbitrary provider URLs/tool names entered through Settings.
- Python sidecar/GitHub Actions worker in this phase.
- Paid provider upgrades.

5. Non-negotiable invariants

- Provider routing is deterministic; no LLM participates.
- US and India resolve independently. No cross-market or cross-currency merge.
- Every scored field records provider, basis, period, currency, observation time, retrieval time, and quality state.
- unknown quota never means unlimited.
- A missing provider/config/cache read fails unavailable, never fabricated neutral.
- Structurally inapplicable evidence remains distinct from unavailable evidence.
- A provider switch must not silently change metric meaning (for example forward P/E cannot replace TTM P/E under the same canonical field).
- Policy changes apply only to future research runs. Each run freezes one active policy version per market.
- A provider outage cannot touch cash, positions, proposals, or broker orders.
- Database configuration can select only providers/capabilities compiled into the allowlisted code registry.
- Raw provider prose is never inserted directly into an LLM prompt. Only schema-validated, bounded canonical fields may be rendered.
- Existing scoring availability-mask + renormalization semantics remain intact.
- A policy cannot become production-active if its only material effect is removing scored evidence in a way that newly creates entry eligibility. On ly/Off for score-affecting intents remain diagnostic/shadow unless separately approved as a scoring-policy change.
- Router price intents are research/chart evidence only. Execution sizing, fills, stops, broker reconciliation, and order previews retain their existing verified money-path quote sources until separately architected.

6. Product behavior

Default owner experience

Settings -> Data gains two views:

- Routing Policy - the primary view.
- Provider Health - capacity, credentials, freshness, errors, and diagnostics.

Routing Policy is a market-tabbed matrix:

Evidence	Mode	Preferred	Effective chain	Freshness	Status
Fundamentals	Auto	Recommended	Finnhub -> Webull -> Yahoo -> AV	14d	Healthy
Analyst consensus	Auto	Webull	Webull -> Finnhub	3d	Healthy
Daily prices	Auto	Massive	Massive -> EODHD -> AV	EOD	Healthy
Sentiment	Auto	GDELT	StockTwits -> GDELT -> AV reserve	1d	Degraded

The provider dropdown lists only validated providers supporting that market and intent. A provider must be `production_eligible` for score-affecting intents; contract validity alone is insufficient. Unhealthy, disconnected, unentitled, or contract-invalid providers are disabled with a short reason.

Modes

- Auto - use the code-owned recommended chain, health, fresh cache, and capacity.
- Prefer - place the selected provider first, then use the safe Auto fallbacks.
- Only - use only the selected provider/cache; unavailable means unavailable.

This is an advanced diagnostic option and displays a starvation warning. For a scored intent it is shadow-only.

- Off - intent is intentionally unavailable. For scored intents this is

shadow-only because removing a dimension can change renormalized scores.

Owner should not manage quotas routinely

Provider limits and pacing defaults live in the code registry and are updated once per provider adapter. Settings shows them but does not require repeated adjustment. Advanced controls permit conservative overrides only:

- lower daily cap;
- larger minimum interval;
- larger reserved-call buffer;
- disable provider;
- shorten stale allowance.

The UI cannot raise a limit above the code-known ceiling unless the provider's entitlement/tier is also explicitly changed and contract-probed.

Change and rollback UX

- Save creates an immutable policy version; it does not mutate the active pointer.
- A confirmation summarizes changed rows and says "Applies next research run."
- Activate atomically updates the active-policy pointer for one market.
- Restore Auto creates/activates a new all-Auto version.
- Recent versions show who changed them, when, and a compact diff.
- Disabling the router feature flag immediately returns that market to the legacy

chain without deleting policies or cache.

7. C4 system context

```
C4Context
title System Context - Kairos Data Source Policy

Person(owner, "Vaibhav", "Chooses evidence-source policy")
System(kairos, "Kairos FinanceOS", "Research, scoring, paper and live workflows")
System_Ext(usProviders, "US Data Providers", "Webull, Finnhub, SEC, Massive, Yahoo, AV")
System_Ext(indiaProviders, "India Data Providers", "Kite, Upstox, Yahoo, NSE, GDELT")
SystemDb_Ext(supabase, "Supabase", "Policy, cache, queue, health, evidence ledger")

Rel(owner, kairos, "Reviews health and activates policy", "HTTPS")
Rel(kairos, usProviders, "Fetches allowlisted US evidence", "HTTPS/MCP")
Rel(kairos, indiaProviders, "Fetches allowlisted India evidence", "HTTPS")
Rel(kairos, supabase, "Freezes policy and persists evidence state", "PostgREST/RPC")
```

8. C4 containers

```
C4Container
title Container Diagram - Evidence Routing

Person(owner, "Vaibhav", "Owner")
System_Ext(providers, "External Providers", "Structured evidence APIs and MCP")

System_Boundary(kairos, "Kairos") {
  Container(settings, "Settings Data UI", "Next.js Client", "Policy and health controls")
  Container(api, "Policy and Diagnostics API", "Next.js Route Handlers", "Owner-gated reads and version activation")
  Container(research, "Research and Prewarm", "Next.js Agents", "Requests canonical evidence intents")
  Container(router, "Canonical Evidence Router", "TypeScript", "Deterministic cache, policy and fallback resolution")
  Container(adapters, "Provider Adapters", "TypeScript", "Allowlisted typed provider contracts")
  ContainerDb(db, "Evidence Control Store", "Supabase Postgres", "Versions, cache, leases, queue, health and ledgers")
}

Rel(owner, settings, "Selects Auto/preferred/off", "Browser")
Rel(settings, api, "Reads health and proposes versions", "JSON/HTTPS")
Rel(api, db, "Creates and activates policy versions", "PostgREST/RPC")
Rel(research, router, "Resolves evidence intent", "TypeScript")
Rel(router, db, "Reads frozen policy/cache and records attempts", "PostgREST/RPC")
Rel(router, adapters, "Calls validated capability", "Typed interface")
Rel(adapters, providers, "Fetches data", "HTTPS/MCP")
Rel(adapters, db, "Stores canonical cache result", "PostgREST")
```

9. Router components and dynamic flow

```
C4Component
  title Component Diagram - Canonical Evidence Router

  Container(research, "Research Agent", "Next.js", "Scoring workflow")
  ContainerDb(db, "Supabase", "Postgres", "Durable control state")
  Container_Ext(provider, "External Provider", "HTTP/MCP", "Evidence source")

  Container_Boundary(router, "Evidence Router") {
    Component(intentCatalog, "Intent Catalog", "TypeScript", "Canonical schemas and applicability")
    Component(policyResolver, "Policy Resolver", "TypeScript", "Loads frozen version and provider order")
    Component(cacheResolver, "Cache Resolver", "TypeScript", "Fresh/stale provider snapshots")
    Component(capacityGate, "Capacity Gate", "Supabase RPC", "Atomic budget and pacing lease")
    Component(adapterRegistry, "Adapter Registry", "TypeScript", "Allowlisted provider capabilities")
    Component validator, "Schema Validator", "TypeScript", "Canonicalizes and rejects bad payloads")
    Component(auditWriter, "Audit Writer", "TypeScript", "Provenance, attempts and health")
  }

  Rel(research, intentCatalog, "Requests intent")
  Rel(intentCatalog, policyResolver, "Supplies capability requirements")
  Rel(policyResolver, cacheResolver, "Checks ordered candidates")
  Rel(cacheResolver, capacityGate, "Requests live-call lease on cache miss")
  Rel(capacityGate, adapterRegistry, "Permits bounded call")
  Rel(adapterRegistry, provider, "Calls allowlisted operation", "HTTPS/MCP")
  Rel(adapterRegistry, validator, "Returns provider payload")
  Rel(validator, auditWriter, "Records accepted/rejected evidence")
  Rel(auditWriter, db, "Persists state")
  Rel(cacheResolver, db, "Reads cache")
  Rel(policyResolver, db, "Reads policy version")
```

Resolution sequence

- Validate market/symbol/intent and determine structural applicability.
- Load the policy version frozen at research-run start.
- Build provider candidates from the code registry and policy mode.
- Remove providers that are disabled, contract-invalid, unauthorized for the market/intent, circuit-open, or known to lack entitlement.
- Read fresh canonical cache in provider order.
- If no fresh hit, atomically reserve budget and pacing for at most two synchronous provider attempts within the caller's wall-clock budget.
- Validate provider payload and canonical metric semantics.
- Persist canonical cache + append-only evidence/attempt records before returning evidence that can influence scoring.
- If live fetch cannot proceed, return policy-allowed stale evidence and enqueue a refresh job. Otherwise return typed unavailable status.
- Never spin-wait for a provider slot inside Vercel.

10. Canonical intent catalog

Initial intent IDs are code constants, not arbitrary DB strings:

```
| Intent | Markets | Canonical result | Default fresh TTL | Stale ceiling |
| `price.quote` | US/India | price, currency, market time | 2 min/session | 1 day, display only |
| `price.daily_bars` | US/India | adjusted OHLCV series | current EOD | 3 trading days |
| `fundamentals.reported` | US/India | margin, ROE, EPS, revenue growth | 14 days | 45 days |
| `fundamentals.valuation` | US/India | TTM/forward P/E with basis | 3 days | 14 days |
```

```
| `analyst.consensus` | US initially | rating counts, target range, forecast EPS | 3 days | 14 days |
| `sentiment.news` | US/India | aggregate tone + sample count | 1 day | 3 days |
| `insider.transactions` | US/India if supported | normalized filings | 1 day | 7 days |
| `events.earnings` | US/India | event time and estimates | 1 day | no stale past event |
| `events.corporate_actions` | US/India | dividend/split/action | 1 day | no stale past ex-date |
| `macro.regime_inputs` | market-scoped | point-in-time macro values | provider cadence | one cadence |
```

TTL values are policy defaults, not proof that the underlying observation is current. `observedAt/periodEnd` remain explicit.

11. Canonical contracts

```
type Market = "us" | "india";
type EvidenceIntent =
  | "price.quote"
  | "price.daily_bars"
  | "fundamentals.reported"
  | "fundamentals.valuation"
  | "analyst.consensus"
  | "sentiment.news"
  | "insider.transactions"
  | "events.earnings"
  | "events.corporate_actions"
  | "macro.regime_inputs";

type EvidenceQuality =
  | "fresh"
  | "stale"
  | "partial"
  | "conflict"
  | "quarantined"
  | "unavailable"
  | "not_applicable";

interface FieldProvenance {
  providerId: ProviderId;
  providerField: string;
  basis: "ttm" | "forward" | "annual" | "quarterly" | "spot" | "eod";
  periodEnd?: string;
  observedAt?: string;
  retrievedAt: string;
  currency?: "USD" | "INR";
  unit: "fraction" | "currency" | "per_share" | "count" | "ratio" | "text";
}

interface CanonicalField<T> {
  value: T;
  provenance: FieldProvenance;
}

interface EvidenceEnvelope<T> {
  schemaVersion: "evidence-v1";
  market: Market;
  symbol?: string;
  intent: EvidenceIntent;
  quality: EvidenceQuality;
  payload: T | null;
  provenance: FieldProvenance[];
  providersAttempted: ProviderId[];
  policyVersionId: string;
  cacheState: "fresh" | "stale" | "miss";
  unavailableReason?: UnavailableReason;
  resolvedAt: string;
}
```

`UnavailableReason` is an enum including `auth_missing`, `entitlement_missing`, `contract_invalid`, `rate_limited`, `daily_budget`, `timeout`, `provider_error`, `schema_invalid`, `genuine_no_data`, `chain_exhausted`, `disabled_by_policy`, and `not_applicable`.

12. Provider adapter contract

```
interface ProviderAdapter<I extends EvidenceIntent> {
    readonly providerId: ProviderId;
    readonly intent: I;
    readonly contractVersion: string;
    fetch(request: ProviderRequest<I>, ctx: ProviderCallContext): Promise<ProviderResult<I>>;
    validate(raw: unknown): ProviderResult<I>;
    toCanonical(result: ProviderResult<I>): CanonicalPayloadByIntent[I];
}
```

Provider specs are code-owned:

```
interface ProviderSpec {
    id: ProviderId;
    label: string;
    transport: "http" | "mcp";
    markets: readonly Market[];
    capabilities: readonly EvidenceIntent[];
    dailyLimitState: "known" | "none" | "unknown";
    dailyLimit?: number;
    rateLimitState: "known" | "none" | "unknown";
    rateLimitCalls?: number;
    rateLimitWindowSeconds?: number;
    minIntervalMs: number;
    reserveCalls: number;
    entitlementRequired: boolean;
    credentialRef?: string;
    trustTier: 1 | 2 | 3 | 4 | 5;
    official: boolean;
}
```

DB overrides can only make these limits more conservative unless a code change updates the provider entitlement/ceiling.

13. Webull adapter specification

Allowed research tools

The research adapter allowlist contains only:

- get_analyst_rating
- get_analyst_target_price
- get_stock_forecast_eps
- get_financial_indicators
- later, after fixtures: get_income_statement, get_balance_sheet, get_cash_flow, get_company_profile, get_financial_alert, get_stock_industry_comparison

No tool name is read from DB or request input. No order tool is imported.

Analyst normalization

- Send { symbol, category: "US_STOCK" }.
- Validate symbol/category echoed by Webull.
- Normalize string numeric fields with finite/range checks.
- Consensus weights: strong_buy=100, buy=80, hold=50, under_perform=20, sell=0.
- Compare the sum of rating buckets with number; if mismatch exceeds one, mark

partial/conflict and preserve raw counts without producing a normalized score.

- Target price preserves mean/median/high/low/currency/effective date.
- Forecast EPS selects the nearest future reported=false fiscal period. It never treats historical actual EPS as forecast EPS.

Fundamentals normalization

- Request explicit type (ANNUAL initially) and bounded count (2-5).
- Parse each metric array by fiscal year/period; never flatten arrays generically.
- net_margin and roe are stored as fractions, not percentages.
- Preserve provider currency and period.
- Do not infer TTM from annual data. Add quarterly/TTM derivation only under a separate tested adapter version.
- Map only explicitly validated fields. Unknown numeric leaves remain diagnostics, not canonical scoring inputs.

Caching and failure behavior

- Analyst TTL 3 days; financials TTL 14 days.
- Negative cache only deterministic genuine_no_data for 6 hours.
- Auth, entitlement, timeout, schema, and provider errors are not long negative cached; they update health/circuit state and may use stale canonical cache.
- One MCP session may be reused, but calls are sequential until Webull documents or tests prove concurrent calls on one session are safe.
- Token and provider error redaction remains mandatory.

Scoring posture

Webull analyst evidence remains narrative/measurement-only. It cannot change the deterministic analyst score until it has sufficient point-in-time outcomes and an approved scoring architecture. Webull reported fundamentals may become a fallback only after shadow comparison proves field semantics against existing sources.

14. Database design

Before implementation, list live migrations/tables/functions. Migration 176's provider_pacing/try_acquire_provider_slot exists in production but is not tracked in this repository; the implementation must first add an idempotent repair migration matching the live schema. Never blindly recreate or rename it.

evidence_policy_versions - immutable

Column	Type	Rule
`id`	uuid PK	generated
`market`	text	checked `us`/`india`
`version`	int	unique per market
`router_enabled`	bool	false during rollout
`created_by`	uuid	owner
`created_at`	timestampz	audit
`change_note`	text	bounded owner note

Rows never change after insertion. UPDATE/DELETE is trigger-blocked.

active_evidence_policy - mutable pointer

Column	Type	Rule
`market`	text PK	`us`/`india`
`policy_version_id`	uuid FK	same-market immutable version
`activated_by`	uuid	owner
`activated_at`	timestampz	audit

The activation RPC updates this pointer under a per-market advisory lock. A trigger or RPC validation prevents a US pointer from referencing an India policy. Policy history remains immutable and rollback means pointing to a prior version or creating a new all-Auto version. Activation also validates that the target version has exactly one valid rule for every required intent before changing the pointer.

evidence_policy_rules - immutable children

Column	Type	Rule
`policy_version_id`	uuid FK	cascade prohibited
`intent`	text	code-validated enum
`mode`	text	`auto`/`prefer`/`only`/`off`
`preferred_provider`	text nullable	required for prefer/only
`max_age_seconds`	int	conservative bounded range
`stale_max_seconds`	int	>= max age, bounded
`max_sync_attempts`	int	0-2
`advanced_config`	jsonb	future compatible, size bounded

Primary key (policy_version_id, intent).

provider_runtime_config - current operational controls

Column	Type	Rule
`provider_id`	text PK	must exist in code registry at runtime
`enabled`	bool	false blocks live calls, cache may remain visible
`daily_limit_state`	text	`known`/`none`/`unknown`
`daily_limit_override`	int nullable	cannot exceed code ceiling
`rate_limit_state`	text	`known`/`none`/`unknown`
`rate_limit_calls_override`	int nullable	cannot exceed code ceiling
`rate_window_seconds_override`	int nullable	cannot shrink code window
`min_interval_ms_override`	int nullable	cannot be lower than code floor
`reserve_calls_override`	int nullable	>= code minimum

Owner-read; writes only through owner-gated API using the service role after code validation. No direct authenticated writes.

provider_capability_status - provider/intent maturity

Column	Type	Rule
`provider_id`/`market`/`intent`	text	composite PK
`contract_state`	text	`unverified`/`valid`/`invalid`
`maturity_state`	text	`discovered`/`contract_valid`/`shadow_validated`/`production_eligible`
`entitlement_state`	text	`unknown`/`active`/`inactive`
`contract_version`	text	must match code adapter
`last_probe_at`	timestampz	operational evidence
`last_shadow_evaluation_id`	uuid nullable	proof link

Maturity is capability-specific: Webull analyst can be eligible while Webull fundamentals remains shadow. The owner API may disable a capability, but promotion to `production_eligible` requires a code-known adapter version plus a passing shadow evaluation. A DB row can never create a capability absent from the code registry.

evidence_policy_evaluations - append-only shadow proof

Stores candidate policy version, baseline production version, market, symbol/run sample, coverage delta, schema failures, provider disagreement, score delta, entry-eligibility flips, call usage, and pass/fail reasons. Production activation of a score-affecting policy requires a passing evaluation and explicit owner action.

evidence_cache_v2 - mutable current cache

```
| Column | Type | Rule |
| `market`/`symbol`/`intent`/`provider_id` | text | composite identity |
| `request_fingerprint` | text | contract + normalized request hash |
| `schema_version` | text | canonical schema |
| `payload` | jsonb | canonical only, size bounded |
| `provenance` | jsonb | canonical field/source sidecar; array only |
| `quality_state` | text | fresh/partial/conflict/quarantined |
| `observed_at`/`period_end` | timestampz/date | semantic time |
| `fetched_at`/`expires_at`/`stale_until` | timestampz | cache policy |
| `currency`/`basis` | text | checked canonical values |
| `payload_hash` | text | integrity/dedup |
```

Primary key (market, symbol, intent, provider_id, request_fingerprint). Market-wide intent uses a reserved symbol such as `__MARKET__`, never null ambiguity.

provider_call_ledger - append-only operational audit

Records provider, intent, market, symbol hash/plain symbol, run ID, policy version, cache outcome, lease outcome, started/completed timestamps, latency, HTTP/MCP status, normalized error code, response bytes, and contract version. It stores no tokens, URLs containing keys, headers, or raw error bodies.

provider_refresh_jobs - durable queue

One active job per (market, symbol, intent, provider_id, request_fingerprint). Claiming uses a service-role-only `claim_provider_refresh_jobs()` RPC with `FOR UPDATE SKIP LOCKED`, lease expiry, bounded batch size, attempt count, and `next_attempt_at`. Jobs dead-letter after bounded retries and raise System Health issues.

Existing tables retained

- `evidence_records` remains the immutable decision evidence ledger.
- `provider_budget` may remain as the daily rollup but becomes derived from or atomically updated alongside `provider_call_ledger`.
- `av_cache` remains during compatibility rollout; it is not the router cache.
- `provider_pacing` remains the atomic cross-process lease table.

RLS and grants

- Enable RLS on every new table.
- Owner-email SELECT policy for policy/health/cache summaries where UI needs it.
- No anon access.
- No direct authenticated INSERT/UPDATE/DELETE.

- Service role performs agent writes.
- SECURITY DEFINER RPCs set `search_path`, validate all enums/ranges, revoke PUBLIC, and grant only `service_role`.
- Append-only triggers block UPDATE/DELETE on policy versions, rules, evaluations, and call ledger; only the active-policy pointer is mutable through its service-role RPC.

15. Policy APIs

GET /api/settings/data-routing?market=us

Owner-only. Returns active version, rules, resolved effective chains, provider availability, latest unactivated version, and change history. It never returns credential values.

POST /api/settings/data-routing/versions

Owner-only. Validates the complete matrix server-side and creates a new immutable version without activating it. A service-role RPC allocates the next market version under an advisory lock and inserts the header + complete rule set in one transaction. It rejects unsupported market/intent/provider combinations.

POST /api/settings/data-routing/activate

Owner-only. Calls atomic activation RPC. Activation never starts a research run and never calls a provider. It affects only future runs. For a score-affecting change, the RPC requires a passing `evidence_policy_evaluations` row against the currently active baseline and explicit owner confirmation. Non-scored display/narrative policy may activate after contract validation without the score-delta gate.

POST /api/settings/data-routing/restore-auto

Owner-only. Creates and activates a new default version; does not mutate history.

GET /api/settings/data-providers

Upgrade the existing endpoint to report daily-limit state, rate-limit state, known limits, entitlement, contract status, cache hit rate, last success, consecutive failures, circuit state, and source freshness. Remove the claim that every null daily budget is uncapped.

16. Capacity, pacing, and health

Atomic call admission

One RPC combines daily-budget reservation and minimum-interval lease. Counting a call before external spend is conservative. The ledger distinguishes reserved, started, and completed so unused reservations can be measured but never reused in a race-prone way.

Circuit breaker

Per provider + intent:

- open after 3 consecutive contract/auth/schema failures;

- transient timeout/5xx opens only after a higher threshold;
- cooldown is bounded and visible;
- one half-open probe is leased atomically;
- genuine no-data for one symbol does not degrade global provider health.

Health states

healthy | degraded | exhausted | disconnected | unentitled | contract_invalid | unknown.
Health affects Auto routing but never alters a frozen canonical payload.

Telemetry

- cache hit/stale/live/miss rates;
- success and schema-rejection rates by provider/intent;
- p50/p95 latency;
- fresh coverage by market/intent;
- calls and reserved headroom;
- fallback frequency and provider disagreement;
- refresh queue depth/oldest age/dead letters.

Existing data-availability:<market>:<dimension> alerts remain. Add provider-specific issues only when actionable; avoid one issue per symbol.

17. Fundamental merging and conflict rules

Fundamentals are not a first-provider-wins blob. Canonical fields resolve separately:

- provider must support the required basis;
- freshest valid field in policy order wins;
- no blending of annual/quarterly/TTM/forward values;
- source-specific fields may coexist in one envelope with field provenance;
- material same-basis disagreement marks conflict and excludes that field until

a trusted-tier rule resolves it or the conflict clears;

- a partial envelope is available only if it satisfies the intent's minimum field

contract; otherwise it is unavailable and scoring renormalizes.

Compatibility phase maps canonical fields into the legacy AV-shaped object only at the scorer boundary. The mapping includes a provenance sidecar and is deleted after the scorer consumes canonical fields directly.

18. Security and abuse analysis

- SSRF/tool injection: provider URLs and MCP tool names exist only in code.
- Prompt injection: discard unstructured provider prose; render bounded canonical values through templates. Filing/news text remains in its separate sanitized path.
- Prototype/property abuse: validators create new plain canonical objects and use

own-property checks; no arbitrary dotted-path traversal over provider objects.

- Credential leakage: service role and OAuth tokens remain server-side; logs store normalized error codes, never raw authorization/URL/header/error payloads.
- Payload bombs: response byte limit, JSON depth/array count bounds, timeout, and per-tool result schemas.
- Cross-market contamination: cache and policy keys include market; adapters check provider market/category and canonical currency.
- Configuration escalation: Settings cannot add order capability, tool names, scopes, URLs, or raise unknown quotas to unlimited.
- Money-path isolation: router modules are imported by research/prewarm/read APIs, not broker adapters or `execute-order.ts`.

19. Rollout and reversibility

Phase 0 - schema reality and fixtures

- Verify live tables/functions/migrations, especially out-of-band migration 176.
- Capture sanitized contract fixtures for the Webull read tools and representative US symbols: AAPL, MSFT, JPM, BRK.B, one ETF, one no-data symbol.
- Add tests before changing the adapter.

Phase 1 - repair Webull without scoring impact

- Add required `US_STOCK` category and explicit parsers.
- Persist canonical Webull analyst/fundamental cache.
- Keep analyst narrative/measurement-only.
- Keep Webull fundamentals shadow-only; compare with Finnhub/Yahoo for at least five trading days and report coverage/disagreement.
- Remove or narrow the diagnostic tools route.

Phase 2 - canonical core behind flags

- Add canonical contracts, intent catalog, adapter registry, durable cache, call ledger, pacing repair migration, and refresh queue.
- Seed one all-Auto policy per market with `router_enabled=false`.
- Dual-resolve legacy and canonical routes; canonical output is logged only.

Phase 3 - Settings policy control

- Add owner APIs and Routing Policy/Provider Health UI.
- Activate policy versioning while router remains shadow-only.

- Browser-test desktop/mobile, disabled options, version diff, restore Auto, and unknown-quota labels.

Phase 4 - controlled research cutover

- Enable router for paper research in US only.
- Compare availability, canonical scores, provider calls, and decision deltas.
- Enable India independently after its own evidence.
- No live-autonomy change; signals continue through all existing gates.

Phase 5 - cleanup

- Move remaining direct provider calls behind adapters.
- Replace inaccurate evidence source labels.
- Retire provider-first abstraction and av_cache only after no callers remain.
- Update architecture/database/cron docs and system map.

Disable/rollback

Set the per-market router feature flag false. The legacy chain resumes next run; policy versions, ledgers, and cache remain for audit. No migration rollback or data deletion is required.

20. Required tests

Unit/contract

- Every adapter fixture validates and canonicalizes expected fields.
- Webull missing category fails test; adapter always supplies US_STOCK.
- EPS chooses nearest future unreported est row.
- Analyst denominator includes under_perform; count mismatch becomes conflict.
- Financial period arrays preserve basis/period/currency and fraction scale.
- Unknown quota is not treated/displayed as unlimited.
- Unsupported provider/intent/market combinations are rejected.
- Forward and TTM fields never substitute for one another.
- Raw untrusted keys/prose cannot enter canonical objects/prompts.

Router

- Frozen policy is stable when owner activates a new version mid-run.
- Auto/prefer/only/off produce deterministic candidate orders.
- Only/off on scored intents cannot production-activate without separate approved scoring-policy treatment.
- Fresh cache avoids provider calls.

- No lease returns stale/enqueues; never spin-waits.
- Max two synchronous attempts and hard wall-clock bound.
- Market/currency keys cannot cross-hit cache.
- Contract-invalid provider is skipped in Auto.
- Provider-only mode returns unavailable rather than hidden fallback.
- Failure to write trade-affecting evidence blocks that evidence from scoring.

Database/security

- One active-policy pointer per market under concurrent activation.
- Score-affecting activation fails without a passing evaluation against the current baseline, and a stale evaluation cannot authorize a newer version.
- Append-only tables reject update/delete.
- Queue claims cannot double-lease.
- anon/authenticated writes are denied; owner reads work.
- SECURITY DEFINER functions have fixed search path and service-role-only execute.
- No secrets/raw token-bearing errors are stored.

Integration/product

- Research run stamps policy version and exact field provenance.
- Shadow evaluation reports entry-eligibility flips, not only average score delta.
- Availability mask and weight renormalization match legacy semantics.
- Webull outage does not change India and cannot reach any order path.
- Settings dropdown contains only validated market-capable providers.
- Restore Auto creates a new auditable version.
- Browser verifies responsive table/control layout and no text overlap.

Release gates

- `npx tsc --noEmit`
- full unit/integration test suite
- `npm run build`
- dependency/security audit
- schema verification against live Supabase after migration
- read-only production contract probes
- Playwright Settings checks at desktop and mobile widths
- explicit proof: Webull `orderCapable=false`, no order scope, no order calls

21. Product/trader acceptance criteria

The feature is successful when:

- Vaibhav can leave every row on Auto and never rebalance quotas manually.
- Switching a provider needs one policy activation, not agent code edits.
- Adding a tool/provider requires one adapter + registry entry + fixtures.
- Every research decision explains which sources and periods supplied its fields.
- Provider degradation is visible before it silently starves a scoring dimension.
- Provider changes do not produce phantom score jumps from unit/basis drift.
- US and India can be enabled, disabled, and rolled back independently.
- No data-source control can place or authorize a trade.

22. Recommended initial defaults

Keep these as reviewed defaults, not permanent truths:

- US reported fundamentals: Auto; FintHub first while Webull shadows. Promote

Webull fallback only after comparison passes. SEC remains excluded until its period/concept derivation is corrected.

- US analyst consensus: Webull primary, narrative/measurement-only.
- US bars/quotes: existing Massive-first chain.
- India fundamentals: Yahoo monitored single-source with durable stale cache;

do not pretend resilience is solved.

- India bars/quotes: existing Upstox/Kite/Yahoo roles, based on entitlement.
- Sentiment/insider/macro: preserve current chains during router shadow phase.
- All policy modes: Auto.
- Router flags: off until dual-run evidence passes separately per market.

23. Riskiest assumptions to validate first

- Webull financial ratios remain available under the owner's authorized capability and subscription during unattended cron runs, not just interactive probes.

- Provider values that share a label also share the intended period/basis/unit.
- The legacy scoring path can be compatibility-mapped without changing historical score semantics before the canonical scorer migration.

The first implementation proof should therefore be a five-day, no-score-impact Webull contract/coverage comparison with durable fixtures and provenance.

24. Review questions for Claude

- Is immutable per-market policy versioning sufficient for reproducible research, or should each run copy the complete rule JSON into its run ledger as well?
- Should provider call admission extend the existing provider_pacing RPC or use

a new combined budget+pacing RPC after live schema verification?

- Are the canonical intent boundaries narrow enough to prevent metric-basis drift?

- Is two synchronous provider attempts the correct Vercel bound, with remaining work delegated to the refresh queue?

- Should Webull fundamentals remain shadow for five trading days or require a larger symbol/sector sample before becoming a fallback?

- Does any existing agent still consume direct provider payloads in a way this migration order misses?

decision review - Feature Architecture

Source: features\decision-review\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Decision Review / Counterfactual - Feature Architecture

Status: IMPLEMENTED (Phase A - measure-only, read-only). No migration, no writes.

Last updated: 2026-07-15

Build note (2026-07-15): Shipped as a read-only tab in Research Journal.

Files: app/api/decision-review/route.ts (owner-gated SELECT-only join of decision_observations x observation_labels x v_decision_quality) and components/dashboard/DecisionReviewPanel.tsx (4th tab: Daily Funnel | Evolution | Score Tracker | Decision Review, /dashboard/research-journal?tab=review).

DB-reality deviations from the draft below:

- No v_decision_review view created - used the API-only deterministic join

(?9 open decision 3, "API-only path avoids a migration entirely"). Read-only, no migration, honors the hard "no migration" boundary.

- Horizons surfaced are data-driven, not fixed at 2/5/20. As of build, only horizon_days 2 and 5 have matured in observation_labels; 10/20 back-fill as they mature. The API returns the canonical horizons that actually have labels; the "1d" question (?4) is moot - 1d is not in the pipeline and none was added.

- 20-floor gate is live (MATURED_FLOOR = 20): a cohort with fewer matured outcomes at a horizon renders "N/20 matured - insufficient sample, not yet actionable"; at/above the floor it shows hit-rate + avg fwd + avg alpha.

- Per-symbol attribution is decorative, always tagged "n=1 ? ILLUSTRATIVE" (locked owner decision). No LLM prose pass was wired (kept fully deterministic).

Update this file when: the per-symbol decision-review data contract, the attribution method, or the UI surface changes; keep it aligned with docs/arch/09-learning-loop.md (Performance Truth / P1 gate) and

1. One-line intent

For a single symbol we scored on date D, show: what we scored (analyst_score

- the 5 dimension scores), how the stock actually moved over the next

1d / 5d / 20d (market-local adjusted close), the gap (score-implied direction vs realized), which dimension most plausibly pushed the score wrong, how confident that read is, and a plain-English "what we'd learn."

This is the missing per-symbol post-mortem. We already measure *aggregate* predictive power (EdgeIC on the broad universe; v_decision_quality per-decision data quality). Decision Review is the per-decision, per-symbol surfacing of the already-existing decision_observations x observation_labels ledger. It is the concrete UI for the "opportunity-level IC" work that docs/arch/09-learning-loop.md and the P1 gate already name as the next measurement layer.

It aligns with item #4 "Counterfactual trade journal" on the native roadmap in features/external-research-integrations/DEEP_CANDIDATE_CAPABILITY_AUDIT.md ("Advisory history only; never changes ledgers, positions, prices, or trades").

2. Hard boundaries (non-negotiable)

- Read-only / advisory / measurement-only. Decision Review reads existing

records and renders them. It never writes to agent_signals, paper_trades, learning_priors, strategy_versions, weights, genome, or any order path.

- Not a parallel truth layer. It does not create a new provenance/outcome

store. It reads the append-only decision_observations ledger (059) and the matured observation_labels (060) - the *same* records the LearnerAgent and Performance Truth Layer read. If a number is shown here, it came from those tables (or a deterministic view over them), never from a fresh recomputation that could diverge from the ledger.

- LearnerAgent remains the only thing that changes weights, on its own weekly

evidence, behind the Phase-0 (10+ closed trades) gate. Decision Review can *inform* the owner and *feed the same evidence tables* the LearnerAgent already reads, but it proposes nothing and mutates nothing.

- No LLM sets any number. Every score, return, gap, and attribution value is

deterministic SQL/arithmetic over the ledger. An LLM may only render the already-computed numbers into the one-line "what we'd learn" *prose* (optional, and clearly labeled as narration of fixed numbers - same posture as existing thesis text).

- Market-local, never mixed. US decisions are compared to USD prices and SPY;

India decisions to INR prices and ^NSEI. A view/API filtered by market never joins a USD entry price to an INR forward path. Currency is carried on every row (decision_observations.currency).

3. Data sources - everything needed already exists

3.1 decision_observations (migration 059) - the point-in-time score

Append-only (a mutation trigger dobs_block_mutation blocks UPDATE/DELETE). One row per candidate scored by ResearchAgent (filled OR rejected). Columns this feature reads:

Column	Use in Decision Review
`id`, `ts`, `market`, `symbol`	Row identity, market scope, score date **D**
`analyst_score`	The headline score under review
`fundamental_score`, `technical_score`, `sentiment_score`, `macro_score`, `insider_score`	Per-dimension scores (the attribution inputs)
`direction`	long / short / hold as scored (defines the "was it right?" sign)
`weights_used` (jsonb)	Dimension weights actually applied -> dimension contribution = weight x (score-50)
`availability_mask` (jsonb)	Which dimensions were real vs unavailable - an unavailable dim cannot be "wrong"
`features` (jsonb)	Raw sub-features (already parsed by `v_decision_quality`)
`price_at_decision`, `currency`	Entry basis (native currency) - the anchor for forward returns
`entry_eligible`, `action`, `score_threshold`	Whether we would have acted; held vs skipped context
`signal_id`	Link to `agent_signals` when a signal was written
`mandate_id` (migration 135)	Mandate/benchmark context (Swing US 2-20d -> V00/SPY; Swing India 2-20d -> ^NSEI)

3.2 observation_labels (migration 060) - the matured forward outcome

One row per (observation_id, horizon_days), written only after maturity by the label-maturation cron (app/api/agents/label-maturation/route.ts) so features can never see the future. Horizons already computed: 2, 5, 10, 20 trading days.

Column	Use
`horizon_days`	2 / 5 / 10 / 20 - maps to the "next 1d/5d/20d" UI (see ?4 note on the 1d question)
`fwd_return`	`(exit-entry)/entry - 10bps` cost haircut - the actual move
`benchmark_return`	Same-window SPY (us) / ^NSEI (india) return
`benchmark_neutral_return`	`fwd_return - benchmark_return` - alpha vs beta separation
`max_adverse_excursion` / `max_favorable_excursion`	Path risk inside the window (was the thesis ever right/wrong intraperiod)
`entry_price`, `exit_price`	Displayed anchors
`matured_at`	Freshness / "not matured yet" state

3.3 v_decision_quality (migration 122) - per-decision dimension bookkeeping

Already computes, per observation: applicable_dims, real_dims, missing_dims, degraded_dims, decisive_dim (highest-weight real dimension), data_confidence, confidence_band (high/med/low). Decision Review reuses this for the confidence column and to gate attribution to *real, non-degraded* dimensions only.

3.4 edge_ic_history / EdgeIC (migrations 132; lib/edges/ic.ts) - the aggregate we EXTEND

EdgeIC measures cross-sectional rank-IC of *edges* on the *broad candle universe*. Decision Review is the complementary view: not "does this factor rank winners across the universe" but "for this one decision on this one symbol, how did our composite score line up with what happened". Decision Review references EdgeIC as context ("technical-momentum edge IC at 20d = 0.04, t=2.1") but does not recompute or duplicate it, and never writes to edge_ic_history.

3.5 Forward-price source (for spot-checking / rendering only)

The forward returns themselves come from observation_labels (already matured). If the UI wants to *render a small price sparkline* from D to D+20, it reads cached candles the same way maturation does - price_cache then fetchUSCandles fallback (lib/data/candles.ts) for US, fetchIndiaCandles (lib/india-data.ts) for India. The numbers of record are always the label row, not a fresh fetch, to prevent drift from the ledger.

3.6 New table/column decision - prefer none; one optional derived view

- No new base/truth table. The ledger already holds score + outcome.
- No new provenance layer. decision_observations is the provenance.
- One optional read-only VIEW v_decision_review (deterministic join of 3.1 x 3.2

x 3.3, per market, per horizon) MAY be added to keep the query in one audited place. A view is not a truth layer - it stores nothing. Open question in ?9 is whether even this is worth it vs. an API-only join.

- The "opportunity-level IC" columns the roadmap already reserved

(strategy_evaluations.opp_*, null until P1 per docs/arch/09-learning-loop.md) are the correct home for any aggregate roll-up this feature surfaces - extend those, do not invent siblings.

4. Point-in-time forward returns (no look-ahead, market-local)

The maturation cron already implements this correctly; Decision Review consumes it and must not re-derive it differently.

- Entry anchor = decision_observations.price_at_decision (the close/quote used at scoring time), else the first candle on/after date D. Native currency.

- Window = the first H candles strictly after the entry date

(c.date > entryDate) - so D itself is never counted, and a horizon is only labeled once H trading days have actually elapsed (maturity). This is what prevents look-ahead: an un-elapsed horizon has no label row and renders as "maturing".

- Return = (exit - entry)/entry - 0.001 (10 bps round-trip cost haircut, LABEL_COST_HAIRCUT).

- Benchmark = SPY for us, ^NSEI for india, same window, same rule ->

benchmark_return; benchmark_neutral_return = fwd - benchmark isolates alpha.

- Market isolation = every query filters market and reads currency; a US label

is never compared to an INR benchmark and vice-versa. Mandate benchmark (VOO/^NSEI) comes via mandate_id.

The "1d" question. Labels exist at 2/5/10/20. The task asks for 1d/5d/20d. Two honest options (see ?9): (a) surface 2d/5d/20d and label the near bucket "~1-2d" (zero new pipeline work, no new provider calls); or (b) add horizon_days = 1 to the HORIZONS array in the maturation cron (one-line change, still measure-only, back-fills naturally). Recommendation: (a) for the first cut - 1d is the noisiest, least decision-relevant horizon for a 2-20d swing mandate, and adding it risks implying precision we don't have.

5. The per-symbol view (what the screen shows)

For a chosen (symbol, market) and a chosen decision date D (or the latest matured decision), one Decision Review card per horizon, plus a symbol-level history table.

5.1 Header - the decision under review

- Symbol, market badge, date D, analyst_score, direction, whether it was entry_eligible / acted / skipped, confidence_band (from v_decision_quality), mandate + benchmark.

5.2 Score vs actual move (per horizon 2/5/20)

- Scored: analyst_score and score-implied direction.
- Actual: fwd_return, benchmark_neutral_return, MAE/MFE (path).
- Gap: a deterministic, sign-aware gap metric:
- Directional hit/miss: did sign(direction) match sign(benchmark_neutral_return)?

- Magnitude gap: score mapped to an *ordinal* expectation bucket (high score ? expect top-tercile forward return) vs realized tercile - ordinal, not a fabricated regression (we deliberately avoid implying the score is a calibrated return forecast; it isn't).

5.3 Per-dimension attribution - "which dimension was most wrong"

Deterministic, and honest about its limits:

- Only dimensions that were real and non-degraded (`availability_mask + v_decision_quality.degraded_dims`) are eligible - an unavailable dimension cannot be blamed.

- Contribution of dimension *d* to the decision = `weights_used[d] x (score_d - 50)`

(how far, and in which direction, *d* pushed the composite off neutral).

- "Most wrong" = the eligible dimension whose contribution most opposed the realized `benchmark_neutral_return` sign, weighted by `|contribution|`. Example: score was pushed long mostly by sentiment (+high contribution), stock fell vs benchmark ? sentiment is flagged as the most-wrong dimension for this decision.

- "Most right" symmetric, for balance.

- Displayed as a small horizontal contribution bar, colored by whether each dim helped

or hurt this realized path - explicitly labeled "for this one outcome," never as a causal weight-change recommendation.

5.4 Confidence

- `confidence_band` from `v_decision_quality` (data completeness), plus a

sample/maturity caveat: a single symbol-decision is `n=1`. Per-symbol attribution is always shown with an explicit "`n=1` anecdote - see aggregate below" marker.

5.5 Plain-English "what we'd learn"

- A templated, deterministic sentence assembled from the computed fields, e.g.:

"On 2026-06-30 we scored NVDA 78 (long), driven mainly by technical (+12) and sentiment (+9). Over 20d it returned -4.1% vs SPY (-5.3% alpha). Sentiment most opposed the outcome. This is one decision (n=1); the sentiment dimension's 20d IC across all decisions is +0.01 (t=0.7, not significant) - no weight change is warranted on this evidence."

- Optional LLM pass only *rephrases* this fixed string; it introduces no new numbers.

5.6 Symbol history + aggregate roll-up (the honest layer)

Because `n=1` per decision is meaningless, the symbol page's default emphasis is the aggregate over that symbol's decisions (and, more importantly, over the regime/dimension cohort): hit-rate, mean `benchmark_neutral_return`, and a per-dimension "how often did this dim's push agree with the outcome" - only shown once the cohort clears a sample floor (reuse the Performance Truth 20-trade / P1 honesty rule; below floor -> `insufficient_sample`, no fabricated precision).

6. C4 sketch

6.1 Context

```
flowchart TB
  Owner([Owner / operator]) -->|reads, never edits| DR[Decision Review\ntab in Research Journal]
  DR -->|read-only SQL| LEDGER[(decision_observations 059\nobservation_labels 060\nv_decision_quality 122)]
  MAT[label-maturation cron\nnighly, existing] -->|writes matured labels| LEDGER
  RA[ResearchAgent\nexisting] -->|append-only score rows| LEDGER
  DR -.reads for context only.-> EIC[(edge_ic_history 132)]
  DR -.n=1 caveat points owner to.-> PT[Performance Truth / P1\nstrategy_evaluations.opp_*]
  DR -. never writes .-x LEARN[LearnerAgent / weights / orders]
```

6.2 Container

```
flowchart LR
  subgraph UI[Next.js dashboard]
    TAB[/dashboard/research-journal?tab=review\nDecisionReviewPanel.tsx/]
  end
  subgraph API[Next.js route handlers - read only]
    EP[GET /api/decision-review\n?market&symbol&date&horizon]
  end
  subgraph DB[Supabase Postgres]
    V[(v_decision_review\noptional deterministic view)]
    O[(decision_observations)]
    L[(observation_labels)]
    Q[(v_decision_quality)]
  end
  TAB --> EP --> V
  V --> O & L & Q
  EP -. optional sparkline .-> PC[(price_cache / providers)]
```

Everything in the API/DB path is SELECT-only. No route in this feature issues INSERT/UPDATE/UPSERT to any table.

7. Where it lives in the UI

A fourth tab in the existing Research Journal page (app/dashboard/research-journal/page.tsx), sibling to Score Tracker:

Daily Funnel | Evolution | Score Tracker | Decision Review

- Reached via /dashboard/research-journal?tab=review (mirrors the existing ?tab=scores redirect pattern; a /dashboard/decision-review route may redirect here for discoverability).
- Market-scoped via the existing market toggle/param (?market=us|india); the panel never renders mixed-market rows.
- Deep-link from Score Tracker: "review this decision" on any historical score point opens the same symbol/date in Decision Review. Deep-link from the Research Journal Daily Funnel row and from /dashboard/symbol/[symbol].
- Reuses existing dashboard chrome (PageHeader, theme tokens T, tab button styling)
- no new layout system, per Drift Prevention.

8. Phased rollout (measure-only first)

Phase A - Surface (measure-only, no new pipeline). Read `decision_observations x observation_labels x v_decision_quality`; render the per-horizon card (2/5/20), score-vs-actual, and the symbol history table. Attribution shown but heavily n=1-caveated. No migration beyond the optional `v_decision_review` view. Ships behind the same measure-only posture as EdgeIC.

Phase B - Attribution + aggregate honesty. Add the deterministic per-dimension contribution/most-wrong logic and the cohort/regime aggregate roll-up gated by the 20-trade/P1 sample floor. Wire the "what we'd learn" template. Still writes nothing.

Phase C - Optional 1d horizon + Performance Truth link. If owner wants true 1d, add `horizon_days = 1` to the maturation HORIZONS (one-line, measure-only). Surface the aggregate per-dimension agreement into the reserved `strategy_evaluations.opp_*` columns as read-through display, feeding the *same* evidence the LearnerAgent already consumes - never a new recommendation channel.

Never a phase: auto-adjusting weights from Decision Review. That stays the LearnerAgent's weekly, gated job.

9. Acceptance tests

- Read-only proof: grep the feature's API + panel for any write verb
(`insert | update | upsert | delete | .rpc()`) -> zero hits against ledger/weight/order tables.
- No look-ahead: a decision whose newest horizon hasn't elapsed shows "maturing" and no fabricated return; forcing a render never fetches post-cursor candles for the number of record (it reads only the existing label row).
- Market isolation: a US symbol page never displays an INR price or an ^NSEI benchmark; India never shows SPY. Currency badge matches `decision_observations.currency`.
- Ledger fidelity: every number on the card equals the corresponding `decision_observations / observation_labels` field byte-for-byte (no recompute drift).
- Attribution eligibility: a dimension with `availability_mask[d] = false` or in `degraded_dims` is never flagged "most wrong."
- Sample honesty: with a below-floor cohort, the aggregate panel shows `insufficient_sample`, not a number (mirrors Performance Truth's 20-trade rule).
- Determinism: same inputs -> same card on repeat load; the optional LLM prose pass, if disabled, leaves every number and the templated sentence intact.
- Boundary: disabling/deleting this feature changes no ledger row and no weight - it is pure projection.

10. Riskiest assumption (call it out loud)

Forward-return attribution ? causation, and per-symbol samples are tiny (n?1). Flagging a dimension as "most wrong" for a *single* realized path is an anecdote: the stock's move is dominated by market/sector beta and idiosyncratic noise, not by which sub-score we over-weighted. If the UI presents per-symbol attribution as actionable, the owner (or a future LearnerAgent reading it) could chase noise - exactly the overfitting the Phase-0 "10+ closed trades" rule and the EdgeIC t-stat hurdles exist to prevent.

Mitigation (locked into the design): per-symbol/per-decision attribution is always labeled $n=1$ and is decorative; the decision-relevant number is the aggregate/regime/dimension cohort roll-up, shown only above the Performance-Truth sample floor, with the aggregate EdgeIC t-stat displayed alongside so a non-significant dimension is visibly non-significant. Decision Review informs; it never proposes, and weight change stays the LearnerAgent's gated, aggregate-evidence job.

11. Open decisions for owner / Claude

- (Biggest) Attribution methodology & how loudly to show per-symbol "most wrong":
is the deterministic $\text{weight} \times (\text{score} - 50)$ vs realized-sign contribution rule the right definition, and should per-symbol attribution be shown at all in Phase A or deferred until the aggregate cohort layer (Phase B) exists to anchor it? (This is the single point where the feature could either become genuinely useful or quietly encourage overfitting.)
- Add `horizon_days = 1` now (true 1d) or surface 2/5/20 and label the near bucket
"-1-2d"? (Recommendation: defer 1d.)
- `v_decision_review` deterministic view vs API-only join? (Both are read-only; the view centralizes the contract, the API-only path avoids a migration entirely.)

downside hedging - Feature Architecture

Source: features\downside-hedging\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Downside Hedging - Feature Architecture

Status: APPROVED FOR BUILD (2026-07-15). US PAPER ONLY. SHIPS OFF.

Purpose

Add bounded downside protection without true short selling, margin, options, or leveraged products. Kairos may buy an explicitly allowlisted unleveraged inverse ETF as a small portfolio hedge after deterministic macro and market confirmation. The hedge is portfolio insurance, not an alpha candidate.

Locked Decisions

- US paper book only. No India proxy is invented.
- Initial instruments: SH and PSQ, both -1x daily-reset inverse ETFs.
- Every 2x/3x, volatility, commodity-inverse, and single-name inverse product remains blocked.
- Normal research, PaperTrader, Trader, rotation, and live execution continue to reject every leveraged/inverse ETF. Only the dedicated paper-hedge RPC has a narrow SH/PSQ exception.
- No LLM may arm, select, size, enter, exit, or mutate the hedge.
- Entry is cash-funded. Capital Rotation cannot sell an alpha holding to fund it.
- Default target is 5% NAV; hard ceiling is 8%; one hedge position maximum.
- Maximum holding period is five market days. Normal stop protection still works.
- Performance is judged on whole-book drawdown, volatility, turnover, and benchmark-relative return after hedge drag, never hedge P&L alone.

Deterministic State Machine

off -> armed -> active -> exit_pending -> cooldown -> off

Entry requires both:

- Macro confirmation: latest US macro_regime.danger_score >= 60.
- Market confirmation: SPY is below its 50-session SMA and either 20-session return <= -4% or 20-session drawdown <= -6%.

The condition must persist for two completed evaluations. PSQ is selected only when QQQ is at least three percentage points weaker than SPY over the same 20-session window; otherwise SH is used.

Exit requires danger <= 45, SPY above SMA50, and non-negative five-session return, persisting for two evaluations. The five-market-day time stop and 5% price stop can exit sooner. Missing/stale data never enters and never fabricates an exit.

Data Model

- `downside_hedge_config`: US-only controls, seeded disabled.
- `downside_hedge_state`: mutable current state and confirmation streaks.
- `downside_hedge_events`: append-only evaluation/lifecycle ledger.
- `paper_positions.position_role` and `paper_trades.position_role`: alpha or hedge; hedge trades are always `excluded_from_learning=true`.

Owner-authenticated users may read configuration/state/events through owner RLS. Only service-role routes/RPCs write. Anonymous access is revoked.

Execution

POST `/api/agents/downside-hedge` runs after the US close. It loads macro state and point-in-time daily bars, evaluates the pure state machine, and appends an event.

- `enabled=false`: no evaluation or provider call.
- `enabled=true`, `paper_execute_enabled=false`: shadow measurement only.
- both true: an enter decision creates a non-alpha control signal and calls

`execute_paper_hedge_fill`, which rechecks configuration, state, allowlist, one-position limit, cash, and NAV cap under DB row locks before delegating to the existing atomic paper-fill function.

- an exit decision marks only the hedge position `hedge_exit`; `PositionMonitor` closes it through the existing paper accounting path.

`PositionMonitor` never applies ordinary analyst-score/direction exits to a hedge. It retains price stop and time stop behavior. After a hedge disappears, the next controller run reconciles `exit_pending/active` to cooldown before re-entry.

Product Surface

Settings -> Agents contains a Downside Hedge panel showing state, latest reason, target/cap, instruments, and separate Shadow and Paper Execution toggles. Paper execution cannot be enabled unless shadow is enabled. There is no live toggle.

Acceptance Criteria

- Default/live DB state is OFF for both flags.
- Generic research, paper, trader, rotation, and live gateway still block SH, PSQ, and all leveraged/inverse products.
- LLM text cannot affect any evaluator result.
- One failed/missing/stale input cannot create an entry.
- RPC rejects non-US, non-allowlisted, over-cap, non-cash-funded, duplicate, or improperly sourced fills.
- Hedge trades cannot enter learner/validation datasets.
- US and India books are never summed or mutated together.
- Typecheck, full Vitest, production build, live migration/RLS/OFF-state checks,

and Vercel deployment pass without placing a live order.

earnings aware risk - Feature Architecture

Source: `features\earnings-aware-risk\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Feature Architecture: Earnings-Aware Risk

Status

Architecture status: Approved P0 - implemented 2026-07-29 Architecture approved: Yes Approved scope: Measure-only US paper/live annotations, India proximity, display-only holding warnings, normalized append-only evidence Implementation allowed: P0 only

No behavioral policy is approved. `block` and `size_down` remain unavailable.

Decision

Use earnings proximity and an options-derived move proxy as risk context, not as a sixth alpha score.

Build the first release as measure-only:

- US paper: annotate every otherwise-eligible entry.
- US live: annotate, while preserving the existing Alpha Vantage earnings

blackout exactly as-is.

- India: annotate earnings proximity when available; options risk is

unavailable.

- Existing positions: surface a warning only. `PositionMonitor` and live exits do not change.

No block, size reduction, stop widening, score change, promotion input, or automated exit is allowed until the shadow record passes the acceptance gates in this document and the owner separately approves the behavior.

Why This Belongs In Risk, Not Alpha

Short-dated option prices around a known earnings event contain information about the market price of uncertainty. They do not provide a reliable sign for the stock's next move. Skew and flow may contain directional information in some settings, but retail-visible volume cannot reliably identify opening versus closing trades, buyer versus seller initiation, or multi-leg hedges.

Kairos may therefore use:

- earnings timing as event-risk metadata;
- a same-expiry ATM straddle as an expiry-bounded move proxy;
- the ratio of that proxy to the planned stop distance as context.

Kairos must not use:

- put/call ratio, "unusual calls", or skew as automatic long conviction;
- the move proxy as a directional forecast;
- options fields in `analyst_score` or the `LearnerAgent` objective.

This feature does not need directional IC validation because it makes no directional alpha claim. It does need operational and calibration validation: coverage, quote quality, date accuracy, implied-versus-realized move calibration, and counterfactual effects on entries and outcomes.

Verified Current State

```
| Area | Current behavior |
| US PaperTrader | No earnings gate or annotation at the fill choke point |
| US TraderAgent | Already blocks proposals from 2 calendar days after through 5 calendar days before an
Alpha Vantage earnings date |
| Existing live gate | Fetches Alpha Vantage directly, fails open on timeout/unavailable data, and is not
shared with PaperTrader |
| `lib/data/earnings.ts` | Finnhub US and Yahoo India proximity helper; returns days only and loses source/
as-of/confidence metadata |
| `lib/data/earnings-pit.ts` | Captures point-in-time earnings vintages |
| `lib/options-signal.ts` | On-demand Yahoo nearest-expiry chain; not on a trading path |
| Current `ivPercentile` | Not a 52-week IV percentile. It locates ATM IV inside the current chain's strike-
IV range |
| Current unusual-flow labels | Comments and summary overstate contracts as "smart money" evidence |
| Paper audit | `pipeline_stage_events` records stage decisions |
| Live audit | `trade_proposals.risk_check_reasons` records proposal risk checks |
```

The new feature must not accidentally replace or weaken the existing live blackout. A unified policy can replace it only after shadow coverage parity and an explicit owner-approved cutover.

Verified Broker Data Surfaces

The connected broker surfaces were re-probed on 2026-07-29. Availability is useful, but tool presence alone is not evidence that a field is correctly timestamped, complete, or stable enough for a money-path decision.

```
| Source | Earnings | Options | Verified state | Intended role |
| Finnhub | Yes | No | Existing US `fetchDaysToEarnings()` source | Current US calendar source and
independent fallback |
| Alpha Vantage | Yes | No | Existing live TraderAgent blackout source | Preserve until unified resolver
proves parity |
| Webull MCP | Yes | No | Live `tools/list`: 71 tools. `get_stock_earnings_calendar` returned AAPL dates,
actual/estimated EPS, and actual/estimated revenue | US earnings-date cross-check and optional estimate
context |
| Robinhood MCP | Yes | Yes | Live `tools/list`: 52 tools, including `get_earnings_calendar`,
`get_earnings_results`, `get_option_chains`, `get_option_quotes`, and `get_option_historical` | Candidate
broker-backed earnings cross-check and preferred US options source after payload probe |
| Yahoo | Yes for India; US fallback possible | Yes | Existing India calendar helper and current on-demand
US options implementation | India calendar source; shadow-only US options fallback |
```

Webull's published Cloud MCP surface does not expose an equity-options chain, despite exposing stock quotes and many fundamentals tools. Do not infer options support from Webull's brokerage product or separate trading documentation.

Robinhood is the strongest candidate for the US move proxy because the connected MCP advertises contract discovery plus current quotes. Before selection as a provider, an owner-run allowlisted probe must verify:

- exact expiry and strike fields;
- same-contract bid, ask, sizes, timestamp, and open interest;
- whether quotes are real-time, delayed, indicative, or entitlement-dependent;
- response pagination and maximum contracts per request;
- behavior outside market hours and for illiquid names;
- rate limits and token/session failure behavior.

Broker OAuth calls do not consume Finnhub, Alpha Vantage, or Yahoo quotas, but they are not quota-free by assumption. They require independent pacing, provider-call accounting, caching, and health monitoring. A disconnected broker must become unavailable; it must never silently change entry behavior.

Source resolution

Earnings dates should be prewarmed and resolved from the existing point-in-time calendar cache rather than fan-out to three providers synchronously at every entry. Finnhub, Webull, and Robinhood observations retain source and as-of metadata. A disagreement beyond one market session is `conflict`, not a majority vote.

For US option snapshots:

- Robinhood is the preferred candidate only after its payload probe passes.
- Yahoo remains a shadow-only fallback with an explicit `unofficial-source` tag.
- Webull returns `unsupported`, not `unavailable`, for options.
- No broker order or option-order tool is used. Read and execution allowlists

remain separate.

Risk Contract

Earnings event

```
type EarningsEventRisk = {
  market: "us" | "india";
  symbol: string;
  reportDate: string | null;
  reportSession: "bmo" | "amc" | "during_session" | "unknown";
  sessionsUntilReport: number | null;
  source: string | null;
  observedAt: string;
  confidence: "confirmed" | "estimated" | "unknown";
  status: "available" | "unknown" | "conflict";
};
```

Calendar sessions, not UTC duration or raw calendar-day subtraction, determine proximity. If sources disagree beyond one market session, status is `conflict`; the system must not silently choose the more convenient date.

Options move proxy

```
type EarningsMoveProxy = {
  market: "us";
  symbol: string;
  observedAt: string;
  quoteAsOf: string | null;
  reportDate: string;
  reportSession: EarningsEventRisk["reportSession"];
  expiry: string;
  spot: number;
  strike: number;
  callBid: number;
  callAsk: number;
  putBid: number;
  putAsk: number;
  callMid: number;
  putMid: number;
  moveProxyPct: number;
  stopDistancePct: number | null;
  stopToMoveRatio: number | null;
  quality: "usable" | "wide_spread" | "stale" | "illiquid" | "unavailable";
  reason: string;
};
```

The calculation is:

```
moveProxyPct = (ATM call mid + ATM put mid) / contemporaneous spot
```

It is labelled expiry-bounded ATM straddle move proxy, not "earnings implied move". The premium contains event variance, non-event variance through expiry, volatility risk premium, rates/dividends, and bid/ask effects.

Deterministic Chain Rules

- Select the earliest standard expiry that is after the earnings event and leaves at least one tradable post-event session. Account for BMO versus AMC; unknown timing makes the proxy lower-confidence.
 - Fetch that exact expiry. The current Yahoo call only reads `options[0]` and is insufficient.
 - Select the nearest strike present in both calls and puts.
 - Use mids only when both bid and ask are finite, non-negative, and ask is not below bid. Last trade is never a substitute.
 - Require a configurable maximum spread-to-mid ratio on each leg, nonzero open interest or quoted size, a fresh quote timestamp, and a contemporaneous spot.
 - Reject crossed, stale, zero-premium, missing-leg, or implausible results.
 - Cache the raw normalized snapshot by ``(symbol, expiry, observed market session)``. Never overwrite a snapshot used by a decision.
 - Every provider is fail-soft and source-labelled. Robinhood requires an allowlisted payload probe; Yahoo is unofficial and remains a shadow fallback. A capability probe and provider-health metric are required before shadow runs.
- The first build must rename or remove the current `ivPercentile` label and remove "smart money" claims from unusual-flow summaries. Neither field is part of this feature's decision contract.

Policy

```
type EarningsRiskPolicy = {
  version: number;
  mode: "shadow" | "block" | "size_down";
  market: "us" | "india";
  proximitySessions: number;
  maxLegSpreadToMid: number;
  maxQuoteAgeSeconds: number;
  minOpenInterest: number;
  sizeMultiplier: number;
};
```

Initial production configuration is permanently pinned to shadow until an owner-approved migration/config change activates another mode.

`earningsRiskVerdict()` is pure and deterministic. It may only preserve or reduce entry eligibility/size. It can never:

- increase a score, size, price target, or holding horizon;
- widen or remove a stop;
- suppress a sell or any `PositionMonitor`/live-exit action;
- convert unknown data into "no earnings";
- borrow US options evidence for India.

In shadow mode, unavailable data records unknown and does not block. In a future active mode, unavailable behavior must be decided explicitly; it cannot inherit a generic fail-open default.

Money-Path Integration

Paper

Run the pure verdict after the fill-bound stop/target plan exists and before the atomic `execute_paper_fill` call. In P0, only log the counterfactual verdict to `pipeline_stage_events`; do not alter the RPC inputs or claim lifecycle.

Capital rotation must use the same annotation contract. A rotation cannot bypass measurement merely because it enters through the slot/cash replacement path.

Live

Run the shared annotation before proposal insertion and persist it inside `trade_proposals.risk_check_reasons`.

The existing Alpha Vantage -2/+5 blackout remains authoritative during P0. The new resolver must not catch an error and skip that existing gate. Replacement requires:

- coverage parity over predeclared observations;
- no regression in known-date blocking;
- an owner-approved policy version;
- one atomic cutover that removes the direct Alpha Vantage path.

Autonomous live execution must re-check the persisted policy version and verdict freshness before submitting an approved proposal, just as it re-checks other latched money-path controls.

Existing positions

P0 may show:

- earnings date/session;
- sessions remaining;
- usable move proxy;
- stop-to-move ratio;
- data-quality state.

It must not cause PositionMonitor or the live exit monitor to trim, close, move a stop, or alter a target. Gaps can jump resting or synthetic stops, so `stopToMoveRatio < 1` is a warning, not proof that the stop is wrong.

Persistence And Provenance

Do not create a parallel evidence truth layer.

- Source payload/fingerprint references use existing `evidence_records`.
- Paper decision context uses `pipeline_stage_events.detail`.
- Live decision context uses `trade_proposals.risk_check_reasons`.
- If cross-flow analytics cannot be made reliable over those ledgers, add one

append-only `earnings_risk_observations` table only after schema approval. It must reference the existing signal, paper event/proposal, evidence record, policy version, and source snapshot rather than duplicating source truth.

No raw option chain is exposed to the browser. Owner-facing APIs return only the normalized fields above. RLS is owner-read; writes are service-only and append-only.

Shadow Metrics And Acceptance Gates

Predeclare the shadow window before collecting outcomes. Minimum gates:

- At least 60 otherwise-eligible US entry decisions and at least 20 distinct earnings events.
- Earnings-date coverage at least 95% for symbols where the legacy live gate found a date.
- Date disagreement, reschedule, and unknown rates reported separately.
- Usable option snapshot coverage reported by liquidity tier; no silent exclusion of thin names.
- Compare proxy with absolute close-to-close and close-to-next-open event moves.
- Report empirical exceedance rates for 0.5x, 1.0x, and 1.5x proxy bands.
- Counterfactual report: entries blocked/sized down, later P&L, MAE/MFE, stop-outs, and sample sizes. No claim based only on average P&L.
- No paper/live eligibility or size change while mode is shadow.

These observations test calibration and product usefulness, not directional IC. A future block/size-down proposal must specify its expected trade-count cost and must default to no change when evidence is inconclusive.

Build Order

- Correct misleading existing option labels and add golden parser tests.
- Add bounded read-only Robinhood earnings/options contract probes and persist only schema fingerprints plus normalized test results.
- Add Webull earnings into the source-aware, session-aware earnings-event resolver; retain Finnhub/Alpha Vantage observations through cutover.
- Add exact-expiry Robinhood fetch, Yahoo fallback, and pure quote-quality/move-proxy calculation.
- Add pure `earningsRiskVerdict()` with shadow as the only enabled mode.
- Wire paper, rotation, and live annotation without changing behavior.
- Add owner-facing Backtest/Risk visibility and provider-health coverage.
- Accumulate the predeclared shadow record.
- Review evidence and separately decide whether to keep annotation-only, activate bounded size-down, or reject the behavioral feature.

Tests Required Before P0 Ships

- BMO, AMC, unknown timing, weekend, holiday, and rescheduled event cases.
- Exact expiry selection and same-strike call/put pairing.
- Missing leg, zero bid, crossed quote, stale quote, wide spread, and no-OI cases.
- US/India isolation.
- Paper fill and capital-rotation paths both produce annotation.
- Existing live blackout still blocks every case it blocked before.
- Annotation failure cannot alter eligibility, sizing, or exits.
- Idempotent retries do not create conflicting observations.
- Browser payload excludes raw chain data.

Sources

- Cboe explains that short-dated options around earnings primarily indicate the magnitude, not direction, of the expected reaction and compares straddle pricing with historical moves:
<https://www.cboe.com/insights/posts/what-options-data-may-indicate-about-mag-7-earnings>
- Chung and Louis document that earnings-event option returns and implied versus realized volatility vary with pre-event conditions, which is why calibration cannot be assumed:
https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2886040
- SEC material notes that disseminated option quotes can be stale during market data stress, supporting explicit freshness and quote-quality gates:
<https://www.sec.gov/rules-regulations/2001/09/firm-quote-trade-through-disclosure-rules-options>

Owner Decision

Recommended approval, when requested:

- P0 measure-only implementation;
- US paper + US live annotation;
- India proximity annotation only;
- existing live blackout preserved;
- existing positions display-only;
- no scoring, sizing, entry, stop, target, or exit behavior change.

Do not approve block or size_down yet.

P0 Implementation Result

Implemented in `lib/risk/earnings-risk.ts` and recorded in `IMPLEMENTATION_RESULT.md`. Production is pinned to policy version 1 mode='shadow'; the database rejects any other mode and rejects `behavior_changed=true`.

Holdings coverage extension (2026-07-30)

The original P0 hooks observed an options move proxy only after an entry reached PaperTrader or TraderAgent. A bounded weekday US holdings monitor now reads open paper positions plus the latest complete per-account live-risk snapshots and records `decision_kind='holding'` observations. Event discovery merges the persisted PIT cache with one bounded Robinhood calendar read, so a cache miss does not hide a known holding's earnings date. Per-symbol earnings/options calls are requested only when that event is inside the holding horizon. This extends risk visibility to names already owned without turning options into an all-watchlist provider fan-out or a directional score. India remains proximity-only and has no scheduled options leg.

earnings repricing barrier - Feature Architecture

Source: `features\earnings-repricing-barrier\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Earnings Repricing Barrier

Status: Approved corrective safety change, implemented 2026-07-30.

Problem

Daily technical inputs can remain pinned to the pre-earnings close after an AMC or BMO release. On 2026-07-30, MSFT was a held position and received a deterministic short from a July 29 close despite a July 29 after-close result. That is a stale-data decision, not an informed reaction.

Rule

When the existing PIT earnings calendar has a recent event that has already occurred, a research score cannot direct an entry or a score/direction exit until the daily candle series contains the event reaction. A before-open report may use the completed report-date bar; an after-close or unknown-session report requires a later bar. A known past event activates the barrier even when the actual-result feed is late.

- New candidate: direction becomes neutral and is ineligible for PaperTrader.

The signal remains session-validated so consumers see this current abstention rather than falling back to an older pre-event direction.

- Held position: direction becomes neutral, preventing stale score/direction

exits. Price stops, targets, trailing stops, and time exits are unchanged.

- The calendar lookup is read-only and provider-free. It never infers that an earnings beat means buy, and it never uses options as directional alpha.

- Calendar read failure is an unknown control-plane state and therefore writes a current neutral abstention. It cannot authorize a stale score. The lookup has no seven-calendar-day expiry: the most recent prior event remains checked until a qualifying post-event bar exists.

- Every collector uses the same real YYYY-MM-DD validator. Impossible values such as 2026-02-31 are unavailable, never normalized into another day.

Scope

No migration, new provider, options call, score-weight change, manual-live change, or paper/live order is added. The existing next research session after a complete post-event daily bar resumes normal deterministic scoring.

edge calibration readiness - Feature Architecture

Source: features\edge-calibration-readiness\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Edge Calibration Readiness Monitor

Status: Approved Owner: Kairos Decision date: 2026-07-21 Scope: Deterministic, measure-only US and India evidence governance

1. Problem

Weekly EdgelC runs collect immutable factor evidence, but Kairos does not currently answer three operational questions reliably:

- How many independent weekly windows have accumulated for each factor, market, and horizon?

- Has collection stalled or degraded?

- When is the evidence sufficient to build the next validation phase or request a shadow review?

Relying on a human to remember to check in six to eight weeks is fragile. Treating a single shadow_eligible IC row as readiness is worse: it ignores stability, point-in-time quality, costs, turnover, and multiple testing.

2. Decision

Build a deterministic readiness monitor over the existing append-only edge_ic_history ledger. It publishes progress to the Edge dashboard and emits a one-time informational notice when a factor crosses a governance milestone.

It has no LLM and no authority over scoring, signals, mandates, positions, cash, proposals, brokers, or orders.

3. Milestones

3.1 collecting

Fewer than six independent market-wide windows exist. Windows use the latest snapshot for each window_end and must be at least five calendar days apart, so manual reruns and provider corrections do not inflate progress.

3.2 needs_stability

Six windows exist, but the bounded historical diagnostic does not satisfy every Kairos policy gate:

- minimum observation count of 72 in every selected window;
- positive IC in at least five of six windows;
- median IC at least 0.02; and
- median Newey-West t-statistic at least 1.5.

These thresholds are conservative Kairos governance policy, not a claim of universal statistical truth. They may be versioned later; old results remain immutable.

3.3 ready_for_validation_build

The historical stability gates pass, but fewer than four independent rows carry the future evidence quality `pit_walk_forward_cost_adjusted_fdr` with non-null turnover and net-of-fee IC. This milestone means only: build or run the point-in-time, walk-forward, cost-adjusted, multiple-testing-controlled validation layer.

3.4 ready_for_shadow_review

At least four independent validation windows exist and all of these gates pass:

- evidence quality is exactly `pit_walk_forward_cost_adjusted_fdr`;
- net-of-fee IC is positive in at least three of four windows;
- median net-of-fee IC is at least 0.01; and
- turnover is finite and non-negative in every validation window.

This remains a review milestone. It never changes an Edge lifecycle state and never grants production, paper, or live trading permission.

3.5 FDR Contract Before Any Validation Row Can Qualify

`pit_walk_forward_cost_adjusted_fdr` is not a label a route may set by convention. Before the first row claims that evidence quality, the validation design must specify and persist: the complete predeclared trial family (factor formula/version, market, horizon, segment, and parameter variants), the family size, raw p-value, and the Benjamini-Hochberg procedure at $q = 0.05$. The correction is applied independently per market and validation family; US and India are never pooled. Missing/non-finite p-values, an unknown family size, or a changed family definition fail closed and keep the row below the validation milestone.

The current EdgeIC implementation does not yet supply this contract or immutable candle vintages. Consequently no current row may claim `pit_walk_forward_cost_adjusted_fdr`, and the monitor may report only collection or historical-stability progress until a separate approved validation-layer build.

4. Data Model

`edge_readiness_status` is a latest-state projection keyed by (`edge_id`, `market`, `horizon`):

- stage and policy version;
- observed/required windows and positive-window count;
- median IC/t-stat, minimum observations, latest window/session;
- validation-window counts and median net-of-fee IC;
- deterministic gate details;
- separate first-notified timestamps for each readiness milestone; and
- evaluation timestamp.

The immutable source remains `edge_ic_history`. The projection may be updated because it is not an evidence ledger. RLS is enabled and access is service-role only, matching `edge_market_status`.

5. Scheduling And Alerts

A provider-free daily cron runs after the Monday EdgeIC time slot. It:

- reads only market/all rows;

- collapses same-window provider revisions to the latest snapshot;
- evaluates every factor/market/horizon independently;
- upserts the latest projection;
- emits at most one batched informational notice per market/milestone/run for newly qualifying factor-horizons;
- emits a warning when a market has no successful IC snapshot for more than ten days; and
- resolves the stall warning after recovery.

Daily evaluation is deliberately cheap and idempotent: it reads only Supabase rows and makes no market-data request. This lets a missing weekly IC run become visible on day 11 rather than waiting for another weekly monitor invocation.

One-time readiness notices use separate milestone timestamps in the projection. Dismissing a notice does not cause it to reopen every week, and a regression from a later milestone cannot re-notify an earlier one. A later policy/formula version gets a new identity and can notify independently.

6. Dashboard

The Edge page shows:

- market and horizon;
- stage;
- weekly-window progress;
- positive-window count;
- median IC and minimum sample;
- validation-window progress; and
- the next required action.

No visible status may use the phrase "trade ready." Empty or failed reads render as unavailable, never as zero evidence or ready.

7. Safety And Failure Modes

- US and India are never combined.
- Sector rows are diagnostic and never satisfy market readiness.
- Same-window reruns cannot increase the weekly-window count.
- Legacy/unverified rows may count only as retrospective diagnostics, never as validation windows.
- Null/NaN metrics fail their gate.
- A route or DB failure reports a monitor warning but cannot affect EdgeIC itself.
- Readiness alerts are informational; stalled collection is warning severity.
- No provider calls are made by the monitor.

8. Acceptance Criteria

- Pure evaluator tests cover deduplication, weekly spacing, thin samples, unstable signs, historical readiness, validation readiness, market isolation, and nulls.
- Route source contains no score, signal, mandate, position, cash, proposal, broker, or order write.
- One-time notification state survives alert dismissal.
- Dashboard exposes truthful progress and the next gate.
- Cron, migration, RLS, TypeScript, tests, production build, and live smoke pass.

edge factor discovery - Feature Architecture

Source: features\edge-factor-discovery\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Edge/Factor Discovery + Signal Validation + Regime Filter

Status: P0/P1 MEASURE-ONLY BUILT. P2+ NOT APPROVED OR WIRED. Author: Claude (Kairos). Date: 2026-07-08. Reviewed/updated by ChatGPT (Codex), 2026-07-08 - source-backed methodology review through 2026. Intended reviewers: other LLMs / quant-literate readers. This doc is written to be self-contained enough to critique without the codebase in hand.

0. TL;DR

Kairos today generates trade ideas by having an LLM read ~15 news headlines and invent "themes," then scores candidates on 5 soft dimensions and paper-fills the top ones. That makes the LLM the alpha source - a black-box, news-chasing, overfit-prone origin that the systematic-trading literature explicitly warns against.

This proposal moves Kairos to the standard institutional shape:

- Discovery = a catalog of explicit, academically-grounded EDGES (factor, technical, calendar, structural, information), each a deterministic formula - not an LLM guess.

- Every signal must earn its way in via an Information Coefficient (IC)

stability gate (rolling IC, IR, t-stat, decay) before it is allowed to size normal paper capital or become live-eligible. Tiny, capped exploratory paper may be allowed for candidate edges so the system can learn without pretending the edge is proven.

- A ranked Regime Filter starts as a continuous size scaler, scored on protection / cost / stability / cross-market reach, replacing today's implicit, score-only regime handling. A hard on/off switch is a later, evidence-gated owner decision.

- The LLM is demoted from alpha source to research synthesizer:

hypothesis generation from literature, explanation, code/report drafting, orchestration. It never originates the number that sizes a position.

- Validation stays fees-first + walk-forward OOS + Monte Carlo (extends the existing Validation Engine), so nothing trades live without surviving that.

The existing evolution machinery (genome, champion/challenger, shadow decisions, learner, calibration, Build 4a slip) is reused, not replaced - this proposal gives it a better *input* (validated factor signals) and a better *gate* (IC + regime), rather than rebuilding the loop.

0A. ChatGPT reviewer correction - methodology update

The direction is correct, but the first draft over-restricts exploration and overstates regime gating. The safer target architecture, based on current factor-research practice through 2026, is:

- ThemeScout remains attention discovery only. It may add candidates to the watchlist, but it never creates tradeable alpha or trade permission.

- EdgeScout / FactorScout becomes the deterministic alpha factory. It

computes formula-based edge values and validates them statistically.

- Candidate edges may enter shadow/exploratory paper with tiny caps before full

IC proof. Otherwise the system cannot explore. Normal paper sizing and any live eligibility still require validation.

- Regime starts as a continuous size scaler, not a hard on/off switch. A hard block can be added later only after measured evidence and owner approval.

- Point-in-time data becomes a first-class gate. No factor result is trusted

unless every input has source, as_of_date, available_at, revision policy, and corporate-action adjustment policy.

- Long-only validation is required. Kairos does not run long/short books; IC

is useful, but the decisive test is whether the top-ranked long-only bucket beats cash/benchmark after realistic costs.

- Newly discovered factors face stricter multiple-testing gates than

academically-prioried factors. A t-stat around 2 can be enough for known priors to enter shadow/paper; newly discovered/data-mined edges need a higher hurdle such as t-stat > 3 or FDR/q-value control before active/live eligibility.

1. Motivation - what the literature says vs. what we do

Sources the owner supplied (SetupAlpha Medium series: "Scientific Workflow for Generating Alpha 2026", "60+ Market Edges 2026", "20 Trend-Based Regime Filters"; plus Investopedia on combining technical+fundamental), cross-checked against standard references (Grinold & Kahn *Active Portfolio Management* - the IC/IR framework; Asness/AQR on backtest overfitting; Lopez de Prado *Advances in Financial ML* - combinatorial purged CV, deflated Sharpe).

```
| Principle | Kairos today | Gap |
| Ideas come from **academic factors / documented edges** (SSRN, arXiv), not vibes | LLM invents "themes"
from 15 headlines | No factor basis; news-chasing |
| **Test the signal first** - rolling IC/IR stability, t-stat, judged at the *median* parameter (not the
best) | Whole *strategies* are walk-forward-validated, but individual signals are not IC-tested | No per-
signal stability gate |
| **Regime filter** as explicit scaler first, ranked on protection/cost/stability/**reach** (works
SPY+QQQ+India proxy, not one chart) | Implicit, score-threshold only; kill-switches are *risk* not *regime*
| No measured regime scaler yet |
| **Fees-first, worst-case** friction | Build 4a slip tracking + notional caps | Partial - decent |
| **Stack multiple edge categories** | Single LLM-score edge | One edge, not stacked |
| **Understand WHY (causal); never trade a black-box signal** | LLM *is* the signal (black box) | Inverts
the principle |
| **Judge by out-of-sample, not in-sample; expect overfit if you test many signals** | Shadow decisions +
validation exist | IC/deflated-Sharpe discipline not enforced at signal level |
```

The central inversion: the LLM should explain and orchestrate, not originate alpha. Alpha comes from validated, causal edges.

2. Design goals & non-goals

Goals

- Replace LLM-as-alpha with a deterministic edge library whose signals are reproducible and inspectable.

- Add an IC-stability gate so only signals with a real, stable, positive edge (after decay + fees) can influence normal sizing or live eligibility.

- Add a ranked regime filter as an explicit, market-portable size scaler first; hard blocking requires later evidence and owner sign-off.

- Keep the LLM for what it is good at: literature -> hypotheses, and results ->

plain-language explanation.

- Reuse the existing validation/evolution/calibration machinery.
- Preserve every money-safety invariant (Section 9).

Non-goals

- Not proposing HFT / intraday microstructure (data + latency out of scope).
- Not removing the LLM (it moves roles).
- Not a from-scratch backtester - extend the current Validation Engine.
- Not auto-promoting anything to live - owner click + gates remain.

3. Architecture overview

```

????????????????????????????????????????????????????????????
?  EDGE LIBRARY (deterministic formulas, versioned)  ?
?  factor ? technical ? calendar ? structural ? information  ?
????????????????????????????????????????????????????????????
      ? compute per (symbol, date)
      ?
????????????????????????????????????????????????????????????
?  SIGNAL STORE  edge_signals(symbol,date,edge_id,value,z)  ?
????????????????????????????????????????????????????????????
      ? rolling evaluation
      ?
????????????????????????????????????????????????????????????
?  IC/IR VALIDATION GATE  ?
?  rolling IC, IR, t-stat, half-life/decay, turnover, net-of-  ?
?  fee. Promotes edge -> {candidate|active|benched|retired}  ?
????????????????????????????????????????????????????????????
      ? active edges only
      ?
????????????????????????????????????????????????????????????
?  ENSEMBLE / COMPOSITE SCORE  (IC-weighted blend of active  ?
?  edges) -> feeds the EXISTING champion genome weights/sizing ?
????????????????????????????????????????????????????????????
      ? gated by
      ?
????????????????????????????????????????????????????????????
?  REGIME FILTER (ranked, per market) -> advisory/size scale  ?
????????????????????????????????????????????????????????????
      ?
existing pipeline: research signals -> paper-trade (freshness/claim) ->
PositionMonitor -> Learner -> Validation Engine (WFO, MC) -> promote

```

LLM (DeepSeek/Claude via agent_config) sits BESIDE this: proposes new edges from literature (goes into the library as `candidate`, never auto-active), and writes the human explanation of each decision. Never a signal value.

4. The Edge Library

A versioned catalog. Each edge is a pure function $f(\text{symbol}, \text{date}, \text{history}) \rightarrow \text{number plus metadata}$. Categories mirror the "60 edges" taxonomy:

- Factor (cross-sectional, academically documented): 12-1 momentum, value (E/P, FCF yield, B/P), quality (ROIC, gross-profitability, accruals), low-volatility, size, short-term reversal, earnings-revision/PEAD, investment/asset-growth.
- Technical / trend: distance to SMA/EMA/KAMA/WMA, breakout (n-day high),

RSI, MACD state, ATR-normalized momentum.

- Calendar / time: turn-of-month, day-of-week, holiday drift, seasonality,

FOMC drift - used as *conditioners*, not standalone.

- Structural: index-inclusion drift, ETF-flow pressure, sector rotation, post-earnings-announcement-drift window.

- Information (later phase; needs alt-data): insider-cluster buys, 13F flow,

news-sentiment surprise vs. estimate. (This is where the *current* LLM/news work is repurposed - as an *information edge input*, not the whole engine.)

Metadata per edge (drives the catalog table): edge_id, name, category, formula_spec (whitelisted grammar - reuse the existing Feature Registry grammar), inputs[], rationale (WHY it should predict returns), expected_sign, horizon_days, data_source, references[], status.

Reuse: the Feature Registry (migration ~065-era, whitelisted-grammar-only) already exists for exactly this "safe formula" purpose - extend it, don't reinvent.

5. Signal computation + store

Batch job (new cron or fold into research/cron) computes each active/candidate edge for the tradable universe daily, cross-sectionally z-scores it (so edges are comparable), and writes:

```
edge_signals(symbol, date, edge_id, raw_value, z_value, universe_id).
```

Cross-sectional standardization + winsorization is the standard pre-IC step. Neutralization (sector/beta) optional per edge (flag in metadata).

6. The IC/IR validation gate (the heart of it)

For each edge, on a rolling window, compute against forward returns $r_{\{t \rightarrow t+h\}}$ at the edge's horizon h:

- $IC_t = \text{Spearman rank-corr}(z_value_t, forward_return_{\{t \rightarrow t+h\}})$ per period.

(Rank IC is robust to outliers vs. Pearson.)

- Mean IC, IC volatility, IR = mean(IC)/std(IC) (Grinold's

fundamental law: IR ? IC ? ?breadth).

- t-stat of IC series vs. 0; require significance.

- Decay / half-life: IC by horizon (1,5,10,20d) - is the edge fresh or stale?

- Turnover implied by the signal -> fee drag (fees-first: net-of-cost IC).

- Stability: rolling IC must not flip sign / collapse (judge the *plateau*,

not the peak - the article's core point about not cherry-picking the best parameter).

Promotion state machine (per edge, per market): candidate -> measure_only -> shadow -> exploratory_paper -> active_paper -> live_eligible -> live_approved.

- candidate -> measure_only: deterministic formula compiles, inputs exist, and the economic rationale is explicit.

- measure_only -> shadow: point-in-time input availability is proven and the

signal has enough historical observations to compute rank-IC.

- shadow -> exploratory_paper: small positive gross IC, positive direction in

multiple windows, and no major data-quality defects. Allocation is tiny and capped; this is exploration, not proof.

- exploratory_paper -> active_paper: mean rank-IC $\geq \tau_{ic}$ (e.g. 0.02-0.03 for

daily equity factors), IR $\geq \tau_{ir}$, positive net-of-fee IC, stability over K windows, turnover/liquidity acceptable, and correlation to existing active edges < 0.8 .

- active_paper -> live_eligible: WFO/OOS + Monte Carlo pass, benchmark-relative

long-only top-bucket alpha positive after costs, and decay monitor clean.

- live_eligible -> live_approved: owner approval only.

Statistical hurdle:

- Academically-prioried/known factors may enter shadow or exploratory paper with

t-stat ≥ 2 when other gates pass.

- Newly discovered/data-mined factors need a stricter multiple-testing hurdle

before active/live eligibility: t-stat > 3 or explicit FDR/q-value control.

- Overlapping forward returns require autocorrelation-aware standard errors

(e.g. Newey-West) rather than naive IID t-stats.

active_paper -> benched when rolling IC/alpha decays below a floor for M windows. benched -> retired after prolonged failure. All transitions are logged and owner-visible.

This gate is what stops "test 100 signals, one looks amazing by luck" (the overfitting failure mode). Complement with deflated Sharpe / multiple-testing correction (Lopez de Prado) on the number of edges trialed.

7. Ensemble -> existing genome

active edges are blended into a composite cross-sectional score per symbol, weighted by each edge's recent IC/IR (shrunk toward equal-weight to avoid over-fitting the weights - this is where the existing Learner + genome weights plug in: the genome already carries per-dimension weights and the learner already mutates them evidence-bound). The composite replaces today's "5 soft LLM dimensions" as the score that drives entry threshold + Kelly sizing (which already read the champion genome - see genome-live.ts).

Net effect: minimal change downstream. research_agent emits the composite score into agent_signals; everything after (paper-trade freshness/claim, monitor, learner, validation) is untouched.

8. Regime filter (ranked, portable, scaler first)

A dedicated module, evaluated per market, that outputs {state: risk_on | risk_off | neutral, scale $\in [0, 1]$ }. In the first implementation this is advisory/measure-only, then a continuous size scaler. It must not become a hard new-entry block until the scaler has measured protection/cost evidence and the owner explicitly approves the harder behavior. Exits always remain allowed.

Candidate filters (implement several, rank them, pick per the article's rubric): SMA(100-300) above/below, EMA, KAMA (adaptive - caught COVID in the study), WMA, Hull, TEMA, n-day-high freshness, higher-lows, trend-quality (R² of price vs. time), price-within-x%-of-high. Rank each on:

- Deal - MAR / return-vs-drawdown improvement over buy-and-hold.

- Crash coverage - % of each bear (dot-com, 2008, COVID, 2022) avoided.
- Cost - annual return given up (the insurance premium).
- Reach - must work on SPY and QQQ and (crypto/India proxy), scored

at the median parameter across its whole range, not the best.

Chosen filter (+ runner-ups as an ensemble vote) becomes the regime scaler. Stored in `regime_filters` (definition + rolling scorecard) and `regime_state` (current per market). This augments the implicit score-only regime and the `macro_regime` table's advisory-only role (macro stays as a **slow** overlay).

Push-back note (CLAUDE.md): the repo's locked decision says **no explicit bull/bear switching** - "scoring should adapt." This proposal re-opens that decision deliberately, because the regime-filter literature is strong and the current implicit approach has no measured protection/cost. Flag for owner: this is a conscious contradiction of an existing locked rule and needs explicit sign-off. The approved first cut honors the old rule: keep regime as measure/advisory first, then a **sizing scaler** only. Never hard on/off without a new owner approval.

9. Money-safety invariants (unchanged, must hold)

- Additive migrations only; verify applied before schema-coupled code ships.
- No autonomous live trading; owner click + `requireOwner` on every live order.
- No LLM places/cancels live orders, mutates money limits, mutates the active

live strategy, or approves its own promotion. Here the LLM is even **further** from execution (it only proposes catalog entries as candidate).

- Append-only ledgers (`paper_trades`, `paper_order_events`) - inserts only.
- US(USD)/India(INR) money limits stay currency-separated.
- Long-only new positions; SELL only if held.
- Nothing goes active/live without passing IC gate and the existing

fail-closed Validation Engine (WFO OOS + Monte Carlo) and owner promotion.

10. Data model (new, additive)

- ``edge_catalog(edge_id, name, category, formula_spec, inputs, rationale, expected_sign, horizon_days, data_source, references, status, created_at)``

- `edge_signals(symbol, date, edge_id, market, raw_value, z_value, universe_id)`
(partitioned/indexed by date+market; this is the big table)

- ``edge_universe_members(universe_id, market, symbol, as_of_date, source, included_reason, created_at)`` - records exactly which symbols were eligible for a universe/date so IC results are auditable and survivorship bias is visible.

- ``edge_signal_inputs(edge_signal_id, input_name, source, as_of_date, available_at, revised_at, adjustment_policy, raw_ref)`` - mandatory audit trail for point-in-time validation. No edge may be promoted if its inputs cannot prove when the app could have known the value.

- ``edge_ic_history(edge_id, market, window_end, ic, ic_ir, t_stat, horizon, net_of_fee_ic, turnover, status_after)``

- `regime_filters(filter_id, definition, params, scorecard jsonb, active bool)`
- `regime_state(market, date, state, scale, filter_id, detail)`

All read-heavy analytics; no changes to money tables.

11. Integration points (existing files, indicative)

- `lib/research-agent.ts` - replace 5-dim soft score with the composite

edge score; keep LLM call only for the written thesis/explanation. (Path verified: it is `lib/research-agent.ts`, not `lib/agents/research-agent.ts`.)

- Feature Registry / whitelisted grammar - `lib/validation/feature-compiler.ts`

(verified) hosts the safe-formula grammar; extend it for edge formulas.

- `lib/validation/engine.ts` (Validation Engine, verified) + `app/api/agents/backtest/*`

- add IC gate + deflated-Sharpe; extend WFO to the edge composite.

- `lib/validation/genome-live.ts` + Learner - consume IC-weighted edge blend as the weights it tunes (evidence-bound mutation already exists).

- `app/api/agents/paper-trade/route.ts` - unchanged (freshness/claim/sizing all reused); it just fills better-founded signals, now also gated by `regime_state`.

- Macro Sentinel (`macro_regime`) - demoted to slow overlay beneath the fast regime filter.

- Dashboard - new read-only panels: edge catalog + IC scorecards, regime state.

12. Rollout phases (each independently shippable + measurable)

Current reality (2026-07-18): P0 and P1 exist, but the first broad run correctly collapsed the initial watchlist-only IC results to approximately zero. No edge is shadow-eligible in production. Migration `20260718140000_edge_evidence_collection_hardening` restarts bounded prospective US/India collection, stores sample/provenance metadata, and moves lifecycle state from the misleading `global_edge_catalog.status` to `edge_market_status`. Historical IC remains labeled `retrospective_current_universe`; it cannot authorize P2.

- P0 - Edge library + signal store (measure-only). Implement 6-8

price/volume technical edges only, compute daily, store `edge_signals`, and record the exact `edge_universe_members` snapshot. No effect on trading.

- P1 - IC gate + dashboard. Compute rolling IC/IR/t-stat; classify edges;

surface scorecards. Still measure-only. This alone tells us whether ANY current idea has real edge.

- P2 - Composite score in SHADOW. Blend active edges; run alongside the live

LLM/current score via the existing `shadow_decisions` machinery (no fills). Compare current score vs. factor score vs. blended score on live, contemporaneous opportunities.

- P3 - Exploratory paper. Candidate/known-prior edges that clear data-quality

and early IC checks may receive tiny capped paper allocation. This is explicit exploration, not proof, and is excluded from live eligibility until later gates pass.

- P4 - Regime filter (measure + rank). Implement filters, score them on history, and run as an advisory/continuous size scaler first. Hard blocking is a later owner decision after evidence.
 - P5 - Active paper. Validated composite score + regime scaler drive the PAPER book. LLM is demoted to explanation. Watch calibration + Performance-Truth for several weeks.
 - P6 - Live (owner-gated, unchanged safety). Only after WFO OOS + Monte Carlo
 - long-only benchmark-relative alpha + owner promotion, and only for edges that stayed active_paper through the soak.
- Each phase reuses existing infra (shadow, validation, calibration, genome), so risk is incremental and reversible.

P1 evidence-hardening gate before P2

P2 stays blocked until all of the following are true per market:

- prospective daily captures have real observed_at timestamps and immutable input fingerprints; legacy synthetic next-session timestamps are labeled legacy_unverified;
- n_obs, universe size, sampled dates, provider coverage, and evidence quality are persisted and visible;
- lifecycle status is market-scoped; US and India cannot overwrite each other;
- at least 60 prospective trading sessions exist (120 preferred for 20-day horizons), and historical results are not represented as point-in-time proof;
- turnover and cost-adjusted long-only bucket alpha are populated before any paper-capital phase;
- Fibonacci, if added, is measure-only and must beat non-Fibonacci placebo levels.

13. Risks / failure modes to critique

- Small-sample IC noise: with a tiny paper history, IC estimates are unstable.
Mitigation: compute IC on a broad historical universe (not just our filled trades) via the price-history backfill; treat live paper as confirmation only.
- Multiple-testing / overfitting the catalog: trialing many edges inflates false positives. Mitigation: deflated Sharpe, hold-out, cap the number of simultaneously-active edges, prefer academically-prioried edges.
- Regime filter = one more overfit dial: the article shows band/adaptive filters wobble. Mitigation: judge at median parameter + require cross-market reach; prefer the simplest filter that clears the bar.
- Data coverage for India: factor inputs on NSE via Yahoo/Kite are thinner - IC gate may bench most edges there initially (acceptable; fail-closed).
- Turnover/fees eat factor edges: enforce net-of-fee IC from the start (fees-first).
- Contradicts the locked "no explicit regime" decision - see ?8; needs owner

sign-off.

14. Open questions for reviewers (LLMs / quants)

- Is rank-IC + IR + t-stat + deflated-Sharpe a sufficient signal gate, or should we add combinatorial purged cross-validation (CPCV) from day one?
- IC thresholds: what τ_{ic} / τ_{ir} are defensible for daily US equity factors on a ~1-3k name universe? (We proposed $IC \geq 0.02-0.03$, $IR \geq -0.3$ - too lax/strict?)
- Regime as hard on/off vs continuous sizing scaler - which is more robust given it will also gate India with thin data?
- Ensemble weighting: IC-weighted vs. equal-weight vs. risk-parity across edges - which best resists weight overfitting at our scale?
- Where exactly should the LLM stay useful - only literature->hypothesis and explanation, or also as a *conditioner* (e.g. news-surprise as one information edge feeding the IC gate like any other)?
- Minimum history before an edge may go active (calendar time vs. number of independent IC observations)?
- Anything structurally missing vs. how real agentic-quant platforms (e.g. QuantConnect + factor libraries, WorldQuant-style alpha pools) are built?

15. References

- Grinold & Kahn, *Active Portfolio Management* - IC/IR, fundamental law of active management (IR $\propto$ IC²/breadth).
- Lopez de Prado, *Advances in Financial Machine Learning* - deflated Sharpe, purged/combinatorial CV, backtest overfitting.
- Asness / AQR - factor premia, implementation shortfall, overfitting warnings.
- Owner-supplied: SetupAlpha "Scientific Workflow 2026", "60+ Market Edges 2026", "20 Trend-Based Regime Filters" (Medium); Investopedia "Blend Technical and Fundamental Analysis".
- Existing Kairos infra this builds on: Feature Registry (whitelisted grammar), Validation Engine (WFO, fail-closed), shadow_decisions, strategy genome + Learner (evidence-bound), calibration, Build 4a slip tracking.
- Harvey, Liu & Zhu, "...and the Cross-Section of Expected Returns" - new/data-mined factors need higher multiple-testing hurdles; t-stat > 3 is a useful rule of thumb for newly discovered factors.
- Bailey & Lopez de Prado, "The Deflated Sharpe Ratio" - selection bias and multiple testing inflate backtest Sharpe; use DSR/PSR-style correction.
- Benhenda, "Look-Ahead-Bench" (2026) - point-in-time LLM finance workflows must explicitly test for look-ahead bias and decay across temporally distinct market regimes.

- Azevedo, Hoegner & Velikov, "The Expected Returns on Machine-Learning Strategies" - ML/anomaly strategies can work after costs, but only with careful treatment of transaction costs, post-publication decay, and stale historical data.

- Baldi-Lanfranchi, "Transaction-cost-aware Factors" (2024) - factor construction should optimize the trade-off between exposure and rebalancing costs; especially relevant for high-turnover edges like momentum.

- Research Affiliates / Robeco / Man Group factor implementation notes - practical factor investing must account for turnover, liquidity, crowding, risk management, and investability, not just gross historical returns.

No implementation until owner approves + reviewers vet. Theme Scout has been reverted to a plain news-driven scout in the interim (it is not the alpha source).

Reviewer changelog (ChatGPT, 2026-07-08)

- Updated author line to record ChatGPT/Codex methodology review.
- Corrected the TL;DR: IC validation is required for normal sizing/live eligibility, but tiny capped exploratory paper is allowed so the system can learn.
- Corrected the regime design: start with continuous size scaling; hard on/off is deferred until evidence and owner approval.
- Added Section 0A with the intended separation: ThemeScout = attention discovery; EdgeScout/FactorScout = deterministic alpha factory.
- Replaced the simple candidate -> active promotion with a full lifecycle: candidate -> measure_only -> shadow -> exploratory_paper -> active_paper -> live_eligible -> live_approved.
- Added stricter statistical gates for newly discovered/data-mined edges: $t\text{-stat} > 3$ or FDR/q-value control before active/live eligibility.
- Added autocorrelation-aware standard error requirement for overlapping forward returns.
- Added mandatory point-in-time input audit table `edge_signal_inputs`.
- Updated rollout to include exploratory paper and active paper as separate phases.
- Added recent/current references through 2026 covering look-ahead bias, transaction-cost-aware factors, ML strategy implementability, and multiple testing.

event aware fundamentals adr - Feature Architecture

Source: `features\event-aware-fundamentals-adr\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Event-Aware Fundamentals and Exchange-Listed ADRs

Status: Approved for implementation by owner on 2026-07-31

Decision

Kairos will reuse company-reported facts across agents through one event-aware freshness policy and will treat reviewed US exchange-listed ADRs as a distinct instrument kind. ADRs may be researched and paper-traded in the US/USD book. Live compatibility uses every existing pause, kill-switch, account, portfolio, broker-review, and order-acknowledgement gate; this feature does not enable live trading or assert that a broker supports a symbol.

Why

ResearchAgent and ThemeScout currently share same-day provider cache rows, but the active Finnhub metric/profile path normally polls again the next UTC day. Reported margins and revenue do not require daily polling outside an event window, while valuation and earnings dates do. Treating the whole provider response as one freshness class either wastes calls or serves stale valuation.

ADRs also cannot be treated as ordinary domestic issuers. Foreign private issuers generally do not provide Section 16 Form 4 evidence, and some providers resolve an ADR symbol to the foreign underlying. A live 2026-07-31 probe for SKHY returned Korean-underlying EPS from Finnhub but USD ADS-basis fields from Yahoo. Mixing those units can corrupt valuation and comparability.

Verified SK hynix Identity

- Current US instrument: SKHY, Nasdaq Global Select Market.
- Temporary when-issued symbol: SKHYV, retired after 2026-07-10.
- Each SKHY ADS represents one-tenth of a KRX 000660 common share.
- Legacy HXSCL is a restricted/limited OTC GDR and is not a substitute.
- Sources: Nasdaq Trader Alert 2026-37, SEC Form F-1/424(b)(4), Citi DR record.

Only SKHY is added to the reviewed ADR registry. OTC/foreign-exchange symbols remain excluded from automatic discovery and live execution.

Freshness Contract

One batch lookup of `earnings_calendar` annotates symbols before provider work:

```
| State | Reported-fact freshness |
| Report date within the prior 3 days or next 14 days | 1 day |
| No near event, but a calendar record exists | 7 days |
| Earnings date unknown | 7 days, fail conservatively |
```

Issuer profile/classification uses 30 days. Price bars remain completed-session daily data. Earnings dates remain daily inside the existing calendar path. Because the legacy Finnhub metric response mixes reported and valuation fields, its maximum freshness is seven days rather than Router's future 14-day reported contract. The Router may later split `fundamentals.reported` (14 days) from `fundamentals.valuation` (3 days) after parity approval.

An event-window refresh does not change scores by itself. It only makes the next normal scoring pass fetch current facts. Provider failures continue to return bounded cached data or unavailable; they never fabricate a fresh observation.

Agent Boundaries

- ResearchAgent is the only authoritative scorer and receives the annotated TTL.
- ThemeScout validates ticker existence with a quote/instrument lookup, not a complete fundamental fetch. Its candidates are still scored later by ResearchAgent.
- Prewarm uses the same annotated symbol entries and cache rules.
- Router shadow collection remains separate and cannot affect scores or orders.
- Learner records `asset_class='adr'`, preventing ADR evidence from being silently pooled as a domestic issuer when segment diagnostics become available.

ADR Scoring Contract

- Market/currency: US / USD, never cross-summed with the India/INR book.
- Technical, sentiment, US macro, options, and analyst applicability match a US-listed equity when the source supports the symbol.
- Fundamental source order: ADS-aware Yahoo first, then only compatible fallbacks.
- Insider/Form 4: structurally not applicable and excluded from weight renormalization, not assigned a neutral score.
- New ADRs require a reviewed registry entry or a future verified instrument classifier. Symbol suffix guessing is forbidden.

Trading Contract

- PaperTrader accepts `adr` as a US asset class and uses USD/fractional paper sizing.
- Live fund-concentration classification treats `adr` as an equity, not an ETF.
- The Robinhood MCP `review_equity_order` remains mandatory before placement.
- A broker rejection or unsupported symbol fails closed. No fallback to an OTC symbol, foreign listing, or alternate broker occurs automatically.
- OTC instruments require a separate design: whole-share sizing, limit-day-only orders, liquidity/spread gates, market-tier and disclosure checks, and explicit broker support. They are out of scope here.

Acceptance Criteria

- SKHY is classified as `adr`, enters US research from the watchlist, and persists `asset_class='adr'`.
- SKHY fundamentals prefer Yahoo ADS-basis values; Finnhub's Korean-underlying profile cannot win the ADR chain.
- ADR insider evidence is inapplicable and consumes no insider provider call.

- An eligible long SKHY signal can enter the paper selection/fill path without a special-case rejection.
- Live classification recognizes ADR as non-fund equity but every existing live and broker review gate remains in force.
- ThemeScout ticker validation consumes no full-fundamental call.
- Near-earnings symbols use one-day reported freshness; ordinary symbols use seven days; profiles use thirty days.
- Tests, architecture chapters, system map, and research diagram agree.

Reversal

Remove SKHY from the reviewed registry and restore one-day freshness constants. No trade, historical signal, or immutable evidence row is rewritten.

exogenous risk evidence - Feature Architecture

Source: features\exogenous-risk-evidence\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Exogenous Risk Evidence Layer

Status: DESIGN DRAFT. NOT APPROVED FOR IMPLEMENTATION.

Date: 2026-07-26

Owner decision required: approve the phased, observation-first build before code

or migrations begin.

1. Decision

Build one canonical, point-in-time evidence layer for facts that affect markets but do not belong to a company's fundamental, technical, insider, or ordinary news-sentiment score.

The layer has three deliberately separate classes:

- Domestic regimes - US macro/Fed and India macro/RBI.
- Global spillovers - Fed impact on India, global oil, USD/INR, and

global risk conditions.

- Discrete shocks - tariffs, sanctions, supply disruptions, and conflicts.

The first releases are record-only and shadow-only. They cannot alter a score, direction, position size, paper trade, live trade, stop, target, or broker order. Admission to a money path requires separately measured, point-in-time, market-and-sector-specific incremental value.

2. Why This Is Needed

Kairos currently has real but fragmented coverage:

```
| Input | Current state | Problem |
| US macro / Fed | US-only FRED MacroSentinel; FOMC ledger is record-only | Correct for the US, but cannot be reused as an India domestic regime. |
| India FII/DII | NSE factual line in the India thesis | Useful daily flow context, but not a regime and not scoreable alone. |
| Oil, gold, crypto products | Some ETFs/watchlist symbols and Markets display tiles | A visible instrument is not a validated factor or a symbol-exposure model. |
| Tariff and conflict news | May appear in per-symbol sentiment/LLM prose | No authoritative event identity, effective time, exposure mapping, or auditable impact record. |
| India macro | Explicitly unavailable in scoring | This is safer than borrowing the US regime, but leaves a genuine coverage gap. |
```

The bad alternative is a universal "risk score" that mixes stale CPI, a same-day oil move, an LLM's interpretation of a conflict, and US Fed policy, then applies it to every US and Indian symbol. That would be non-reproducible and would make the earlier India-US macro leakage possible again.

3. Invariants

- No cross-market substitution. India never reads a US domestic regime; the

US never reads an India domestic regime. Global observations are explicitly tagged `scope=global_spillover` and have separate US/India transmission rules.

- Point in time. Every observation records `observed_period`, `published_at`, `available_at`, source URL, source revision/vintage when available, and raw-payload fingerprint. A later revision never rewrites a value knowable at an earlier decision time.
- No fabricated neutral. Missing, stale, partial, or contradictory evidence is `unavailable`, not a score of 50.
- Deterministic ingestion and math. No LLM classifies an event, chooses a sector, computes a regime, or changes an exposure. LLMs may only summarize already persisted evidence for display.
- Off the money path until proven. Research, paper, live, exits, sizing, allocation, and brokers do not import this layer in P0-P3.
- Append-only evidence. Corrections append a new observation/supersession; they do not rewrite historical facts or impact calculations.
- Market-local accounting. US results remain USD/SPY-or-VOO-relative; India results remain INR/NIFTY-relative. They are never cross-summed.

4. Source Policy

4.1 Approved primary sources

Evidence	US	India	Cadence	Initial role
Domestic policy rate / decision	Existing FRED target range + FOMC ledger	RBI MPC decision and RBI DBIE policy-rate series	Event / daily	Domestic regime evidence
Inflation / activity	Existing FRED series	MoSPI CPI and IIP releases	Monthly	Domestic regime evidence
Rates / financial conditions	Existing Treasury/FRED series	RBI DBIE 10Y G-Sec, policy corridor, liquidity, INR/USD	Daily/monthly	Domestic regime evidence
Portfolio flows	Existing US risk indicators only	NSE FII/DII; SEBI/NSDL reference fallback	Daily	India flow overlay
Oil	EIA Brent/WTI	PPAC Indian Basket, with EIA Brent as global comparator	Daily/weekly	Global spillover
Official trade actions	USTR / Federal Register	Indian Ministry of Commerce / DGFT notices when a machine-readable, timestamped source is proven	Event	Discrete-event ledger

The primary India authority is RBI DBIE for monetary/financial series and MoSPI for CPI/IIP. NSE FII/DII is not promoted into a domestic regime by itself.

4.2 Explicitly deferred or prohibited sources

- Do not scrape CME FedWatch, news pages, social posts, or an RBI page to invent a market-implied policy expectation. A future expectation adapter needs a lawful, timestamped, pre-decision source.
- Do not use Yahoo, TradingView, a crypto exchange, or a chart tile as the authoritative macro source. They may be display/fallback sources only after a separate source-policy approval.
- Do not use a general LLM, GDELT tone, or arbitrary headlines to produce a tariff/war severity score.
- Gold:silver ratio, crypto breadth, and generic geopolitical sentiment are not P0 inputs. They are hypothesis candidates only after the core evidence layer works.

5. Canonical Data Contracts

The implementation must extend the existing evidence/provenance conventions, not create a disconnected second audit system. It should reference existing `evidence_records`, `provider_call_ledger`, `evidence_cache_v2`, `frozen_symbol_daily_returns`, and the `US policy_rate_events` ledger by stable IDs or fingerprints.

5.1 exogenous_observations

Append-only raw facts, one record per source observation/vintage:

```
type ExogenousObservation = {
  id: string;
  market: "us" | "india" | "global";
  scope: "domestic" | "global_spillover";
  seriesKey: string; // e.g. india.cpi_combined_yoy, global.brent_usd
  value: number | null;
  unit: string;
  observedPeriod: string;
  publishedAt: string;
  availableAt: string;
  source: string;
  sourceUrl: string;
  sourceRevision: string | null;
  payloadFingerprint: string;
  quality: "fresh" | "stale" | "partial" | "unavailable";
};
```

No data consumer receives a naked number without its availability and as-of metadata. Source cache keys must include the source, series, market, period, and revision identity.

5.2 market_regime_runs

Immutable output of a versioned deterministic regime function. It must have a `market` column from day one. This is the eventual replacement for reading the unmarketed legacy `US macro_regime` table; it does not modify that table in the first phase.

```
type MarketRegimeRun = {
  id: string;
  market: "us" | "india";
  domesticState: "supportive" | "neutral" | "adverse" | "unavailable";
  globalSpilloverState: "supportive" | "neutral" | "adverse" | "unavailable";
  formulaVersion: string;
  inputFingerprint: string;
  computedAt: string;
  eligibleInputCount: number;
};
```

Domestic and global states are separate fields. They must never be added into a single value without a validated formula version.

5.3 exogenous_events and event_impact_observations

An event carries an authoritative source, effective time, affected jurisdictions, and a bounded taxonomy (`policy_rate`, `tariff`, `sanction`, `supply_disruption`). It does not store a model-generated severity or inferred symbol list. A separate, versioned exposure map may connect a verified event class to sectors or instruments. Impact rows calculate 1/5/20 session raw and same-market benchmark excess returns only from frozen return evidence.

Existing FOMC tables remain the authoritative US policy implementation. A later adapter links them to this common contract rather than copying or rewriting them.

6. Transmission Maps

There is no valid all-symbol response to oil, Fed, tariffs, or war. Every future mapping must be versioned, deterministic, and conservative:

Driver	US examples	India examples	Rule before admission
Fed / US yields	long-duration technology, banks, REITs	FPI-sensitive financials, INR-sensitive importers/exporters	Measure separately by market and sector.
Oil rise	energy producers/refiners may benefit; airlines may suffer	upstream producers may benefit; airlines, chemicals, OMCs may suffer	Require the instrument registry/exposure tags; unknown exposure means no effect.
INR depreciation	US domestic impact usually none	exporters and import-dependent firms differ	Use RBI exchange evidence and a reviewed tag, never a ticker-name heuristic.
Tariff	importer/exporter and supply-chain effects differ	India exporter/importer effects differ	Only an explicit, reviewed exposure link can produce a hypothesis.
Crypto move	crypto ETFs, miners, exchanges	no general India equity effect assumed	Apply only to explicitly classified crypto exposures.
Gold:silver ratio	no default equity implication	no default India equity implication	Research-only until an IC study clears.

7. Phased Build Order

P0 - Truth and observability (2-3 engineering days)

- Publish a source capability matrix: authority, cadence, expected delay, allowed use, fallback, staleness limit, and API quota/cost.
- Add System Health checks for source freshness and observation coverage, not one alert per missing symbol.
- Audit and relabel existing Markets tiles as display-only where appropriate.
- Remove the now-harmless macro-read-india no-op cron after verifying no consumer depends on it.

Acceptance: the UI and health surface distinguish not built, unavailable, stale, and display-only; no money-path import changes.

P1 - India domestic macro shadow (5-7 engineering days)

- Implement server-only official adapters for RBI DBIE and MoSPI. Start with repo rate, 10Y G-Sec, INR/USD, CPI Combined YoY, and IIP growth.
- Persist release vintages in `exogenous_observations`; capture source release time rather than using fetch time as a fake release time.
- Add daily frozen NSE FII/DII snapshots with source date, bounded stale policy, and no accidental reuse of the `gdeLt` provider identity.
- Compute an India domestic regime in `market_regime_runs`, shadow-only.
- Compare regime outputs to NIFTY/sector outcomes by released-data time, with no score or trade effect.

Acceptance: an India run can be reproduced using only observations available at that run's `computedAt`; a failed source yields `unavailable`, not `calm`; the US path cannot query India records and vice versa.

P2 - Global spillover evidence for both markets (4-6 engineering days)

- Link the existing FOMC outcome ledger to the evidence contract; do not duplicate its event or impact records.

- Add Fed-to-India as a separate global-spillover observation, never an India domestic-policy observation.

- Add Brent/WTI from EIA for the US and the PPAC Indian Basket for India.

- Add a versioned, read-only oil/FX/FPI exposure hypothesis map that defaults to no mapping for unknown symbols.

- Record per-sector, per-market impact observations from frozen returns.

Acceptance: a Fed shock can be shown as global context for India while the India domestic regime remains unchanged; all displayed effects state their source and as-of time.

P3 - Discrete-event ledger (5-8 engineering days)

- Implement an event ledger for a small authoritative taxonomy only.

- Ingest USTR/Federal Register trade actions and a separately validated Indian official trade-notice source. Start with policy/tariff events, not wars.

- Permit an owner-reviewed event record for conflicts/supply disruption when an official source and effective timestamp exist. The application must not auto-infer an event from news sentiment.

- Record impacts and candidate sector hypotheses; display them as observation, not trade recommendations.

Acceptance: every event has a source URL and effective/known-at time; a missing exposure link cannot suppress or promote a symbol.

P4 - Hypothesis evaluation (minimum 8-12 weeks of new evidence; no fixed calendar promise)

- Use the existing point-in-time replay, Edge IC, FDR, and walk-forward infrastructure to test each *one* hypothesis separately.

- Require adequate event count, market/sector sample floor, post-cost result, stability across windows, and no worse drawdown/turnover than the baseline.

- Promote at most one validated, narrow use at a time: initially a subtractive

Long -> neutral eligibility guard in paper shadow. Never promote an event feature that directly creates a buy, exits a holding, or widens risk limits.

- Require an owner sign-off and a reversible feature flag before paper action; live remains disabled pending a separate approval.

The key constraint is data time, not coding time. CPI/IIP and RBI decisions are monthly/event-driven, so reliable validation cannot be compressed into a week.

8. Explicit Deferrals

- Gold:silver ratio: add only as a shadow Edge-lab factor after P2. It has no default portfolio action because its equity relationship is regime dependent.
- Crypto macro factor: add only for instruments classified as crypto exposure; a broad US/India equity penalty is unsupported.
- War score: do not build one. Record verified events and their observed impacts; the product must not pretend it can quantify an unfolding conflict.
- Policy expectations: retain unavailable until a lawful pre-decision feed is selected. Official outcomes cannot reconstruct prior market expectations.
- Live usage: completely out of scope.

9. Safety, Security, and Rollback

- All ingestion routes are server/cron-only and owner-triggerable; browser code reads a redacted, owner-authenticated projection only.
- New evidence and impact tables use RLS deny-by-default, no anon grants, service-role-only writers, append-only triggers, indexes on (market, series_key, available_at) and event/impact keys.
- Raw payloads are bounded, schema-validated, and fingerprinted; no cookies, headers, tokens, or arbitrary fetched HTML enter the database or UI.
- Every phase has a feature flag whose disabled state leaves today's US macro, India scoring, paper, live, and exit behavior byte-for-byte unchanged.
- Rollback disables readers/jobs; it never deletes evidence or rewrites past signals/trades.

10. Validation Plan

- Adapter fixtures: normal release, revision, late release, malformed payload, source throttle, stale fallback, duplicate fingerprint, and unavailable.
- Boundary tests: US consumers cannot access India domestic observations; India cannot access US domestic observations; global observations cannot be mislabeled as domestic.
- PIT replay tests: a value published after as_of is rejected even when its observation period is earlier.
- Falsification tests: unknown symbol exposure, absent benchmark, missing expected policy rate, incomplete return horizon, and missing source must all leave scores/directions/orders unchanged.
- Production proof before any promotion: write fresh source rows, then verify their as-of metadata, regime run fingerprints, market isolation, and a UI projection with no provider URL/token leakage.

11. Decision Record

- Why: The user needs auditable macro and event context across both books without mixing USD/INR, importing unreliable news judgement, or spending research-provider quota.
- Who: Vaibhav operating separate US and India paper/live portfolios.
- ROI: Better diagnosis and future evidence-backed risk gating; no promised alpha until the layer passes the existing validation process.
- Build decision: GO for P0-P3 record/shadow architecture; DEFER all score, paper, live, exit, gold/silver, crypto, and war-score admission.

12. References

- RBI DBIE: <https://dbieold.rbi.org.in/DBIE/>
- MoSPI release calendar: https://mospi.gov.in/sites/default/files/Advance_Release_Calendar.pdf
- SEBI FPI source index: <https://www.sebi.gov.in/curation/fpi.html>
- RBI MPC schedule: <https://www.rbi.org.in/scripts/PublicationsView.aspx?id=23139>
- EIA oil data: https://www.eia.gov/dnav/pet/pet_pri_spt_s1_w.htm
- PPAC Indian Basket: <https://ppac.gov.in/index.php/prices/international-prices-of-crude-oil>
- USTR Section 301 source: <https://ustr.gov/issue-areas/enforcement/section-301-investigations>

external research integrations - Feature Architecture

Source: features\external-research-integrations\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Governed External Research Integrations

Status: REVISED DRAFT FOR CLAUDE + OWNER REVIEW. DESIGN ONLY.

Date: 2026-07-13

Owner decision requested: approve an extensible but zero-trust way to add

selected GitHub projects, libraries, and skills to Kairos without granting

them broker, production database, or money-path authority.

Build gate: no implementation, dependency installation, migration, provider

activation, credential issue, deployment, or order-path change until this

document is reviewed and Vaibhav says Proceed/Code it.

Document Authority

This file is the only implementation authority for this feature. Before a build starts, its approved version must contain the precise data contract, migration plan, rollout and acceptance criteria. The catalog, Vibe deep-dive, and deep candidate audit in this directory are evidence-only intake records: they explain why a capability was accepted, deferred, or rejected, but never override this architecture or authorize implementation.

The first native capability remains blocked until Evidence Router shadow parity and the entry-eligibility-flip guard are complete. It must extend existing Kairos records, including `experiment_runs` and immutable `evidence_records`; it must not introduce another provenance truth layer.

1. Decision

Kairos remains the system of record, the Performance Truth Layer, and the sole money path. External repositories and skills are never installed into the Next.js application as trusted agents. They are admitted one at a time as version-pinned, capability-scoped Research Integrations.

This document does not approve a Quant Research Worker now. The immediate work is the Canonical Evidence Router Phase 2 foundation, followed by narrowly scoped in-repository indicator hygiene where it has proven value. The worker framework is reserved for the first genuinely untrusted or LLM-driven external integration, such as a Vibe-derived advisory workflow.

When that future worker is justified, it receives immutable router evidence envelopes plus a Kairos universe snapshot; it may calculate approved research artifacts and returns schema-validated results through a narrow Kairos gateway. It cannot read broker credentials, submit/preview/cancel an order, change strategy configuration, mutate ledger rows, or call production Supabase with a service-role key.

An Integration Registry controls whether a capability is disabled, in contract test, research-only, shadow-only, or retired. There is no execution-capable state for a third-party integration.

This gives Kairos a durable answer to both questions:

- New useful repositories can be added later without rewriting agents.

- A repository update, compromise, deletion, license change, or provider outage cannot silently affect a real account or an existing decision.

2. Why This Is Better Than "Just Use Skills"

Skills and open-source projects are useful accelerators, but they are not a durable trading product boundary. They can change dependencies, prompts, network behavior, data sources, licenses, or security posture without notice. Kairos has requirements that a generic skill does not own:

- US/USD and India/INR pools are never cross-summed.
- Paper and live books are distinct, with live approval explicit by default.
- A score, evidence snapshot, candidate strategy, fill, and outcome are independently auditable.
- No LLM is permitted on a deterministic decision, sizing, or execution gate.
- Provider quotes used for execution remain separate from research evidence.

The right use of a skill is therefore: generate or test a hypothesis inside a constrained research workflow, not decide or execute a trade.

3. Goals And Non-Goals

Goals

- Add a new library, skill, or permitted fork through a repeatable review and release process.
- Make useful capabilities portable: indicator calculation, hypothesis generation, adversarial research, backtesting, document extraction, and validation reports.
- Make every output reproducible from its pinned integration version, input snapshot, policy version, and strategy version.
- Reuse and extend the existing `decision_observations`, `strategy_versions`, `experiment_runs`, immutable `evidence_records`, validation, Performance Truth, benchmark-alpha, and evidence-ledger architecture rather than create a parallel learning/trading system.
- Make disable and rollback immediate and harmless to existing Kairos flows.

Non-goals

- No external repository is a broker, order router, portfolio source of truth, or direct Supabase client.
- No arbitrary GitHub URL, package name, container image, prompt, provider URL, MCP tool, or command may be entered from Settings.
- No self-updating integrations, floating `latest` tags, or remote code fetch at runtime.
- No third-party code is copied before a license and provenance review.

- No model training, autonomous live trading, or new data-provider entitlement

is enabled by this feature.

- No worker, job-token issuer, worker table, or Settings marketplace is built merely to calculate indicators Kairos already obtains or can compute as pure, tested TypeScript.
- No OpenBB code/service is used in the initial plan. Its current AGPLv3 license requires a separate legal/product decision.

4. Architecture Principles And Invariants

- Kairos owns orders. Only Kairos-owned, reviewed broker adapters can reach a broker. The existing gate sequence is repeated immediately before submit.
- Read-only by construction. Worker input is a snapshot; its write path is limited to an append-only integration run/artifact ledger.
- No credentials cross the boundary. A worker never receives broker tokens, OAuth cookies, CRON_SECRET, Supabase service-role credentials, user browser cookies, or raw vault material.
- No network by default. A research worker uses supplied snapshots. Any approved outbound provider access is an explicit, domain-allowlisted adapter owned by the Canonical Evidence Router, never a repository-defined URL.
- Determinism on the money path. LLM and skill output is advisory text or a structured proposal. It cannot set scores, direction, sizing, a gate result, or an execution action.
- Schema before prose. All cross-boundary input and output is validated against a versioned JSON schema. Free-form repository/provider text is never interpolated directly into an LLM prompt or a trading decision.
- Market isolation. Every snapshot, run, artifact, experiment, and result has exactly one market and native currency. No aggregate result can mix us/USD and india/INR.
- Point-in-time fidelity. Inputs carry `as_of`, `retrieved_at`, source provenance, price adjustment basis, and policy version. The worker cannot query a newer fact when replaying a historical date.
- Failures are abstentions. Timeout, invalid output, quota exhaustion, missing price, unexpected network attempt, or policy mismatch returns an unavailable result. It never manufactures neutral evidence or a pass.
- Versioned and reversible. Every run records its exact build digest and configuration digest. Disabling an integration stops new runs without changing existing scores, experiments, paper positions, or orders.

5. C4 Context

```
C4Context
title System Context - Governed External Research Integrations

Person(owner, "Vaibhav", "Reviews integrations, research results, and live proposals")
System(kairos, "Kairos", "System of record, evidence, validation, paper/live governance")
System_Ext(github, "Approved upstream projects", "Libraries, skills, and source repositories")
System_Ext(data, "Approved market data providers", "Reach Kairos only through typed evidence adapters")
System_Ext(brokers, "Broker accounts", "Reach Kairos-owned broker adapters only")

Rel(owner, kairos, "Reviews, enables, disables, and promotes", "HTTPS")
Rel(github, kairos, "Provides reviewed source/release metadata only", "Release review")
Rel(kairos, data, "Retrieves typed research evidence", "HTTPS via allowlisted adapters")
Rel(kairos, brokers, "Submits owner-approved orders", "Kairos-owned broker adapter")
```

Upstream projects do not connect to brokers, accounts, or the primary Kairos database. A GitHub project is source material, not a peer system.

6. C4 Container Boundary

```
C4Container
title Container Diagram - Kairos Research Integration Boundary

Person(owner, "Vaibhav", "Owner")

System_Boundary(kairos, "Kairos") {
  Container(web, "Kairos web/API", "Next.js 15", "Settings, research, validation, paper/live approvals")
  ContainerDb(db, "Kairos database", "Supabase Postgres", "Evidence, decisions, ledgers, policies, artifacts")
  Container(router, "Canonical Evidence Router", "Kairos TypeScript", "Typed provider routing, cache, pacing, provenance")
  Container(workerApi, "Future Research Worker Gateway", "Kairos API", "Issues scoped jobs and validates results")
  Container(worker, "Future isolated compute container", "Pinned container in GitHub Actions", "No secrets or network; calculates approved research artifacts")
  Container(exec, "Kairos Execution Gateway", "Kairos TypeScript", "Final owner/gate/broker checks; never callable by worker")
}

System_Ext(data, "Market-data providers", "Typed evidence providers")
System_Ext(broker, "Broker APIs/MCP", "Real account access")

Rel(owner, web, "Controls policy and reviews artifacts", "HTTPS")
Rel(web, db, "Reads/writes governed records", "Supabase")
Rel(web, router, "Requests evidence intents", "In-process")
Rel(router, data, "Fetches allowlisted typed evidence", "HTTPS")
Rel(web, workerApi, "Creates research job", "Internal API")
Rel(workerApi, db, "Reads frozen snapshots; appends artifacts", "Restricted DB role")
Rel(workerApi, worker, "Future: dispatches immutable job", "GitHub Actions workflow")
Rel(worker, workerApi, "Future: writes result file for trusted wrapper", "No network from compute container")
Rel(web, exec, "Sends owner-approved proposal only", "In-process")
Rel(exec, broker, "Submits after final checks", "Kairos-owned adapter")
```

The future compute container has no relationship to the broker, credentials vault, browser, data providers, or callback endpoint. A trusted Actions workflow wrapper, outside the `--network none` compute container, validates the output and posts it to Kairos using a short-lived callback proof. The wrapper is therefore a reviewed Kairos component, not third-party code. The absence of a worker-to-broker or worker-to-provider path is an intentional security control, not a convention.

7. Capability Classes And Authority Matrix

Class	Examples	Can read	Can write	Never allowed
`indicator`	TA-Lib, internally reimplemented formula	Frozen OHLCV snapshot	Derived indicator artifact	Network, LLM prompt, score/order mutation
`backtest`	Kairos replay engine, approved quant library	Frozen decision/evidence/price snapshots	Experiment result artifact	Live/current provider fetch, broker access
`research_advisory`	Vibe-derived research workflow	Bounded facts, approved documents, prior artifacts	Hypothesis or critique artifact	Direct score/direction/order output
`document_extract`	Parser/OCR library	User-approved document copy	Structured extraction artifact	External upload, unrestricted filesystem
`data_adapter`	Kairos-owned Yahoo/EDGAR/Webull adapter	Intent and provider config	Evidence cache/ledger only	Arbitrary URLs or raw MCP tools

There is deliberately no execution, portfolio_write, config_write, database_admin, or broker_read third-party capability class.

State machine

proposed -> license_review -> contract_test -> research_only -> shadow_only
-> retired

- proposed: metadata exists, no code/image installed or run.
- license_review: source and license are reviewed; still cannot run.
- contract_test: pin runs only on synthetic fixtures with network disabled.
- research_only: may create visible, non-actionable artifacts from snapshots.
- shadow_only: its structured hypothesis may be evaluated by Kairos's existing

deterministic shadow/validation flow, still with no fills or proposals.

- retired: no new jobs; historic results remain immutable and visible.

No transition from any integration state can enable paper fills, live proposals, or live orders. Those are controlled only by existing Kairos strategy and broker governance states.

8. Integration Registry

The registry is code-owned plus database-configured. Database rows select only compiled, reviewed integration identifiers; they cannot name a URL, command, Docker image, npm package, Python package, or MCP tool.

Proposed logical records

Record	Purpose	Required fields
`integration_catalog`	Immutable identity of a reviewed integration	`id`, `slug`, `capability_class`, `source_repo`, `license_spdx`, `owner`, `created_at`
`integration_releases`	One reviewed build/version	`integration_id`, `source_commit`, `release_tag`, `artifact_digest`, `dependency_lock_digest`, `review_status`, `security_scan_at`
`integration_policies`	Per-market enablement and resource limits	`release_id`, `market`, `state`, `max_jobs_day`, `timeout_seconds`, `cpu_limit`, `memory_limit`, `effective_at`, `policy_version`
`integration_runs`	Append-only job header	`run_id`, `release_id`, `policy_version`, `market`, `currency`, `snapshot_hash`, `status`, `started_at`, `ended_at`
`integration_artifacts`	Append-only validated outputs	`run_id`, `artifact_type`, `schema_version`, `payload_hash`, `storage_ref`, `quality_status`, `created_at`
`integration_security_events`	Blocked/failed security events	`run_id`, `event_type`, `severity`, `redacted_context`, `created_at`
`upstream_watch_events`	Detected upstream changes	`integration_id`, `observed_ref`, `change_kind`, `review_status`, `observed_at`

These are design names, not approved migrations. All financial and research history remains append-only. RLS must be owner-only for UI reads; the worker uses a job-specific backend credential that can create only its own run/artifact rows through narrowly scoped RPCs.

Required release manifest

Every release must contain a checked-in manifest such as:

```
type IntegrationManifest = {
  slug: "ta_lib_indicators" | "vibe_research_adapter";
  capability: "indicator" | "research_advisory" | "backtest";
  sourceRepo: string;
  sourceCommit: string;          // full immutable SHA, never a branch
  licenseSpdx: string;
  artifactDigest: string;        // image/package digest
  dependencyLockDigest: string;
  inputSchemaVersion: string;
  outputSchemaVersion: string;
  networkMode: "none";
  permittedMarkets: Array<"us" | "india">;
  permittedArtifacts: string[];
};
```

The manifest is reviewed code. Settings can enable only a manifest that has an approved release record.

9. Data Contracts

9.1 Frozen worker input (future only)

The Canonical Evidence Router is the only evidence/provenance contract. A future gateway does not re-create provider, freshness, quality, policy, or field provenance types. It freezes the router's existing EvidenceEnvelope values and adds only the market-local universe and permitted existing Kairos references. This prevents a parallel truth layer.

```
type ResearchSnapshot = {
  snapshotId: string;
  snapshotHash: string;
  market: "us" | "india";
  currency: "USD" | "INR";
  asOf: string;
  createdAt: string;
  evidencePolicyVersionId: string;
  strategyVersionId?: string;
  mandateId?: string;
  universe: Array<{ symbol: string; assetClass: string }>;
  evidence: Array<EvidenceEnvelope<unknown>>;
  decisionObservations?: Array<{ observationId: string; decisionAt: string; features: unknown; outcome?:
  unknown }>;
};
```

Rules:

- The snapshot includes one market and one currency only.
- asOf is a hard ceiling. Historical replay uses only evidence known at or before that instant.

- EvidenceEnvelope retains the Canonical Evidence Router's intent, quality,

cache state, provider attempts, policy version, unavailable reason, and field-level provenance. The integration layer adds none of these concepts.

- The worker sees normalized router facts, not provider tokens, URLs, cookies, raw credential errors, browser sessions, or user account data.

- Live position quantities/account balances are absent unless a future, separate architecture explicitly permits a redacted risk-analysis snapshot. That future capability still may not execute.

9.2 Worker result envelope

```
type WorkerResult = {
  runId: string;
  snapshotHash: string;
  integrationSlug: string;
  sourceCommit: string;
  artifactDigest: string;
  resultKind: "indicator_set" | "backtest" | "hypothesis" | "critique" | "document_extract";
  status: "completed" | "abstained" | "invalid" | "failed";
  quality: {
    warnings: string[];
    inputCoveragePct: number;
    reproducible: boolean;
  };
  payload: unknown;
  producedAt: string;
};
```

The gateway verifies `runId`, snapshot hash, manifest identity, schema, bounded payload size, numeric finiteness, market/currency match, and allowed artifact type. Any failure becomes `invalid`, emits a security/health event, and stores no candidate strategy, score, or trade action.

9.3 Artifact-specific restrictions

- An `indicator_set` contains derived values and formula/version metadata, never a buy/sell recommendation.
- A `backtest` includes assumptions, universe, date bounds, transaction costs, price basis, benchmark, and point-in-time coverage. It cannot mark itself eligible or promoted.
- A `hypothesis` contains a human-readable proposal plus a structured, whitelisted feature/gate specification. It cannot contain executable code.
- A `critique` may identify evidence gaps or contradictory assumptions, but it cannot revise a score, ledger, or configuration.

10. Controlled Flows

A. Research advisory flow (Vibe-style integration)

- Owner enables a pinned `research_advisory` release for one market in `research_only` state.
- Kairos gathers already validated evidence through the Canonical Evidence Router and freezes a bounded snapshot.
- Gateway sends the snapshot and an allowed task template to the worker.
- Worker returns a hypothesis/critique artifact with citations to snapshot IDs, not external free-form sources.
- Kairos renders it as External Research - Advisory with evidence coverage, version, and warnings.
- A human may convert a suitable hypothesis into an existing Kairos feature registry proposal. The whitelisted feature grammar and Validation Engine remain the only path toward shadow evaluation.

The artifact cannot change agent_signals, strategy_versions, a paper order, or a broker proposal by itself.

B. Indicator/backtest flow (TA-Lib-style integration)

- Research or an owner-triggered experiment requests an approved indicator or backtest capability.
- Gateway creates a market-local point-in-time snapshot, including adjusted price basis and price/evidence provenance.
- Worker computes output with no network.
- Gateway validates and persists an artifact.
- The existing deterministic validator, not the worker, performs walk-forward folds, purging/embargo, cost treatment, benchmark calculation, and promotion gate checks.
- A passing result may create a validation_experiments record and at most a shadow_paper candidate under existing owner/automation controls.

C. Current money path remains unchanged

```
sequenceDiagram
    participant W as External Worker
    participant K as Kairos Research/Validation
    participant P as PaperTrader
    participant O as Owner
    participant E as Kairos Execution Gateway
    participant B as Broker

    W-->>K: Advisory artifact or derived metrics only
    K-->>K: Validate deterministically and optionally shadow
    K-->>P: Existing approved champion signal only
    P-->>K: Existing paper ledger events
    K-->>O: Existing live proposal, if independently eligible
    O-->>E: Explicit approval
    E-->>E: Recheck all money-path gates
    E-->>B: Submit through Kairos-owned adapter
```

There is intentionally no arrow from W to P, E, or B.

11. Security Threat Model And Controls

```
| Threat | Failure scenario | Required control |
| Supply-chain compromise | A dependency or upstream release becomes malicious | Pin full commit + artifact digest; lock dependencies; SBOM; vulnerability/license scan; review diff; no auto-update |
| Prompt injection | A news article, README, or provider string asks a worker/LLM to change behavior | Treat external text as data; schema/bound length; no tool instructions from artifacts; advisory output only |
| Credential theft | Third-party code reads env/vault/browser token | Dedicated worker runtime has no secrets, host mounts, shell access, or production env |
| Broker bypass | Worker calls a broker/API/MCP tool directly | Network deny by default; no broker DNS/egress; no credentials; execution gateway rejects non-Kairos callers |
| Database corruption | Worker obtains broad database access | Job-scoped signed token plus append-only RPCs; no direct database connection; RLS/RPC scope tested |
| Data exfiltration | Worker sends account/history data to an arbitrary host | No egress by default; only sanitized snapshot; outbound block logged as security event |
| Replay leakage | Backtest reads data that was unavailable at decision time | Snapshot `asOf`, evidence observation/retrieval times, sealed replay accessor, snapshot hash |
| Resource exhaustion | Bad library hangs, forks, or allocates memory | Per-run CPU/memory/time/file-size limits; concurrency cap; cancel watchdog; queue isolation |
| Dependency drift | A version update changes numerical results silently | Golden fixtures, deterministic result hashes/tolerances, release comparison report, manual promotion |
| License contamination | Code is copied from incompatible source | License review before source import; source/commit attribution; permitted fork only when license allows |
```

Security controls that are mandatory before any worker exists

- Separate runtime identity from the Next.js/Vercel runtime.
- No Docker socket, host filesystem mount, SSH agent, Windows user profile, or .env* file in the worker environment.
- Read-only filesystem except an ephemeral working directory.
- Egress disabled. The first release must require zero network.
- A short-lived, single-use job token bound to runId, artifact digest, snapshot hash, expiry, and permitted result schema.
- Gateway idempotency: a job can append its result once; retries cannot duplicate strategy candidates or artifacts.
- Redacted logs: no raw snapshot payloads, access tokens, provider errors, account numbers, or LLM prompts in central logs.
- A global integration kill switch plus per-integration/per-market disable.
- Worker output parser rejects prototype-pollution keys, unexpected nested data, executable code, URLs outside cited snapshot references, and non-finite values.

12. Upstream Review And Update Policy

An upstream update is an input to a review queue, never a deployment event.

Intake checklist for any new repository

- Establish the desired capability and confirm Kairos does not already own it.
- Record repository owner, canonical URL, full commit, release/tag, license, language/runtime, maintenance signal, dependency graph, and market relevance.
- Check license compatibility before copying code or creating a fork.
- Review source for secrets, shell execution, dynamic code loading, outbound network behavior, telemetry, subprocesses, filesystem access, and credential assumptions.
- Define the minimum snapshot and artifact schema before installation.
- Run only synthetic fixtures with network disabled.
- Add golden tests, failure tests, resource-limit tests, and a kill-switch test.
- Approve research-only state manually. No installation can skip these steps.

Update lifecycle

- A scheduled metadata-only watcher records a new upstream release/commit.
- A reviewer compares license, manifest, dependency lock, SBOM, and relevant source diff against the pinned release.
- A candidate image is built in an isolated CI environment and scanned.

- It runs fixture and regression suites against the same snapshots as the old release. Numerical output differences require an explanation.
- The candidate may enter `contract_test`; it cannot replace the active pin.
- An owner promotes the release by changing the active registry pointer.
- The old release remains available for rollback until the configured retention period has elapsed.

If an upstream repository is deleted or abandoned, active pinned artifacts remain reproducible. For a capability worth keeping, Kairos may maintain a private or internal mirror/fork only after its license permits that. Kairos must never depend on cloning GitHub at request time.

13. Candidate Mapping From Reviewed Projects

Candidate	Valuable capability	Initial treatment	Explicit boundary
TA-Lib	Standard technical indicators	Deferred candidate only if pure TypeScript/provider parity is insufficient	No worker or native dependency merely for indicator parity; never an execution/scoring shortcut
Vibe-Trading	Research-team workflows, hypothesis/critique artifacts, backtest evidence patterns	Later `research\_advisory` adapter after source/license/security review	No direct execution, source/provider calls, or automatic strategy writes
ML4T	Walk-forward/backtest methodology and test corpus ideas	Reference only	Reimplement only narrow, reviewed methods in Kairos tests
Qlib	Experiment/data/feature lifecycle concepts	Reference only until a later model-research phase	No framework import into money path
OpenBB	Provider capability/adaptor architecture	Architecture reference only	No source/runtime use under this plan because current repo is AGPLv3
FinRL, Qbot, strategy collections	Hypothesis discovery	Reference only	No imported strategy may bypass existing validation/promotion gates
QuantDinger, AI Berkshire	Explainability/research-journal ideas	Product/reference only	No security/execution code reuse without separate review

Every candidate must pass the intake checklist independently. A positive review of one project does not create trust in a future version, fork, package, or skill.

14. Product Surface

Add one Settings area, Research Integrations, only after the backend safety model exists. It is operational, not a marketplace.

Registry view

For each approved integration show:

- Capability and plain-language purpose.
- Market scope and current state.
- Pinned source commit/release, artifact digest, and license.
- Last contract test, last run, error rate, timeout rate, and output coverage.
- Inputs it can receive and artifacts it can produce.
- A clear statement: Cannot access accounts or submit orders.
- Enable/disable control requiring owner confirmation; disable is immediate.

Artifact view

Research/Journals can show an externally produced artifact only with:

- Advisory or Derived Indicator label.
- Snapshot as-of time, market/currency, policy and strategy versions.
- Integration version and data coverage warning.
- Evidence links to Kairos snapshots.
- A visible path to inspect the deterministic validation result, if one exists.

No UI wording implies an artifact is a trade command, prediction guarantee, or live-account recommendation.

15. Observability And Health

Track per integration, release, market, and artifact type:

- jobs requested/started/completed/abstained/invalid/failed;
- queue age, runtime, CPU/memory limit hits, and timeout rate;
- snapshot coverage and stale/unavailable evidence ratio;
- schema validation failures and blocked egress attempts;
- result reproducibility/hash mismatch rate;
- artifact-to-hypothesis, hypothesis-to-validation, and validation-to-shadow

conversion rates;

- zero financial metric attribution: integrations do not get credit for returns

until an independent Kairos strategy lifecycle has evidence.

Health degradation disables new jobs at the policy layer. It does not delete artifacts or alter an existing champion/paper/live position.

16. Phased Build Order

Prerequisite - Canonical Evidence Router Phase 2 foundation

This is the next implementation work, not the integration framework. It creates the one provenance/capacity substrate that future integrations must consume:

- canonical contracts and intent catalog;
- immutable policy versions and active market pointers;
- durable evidence cache, provider-call ledger, refresh queue, and pacing repair;
- typed adapter registry and independently testable adapters; and
- all-Auto policy seeds with `router_enabled=false` and dual-resolution logging.

Its disjoint foundation work may be parallelized in isolated worktrees: policy migration, cache/ledger/queue migration, canonical contracts, capacity RPC repair, and one adapter per worker. The router stays shadow-only until its own Phase 2 evidence proves score/availability/entry-eligibility parity. Do not build its research cutover in parallel with the foundation.

Exit gate: canonical and legacy outputs are available for comparison; no provider choice can affect production scoring, paper fills, or live proposals.

Phase 1 - In-repository technical indicator hygiene

Do not create a worker just to run TA-Lib. First inventory the 6-10 technical formulas Kairos actually uses. For each, choose either the current validated provider value or a small pure TypeScript implementation using Kairos price snapshots. Add formula/version metadata and golden fixtures against known values.

- No new external runtime or native dependency by default.
- No scoring-weight, entry, paper-fill, or execution change in this phase.
- Do not duplicate an Alpha Vantage/other router value without a documented

semantic reason (period, smoothing, adjustment basis, or availability).

Exit gate: exact formula semantics, price basis, warm-up behavior, missing-data behavior, and fixture tolerances are documented and tests are green.

Phase 2 - Deferred isolated compute framework

Build this only after a genuine untrusted/LLM-driven integration has a narrowly defined value that cannot be met in-repo. The initial cloud-only target is a GitHub Actions job using a pinned Ubuntu runner and a pinned compute image.

- The workflow wrapper receives a single-use dispatch/callback proof; the

`docker run --network none` compute container receives neither that proof nor any other secret.

- The compute container writes a bounded result file; the reviewed wrapper

validates it and performs the authenticated callback to Kairos.

- Pin every workflow action by full commit SHA, restrict GITHUB_TOKEN

permissions, do not expose Actions secrets to the compute container, and do not upload market snapshots as public artifacts.

- GitHub Actions is quota-limited for private repositories. Dispatch must stop at

a conservative included-minute/storage threshold and paid overage must remain disabled. It is free only while within the account's included allowance.

- A future persistent/free-cloud fallback, if truly needed, requires a separate

deployment, network, and secrets architecture review. Do not substitute a local Windows service.

Exit gate: synthetic jobs prove the container has no egress/secrets, the wrapper is the only callback identity, results are reproducible, and disabled state cannot dispatch work.

Phase 3 - External advisory research adapter (Vibe candidate)

- Complete source-license-security review of the exact Vibe release first.
- Add a narrowly defined advisory task template and structured hypothesis/critique

output schema, using only frozen EvidenceEnvelope-derived snapshots.

- Permit no external tool/network use from the compute container.
- Require a human conversion into the existing feature registry proposal flow.

Exit gate: prompt-injection corpus passes; artifacts cannot alter a score, strategy, paper position, proposal, or execution path; owner finds UI useful.

Phase 4 - Optional capability expansion

Potential additions are evaluated individually: document extraction, additional indicator libraries, Qlib-inspired experiment tooling, or a permitted local fork. Each repeats the same intake and rollout process. There is no blanket approval for a GitHub topic, organization, or skill collection.

17. Rollback And Incident Response

Normal disable

Owner disables an integration globally or per market. Gateway rejects queued/new jobs immediately. In-flight jobs are cancelled or their results rejected. Existing artifacts remain visible as historical records, clearly marked disabled.

Security incident

- Disable the affected release globally.
- Revoke its job-token issuer/worker identity.
- Block the image/artifact digest from future dispatch.
- Preserve redacted run/security evidence for review.
- Confirm no broker, vault, or direct database path existed; inspect gateway audit events for rejected attempts.
- Roll back to a previously approved release only after its digest and tests are reverified.

No incident response action mutates historical financial ledgers. Any questionable artifact is marked `security_quarantined` and excluded from future research views and validation inputs.

18. Acceptance Criteria

The future isolated-compute implementation is acceptable only when all are proven:

- A disabled integration cannot start a job.
- A worker has no broker/vault/Supabase-service-role/browser credentials.
- DNS/egress to broker domains, arbitrary hosts, and provider domains is blocked.
- A worker cannot reach any order, proposal, settings, or direct DB endpoint.
- A job token cannot be reused, altered for another snapshot, or used after expiry.
- Invalid, oversized, non-finite, unexpected-schema, or wrong-market output is rejected and produces no candidate strategy or score mutation.
- A US job cannot contain or create India/INR data, and vice versa.
- Re-running the same pinned release against the same snapshot produces the same result within defined numerical tolerance and records both provenance.
- A worker timeout/kill/egress attempt fails closed and leaves existing Kairos research, paper, live proposals, and positions unchanged.
- Upstream update detection cannot update an active release automatically.

- The UI accurately states state, pin, inputs, outputs, health, and lack of account/execution access.
- Existing benchmark-alpha, capital-rotation, learner, PaperTrader, PositionMonitor, and broker tests remain unchanged/green with every integration disabled.

19. Open Decisions For Claude And Owner

- Future worker hosting: GitHub Actions is the proposed cloud-only initial runtime, but private repositories have included-minute/storage quotas rather than an unconditional free tier. Recommendation: use it only with a strict no-paid-overage dispatch guard; do not run a local Windows service and do not run native worker compute inside Vercel.
- Artifact storage: Supabase JSONB versus object storage with hashed references. Recommendation: small structured results in Postgres; large reports/arrays in private storage with immutable hash references.
- License policy: which licenses are approved for dependency, permitted fork, reference-only, and prohibited categories. Recommendation: explicitly prohibit copyleft runtime dependencies until reviewed; record SPDX per release.
- Vibe source boundary: whether to reimplement a minimal adapter from public behavior, use a permitted pinned fork, or keep it as a manual/offline research tool. Recommendation: prove utility manually first, then choose the smallest permitted integration.
- Data Policy dependency: should any provider-dependent integration wait for the Canonical Evidence Router? Recommendation: yes. Its snapshot must embed the router's EvidenceEnvelope, not a second provenance type. In-repo formula fixture work can proceed without provider access but cannot create a new competing evidence path.
- Review authority: who may move `contract_test` to `research_only`. Recommendation: owner approval plus a completed security/code review; no automation can promote a release.

20. What This Document Explicitly Does Not Approve

- Installing a GitHub project directly into the Next.js app.
- Executing a repository's install script on the developer workstation.
- Giving Vibe, TA-Lib, Qlib, OpenBB, or any future integration a live account, broker, MCP, API-vault, service-role, or shell credential.
- Replacing Kairos's deterministic scoring, validation, benchmark, paper ledger, capital rotation, or execution gateway.
- Enabling Webull orders, autonomous live trading, or a new provider from this architecture.
- Treating a repository's README, stars, claimed backtest, or LLM output as evidence of trading edge.

21. Recommended Owner Decision

Approve the authority boundary in principle, but do not build a worker now. Build the Canonical Evidence Router Phase 2 foundation behind flags and shadow comparison first. Then do the small in-repository technical-indicator hygiene work only where it has a demonstrated gap. Vibe and every other untrusted repository remain deferred until there is a concrete need for the isolated GitHub Actions compute pattern and its boundary has been independently reviewed.

external research shadow - Feature Architecture

Source: features\external-research-shadow\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

External Research Shadow Runtime

Status: REVIEWED DESIGN DRAFT. NOT APPROVED FOR IMPLEMENTATION.

Last reviewed: 2026-07-15 by Codex.

Authority: subordinate to features/external-research-integrations/FEATURE_ARCHITECTURE.md.

Scope: optional, record-only research compute. No money-path authority.

1. Decision

Do not run a "self-sourcing" external repository as the first phase. Self-sourcing requires outbound network and usually credentials, which contradicts the required `docker run --network none` compute boundary. Independent data can be valuable, but acquisition must occur in a reviewed Kairos-owned adapter before untrusted compute.

The only approved conceptual runtime is:

- trusted Kairos acquisition freezes a minimal public-market-data snapshot;
- untrusted, pinned compute receives that snapshot with no network or secrets;
- a trusted wrapper validates and ingests a bounded artifact; and
- Kairos evaluates it as research evidence only.

No external repository directly becomes a Kairos agent, candidate-discovery source, Challenger, scorer, provider adapter, or broker dependency. A useful result may motivate a separately specified, clean-room, Kairos-owned deterministic feature.

2. Preconditions

Do not build this runtime until all are true:

- Router cutover and eligibility guard are complete for the target market;
- experiment lineage can reference Router policy/evidence fingerprints without a third provenance system;
- an exact repository commit, license, transitive dependency lock, and capability have passed source/security/legal review;
- the same hypothesis cannot be tested more cheaply and safely in native Kairos;
- an owner-approved resource budget, kill switch, retention policy, and threat model exist; and
- a synthetic fixture run passes before any real snapshot is exported.

This makes the feature optional and later than native experiment lineage.

3. Trust Boundaries

Trusted control plane

A Kairos-owned workflow repository and reviewed wrapper may:

- construct a job manifest from an approved integration registry entry;
- download a prebuilt, digest-pinned compute image or build from a reviewed lock;
- place the immutable input snapshot in a bounded read-only mount;
- launch `docker run --network none` with hard resource restrictions;
- validate/scan the output after the container exits; and
- upload only the accepted artifact and trusted wrapper logs.

Untrusted compute plane

The repository code receives:

- no GitHub token, OIDC token, environment secret, provider key, broker token,

Supabase credential, browser cookie, portfolio/account data, or callback proof;

- no outbound or inbound network;
- no host Docker socket, privileged mode, device access, or writable host path;
- a read-only input mount and one size-bounded output directory;
- a read-only root filesystem where compatible, non-root UID, dropped capabilities,

PID/memory/CPU/time limits, and a process/file-size limit; and

- one approved command with no user-supplied shell fragment.

The repository cannot upload artifacts itself. The trusted wrapper regains control after exit, rejects unsafe files/links/archives, validates JSON, and uploads only the normalized result.

GitHub-hosted runner ephemerality is defense in depth, not the sandbox. Job-level Actions container : is not treated as proof of network isolation; the reviewed wrapper must execute the explicit no-network container and test egress denial.

4. GitHub Actions Security Baseline

- workflow exists only in a Kairos-owned private control repository;
- no `pull_request`, `pull_request_target`, `issue-comment`, or `fork-controlled` trigger;
- manual/scheduled dispatch selects only code-known integration IDs;
- top-level permissions: `{ contents: read }` or stricter; no write permission;
- no cloud OIDC, deployment environments, production secrets, or persistent runner;
- every third-party Action is pinned to a full commit SHA and reviewed;
- no untrusted cache restore, reusable workflow from a floating ref, or mutable tag;
- concurrency is one, timeout and daily job caps are enforced before dispatch;
- no paid overage; quota exhaustion is an abstention;
- source/image/dependency digests, action SHAs, runner image, and wrapper version are

written to the run record; and

- hostile-output fixtures test ANSI/log injection, symlinks, path traversal, zip bombs, huge JSON, NaN/Infinity, deep nesting, HTML/Markdown/script, and bad market.

5. Immutable Snapshot Contract

Do not export mutable evidence_cache_v2 rows directly. Create an immutable research snapshot manifest that references existing evidence and lineage records:

```
type ExternalResearchSnapshot = {
  snapshotId: string;
  snapshotHash: string;
  schemaVersion: string;
  market: "us" | "india";
  currency: "USD" | "INR";
  asOf: string;
  policyVersionId: string;
  experimentRunId: string;
  universeVersion: string;
  symbols: string[];
  evidenceRefs: Array<{
    evidenceRecordId: string;
    envelopeHash: string;
    intent: string;
    availableAt: string;
  }>;
  files: Array<{ path: string; sha256: string; bytes: number }>;
};
```

The materialized files contain only the minimum normalized public-market facts needed by the capability. Exclude owner identity, watchlist rationale, account, holdings, orders, fills, cash, broker, credentials, internal prompts, and raw provider responses. A historical snapshot enforces availableAt <= asOf.

Provider acquisition remains in Router-owned adapters with normal quota, license, and terms controls. An external project's data loader, URL, key, browser automation, or MCP tool is never admitted into the untrusted container.

6. Output Contract

```
type ExternalResearchArtifact = {
  schemaVersion: string;
  runId: string;
  snapshotHash: string;
  integrationId: string;
  sourceCommit: string;
  imageDigest: string;
  market: "us" | "india";
  asOf: string;
  status: "completed" | "abstained";
  artifactKind: "hypothesis" | "critique" | "numeric_signal" | "backtest_report";
  coverage: { eligibleN: number; resolvedN: number };
  payload: unknown;
};
```

The trusted gateway checks exact IDs/hashes, market/currency, schema/version, bounded strings/arrays/numbers, finite numeric values, symbol allowlist, payload size, and allowed artifact kind. Rationale is stored/rendered as escaped plain text; URLs, executable code, raw HTML/Markdown, files, and tool instructions are rejected. Invalid output records a run failure but does not persist a partial advisory.

7. Records Without A Parallel Truth Layer

Extend the approved experiment-lineage model with:

- external_research_runs: experiment ID, integration release, snapshot fingerprint,

wrapper/action/image digests, resource use, status, and failure code;

- external_research_artifacts: run ID, kind, schema, payload hash, storage reference, validation status, and retention class; and

- security events containing normalized codes only.

These records reference existing experiment_runs, Router policy/evidence records, and labels. They do not copy source provenance into a third ledger. Do not create an external_advice table that pretends a repo generated a Kairos decision, and do not manufacture decision_observations to improve sample size.

RLS is owner-read; only a service-role RPC owned by the trusted gateway may append a result for the exact issued run. Tables are append-only and reject update/delete.

8. Evaluation

External output is compared only on a predeclared common universe, as-of time, horizon, benchmark, and cost model. Numeric signals must be alignable to existing observations without selecting only names the repo liked. Report missingness and abstentions.

Every repository, commit, prompt/configuration, signal definition, regime slice, and horizon is part of the trial family. Use walk-forward/holdout evaluation and DSR/PBO governance. "Who beats us where" is not a product claim unless confidence, coverage, multiple testing, and incremental value over the champion are shown.

An external artifact can remain advisory indefinitely. Promotion means writing and validating a native Kairos feature under a new architecture; it never means wiring the upstream runtime into candidate discovery or trading.

9. Licensing And Supply Chain

Runtime execution does not make license obligations disappear. Before admission:

- verify the exact commit's license and every bundled/transitive dependency;
- define whether the use is copy, modification, distribution, or network service;
- obtain explicit legal/product approval for copyleft, source-available, unclear, missing, or changed licenses;
- generate/store SBOM and vulnerability/malware scan results;
- mirror source/artifacts only when the license permits it; and
- never auto-sync upstream. A new commit is a new release and repeats review/tests.

Clean-room reimplementation applies to ideas later rebuilt in Kairos. It is not a substitute for complying with the license of code actually executed.

10. Phases

- P0 synthetic harness: Kairos-owned toy program, synthetic data, egress-denial and hostile-output tests. No third-party repo and no production snapshot.
- P1 one reviewed capability: one permissively licensed, exact-pinned repo on a minimal immutable snapshot, record-only, owner-triggered, no LLM required.
- P2 repeated shadow: scheduled bounded runs after retention/cost review; common-universe evaluation, still no influence.

- P3 native hypothesis: if evidence warrants, design a separate clean-room

Kairos feature and send it through normal measure/shadow/validation governance.

Vibe is not automatically P1. Its breadth and agent/tool surface increase review cost; a minimal adapter must be justified and source-audited first.

11. Disable, Incident Response, Acceptance

A code-owned global kill switch and per-integration policy stop dispatch immediately. Revoke wrapper credentials, disable workflow, quarantine unvalidated artifacts, and retain immutable audit metadata. Existing Kairos research/scoring/trading continues unchanged because there is no runtime dependency.

Acceptance requires a proven egress-denial test, zero secrets exposed to compute, hostile artifact rejection, reproducibility from hashes, per-market isolation, quota stop, append-only/RLS tests, and explicit proof that no import/call path reaches signals, strategy activation, broker adapters, or order execution.

12. Owner Decisions Before Build

- Whether this optional framework is worth building after native lineage.
- The first narrow capability, exact commit, and license class.
- Maximum daily Actions minutes/storage and retention.
- Whether private GitHub Actions is acceptable as the initial control plane after the sandbox threat model is independently reviewed.

external skill observation - Feature Architecture

Source: `features\external-skill-observation\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

External Skill Observation Adapter

Status: *DRAFT - NOT APPROVED FOR IMPLEMENTATION*

Scope: *one optional, record-only experiment; no trade influence*

Decision

Do not wire Claude interactive skills, their prompts, or their broker/data tools into ResearchAgent. `nse-trading-toolkit` is a prompt workflow with optional Groww MCP and Yahoo/yfinance dependencies, not a deterministic library with a stable output contract. `garchmethod` may be useful later for a volatility hypothesis, but it is not evidence that its code or recommendations should enter the money path.

The smallest credible experiment is a Kairos-owned deterministic GARCH(1,1) observation calculated from Kairos-frozen daily returns, written beside existing research evidence for a single market. It is not sourced from the installed skill and does not consume a provider key beyond the candle data Kairos already owns.

Boundaries

- Input: one market-local, immutable daily-return snapshot; no holdings, account, broker, token, prompt, or external MCP access.
- Compute: a pinned, reviewed implementation in an isolated offline job, or clean-room native deterministic code after a separate license/security review.
- Output: `garch_vol_forecast`, model/version/hash, return-window end date, convergence status, and unavailable reason. Finite values only.
- Consumer: evidence ledger/read-only dashboard only. It cannot modify score, rank, sizing, stop, target, eligibility, PaperTrader, PositionMonitor, proposals, or orders.

Why This Is The Right First Step

It tests a measurable claim: whether a market-local volatility forecast improves forecast calibration or risk diagnostics over realized-volatility baselines. It avoids the unsafe alternative of treating a natural-language trading framework as an agent input, and it preserves the option to reject the idea with no production impact.

Evaluation And Promotion

Compare the observation with predeclared realized-volatility baselines using frozen walk-forward windows, separately by market, with sample floors and costs where a future sizing simulation needs them. Register every model/order/window variant in the trial family. A later sizing change requires its own approved architecture and must apply in PaperTrader's sizing construction before the execution gateway, never inside the gateway.

Explicit Non-Goals

- No automated invocation of Claude skills, Python scripts, Groww/TradingView MCP,

yfinance, or upstream repo data loaders.

- No API-key sharing, network-enabled GitHub compute, external code execution, or provider-quota bypass.

- No direct VCP/CANSLIM/sector-rotation feature added to agent_signals.

- No use for US and India together; each model, evidence set, and conclusion is local.

Prerequisites And Acceptance

Start only after Router cutover/lineage prerequisites and sufficient labeled outcomes exist for the target market. The build must prove no call path from the observation to trading, validate deterministic replay from a snapshot hash, report unavailable rather than fabricate a forecast, and retain the full trial lineage. Any external repository execution remains governed by features/external-research-shadow/FEATURE_ARCHITECTURE.md.

forecast calibration - Feature Architecture

Source: features\forecast-calibration\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Forecast Calibration and Earnings-Event Assurance

Status: Approved by owner direction and implemented in phases on 2026-07-30.

Problem

Kairos already stores each deterministic score, the indicative stop/objective plan, and immutable 2/5/10/20-session realized paths. Those records are not joined into an explicit plan-calibration view or exposed as a bounded LearnerAgent tool. Separately, earnings/options risk is measured only after an entry reaches PaperTrader or TraderAgent, so an existing holding can lack a fresh pre-event volatility observation.

The MSFT July 2026 incident exposed a related freshness defect: a pre-event daily bar was allowed to produce a post-event direction.

Semantic Boundary

The indicative profit_objective is an exit-policy level, not a predicted terminal price. Retrospective language must therefore report:

- decision/reference price;
- stop floor and profit objective;
- planned horizon;
- realized horizon close;
- maximum favorable and adverse excursion;
- whether the path reached the objective or breached the stop; and
- the fraction of the objective reached.

It must not call the objective a forecast mean or describe a one-stock miss as evidence that a strategy parameter is wrong.

Architecture

```
flowchart LR
    D["Immutable decision_observations<br/>score + trade plan"] --> C["Deterministic plan calibration"]
    L["Immutable observation_labels<br/>return + MFE + MAE"] --> C
    C --> U["Decision Review<br/>per-symbol illustrative path"]
    C --> A["Market/horizon cohorts<br/>N and readiness"]
    A --> LA["LearnerAgent read-only tool<br/>hypothesis prose only"]
    A --> RR["Existing MAE/MFE policy<br/>N >= 60, deterministic"]
    E["PIT earnings calendar"] --> G["Post-event repricing barrier"]
    G --> S["Current neutral signal<br/>until reaction bar exists"]
    H["US paper/live holdings"] --> O["Bounded earnings/options shadow"]
    E --> O
    O --> R["Existing append-only<br/>earnings_risk_observations"]
```

Deterministic Calibration

The reusable calculator consumes one original trade plan and one matured label. It refuses malformed, non-candidate, stale, or currency-incoherent plans.

For the label whose horizon exactly matches horizon_sessions:

- `objective_reached = MFE >= target_pct;`
- `stop_breached = MAE <= -stop_loss_pct;`
- `objective_reach_ratio = MFE / target_pct;`
- terminal return, benchmark-neutral return, MFE, and MAE remain distinct.

MAE and MFE alone do not reveal event order. If both levels were touched, the calculator reports both and never claims which exit would have happened first.

Per-symbol output is always illustrative. Cohorts become reviewable at 20 matured paths, while risk-parameter admission keeps the existing stricter floor of 60 same-market, same-horizon eligible-long labels.

LearnerAgent Boundary

The LLM may:

- read deterministic aggregate calibration;
- explain likely failure modes;
- write a hypothesis for future validation; and
- point to insufficient evidence.

The LLM may not:

- calculate labels or overwrite realized outcomes;
- change stop, target, horizon, score, or weights from one event;
- treat personal trade history as market-wide evidence; or
- bypass the existing challenger/promotion gates.

The existing deterministic MAE/MFE policy remains the only automatic stop/objective adjustment path and requires at least 60 matching labels.

Options Monitoring

Options remain US earnings-risk evidence, never a sixth directional score. Cboe's own explanation is explicit that earnings options primarily estimate move magnitude rather than direction.

The scheduled shadow monitors only:

- open US paper positions; and
- symbols in the latest complete US live holding-risk snapshots.

It deduplicates symbols, caps work per run, and discovers events from the persisted cache plus one bounded Robinhood calendar call. That batch fallback closes a cold-cache gap without creating N per-symbol requests. It requests an option chain only when the event falls inside that position's horizon. Otherwise-eligible entries continue to be measured at the existing PaperTrader/TraderAgent choke points. Rejected/watchlist symbols do not consume option calls merely to inflate a sample.

Every observation is append-only, idempotent per market session, and `behavior_changed=false`. It cannot score, size, enter, exit, or mutate a position.

Earnings Repricing Rules

- Before-market-open result: the report-date daily bar can contain the reaction.

- After-close result: a daily bar strictly after the report date is required.
- Unknown session: use the conservative after-close rule.
- A known event that is already in the past activates the barrier even when the actual-result feed is late.
- A current neutral signal is written so consumers cannot fall back to an older directional signal.
- Mechanical stop, target, trailing-stop, and time exits remain independent.

Acceptance Criteria

- MSFT-style AMC input plus only the report-date bar produces neutral.
- BMO input plus a completed report-date bar releases the barrier.
- A late actual feed cannot reopen stale scoring after a known past event.
- Decision Review shows plan versus realized path without calling the objective a terminal-price prediction.
- LearnerAgent receives only aggregate deterministic calibration and cannot mutate risk parameters from it.
- The holding monitor writes only shadow observations and never calls options for India or for an event outside the position horizon.
- No US/India data is aggregated together.

Sources

- Cboe, "What Options Data May Indicate About Mag 7 Earnings": <https://www.cboe.com/insights/posts/what-options-data-may-indicate-about-mag-7-earnings/>
- Rompolis, "Pricing event risk: evidence from concave implied volatility curves," Review of Finance (2025): <https://doi.org/10.1093/rof/rfaf016>
- Novy-Marx, "Backtesting Strategies Based on Multiple Signals," NBER Working Paper 21329: <https://doi.org/10.3386/w21329>

health control and paper pnl - Feature Architecture

Source: `features\health-control-and-paper-pnl\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Health Control And Paper P&L

Status: APPROVED AND IN BUILD (owner request, 2026-07-20)

Problem

- A kill-switch trip latches `market_controls.trading_enabled=false`, but the Settings market toggle only changes `strategy_config`. The user therefore cannot inspect or safely reset the actual latch from the UI.
- `paper_trades.realized_pnl` is correctly null while a trade is open, but the Trades tables render only that terminal value. The current mark-to-market P&L already available on `paper_positions` is not shown beside the open lot.
- Live holdings already show current P&L. Live orders and research signals do not share one canonical, cross-broker execution ledger and must not be presented as if uploaded history were broker-confirmed live trades.

Design

Guarded reset

- Add an owner-only Settings API that returns each market latch and reset reason.
- A reset names the market and the book that tripped (paper or live).
- The server reruns `checkKillSwitches` for that exact book before enabling the market latch. An unsafe, stale, or missing live baseline refuses the reset.
- A successful reset clears the matching health issue and writes an immutable `decision_journal` entry. It never changes thresholds, global pause, security lock, broker credentials, account selection, or live-autonomy flags.
- Future trip reasons and issue keys include the book. The market latch remains intentionally conservative and shared: a trip in either book blocks new exposure for that market until its originating book passes a guarded reset.

Current paper P&L

- Keep Realized P&L terminal and immutable.
- Add Current P&L beside it for open BUY lots only.
- Compute deterministically from persisted values already fetched:
$$(\text{paper_positions.current_price} - \text{paper_trades.fill_price}) * \text{paper_trades.qty.}$$
- Join strictly by normalized market + symbol. Multiple open lots use the same current mark but retain their own fill price and remaining lot quantity.

- Missing/non-finite marks, unmatched lots, SELL rows, and closed trades render unavailable; they never substitute fill price or zero.
- Expose mark timestamp in the cell tooltip. No provider call is added.

Live parity boundary

- Live Portfolio remains the source for broker holdings and unrealized P&L.
- Research signals remain market-level evidence, not account-level executions.
- A future live Trades tab may read only reconciled broker_orders, scoped to market + active account + live environment. Kite/Webull/Robinhood parity must be proven before calling it unified. Uploaded CSV decisions remain analysis, not the live order ledger.

Acceptance criteria

- Settings shows the real per-market latch and its reason.
- Reset cannot succeed unless the originating book currently passes all brakes.
- Successful reset is audited and resolves only matching kill-switch alerts.
- Open paper trade rows show current amount and percent at the persisted mark; closed rows show only realized P&L.
- US and India are never joined or summed; currency follows the selected market.
- No external API call, LLM call, broker call, order, threshold, or autonomy flag is introduced by the P&L display.

historical evidence intake - Feature Architecture

Source: features\historical-evidence-intake\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Governed Historical Evidence Intake

Status: APPROVED for implementation by owner direction on 2026-07-29 Scope: offline acquisition, validation, normalization, and historical replay only Money-path status: unreachable; no scorer, trader, monitor, cron, or broker reads this store

1. Decision

Kairos will maintain a local, immutable historical-evidence store for deterministic backtests. It will acquire official bulk data where available, pin any community artifact to an exact commit, validate and hash every file, quarantine failures, and feed normalized records only to the existing sealed replay layer.

This is not a universal market-data lake. The useful first scope is:

- India daily equity prices and point-in-time traded universe from NSE bhavcopy.
- India historical corporate actions from NSE, required before adjusted-return use.
- US as-filed fundamentals from SEC quarterly Financial Statement Data Sets.
- US macro vintages from FRED/ALFRED for as-known regime replay.
- A diagnostic-only S&P 500 historical-membership fixture for cross-checks.

No free GitHub repository found in the intake review replaces a survivor-bias-free US security master plus adjusted prices and delisting returns. Kairos must continue using its existing Massive point-in-time membership/price entitlement for recent US experiments or purchase licensed data for older promotion-grade US tests.

2. Why Local, Not Supabase

The raw SEC archive is gigabytes and India daily files are hundreds of megabytes. Uploading those rows to Supabase would waste the free database/storage budget and make local reproducibility depend on network access.

Raw and normalized files live under:

```
%KAIROS_EVIDENCE_DIR%
  raw/<source>/<version>/...
  normalized/<dataset>/<version>/...
  manifests/<dataset-id>.json
  quarantine/<dataset-id>/...
  catalog.json
```

Default: %USERPROFILE%\kairos\evidence. The store is outside the repository and outside OneDrive. Only small schemas, acquisition code, tests, and example manifests are committed.

Supabase receives only the existing immutable experiment identity, dataset fingerprint, result summary, and (when explicitly materialized) sealed replay packet metadata. Raw bulk rows never enter a browser or Vercel runtime.

The executable boundary is defined in features/local-historical-replay/FEATURE_ARCHITECTURE.md: a local, network-free worker verifies these manifests, predeclares one immutable experiment, computes the diagnostic, and uploads only compact fingerprints/results. The application does not read this directory and the raw store is not uploaded anywhere.

3. Source Decisions

Source	Capability	Authority	Intake decision	Evidence class
SEC Financial Statement Data Sets	quarterly ZIPs of primary statements, as filed, 2009+	SEC official	adopt; prefer over bulk `companyfacts.zip` for historical normalization	promotion-capable after metric tests
SEC `companyfacts.zip`	all XBRL company facts, nightly	SEC official	catalog only; do not download by default (1.29 GB compressed and much larger expanded)	capture/reference
NSE daily bhavcopy	all daily CM rows, OHLC/volume/trade fields	NSE official	adopt	promotion-capable only with actions and coverage
NSE corporate-actions archive	splits, bonuses, dividends and effective dates	NSE official	adopt; adjusted returns refuse without it	promotion-capable after reconciliation
FRED/ALFRED	observation vintages and real-time periods	Federal Reserve Bank of St. Louis	adopt exact series used by MacroSentinel	promotion-capable
`chartiny/nse-\*`	normalized mirrors of NSE reports	community, MIT repo	fallback/probe only; pin commit and cross-check official samples	diagnostic
`jugaad-data`	NSE/RBI downloader and caching patterns	community	reference implementation only; do not add Python runtime dependency	reference
`hanshof/sp500\_constituents`	daily historical S&P membership since 1996	community, MIT	adopt as a small diagnostic fixture, never as sole promotion universe	diagnostic
`FinanceDatabase`	current global symbol/classification catalog	community, MIT	optional display/entity-resolution input only	context
TradingView CSV	manually loaded chart bars/indicators	user-licensed manual export	accepted only for named-symbol diagnostics; never proves a PIT universe	diagnostic
FNSPID and similar news corpora	historical prices/news	research/community corpus	defer pending license, source, timestamp, and leakage audit	quarantined

Repository code is never dynamically executed. Data files are downloaded over HTTPS; community files are pinned to immutable commit URLs. GitHub outages cannot remove already-hashed local copies.

4. Manifest Contract

Each acquisition writes a manifest before normalization:

```
interface EvidenceDatasetManifestV1 {
  schemaVersion: "kairos.evidence-dataset.v1";
  datasetId: string;
  sourceId: string;
  sourceAuthority: "official" | "community" | "manual";
  evidenceClass: "promotion_candidate" | "diagnostic" | "reference";
  market: "us" | "india" | "global";
  dataKinds: Array<"ohlc" | "universe" | "fundamental" | "macro" | "corporate_action">;
  sourceVersion: string;
  retrievedAt: string;
  files: Array<{
    relativePath: string;
    sourceUrl: string;
    sha256: string;
    bytes: number;
    mediaType: string;
  }>;
  coverage: { start: string; end: string; expectedFiles?: number; receivedFiles: number };
  normalization: {
    schemaVersion: string;
    codeGitSha: string;
    codeSha256: string;
    status: "pending" | "valid" | "quarantined";
  };
  limitations: string[];
  datasetFingerprint: string;
}
```

The dataset fingerprint is SHA-256 over canonical manifest fields and ordered file hashes. A changed upstream file creates a new dataset version; it never mutates an existing valid version.

5. Canonical Records

Daily bar

market, exchange, symbol, session_date, open, high, low, close, volume, turnover, currency, price_basis, source_file_sha256

Raw exchange prices remain raw. Adjusted prices are a derived dataset with a separate fingerprint and the exact corporate-action manifest in its lineage.

Fundamental fact

cik, accession, symbol_at_filing, form, filed_at, accepted_at, period_start, period_end, fiscal_year, fiscal_period, taxonomy, tag, unit, value, statement, source_file_sha256

Reads use `accepted_at <= as_of`; if the compact SEC data exposes only filing date, daily replay treats it as available after that session and records the coarser clock. Amendments are additional accessions, not in-place replacements.

Macro observation

series_id, observation_date, realtime_start, realtime_end, value, unit, frequency, source_file_sha256

An as-of read selects the row whose real-time interval contains the decision date. Latest revised values must never be stamped onto historical decisions.

Corporate action

market, symbol, action_type, ex_date, record_date, ratio_or_amount, currency, announced_at, source_file_sha256

Unsupported or ambiguous actions quarantine the affected symbol/date range. The normalizer never guesses a split ratio from a price jump.

6. Validation and Quarantine

All gates fail closed:

- HTTPS and allowlisted host.
- Successful status and non-empty body.
- SHA-256 and byte count recorded.
- Expected archive members and headers present.
- Dates parse and remain inside requested coverage.
- OHLC invariants: positive prices, $high \geq \max(open, close)$, $low \leq \min(open, close)$, non-negative volume.
- One canonical row per source key; conflicting duplicates quarantine.
- Market/currency contract: US/USD, India/INR.
- Session coverage report distinguishes exchange holidays from missing files.
- Corporate-action completeness is required for adjusted-return datasets.

- Symbol mappings are time-varying; current ticker text is not a permanent ID.
- No normalized record may have `knowable_at > replay_as_of`.

Quarantined files remain preserved with reasons and hashes but cannot be resolved by the replay accessor.

7. Existing Architecture Integration

This extends, rather than duplicates, current truth layers:

- `lib/replay/packet-assembler.ts` remains the sole raw-to-frozen boundary.
- `lib/replay/sealed-accessor.ts` remains the network-free read boundary.
- `edge_universe_members` remains the persisted PIT universe table.
- `fundamental_facts` remains the live forward-capture ledger.
- `backtest_experiments` remains the immutable experiment identity.

The local resolver emits the existing `RawRecord` contract, extended additively with first-class `macro` and `corporate_action` item types. It does not add a second scoring API or a second experiment ledger.

Default resolver order for offline replay:

- exact immutable local dataset named in the experiment manifest;
- existing persisted PIT snapshot with matching policy/fingerprint;
- refuse.

There is no live-provider fallback inside a bound replay.

8. Safety Boundary

- No environment switch can make this store affect live scoring.
- No API route exposes raw files.
- No cron downloads or normalizes data in P0.
- No LLM parses, validates, transforms, or selects financial rows.
- Community and manual sources are diagnostic by default.
- A promotion experiment must name the exact dataset fingerprint before the run.
- India and US datasets, currencies, universes, actions, and results remain separate.

9. Build Order

- Manifest, hashing, allowlist, atomic download, quarantine, and catalog CLI.
- FRED/ALFRED exact-vintage acquisition for the existing MacroSentinel series.
- SEC quarterly ZIP acquisition and schema validation; selective normalization

for the experiment universe rather than expanding every fact permanently.

- NSE official bhavcopy acquisition and normalization.
- NSE corporate-action acquisition and adjusted-return derivation.
- Local replay resolver to `RawRecord`, plus sealed-accessor tests.
- Diagnostic community fixtures pinned to commits.

- Coverage report; only then run new India and macro PIT diagnostics.

10. Acceptance Criteria

- Re-running an acquisition with unchanged bytes is idempotent.
- Changed bytes at the same URL produce a new version or quarantine; never overwrite.
- Interrupted downloads leave no valid manifest.
- Path traversal from ZIP member names is rejected.
- Invalid rows cannot appear in normalized output.
- An as-of macro/fundamental read cannot see later revisions/amendments.
- India adjusted returns refuse when corporate actions are missing.
- A replay bound to a dataset works with network disabled.
- No import changes any current score, signal, paper position, broker proposal, or order.
- Catalog output names coverage, source authority, evidence class, limitations, and exact fingerprints without exposing secrets.

11. Residual Limitations

- Free US history older than the existing Massive entitlement remains unsuitable for broad promotion-grade, survivor-safe equity tests.
- SEC statements solve fundamentals, not prices, corporate actions, or delisting returns.
- NSE bhavcopy gives a point-in-time traded set, not necessarily every temporarily suspended but still listed security.
- Community historical-index files are useful cross-checks, not authoritative truth.
- Historical news/sentiment remains deferred because publication-time corrections, corpus selection, redistribution rights, and prompt-injection surfaces are harder than the incremental value currently justifies.

12. Implementation Record

Implemented on 2026-07-29. The exact current manifests, fingerprints, coverage, row counts, and residual exclusions are recorded in `ACQUISITION_RECORD.md`. Only entries marked `currentNormalizer: true` by the catalog are candidates for new experiment binding.

historical replay harness - Feature Architecture

Source: features\historical-replay-harness\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Historical Replay Harness - Frozen Point-in-Time Eligibility

STATUS: SUPERSEDED IN PART. The sealed primitives were implemented. The

parallel storage proposal below is historical context and is not authoritative

for new bulk-data runs.

New bulk-data experiments follow

features/local-historical-replay/FEATURE_ARCHITECTURE.md and extend the existing

immutable backtest_experiments ledger.

Last updated: 2026-07-29 Owner: Vaibhav Requirement: CLAUDE_CODE_POST_UPGRADE_FIX_PROMPT.md - Required test 15.

1. The requirement (verbatim)

Test 15: "MU/INTC/SNDK/GME-style frozen historical packets run without future

data and report whether/when each model becomes eligible - no handpicked future-return

dates."

And its sibling, test 14 (the guard this harness must not violate):

Test 14: "point-in-time label/universe fixtures cannot access future records."

Read together: build a replay that, for a handful of named symbols, freezes the world as it was on a sequence of past as-of dates, runs the real scoring -> calibration -> eligibility pipeline against each frozen slice, and reports the first date each model would have cleared its eligibility gate - chosen by the pipeline, never cherry-picked by a human who already knows the outcome.

The adversarial framing from the prompt applies: "Treat real money as adversarial." The whole point of this harness is to make it *impossible* to accidentally answer "would this model have caught the MU/INTC run-up?" with a number that secretly peeked at the run-up.

2. What "eligible" already means in this codebase

The harness does not invent an eligibility concept. It replays the gates that already exist:

```
| Gate | Where it lives | What it decides | | |
| **Calibration OOS gate** | `lib/validation/calibration.ts` -> `acceptCalibrationOOS()` / `fitAndStoreCalibration()` | A `pwin_logistic` model artifact is accepted only when its walk-forward holdout has >=30 usable rows, non-degenerate outcomes, and ECE <= 0.1. Fail-closed. |
| **Validation Engine gates** | `lib/validators/backtest.ts`, `app/api/agents/backtest/route.ts` (per `docs/arch/09-learning-loop.md`) | Sharpe >= 0.5 and win rate >= 40% on walk-forward held-out slices -> `eligibility_passed`. |
| **Edge IC classifier** | `lib/edges/ic.ts` -> `classify()` | An edge is shadow_eligible only at meanIC >= 0.02 and `|t|` >= 2.0 with n >= 12. |
| **Thin-evidence / abstain gate** | `lib/scoring/weighted-score.ts` -> `computeWeightedAnalystScore()`,
```

``isThinEvidence()`` | Score is meaningful only with ≥ 2 usable dimensions. |

"Model becomes eligible" in test-15 language = the earliest as-of date at which one of these gates flips from fail to pass, using only data dated $\leq$ that as-of date. That is the single number the harness reports per (symbol-cohort, model, setup, horizon).

3. What is already in place (reuse, don't rebuild)

The PIT machinery is largely built; this harness assembles frozen inputs *for* it.

- Walk-forward with purge + embargo - `lib/learning/dataset.ts` -> `walkForwardFolds()`.

Pure function, already unit-tested in `tests/walk-forward.test.ts` for the property that no train row's label window overlaps its fold's test window. This is the time-ordering spine.

- Calibration replay - `lib/validation/calibration.ts` fits a separate model on each

fold's train partition and predicts only that fold's test partition (the file's own comment flags the prior full-data leak it fixed). ECE is computed on the holdout only.

- As-of candle replay - `lib/edges/ic.ts` -> `computeEdgeIC()` already samples historical

benchmark trading days as as-of dates and slices candle history up to each, leaving room for the forward horizon. This is the pattern to generalize.

- Shared scoring contract - `lib/scoring/weighted-score.ts` guarantees the replay scores

a candidate with the *exact* rule `ResearchAgent` runs live (the module exists precisely so offline replay and live scoring can't diverge).

- Temporal evidence fields - `lib/data/evidence.ts` `EvidenceRecord` already carries

`observed_at` / `effective_at` / `retrieved_at`. These are the columns the sealed accessor filters on.

The leak surfaces this harness must close

- `providerCachedFetch` is keyed to today - `lib/data/provider-fetch.ts` writes/reads

`av_cache` by `cache_key` + `cache_date` where `cache_date = new Date().slice(0,10)`. Live scoring always pulls "latest." A replay that called it would get *today's* data stamped onto a 2022 as-of date. The harness must never call the live fetch path.

- `fetchOverview()` returns current TTM fundamentals - `lib/data/provider-interface.ts`

AV/FMP overview endpoints return *trailing-twelve-month as-of-now* ratios with no report period. Replaying these against a past date silently injects future earnings. This is the single most dangerous leak (fundamentals are the slowest-moving, highest-hindsight input).

- Universe is static & survivorship-biased - `lib/edges/universe.ts` says so in its own

header: current-liquid names only, "chosen with hindsight." SNDK (SanDisk) and delisted names are absent; GME's meme-era liquidity profile is not reconstructable from a today list. Frozen packets must carry their own point-in-time universe membership + tradability.

4. Harness architecture

Three components, each a thin layer over existing code.

```
????????????????????????????????????????????????????????
?  A. Packet Assembler (offline, one-time)  ?
```

```

? per (symbol, as-of date):
? * prices <= as-of (AV/FD historical) ?
? * fundamentals AS-OF (report_period <= d) ?
? * news <= as-of (published <= d) ?
? * universe/tradability flags as of d ?
? -> freeze to replay_packet_items + manifest ?
????????????????????????????????????????????????????????
? frozen, hashed, immutable
????????????????????????????????????????????????????????
? B. Sealed Replay Cursor
? * SealedDataAccessor(asOf) ?
? * rejects ANY item dated > asOf (throws) ?
? * feeds computeWeightedAnalystScore + ?
? calibration fit/predict + IC classify ?
? * steps as-of forward one trading day ?
????????????????????????????????????????????????????????
? per-date gate verdicts
????????????????????????????????????????????????????????
? C. Eligibility Reporter
? * first as-of date each gate flips to pass ?
? * or "never eligible in window" ?
? * forward return AFTER that date, reported ?
? as consequence, never as selection input ?
????????????????????????????????????????????????????????

```

A. Packet assembler - how a "frozen packet" is built

A packet = one symbol x one as-of date x the full set of scoring inputs, each stamped with the date it was knowable. A cohort = the MU/INTC/SNDK/GME set of symbols. A run = a cohort replayed across a contiguous sequence of as-of dates.

Freezing rules per input type:

- Prices (OHLCV): use provider historical endpoints that accept an explicit end date

(AV TIME_SERIES_DAILY full, FinancialDatasets get_stock_prices with date range, Yahoo/Kite for India). Keep only candles with date <= asOf. Prices are the *easy* input - they're already dated per row and computeEdgeIC already slices them this way.

- Fundamentals: this is the hard one and the reason the harness exists. Do not use

fetchOverview(). Use period-stamped statements: FinancialDatasets get_financial_metrics / get_income_statement etc. carry a report_period and an SEC filing/accepted date. Freeze the *most recent statement whose filing/acceptance date <= asOf* - not whose fiscal period <= asOf (a Q4 result for period ending Dec 31 is not public until the ~Feb filing). If a provider only gives fiscal period and not filing date, add a conservative publication lag (e.g. period_end + 45-75 days) and record that assumption in the packet manifest so the honesty of the freeze is auditable.

- News / sentiment: keep only items with publishedAt <= asOf (FinancialDatasets/AV news

carry timestamps). Sentiment aggregates recomputed from the filtered set only.

- Universe / tradability: each packet carries an explicit tradable, adv, price,

market_cap, spread_ok snapshot as of the date - so the eligible-universe floors from Phase 2 of the fix prompt can be replayed without leaking today's liquidity.

Assembly is one-time and offline. Once a packet is written and hashed it is immutable. Re-running a replay reads frozen packets; it never re-fetches. This also means the same run is byte-for-byte reproducible (a manifest hash proves the inputs didn't move) - matching the dataset_hash discipline already in fitAndStoreCalibration.

B. Sealed replay cursor - the leak-prevention mechanism

The core safety primitive is a SealedDataAccessor bound to a single asOf cursor. Every read of a packet item goes through it, and it throws (does not filter silently) if asked for, or handed, any record whose knowable-date is after asOf:

- Construction: new SealedDataAccessor(asOf, packetItems).

- It pre-partitions items and, on any accessor call, asserts `item.knowable_at <= asOf` for every item it returns. A single violation raises `FutureDataLeakError` and aborts the run.

- It refuses to reach the network at all - it has no provider client. The only way to get

data into it is a pre-frozen packet. This structurally forecloses leak surface #1 (the today-keyed live cache) because the live fetch path is simply not reachable from replay.

- The scoring/calibration functions are called with the accessor's outputs, not raw providers.

`computeWeightedAnalystScore` and `predictPWin` are already pure over their inputs, so no change to them is needed - only the `*source*` of their inputs changes.

Why throw instead of filter: filtering hides bugs. If packet assembly ever mis-stamps a record, a silent filter would drop it and the run would "look fine." A throw turns a latent leak into a loud, testable failure - which is exactly what test 14 asserts and what the fix prompt's "fail loudly" doctrine demands.

The cursor steps forward one trading day at a time across the run window. At each step it re-seals to the new `asOf` and re-runs the gates. Walk-forward calibration inside a step still uses `walkForwardFolds()` with its existing purge/embargo - so there are two layers of time-safety: the outer sealed cursor (no packet item after `asOf` enters the step at all) and the inner purge/embargo (no train label window bleeds into a test fold).

C. Eligibility reporter - the honest output

For each (cohort-symbol or cohort, model, setup, horizon), the reporter walks the per-date verdicts in ascending date order and records the first date the gate passes:

symbol	model	setup	horizon	first_eligible_asof	gate_margin	n_oos	ece
MU	quality_catalyst_moment	breakout	10d	2023-01-19	+0.031	41	0.074
MU	turnaround_inflection	inflection	20d	2022-11-08	+0.012	36	0.088
INTC	turnaround_inflection	inflection	20d	never (window ends 2024-06-30)	-	-	-
SNDK	(delisted 2016; excluded	-	see ?8 open question)				
GME	fast_breakdown_defense	crowding	5d	2021-01-27 (VETO fired, not entry)			

- `first_eligible_asof` is selected by the gate, not by a human. There is no

"pick the date MU doubled" input anywhere in the pipeline.

- Forward return is reported only as a consequence column, computed strictly after

`first_eligible_asof`, and is never fed back into selection, sizing, or gate thresholds within the run. It answers "and then what happened?" - it does not choose the date.

- "never eligible in window" is a first-class, honest result. A model that never clears its

gate on MU across the whole replay window reports `never`, not a fudged partial pass.

5. Proposed storage schema (describe only - DO NOT create)

Four additive tables. Names/shapes are a proposal; the migration is out of scope for this draft and must not be applied without approval (and, per the global schema rule, verified against the target DB before any dependent code ships).

- `replay_packets` - one row per (symbol, as-of date). Columns (proposed):

`id`, `cohort` (text, e.g. `semis_memory_2022`), `symbol`, `market`, `as_of` (date), `manifest_hash` (sha256 over the frozen item set), `publication_lag_assumptions` (jsonb), `created_at`. Immutable after write.

- `replay_packet_items` - the frozen inputs. `id`, `packet_id` (fk), `item_type`

(ohlcv|fundamental|news|universe), `knowable_at` (timestampz - the date this record was public), `source`, `source_tier`, `payload` (jsonb), `payload_hash`. The `knowable_at <= packet.as_of` invariant is what the sealed

accessor enforces at read time and what a DB check/backfill test asserts at write time.

- `replay_eligibility_runs` - one row per replay execution. `id`, `cohort`, `model_kind`, `setup`, `horizon_days`, `window_start`, `window_end`, `packet_manifest_hash` (proves which frozen inputs were used), `code_git_sha`, `created_at`.

- `replay_eligibility_events` - per (run, symbol, as-of) gate verdict. `run_id`, `symbol`, `as_of`, `gate` (`calibration_oos|validation|ic|thin_evidence|breakdown_veto`), `passed` (bool), `margin` (numeric), `n_oos`, `ece`, `detail` (jsonb). The reporter's "first_eligible_asof" is a `MIN(as_of) WHERE passed` query over this table.

Reuse existing tables where possible: labels/returns can still live in `observation_labels`, and the accessor can read `evidence_records` filtered by `effective_at <= as_of` for any inputs already captured there, rather than duplicating them into packets.

6. How it plugs into the existing walk-forward code

- The harness imports `walkForwardFolds` and `loadLabeledDataset` from

`lib/learning/dataset.ts` unchanged - but feeds them observations assembled from sealed packets rather than the live `decision_observations` join.

- It imports `fitCoefficients` / `predictPWin` / `acceptCalibrationOOS` from

`lib/validation/calibration.ts` unchanged. The eligibility verdict per as-of date is literally `acceptCalibrationOOS(...).accepted` computed on the packet-sealed folds.

- It imports `computeWeightedAnalystScore` from `lib/scoring/weighted-score.ts` so the replay's `analyst_score` is identical to production's.

- The only *new* code is: (a) the packet assembler, (b) `SealedDataAccessor` +

`FutureDataLeakError`, (c) the reporter, (d) the cohort fixtures. The gates themselves are reused verbatim - this is what makes the report trustworthy (it tests the real gates, not a reimplementation, satisfying the fix prompt's "call the actual services, not duplicate pure logic" rule).

7. Phased build plan & effort

Phase	Deliverable	Effort (ideal-dev-days)
P0	- sealed accessor + leak test <code>`SealedDataAccessor`</code> , <code>`FutureDataLeakError`</code> , and a unit test proving a deliberately future-stamped item throws (this alone satisfies test 14's spirit). Pure, no network, no DB.	1.0
P1	- packet assembler (prices + news) Offline builder for OHLCV and news packets from historical provider endpoints; manifest hashing; immutability. Prices/news are date-stamped so this is mostly plumbing.	1.5
P2	- fundamentals as-of Filing-date-based fundamental freezing incl. the publication-lag assumption + manifest recording. The hard, high-value phase.	2.0
P3	- replay cursor + reporter Step-forward cursor wiring the sealed accessor into the reused gates; <code>`replay_eligibility_events`</code> writes; first-eligible query + report table/JSON.	2.0
P4	- cohort fixtures + test 15 MU/INTC/GME (and SNDK caveat) fixtures; the actual <code>`tests/replay-eligibility.test.ts`</code> asserting each model's first-eligible date is gate-selected and no future data is touched.	1.0
P5	- schema migration (gated) The four tables, additive, applied only after owner approval + target-DB verification per the global schema rule.	0.5
Total		**~8 dev-days**

P0 is independently valuable and low-risk - it can land and satisfy the test-14 guard before the heavier packet work. Recommend building P0->P4 as offline/measure-only (no money path touched, consistent with the fix prompt's "shadow/measure-only first" doctrine); P5 last.

8. Risks / open questions

- Fundamental publication dates. The whole honesty of the fundamentals freeze rests on knowing the **filing/acceptance** date, not the fiscal period. If the chosen provider only exposes fiscal period, the publication-lag heuristic is an assumption that must be visible in every report. Open question: does `FinancialDatasets.get_filings/get_income_statement` reliably return an accepted/filed date for the 2020-2023 window for these symbols? Needs a spike before P2.
- SNDK (SanDisk) was delisted (acquired by Western Digital, 2016). Historical data for a delisted ticker is spotty and the current providers/universe don't carry it. Options: (a) drop SNDK and document why, (b) source a one-off frozen CSV for it, (c) substitute a still-listed memory/NAND name (WDC/MU) and note the substitution. Owner decision needed. The requirement says "SNDK-**style**", so a documented substitution is likely acceptable.
- GME is an exit/veto case, not an entry case. The honest expected result for GME is that the fast-breakdown/crowding defense (fix prompt Phase 2 ?4) fires a veto, not that an entry model becomes "eligible." The reporter must represent veto-fired distinctly from entry-eligible so GME isn't scored as a missed long.
- Small-n calibration on a 4-symbol cohort. `acceptCalibrationOOS` needs ≥ 30 OOS rows.
A single symbol across a replay window won't reach that from its own observations. The cohort/universe must be broad enough at each as-of date to produce a real holdout - i.e. the replay universe is the **point-in-time liquid set**, and the named symbols are the ones we **report** on, not the entire training set. This must be designed in from P3.
- Corporate actions (splits/dividends) inside the window. Frozen prices must be split-adjusted **as known at each as-of date**, which is subtle (a split that happens after asOf must not be retro-applied to pre-asOf candles). Needs explicit handling in P1.
- Cost/rate limits of historical backfill. Assembling multi-year daily packets for a cohort burns provider budget (`provider-fetch.ts` daily caps). Assembly should be a throttled, resumable one-time job, not a cron.

9. What this harness does NOT do

- It is not a backtester and does not simulate P&L, fills, slippage, or position sizing.
It answers one question: **when would each model have become eligible?** Realized-return backtesting is the Validation Engine's job (`lib/validators/backtest.ts`); this harness only reports the forward return **after** the eligibility date as a read-only consequence.
- It does not touch the money path. No orders, no `strategy_config`, no live/paper execution, no autonomous flags. Consistent with the fix prompt: measure/shadow only.
- It does not promote or demote any model. Eligibility here is a historical diagnostic, not a live promotion signal. Promotion stays governed by the existing Validation Engine + owner review (`docs/arch/09-learning-loop.md`).
- It does not fix survivorship bias in the live universe (`lib/edges/universe.ts`). It works around it **for the replay** by carrying per-packet universe membership, but building a true versioned PIT index membership (fix prompt Phase 2 ?2) is a separate effort.
- It does not hand-pick outcome dates - by construction. The absence of any "future return

date" input is the feature, and P0's leak test is what proves it.

- It does not re-fetch on replay. Once packets are frozen, a run is a pure function of frozen inputs + gate code, reproducible via the manifest hash.

10. Docs to update when this ships (per CLAUDE.md)

- docs/arch/09-learning-loop.md - new "Historical Replay Harness" section under Validation.
- docs/arch/04-database-schema.md - the four replay_* tables (only after migration applied and verified).
- public/agent-diagrams/system-map.json - only if the harness becomes a scheduled/agent flow (currently proposed as an offline tool, so likely no system-map change).
- This file: flip STATUS to approved and record the approval date.

holding risk daily - Feature Architecture

Source: features\holding-risk-daily\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature: Daily Per-Holding Risk Analytics

Status: SHIPPED - current formula hr -v3 Last updated: 2026-07-21 Owner: Vaibhav Update this file when: the per-holding risk score formula, the strategy-decision rules, the daily cron cadence, or the snapshot schema changes.

Sector-cap breach allocation (hr -v1 -> hr -v2, 2026-07-16): the posture rule

below ("hard concentration/cluster breach -> trim") was wrong for the SECTOR cap.

A sector breach is a property of the sector, not of any name in it, so it gave

every holding in the sector the identical trim with no size. The per-name

allocator - which names absorb the breach, how much each gives up, and why a name

was NOT selected - is specified in

features/risk-sector-breach-allocation/FEATURE_ARCHITECTURE.md, which is the

authority for that rule, its NAV denominator, and the read-only advisory labelling.

This file remains the authority for the score, the cron, the snapshot schema, and

the LLM prose boundary. No migration; formula_version carries the change.

Account-intent correction (hr -v2 -> hr -v3, 2026-07-21): a generic Kairos

trading cap is not an approved sell mandate for a long-term/read-only account.

Concentration breaches on every live account now produce review, never trim.

Broad asset-class sleeves (Diversified Equity, International Equity, Fixed

Income, Commodities, Digital Assets) are not equity sectors and are excluded

from max_sector_exposure_pct. Correlation remains measured but produces

review until a deterministic allocator identifies the holding and quantity.

Having an order path is not proof of an account-specific concentration mandate.

A future trim requires both that mandate and a deterministic executable share-

quantity plan. Protective-stop/thesis exit_review precedence is unchanged. No

schema or order-path change.

Stored hr -v1/hr -v2 rows remain immutable audit evidence, but current Risk

Analytics and newsletter read surfaces expose their original posture separately

and render legacy trim as review with an explicit historical-reason label.

Intent (owner, verbatim)

Canonical account labels (2026-07-21): broker + account_id remains the stable identity and join key. The latest verified live_account_snapshots.nickname is presentation metadata only. Live Portfolio and Risk Analytics render <nickname> ? <broker> <last4>; if the nickname is unavailable they render <broker> <last4>. The read API enriches historical runs at response time, so immutable risk evidence is never rewritten when an account is renamed. Full account IDs remain authenticated API request keys but are never rendered as visible account labels; raw snapshot rows are not returned. Nickname lookup failure degrades to the masked fallback without blocking risk results.

Risk analytics on left panel - add live OTHER accounts, each holding's risk analytics, score, and details of risk and what should be our strategy for each one of them, updated everyday.

Decisions (approved 2026-07-11)

- Strategy source: Hybrid. A deterministic engine computes the per-holding risk score (0-100) AND the risk posture (hold | review | trim | exit_review). A risk-only engine must not recommend adding capital: add requires a separate current, actionable, deterministic alpha signal plus the existing portfolio/execution gates. The risk page may display add_capacity=true (risk capacity exists), but that is not an add recommendation. An LLM writes only the human-readable explanation around that fixed decision - it cannot change the score or the action. Complies with PROJECT_DECISIONS ("deterministic services calculate risk; LLMs may propose/explain") and the safety rule ("LLMs may not control money limits, accounts, promotion, order submission").
- Scope: all live accounts. Every Robinhood account (Trading 605420660, read-only 965848641, any others) + Kite India. The strategy line is advisory for all accounts and wired to NOTHING - it never reaches the order path. Only 605420660 can ever act on a call, and only via the existing owner-click Execution Gateway.

Non-goals / guardrails

- No auto-execution. Strategy text is display-only.
- New positions long-only; exit/trim calls are advisory, not orders.
- Additive migration only; daily snapshots are an append-only history table.
- No cross-currency roll-up. USD and INR accounts are scored and trended independently; the feature never adds USD market value to INR market value.
- A score is a risk-control pressure index, not a probability of loss and not a return forecast. UI copy must not present it as predictive confidence.
- Missing/stale inputs produce insufficient_data or a lower

data_confidence; the engine must not replace unavailable beta, correlation, sector, quote, or cost-basis evidence with a confident neutral score.

- Unrealized loss alone never creates an exit_review. Selling solely because a

holding is down is a loss-chasing rule. Exit review requires a deterministic protective-stop/thesis-break/hard-risk-breach reason with its evidence recorded.

- Read-only accounts stay read-only. Without an approved account-specific

objective/cap mandate, concentration produces review, not a sell instruction. A verified protective-stop/thesis break may still produce advisory exit_review.

Data model (additive)

holding_risk_runs (append-only, one row per market/account computation):

- id uuid, run_key text unique, market, currency, broker,

account_id, account_label, status (running|complete|failed|partial)

- source_captured_at, started_at, completed_at, formula_version,

input_hash, data_confidence, missing_inputs text[], error

- run_key is deterministic for `market x account x local trading date x

formula_version x input_hash`; concurrent/retried crons claim it through a unique insert. Do not use check-then-insert idempotency.

holding_risk_snapshots (append-only, one row per run x holding):

- id, run_id uuid FK holding_risk_runs(id), captured_on date,

market, currency, broker, source, source_captured_at, account_id, account_label

- symbol, sector, qty, current_price, average_cost,

market_value, weight_pct, beta, realized_vol_pct, unrealized_pnl_pct

- holding_risk_score int, risk_label, risk_drivers jsonb (deterministic detail)

- risk_posture text (hold|review|trim|exit_review),

action_reason text (deterministic), add_capacity boolean (never an order signal)

- data_confidence numeric, missing_inputs text[], formula_version text

- strategy_note text (LLM prose, nullable - populated best-effort, never blocks)

- created_at

account_risk_snapshots (new append-only table, one row per holding_risk_runs.id): the existing in-code RiskMetrics roll-up persisted so the page can show ?-vs-prior-comparable-run and a trend. This table does not exist today; "existing" refers only to the TypeScript RiskMetrics contract. Store currency, source_captured_at, formula_version, data_confidence, and missing_inputs here too.

Schema rules:

- Unique (run_id, symbol) for holding rows and unique run_id for account

rows. Tables have INSERT-only service-role policies plus UPDATE/DELETE-blocking triggers. Reruns insert a new run only when inputs/formula changed; they never upsert or rewrite a prior snapshot.

- Owner-authenticated SELECT only; no anon access. Account identifiers and

positions are sensitive.

- Numeric checks: `finite, market_value >= 0, 0 <= weight_pct <= 1,`
`0 <= score <= 100, 0 <= data_confidence <= 1.`
- No cascade that can delete snapshot history. A failed run remains as evidence.

Compute path

- Add a pure, versioned module `lib/risk/holding-risk.ts` with
`computeHoldingRisk(holding, ctx) -> { score, label, drivers[], riskPosture, actionReason,`
`addCapacity, dataConfidence, missingInputs }`. Pure function; `ctx` carries account/portfolio totals,
owner-approved risk limits, sector exposure, correlation evidence, beta/volatility, quote freshness, and data availability. Do
not enlarge `lib/portfolio-risk.ts` further; reuse its types/calculations where sound, but keep daily snapshot logic
isolated and testable.
- V1 score is formula-versioned and based on limit utilization, not
uncalibrated "TBD" weights:
 - name concentration: 30 points, scaled against
`max_name_exposure_pct;`
 - sector concentration contribution: 20 points, scaled against
`max_sector_exposure_pct;`
 - volatility/beta contribution: 15 points, scaled against the portfolio
volatility/beta budget, only when real price evidence is available;
 - correlated-cluster contribution: 15 points, from computed aligned-return
correlations, never the small static `KNOWN_CORR` map alone;
 - drawdown/stop-distance pressure: 10 points; drawdown alone cannot trigger exit;
 - liquidity/event/idiosyncratic flags: 10 points when supported by fresh data.
- Each component is clamped 0-its cap and recorded in `risk_drivers`.
 - If required structural fields (`qty`, `price`, `market value`, `account total`,
currency) are missing/non-finite, return `insufficient_data` with no numeric score. Optional missing dimensions reduce
`data_confidence` and are excluded from the numerator without renormalizing the score to appear fully confident.
 - Posture precedence: verified protective-stop/thesis-break ->
`exit_review`; direct-name, allocated real-sector, and correlated-cluster concentration produce review for every account
because global references are not account-specific sell mandates; incomplete or conflicting evidence -> review; otherwise
hold. `add_capacity` means only that owner-approved risk limits have room.
 - New cron POST `/api/agents/holding-risk` (cron-secret gated):
 - accept only `market=us|india`; use verified market calendars and the
market-local completed session date;
 - acquire the append-only run claim before external/LLM work;
 - fetch all live accounts (reuse `fetchRobinhoodBrokerAccounts` / Kite), but

persist only broker-verified account identities. Today `fetchKiteBrokerAccount()` reports placeholder `accountId="kite_india"`; it must instead return the verified Kite user_id that matches `active_account_india`, or the run fails closed for that account;

- Robinhood source (shipped 4f0bec7, 2026-07-12): `fetchRobinhoodBrokerAccounts`

no longer uses the classic REST client (`api.robinhood.com` 401s rejected `client_id` for the MCP-scoped vault token - it never returned holdings). It now calls `captureAllRobinhoodAccounts()` in `lib/robinhood-mcp.ts`: one agentic-MCP session, `get_accounts` enumerates all 6 RH accounts, per-account `get_equity_positions` + `get_portfolio`, and a batched `get_equity_quotes` over deduped held symbols to price holdings. `agentic_allowed=false` gates order placement only, not reads, so all 6 accounts feed risk while order placement stays restricted to 605420660. The same capture backs `refreshViaMcp()` (all 6 accounts upserted into `live_account_snapshots`, not only the active one);

- capture one coherent, bounded input snapshot per account. Never combine a

Robinhood account timestamp with another account's holdings or a later quote; record source timestamps and reject/mark stale inputs;

- compute per-holding + per-account risk deterministically,

- LLM pass (hybrid): given the fixed score+action, write `strategy_note` per

holding. The prompt contains only structured facts; output is length-bounded, schema-validated, and cannot introduce new numbers/actions. Batch notes to control cost. Best-effort; empty note on LLM failure never blocks the row.

- insert snapshots and mark the run complete in one database transaction/RPC.

Never upsert append-only snapshot rows. Partial DB failure marks the run failed/partial and does not publish it as latest.

- Keep `GET /api/portfolio/holdings` unchanged for live view. Add owner-gated

`GET /api/portfolio/risk-daily?market=us|india&accountId=...` that returns the latest complete run plus the previous complete run with the same account/market/currency/formula version. Do not compare across a formula change.

UI (PortfolioRiskPage.tsx)

- Holdings table gains columns: Risk (score chip, colored) and Action

(hold/review/trim/exit-review pill). `insufficient_data` is shown explicitly, not as risk 0.

- Row expander shows `risk_drivers` bullets + `strategy_note` prose + ? vs

yesterday.

- Header cadence flips as -needed -> daily with "Last computed: <date>" from the

snapshot, the source-data as-of time, formula version, and confidence. The live Refresh remains separate and must not silently overwrite the daily historical run.

- Account tabs never aggregate currencies. Read-only accounts show

"advisory only"; the agentic trading account still offers no order button from this feature.

- Account tabs use the canonical presentation label above. Never use a transport-

adapter fallback label or expose a full broker account number in UI copy.

Cron

holding-risk - daily, per market (?market=us, ?market=india), after the respective exchange close and after the authoritative account snapshot refresh. If the refresh is absent/stale, publish a failed/insufficient-data run, not yesterday's risk as today's. The route is owner-or-valid-cron-secret gated, concurrent runs are serialized by the DB run claim, and provider/LLM failures are recorded per account. Exact times and dependency ordering go in docs/arch/05-crons-and-scheduling.md on ship.

Failure and safety behavior

- Account enumeration/config/profile error: fail that account closed; do not substitute another account or the combined portfolio.
- Stale quote/account snapshot: no actionable posture; insufficient_data.
- Missing sector/beta/correlation: lower confidence and list the missing inputs; never silently use "Other", beta 1.0, or zero correlation as verified fact.
- LLM unavailable/invalid: persist deterministic row with null note.
- Snapshot insert/transaction failure: run is failed and is not returned as latest.
- Duplicate cron/retry: unique run claim returns the existing run; no duplicate holding rows and no mutation.
- Formula change: new formula_version; trends/deltas are not compared across versions without an explicit compatibility transform.
- Corporate actions/symbol changes: preserve broker symbol and normalized symbol, with the raw broker payload hash/provenance; never infer P&L across a split from unadjusted values.

Required verification before ship

- Pure formula tests at every threshold and with NaN/Infinity/null inputs.
- US and India fixtures proving no currency/account cross-contamination.
- Duplicate/concurrent cron test and transaction rollback test.
- Stale/missing broker snapshot and provider failure tests.
- Kite token user-id mismatch test; unknown Robinhood account remains advisory.
- Append-only trigger tests (UPDATE/DELETE rejected) and owner-only RLS tests.
- LLM output cannot alter score, posture, drivers, symbols, or numbers.
- Clean migration replay, tsc --noEmit, full unit suite, build, and authenticated Risk page smoke test.

Cross-doc updates required on ship (same commit)

- docs/arch/05-crons-and-scheduling.md - new daily cron.
- docs/arch/08-risk-and-safety.md - advisory-only strategy line, read-only scope.
- docs/arch/04-database-schema.md - two new tables.
- public/agent-diagrams/system-map.json - new HoldingRisk node + history entry.

- Migration verified applied to prod DB before the reading code ships.

Reviewer changelog (ChatGPT)

- Corrected the append-only/upsert contradiction by adding run-level idempotency, insert-only snapshots, immutable triggers, and a publish-only-complete rule.
- Split risk capacity from alpha: removed risk-only add recommendations and prohibited loss-only exit calls.
- Defined V1 score semantics/components so the implementation is not left to arbitrary thresholds; added confidence/abstention for missing data.
- Added market/currency/account/source timestamps and prohibited USD/INR or cross-account aggregation.
- Corrected the Kite reuse assumption: current broker code uses the placeholder `kite_india`; daily risk must use the broker-verified Kite `user_id`.
- Added coherent source snapshots, stale-data behavior, transactional publication, concurrent cron handling, formula versioning, security/RLS, failure modes, and acceptance tests.

hybrid stop - Feature Architecture

Source: `features\hybrid-stop\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Hybrid Protective Stops

Status: P1 implemented: read-only coverage and autonomous-entry interlock.

Broker placement/cancel/replace remains deliberately unimplemented and disabled.

Scope: US and India live positions. Paper remains unchanged unless separately approved.

Current status and phasing (2026-07-30)

The production schema was checked directly on 2026-07-30. `protective_orders`, `protective_order_events`, and `strategy_config.protective_orders_enabled` exist; the flag is false and there are zero protective-order rows. Migration presence is not treated as deployment proof.

P1 is active but cannot send broker traffic:

- `lib/protective/coverage.ts` accepts protection only when exactly one matching broker order is active, unexpired, has exactly one broker identifier, and has protected and reconciled quantities that exactly match the reconstructed holding. Partial, oversized, duplicate, expired, or unreconciled records are unprotected; an oversized SELL floor could create a short after a trigger. The aggregate coverage result must also be explicitly true: unknown/inconsistent aggregate state with no per-position detail still blocks entry.

- `runAutonomousLive()` blocks new autonomous live BUYs before provider work or a

proposal write unless the feature flag, a future broker-specific placement and reconciliation worker, and full coverage for all managed live positions pass.

- The worker availability is a source-level false constant. A Settings, database,

environment, or LLM change cannot turn it on.

The existing-position interlock is necessary but not sufficient for P2. A future BUY workflow must prove how the newly filled quantity becomes broker-protected without an unbounded gap; merely changing the source constant to true is not an approved placement implementation.

P1 does not place, amend, cancel, or query a broker order. It does not alter PaperTrader, PositionMonitor, manual live orders, scoring, learning, or the existing synthetic live-exit monitor. Broker-hosted protection remains a separate P2 activation and canary decision.

Shadow scaffold status (2026-07-18)

The owner approved BUILDING the shadow scaffold - everything UP TO the placement line, and it STOPS there. No live broker order is placed. Not a \$1 test. Ever. The spec below remains authoritative for the eventual live design; this section records what shipped as inert scaffolding.

Built (all shadow / no placement), under `lib/protective/`:

- `capabilities.ts` - the broker-neutral `BrokerProtectiveCapabilities` matrix + `evaluateProtection()` (capability-driven; no flat broker boolean; no eligible multi-day order -> unprotected-by-broker; GTC ? every-session triggering).
- `kite-capabilities.ts` - Kite's capability declared from the PROVEN GTT code

(gtt_limit weaker protection; DAY-only SL-M rejected as a multi-day floor).

- disaster-floor.ts - pure computeDisasterFloor({mode, distance, ...}). Default

mode = wider_disaster_floor (Q1 unanswered); distance is a config input, no hardcoded value; monotonic ratchet (a falling high-water mark can't lower it).

- reconcile.ts - pure reconcileProtectiveOrder() (out-of-band triggers,

partial fills, cancels, expiry, broker edits, corp-action drift; unknown -> needs_reconcile; trigger-without-fill never closes the book).

- state.ts - the protective_order record shape + status machine + exit

provenance(protective_disaster_floor / learning_scope = risk_policy_only)

- long-only / cancel-before-replace invariants.

- placement-gate.ts - THE MONEY LINE. False-by-default

protective_orders_enabled flag gates ALL placement and stays false; planProtectivePlacement() never calls a broker.

Schema (applied, still inert): supabase/migrations/20260718000000_protective_orders_shadow.sql, hardened by 20260718130000_harden_protective_orders_invariants.sql. The schema is only a ledger/control plane until a separately approved P2 broker worker exists.

Tests: tests/protective-hybrid-stop.test.ts - all 14 spec acceptance tests, each falsifiable (mutation-verified).

Still gated behind owner approval before anything goes live: Q1 (touch semantics), floor distance, ratchet step/cadence, and the Kite-first vs Webull-first build sequence - see "Open Owner Decisions" below.

Decision

Kairos currently owns close-based stop, target, trailing, thesis, and time exits. A broker-hosted order changes an analytical close-based rule into an intraday-touch execution rule. That is a strategy change, not merely outage protection.

The recommended design is therefore a wider disaster floor, not a duplicate of the analytical stop:

- Kairos remains authoritative for analytical exits and the trailing ratchet.
- The broker holds a wider, static protective floor for app or scheduler outages.
- Broker fills are reconciled back into the Performance Truth Layer.
- A broker floor may only move upward and may never increase risk.

Do not activate placement until the owner explicitly approves touch semantics, floor distance, post-fill policy, and the relevant broker-specific P2 canary.

Verified Broker Capabilities

Broker	Verified capability	Constraint	Design consequence
Kite	GTT `single`/`two-leg` trigger with child orders represented as `LIMIT`	A trigger creates a limit order; a gap can leave it unfilled. GTT can be modified with `PUT` and returns an expiry.	Use a stop-limit disaster floor, modify in place, and surface unfilled-trigger risk.
Kite regular order	`MARKET`, `LIMIT`, `SL`, `SL-M`; DAY/IOC/TTL validity	A regular protective order is not a multi-day substitute for GTT.	Do not renew DAY stops as hidden cron state.
Robinhood agent API	Existing captured tool schema is insufficient proof for a new stop integration	Public support pages do not define Kairos's current MCP order arguments.	Require a fresh live `tools/list` schema capture before implementation.
Webull	Trading API documents stop orders; Cloud MCP is query-only	Trading API is a different signed integration. Trailing-stop orders are DAY-only in the documented stock API.	Out of scope; never infer MCP writes.

Broker-Neutral Capability Matrix

Protection is driven by a broker-neutral state machine that reads declared capabilities per adapter - never by per-broker if branches in the protocol. The credential store (`api_key_vault`) is generic; broker behavior is conditional, so each adapter declares only combinations it has proved.

```
BrokerProtectiveCapabilities {
  scope: {
    market: Market
    instrumentTypes: InstrumentType[]
    sides: ("sell_long")[]
    accountModes: AccountMode[]
  }
  orders: Partial<Record<"stop_market" | "stop_limit" | "gtt_limit", {
    timeInForce: ("day" | "gtc" | "broker_managed")[]
    sessions: ("regular" | "extended" | "overnight")[]
    updateMode: "modify_in_place" | "cancel_replace" | "none"
    maxLifetimeDays: number | null
    oco: boolean
  }>>
}
```

- A flat broker-level boolean is forbidden. Support varies by order type, instrument, session, TIF, account entitlement, and environment. For example, Webull documents a GTC stop-market in CORE, but that does not prove the same stop works in extended or overnight sessions.
- GTC means the order persists across trading days; it does not mean the order can trigger in every session. Session eligibility is separate.
- An adapter with no eligible multi-day order for the exact position is declared synthetic-only - the position is flagged unprotected-by-broker, never silently treated as protected.
- Preference order: stop-market where verified (designed to trigger a market order, without a guaranteed execution or fill price) -> GTT/stop-limit as weaker protection with unfilled-trigger risk surfaced in the UI.
- When a broker adds a verified capability, its adapter updates the matrix after a fresh schema capture; the shared protocol is unchanged.

Protection Is Mitigation, Not a Guarantee

State this honestly in code comments and the UI - do not let it read as "loss protection":

- A stop-market is designed to trigger a market order, but neither execution nor price is guaranteed during gaps, halts, venue/broker outages, or rejected orders. A floor at \$90 can fill at \$75 or remain unresolved.
 - A stop-limit / GTT limit child controls price but may never fill on a gap -> the position stays unprotected, which must be reported, not called filled.
 - Stops generally cannot execute while the session is closed. A resting floor does not cover the gap between close and the next open.
- The honest label is outage + catastrophic-loss mitigation, not guaranteed loss protection.

Required State Model

One authoritative `protective_order` record per live position and broker:

- position_id, broker_account_id, market, symbol
- broker_order_id or Kite trigger_id
- mode = disaster_floor
- analytical stop, broker floor, trigger and limit prices
- status: placing, active, triggered, filled, canceling, canceled, failed, needs_reconcile
- broker version/update timestamp, expiry, last reconciliation timestamp
- immutable reason and correlation/idempotency id

The record references the existing position and execution ledger. It must not create a parallel position, cash, or P&L truth layer.

Exit provenance. A disaster-floor fill closes the position with `exit_reason = protective_disaster_floor`, DISTINCT from every strategy exit (stop, target, trailing, thesis, time). The Learner excludes it from signal-weight attribution and genome promotion, while the Performance Truth Layer, NAV, realized P&L, drawdown, mandate evaluation, and risk-policy evaluation MUST include the loss. A generic `excluded_from_learning = true` is insufficient because the existing evaluation engine also filters that field. Use explicit attribution such as `learning_scope = risk_policy_only` (exact schema decided during implementation). Broker execution quality is evaluated separately from the strategy signal that originally opened the position.

Execution Protocol

Entry

- Pass all existing money-path gates and place the live BUY.
- Reconcile the BUY fill and held quantity.
- Compute a deterministic disaster floor from the approved mandate.
- Reserve one protective-order intent using a stable idempotency key.
- Place the broker floor and persist its broker id and expiry.
- If placement, persistence, or immediate read-after-write reconciliation fails,

create a critical health issue, mark the position unprotected, and latch a broker-account + market entry circuit breaker. Existing exits and risk-reducing protection writes remain enabled. Clear the latch only after a later reconciliation proves the correct active protection and quantity.

Ratchet

- Only update when the approved floor has risen by the minimum step.
- Kite uses modify-in-place for the existing GTT where possible.
- Any cancel/replace broker must confirm cancellation before replacement.
- Never lower a floor and never leave two sell orders capable of exceeding the reconciled holding.

Explicit Exit

- Read and lock the current protective-order state.
- Cancel the resting protection and confirm cancellation.

- Re-read reconciled held quantity.
- Submit a SELL no larger than that quantity.
- If cancellation cannot be confirmed, fail closed and alert. Do not submit a

competing SELL that could create an accidental short.

The current standalone Kite path rejects a partial explicit SELL when a resting GTT exists because it cannot atomically rebuild protection for the residual.

The standalone Kite route now follows this cancel-before-sell rule; a future shared implementation must reuse the protocol rather than add a second path.

Reconciliation

Every live snapshot must detect out-of-band triggers, partial fills, cancellations, expiry, and broker-side edits. Reconciliation closes or adjusts the Kairos position using actual fills and appends execution evidence. Unknown state is `needs_reconcile`, never assumed active or canceled.

Corporate actions and broker expiry are first-class reconciliation inputs. A split, symbol change, delisting, stale instrument id, or approaching GTC/GTT expiry invalidates the assumed price/quantity and requires a fresh broker snapshot before replacement. Renewal uses the same cancel/replace quantity invariant and may not create overlapping sell capacity.

Safety Invariants

- No LLM computes, places, modifies, or cancels protection.
- US and India accounts, cash, orders, and quantities never cross.
- Existing pause, market controls, kill switch, drawdown breaker, approval mode, notional limits, and account allowlist all remain upstream of new entries.
- Entry halts never block explicit exits, reconciliation, placement of missing protection on an already-held position, or an upward risk-reducing ratchet. They block lowering/removing protection except inside the locked explicit-exit or verified replacement protocol.
- Total executable SELL quantity can never exceed reconciled held quantity.
- A trigger without a confirmed fill does not close the book.
- A gap through a Kite limit child is reported as unprotected, not called filled.
- Expired protection is a critical health state.
- Paper and live outcomes are not mixed across different exit semantics.

Acceptance Tests

- A cancel failure prevents the explicit SELL.
- A replace cannot create two executable protective orders.
- A falling high-water mark cannot lower the floor.
- A Kite trigger with no fill remains open/unprotected and raises an issue.
- An out-of-band fill reconciles with actual quantity and price.
- Partial fills leave the correct residual protection.
- Expiry and broker-side cancellation are detected.

- Every pause/kill/account/approval gate blocks new entries; an entry halt cannot block exits or risk-reducing protection repair on an existing position.
- US and India fixtures cannot read or mutate the other market.
- Paper results remain on their existing close-based semantics.
- A disaster-floor fill records `exit_reason = protective_disaster_floor`, is excluded from signal-weight attribution, and remains included in P&L, drawdown, mandate, and protection-policy evaluation.
- An adapter with no eligible multi-day protective order yields an unprotected-by-broker position, never a silently-protected one.
- Capability fixtures reject unsupported order-type/TIF/session/account combinations; GTC alone never implies extended-hours triggering.
- Split, expiry, partial-fill, and broker-edit fixtures cannot leave aggregate executable SELL quantity above the reconciled holding.

Open Owner Decisions

- Approve broker-hosted touch execution at all?
- Disaster-floor distance: fixed percentage beyond analytical stop, volatility multiple, or mandate-specific value? Recommendation: mandate-specific ATR rule with a hard maximum loss bound.
- Minimum ratchet step and update cadence.
- Whether paper receives a separately versioned touch-fill simulation before live.
- Build sequence - genuine tension, owner call:
- **Kite-first** (this spec's build order): lifts already-written, proven GTT code into the shared path and delivers India protection without waiting for a new broker entitlement.
- **Webull-first** (Codex's recommendation): build the signed `webull_trade` adapter as a reference against a richer capability set + a sandbox, but delays any floor until Webull entitlement and lifecycle proof are complete. The tradeoff is ship-protection-soonest vs build-the-abstraction-right-first. My lean: adopt the capability-flag interface up front (from Codex) but implement Kite first against it, since its code is proven and it delivers protection before the Webull entitlement/sandbox path is even confirmed.

Build Order After Approval

- Capture current broker schemas and write failure-injection contract tests.
- Add protective-order state and reconciliation without placing orders.
- Shadow the computed floor and broker capability status.
- Implement Kite sandbox/manual path first.
- Implement a verified Robinhood path only after live schema capture.
- Run a single owner-approved small live test per market.
- Keep Webull in its separate feature and disabled.

Primary Sources

- Kite GTT lifecycle and LIMIT child orders: <https://kite.trade/docs/connect/v3/gtt/>
- Webull stock order types, TIF, sessions, and examples: <https://developer.webull.com/apis/docs/trade-api/stock/>
- Robinhood's public agent tool inventory (schema still requires live capture):
<https://robinhood.com/us/en/support/articles/trading-with-your-agent/>

india markets parity - Feature Architecture

Source: `features\india-markets-parity\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

India Markets Verification And Hardening

Status: REVIEWED DESIGN DRAFT. NOT APPROVED FOR IMPLEMENTATION.

Last reviewed: 2026-07-15 by Codex.

Scope: India Markets page correctness and resilience; no scoring or trading.

1. Correct Current-State Assessment

India Markets is implemented, but not yet production-proven:

- MarketsPage displays NIFTY 50, SENSEX, BANK NIFTY, India VIX, and ten sector-index tiles.
- IndiaBreadth fetches only the first ten fixed NIFTY-50 constituents. It is a sample, not NIFTY-50 market breadth.
- The client component imports `lib/india-data.ts` and calls Yahoo directly.

Browser-side Yahoo access cannot rely on the server-only User-Agent header or Next.js revalidation semantics and bypasses Router pacing, durable caching, provenance, and health telemetry.

- IndiaGapNote conflates a transient India-source failure with a structural absence of an India equivalent.
- The page heading remains US-specific when India is selected.

The feature is therefore a verification and hardening project, not a greenfield parity build. Sector and breadth UI must not be built twice.

2. Invariants

- The browser never calls Yahoo, NSE, RBI, Kite, Upstox, or a provider directly.
- US/USD and India/INR snapshots, caches, labels, and health are isolated.
- Missing or partial data is displayed as unavailable/partial, never zero or a fabricated neutral market state.
- Markets remains advisory. No tile changes scores, sizing, orders, or exits.
- An unofficial source is capability-probed, cached, paced, and fail-soft; it is never described as guaranteed, unlimited, or an authoritative NSE feed.
- Structural product gaps and transient operational failures use different UI states and System Health codes.

3. Target Contract

Create one owner/cron-gated server boundary that returns a frozen India market snapshot. It must consume Canonical Evidence Router contracts or a temporary compatibility adapter that is explicitly retired at Router cutover.

```
type IndiaMarketsSnapshot = {
  market: "india";
  currency: "INR";
  asOf: string;
  fetchedAt: string;
  status: "complete" | "partial" | "unavailable";
  policyVersionId: string;
  indices: Array<{
    symbol: string;
    label: string;
    price: number;
    changePct: number;
    observedAt: string;
    source: string;
    quality: "fresh" | "stale";
  }>;
  sectors: Array<{
    symbol: string;
    label: string;
    price: number;
    changePct: number;
    observedAt: string;
    source: string;
    quality: "fresh" | "stale";
  }>;
  breadth: null | {
    universeId: "nifty_50";
    universeAsOf: string;
    eligibleN: number;
    resolvedN: number;
    advanced: number;
    declined: number;
    unchanged: number;
    unavailable: number;
    coveragePct: number;
    quality: "complete" | "partial";
  };
  unavailable: Array<{ component: string; reasonCode: string }>;
};
```

The response contains no raw provider payload, token-bearing error, URL, cookie, or arbitrary prose. Numeric fields must be finite and currency/basis validated.

4. Data Acquisition

- Use server-side, code-allowlisted adapters only.
- Prefer a fresh durable cache; serve bounded stale data with an explicit timestamp when allowed; enqueue bounded refresh work rather than burst on page load.
- Fetch index/sector symbols with bounded concurrency and atomic provider pacing.
- Use Kite/Upstox only when the owner's entitlement and contract allow it. Provider selection is policy-controlled, not hardcoded in the component.
- Yahoo is an unofficial fallback. Contract-test symbol semantics and adjustment basis; treat CORS, throttling, schema drift, and no-data as expected failures.
- Never scrape arbitrary NSE/RBI pages from a client or silently change source.

5. Breadth Correctness

Choose one honest product:

- Full NIFTY-50 breadth: use a versioned, current constituent snapshot; resolve

all eligible names from a common observation window; show breadth only at a pre-approved coverage floor and report resolvedN/eligibleN.

- Ten-name sample: retain current behavior but rename it "10-name NIFTY sample,"

disclose coverage, and never use the word breadth or compare it with full US breadth.

Recommended target is full NIFTY-50 breadth built asynchronously from cache, not 50 provider calls during page render. Constituents need effective dates so historical replay does not use today's index membership. Advance/decline compares the current session's valid reference close on a consistent adjustment basis; unchanged and unavailable names remain in the denominator report.

6. Product States

- loading: bounded skeleton while the server request is active.
- complete: data plus asOf/source-quality summary.
- partial: render available rows and explicit coverage; no healthy-looking total.
- temporarily_unavailable: India capability exists but failed/staled out.
- not_supported: no approved India equivalent exists.

The India page heading and explanatory copy name India instruments. Leveraged bull/ bear pairs remain not_supported; they must not be synthesized. TradingView embeds are optional third-party display surfaces, not a parity requirement or evidence.

7. Macro And Regime Boundary

Do not build an "India recession sentinel" in this feature. RBI policy rate, CPI, WPI, IIP, yields, INR, and FII/DII flows differ in release cadence, revision, economic meaning, and source authority. FII/DII flow is not a macro regime proxy.

A future India macro architecture must define official series, publication and revision vintages, release calendar, staleness, deterministic regime math, and validation. Until then, show a structural gap. LLM prose cannot create regime math.

8. Verification Plan

- Add server-adapter fixture tests for success, partial payload, throttling, schema drift, stale fallback, bad currency, and non-finite values.
- Verify no provider hostname appears in the client bundle/network log.
- Browser-test the actual context switch, persistence, refresh, failure, and stale states; ?market=india is not the context contract unless separately designed.
- Verify India requests never read/write US cache keys and vice versa.
- Verify the page always exits loading after timeout/error.
- Verify full breadth coverage arithmetic and constituent-version identity, or verify the ten-name product is consistently labeled as a sample.
- Confirm desktop/mobile layout and market-specific heading with Playwright.

9. Build Order

- Live-test the current market switch and capture current browser/network failures.
- Add the server-side snapshot contract and cache compatibility layer.
- Remove direct client imports/calls to `lib/india-data.ts`.
- Correct headings, transient-vs-structural states, and timestamps.
- Decide full breadth versus explicitly labeled sample; implement only one.
- Add System Health aggregation by component/provider, not one alert per symbol.
- Revisit optional India-only displays after source contracts are proven.

10. Rollback And Acceptance

Disable the India snapshot endpoint/flag and render an honest temporary-unavailable state; US remains untouched. Acceptance requires zero direct browser provider calls, reproducible market-local cache keys, bounded loading, explicit partial coverage, and no scoring/order imports. No Supabase migration or provider activation is authorized by this document until the design is owner-approved.

11. Implementation Notes (2026-07-15)

Built to this design. Files/routes:

- `lib/india-markets/adapters.ts` - server-only, allowlisted Yahoo chart adapter.

Pure validate core `toAdapterQuote(symbol, FetchOutcome, now)` handles success / no_data / throttled / http_error / network_error / bad_currency (currency must be INR) / non_finite. `fetchQuotes` runs bounded concurrency + pacing. Never called authoritative.

- `lib/india-markets/constituents.ts` - versioned NIFTY-50 snapshot

(`universeId:"nifty_50", universeAsOf:"2026-07-01", BREADTH_COVERAGE_FLOOR=0.8`).

- `lib/india-markets/snapshot.ts` - types + pure `buildSnapshot / computeBreadth`.

Status: `unavailable` (0 index rows) / `complete` (all indices+sectors + breadth complete) / `partial` (anything else). Breadth counts only resolved names; unresolved stay in the denominator (`resolvedN/eligibleN, coveragePct`).

- `app/api/markets/india/route.ts` - the ONLY India market-data surface.

GET = cache-only and never contacts Yahoo from a page request. POST = owner/cron-gated FULL fill incl. NIFTY-50 breadth through one deduplicated, paced stream; persistence failure returns an error instead of false success. System Health aggregates ONE issue by overall status (`india-markets-degraded`), not per-symbol.

Breadth choice: Option 1 (full NIFTY-50 breadth), built asynchronously by the scheduled POST fill (not during page render), with a versioned constituent snapshot and 80% coverage floor. The GET path carries the last full breadth block forward and shows `temporarily_unavailable` until the first fill runs.

Cache isolation: new India-only table `india_market_snapshot` (migration 20260715160000), fully separate from US `price_cache`. India data is never written to `price_cache` and vice-versa - no cross-read is possible.

Cron: 20260715160500_India_market_snapshot_cron.sql plus audit correction 20260715220000_... - post-close full fills at 10:15 / 10:35 UTC (15:30 IST close, no DST) via `kairos_call_agent` (injects `x-cron-secret`). Both migrations applied to the target DB and verified.

Client (`components/dashboard/MarketsPage.tsx`): removed all direct `lib/india-data` Yahoo calls; India path fetches `/api/markets/india` with a 15s client abort so loading always exits. Product states: loading skeleton, complete/partial (available rows + explicit coverage), `TempUnavailableNote` (transient), `NotSupportedNote` (structural -

leveraged pairs / macro sentinel / TradingView, never synthesized). Market-aware heading names India instruments.
Verified: no provider hostname in the client static bundle; the markets page chunk references /api/markets/india.
Gates: tsc, vitest (423 pass), next build all green.

international equity allocation - Feature Architecture

Source: `features\international-equity-allocation\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

International Equity Exposure Architecture

Status: P0-P2A SHIPPED (read-only). P2 comparative study and P3-P4 remain unapproved.

Date: 2026-07-27

P0-P2A shipped: current US paper-book visibility plus an append-only VXUS

policy, issuer-mandate snapshot, and weekly operational-shadow assessment. It

has no target, provider runtime, allocation, paper-trading, or live-trading

effect. Comparative P2 and P3-P4 require separate approval.

P0 Implementation (2026-07-27)

- `lib/allocation/international-exposure.ts` calculates a current US paper-book view from existing persisted paper marks only. It uses a deliberately narrow, static country-ETF map; other ETFs remain unavailable rather than inferred.
- `components/dashboard/InternationalExposurePanel.tsx` renders the read-only Portfolio panel with a disabled/no-action state, recognized-country exposure, unclassified-ETF disclosure, and cost-mark disclosure.
- The panel is hidden on the India view. It cannot read India holdings or mix USD and INR. No position, policy, instrument registry, provider ledger, or money-path writer was added.

P1 Implementation (2026-07-27)

- Migration `20260727100000_international_allocation_p1.sql` added service-only `international_allocation_policies`, append-only `fund_exposure_snapshots`, and append-only `international_allocation_assessments`. Browser roles have no table grants and cannot execute the refresh RPC.
- The sole policy is `us_non_us_broad_core_v1: market=us, core=VXUS, construction=broad_core, status=observe`, and null `target/band`. The observed `instrument_registry` row remains `new_entry_allowed=false`.
- The first issuer snapshot preserves Vanguard's broad developed/emerging ex-US mandate and source URL. It is deliberately partial: it does not claim a point-in-time country-weight breakdown.
- The owner/cron-gated assessment route can append a current US paper-book observation only. It calls no provider and cannot generate a candidate, alter allocation, or place an order. P2A weekly scheduling is now active; comparative P2 remains unapproved.

P2A Implementation (2026-07-27)

- `kairos-international-allocation-shadow` runs once weekly at Monday 03:30

UTC through the existing protected Vercel transport.

- Each P2A row records a no-action proposal, explicitly null costs/tax drag, source coverage state, snapshot/position fingerprints, and one shadow week. A partial index-mandate snapshot or unset target can only suppress action.

- This is an operational evidence track for the approved VXUS policy. It does not compare no-international, VXUS, and VEA+VWO strategies yet: the latter two require separately approved policies and point-in-time source snapshots. Country satellites remain out of scope.

1. Decision

Kairos should eventually support non-US equity exposure inside the US/USD book through reviewed, US-listed international ETFs. It must not create an international market, a third cash pool, or an automatic country-ETF trading strategy.

This is strategic geographic diversification, not a short-horizon alpha claim. The first product is a read-only allocation decision surface that answers:

- How much of the US equity sleeve is already exposed outside the US?
- Is the portfolio materially concentrated in US geography, sectors, or common global revenue sources?
- Does the owner's approved target call for a broad non-US core, and is the actual exposure outside its allowed band?
- Is a proposed fund a broad core, a complementary developed/emerging sleeve, or a concentrated country satellite that duplicates existing exposure?

The current allocation core remains off. This proposal does not change `allocation_enabled`, score a symbol, add watchlist candidates, consume a new provider quota, place a paper trade, or place a broker order.

2. Why This Is the Right Boundary

A US broker account can obtain international equity exposure through US-listed funds without operating a foreign-market execution system. A broad vehicle such as VXUS (<https://investor.vanguard.com/investment-products/etfs/profile/vxus>) tracks a developed-and-emerging-markets index excluding the United States. That is materially different from a single-country vehicle such as INDA (<https://www.ishares.com/us/products/239659/INDA>), which the issuer describes as a targeted India exposure and currently has a large financials weight. Broad geographic diversification can reduce home-country concentration, but it is not a promise of higher return or a signal to tactically rotate every week. Vanguard's research (<https://corporate.vanguard.com/content/corporatesite/us/en/corp/articles/making-case-international-equity-allocations.html>) also frames the benefit as diversification and volatility reduction, not a market-timing rule.

The existing India pipeline is a separate INR book. It has its own market hours, research, paper pool, risk controls, and benchmark. A US-listed India ETF is a USD instrument with India geographic exposure; it belongs in the US book if ever approved. It must not be combined with, rebalance against, or treated as a substitute for India-pipeline holdings.

3. Non-Negotiable Invariants

- Two market pools only. A US-listed international ETF has `market=us`, `currency=USD`; the underlying geography is metadata. India remains `market=india`, `currency=INR`. No US/India NAV, P&L, cash, cap, performance, or exposure percentage is cross-summed.

- Strategic sleeve, not a generic candidate. Broad and country ETFs do not enter the ordinary 2-20 day ResearchAgent queue or compete with single-name scores. They use a slow allocation/rebalance process only after approval.
- One core construction at a time. The system may hold either one broad ex-US core, or a deliberate developed-plus-emerging split. It cannot call overlapping cores diversification.
- Country funds are satellites. A single-country ETF never satisfies the international-core target. Its geographic, sector, issuer, and existing-book overlap must be shown before it can be proposed.
- No US-macro proxy. Existing US MacroSentinel cannot decide whether an international ETF is attractive. Future domestic/global evidence is read-only until it demonstrates incremental, point-in-time value by fund archetype.
- No LLM decision authority. An LLM may explain persisted allocation facts; it cannot choose a fund, set a target, determine overlap, or trigger an order.
- Default deny. An unregistered ETF, stale fund facts, unavailable geographic look-through, or ambiguous classification produces unavailable and no proposal, never a neutral or buy recommendation.
- Existing brakes remain authoritative. Any future paper/live action still passes app pause, market controls, kill switch, drawdown breaker, market session, account scope, mandate, caps, idempotency, and live approval gates.

4. Exposure Model

4.1 Portfolio hierarchy

```

US/USD book
-> equity sleeve
    -> domestic US equity exposure
    -> international-core sleeve (optional, slow allocation)
        -> broad ex-US OR developed + emerging split
    -> country satellite sleeve (optional, smaller and independently capped)
India/INR book
-> unchanged India equity/defensive/cash sleeves

```

international_core and country_satellite are US-book sleeve labels, not markets. They extend the existing strategy_sleeves concept only after an approved migration and must retain its per-market key.

4.2 Permitted construction patterns

```

| Pattern | Intended use | Initial status |
| One broad ex-US core, e.g. VXUS | Default diversified international exposure across developed and emerging markets | Candidate for P0 measurement only |
| Developed ex-US + emerging split, e.g. VEA + VWO | Owner deliberately wants a controlled developed/emerging mix | Candidate for P0 measurement only; mutually exclusive with broad core |
| Single-country, e.g. INDA | Explicit small satellite hypothesis or owner-directed concentration | Observe only; not a core and not an ordinary research candidate |
| Multiple India country ETFs, e.g. INDA + EPI + INDY | Apparent variety with heavy geographic overlap | Prohibited without look-through proof and a separate owner decision |

```

The product must not preselect tickers as an investment recommendation. The issuer's current fund facts, expense ratio, liquidity, holdings, index method, and broker fractional eligibility are data to verify at proposal time, not permanent constants in source code.

4.3 Target and band semantics

The owner selects a target as a percentage of the US equity sleeve, not of total personal net worth, because Kairos does not have a complete, reconciled view of every external asset and liability. It shows both denominators clearly:

- US book NAV: useful for execution capacity only.
- US equity sleeve: the only denominator for this allocation policy.
- Known Kairos portfolio: informational only; never represented as total wealth.

No default percentage is set in code. The initial UI asks for a target and rebalancing band only after P0 data has exposed the current geographic concentration. A target of zero remains the shipped default.

Bands create slow actions:

- below lower band: a future proposal may buy the selected core;
- inside band: hold, regardless of recent relative performance;
- above upper band: a future proposal may stop new core buys or propose a slow

rebalance, subject to taxes, costs, and approval;

- unavailable/stale input: hold and surface the gap.

No band breach is an immediate sell signal and no international allocation rebalance can override a protective position exit.

5. What Determines Where Exposure Belongs

5.1 Core decision matrix

Input	What it answers	Allowed initial effect	
Current fund look-through	Country, region, sector, and issuer overlap across held ETFs	Display and duplicate-exposure warning	
Current US-book holdings	Concentration in US/foreign revenue, sectors, and names when reliable tags exist	Display only; unknown exposure remains unknown	
Owner target and band	Desired strategic non-US share of US equity sleeve	Display a deterministic below/inside/above-band state	
Fund eligibility	Registration, exchange, currency, instrument archetype, liquidity, fee, and stale-fact checks	Refuse proposal when incomplete	
Tax and account constraints	Whether a sale/rebalance could be uneconomic	Refuse or require explicit owner approval	
Domestic/global exogenous evidence	Global risk context and country-specific stress	Context only in P0-P2; never a timing score	

5.2 Geographic look-through rules

- Store fund composition snapshots with as_of, source URL, retrieval time, holdings/fund-facts fingerprint, and coverage percent. Never silently use today's holdings for a historical allocation decision.
- Report known, unknown, and overlap portions separately. Do not renormalize known holdings to 100% when coverage is incomplete.
- Use country/region weights first. Sector/issuer overlap is supplementary and must be constrained by source coverage.
- For a country satellite, show overlap with the India/USD ETF sleeve and any directly held Indian ADR/US-listed exposure. It does not read India/INR position data into US sizing.
- A fund-of-funds, derivative, or opaque product needs explicit classification;

it is not inferred from its ticker or marketing name.

6. Data and Architecture Fit

6.1 Reuse, do not create a parallel truth layer

The build must extend these existing contracts:

```
| Existing component | Required use |
| `instrument_registry` | Reviewed archetype, market, currency, execution eligibility, and default-deny behavior |
| `strategy_sleeves` | Eventual US-book target/band configuration; no global singleton setting |
| Existing live/paper position and NAV projections | Current US-book exposure inputs, preserving account scope and currency |
| `evidence_records`, `provider_call_ledger`, `evidence_cache_v2` | Provenance for all externally sourced fund and risk facts |
| `symbol_daily_returns` | Later frozen, same-market measurement only; no current-price backfill for historical claims |
| `features/exogenous-risk-evidence` | Future domestic/global context source; remains observation/shadow-only until independently admitted |
| Existing allocation core | Eventual deterministic band calculation, still off until its current sizing/rebalancer work is completed |
```

New records, if P1 is approved, should be a versioned `fund_exposure_snapshot` and an immutable `allocation_assessment`, each linked to the above evidence fingerprints. They must not replicate prices, positions, evidence cache, or broker account data.

6.2 Required instrument archetypes

```
| Archetype | Market/currency | Research path | Allocation role |
| `international_equity_core` | US/USD | Excluded from generic scoring | Optional strategic core |
| `international_equity_developed` | US/USD | Excluded from generic scoring | Optional complementary split only |
| `international_equity_emerging` | US/USD | Excluded from generic scoring | Optional complementary split only |
| `country_equity_satellite` | US/USD | Excluded from generic scoring | Optional capped satellite, never core |
| `india_equity` | India/INR | Existing India path | Unchanged; not a substitute or counterweight |
```

7. Phased Build and Timing

Calendar estimates are engineering ranges, not evidence promises. A phase does not advance just because its dates have elapsed.

P0 - Inventory and read-only decision surface (2-3 engineering days)

- Add an owner-reviewed starter policy catalog with archetypes and no enabled

instruments. Do not seed an allocation target.

- Inventory existing US-book ETF/ADR exposure and static overlap warnings.

- Build a Portfolio > Allocation read-only section: US book separately,

selected construction status, known/unknown geographic coverage, duplicate warnings, target-band state, data freshness, and no action state.

- Prove that no item enters ResearchAgent, ThemeScout, PaperTrader,

PositionMonitor, capital rotation, or live trader.

Acceptance: with no policy selected the surface states that international allocation is disabled; it neither fetches a new market-data feed nor changes the research backlog or provider-call ledger.

P1 - Authoritative look-through and policy records (3-5 engineering days)

- Add source-backed fund exposure snapshots and the append-only allocation assessment record.

- Enforce one core construction and prohibit undocumented overlap.
- Add fund eligibility/staleness/cost/liquidity checks and account/tax warning states.

- Apply RLS and service-only writer protections; verify the target production schema and client denial before claiming completion.

Acceptance: every displayed country allocation has an as-of source snapshot, coverage percentage, and immutable evidence fingerprint. A stale or partial snapshot yields unavailable and no proposed action.

P2 - Paper allocation shadow (minimum 8-12 weeks of observations)

- Recalculate target-band decisions on a slow cadence (weekly at most) without creating paper positions or consuming broker/provider execution quota.

- Record proposed action, suppressed action, costs, coverage, and exact input fingerprints.

- Compare three mutually exclusive policy families: no non-US sleeve, broad core, and developed/emerging split. Country satellites are evaluated as a separate hypothesis, not merged into the core result.

- Measure turnover, costs, dividend/tax assumptions, drawdown, concentration, and exposure stability. No retrospective current-holdings reconstruction.

Acceptance: zero cross-currency records, zero generic ResearchAgent entries, and reproducible weekly proposals. Operational evidence is healthy for at least 8-12 weeks; long-horizon performance claims remain unproven.

P2B - Historical static-allocation diagnostic (implemented, not a promotion gate)

- Backfill only V00 and VXUS adjusted-close history through two bounded, cache-only Yahoo chart requests. Massive's production entitlement returned only about two years of archive data, below the required history floor. This does not add either symbol to Markets tiles, research, scoring, or execution.

- Compare one predeclared US/USD policy pair only: 100% V00 against `80% VOO / 20% VXUS`, monthly rebalanced at the matched-session close with a recorded 5 bps one-way cost assumption. The 20% is a diagnostic constant, not an owner target or recommendation.

- Require at least three years of matched sessions. Until then, persist and display `insufficient_history`; never fill missing dates from current prices or a provider call in the replay request.

- Persist the exact cache/configuration fingerprint and result in a service-only append-only replay ledger. Show return, volatility, drawdown, information ratio, cost drag, and three contiguous reporting windows, but never designate a winner or alter policy state.

Acceptance: the replay makes zero provider calls, produces no paper/live order, does not mutate a target/band/status, and clearly says it is not a reconstruction of the Kairos paper/live book. It is useful historical context only; PIT fund composition,

distributions, taxes, and a separately predeclared owner target are still required before a performance-oriented promotion.

P3 - Paper execution, only after P2 and allocation-core completion

- Finish the already-deferred allocation sizing/rebalancer work with a US-only international sleeve and a band/deadband.
- Require a separate paper-only feature approval and a tax/cost-safe proposal ledger. Rotation cannot use an international sleeve to bypass its own gates.
- Restrict actions to the selected core construction and owner cap. Country satellites remain disabled unless their separate study passes.

Acceptance: an action is serialized per US pool, idempotent, gate-complete, and cannot force-close a single-name position or override a stop/target.

P4 - Live consideration (no fixed date)

Requires all of: explicit owner approval, paper evidence, longer-horizon historical/PIT evaluation, broker and fractional-share eligibility, tax review, and a separate live-order architecture review. It must start in manual approval mode. No inference from the presence of a Webull or Robinhood connection is permission to enable it.

8. Validation Standard

This architecture deliberately does not promise that international exposure will outperform US equities. The validation question is whether a chosen policy delivers the stated diversification objective after realistic implementation costs, without creating unacceptable concentration or churn.

Before a performance-oriented promotion, require:

- Point-in-time fund composition and return inputs, including delistings, distributions, fees, rebalances, and corporate actions as applicable.
- Walk-forward comparisons against the policy's declared baseline, with no target tuning on the test window.
- A long enough span to include distinct currency/rate/ regime periods. An 8-12 week shadow proves operations, not strategic allocation quality.
- Costs, spreads, taxes, and account constraints applied before any claimed benefit.
- Market-local reporting: US allocation results in USD and against its selected US-book benchmark; India results, if separately designed later, in INR and against NIFTY. Never create an aggregate alpha number.

Until then, global risk evidence and relative strength can explain context only; they cannot time entry or exit from the international sleeve.

9. UI Contract

The eventual Portfolio > Allocation section is a dense operational surface, not a recommendation feed. It shows:

- US-book current/target/band values and the explicit denominator;

- selected construction (disabled, broad core, or developed + emerging);
- country/region weights with known/unknown coverage;
- detected duplicate exposure and country-satellite caps;
- source as-of time, quality, and missing-data reason;
- current policy result: disabled, hold, below band, above band, or unavailable;
- a full audit trail of inputs and hypothetical actions.

It must visibly distinguish an educational allocation observation from an approved paper proposal and from a broker order. No buy/sell button appears in P0-P2.

10. Explicit Non-Goals

- No direct foreign exchange or foreign-broker trading.
- No third market or pseudo-currency pool.
- No use of India paper/live cash to fund a US-listed India ETF.
- No automatic country rotation, momentum chasing, or weekly macro timing.
- No addition of VXUS, VEA, VWO, INDA, EPI, INDY, or similar funds to the generic stock watchlist merely because they are available.
- No use of an LLM, TradingView, social/news sentiment, or a chart pattern to choose geographic allocation.
- No live allocation change, provider quota expansion, or API key requirement in P0-P2.

11. Owner Decisions Needed Before P1

- Is the first objective broad diversification only, or is a tactical country satellite allowed at all? Recommendation: broad diversification only.
- What is the desired non-US share of the US equity sleeve and allowed deadband? Recommendation: choose after P0 displays actual look-through; default remains zero until explicitly set.
- Is the permitted construction one broad core, or a developed/emerging split? Recommendation: one broad core initially because it has the least overlap and operating complexity.
- Are any US-book accounts tax-sensitive enough that a rebalance must always be manual? Recommendation: yes by default until tax-lot and account-policy facts are fully modeled.
- Should any country satellite be evaluated later? Recommendation: defer it until the broad-core policy and look-through data are operating correctly.

12. Recommended Next Step

Approve P0 only. It is the smallest useful answer to "when, how, and where": show existing exposure, select no more than one construction, make overlap visible, and keep all trading paths unchanged. P1-P4 remain gated by P0 evidence and separate approvals.

known anomalies - Feature Architecture

Source: features\known-anomalies\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Known-Anomaly Research Backlog

Status: REVIEWED DESIGN DRAFT. NOT APPROVED FOR IMPLEMENTATION.

Last reviewed: 2026-07-15 by Codex.

Scope: measure-only anomaly research. No score, rank, sizing, order, or exit effect.

Market applicability: US feasibility first. India is unavailable until an

independently validated point-in-time event and expectation contract exists.

1. Decision

Do not begin by implementing PEAD as a scored feature. Begin with a bounded US data-feasibility study. Kairos currently stores upcoming earnings estimates, but the refresh path writes `eps_actual = null`; Webull supplies a current consensus snapshot, not a durable pre-announcement estimate history. The app therefore does not yet possess the point-in-time actual/expectation pair required for PEAD or the vintages required for revision momentum.

This backlog is subordinate to features/advanced-learning and the existing Edge/Validation governance. Each distinct signal definition is a separate trial and counts toward multiple-testing controls. A combined signal is another trial, not a free improvement.

2. Non-Negotiable Invariants

- No LLM output changes a deterministic score, eligibility result, size, order, stop, target, or exit.
- Every feature is point-in-time: `available_at <= decision_at`. Event date alone is insufficient.
- US and India are evaluated separately. Currency and return pools never mix.
- A negative observation is measurement only. It cannot enter downside hedging, suppress candidates, or create a short without a separately approved feature.
- Promotion defaults to no change and uses the existing Challenger, validation, lineage, and Performance Truth layers.
- Data comes through the Canonical Evidence Router and existing evidence provenance. Do not create an anomaly-specific provider or truth layer.
- The existing `post_earnings_drift` archetype is not evidence that PEAD exists: it currently reacts to pre-earnings proximity and reweights existing dimensions. A true PEAD edge must use a different versioned identifier until that naming collision is resolved.

3. P0: US PEAD Data Feasibility

Required event record

For each issuer/reporting period, the immutable event snapshot must contain:

- canonical symbol and issuer identity;
- fiscal period and period end;
- announcement timestamp and session (before_open, during_session, after_close, or unknown);
- first-reported actual EPS, basis, currency, source, and available_at;
- last valid consensus EPS strictly before the announcement, snapshot timestamp, contributing analyst count, basis, currency, and source;
- later correction/restatement links without overwriting the original value;
- split/corporate-action adjustment metadata; and
- Router policy version and evidence-record references.

Unknown announcement session, incompatible EPS bases, a post-release estimate, zero/near-zero scaling denominator, or missing provenance causes abstention.

Candidate surprise definitions

Pre-register and evaluate separately:

- $\text{pead_analyst_price_scaled_v1} = (\text{actual_eps} - \text{consensus_eps}) / \text{pre_event_price}$.
- $\text{pead_analyst_abs_scaled_v1} = (\text{actual_eps} - \text{consensus_eps}) /$

$\max(\text{abs}(\text{consensus_eps}), \text{epsilon})$, with a documented epsilon and winsorization.

- A time-series SUE only after a restatement-safe historical EPS series exists;

scale the unexpected change by its own historical variability.

Do not divide surprise by analyst dispersion. Dispersion measures disagreement, may be absent or near zero, and can create unbounded values. It may be tested as a separate conditioning variable once point-in-time analyst-level coverage exists.

Feasibility output

P0 produces a coverage report, not edge_signals:

- eligible events / all events by year, sector, market-cap bucket, and session;
- consensus age and analyst-count distributions;
- basis/currency conflicts and correction rates;
- source outages and survivorship/delisting coverage; and
- estimated provider call/storage cost.

Proceed only if the owner approves explicit coverage floors after seeing this report. Do not label PEAD "zero data risk" or "data already available."

4. P1: Measure-Only PEAD Edge

After P0 approval:

- Freeze the definition, universe, horizon set, cost model, and winsorization.
 - Assign returns from the first tradable session after the verified publication time. An after-close result cannot use that same close.
 - Evaluate fixed forward horizons aligned with Kairos styles, with overlapping-return robust errors, sector/year breakdowns, event clustering, liquidity, delistings, and realistic spread/slippage.
 - Evaluate long-only behavior. Negative-surprise names are an excluded/observed cohort, not automatic shorts.
 - Write measure-only edge observations linked to existing decision observations and evidence snapshots. Never manufacture a decision observation solely because an external event exists.
 - Count every definition, horizon, filter, and combination in the declared trial family used by DSR/PBO and promotion review.
- Only an approved, out-of-sample Challenger may later consume the edge.

5. Deferred Candidates

Analyst-revision momentum

Blocked until Kairos stores immutable estimate vintages. A current Webull target or forecast cannot reconstruct revisions. A valid contract needs estimate timestamp, fiscal period, basis, analyst count, revision breadth, stable analyst universe, and strict pre-decision availability. Evaluate EPS and target revisions separately.

Filing-language change (Lazy Prices)

US-only feasibility initially. Use point-in-time EDGAR filings, issuer/form/period identity, acceptance timestamp, amendment links, section-aware extraction, and a versioned tokenizer/diff algorithm. Never compare a filing with a later amendment that was not yet known. Plain deterministic similarity is the first candidate; embeddings or LLM interpretation require a separate untrusted-compute review.

PEAD.txt-style textual surprise

This is not generic earnings-call tone. The cited research constructs a numerical text-based earnings-surprise measure. Reproducing it requires a separately reviewed document corpus, labels, training/evaluation protocol, and point-in-time controls. It is not an incremental prose feature for P1.

Short interest, 13F flows, options skew, and seasonality remain intake ideas only.

6. Validation And Promotion Gates

- common, point-in-time universe and benchmark;
- market-local net returns after costs;
- walk-forward splits with an untouched final holdout;
- sample floors per year/regime/sector, not only aggregate N;

- Newey-West or event-cluster-aware uncertainty where observations overlap;
- declared trial-family count and advanced-learning DSR/PBO controls;
- incremental value over the current champion, not standalone significance;
- stable effect direction and no dependency on one provider/year/sector; and
- owner-approved Challenger promotion. No automatic promotion.

7. Build Order

- Resolve the `post_earnings_drift` naming/semantic collision.
- Specify and capability-probe a US point-in-time earnings contract.
- Run P0 coverage only; retain no scoring effect.
- If approved, add immutable event snapshots through Router provenance.
- Build one pre-registered PEAD definition and evaluation.
- Add other definitions one at a time as counted trials.
- Consider revision momentum, filing changes, and India only after their own evidence contracts pass feasibility.

8. Acceptance And Rollback

P0 passes when every reported numerator/denominator is reproducible from immutable source references and no post-event fact enters a pre-event snapshot. P1 passes only when reruns from the same snapshot produce byte-equivalent feature values and the edge remains measure-only. Disable by stopping new edge computation; immutable history remains. No positions, signals, policies, or orders require rollback.

8b. Implementation Note - Data Capture Enabler (2026-07-15)

Build-order steps 1-2 (plus the vintage table and a feasibility read) are implemented as data capture only. No scored feature, no `edge_signals`, no scoring/sizing/order/exit effect exists. Deterministic; no LLM.

- Naming collision (step 1) resolved. `lib/scoring/archetypes.ts`: the archetype formerly `id: "post_earnings_drift"` / label "Post-Earnings Drift" is renamed to `id: "pre_earnings_proximity_reweight_v1"` / "Pre-Earnings Proximity Reweight". Behavior (weights, `daysToEarnings <= 10` routing, shadow role) is unchanged - only the identifier/label/comment. The `pead_*` namespace is now reserved for a future true surprise-based edge. Historical `shadow_decisions.setup_type = 'post_earnings_drift'` rows remain queryable under the old string; the rename starts a new series going forward.

- First-observed provider actuals (step 2). Migration

`20260715210000_earnings_pit_actual_capture_columns.sql` adds `eps_actual_first`, `revenue_actual_first`, `actual_available_at`, `announcement_session`, `eps_basis`, `actual_currency`, `actual_source`, `restated_eps`, `restated_available_at`, `restated_source`, `market` to `earnings_calendar`. `lib/data/earnings-pit.ts` fills `eps_actual_first` once and never overwrites it; migration `20260715220000_...` enforces that immutability in Postgres. A later differing provider print is logged to `restated_eps`. This is the first value Kairos observed, not proof that a sparse polling cadence captured the issuer's literal first print. Source: Finnhub earnings calendar (free, no daily cap, already wired); its `hour` field maps to the announcement session.

- Consensus vintages (step 3). Append-only table

earnings_consensus_snapshots (migration 20260715210100_ . . .) accumulates the last-valid pre-announcement consensus. A new vintage is appended only when the consensus changes since the last snapshot; a database trigger blocks UPDATE/ DELETE. The conservative timestamp rule rejects consensus captured on or after the US report date, preventing delayed actual publication from creating look-ahead. analyst_count is null on the free FintHub calendar (surfaced in the coverage report). FintHub also does not identify GAAP versus adjusted EPS, so provider identity is never treated as accounting basis; these observations remain measurement-only and ineligible until another source proves basis comparability.

- Feasibility read (?3). GET /api/calendar/earnings/coverage, owner-gated

(requireOwner), read-only. Reports eligible events (both a first-reported actual AND a pre-announcement consensus vintage with explicitly matching basis/currency) by year and session, plus analyst-count coverage, corrections, and basis conflicts. It is a coverage report, not edge_signals.

- Cadence. kairos-earnings-pit-capture invokes POST /api/calendar/earnings/refresh

daily at 02:10 UTC; the capture runs after the cache bust, fail-soft. RLS is on for both tables (authenticated-read, service-role-write), verified via information_schema + pg_policies.

9. Primary Research References

- Bernard and Thomas, post-earnings-announcement drift:

<https://www.jstor.org/stable/2491062>

- Philadelphia Fed, PEAD.txt project and revisions:

<https://www.philadelphiafed.org/the-economy/banking-and-financial-markets/pead-txt-post-earnings-announcement-drift-using-text>

- Cohen, Malloy, and Nguyen, Lazy Prices:

<https://onlinelibrary.wiley.com/doi/abs/10.1111/jofi.12885>

10. Owner Decisions Before Build

- Approve P0 as US coverage measurement only.
- Approve one analyst-surprise definition after the coverage report; do not approve a combined definition in advance.
- Set coverage and sample floors from observed feasibility, not guesses.
- Keep India blocked until a separate point-in-time data proof exists.

learning core - Feature Architecture

Source: `features\learning-core\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Learning Core Rebuild - Feature Architecture

Status (2026-07-06): FULL ROADMAP BUILT. Phase 1 (decision ledger), all P0 improvements (transactional fill RPC, cron-gap detector, vitest suite), Portfolio Constructor, Phase 2 (Validation Engine + fail-closed promotion + calibrated Kelly sizing + dynamic R:R), Execution Gateway paper-stage (Alpaca), and Phase 3 (typed strategy genome, feature registry + whitelisted compiler, shadow decisions, regime features, governance rewiring) - all built, migrated live (057-065, 068-071), and passing 85 tests + tsc + build. Two critical pre-existing bugs discovered and fixed along the way via live schema access: `paper_order_events.signal_id` type mismatch (Decision 34) and the Trade Queue reading a dead `trade_queue` table instead of `trade_proposals` (Decision 36). Genome/feature-registry/shadow are all opt-in and inert by default - nothing changes live behavior until a human deliberately activates them. Remaining out-of-scope (spend/hosting decisions, not specs): paid data tier, cloud infra migration. Origin: Codex agent-architecture review (`CODEX_AGENT_REVIEW_RESULT.md`). Verdict: Kairos is a governed adaptive **scoring** system, not yet self-evolving. Blocker: no point-in-time, policy-aware, out-of-sample validation that proves a change **causes** reproducible improvement.

Goal: Turn the learning plane into a statistically trustworthy, self-evolving engine - the "world-class evolving quant agent" bar - without weakening the existing control plane (deterministic scores, immutable challengers, per-market champions, human live-capital gate).

Design constraints (keep): deterministic scoring (no LLM numbers); LLM = hypothesis/feature proposal + narrative only; per-market isolation; human is the FINAL live-capital gate; everything guarded/resilient; free-data-first.

The three phases

Phase 1 - Scientific ground truth (KEYSTONE)

Give the learner a valid observation->outcome dataset. Nothing above this is trustworthy until it exists.

- `decision_observations` (new, immutable, append-only): one row per SCORED candidate - including rejected ones (not just filled longs) - capturing the point-in-time state:
 - `id`, `ts`, `market`, `symbol`
- `code_version` (git sha), `strategy_version_id`, `weights_snapshot_hash`, `data_availability_mask` (which of the 5 dims had real data)
- raw + transformed features (the dimension sub-inputs, not just the 5 scores): stored as `features jsonb`
- `analyst_score`, per-dim scores, `entry_eligible` (passed threshold?), `action` (bought/skipped + reason), `fill_price/qty` if filled, `exit_policy_version`
- Written by ResearchAgent for EVERY candidate it scores (filled or not).
- `observation_labels` (new): forward outcomes computed AFTER horizon maturity, never before:
- `observation_id`, `horizon` (2d|5d|10d|20d), `fwd_return`, `benchmark_neutral_return`, `sector_neutral_return`, `max_adverse_excursion`, `max_favorable_excursion`, `matured_at`
- Uses corporate-action-adjusted prices; realistic spread/slippage/fees. A nightly "label maturation" job fills these as each horizon comes due.
- Realized policy P&L (`paper_trades`) is kept SEPARATE as an execution-policy label, not the alpha label.

- Walk-forward dataset builder: purged + embargoed folds around validation boundaries so overlapping swing returns aren't leaked across train/test.
- Quarantine 10yr personal trades from alpha: they feed the Mentor/behavioral model only (provenance + policy-era tags); may generate hypotheses but can't satisfy mutation sample sizes or update alpha weights.

Migrations: 059_decision_observations.sql, 060_observation_labels.sql. Jobs: POST /api/agents/label-maturation (nightly cron, per market). ResearchAgent writes observations inline.

Phase 2 - Validation Engine (make evidence mandatory)

- Deterministic validation service (lib/validation/): fit regularized multivariate models (ridge/elastic-net for excess return, logistic for sign, rank-IC/Spearman for cross-sectional rank) with sector/beta/vol/regime controls; block/bootstrap uncertainty; nested walk-forward + a locked final holdout.
- Champion-vs-challenger on paired opportunity sets (same symbols/periods): paired return differences, posterior P(improvement), Sharpe/Sortino, drawdown, turnover, calibration, exposure stability, per-regime, effective sample size (clustered/overlap-adjusted), multiple-testing correction.
- Promotion state machine (server-enforced, fail-closed): draft -> offline_testing -> shadow_paper -> eligible -> approved. promote_champion REJECTS unless a signed Validation Engine result on a frozen dataset exists. Human approval remains the final live gate - after evidence, not instead of it.
- Promotion objective = risk-adjusted return (net expected log-growth / Sharpe under fixed exposure), NOT win-rate - so R:R and expected value drive selection (see the Risk-Reward workstream above).
- Calibrated P(win) + sizing/exit module: fit a calibrated probability + a separate, independently-validated sizing/exit policy against the MAE/MFE labels (review #4). This is where risk-reward starts being *learned* instead of hand-set.
- LearnerAgent's update_signal_weight no longer picks the number - it proposes a hypothesis; the Validation Engine fits + gates.

Phase 3 - Controlled evolution (genuinely evolve)

- Typed strategy genome (extend strategy_versions): feature defs/transforms, missing-data policy, entry/rank threshold, universe/liquidity filters, horizon, exit family, conviction-scaled sizing (fractional-Kelly / vol-target), dynamic ATR/pattern-conditioned R:R, exposure limits, regime router - each with an approved search domain. Sizing + R:R are now first-class evolvable genes (see Risk-Reward workstream), validated like alpha. Evolve ONE layer at a time under nested validation. Hash-bound to signals/experiments/trades.
- Feature registry + discovery: LLM proposes a machine-readable feature spec (rationale, formula, inputs, lag, expected sign, horizon, falsification test); a deterministic compiler replays it point-in-time, tests incremental value, quarantines until it passes; monitors rolling IC/decay and auto-demotes dead features.
- Shadow A/B + bounded exploration: fan every daily observation to champion + a small set of challengers (record hypothetical decisions/fills); paper-only risk-capped contextual bandit for exploration; live capital stays champion-only with auto-rollback on drift/drawdown.
- Regularized regime conditioning: point-in-time regime features (trend/vol/breadth/rates/dispersion); mixture-of-experts / partial-pooling so sparse regimes shrink to global. No brittle bull/bear switches.
- Predeclared promotion objective (e.g. benchmark-neutral expected log-growth under fixed exposure) + non-inferiority gates; versioned, requires approval to change.

Cross-cutting workstream - Risk-Reward Optimization (first-class goal)

Problem today: sizing is a flat `position_size_pct` (10%) and R:R is a fixed profile ratio (~2.85:1) - identical on a barely-qualifying and a max-conviction trade. Conviction only flips a binary entry gate; it never sizes the bet or shapes the payoff. Risk-reward is a hand-set constant, entirely outside the learnable surface. A world-class agent must press when the edge is real and confident, and stay small/flat otherwise.

This spans all three phases:

- Phase 1 (ground truth): `observation_labels` capture MAE/MFE (max adverse/favorable excursion) per candidate per horizon - the raw material for learning "how much room does this pattern need, how far does it run." Also `benchmark_neutral_return` so reward is measured as alpha, not beta.
- Phase 2 (learn + validate the R:R module):
 - Fit a calibrated `P(win)` / expected-payoff model (logistic for sign, quantile/regression for magnitude) - NOT the raw `analyst_score`, which is uncalibrated.
 - Learn sizing + exit as a SEPARATE validated module (review #4) against MAE/MFE + return labels: how big, where the stop, where the target - as functions of conviction, volatility, and pattern.
 - Promotion objective = risk-adjusted return (net expected log-growth / Sharpe under fixed exposure limits), NOT win-rate. So the engine optimizes expected value and R:R, not hit-rate - a 45%-win, 3:1-payoff strategy must be able to win promotion over a 60%-win, 1:1 one.
- Phase 3 (make it evolvable + conditional - part of the genome):
 - Conviction-scaled sizing - fractional-Kelly or volatility-target sizing from calibrated `P(win)` x payoff, capped by exposure/concentration limits. Bet bigger only where edgexconfidence is high; never full-Kelly (ruin risk).
 - Dynamic R:R - ATR/volatility- and pattern-conditioned stops & targets (e.g. stop = $k \times \text{ATR}$, target sized to the pattern's historical MFE), replacing the fixed 7/20.
 - Regime-conditioned posture - the regime router (P3) adjusts sizing aggression + R:R by regime (e.g. tighter/smaller in high-vol reversals).
 - All bounded by the risk layer + human live-capital gate; sizing/exit changes go through the same walk-forward validation + promotion gate as alpha changes.

Guardrails: fractional-Kelly ($\leq$?-Kelly) not full; hard per-name and per-sector exposure caps stay human-set (moral-hazard rule); sizing model validated out-of-sample before it can move real size; auto-rollback on drawdown.

Phase 1 detail (the piece up for approval now)

Why first: every other phase optimizes against a label. If the label/dataset is biased (symbol-join, filled-longs-only, policy-P&L, leakage), the whole engine optimizes noise. Phase 1 is the ground truth.

Scope of the build:

- Migration `059_decision_observations` + `060_observation_labels` (+ indexes; RLS off like sibling agent tables). Applied manually (MCP denied) - I'll hand you the SQL + full paths.
- ResearchAgent (`lib/research-agent.ts`): after scoring each candidate, write a `decision_observations` row (features + availability mask + action) - for filled AND skipped candidates. Guarded/resilient (no-op if table absent).
- Label maturation job `app/api/agents/label-maturation/route.ts` + cron: nightly, per market; for observations whose horizons have matured, compute fwd/benchmark-neutral returns from Yahoo/price_cache and write `observation_labels`. Idempotent.
- Walk-forward dataset builder `lib/learning/dataset.ts`: assemble purged/embargoed folds from observations+labels (read-only; consumed by Phase 2).

- Learner: repoint `query_signals_with_outcomes/query_score_correlation` to read the matured label dataset (horizon-aligned, all candidates) instead of the symbol/policy-P&L join; mark personal-trade tools as behavioral-only (no mutation authority).
- Docs + system-map (new LEDGER node) + PROJECT_DECISIONS (Decision 33).

Not in Phase 1: the statistical fitting, promotion gate, genome, shadow - those are Phase 2/3.

Risk/cost: additive, resilient; no change to live behavior until observations accrue. Storage grows (~N candidates/day/market x features) - bounded, prune >18mo.

Phase 1 - RESOLVED decisions (refinement, 2026-07-06)

- Feature granularity = full raw features, for free. `computeScores` already returns an evidence object per dimension holding the raw sub-inputs (`pe_ratio`, `profit_margin`, `roe`, `eps`, `revenue_growth_yoy`, `analyst_target`, `sector`, `industry`, `sentiment_bull_bear_%`, `macro_danger`, `RSI/MA technicals`, `insider`). Phase 1 stores this whole evidence blob as `decision_observations.features` - no refactor of the scorer. This future-proofs Phase 3 feature discovery (raw inputs are captured from day one, not just the 5 scores).
- Observation trigger = every symbol that reaches `computeScores` (holdings + screener candidates + NIFTY candidates), whether it fills or is skipped/rejected. That's the honest "all candidates incl. rejected" set. Symbols filtered out *before* scoring (no features) are NOT logged.
- Horizons = 2 / 5 / 10 / 20 trading days. Active markets only (write observations only for markets currently in `market_focus`) - matches the 2-20d thesis, no wasted storage. [user-approved 2026-07-06]
- Label sources / benchmarks: US fwd prices from `price_cache/AV/Massive`, benchmark SPY. India from Yahoo .NS candles, benchmark ^NSEI. Phase 1 computes `fwd_return`, `benchmark_neutral_return`, and `max_adverse/favorable_excursion` (all cheap from daily candles). Sector-neutral return is DEFERRED to Phase 2 (needs peer-set construction). Prices corp-action-adjusted (Yahoo/AV adjusted series); a fixed cost/slippage haircut applied.
- No backfill. Point-in-time features can't be honestly reconstructed for past signals, so the ledger starts fresh from deploy. `signal_score_history` stays as-is for the Score Tracker chart. The dataset simply accrues from go-live.
- Phase 1 improves the DATA, not yet the METHOD. Once matured labels exist, the learner's correlation tools read the horizon-aligned, all-candidate, `signal_id`-correct dataset (already fixed the symbol-join bug) - but still use simple correlation, clearly flagged as INTERIM. The regularized multivariate fit + walk-forward promotion GATE is Phase 2. This keeps Phase 1 shippable and low-risk.
- Retention: prune `decision_observations` + `observation_labels` older than 18 months (a later cron); bounded storage.

Volume estimate: ~10-20 scored candidates/market/day x 2 markets ? 30-40 rows/day + 4 label rows each -> well under 1M rows/year. Trivial for Postgres.

Deliverables locked for Phase 1: migrations 059/060 (manual apply), ResearchAgent observation write, `label-maturation` cron (+ register-tasks entry), `lib/learning/dataset.ts` walk-forward builder, learner repoint + personal-trade quarantine, docs + system-map (LEDGER node) + Decision 33.

Build 1 - Genome as LIVE control (2026-07-08, user-approved)

Gap found: Phase 3 shipped a typed, hard-bounded StrategyGenome + feature registry (lib/validation/genome.ts, migrations 063/064) that is written and hashed onto every signal for provenance - but no live code read it. The decision path took score_threshold from strategy_config and derived exit/sizing from hardcoded percentiles (0.25/0.75, horizon 10, cap = position_size_pct). So a challenger that the Validation Engine proved better by tightening a stop or raising the threshold got promoted, yet traded identically to the champion - the genome was a manifest, not a control. Build 1 closes that: the promoted champion's genome now governs live behavior.

Wired (additive, no new migration - genome column already live):

- lib/validation/genome-live.ts (new): loadChampionGenome(supabase, market) -

reads the promoted champion strategy_versions.genome, hydrates onto DEFAULT_GENOME, re-checks bounds, and returns {genome, source, hash}. Best-effort: missing champion / null genome / out-of-bounds -> DEFAULT_GENOME (source: "default"), never throws.

- lib/research-agent.ts: entry threshold reads champion.genome.entry.score_threshold

(genome added to the champion select), falling back to strategy_config.

- app/api/agents/paper-trade/route.ts: getGlobalMaeMfePercentiles now takes

the genome's horizon_days + exit.stop_mae_pctile/target_mfe_pctile; Kelly sizing takes sizing.cap_pct/floor_pct and sizing.mode selects half-Kelly vs flat. Genome source/hash stamped into the portfolio-constructor stage log.

Money-safety invariants (locked):

- Genome cap clamped to the owner's strategy_config.position_size_pct -

min(genome.cap_pct, position_size_pct). The learning loop can size DOWN but NEVER raise per-position exposure above the owner-set limit. Money limits stay human. Portfolio Constructor name/gross/vol caps still run after.

- Behavior-preserving by construction. DEFAULT_GENOME = `{score_threshold 60,

stop_mae_pctile 25, target_mfe_pctile 75, horizon 10, cap 10, floor 2}`, the exact values the live path used before. A market with no genome-bearing champion is byte-for-byte unchanged. New behavior triggers only after a genome-bearing challenger passes the fail-closed validation gate AND is owner-promoted - the most-gated path in the app. No autonomy change; owner click still required for live.

Deliberately OUT of Build 1 (deferred, not dropped): universe.* (screener routing), features.included + regime.router (need feature-registry activation - Build 5-adjacent). Shadow-version decisions still score off the champion threshold, not each shadow version's own genome (separate enhancement).

learning integrity - Feature Architecture

Source: `features\learning-integrity\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Learning Integrity - Taint Detection, Exclusion & Learner Recovery

Status: DRAFT - awaiting approval. No implementation code until approved. Author: Claude (Opus 4.8), reviewed/updated by ChatGPT, 2026-07-08. Decision inputs (user, 2026-07-08):

- Primary mechanism: tag + exclude (non-destructive; deletion optional).
- Auto-taint threshold: moderate (`data_confidence < 0.5`) - conditional on a proven-correct, deterministic confidence calculation.
- Scope now: architecture draft only.

1. Problem

The research scorer can emit a high-conviction signal from partial data - for example, GLD/SGOV/IBIT on 2026-07-07 scored high while the logged evidence showed missing fundamentals, zero technical candles, and unknown macro. The weighted score then renormalized onto the dimensions that happened to be present.

Those decisions can flow into paper/live books and, after outcomes mature, into the LearnerAgent and Validation Engine. Bad data can therefore become bad trades, bad labels, and bad strategy mutations.

We need:

- A deterministic way to detect low-evidence decisions from already-logged facts.
- A non-destructive way to stop them polluting learning and validation datasets.
- A recovery path if a bad learner mutation still lands.
- A reusable, safe teardown path to replace ad-hoc manual SQL cleanup.

2. Non-goals

- No re-fetching or re-scoring historical decisions.
- No live trading-path dependency on this layer.
- No auto-delete by default; delete/reset remains manual, backed up, and dry-run-gated.
- No implementation code until Vaibhav approves this architecture.

3. Existing primitives to reuse

```
| Need | Existing table/column |
| Per-decision evidence | `decision_observations`: append-only; `availability_mask` jsonb, `features` jsonb
(`weighting.{base_weights,applied_weights,included_dims}`), `weights_used`, `signal_id` |
| Forward labels | `observation_labels`: `observation_id`, horizon returns, MAE/MFE |
| Versioned strategies / rollback | `strategy_versions`: `weights_snapshot`, `genome`, `is_champion`,
`parent_version_id`, `promoted_at`, `retired_at`, `state`, `rejection_reason` |
| Prior history | `learning_priors_history`: `prior_id`, `confidence`, `enabled`, `reason`, `changed_by`,
`changed_at` |
| Shadow / challenger eval | `shadow_decisions`: `observation_id`, `policy_version_id`, `would_enter`,
`score` |
| Paper trade to decision link | `paper_trades.signal_id` -> `decision_observations.signal_id` |
| Live order to decision link | `broker_orders.proposal_id` -> `trade_proposals.id` -> `trade_proposals.
signal_id` -> `decision_observations.signal_id` |
```


Current schema constraints that matter

- `decision_observations.id` is `bigserial`.
- `decision_observations.signal_id` is `uuid`.
- `paper_trades.signal_id` is `uuid`.
- `broker_orders` currently has no `signal_id`; live quality must join through `proposal_id`.
- `trade_proposals.signal_id` is currently declared as `bigint` in migration 037. It must be migrated to `uuid` before live-order taint joins are trusted.
- `decision_observations` is append-only by trigger; do not add mutable taint columns there.
- `observation_labels` and `shadow_decisions` reference `decision_observations(id)` with `on delete cascade`; the reset/teardown feature must never delete `decision_observations`.
- Any code that reads new quality/tag columns must ship only after the additive migration is applied. No schema-coupled runtime fallback should silently treat missing columns as "clean."

Implication: rollback is a strategy-version pointer operation, not data surgery. The taint layer should be a thin additive layer over mutable trade/order projections plus a deterministic view over the immutable observation ledger.

4. Design

4.1 `v_decision_quality` - deterministic quality view

Create a read-only SQL view `v_decision_quality` over `decision_observations`. It must not mutate the append-only ledger. It covers both historical and future rows.

Concept:

```
applicable_dims    = structural scoring dims derived from the logged row only
applicable_weight  = sum(base_weights[d]) for d in applicable_dims
real_dims          = dims in applicable_dims where availability_mask[d]=true and dim is not degraded
available_weight   = sum(base_weights[d]) for d in real_dims
data_confidence    = available_weight / applicable_weight
```

Use `features.weighting.base_weights` for `base_weights`. Do not use `weights_used` or `features.weighting.applied_weights` as the denominator; those are post-renormalization weights and can make a one-dimension decision look fully covered.

The view must not call or guess the TypeScript `applicableDimensions()` function. It has to derive structural applicability from values already logged in `decision_observations`:

- `market = 'india'`: applicable scoring dims are `fundamental`, `technical`, `macro`.
- `features.fundamental.is_etf = true`: applicable scoring dims are `technical`, `sentiment`, `macro`.
- US non-ETF: applicable scoring dims are `fundamental`, `technical`, `sentiment`, `macro`, `insider`.
- US ADR insider exclusion is not reliably reconstructable from current rows unless a structured flag is logged. Until then, if `availability_mask.insider=false` and the symbol is in a small reviewed ADR allowlist, the SQL view may exclude insider; otherwise set `quality_status='unknown'` rather than pretending certainty.
- If `applicable_weight <= 0`, malformed base weights, or a missing base-weight key would affect the denominator, emit `data_confidence=NULL` and `quality_status='unknown'`; never divide by applied/renormalized weights as a fallback.

The view emits:

- observation_id
- signal_id
- market
- symbol
- applicable_dims text[]
- real_dims text[]
- missing_dims text[]
- degraded_dims text[]
- decisive_dim text
- data_confidence numeric
- confidence_band text (high >= 0.75, med >= 0.5 and < 0.75, low < 0.5)
- quality_status text (ok or unknown)
- taint_reason text

4.2 Degraded-dimension rules

A dimension is degraded when it is technically present but not real evidence.

Historical fallback rules:

- fundamental: evidence note contains "No fundamental data available" or provider/rate-limit text. ETF "no company fundamentals" is structural, not degraded.
- macro: features.macro.regime = 'unknown'.
- technical: features.technical.dataPoints < 15.
- sentiment: no provider-backed basis, e.g. bullish/bearish both absent/zero and no AV/news sentiment.
- insider: unavailable/too few transactions text or availability_mask.insider=false.

String matching is a historical bridge only. For future rows, ResearchAgent should log a structured quality object:

```
{
  "quality": {
    "fundamental": { "state": "ok|missing|degraded|inapplicable", "reason": "..."},
    "technical": { "state": "ok|missing|degraded|inapplicable", "reason": "..."},
    "sentiment": { "state": "ok|missing|degraded|inapplicable", "reason": "..."},
    "macro": { "state": "ok|missing|degraded|inapplicable", "reason": "..."},
    "insider": { "state": "ok|missing|degraded|inapplicable", "reason": "..."}
  }
}
```

v_decision_quality should prefer features.quality and fall back to legacy string heuristics only for older rows.

4.3 Correctness requirements

These are acceptance criteria:

- Pure / no I/O. The quality calculation reads only database columns. No network, LLM, live prices, or provider calls.
- Reads logged weights. It uses features.weighting.base_weights and the scorer's own logged evidence. It does not reconstruct live state from current config.
- Two-stage fail behavior.

- During measure-only rollout, malformed/missing weighting, availability_mask, or structural inputs produce data_confidence = NULL, quality_status='unknown', and no automatic taint write.
- After golden tests and historical flag-rate checks pass, learner and validation datasets must require quality_status='ok' and excluded_from_learning=false. Unknown quality is not proof of bad data, but it is also not safe training evidence.
- Trading path stays independent. Order submission and portfolio code must not depend on this layer to place orders. This layer can tag, explain, and filter learning; it cannot be a new order-execution gate unless separately approved.
- Flag != exclude. Computed confidence and mutable exclusion are distinct. A bad flag is reversible by owner override.
- Golden tests + monitor. Unit/golden tests assert worked examples against real or fixture rows. A daily System Health reporter emits tainted/unknown percentages by market.

Moderate auto-flagging (data_confidence < 0.5) turns on only after all of the above are green. Until then, ship measure-only.

4.4 Worked checks

These cases must pass as golden tests:

- Bad GLD/SGOV/IBIT-style ETF row: applicable {technical, sentiment, macro}; technical missing, macro degraded, only sentiment real -> confidence roughly sentiment_weight / (technical + sentiment + macro) -> below 0.5 -> tainted.
- Legit ETF with real technicals, real macro, real sentiment -> confidence near 1.0 -> clean.
- Full-coverage US equity with all applicable dims real -> confidence 1.0 -> clean.
- India equity with real fundamentals, real technicals, real macro -> confidence 1.0 -> clean.
- Malformed historical row with no features.weighting.base_weights -> quality_status='unknown', data_confidence=NULL, not auto-tainted during measure-only.

5. Trade/order tagging

Add nullable columns, additive and safe:

- paper_trades: data_confidence numeric, quality_status text, tainted boolean default false, taint_reason text, excluded_from_learning boolean default false
- broker_orders: same five columns
- observation_labels or the ledger-quality join path: must expose equivalent quality/exclusion fields to validation and feature-learning code before any challenger can use post-feature data.

Population:

- At creation - paper: PaperTrader joins v_decision_quality by paper_trades.signal_id -> decision_observations.signal_id and writes the five fields.
- At creation - live: Execution Gateway joins by broker_orders.proposal_id -> trade_proposals.id -> trade_proposals.signal_id -> decision_observations.signal_id.
- Backfill: One-time job tags existing paper_trades and broker_orders.
- Override: Owner can flip excluded_from_learning manually; every override writes decision_journal or an admin audit row.

Taint rule after enablement:

```
if quality_status='ok' and data_confidence < 0.5:
```

```

tainted = true
excluded_from_learning = true
taint_reason = 'low data_confidence: <degraded/missing dims>'
else if quality_status='unknown' or data_confidence is null:
  measure-only: leave untainted
  learning-enabled: exclude from learner/validation until reviewed

```

6. Learner and validation filtering

The primary pollution fix is not deletion. It is dataset filtering.

Closed-trade learner queries must include:

```

coalesce(excluded_from_learning, false) = false
and coalesce(tainted, false) = false
and data_confidence is not null
and coalesce(quality_status, 'unknown') = 'ok'

```

Apply the equivalent filter to every learning input path:

- app/api/agents/learner/route.ts closed paper_trades queries.
- computeScoreCorrelation() fallback over paper_trades.
- lib/learning/dataset.ts::loadLabeledDataset() / validation engine reads from decision_observations x observation_labels.
- app/api/validation/feature-check/route.ts feature IC evaluation.
- shadow_decisions promotion metrics.

For ledger-based paths, do one of:

- Add data_confidence, quality_status, and excluded_from_learning to observation_labels at label creation time; or
- Join v_decision_quality by observation_labels.observation_id.

Do not depend only on paper_trades tags. The validation engine and feature discovery learn from the observation ledger, not only filled paper trades.

7. Learner mutation recovery

Reuse strategy_versions and learning_priors_history.

- Every weight mutation should already create a new strategy_versions challenger row with parent_version_id and weights_snapshot.
- Recovery is: retire the bad version, re-promote the parent for the same market, and log the reason.
- revert_learner(market, to_version?): owner-gated admin op. to_version defaults to current champion's parent.
- Prior restoration should use learning_priors_history only when the mutation actually changed priors. If prior changes are unrelated user edits, do not roll them back blindly.

Mutation guardrails:

- Reject a candidate trained on any tainted/excluded trade or unknown-quality ledger row after enablement.
- Reject a candidate if validation used any row with quality_status <> 'ok' or data_confidence is null.
- Clamp per-run weight deltas server-side.

- Shadow quarantine: a new candidate must accumulate K shadow outcomes before promotion to champion / production sizing.

8. Reusable safe teardown

Owner-gated admin route. Dry-run is the default.

Verbs:

- `exclude_tainted?market=X`: flags only, no deletion.
- `reset_paper?market=X&dryRun=true`: returns counts and exact operations.
- `reset_paper?market=X&confirm=true`: creates backup under backups/, then transactionally resets mutable paper state.

`reset_paper` may touch only mutable projections for the chosen market:

- `paper_portfolio`
- `paper_positions`
- `paper_trades`
- pending paper-only proposals/orders, if explicitly scoped

It must not delete or mutate:

- `decision_observations`
- `observation_labels`
- `shadow_decisions`
- `paper_order_events`
- `learning_log`
- `strategy_versions`
- `evidence_records`

The reset route must use service-role only, owner auth, dry-run counts, backup-first, and explicit market scoping.

9. What each part protects against

```
| Failure | Guard |
| Bad data -> high score | `v_decision_quality.data_confidence` measures evidence coverage |
| Tainted trade -> learner | `excluded_from_learning` + quality filters |
| Tainted observation -> validation/feature learning | `v_decision_quality` join or label-level quality
columns |
| Confidence calc bug -> silent mass exclusion | measure-only rollout, unknown state, golden tests, monitor,
owner override |
| Bad mutation lands | champion-parent revert + shadow quarantine |
| Manual cleanup breaks app | dry-run + backup + append-only-respecting reset |
```

10. Rollout

- Phase 1 - Measurement: `v_decision_quality` view, structured future `features.quality` logging, golden tests, monitor. No auto-taint yet.

- Phase 1B - Pollution-proofing: trade/order taint columns, observation-label or ledger-quality join, backfill, learner/validation/feature filters. Enable moderate auto-flag only after golden tests and historical flag-rate review are green.
- Phase 2 - Mutation safety: `revert_learner()`, mutation guardrails, shadow quarantine.
- Phase 3 - Ops: admin exclude/reset route with dry-run and backup.

11. Open questions

- K for shadow quarantine. Default proposal: 10 closed shadow outcomes per market, but validate against observed signal frequency.
- Exact `features.quality` shape for future rows.
- Historical degraded-dim heuristics for old rows.
- `trade_proposals.signal_id` type migration from `bigint` to `uuid`.
- Moderate threshold 0.5: validate flag rate across full history before enabling. It should catch the true bad set, not a broad percentage of normal decisions.

12. Diagram / system-map impact

The learner `<- trades <- decision_observations` flow gains a quality gate. On build, update `public/agent-diagrams/system-map.json` and the affected learner/paper-trader diagrams, then append a history entry per project convention.

Reviewer changelog (ChatGPT)

- Section 3: Updated author line to "reviewed/updated by ChatGPT, 2026-07-08" as requested.
- Section 3: Added an explicit migration-order rule: code that reads new taint/quality columns must ship only after its additive migration is applied, and missing columns must not be treated as clean data.
- Section 4.1: Tightened the `data_confidence` denominator rule to fail unknown when structural/base weights are malformed instead of falling back to post-renormalized `applied_weights`.
- Section 5: Added the requirement that validation/feature-learning paths get equivalent quality/exclusion fields through `observation_labels` or a `v_decision_quality` join, not only through `paper_trades`.
- Sections reviewed with no additional change: Sections 1, 2, 7, 8, 9, 10, 11, and 12 were consistent with the known schema/safety constraints after the edits above.

leveraged etf and intraday execution - Feature Architecture

Source: features\leveraged-etf-and-intraday-execution\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Leveraged ETF Sleeve and Intraday Execution Architecture

Status: DRAFT - approval required before implementation Date: 2026-07-26 Scope: US paper book first. India, live trading, inverse ETFs, options, and extended-hours trading are explicitly out of scope.

1. Decision

Kairos should not enable leveraged ETFs through the generic ETF path. A long leveraged ETF is a distinct instrument class with a daily-reset return objective, path-dependent multi-session results, and materially higher gap and execution risk. The correct first product is an isolated, paper-only US sleeve that is disabled by default.

The proposed initial envelope is:

```
| Limit | Initial policy |
| One leveraged ETF | 3% of US paper NAV |
| Entire leveraged ETF sleeve | 5% of US paper NAV |
| Number of leveraged ETF positions | 1 |
| Market | US / USD only |
| Direction | Long leveraged only |
| Default state | Disabled |
```

The 3% and 5% limits are ceilings, not target allocations. A 5% sleeve can lose its entire paid value, or about 5% of the US portfolio at the entry-time NAV, if its products go to zero; a 3x product also creates roughly 15% initial index-equivalent exposure before path effects. Stops do not eliminate gap risk. These are risk-envelope defaults, not evidence-based alpha parameters, and must not be raised automatically.

The broader goal is to borrow institutional *discipline*, not to imitate Renaissance or Medallion. Kairos does not have their proprietary data, dense diversified signal library, low-latency execution, capacity research, or decades of independent outcomes. The relevant, achievable standard is reproducible point-in-time research, realistic costs, strict market-local risk limits, out-of-sample promotion, and operational monitoring. No return target or claim of Medallion-like capability is valid.

Quant capability target

```
| Capability | Kairos direction | What not to pretend |
| Data provenance | Freeze evidence, policy, code, and quote fingerprints per decision | That a retail/free-
data feed is an institutional consolidated low-latency feed |
| Research validation | Point-in-time replay, walk-forward, purging, costs, and regime slices | That a
backtest proves a live edge |
| Portfolio construction | Market-local constraints, concentration/correlation controls, and explicit
sleeves | That a small number of paper outcomes calibrates a universal optimizer |
| Execution | Fresh quote, spread, drift, idempotency, reconciliation, and broker protections | That cron
polling can compete with a market maker or HFT firm |
| Learning | Separate populations and promotion defaults-to-no-change | That an LLM or a short sample can
discover stable alpha |
```

These are the useful quantitative similarities to pursue. Additional indicators, more agents, or frequent retraining are not substitutes for these controls and are not part of this feature.

2. Existing State and Gaps

Existing controls that must remain in force

- lib/trading/symbol-policy.ts blocks leveraged and inverse ETFs from generic research, paper, and live paths.

- `lib/asset-classification.ts` identifies a static set of known US ETFs, including several leveraged products. This is insufficient as an authority for a new money path because it does not model leverage factor, direction, issuer, or status.
- The generic US ETF allocation cap in `lib/trading/execute-order.ts` is a global ETF cap. It is not a leveraged-sleeve cap and cannot authorize a leveraged trade.
- Paper entry attempts currently occur at two deterministic in-session windows: US at 15:15 UTC and 19:15 UTC, and India at 09:40 IST and 13:15 IST. The US wakes are 11:15/15:15 EDT and 10:15/14:15 EST, respectively. They are not random or continuously polling entries, but static UTC cron cannot preserve an exact ET minute across daylight saving.
- The live auto worker and live exit monitor exist behind false-by-default deployment and database gates. They must remain disabled for this feature's paper phases.

Gaps that block an enablement

- Static ticker lists cannot safely classify all future leveraged, inverse, single-stock, or renamed funds.
- The current technical score is a general equity score. It does not measure daily reset/path risk, underlying trend persistence, intraday liquidity, or execution spread for a leveraged product.
- Paper PositionMonitor is not a reliable intraday protective system. A periodic quote check cannot protect an outage or overnight gap.
- Existing paper results are not an isolated leveraged-ETF experiment and cannot justify sizing or live activation.

3. Instrument Classification and Allowlist

Create a server-owned `tradable_instruments` policy record rather than inferring from a ticker. The initial record fields are:

```
type InstrumentPolicy = {
  symbol: string;
  market: "us" | "india";
  currency: "USD" | "INR";
  assetType: "equity" | "etf";
  leverageClass: "none" | "long_2x" | "long_3x" | "inverse" | "leveraged_inverse" | "unknown";
  underlyingSymbol: string | null;
  sleeve: "core" | "leveraged_us" | "blocked";
  enabledForPaper: boolean;
  enabledForLive: boolean;
  effectiveFrom: string;
  effectiveTo: string | null;
  source: string;
  reviewedAt: string;
};
```

Rules:

- Only explicit `long_2x` or `long_3x`, `sleeve='leveraged_us'`, and `enabledForPaper=true` may reach the isolated paper candidate flow.
- `inverse`, `leveraged_inverse`, `unknown`, single-stock leveraged products, crypto

leveraged products, and every unlisted ticker remain blocked. The existing paper- only hedge exception remains separate and may not use this sleeve.

- The first allowlist must contain only broad, liquid, long index/sector products.

It must not include single-stock or crypto leveraged ETFs. Each addition is an owner-reviewed policy change with an audit record, not a watchlist side effect.

- A policy lookup/read error is a money-path denial. It may not fall back to the old static ticker set.

4. Leveraged-ETF Decision Model

No LLM may decide eligibility, instrument class, size, stop, target, or exit. LLM output can explain a completed deterministic decision only.

The leveraged model is a second deterministic gate after the normal new-long gates; it can only reject or reduce an otherwise valid candidate.

Required measurements

```
| Measurement | Purpose | Initial use |
| Underlying trend and persistence | Daily-reset leverage benefits more from persistent than choppy moves |
Require a pre-declared, paper-validated underlying trend regime; no ad-hoc threshold |
| Realized volatility and ATR-normalized range | Detect path-risk / volatility drag conditions | Reject or
reduce exposure above a market-local calibrated ceiling |
| Gap and overnight risk | Daily stops do not control discontinuous loss | Require a maximum expected loss
and separate overnight policy |
| ETF quote age, bid/ask spread, dollar volume | Avoid paying large implementation shortfall | Fail closed
on stale/unavailable quotes or excessive spread; thresholds calibrated from observed data |
| Underlying/ETF tracking sanity | Catch stale, dislocated, halted, or bad price data | Reject on abnormal
divergence, stale underlying, or a trading halt |
| Portfolio beta/correlation | Prevent duplicate exposure (for example, QQQ plus TQQQ) | Enforce against
the US book using the existing correlation framework; never cross-sum India |
| Event risk | Avoid automatic entry immediately before known binary events | Start with an exclusion
policy for underlying index/sector events only where reliable data exists |
```

MACD, RSI, EMA, ATR, volume confirmation, and relative strength may be evaluated as features in the isolated experiment. They are not sufficient by themselves, and none should receive a permanent weight without point-in-time, costed, market-local evidence. Fibonacci levels, Elliott wave, and similar discretionary chart patterns are not part of this money path because they add degrees of freedom without a validated decision contract.

Sizing and exits

```
leveraged_notional = min(
    normal_new_long_notional,
    3% of current US paper NAV,
    remaining 5% US leveraged sleeve capacity,
    liquidity/spread-constrained notional,
    volatility-scaled risk budget,
    all existing name/sector/gross/correlation/cash constraints
)
```

- Recalculate current NAV and all marks in USD at the same decision timestamp.
- Never use cost basis, stale cache values, or cash alone for sleeve capacity.
- A generic 7% stop / 20% target must not be copied blindly. The experiment records

candidate volatility-normalized exits, but the active paper policy starts with a conservative deterministic stop/time-risk plan defined before entry and immutable on the trade record.

- No capital rotation into or out of this sleeve in the first phase. It would mix

the new experiment with the established alpha book and amplify churn.

- The LearnerAgent cannot mutate the core champion or learn core signal weights from these outcomes. Leveraged outcomes have their own evaluation population.

5. Execution Sessions

Regular-session hours are 09:30-16:00 ET for US equities and 09:15-15:30 IST for NSE equities. Kairos must remain regular-session-only. Extended-hours orders, random polling, and "trade whenever a score changes" are out of scope.

The opening and closing periods have higher uncertainty, volatility, and often worse spreads. A study of intraday trading activity finds U-shaped volume and price variability, and liquidity research also documents wider spreads at the edges of the day. Therefore, the product uses fixed decision windows, not a claim that any single minute is universally optimal.

```
| Flow | US equities / ordinary ETFs | India equities | Leveraged-US ETF sleeve |
| Research | Existing morning and afternoon cycles | Existing morning and midday cycles | Reuse only a
fresh same-session deterministic snapshot |
| New entry | Keep the existing two in-session wakes; normalize any future exact local-time policy in the
route, not only in UTC cron | Keep existing 09:40 and 13:15 IST windows | One dedicated paper window,
initially 11:00 ET, after the opening auction settles and before late-day rebalance pressure; never at the
open or within 60 min of close |
| Routine exit review | Existing PositionMonitor policy | Existing PositionMonitor policy | Same end-of-
session plan plus a separately qualified risk monitor |
| Emergency exit | Not a reason to add random entry polling | Not a reason to add random entry polling |
Risk-reducing only: immediately actionable when reliable quote/broker state says the immutable stop/
disaster rule fired |
```

The exact 11:00 ET leveraged entry time is a conservative starting experiment, not a universal alpha claim. It must be compared with the existing fixed windows using timestamped, costed paper evidence before it becomes a permanent policy. A cron may wake the endpoint more broadly, but America/New_York time-window code must be the single execution authority; DST can never silently move a policy window.

Intraday exits: correct order of implementation

- Do not add an intraday exit cron using stale/free daily prices. It could sell on a bad mark and create a false sense of protection.
- Build a quote-quality contract: executable quote timestamp, bid, ask, spread, source health, halt/session status, and broker-held quantity.
- In paper, record every would-exit at a fixed 15-minute cadence, but do not alter the established book until mark quality and false-trigger statistics are known.
- For any future live entry, a broker-resident disaster floor/protective order plus reconciliation is required. The app monitor is a second line, not the primary protection against an outage or gap.
- A risk-reducing verified SELL may run outside entry windows during the regular session. It remains subject to held-quantity/reconciliation guards, never a BUY budget, and never opens a short position.

6. Evidence and Promotion

The sleeve has a separate immutable experiment lineage. Each decision stores the instrument-policy version, underlying and ETF quotes, features, rule version, entry window, model outcome, transaction-cost assumptions, and market-local session date.

Required evidence before any expansion:

- point-in-time replay with correct fund availability and no survivorship additions;

- walk-forward evaluation and purged/time-separated validation;
- costs and conservative bid/ask/slippage sensitivity, not close-to-close returns;
- distinct regime slices (trending, high-volatility, mean-reverting) and a minimum number of independent setups, not daily observations counted as independent;
- comparison with the same underlying unlevered ETF and a cash/no-trade baseline;
- maximum drawdown, gap, false-exit, quote-quality, and reconciliation reporting;
- an owner-approved paper evidence threshold. Failure means no change, not a lower threshold.

No result from this experiment may promote a core strategy version, relax a generic ETF cap, expand the live auto lease, or change India logic.

7. Phased Delivery

L0 - policy and observability

- Add the classified instrument registry, owner-visible sleeve status, and a strict default-deny rule.
- Add a pure, tested market-local execution-window policy. It changes no existing schedule or order behavior in L0; later phases use it to enforce an approved ET window irrespective of UTC/DST.
- Keep all leveraged instruments blocked from generic research and execution.
- Add no cron, no provider, no live broker call, and no scoring influence.

L1 - measure-only research

- Produce deterministic candidate feature records only for explicitly allowlisted instruments, paced separately from the core research quota.
- Capture quote quality, regime, normalized volatility, and would-enter/would-reject decisions. No paper fills.

L2 - isolated paper sleeve

- Enable one US paper-only position under the 3%/5% envelope.
- Use the dedicated 11:00 ET entry window, immutable entry plan, and separate outcomes/analytics.
- Enable 15-minute *would-exit* observations only after quote contract tests pass.

L3 - paper risk monitor

- If L2 shows reliable marks, activate deterministic risk-reducing paper exits at the approved cadence. Continue end-of-session monitoring as the fallback.
- Demonstrate no duplicate exits, no stale-mark exits, and correct market-local session behavior in failure injection.

L4 - live design review only

- This is not authorization to trade live. It requires broker-native protective support, reconciliation, a live-specific allowlist, live account mandate, small owner-approved lease, and a separately approved architecture.

8. Acceptance Criteria

- An unclassified, inverse, leveraged-inverse, single-stock, crypto, India, or expired allowlist instrument is denied before any score, claim, fill, or order.
- The sleeve cannot exceed 3% for one name or 5% in aggregate using current US NAV.
- India INR holdings and capacity never participate in US sleeve calculations.
- A missing policy, quote, mark, NAV, spread, or correlation read rejects a new leveraged entry; it never falls back to the generic ETF path.
- Leveraged results are excluded from core LearnerAgent promotion/weight mutation.
- A repeated worker, stale quote, quote divergence, halt, partial exit, broker uncertainty, or kill switch cannot create a duplicate order or oversell.
- New entries occur only in their approved regular-session window. Verified risk-reducing exits may occur during regular session outside that window.
- No production flag, mandate, account, generic ETF cap, or live order capability changes in L0-L3.

9. Sources

- FINRA, The Lowdown on Leveraged and Inverse Exchange-Traded Products (<https://www.finra.org/investors/insights/lowdown-leveraged-and-inverse-exchange-traded-products>): daily reset and the greater divergence caused by leverage and volatility.
- FINRA, Non-Traditional ETFs FAQ (<https://www.finra.org/rules-guidance/key-topics/etf/non-traditional-etf-faq>): suitability must consider volatility, leverage, and holding period.
- Wei (1992), Intraday Variations in Trading Activity, Price Variability, and the Bid-Ask Spread (<https://doi.org/10.1111/j.1475-6803.1992.tb00804.x>): empirical U-shaped intraday activity and variability.
- NYSE, Trading Information (https://www.nyse.com/trade/trading-information?os=io_): US core session hours.
- NSE, Market Timings and Holidays (<https://www.nseindia.com/resources/exchange-communication-holidays>): NSE regular equity session hours.

10. Explicit Non-Goals

- No claim of Renaissance/Medallion equivalence.
- No HFT, market making, order-book prediction, extended-hours trading, options, inverse ETF trading, or cross-market/currency netting.
- No new LLM authority, external GitHub skill runtime, or provider-quota increase.
- No live leveraged trade, even manually, under this architecture without the L4

approval and a distinct live safety review.

live auto trading - Feature Architecture

Source: features\live-auto-trading\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Live Auto Trading - Feature Architecture

Last updated: 2026-07-10 Status: IMPLEMENTING PA3 - per-market off/manual/autonomous mode (approved by owner)
Authors: Claude Sonnet 4.6; security and execution architecture reviewed by ChatGPT/Codex, 2026-07-10 Scope:
Per-market autonomous mode (US via Robinhood direct REST; India via Kite REST). Both markets support off/manual/autonomous independently.

1. Safety correction and goal

The previous draft was not safe to implement. It used the wrong Robinhood account number (605420606), bypassed the hardened Execution Gateway by calling `submitRobinhoodOrder()` directly, used a race-prone read/sum daily cap, allowed a fallback NAV, and proposed auto BUY before live exit/reconciliation was complete.

Authorized order account: 605420660 only. Read-only account 965848641 may be used only for the explicitly approved holdings-research path and must never price, size, authorize, or execute an order for the agentic account.

The goal remains valid: after Kairos proves a scoring version in shadow/paper evidence, Vaibhav may enable an autonomous risk envelope. Individual orders inside that envelope need not require a click. Enabling autonomy does not allow an LLM to control money limits, activate scoring versions, choose accounts, or bypass deterministic gates.

2. Required architecture

There must be one shared execution kernel for manual and autonomous orders.

```
Research/Scoring -> eligible signal -> proposal
                        ?
                        authorization context
manual_owner | autonomous_worker
                        ?
                        executeApprovedProposal()
(same account/risk/quote/budget/order gates)
                        ?
                        broker adapter
                        ?
broker_order + append-only events
                        ?
sync/reconcile partial fill/fill/cancel/reject
                        ?
live position + protective exit
```

Shared execution kernel

Extract the money-moving logic in `app/api/broker/orders/route.ts` into a server-only function, for example:

```
executeApprovedProposal({
  proposalId,
  env: "live",
  actor: { kind: "owner", userId } | { kind: "autonomous_worker", runId },
})
```

The owner route keeps `requireOwner()` and calls this function. A new cron/worker route uses `verifyCronSecret()`, obtains a single-run lease, and calls the same function. Neither caller may invoke `submitRobinhoodOrder()` directly.

The kernel owns all gates: broker/account resolution, autonomy policy, kill switches, signal quality, scoring-version eligibility, fresh account state, per-order cap, portfolio limits, quote freshness/drift, held-position SELL, atomic daily budget reservation, broker schema/review, idempotency, durable state transitions, and alerts.

3. Authorization and activation

Autonomy uses independent keys that all must pass:

- deployment flag `AUTONOMOUS_LIVE_ENABLED=true` (false/absent by default);
- `strategy_config.autonomy_level = 'L4_live_small_auto'`;
- `live_auto_enabled = true`;
- unexpired `live_auto_enabled_until` lease;
- global and US trading switches on;
- Robinhood MCP enabled and token healthy;
- active account resolves from the allowlist to 605420660 with role `trading`;
- active scoring strategy is `live_approved` with linked validation evidence;
- no unresolved critical trading/data/reconciliation alert.

Unknown or missing values fail closed. Do not replace the deployment kill constant with only a DB boolean. The environment flag is a release control; the DB toggle is an owner runtime control.

Settings changes require owner authorization, CSRF/request guards, a recent session or vault-PIN re-auth, typed `ENABLE AUTO`, and an append-only decision-journal record containing old/new policy, actor, timestamp, and expiry. L4 begins as a maximum 24-hour renewable lease. A future L5 design may extend the lease only after evidence.

LLMs and agents cannot change any activation key, cap, account, strategy lifecycle, or lease.

4. Database changes (additive)

Use the next real migration number. Apply and verify migrations before code.

`strategy_config`

```
live_auto_enabled boolean not null default false
live_auto_enabled_until timestampz
live_auto_policy_version integer not null default 1
live_auto_daily_cap_usd numeric
live_auto_max_per_order_usd numeric
live_auto_min_evidence_confidence numeric
live_auto_max_open_positions integer
live_auto_max_orders_per_day integer
```

Add non-negative/range checks. Defaults must be conservative and never larger than the existing `max_order_notional_usd`, `max_daily_notional_usd`, or `max_daily_trades`; runtime uses the minimum of global/account/auto limits. Do not create parallel caps that can enlarge an existing cap.

`trade_proposals`

Add `execution_mode ('manual'|'auto')`, `policy_snapshot jsonb`, `auto_run_id uuid`, and `auto_decided_at`. Existing proposal statuses should remain business-intent statuses. Do not overload them with broker fill state; `broker_orders` owns execution state.

Add only schema-compatible status values after inspecting the current CHECK constraint. Suggested proposal states: pending_review, approved, queued_auto, submitted, manual_review_required, failed, expired, cancelled. Ambiguity lives on the broker order as unknown_needs_reconcile.

Budget reservation and actor audit

The current reserve_live_order_budget serializes per market/day correctly, but hardcodes approved_by_user=true. Do not use it unchanged for autonomous orders.

Create a new versioned RPC (for example reserve_live_order_budget_v2) rather than accidentally creating an overloaded function with ambiguous PostgREST resolution. It must:

- be SECURITY DEFINER only if necessary;
- set a fixed safe search_path;
- validate every enum/value and reject null/invalid notional for live BUY;
- accept execution_actor and persist approved_by_user = (actor='owner');
- use a market-local-day advisory transaction lock;
- count reserved/submitted/partial/unknown BUY notional so timeouts cannot reopen budget;
- exempt held-position SELL exits from BUY budget;
- insert the pending_submit broker row in the same transaction;
- be revoked from PUBLIC, anon, and authenticated, granted only to service_role;
- preserve the unique active-order-per-proposal backstop.

RLS remains enabled on exposed tables; service-role-only tables have explicit revoked grants. Run Supabase advisors and verify RPC permissions.

Append-only execution events

Add broker_order_events as an append-only lifecycle ledger if it does not already exist. broker_orders remains mutable current state. Each transition records broker/order IDs, event type, quantities, prices, raw hash/redacted payload, actor/run, and timestamp. Never store tokens.

5. Autonomous proposal eligibility

A proposal may be queued for auto execution only when all are true:

- market is US;
- account target is exactly 605420660 after allowlist resolution;
- side is BUY for a new long or SELL for a verified held position;
- signal source is deterministic and scoring strategy lifecycle is live_approved;
- signal/quote/proposal are within configured freshness windows;
- evidence confidence meets the auto threshold with no override;
- no LLM veto, taint, unknown quality, or unresolved provider substitution;
- mandate permits the asset/setup;
- no earnings/ex-dividend/corporate-action blackout required by the approved setup policy;

- a protective exit can be established and monitored;
- proposal has a deterministic idempotency key and policy snapshot.

`analyst_score >= 80` alone is not eligibility. A 0-100 heuristic is not a calibrated probability. Prefer validated `expected_return_bps`, `uncertainty`, `setup lifecycle`, and `evidence gates` when available.

The autonomous path has no `acceptLowQuality` or `acceptPortfolioRisk` override. Those remain owner-only manual actions with audited reasons.

6. Pre-submit gate order

Every gate is rerun immediately before broker submission:

- actor authorization and single-worker lease;
- policy lease/version and current strategy lifecycle;
- trading flags, broker token, and account allowlist;
- kill switches and critical-alert check;
- proposal expiry, source/version, direction, quality, and mandate;
- fresh agentic-account portfolio/positions; no fallback NAV;
- valid integer quantity and held quantity for SELL;
- minimum of per-order/global/auto caps in USD;
- portfolio concentration/correlation/volatility limits;
- fresh executable quote, max age, spread, and price-drift collar;
- atomic daily budget/idempotency reservation;
- `review_equity_order` with symbol/side/qty/account echo verification;
- broker submit;
- durable result/event write before returning.

Any read error, null, stale state, malformed preview, or failed audit on a money path fails closed. Do not use `FALLBACK_NAV`. Do not downgrade a failed auto order to an immediately executable manual command; create `manual_review_required` and force a new owner review/fresh gates.

For swing entries, prefer a marketable limit with a price collar if the MCP tool schema supports it. If only market orders are supported, constrain liquidity/spread/time window and document that risk. Never guess MCP parameters; discover the live tool schema and make the preview echo mandatory.

7. Exit protection is a launch blocker

Autonomous BUY cannot launch before an operational live exit path exists.

Minimum:

- current agentic holdings synced frequently during market hours;
- protective stop policy recorded at entry;
- deterministic held-position SELL through the same Execution Gateway;
- SELL quantity capped to verified available quantity after open orders/partial fills;

- stop/time/target/kill-switch exits prioritized above new entries;
- protective SELL is never blocked by BUY notional/daily budget;
- if broker-native stop/bracket support exists, preview and place it deterministically;
- if no broker-native protection exists, the runtime must have heartbeat/staleness alerts and the L4 allocation must remain tiny.

If protective exit cannot be installed or the monitor is stale, reject the BUY and disable/expire L4. Tax or dividend preferences may influence normal exits but cannot delay a risk stop.

8. Order lifecycle and reconciliation

External exactly-once execution cannot be assumed. Use durable at-most-once intent plus reconciliation:

- reserve before submit;
- send a client idempotency key if supported;
- ambiguous timeout/no broker ID -> unknown_needs_reconcile, keep budget reserved, block symbol/proposal retry;
- clean rejection -> error/rejected, release only according to explicit RPC policy;
- partial fill -> persist filled and remaining quantities; never blindly resubmit remainder;
- order-sync worker queries broker order state during market hours and resolves submitted/partial/unknown states;
- reconciliation compares broker orders, fills, positions, proposals, and local ledger;
- any mismatch opens a critical alert and blocks new autonomous entries while allowing verified risk-reducing exits.

Kill-switch behavior: stop new orders and attempt to cancel resting entry BUY orders if an authenticated, tested cancel tool exists. Never cancel protective SELL orders automatically. If cancel capability is unavailable or fails, alert critically and instruct broker-level intervention.

Email is notification only. Durable DB alerts/events are the source of truth; email failure must not erase the order result.

9. Position sizing

Sizing is deterministic and bounded:

```
raw opportunity size = portfolio constructor(calibrated edge, uncertainty, volatility, correlation)
final notional = min(raw size,
    global per-order cap,
    auto per-order cap,
    remaining daily auto budget,
    mandate/name/sector/gross limits,
    available buying power)
```

If calibrated edge or NAV is unknown, autonomous BUY size is zero. Paper and manual pathways may show a proposal, but auto must abstain. Integer quantity rounds down; zero shares means no order. Recompute with a fresh quote immediately before reservation.

The LLM cannot size. A "great opportunity" can increase size only through the validated numeric opportunity model and only up to every hard ceiling.

10. Rollout

PA0 - architecture/schema/UI only

- additive migrations, RLS/grants/checks;
- Settings card and audited time-bounded enablement;
- deployment flag remains false;
- no autonomous order route.

PA1 - shadow autonomous decisions

- extract and test shared execution kernel;
- auto worker evaluates proposals and records would-submit/would-block reasons;
- no broker submit;
- compare shadow decisions to manual outcomes for at least the approved evidence window.

PA2 - reconciliation and live exits

- broker-order event ledger and sync worker;
- partial-fill/unknown reconciliation;
- deterministic live held-position exits/protective policy;
- chaos tests for timeouts, duplicate cron, stale account, quote failure, and DB failure.

PA3 - tiny L4 live

- owner promotes a scoring version to `live_approved`;
- explicit deployment flag + 24-hour DB lease;
- one order at a time, tiny caps, liquid allowlist, no unresolved critical alerts;
- watched first executions and rollback drill.

PA4 - evidence-based expansion

Increase limits only by owner action after net live results, reconciliation accuracy, drawdown, and operational SLOs pass. No automatic cap expansion.

India auto trading is a separate architecture. It must use Kite's official HTTP API through the same execution kernel, INR-only limits, CNC product, exchange/order validation, token health, and live exit/reconciliation parity.

11. Acceptance tests

- Any account other than 605420660 is rejected before reserve/preview/submit.
- Auto caller cannot call the broker adapter directly; static test verifies only shared kernel imports it.
- Deployment flag false, expired lease, invalid autonomy level, or DB read error blocks auto.
- Two concurrent runs for one proposal yield one reservation and at most one submit intent.

- Concurrent different proposals cannot exceed daily count/notional.
- Auto broker row records `approved_by_user=false` and autonomous run ID.
- Unknown outcome remains budget-reserved and cannot be retried.
- Preview mismatch on account/symbol/side/qty blocks submit.
- Missing/stale NAV or holdings blocks BUY and SELL authorization as appropriate; no fallback NAV.
- Low/unknown quality and portfolio-risk overrides are impossible in auto.
- Scoring version not `live_approved` cannot auto trade regardless of score.
- Partial fill cannot cause oversell or automatic remainder resubmit.
- Protective exit failure blocks new entry.
- Kill switch blocks entries, preserves/permits risk-reducing exits, and handles resting BUY cancellation safely.
- Secrets/tokens never appear in logs, events, emails, or raw payloads.

13. PA3 Implementation Details (2026-07-10)

Per-market mode

New columns on `strategy_config` (migration 141):

- `live_auto_mode_us` TEXT DEFAULT 'manual' CHECK IN ('off','manual','autonomous')
- `live_auto_mode_india` TEXT DEFAULT 'manual' CHECK IN ('off','manual','autonomous')

Three modes per market:

- `off` - autonomous-live cron skips this market entirely
- `manual` - TraderAgent creates proposals; owner clicks Approve (existing flow, unchanged)
- `autonomous` - execution kernel + live broker REST submit from cron, no per-order click

New cron: autonomous-live (14:00 UTC, weekdays)

Runs AFTER research (13:00 UTC / 9 AM ET) so same-day signals exist. Route: POST

/api/agents/autonomous-live/cron - timing-safe CRON_SECRET auth. Shadow cron at 07:30 UTC unchanged.

US execution path (no MCP in serverless)

Direct Robinhood REST via `lib/brokers/robinhood/rest-client.ts`:

- Token from vault key `ROBINHOOD_MCP_ACCESS_TOKEN`
- GET `https://api.robinhood.com/instruments/?symbol=X` -> instrument URL
- POST `https://api.robinhood.com/orders/` - market, gfd, account=605420660
- `submitRobinhoodOrder()` (MCP) is NOT called - requires live MCP session context unavailable in serverless

India execution path

Calls `placeEquityOrder()` from `lib/kite.ts` directly - same REST path as the owner-click flow. Note: requires fresh daily Kite token; if stale, order = `unknown_needs_reconcile`.

Budget reservation

Uses `reserve_live_order_budget_v2` with `p_execution_actor='autonomous_worker'` which sets `approved_by_user=false`.

Missing migration 139 column fix

`trade_proposals.market` was not added by migration 139 but IS written by `autonomous-shadow.ts` - shadow cron inserts fail. Migration 141 adds it.

12. What Claude must not do

- Do not implement the old direct `submitRobinhoodOrder()` branch.
- Do not use 605420606; it is wrong.
- Do not remove the deployment kill flag.
- Do not build PA3 before PA1/PA2 evidence and Vaibhav approval.
- Do not use `market-data/portfolio` fallbacks on autonomous money paths.
- Do not duplicate the Gateway's checks in `TraderAgent`; extract and reuse one kernel.
- Do not call a same-app HTTP route internally for execution.
- Do not enable autonomous BUY without live exit and reconciliation.
- Do not mutate caps, lifecycle, account, or code through any LLM/tool loop.

live trading hardening - Feature Architecture

Source: features\live-trading-hardening\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature Architecture: Live-Trading Hardening (proposal consolidation, order-status sync, snapshot parse, pre-send confirmation, broker-side protective stops)

Status

Architecture status: P1 + P2 Implemented; P3-P5 Draft (unapproved) Architecture approved: P1 + P2 ("build", 2026-07-07)
Approved scope: P1 (proposal consolidation) + P2 (Robinhood order-status sync) Approved date: 2026-07-07
Implementation allowed: P1 + P2 done; P3-P5 still gated

Implementation notes (2026-07-07)

- P1 - trade_queue retired as a live path. /api/agents/trade, /trade/approve, /trade/reject now return 410. All generation -> /api/agents/trader (canonical trade_proposals). Repointed: AgentsPage "Run TraderAgent" button, the dead /api/markets/smart-money API (now reads trade_proposals aliased), and TradingPage + PortfolioPage approve/reject (-> /api/agents/trader). smart-money/page.tsx already read trade_proposals (the real-money hazard was already patched). Follow-up flagged: TradingPage/PortfolioPage "queue" tabs still render raw agent_signals, so their Approve needs a proposal id - product-UX decision (spawned task).
- P2 - robinhood_mcp adapter getOrder implemented via a deterministic get_equity_orders callTool (mcpToolJson-parsed, escaped-safe) -> state map. The kairos-broker-sync loop now transitions Robinhood orders submitted->filled, writes filled_qty/avg_fill_price to broker_orders and fill_price/fill_qty/filled_at (status=executed) back to the trade_proposal, resolves the order-needs-reconcile System Health alert, and journals the fill.

Still Draft / not built

- P3 live-account snapshot parse fix, P4 pre-send confirmation screen, P5 broker-side protective stops (the real live-risk fix). Unapproved.

Why this feature exists

The in-app Robinhood MCP live-order path now works end-to-end (first real order placed 2026-07-07). But the surrounding machinery has correctness gaps and a real live-risk hole that surfaced during that first order. This feature makes live trading correct, trackable, safe to operate by hand, and - critically - protected by broker-side stops so a live position isn't left unguarded when the app/cron isn't running. It does NOT add autonomous execution (separate, later, deliberately gated) and does NOT add limit-order entries (low value for this app's daily-cadence style).

Sequenced in phases; each is independently approvable and shippable. Recommended order: P1 -> P2 -> P3 (small) -> P4 -> P5.

Non-Goals

- No autonomous live execution. Every live order still requires a human

approve + send. `trading_mode='auto'` stays unimplemented; its architecture is a separate doc drafted only after a paper track record justifies it.

- No limit-order entries. Daily-cadence trades on liquid names are fine with

market orders; limit-order management (unfilled orders, cancel/replace, GTC tracking) is complexity this app's edge doesn't need. Revisit only if a real need appears (e.g. routinely trading illiquid names).

- No standalone live-orders history page in this feature - `broker_orders`

plus Robinhood's own app already hold the record. A minimal orders list may ride along with P4, but a polished history/analytics page is out of scope.

- No change to the paper-trading loop's simulation math.

Phase 1 - Consolidate the proposal system (`trade_queue` -> `trade_proposals`)

Purpose / current bug

Two tables model "a proposed trade":

- `trade_queue` - written by the legacy generator `app/api/agents/trade` and

READ by the Smart Money Trade Queue UI (`/api/markets/smart-money`).

- `trade_proposals` - written by the main `app/api/agents/trader` and the ONLY

table the Execution Gateway (`app/api/broker/orders`) reads.

So the UI shows `trade_queue` rows, but the Gateway acts on `trade_proposals`. The paper/live "send" buttons pass the displayed row's id as a `proposal_id` to the Gateway - which looks it up in `trade_proposals`. If the ids don't align (they don't in general), the Gateway either 404s or, worse, acts on a different proposal with the same id. This is a real-money correctness hazard - it's why the Send-LIVE button was NOT shipped.

Proposed behavior

- `trade_proposals` becomes the single source of truth for proposals.

- `/api/markets/smart-money` (and any UI reading `trade_queue`) reads

`trade_proposals` instead; the Trade Queue renders `trade_proposals` rows, so the id sent to the Gateway is guaranteed correct.

- The legacy `/api/agents/trade` generator (writes `trade_queue`) is retired or

repointed to write `trade_proposals`. Its approve/reject routes (`/api/agents/trade/approve|reject`) are consolidated onto the `trade_proposals` approve/reject the trader route already uses.

- `trade_queue` is left in place (not dropped) but no longer written/read; a

follow-up migration can archive it once verified unused.

Files (verify against live code before building)

- `app/api/markets/smart-money/route.ts` (read `trade_proposals`)

- `components/dashboard/SmartMoneyPage.tsx` (field-name mapping to `trade_proposals` columns; the queued Send-LIVE button becomes safe here)

- `app/api/agents/trade/route.ts` + `trade/approve|reject` (retire/repoint)

- Any other reader of `trade_queue` (grep before changing).

Acceptance criteria

- The Trade Queue UI renders `trade_proposals`; a row's id equals the `proposal_id` the Gateway resolves - verified by sending a paper order and confirming the Gateway operates on the same row shown.
- No code path sends a `trade_queue` id to the Gateway.
- No behavior change to how proposals are generated/approved beyond the table.

Phase 2 - Robinhood order-status sync (unconfirmed -> filled)

Purpose / current gap

After a live Robinhood order is placed it sits at Robinhood state `unconfirmed`, then `fills`. The app never learns: `lib/brokers/adapters/robinhood-mcp.ts`'s `getOrder()` is a stub, and the sync loop (`app/api/broker/orders/sync`) has no Robinhood branch. So `broker_orders.status` stays `submitted` and the fill `price/qty/position` never update in-app.

Proposed behavior

- Implement the Robinhood adapter's `getOrder(brokerOrderId)` via a deterministic MCP `get_equity_orders` (or the single-order variant if the server exposes one) `callTool`, parsed with `mcpToolJson` (the escaped-content-safe helper added when fixing the first order). Map Robinhood order state (`unconfirmed/confirmed/partially_filled/filled/cancelled/rejected`) -> the adapter's `BrokerOrderState.status` union.
- The existing sync cron (`kairos-broker-sync`, every 30 min market hours) already iterates DISTINCT brokers in open orders - it will pick up `robinhood_mcp` orders once `getOrder` works. Update `broker_orders` (`status`, `filled_qty`, `avg_fill_price`, `raw_last_state` redacted) + on fill write back `trade_proposals.fill_price/fill_qty/filled_at` and a `decision_journal` fill entry.

Files

- `lib/brokers/adapters/robinhood-mcp.ts` (`getOrder`, maybe `cancelOrder`)
- `lib/robinhood-mcp.ts` (a `queryRobinhoodOrder(id)` helper if useful)
- `app/api/broker/orders/sync/route.ts` (confirm Robinhood orders flow through)

Acceptance criteria

- A placed Robinhood order transitions in-app `submitted` -> `filled` after the sync runs, with the real fill `price/qty` recorded.
- `mcpToolJson` parsing handles Robinhood's escaped text content (no false "unparseable" like the first order hit).
- Tokens/account numbers never persisted in `raw_last_state`.

Phase 3 - Live-account snapshot parse fix (small)

Purpose / current bug

refreshViaMcp (in app/api/live-account/refresh-snapshot) connected and ran get_accounts/get_equity_positions, but its regex parse of the escaped MCP text content left equity/buying_power/portfolio_value NULL and positions_json empty. So the live holdings view shows nothing.

Proposed behavior

- Reparse using mcpToolJson: extract account equity/buying_power/portfolio_value and the positions array from the parsed object shape (to confirm against a real get_accounts/get_equity_positions response), not a flat regex.
- Same fix applies to robinhoodHeldQty (the sell-if-held gate) - parse the positions via mcpToolJson so live SELLS validate correctly.

Acceptance criteria

- After a cloud MCP snapshot refresh, live_account_snapshots for the trading account shows non-null equity + a populated positions array.
- robinhoodHeldQty returns a correct held quantity for a known holding.

Phase 4 - Pre-send confirmation screen (real-money "don't fat-finger")

Purpose

Today a live order is sent with only a JS confirm() (or a raw console call). Before committing real money the user should see Robinhood's OWN reviewed numbers (from review_equity_order) - symbol, side, qty, order type, estimated price/cost, account - and explicitly confirm THAT.

Proposed behavior

- A new owner-only endpoint POST /api/broker/orders/review runs the Gateway's pre-flight (all gates) + the broker's review_equity_order and returns the reviewed order preview WITHOUT placing anything.
- The Send-LIVE flow becomes: click -> modal shows the reviewed preview (price, qty, est cost, account, cap headroom, any gate warnings) -> user confirms -> Gateway places (reusing the already-fetched review where possible).
- Applies to all brokers (Alpaca/Kite show their equivalent preview).

Files

- app/api/broker/orders/review/route.ts (new, owner-gated)
- lib/robinhood-mcp.ts (a reviewRobinhoodOrder() that returns the parsed

preview; today review happens inside `submitRobinhoodOrder` - split it out)

- `components/dashboard/SmartMoneyPage.tsx` (confirmation modal) - depends on P1

so the button acts on the right table.

Acceptance criteria

- No live order is placed without the user seeing the broker-reviewed numbers and confirming them.
- The review endpoint never places an order (pure preview) - verified.

Phase 5 - Broker-side protective stops (the real live-risk fix)

Purpose / current gap (most important item)

Stops/targets are managed APP-SIDE: `PositionMonitor` polls prices and sends a market exit when a stop/target is hit. Fine for paper (simulated). For LIVE money it's dangerous: if the app or its cron isn't running (deploy down, cron missed, overnight gap), a live position has NO broker-side stop - it can gap against you with nothing protecting it. Real capital needs a resting stop at the broker.

Proposed behavior (to be refined against Robinhood's MCP order schema)

- When a LIVE entry fills, place a corresponding broker-side protective stop
(a resting stop-loss order at the MAE-derived stop price the app already computes), if Robinhood's MCP supports stop orders (confirm via `tools/list` / the `place_equity_order` schema: `type: stop / stop_price`, or a bracket/OCO facility). Prefer a native bracket/OCO if available so the stop auto-cancels on target fill.
- If Robinhood does NOT support a resting stop via MCP: surface this clearly and keep app-side monitoring as the only stop, with an explicit UI warning that live positions are only protected while the app is running (honest about the limitation rather than pretending there's a safety net).
- `PositionMonitor` becomes broker-env aware: for a LIVE position with a broker-side stop already resting, it should NOT also fire its own market exit (avoid double-exit / racing the broker stop). It reconciles against the broker's open orders first.
- All stop placements go through the same Gateway gates + are deterministic (no LLM), same as entries.

Open questions to resolve before building (verify, don't guess)

- Does Robinhood's agentic MCP `place_equity_order` accept `type: "stop" / stop_price`, and/or a bracket/OCO? (Inspect the live tool schema.)
- Partial-fill handling: stop qty must track the actual filled qty, not the ordered qty.
- Cancel/replace semantics for adjusting a trailing stop broker-side vs app-side.

Acceptance criteria

- A LIVE entry fill results in a resting broker-side stop at the intended price

(if supported), verified on the account.

- PositionMonitor does not double-exit a live position that has a broker stop.
- If broker stops are unsupported, the UI states plainly that live stops are app-side only (running-app dependent).

Cross-cutting guardrails (all phases)

- Every live order/stop/cancel goes through the Execution Gateway's full gate

set (owner, CSRF, kill switches, robinhood_mcp_enabled, per-market trading_enabled, notional cap, fresh-quote/drift, allowlist), deterministic, no LLM in the write path.

- needs_reconcile semantics preserved; no auto-retry of any write.
- No secrets/account numbers in persisted raw_last_state.
- robinhood_mcp_enabled remains the fast kill switch; max_order_notional stays a hard cap.

Deferred to their own specs (explicitly not this feature)

- Autonomous live mode (trading_mode='auto': auto-approve + auto-send + enforced limits + master autonomous kill-switch) - separate doc, gated on a paper track record.
- Limit-order entries - only if a concrete need appears.
- Standalone live-orders history/analytics page.

Approval

Architecture approved: No Approved scope: None Implementation allowed: No

local historical replay - Feature Architecture

Source: `features\local-historical-replay\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Local Historical Replay Worker

Status: APPROVED and implemented by owner direction on 2026-07-29 Scope: deterministic offline research, immutable experiment lineage, read-only app visibility Money-path status: unreachable; diagnostic results cannot change scoring, promotion, positions, cash, orders, or broker state

1. Decision

Kairos will run large historical experiments on the operator's machine, where the verified evidence store already lives. Vercel and the browser never read the raw files. Supabase stores only the predeclared experiment, dataset/universe/run fingerprints, compact result summary, and timestamps.

The worker extends `backtest_experiments`, the existing immutable experiment lineage. It does not create another result ledger and does not populate the older `replay_eligibility_*` tables.

Initial executable scope is an India, price-only, cross-sectional OOS diagnostic of `kairos_technical_score_v1` using official NSE bhavcopy. US bulk SEC facts and FRED vintages remain available to later sealed feature studies, but Kairos does not claim a US price backtest until a survivor-safe adjusted-price source is bound.

2. Runtime Boundary

```
flowchart LR
    CLI["Local operator CLI"] --> VERIFY["Verify manifest and every file hash"]
    VERIFY --> PREDECLARE["Insert immutable backtest_experiments plan"]
    PREDECLARE --> REPLAY["Two-pass network-free OOS replay"]
    REPLAY --> COMPLETE["Write compact result and fingerprints once"]
    COMPLETE --> API["Owner-only read API"]
    API --> UI["Backtest historical runs"]
    VERIFY -. "raw files stay local" .-> STORE["User profile evidence store"]
```

The CLI may contact Supabase only to register and complete lineage. After predeclaration, the replay calculation has no provider client and makes no market data API call. It reads the manifest-bound local files only.

3. Immutable Plan

An experiment is inserted before normalized market rows are read. Its plan fixes:

- dataset IDs and expected SHA-256 fingerprints;
- market, data cutoff, date window, horizon, cadence, and fold layout;
- exact edge/formula version and git commit;
- point-in-time universe rule and liquidity lookback;
- minimum cross-section and action-exclusion policy;
- one predeclared variant and diagnostic evidence class.

`plan_fingerprint` is unique. A completed plan is never overwritten. Re-running the same plan returns the existing result; changing any input creates a new plan.

Post-run defects do not rewrite that result. An append-only `backtest_experiment_quality_reviews` row marks the artifact `accepted_diagnostic` or `invalidated`, records the reason, and may point to its replacement. Backtest must show this review state prominently.

4. India Replay Method

- Verify the NSE daily-bar and corporate-action manifests byte-for-byte.
 - First pass builds a trailing 20-session turnover rank from date-ordered bars.
 - On each predeclared non-overlapping as-of date, select the top 200 eligible equities using only information available on or before that date.
 - Reject symbol/date observations with a split, bonus, rights issue, demerger, or ambiguous price-affecting action in the feature/label window. Raw dividends are not converted into total returns, so the run is explicitly diagnostic.
 - Second pass loads only the union of selected symbols plus NIFTYBEES.
 - Compute the exact production technical composite from candles through `as_of`.
- Compute the forward return only after `as_of`; immature labels are dropped.
- Calculate raw-rank Spearman IC per date. No winsorization precedes ranking.
- Aggregate mean IC, sample sigma, IC IR, and Newey-West t-stat.
- Persist the bounded per-date audit series plus dataset, universe, and run fingerprints.
- The result is research evidence, not a simulated portfolio claim. Costs, capacity, tax, dividends, and execution are not represented and the UI must say so.

5. Database Contract

`backtest_experiments.experiment_type = 'historical_replay'` uses the existing service-role-only table and mutation guard. Required plan fields include formula, horizon, validation mode, trial family, universe policy, data cutoff, exact code commit, structured validation spec, unique plan fingerprint, and predeclared variant/count. Completion requires all three SHA-256 fingerprints and a structured result. No anonymous or authenticated table grant is added.

6. App Surface

GET `/api/agents/backtest/historical`:

- calls `requireOwner()` before using the service client;
- returns only bounded historical-replay rows and compact summaries;
- never returns raw evidence paths, credentials, or source files;
- has no POST method and cannot start a process on the user's desktop.

Backtest shows market, status, edge, date range, evaluated/skipped dates, cross-section, mean IC, sigma, HAC t-stat, evidence class, dataset fingerprint, and limitations. It visually separates these runs from legacy signal replay.

7. Safety and Non-Goals

- No LLM in acquisition, features, labels, statistics, or persistence.
- No raw upload to Supabase, Vercel, Git, or the browser.
- No network fallback when a local file is missing or invalid.
- No use by ResearchAgent, PaperTrader, TraderAgent, PositionMonitor, LearnerAgent,

promotion RPCs, strategy policies, or broker adapters.

- No automatic schedule. The operator runs the CLI for an approved fixed plan.
- No inference that one diagnostic run validates a strategy. Promotion remains separately fail-closed under existing PIT/walk-forward governance.

8. Acceptance Gates

- A changed byte or manifest fingerprint aborts before result computation.
- The experiment plan exists before the first normalized row is read.
- No candle after as_of enters feature computation.
- No label is shorter than the declared session horizon.
- Exact-plan reruns are idempotent; completion is write-once.
- Anonymous/authenticated clients still cannot access the table.
- The app API rejects non-owner callers.
- Focused tests, TypeScript, production build, and one real local run pass.

market local risk configuration - Feature Architecture

Source: features\market-local-risk-configuration\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Market-Local Risk Configuration

Status: DRAFT - NOT APPROVED FOR IMPLEMENTATION

Trigger: 2026-07-26 configuration audit

Decision Needed

US and India should have independently configurable execution risk limits. Their genomes already evolve independently, but the owner sizing ceiling is still read from the global `strategy_config.position_size_pct`. That is an unsafe hybrid: a change intended for one currency/book silently changes the other market's maximum new-paper position size.

Do not choose different numeric values speculatively. The correct immediate change is ownership and provenance, not an India-specific risk opinion without sufficient market-local outcomes.

Current State

- `trading_mandates` is per-market and currently controls score threshold, default stop loss, target, holding horizon, open-name cap, and signal freshness.
- Production US and India mandate rows currently both use 60 / 7% / 20% / 10 names. Equal values are valid defaults; they are not evidence that the settings should be permanently shared.
- PaperTrader reads `strategy_config.position_size_pct` as the flat-size fallback and as the cap on either market's champion genome. This is global.
- Existing positions persist their entry stop/target and mandate snapshot. Changing a mandate is gate-only for new entries unless its explicit existing-positions policy is approved and executed.
- The US ETF allocation cap is a US instrument-class policy. It must not be copied to India; a future India fund/ETF sleeve needs a separately classified INR policy.

Scope

- Add `max_position_pct` to `trading_mandates`, constrained to 1-30 and seeded from the owner-approved current global ceiling for each market.
- Make PaperTrader load this strict market-local value before sizing. It becomes the flat fallback and the upper cap for that market's genome/Kelly result.
- Stamp the resolved cap and mandate version into each paper fill's existing mandate snapshot/decision journal record.
- Keep `strategy_config.position_size_pct` as a deprecated UI-preset/template only until settings migration is complete; it must not be an execution fallback once market mandates are required.
- Expose US and India controls separately in Settings with an explicit statement

that edits affect future entries only.

Non-Goals

- No retroactive resize, forced exit, stop rewrite, or cross-market normalization.
- No automatic genome parameter divergence or learner-authorized money-limit change.
- No India ETF policy inferred from US ETF tickers.
- No live-order behavior change; this is first a paper-path configuration correction.

Safety Requirements

- Missing/malformed market mandate or max-position value fails closed for new paper entries; it never falls back to the other market or global value.
- US and India pools, currencies, position counts, NAV, and caps remain isolated.
- A champion genome may size down but never above that market's owner cap.
- Settings writes are owner-authenticated, versioned, audited, and do not alter existing position snapshots.
- Database migration includes a CHECK constraint, RLS verification, and a default-free backfill that explicitly seeds both current market rows.

Acceptance Criteria

- A US cap edit cannot change an India simulated order, and vice versa.
- A missing India cap blocks only India new entries with an auditable reason.
- Existing positions preserve their stored entry plan after a settings edit.
- Tests cover flat and Kelly sizing, both markets, missing configuration, snapshot provenance, and no cross-currency aggregation.
- docs/arch/03-agents.md, 04-database-schema.md, 08-risk-and-safety.md, the System Map, migration tracker, typecheck, tests, build, and production schema/RLS verification are updated before ship.

Build Order

- Approve initial US and India max-position values.
- Add migration and strict mandate type/loader support.
- Change PaperTrader sizing and immutable fill provenance.
- Add separate Settings controls and tests.
- Verify production schema, then observe paper entries before considering live reuse.

markets data - Feature Architecture

Source: `features\markets-data\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Markets Data - Price-Cache Fill Architecture

Status: Implemented (2026-07-15)

Scope: the Markets page display tiles only. Never on the money / scoring / sizing / order path.

Problem

The Markets page (`components/dashboard/MarketsPage.tsx`) renders ~30 ETF tiles:

- Regime proxies - SPY, QQQ, IWM, TLT, IEF, HYG, UUP, GLD, VIXY, DIA
- 11 sector XLs - XLK, XLF, XLE, XLV, XLI, XLY, XLC, XLP, XLU, XLRE, XLB
- Leveraged sentiment pairs - TQQQ/SQQQ, SOXL/SOXS, SPXL/SPXS, FAS/FAZ, UGL/GLL

Each tile group had its own API route that fetched its symbols with a concurrent burst of per-symbol Massive /prev calls on page load:

- `app/api/markets/synthesis/route.ts` - `Promise.allSettled(REGIME_TICKERS.map(fetchQuote))` (8-9 concurrent)
- `app/api/markets/overview/route.ts` - 15 symbols in parallel
- `app/api/markets/quotes/route.ts` - up to 20 symbols in parallel (the leveraged pairs)
- `app/api/charts/sector-returns/route.ts` - 11 sectors, staggered but still bursty

Massive's free tier rate-limits at ~5 requests/minute (confirmed: HTTP 429 - "You've exceeded the maximum requests per minute"). So most of each burst 429'd and `Promise.allSettled` silently dropped the rejected quotes -> IWM/TLT/HYG/UUP/GLD and the leveraged pairs rendered - . Worse, `markets/synthesis` then cached the degraded read (briefings table, `session='synthesis'`) for the whole day, so a single unlucky page load froze a mostly-empty synthesis until someone manually force-refreshed. `charts/sector-returns` reads a `price_cache` table that nothing proactively filled, so Sector Performance / Breadth showed "Price cache empty".

Free-cloud-only constraint: upgrading Massive's tier is off the table. The fix has to live entirely within ~5 req/min.

Solution: one paced daily fill, then read the cache

1. Daily fill job - `app/api/agents/price-cache-fill/route.ts`

Runs pre-market on weekdays (`pg_cron`, see below). Cron-gated (`verifyCronSecret`) for the scheduled run, owner-gated for manual runs (`requireOwner`). `maxDuration = 60`.

Pacing - the key move. Massive is Polygon-compatible, so it exposes the grouped-daily aggregates endpoint:

```
GET https://api.massive.com/v2/aggs/grouped/locale/us/market/stocks/{date}?adjusted=true
```

This returns every US ticker's prior-session OHLC in one call (~12k rows). The fill job makes that single request, filters the result to our 31-symbol universe, and upserts into `price_cache`. So the entire daily fill costs one Massive request - it respects the ~5/min limit trivially, with no burst and no long pacing loop. The most-recent completed session is resolved by walking back from yesterday over weekdays (a market holiday just yields an empty grouped result and we step back another day); at most a handful of paced attempts.

Fallback (only if the grouped endpoint is ever unavailable / entitlement-denied): sequential per-symbol /prev, gated by the shared serverless-safe `try_acquire_provider_slot` lease (12.5 s = 5/min), bounded to a 45 s wall-clock budget and resumable - it only fetches symbols missing the expected session, so the 13:45 retry tick (and subsequent days) drain any remainder. This mirrors the `prewarm / evidence-shadow` bounded-resumable pattern.

Idempotency. Before doing anything (unless `?force=1`), it probes `price_cache` for a marker symbol (SPY) at the most-recent session and skips if already filled. Upserts are keyed by the `price_cache` primary key (`symbol, date`).

Health. Emits a System Health warn (`reportIssue`, key `price-cache-fill-degraded`, auto-expires at UTC midnight) only on a large shortfall (<60 % of the universe filled, or the fallback made zero progress). A clean fill calls `resolveIssue`. A couple of illiquid names missing from one snapshot is normal and does not alert.

2. Tiles read the warm cache

- `markets/synthesis` now reads the latest `price_cache` bar per regime ticker in one batch
- query (`readCachedQuotes`), computing $\text{changePct} = (\text{close} - \text{open}) / \text{open}$ from the cached daily bar (identical to what /prev returned). Only symbols still missing from the cache are backfilled, and that backfill is sequential + lease-gated (`fetchQuotePaced`) so it can never burst. Normal page loads resolve entirely from cache -> zero Massive calls on load.
- `markets/quotes` already had a `price_cache` fallback (it computes change from the last two closes); once the cache is filled the leveraged pairs resolve from it.
 - `charts/sector-returns` reads `price_cache` and is kept warm by the daily fill. Its own lazy backfill remains as a cold-start safety net. Superseded assumption (corrected 2026-07-17): this section previously claimed "the cache accumulates history over successive fills", implying the 1W/1M/3M/6M/1Y windows would fill themselves in. They do not on any useful horizon - the daily fill adds ONE session per weekday, so a 1Y window needs a year of ticks. See "Sector period windows" below for the honesty guard and the explicit backfill that replace it.
 - `markets/overview` keeps its 5-minute in-memory + Next revalidate cache; it benefits from the warm provider state but was not the frozen-for-a-day offender.

3. Never cache a degraded synthesis

`markets/synthesis` now only writes its daily briefings cache when a strong majority of the 7 regime scoring signals resolved (`scored >= 5`). A sparse read (cache not filled yet, or a transient Massive failure) is returned to the caller but left uncached, so the next fill / regeneration completes it instead of freezing it for the day.

4. Macro "what this means for your book" stays fresh

`app/api/agent-mind/macro-read` already regenerates daily via `kairos-macro-read-us/india` crons and returns a stale flag when its cached date `? today`. The `MacroReadCard` on the Markets page now self-heals: on load, if the read is stale it auto-triggers one regeneration (guarded by a `useRef` one-shot, reset when the market flips) so the card is never frozen on a previous day's text. Advisory-only, cached per day -> at most one cheap LLM call per viewer per day.

Schedule

`pg_cron` (migration `20260715140000_price_cache_fill_cron.sql`), verified registered in `prod cron.job` (jobid `71/72`, active):

```
| Job | Schedule (UTC) | Calls |
| `kairos-price-cache-fill` | Weekdays 13:25 | `POST /api/agents/price-cache-fill` |
| `kairos-price-cache-fill-retry` | Weekdays 13:45 | `POST /api/agents/price-cache-fill` |
```

Both fire before the 14:00 UTC morning briefing. 13:25 UTC ? 9:25 AM ET pre-market. EDT/EST caveat: set for EDT (summer); shift +1h at the November changeover, consistent with the other kairos- * jobs.

Invariants preserved

- Free-cloud-only - one Massive request/day; no paid tier, no VPS.
- Deterministic - prices and the regime scoring math are pure data; the LLM is used only for the advisory synthesis / thesis / macro-read prose, never for prices or the regime call.
- Per-market never mixed - this fill is US ETFs only. India tiles keep their own Yahoo/GDELT sources (IndiaSectors, IndiaBreadth, IndiaGapNote); US-only tiles stay US-only.
- Off the money path - price_cache is display data. Nothing here feeds scoring, sizing, or orders.
- Mobile-first - no UI layout changes; the existing responsive tiles just get real data.

Sector period windows - honesty guard + backfill (2026-07-17)

The defect

Markets -> Sector Performance switching 1W/1M/3M/6M/1Y did not change the data. price_cache held exactly two sessions per sector ETF (2026-07-14, 2026-07-15), because the daily fill had only been scheduled two days earlier and adds one session per tick. Every cutoff (now - 7/30/90/180/365) therefore preceded both bars, every window selected the SAME two bars, and every period returned the SAME one-day move.

This was an honesty bug, not just missing data. The guard was `if (candles.length < 2) return null` - a COUNT check, not a SPAN check. Two bars pass it and produce a number the UI renders as "1Y XLK return +2%" when it is a one-day move. A display asserting a period the data cannot support is a false return.

The contract - lib/markets/sector-returns.ts (pure, unit-tested)

A window may only report a return when the cached bars actually span it. Otherwise returnPct: null plus a machine-readable reason the UI renders as what/why/next. Three independent checks, each covering a different failure mode - all are load-bearing:

```
| Check | Constant | Catches |
| Start edge - oldest bar within N days of the cutoff | `START_GRACE_DAYS = 4` | Missing EARLY history on long windows (6 months of data must never be labelled 1Y). 4 days because the longest run of consecutive non-trading calendar days is 3 (holiday-adjacent weekend); exact-match would false-reject every window. |
| Span coverage - `span >= days x floor` | `MIN_SPAN_COVERAGE = 0.4` | DEGENERATE spans on short windows. The absolute grace cannot protect a 7-day window: observed 2026-07-17 with 2 bars, the 1W cutoff (07-10) sat exactly 4 days before the oldest bar (07-14), so both edge checks passed and a 1-day move was served as a "1W return" of -1.11%. 0.4 sits below the worst legitimate 1W (holiday week, span 3/7 = 0.43) and above the degenerate case (1/7 = 0.14). |
| End edge - newest bar within N days of today | `STALE_GRACE_DAYS = 7` | A window ending at a stale point - a "1Y return" ending three weeks ago is not a 1Y-to-today return. |
```

Reason codes: no_data, single_bar, insufficient_history, stale_cache. summariseCoverage() folds the per-sector rows into one note; the UI renders it verbatim (never a bare " - ").

PostgREST 1000-row cap (found by the backfill)

sector - returns batch-read all 11 ETFs unpaginated. PostgREST caps a response at 1000 rows; a full 1Y window is ~273 x 11 ? 3000, so it silently returned only the four alphabetically-first sectors (XLB, XLC, XLE, XLF) complete and the other seven with ZERO rows. Latent while the cache held 22 rows; it bit the instant real history landed. The read is now paginated. The honesty guard degraded correctly here - it reported no_data rather than inventing a number.

Backfill

See docs/arch/05-crons-and-scheduling.md. ~400 days of daily bars for the 11 sector XLs, one provider call per symbol (range/1/day returns the full series in one response), lease-paced at 5/min, wall-clock bounded, resumable across ticks, riding the existing kairos-price-cache-fill schedule. Total cost: 11 requests. Note the fix order - the honesty guard is correct with zero backfill and ships independently of it.

India parity

India has no equivalent defect. MarketsPage gates the whole US analytics block behind `{!isIndia && ...}`; India renders its own SectorHeatmap from NSE sector indices, which is a single-session snapshot with no period selector - so it cannot make a period claim its data cannot support. Structural gaps already carry an explicit `NotSupportedNote`. Per-market and per-currency; nothing is cross-summed. No change was needed and none was made.

Files

- app/api/agents/price-cache-fill/route.ts - new fill job; + sector history backfill (2026-07-17)
- lib/markets/sector-returns.ts - span-aware honesty layer (pure; tests/sector-returns-span.test.ts)
- app/api/charts/sector-returns/route.ts - honesty guard + paginated read (2026-07-17)
- components/charts/SectorPerformanceChart.tsx - insufficient-history / partial-coverage states
- components/dashboard/SectorBreadth.tsx - per-clause scope labelling + reason instead of " - "
- app/api/markets/synthesis/route.ts - cache-first read + strong-majority cache gate
- components/dashboard/MarketsPage.tsx - MacroReadCard self-heal on stale
- lib/schedule.ts - display metadata entry
- supabase/migrations/20260715140000_price_cache_fill_cron.sql - pg_cron schedule
- docs/arch/05-crons-and-scheduling.md - cron chapter entry

mentor agent - Feature Architecture

Source: features\mentor-agent\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature: Mentor AI Agent

Status: DRAFT -> building v1 (2026-07-04).

What's out there (and the gap)

- Trade-journal analytics (Edgewonk, TraderVue, Chartlog): compute stats on your behavior - win rate by setup, time-of-day, position-sizing errors, revenge trading. Descriptive, not coaching. You read charts and self-diagnose.
 - Trading-psychology coaches: human, expensive, not data-grounded in YOUR fills.
 - Robo-advisors / SA "health scores": rate your portfolio, not your *behavior*.
- Nobody combines: your actual decision data + live market regime + a personalized AI coach that reasons about *you* + a learning curriculum with milestones. That's the gap Kairos fills.

What we should have - MentorAgent (true AI tool-use agent)

Not a scorer (the old /api/mentor/evaluate just graded a single thesis). A real agent that, on demand (and feeding the briefing), pulls the user's behavior + learning progress + market context + trading principles, reasons over them, and returns personalized coaching.

Pipeline

runAgentLoop on deepseek - reasoner (tool-calling works; no ANTHROPIC_API_KEY). Tools:

- query_behavior - trade_decisions outcome_score patterns (what the user does well/badly across regimes), buy vs sell accuracy, recurring pattern_tags; closed paper_trades outcomes.
- query_learning_progress - closed-trade count, Phase 0/1 status, recent LearnerAgent hypotheses, rescore flags.
- query_market_context - current macro regime + index posture.
- read_principles - human-written learning_priors (Bayesian market principles).
- finish - structured coaching (below).

Output (stored in mentor_insights)

- grade (0-100) + confidence (0-1) on the user's current trading discipline/progress.
- strengths [] - what they're doing right (grounded in their data).
- focus_areas [] - the 1-3 things to work on, each with why.
- lesson - ONE lesson tailored to the current market regime AND their gaps.
- market_note - how today's regime should shape their behavior.
- next_milestone - the concrete next step (e.g. "close 4 more trades to unlock

weight-tuning; journal a thesis for each").

Governance / honesty

- Advisory/educational only - never places trades, never changes strategy.
- Grounded: reasons only over the user's real data + principles; if data is thin (Phase 0, few trades), it says so and coaches on process, not outcomes.

Surfaces

- /dashboard/mentor - a "Coach" panel: run button + latest insight card.
- Briefing v2 - a Mentor block (see features/briefing).
- Visual Agents - Mentor tagged ? AI Agent (done Phase 4).

API

- POST /api/agents/mentor-coach - runs the loop, stores an insight, returns it.
- GET /api/agents/mentor-coach - latest stored insight.

Storage - mentor_insights

id ? grade int ? confidence numeric ? strengths jsonb ? focus_areas jsonb ? lesson text ? market_note text ? next_milestone text ? model ? tokens_in/out ? created_at. RLS: service all; auth read.

momentum factors - Feature Architecture

Source: features\momentum-factors\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Momentum / Growth Factors - Feature Architecture

STATUS: DRAFT - NOT APPROVED. Proposal for review. No code until explicit sign-off.

Owner decision pending. Created 2026-07-10.

1. Problem

The live deterministic_v1 scorer is tilted value + mean-reversion and structurally fades the exact stocks that become multibaggers (Micron, SanDisk, NVDA, Intel-type cycle-turn / re-rating moves):

- RSI curve peaks at 60-72 and penalizes RSI > 75 - hype movers run overbought for weeks.
- P/E dimension marks high-multiple growth names "expensive" - the re-rating engine (earnings acceleration) isn't measured, only the P/E *level*.
- No relative strength vs market/sector (the top empirical momentum factor).
- No earnings-estimate revision momentum (the re-rating driver).
- No revenue/earnings acceleration (2nd derivative - the cycle-turn detector).
- No 52-week-high proximity / breakout / volume accumulation.

We want to *catch* these earlier - WITHOUT curve-fitting to a handful of remembered winners (survivorship) and without turning a value engine into a muddle that does neither job well.

2. Non-goals / guardrails (push-back mandate)

- Not a hardcoded "hype detector." Every new factor must clear the existing IC gate (EdgeScout/EdgeIC, lib/edges/*) on the broad universe before it can influence live scoring - the same gate that already correctly rejected the naive price/volume edges as noise.
- Not a replacement for the value/quality dimensions. Momentum is ADDED; the champion weights (per market) decide when it dominates - that's what the learning loop is for.
- No look-ahead. All factors computed point-in-time from data available at decision time.
- No money-path change. Scoring only; sizing/gates/broker untouched.

3. Proposed factors (all deterministic, 0-100 or z-scored)

Factor	Definition	Data source	
Relative strength	stock total return - benchmark return over 1/3/6mo, percentile-ranked	daily series (have) + SPY/^NSEI	
Estimate revision	? consensus EPS/target over 30/90d (up = bullish)	FinancialDatasets / AV estimates	
Revenue acceleration	sign+magnitude of QoQ growth 2nd derivative	FinancialDatasets income statements	
Earnings acceleration	same on EPS	FinancialDatasets	
52w-high proximity	`price / 52w_high` (breakout tell)	daily series (have)	
Volume breakout	volume z-score on up-days vs base	candles (have)	

4. Two architecture options (decide at review)

Option A - new 6th dimension momentum_score. Aggregate the factors into one 0-100 dimension; extend the weight vector to 6, extend champion weights_snapshot, validation replay, and genome. Cleanest conceptually; touches the weight schema + validation contract (bigger blast radius).

Option B - factors as IC-gated *features* feeding existing dimensions (RECOMMENDED). Each factor enters feature_registry (migration 064), is IC-validated via the edge lab, and on promotion contributes to the technical (RS, 52wH, volume-breakout) or fundamental (estimate revision, acceleration) dimension via the whitelisted compiler. No new weight slot; the governed discovery path we already built does the work. Slower to "turn on," but validation-first and reversible.

Recommendation: B - it's the path the learning-core was built for and avoids overfitting.

5. Archetype interaction

lib/scoring/archetypes.ts already exists. A "momentum/breakout" archetype would stop hard-fading high RSI *when* RS + acceleration confirm - regime/archetype-routed, not global. In scope as a follow-on once ≥ 1 momentum factor is IC-promoted.

6. Validation gate (hard requirement)

No factor influences a live score until: IC [?] $\geq$ threshold with Newey-West t-stat, out-of-sample across walk-forward folds, on the broad universe (not the 4 winners). Fail-closed. This is the antidote to survivorship.

7. Phasing

- P0 - compute + log the 6 factors as shadow evidence on every candidate (no score effect).
- P1 - IC-measure each in the edge lab; promote only those that clear.
- P2 - wire promoted factors into their dimension (Option B) or the 6th dimension (Option A).
- P3 - momentum archetype (conditional RSI-fade removal); let per-market weights adapt.

8. Open questions for review

- Option A (new dimension) vs B (IC-gated features)?
- Benchmark for RS: SPY/^NSEI only, or sector ETF too?
- Estimate-revision data: FinancialDatasets sufficient, or add a second source?
- Acceptable AV/FD budget increase for the new fetches (day-cache + budget guard applies)?

multi tenant - Feature Architecture

Source: features\multi-tenant\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature Architecture: Multi-Tenant (Friends & Family)

Status

Architecture status: Draft Architecture approved: No Approved scope: None Approved date: None Implementation allowed: No

DESIGN ONLY. No code, no migration file, no migration applied, no deployment.

This document is a proposal awaiting the owner's explicit approval gate

(CLAUDE.md "Architecture-First Mode"). Nothing here ships until the owner says

"Approved / Proceed / Implement this".

Feature Purpose

Let the owner invite a small, vetted circle (friends & family) to use Kairos for their own internal paper/research use. Each invited user gets an isolated account: their own onboarding, their own market-data + LLM + broker keys, their own paper book, and - the interesting part - a fresh champion genome and learning loop that adapts on *their* trades, never on anyone else's. The system explores whether *aggregate, privacy-preserving* meta-insights can be shared across users without ever sharing an individual's weights or trades.

This is the transition from a single-owner personal tool (one hardcoded email, global-singleton tables scoped by RLS to that email) to a narrow multi-tenant tool (N users, every per-user row carries `user_id`, RLS scoped to `auth.uid()`), while keeping the free-cloud-only, deterministic-money-path, and per-market/currency-isolation invariants intact.

User/System Questions This Feature Answers

- How does the owner let a specific friend in - and *only* that friend - without opening public registration?
- When a new user logs in for the first time, what must they provide before any agent runs on their behalf, and how do we stop agents running until they do?
- How is user A's genome/learning kept provably separate from user B's?
- Whose API keys and LLM spend pay for a given user's runs? (Answer: always their own - the owner never pays a guest's usage.)
- What, if anything, is safe to learn *across* users, and how do we share it without leaking any individual's trades or weights?
- How do we migrate the current single-owner data so the owner becomes "user 1" with zero loss?

Scope

This feature includes:

- A tenancy model: `user_id` on every per-user table + Supabase RLS keyed on `auth.uid()` (replacing the owner-email RLS), with an explicit shared-vs-per-user table classification and a single-user->multi-user backfill.
- Owner-approved signup: an invite/allowlist gate. New emails cannot self-activate.
- A guided onboarding flow that blocks all agent activity until complete.
- Per-user genome + per-user learner (the locked 10-closed-trade Phase-0 rule, applied per user per market).
- A cross-user learning design that shares only opt-in, aggregate, k-anonymized meta-insights - never individual weights or trades.
- Security/isolation: RLS proof obligations, per-user vault isolation, owner-admin boundary, per-user cost isolation.
- C4 context + container diagrams, a phased rollout, and acceptance tests.

Non-Goals (explicitly OUT of scope)

- Billing / metering / subscriptions. Friends & family = free. Lago stays a later_product_idea (per features/external-research-integrations/REPOSITORY_CAPABILITY_CATALOG.md). No Stripe wiring beyond the dormant legacy columns already on profiles.
- Public / open self-service signup. Invite-only, owner-gated, forever (for this feature).
- Any shared money path. No pooled capital, no shared book, no cross-user order routing. Each user's book is theirs alone.
- Live trading for invited users, initially. Paper-first. Guests are hard-capped to paper + research. Live broker order placement stays owner-only until a separate, later, explicitly-approved phase.
- Hand-tuning of weights by users. Users influence their own genome only indirectly (risk profile, mandate, and their realized outcomes). No user - not even the owner-for-a-guest - edits a genome's raw weights by hand.
- Changing the deterministic no-LLM-on-money-path rule, per-market/per-currency isolation, or the free-cloud-only constraint.

Current Behavior (verified against code, 2026-07-15)

Kairos is single-user, owner-gated to one email. Concretely:

```
| Layer | Today | File |
| Owner identity | `OWNER_EMAIL = "vterminater@gmail.com"` (hardcoded constant) | `lib/auth/owner.ts` |
| Page gate | `middleware.ts` signs out any session whose `email !== OWNER_EMAIL`; gates `/dashboard`, `/admin` | `middleware.ts` |
```

```
| API gate | `requireOwner()` returns 403 unless `user.email === OWNER_EMAIL && email_confirmed_at` | `lib/
auth/require-owner.ts` |
| RLS | Owner-`email` predicate: `(auth.jwt() ->> 'email') = 'vterminater@gmail.com'` on the sensitive
tables; `service_role` bypasses RLS for all server/agent writes | migrations 142, 144, `20260713112754` |
| Profiles | `profiles` PK = `auth.users.id`; `handle_new_user()` trigger auto-creates a row on signup and
stamps `role='superadmin'` for the owner email | `001_initial_schema.sql` |
```

Which tables already carry `user_id`: only the *legacy consumer-app* tables from the original FinanceOS product - holdings, predictions, watchlist (UNIQUE(`user_id`, `symbol`)), plus profiles (PK = `auth.users.id`). These predate the agent system and are effectively dormant for the agent flows.

Which tables are global singletons with NO `user_id` (owner-scoped only by the email-RLS + the fact that one person uses the app) - the agent system's entire data plane:

- Strategy/config: `strategy_config` (a *single row*), `strategy_versions` (per-market champion/challenger), `agent_config`, `learner_config`, `learning_priors`, `learning_priors_history`, `learning_log`, `signal_weights_history`, `investment_mandates`, `market_controls`, `strategy_validation_automation`, `strategy_sleeves`.
- Research/signals: `agent_signals`, `signal_score_history`, `decision_observations`, `research_packets`, `research_queue`, `edge_signals`, `edge_ic_history`, `universe_snapshot`.
- Paper book: `paper_portfolio`, `paper_positions`, `paper_trades`, `paper_order_events`, `paper_performance`.
- Live book: `broker_accounts`, `live_account_snapshots`, `live_performance`, `broker_orders`, `broker_order_events`, `trade_proposals`, `decision_journal`.
- Learning/RAG: `trade_memories` (pgvector), `experiment_runs`, `strategy_evaluations`, `benchmark_scorecard`, `rotation_events`.
- Secrets: `api_key_vault` - a single global vault keyed by `key_name/provider` UNIQUE; resolution is vault-first / env-fallback, service-role-locked (`lib/llm-keys.ts`, `lib/vault-pin.ts`). LLM keys and broker OAuth tokens all share this one store.

Genuinely-shared market facts (NOT user data - must stay global): `macro_regime`, `macro_signals`, `india_screen_cache`, `evidence_records`, `fundamental_facts`, `corporate_actions`, `benchmarks`, `benchmark_price_observations`, `provider_pacing`, `provider_call_ledger`, `evidence_cache_v2`, the evidence-policy tables, and the market-knowledge principle base (`knowledge/`). These describe *the market*, identically for everyone, and are expensive to recompute per user.

Scheduling today is global: Vercel crons, Windows Task Scheduler (`scripts/run-agents.ps1`), and Supabase `pg_cron` each fire once and operate on the single owner's book, parameterized only by `?market=us|india`. There is no per-user fan-out anywhere.

No onboarding, invite, or allowlist flow exists for users today - this is greenfield.

Proposed Behavior

1. Tenancy model

The core move: add `user_id` `uuid` NOT NULL REFERENCES `auth.users(id)` to every per-user table, and switch RLS from the owner-email predicate to `user_id = auth.uid()`. Shared market-fact tables keep no `user_id` and stay readable by all authenticated users.

1a. Table classification (the load-bearing decision)

```
| Class | Rule | Tables (representative) |
| **Per-user (add `user_id`, RLS `= auth.uid()`)** | Anything that is one user's config, book, signals, learning, or secrets | `strategy_config`, `strategy_versions`, `agent_config`, `learner_config`, `learning_priors`, `learning_log`, `signal_weights_history`, `investment_mandates`, `market_controls`, `strategy_validation_automation`, `strategy_sleeves`, `agent_signals`, `signal_score_history`, `decision_observations`, `research_packets`, `research_queue`, `edge_signals`, `edge_ic_history`, `universe_snapshot`, `paper_portfolio`, `paper_positions`, `paper_trades`, `paper_order_events`, `paper_performance`, `broker_accounts`, `live_account_snapshots`, `live_performance`, `broker_orders`, `broker_order_events`, `trade_proposals`, `decision_journal`, `trade_memories`, `experiment_runs`, `strategy_evaluations`, `benchmark_scorecard`, `rotation_events`, `watchlist`, `holdings`, `predictions`, `briefings`, `newsletters`, `mentor_insights`, `agent_runs`, `agent_alerts`, `llm_call_log`, `rag_traces` |
| **Shared market facts (NO `user_id`, RLS: read-all-authenticated, service-write)** | Identical for everyone; describes the market, not a person | `macro_regime`, `macro_signals`, `india_screen_cache`, `evidence_records`, `fundamental_facts`, `corporate_actions`, `benchmarks`, `benchmark_price_observations`, `provider_pacing`, `provider_call_ledger`, `evidence_cache_v2`, evidence-policy tables, `symbol_blocklist` |
| **Secrets (per-user, but never client-readable)** | Per-user vault rows; service-role only, never returned raw | `api_key_vault` -> **add `user_id`**, change UNIQUE to `(user_id, key_name)` |
```

**strategy_config is the sharpest change. It is a *single-row* table today*

(SELECT ... LIMIT 1 everywhere). Multi-tenant requires it to become

one row per user (drop the singleton assumption; key by user_id). Every

*call site that does "read the config row" must become "read *this user's* config*

row". This is the highest-touch refactor and the biggest source of latent

single-tenant assumptions. Same pattern for paper_portfolio (already per-market;

becomes per-user-per-market) and market_controls.

1b. Composite scoping keys

Kairos already isolates by market (us | india) and currency. Multi-tenant adds user as the outermost scope. The canonical grain of a book row becomes (user_id, market); of a champion, (user_id, market, is_champion). No row is ever summed across user_id, exactly as no row is summed across currency today. All existing per-market uniqueness constraints gain user_id as the leading column (e.g. strategy_versions champion-uniqueness -> UNIQUE(user_id, market) WHERE is_champion).

1c. RLS pattern (replaces owner-email)

For every per-user table:

```
-- read/write only your own rows
CREATE POLICY user_rw ON <table>
FOR ALL TO authenticated
USING (user_id = (SELECT auth.uid()))
WITH CHECK (user_id = (SELECT auth.uid()));
```

service_role continues to bypass RLS - but see ?6: server/agent code must now explicitly stamp and filter user_id on every service-role query, because the DB will no longer protect it (the email predicate is gone). This is the single biggest correctness risk and gets an acceptance test.

Owner/admin visibility (support, debugging) is not a blanket RLS bypass. It is a separate, audited, read-only admin surface (?6) that uses service_role with an explicit user_id filter and writes an access-log row - never an "owner sees all via RLS" shortcut.

1d. Migration from single-user (backfill owner as user 1)

- Resolve the owner's `auth.users.id` (call it `OWNER_UID`).
 - For each per-user table: `ALTER TABLE ... ADD COLUMN user_id uuid;`
then `UPDATE ... SET user_id = OWNER_UID WHERE user_id IS NULL;` then `ALTER COLUMN user_id SET NOT NULL + add FK + index (user_id, ...).`
 - Replace owner-email RLS policies with the `auth.uid()` policies above.
 - `strategy_config / paper_portfolio / market_controls`: the existing rows are stamped `OWNER_UID`; the singleton reads become per-user reads.
 - Shared-fact tables: unchanged except tightening RLS to "read-all-authenticated, service-write" (they were owner-email or service-only before).
 - Backfill is idempotent and reversible (guard on `WHERE user_id IS NULL`), applied per `CLAUDE.md`'s schema-migration rule (verify applied to target DB via `information_schema` before any dependent code ships).
- Because every existing row is stamped `OWNER_UID`, the owner's book, genome, learning history, and vault survive the migration byte-for-byte - the owner simply becomes user 1.

2. Owner-approved signup (invite / allowlist)

No open registration. A new `user_invitations` table is the gate:

Column	Type	Notes
<code>`email`</code>	text PK (citext)	Invited address, normalized
<code>`invited_by`</code>	uuid	Owner's uid
<code>`status`</code>	text	<code>`invited`</code> \ <code>`activated`</code> \ <code>`revoked`</code>
<code>`role`</code>	text	<code>`guest`</code> (default) \ <code>`owner`</code>
<code>`capabilities`</code>	jsonb	e.g. <code>{ live_trading: false, markets: ["us"] }</code> - paper-only by default
<code>`invited_at`</code> / <code>`activated_at`</code> / <code>`revoked_at`</code>	timestampz	

Flow:

- Owner adds an email in an Admin -> Invites panel (owner-only, guarded by a new `requireRole('owner')`).
- The email is added to `user_invitations` (`status='invited'`). Supabase Auth signups are configured invite-gated: a `handle_new_user()` update rejects (or immediately deactivates) any signup whose email is not `status='invited'`. This replaces the "reject everyone but `OWNER_EMAIL`" logic in `middleware.ts`.
- The invited person signs in (magic-link / OAuth). `handle_new_user()` flips the invite to `activated`, creates their profiles row with `role='guest'`, and marks onboarding incomplete.
- `middleware.ts` and `requireOwner->requireUser` change: instead of comparing to a hardcoded email, they check `profiles.role/invite` status. `OWNER_EMAIL` stays only as the bootstrap seed for user 1 and the owner-admin check.
- Revocation: owner sets `status='revoked'`; `middleware` signs the session out and blocks re-entry. (Data is retained, not deleted - deletion is a separate, explicit, out-of-scope action.)

3. Onboarding (agents gated until complete)

A new `onboarding_state` per user (columns on profiles or a small `user_onboarding` table) with a hard gate: no agent, cron, or research run executes for a user until `onboarding_complete = true`. The gate is enforced in the agent entry points (the per-user run resolver, ?7) - a user who hasn't finished onboarding is simply skipped by the schedulers.

Guided steps (a wizard; each writes to the user's own rows):

- What Kairos does - explainer + the hard rules (paper-first, deterministic money path, you bring your own keys, your data is isolated).
 - Market-data keys - the user enters *their own* provider keys (Alpha Vantage, FinancialDatasets, etc.) into *their* vault rows. Nothing runs on the owner's keys. Missing required keys -> that capability stays dark.
 - LLM model + key - the user picks a per-flow model and supplies the matching provider key (reuses the existing Settings -> AI Models UX and `lib/llm-keys.ts`, now `user_id`-scoped). No key -> that flow is disabled for them (never silently billed to the owner).
 - Broker connect (paper-first) - optional; read-only holdings import only.
- Live order placement is not offered to guests in this phase.
- Risk profile + mandate - Conservative | Balanced | Aggressive and a default `investment_mandate` (horizon/benchmark). These seed the user's `strategy_config` and their fresh genome (?4).
 - Confirm - sets `onboarding_complete = true`, which arms their schedulers.

4. Per-user genome + learning loop

Each user gets a fresh champion genome per market, seeded at onboarding from a neutral default prior (the same academic prior the single-owner app seeds today - 5 equal-ish dimension weights + default genome ranges), not from any other user's genome. From there:

- The user's `strategy_versions` rows (`user_id`-scoped) hold *their* champion + challengers. `UNIQUE(user_id, market) WHERE is_champion`.
- Their `LearnerAgent` run reads only their closed `paper_trades` / `decision_observations` and proposes challengers for their genome.
- The locked Phase-0 gate applies per user: weight mutation is blocked until that user has 10+ closed trades in that market. A user with 3 trades stays on the seeded champion; the owner with 200 trades is on their evolved champion. The gate, the auto-guard (last-3-runs win-rate < 35%), and the "size down only, never above the user-set cap" genome rule are unchanged - just per-user.
- Their Validation Engine replays walk-forward on their `decision_observations` only. Their promotion decisions touch only their champion.
- Full isolation invariant: *user A's outcomes never touch user B's weights.*

There is no code path, RLS policy, or query that reads one user's trades to mutate another's genome. This is the divergence acceptance test (?Acceptance): two users fed different trade sets must produce measurably different champions.

Users influence their genome only through risk profile, mandate, and realized outcomes - never by editing raw weights (the CLAUDE.md push-back mandate is preserved and now applies per user).

5. Cross-user learning (the interesting part)

The design question the owner cares about most. Two hard boundaries first, then the narrow safe channel.

Never shared (hard NO):

- Individual weight genomes / `weights_snapshot` / champion parameters.
- Any per-user trade, signal, position, P&L, or `decision_observation`.
- Anything from which a single user's book or identity could be reconstructed.

Already shared, safely (existing priors): the academic principle base (knowledge/) and the deterministic scoring grammar. These are literature-derived priors, identical for everyone, and carry no user data. They stay shared - this is not new leakage.

The proposed narrow channel - opt-in, aggregate-only meta-insights:

A separate, deterministic MetaLearner (offline, owner-run, no money path) computes **cohort-level** observations across users who have opted in, subject to strict privacy guards, and emits them only as priors nudges bounded in magnitude, never as another user's weights:

- Example output: **"across opted-in users, the momentum-dimension weight tended to*

*rise in high-volatility regimes"** - a direction, not a value, not attributable to anyone.

- k-anonymity / min-cohort: an aggregate is emitted only if it is computed over

$\geq K$ opted-in users (proposed $K \geq 5$) **and** $\geq N$ trades, so no cohort can be narrowed to one person. Below the floor -> no insight emitted.

- Aggregation before storage: only means/medians/rank-correlations over the

cohort are ever persisted; raw per-user inputs are read under service-role, reduced in-memory, and discarded. No per-user row is copied into the meta store.

- Overfitting guards: meta-insights are advisory **priors**, bounded to a small

max nudge (e.g. ?X% of a dimension weight), pass the same walk-forward validation before they can influence anyone, and are rate-limited (at most one meta-update per cadence). They can never bypass a user's own Phase-0 gate or promote a user's champion.

- Opt-in + revocable: default OFF. A user opts in explicitly during

onboarding or Settings; opting out removes them from all future cohorts.

- No money path: the MetaLearner writes to a `meta_insights` (aggregate) table

only; it never places an order, moves cash, or auto-mutates a champion.

This gives "collective intelligence" its upside (regime-conditioned prior nudges) while keeping every individual's weights and trades private and every user's book adapting primarily to their own outcomes.

6. Security / isolation

- RLS proof obligation. Every per-user table has an `auth.uid()` policy; a test

suite (?Acceptance) asserts, per table, that user B receives zero rows of user A via the anon/authenticated client. The dangerous change is that removing the owner-email predicate means server code is now the last line of defense on service-role queries - so every service-role read/write must carry an explicit `.eq('user_id', uid)`. A lint/code-review checklist + an integration test that runs each agent as user B and asserts it never reads user A's rows.

- Per-user vault isolation. `api_key_vault` gains `user_id`;

UNIQUE(user_id, key_name).getProviderKey() / setProviderKey() / broker-token resolution all take a user_id. A user can never read or use another user's key; keys remain write-only from the UI and are never returned raw. Env-fallback becomes owner-only (guests must bring their own keys - see cost isolation).

- Owner-admin boundary. The owner is a superadmin for *support* (read-only, audited), not a data-plane superuser. Admin reads go through a dedicated service-role surface that filters by the target user_id and writes an admin_access_log row. No "owner sees everything" RLS bypass.

- Cost isolation. Each user's LLM and market-data calls resolve their vault keys only; env-fallback is disabled for guests. A guest with no key -> the flow is disabled, not silently charged to the owner. llm_call_log is user_id-scoped so spend is attributable per user. This directly satisfies the free-cloud-only / "users bring their own keys" memory constraint.

- Live-trading lockout. Guests' capabilities.live_trading = false is enforced

at the order gate (in addition to the existing 9 safety gates + autonomy ladder). A guest cannot reach a live broker order path regardless of settings.

7. Scheduling (per-user fan-out)

Today's crons operate on one book. Multi-tenant needs the schedulers to fan out over active, onboarded users:

- Each global cron endpoint (/api/agents/research/cron, paper-trade, position-monitor, learner, performance, briefings, ...) changes from "run once" to "for each user where onboarding_complete AND NOT paused, run scoped to that user_id". Shared market-fact refreshes (macro, screen cache, evidence) run once, globally, not per user - so N users don't multiply the market-data spend.

- Free-cloud-only constraint: Vercel cron count/time budgets are fixed, so fan-out

is a sequential loop inside one cron invocation (bounded, small N), not N separate cron entries. Per-user provider pacing (provider_pacing, migration 176) already exists and extends naturally.

- Guests are paper-only, so the live-order crons stay owner-scoped in this phase.

System Architecture

C4 - Level 1: System Context

```
C4Context
title Kairos Multi-Tenant - System Context
Person(owner, "Owner", "Superadmin. Invites users, uses Kairos, runs admin/support")
Person(guest, "Invited User (F&F)", "Paper/research only. Brings own keys")
System(kairos, "Kairos", "Multi-tenant paper-trading + research OS")
System_Ext(supabase, "Supabase", "Postgres + Auth + RLS + pg_cron + pgvector")
System_Ext(llm, "LLM providers", "Anthropic / DeepSeek / Groq / (per-user keys)")
System_Ext(mkt, "Market-data providers", "Alpha Vantage / FinancialDatasets / NSE (per-user keys)")
System_Ext(brokers, "Brokers", "Robinhood / Kite (owner live; guests read-only paper)")

Rel(owner, kairos, "Invites, uses, supports")
Rel(guest, kairos, "Onboards, uses paper+research")
Rel(kairos, supabase, "Per-user RLS data plane, cron fan-out")
Rel(kairos, llm, "Scoped to each user's vault key")
Rel(kairos, mkt, "Shared facts once; user calls on user keys")
Rel(kairos, brokers, "Owner live; guest read-only")
```


C4 - Level 2: Container

```
C4Container
  title Kairos Multi-Tenant - Containers
  Person(owner, "Owner")
  Person(guest, "Invited User")

  Container_Boundary(app, "Next.js app (Vercel)") {
    Container(web, "Web UI", "Next.js/React", "Dashboard, onboarding wizard, Admin-Invites")
    Container(mw, "Auth middleware", "Edge", "Invite/role gate (replaces owner-email gate)")
    Container(api, "API routes", "Route handlers", "requireUser() + user_id-scoped")
    Container(agents, "Agent runtime", "Server", "Research/Trader/Learner/Monitor per-user run resolver")
    Container(sched, "Schedulers", "Vercel cron / pg_cron / Task Scheduler", "Fan out over onboarded users; shared facts once")
    Container(meta, "MetaLearner (offline)", "Server, owner-run", "Aggregate, k-anon, opt-in only. No money path")
  }

  ContainerDb(db, "Supabase Postgres", "Per-user tables (user_id + RLS auth.uid()); shared-fact tables; per-user vault; meta_insights (aggregate)")
  System_Ext(providers, "LLM + market-data + brokers")

  Rel(owner, web, "Invite, use, admin")
  Rel(guest, web, "Onboard, use")
  Rel(web, mw, "Session")
  Rel(mw, api, "role/invite-checked")
  Rel(api, db, "user_id-scoped (RLS + explicit filter)")
  Rel(agents, db, "Reads/writes this user's rows")
  Rel(sched, agents, "Per onboarded user")
  Rel(agents, providers, "User's own keys")
  Rel(meta, db, "Reads opted-in cohort; writes aggregate only")
```

Screen / Page / Module Inventory (new or changed)

- New: Admin -> Invites panel; Onboarding wizard (6 steps); per-user Settings (keys/LLM/risk/mandate now user_id-scoped); opt-in-to-meta toggle.
- Changed: middleware.ts (invite/role gate), lib/auth/require-owner.ts -> add requireUser() / requireRole(), lib/llm-keys.ts + vault helpers (take user_id), every agent entry point (per-user run resolver + onboarding gate), every cron endpoint (fan-out), all strategy_config/paper_portfolio single-row reads.
- New tables: user_invitations, user_onboarding (or columns on profiles), meta_insights (aggregate-only), admin_access_log.
- Altered tables: user_id added to every per-user table (?1a) + RLS swap.

Phased Rollout

```
| Phase | Goal | Exit criteria |
| **P0 - Owner-only (backfill)** | Add `user_id` everywhere, backfill owner as user 1, swap RLS to `auth.uid()`, refactor singleton reads. **Zero behavior change for the owner.** | Owner's book/genome/vault byte-identical post-migration; all agents run scoped to `OWNER_UID`; RLS tests pass with a single user. |
| **P1 - 1 invited user** | Invite flow + onboarding gate + per-user vault + per-user genome seeding + cron fan-out (N=2). | A second user onboards on their own keys, gets a fresh champion, runs paper research; isolation tests prove zero cross-read; owner unaffected. |
| **P2 - N users** | Harden fan-out under cloud budgets; per-user pacing; admin support surface + access log. | N (small) users run within Vercel cron budgets on their own keys; provider pacing holds; admin reads are audited. |
| **P3 - Cross-user meta (opt-in)** | MetaLearner: aggregate, k-anon (K>=5), bounded prior nudges, validated, opt-in. | Meta-insights emit only above the cohort floor; no per-user data in the meta store; nudges pass walk-forward; opt-out removes a user from cohorts. |
| **(Later, separate approval)** | Guest live trading, billing/metering. | Out of scope for this feature. |
```


Acceptance Tests

- Tenant isolation (data plane). As authenticated user B, every per-user table returns 0 rows belonging to user A (direct PostgREST + through each API route).
- RLS policy coverage. Automated check: every table classified per-user has an `auth.uid()` policy and no lingering owner-email or `USING(true)` policy.
- Service-role filter guard. Run each agent as user B; assert (via query log / integration harness) it never reads or writes a row with `user_id = A`.
- Onboarding gate. A user with `onboarding_complete = false` is skipped by all schedulers; no `agent_signals/paper_*` rows are ever created for them.
- Invite gate. A non-invited email cannot activate; a revoked user is signed out and blocked; only `status='invited'` can complete signup.
- Per-user genome divergence. Seed user A and user B with different closed-trade sets past the Phase-0 gate; assert their champion `weights_snapshot/genome` differ and that neither's `LearnerAgent` ever read the other's trades.
- Vault isolation. User B cannot read/use user A's provider or broker key; `getProviderKey(uid=B)` never resolves A's vault row; `env-fallback` is owner-only.
- Cost isolation. A guest with no LLM key gets the flow disabled (not run on the owner's key); `llm_call_log` attributes every call to the correct `user_id`.
- Owner backfill fidelity. Post-migration, the owner's champion, trade ledger, and vault are byte-identical to pre-migration (hash compare).
- Meta privacy. `MetaLearner` emits nothing below K users / N trades; the `meta_insights` store contains no per-user trade, weight, or identity; opting out removes a user from all subsequent cohorts.
- Live lockout. A guest cannot reach any live broker order path irrespective of settings (`capabilities.live_trading=false` enforced at the gate).

Open Decisions (for the owner)

Biggest open decision - how much cross-user aggregate learning to allow, vs full isolation. The spectrum:

- (A) Full isolation. No cross-user learning at all. Simplest, zero leakage risk, most defensible privacy story. Each user is an island; the only shared prior is the existing academic knowledge/ base. Cost: no "collective intelligence" upside.
- (B) Opt-in aggregate nudges (this doc's ?5 proposal). k-anonymized ($K \geq 5$), bounded, validated, revocable prior nudges - direction-only, never values, never attributable. Upside: regime-conditioned collective priors. Cost: real engineering + a standing privacy/overfitting surface to maintain, and a small but non-zero inference-risk to reason about continuously.

Recommendation: ship P0-P2 with (A) full isolation, and treat (B) as a distinct, separately-approved P3 only after there are enough opted-in users for $K \geq 5$ to even be meaningful. Everything else in this doc (tenancy, invites, onboarding, per-user

genome, security) is required regardless of which way (A)/(B) goes.

Secondary open decisions:

- Keep `strategy_config` as one-row-per-user, or split the truly-global knobs

(autonomy master-kill) from per-user knobs into two tables? (Proposal: per-user row; owner keeps a separate global master-kill.)

- Cron fan-out as a sequential loop vs. a durable per-user job queue if N grows

(proposal: sequential loop now; revisit at N beyond a handful).

Update triggers (per CLAUDE.md)

If/when approved and implemented, the SAME change must update: `docs/arch/04-database-schema.md` (new `user_id` columns + RLS), `08-risk-and-safety.md` (guest live-lockout + per-user gates), `09-learning-loop.md` (per-user genome + MetaLearner), `05-crons-and-scheduling.md` (per-user fan-out), `06-env-variables.md` (owner-only env-fallback), `07-coding-conventions.md` (service-role `user_id` filter rule), and `public/agent-diagrams/system-map.json`.

nav restructure - Feature Architecture

Source: `features\nav-restructure\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Feature: Left-nav restructure (funnel order + surface LLM config)

Last updated: 2026-07-12 Update this file when: NAV_SECTIONS changes, a route is added/removed/renamed, or the LLM-config entry point moves.

Problem

- ~20 sidebar items; grouping ("Daily / Portfolio / Agents & Research / Discovery / Learn / Settings") doesn't mirror the user's actual funnel.
- "Morning Briefing" is the home page but reads like a feature, not the app home.
- The per-flow LLM model picker is buried at Agents -> "? LLM Config" tab (default tab is Paper), so users look in Settings and don't find it.
- Three portfolio surfaces (Paper / Live-US / India) read as duplicates.

Decision (this pass - LOW RISK, no routes removed)

Reorder + relabel NAV_SECTIONS in `components/dashboard/DashboardShell.tsx` to follow the funnel Overview -> Markets -> Signals -> Portfolio -> Research -> Discovery -> Learn -> Settings, and:

- Rename the home item "Morning Briefing" -> "Home" (route `/dashboard` unchanged).
- Split the old "Daily" into Overview (Home, Markets) + Signals (Intelligence, Smart Money) - pure regrouping, same routes.
- Rename "Agents & Research" -> "Research"; keep `Agents/History/Research Journal/Scores`.
- Add a "LLM & Models" item under Settings that deep-links to

`/dashboard/agents?tab=llm-config`. `AgentsPage` now reads `?tab=` to open that tab directly. This is the fix for "I can't find where to change the model."

No page component is merged in this pass. All existing routes keep working; only the sidebar order/labels and one new deep-link change.

Done (2026-07-12, second pass - shipped behind redirects, no deep-link 404s)

- Live Portfolio = US + India: `LivePortfolioSwitch` renders US (`LivePortfolioPage`) or India (`IndiaLivePanel`) by the global market switch. `/dashboard/india` redirects.
- Signals = Intelligence + Smart Money: Smart Money is a tab on the Intelligence page (data via owner-gated `/api/markets/smart-money`). `/dashboard/smart-money` redirects.
- Research = Research Journal + Score Tracker: `ScoreTrackerPanel` is a tab on Research Journal. `/dashboard/scores` redirects.
- Agents absorbs Agent History: `AgentHistoryPanel` is a "History" tab; `AgentsPage` reads `?tab=`. `/dashboard/agents/history` redirects.

- Settings absorbs Automation + Admin + LLM Config: AutomationPanel + AdminPanel + LLMConfigPanel are tabs. /dashboard/admin and /dashboard/settings/automation redirect.

Deliberately NOT merged: Paper Portfolio stays separate from Live Portfolio (sim vs real money - kept visibly distinct for safety).

Known cosmetic follow-up

Merged panels (Smart Money, Score Tracker, Admin, Automation, India Live) still render their own PageHeader inside the container that also has one -> a double header on those tabs. Functional; trim by adding an optional embedded prop that suppresses the inner PageHeader when rendered as a tab.

Files

- components/dashboard/DashboardShell.tsx - NAV_SECTIONS array.
- components/dashboard/AgentsPage.tsx - initial tab from useSearchParams().get("tab").

paper exit plan display - Feature Architecture

Source: features\paper-exit-plan-display\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Paper Portfolio Exit Plan Display

Status: approved for implementation (owner requested, 2026-07-21)

Decision

Show one compact Exit plan section for every open paper position. Do not show a single "agent exit price": Kairos can exit through several deterministic rules, and presenting only the profit target would be materially misleading.

Why

The owner needs to understand why an open paper position is still held and which condition would close it. The existing position card shows P&L but hides PositionMonitor's current stop, target, conviction threshold, score freshness, and time horizon.

Scope

- Project a read-only exit plan from the open position, per-market trading mandate, effective champion/user horizon, and latest same-market validated deterministic holding score.
- Show the persisted trailing/protective stop. This is the current executable paper stop and may ratchet upward; it never moves down.
- Show the persisted profit target when present. After a partial target exit the database clears the target, so the UI must say no remaining target rather than retaining a historical number.
- Show current score versus the per-market exit threshold and whether the score is fresh enough to act.
- Show current position age versus the effective time horizon, including whether the position is grandfathered onto its entry horizon.
- Show the first currently met condition using PositionMonitor's precedence: time, score, stop, target.
- Reuse the same projection in the Paper Portfolio Trades table, immediately after Score. Only an open BUY trade in the same market and symbol may receive the current plan; completed trades display their completed state and cannot inherit a later position's plan.

Boundaries

- Display-only. It cannot write signals, positions, trades, mandates, cash, controls, broker orders, or provider state.
- No market-data/provider calls. It reuses persisted position prices and signals.
- US and India are keyed and resolved separately; currencies and signals never cross.
- Missing/stale research means mechanical exits only. It must never be converted to score zero.
- The manual Close Position command remains separate and unchanged.
- This does not add or change an exit rule, threshold, target, stop, or schedule.

Data Contract

PaperExitPlan contains:

- market and position identity
- current persisted price, stop, and optional target
- latest deterministic score, threshold, timestamp, market-session age, and freshness
- open-position weekday age, effective horizon, and horizon source
- current projected state: hold, time_exit_due, score_exit_due, stop_exit_due, or target_exit_due

The projection is assembled on the server. The client receives only display data and performs no decision calculation.

Edge Cases

- No target: display No remaining target; do not invent one.
- No score: display Awaiting held-name research ? mechanical exits active.
- Stale score: display its age and mechanical exits only.
- Partial target: the remaining lot has no target and a breakeven-or-higher persisted stop.
- Hedge positions: omit score exits; retain their stop and effective short horizon.
- Invalid numeric data: display unavailable, never coerce to zero.
- A due state is a projection at the persisted price/time. PositionMonitor remains the sole autonomous execution authority.

Acceptance Criteria

- Every open paper position renders an Exit plan section without changing layout width on mobile.
- Stop, target, score/freshness, and time horizon match the inputs PositionMonitor would read.
- US positions cannot receive India scores or mandates, and vice versa.
- Missing/stale scores cannot produce score_exit_due.
- Null targets remain null.
- The page performs no external provider request and no write.
- Pure projection tests cover precedence, stale/missing score, null target, grandfathered horizon, and market identity.
- The Trades table places Exit plan between Score and Realized P&L, and closed trades never receive a current plan.

paper portfolio benchmarks - Feature Architecture

Source: `features\paper-portfolio-benchmarks\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Paper Portfolio - market-aware benchmarks + page cleanup

Status: SHIPPED. 2026-07-08.

- migration 130 (applied): `paper_performance.bench_nav/bench_return_pct`.
- paper-trade route computes VOO (US) / NIFTY ^NSEI (India) into `bench_* + alpha` for BOTH markets (US also mirrors into legacy `spy_*` for back-compat).
- metrics route surfaces `bench_return_pct` (+ `benchmarkLabel`) so India gets a benchmark.
- PortfolioPage gauge is market-aware ("Alpha vs VOO"/"Alpha vs NIFTY") and reads the stored bench; BenchmarkChart plots the per-market bench line from `bench_return_pct` (dropped the hardcoded US VOO/QQQ fetch - fixed the US-index-on-India leak).
- LiveHoldingsTab is market-aware: US -> Robinhood (\$), India -> Zerodha Kite (?) via new owner-gated `/api/kite/holdings` (read-only). Fixes the Robinhood-on-India leak. Last updated: 2026-07-08

Why

The Paper Portfolio page has three real defects and one missing feature, all surfaced from the live dashboard:

- Benchmark is inconsistent (US) and absent (India).
 - US: the paper-trade route stores alpha vs SPY (`paper_performance.alpha_pct`, `[app/api/agents/paper-trade/route.ts:639]`), but the UI gauge independently fetches VOO and computes a *second, different* alpha labelled "Alpha vs VOO" (`[components/dashboard/PortfolioPage.tsx:199,225,736]`). Two benchmarks for one number.
 - India: the route explicitly skips benchmark computation (`if (market==="us")`), so India stores no benchmark at all - yet the UI still renders the hardcoded "Alpha vs VOO" needle on the India view (reads a meaningless `~+0.0%`).
 - The Risk page footer already promises "US vs SPY, India vs NIFTY" - India has no NIFTY series wired, so the promise is unmet.
 - US-broker text leaks onto the India view (bug). `LiveHoldingsTab()` renders "Robinhood Live Positions" + "Robinhood Trading account 8641" unconditionally (`[PortfolioPage.tsx:535,609]`) - including when `activeMarket === "india"`, whose broker is Kite, not Robinhood. The trade-queue is already market-guarded at `:733`; `LiveHoldings` is not.
 - No benchmark line on the return chart. The "Portfolio vs Benchmark (% return)" chart currently plots only the portfolio return line; the benchmark it names is not drawn. User wants the benchmark drawn alongside portfolio gain/loss, for BOTH markets.
 - General cleanup of the page (redundant/again-fetched benchmark, dead labels).
- Non-goal: this does not touch the fill engine, sizing, money limits, or live-order paths. Read/display layer + one additive benchmark-series write for India.

Proposed design

A. One benchmark per market, computed server-side (single source of truth)

- US benchmark = SPY (already computed and stored in `paper_performance`).

Retire the client-side VOO fetch + client-side alpha. The gauge and the chart both read the stored `spy_return_pct` / `alpha_pct`. Label becomes "Alpha vs SPY". *(Decision point for you: keep SPY, or switch the stored benchmark to VOO? SPY is already wired end-to-end; recommendation: keep SPY, drop VOO.)*

- India benchmark = NIFTY 50 (^NSEI via the existing India quote path -

same Yahoo .NS/index source India NAV already uses). Add its close + return to the India `paper_performance` rows, mirroring the US SPY fields. This is the only data-layer change: the route's per-market perf block gains an India branch that fills `bench_nav/bench_return_pct/alpha_pct` from NIFTY instead of leaving them null.

Schema: `paper_performance` currently has US-specific `spy_nav/spy_return_pct`. Two options:

- (A1, recommended) rename-free, additive: add generic `bench_nav numeric`, `bench_return_pct numeric` columns (nullable). US writes SPY into them (and keeps `spy_*` for backward-compat), India writes NIFTY. UI reads the generic columns. One additive migration, verified applied before UI reads them.
- (A2) reuse `spy_*` columns for both markets (semantically wrong name for India).

Rejected - misleading column name.

B. Market-aware UI (no cross-market leakage)

- Gauge label + benchmark name derived from `activeMarket`: "Alpha vs SPY" (US) /

"Alpha vs NIFTY" (India). No hardcoded "VOO".

- `LiveHoldingsTab` (Robinhood) renders only when `activeMarket === "us"`. On

India, either hide it or show a Kite-scoped equivalent (out of scope here - hide for now, matching the existing trade-queue guard pattern).

C. Benchmark overlay on the return chart (both markets)

- "Portfolio vs Benchmark (% return)" plots two lines: portfolio cumulative %-return

(existing) + benchmark cumulative %-return (SPY US / NIFTY India), both rebased to 0% at the window start, from `paper_performance` series. Legend + the headline number becomes portfolio - benchmark = alpha for that window.

D. Cleanup

- Remove the client VOO fetch/compute path once the gauge/chart read server values.
- Delete the now-dead "Alpha vs VOO" hardcoded label.
- Keep the empty-state copy (US "No open positions...") - correct as-is; it will

populate once the paper-fill fix (migrations 126-128) opens US positions.

Data contract / touch list (for approval)

- Migration (additive): `paper_performance.bench_nav`, `bench_return_pct` (nullable).

Verify applied via `information_schema` before UI reads them.

- app/api/agents/paper-trade/route.ts - per-market perf block: US fills bench_* from SPY (already fetched); add India branch filling bench_* from NIFTY (^NSEI).
- components/dashboard/PortfolioPage.tsx - market-aware label; read stored bench_*; drop client VOO fetch; guard LiveHoldingsTab to US; add benchmark line to the chart.
- Docs: SYSTEM_OVERVIEW.md (benchmark section) + system-map.json only if an agent-to-agent flow changes (it does not - display layer + one perf field).

Locked decisions (2026-07-08)

- US benchmark = VOO. Move the server-side benchmark computation from SPY to VOO in the paper-trade perf block; store into the new generic bench_* columns. Historical spy_* rows are left as-is (VOO?SPY, both S&P 500); the new series is VOO forward. UI label = "Alpha vs VOO". Retire the client-side VOO fetch - the value now comes from the server like India's.
- India benchmark = NIFTY 50 (^NSEI) via the existing India index source.

Written into bench_* on India paper_performance rows. UI label = "Alpha vs NIFTY".

- India live tab = stub a Kite holdings tab. Instead of hiding LiveHoldings on India, render a market-aware live-holdings tab: US -> Robinhood block (existing); India -> a Kite holdings block reading the India account's positions read-only (reuse the existing India live/holdings source the India Live page already uses - no new order path, no new broker-write surface). Guard: Robinhood block renders only for US, Kite block only for India - no cross-market leak either way.

Revised touch list

- Migration (additive): paper_performance.bench_nav, bench_return_pct (nullable).

Verify applied via information_schema before UI reads.

- paper-trade/route.ts: US perf block computes VOO (was SPY) into bench_*; new India branch computes NIFTY (^NSEI) into bench_*.
- PortfolioPage.tsx: market-aware label + benchmark line on the chart; drop client VOO fetch; LiveHoldingsTab becomes market-aware (Robinhood=US, Kite=India read-only).
- Kite holdings read: reuse the existing India live-holdings data source (same one the India Live + Signals page uses); read-only, no order/mutation path.
- Docs: SYSTEM_OVERVIEW.md benchmark section + "Last updated" bump. system-map.json only if a flow changes (it does not).

Awaiting explicit approval to implement (say "approved / implement / code it"). One caveat to confirm: switching US to VOO means the stored benchmark series changes provider mid-history (SPY rows before, VOO after) - acceptable since both track the S&P 500. Flag if you'd rather backfill VOO across history instead.

performance truth - Feature Architecture

Source: features\performance-truth\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Performance Truth Layer - Feature Architecture

Last updated: 2026-07-09 (v2 - post Codex BLOCKER/HIGH review) Status: P0 - awaiting approval before any implementation

Goal

Build a deterministic, mandate-aware, cost-adjusted evaluation layer that answers one question:

"Did this strategy create repeatable, benchmark-relative edge in the mandate

it claims to serve?"

Two evaluation targets (both required):

- Book/trade metrics - what actually happened to the paper book (closed trades + NAV)
- Opportunity-level metrics - what would have happened across the full scored opportunity set (decision_observations x observation_labels)

No live trading impact. No LLM weight optimization. Append-only. Deterministic.

Reuse inventory - what already exists

Analytics math (lib/analytics/performance-metrics.ts)

All math already built. Full reuse, zero changes:

- navToReturns(), sharpe(), sortino(), maxDrawdown()
- expectancy(trades[]) - expects rows with { pnl_pct, outcome } directly (NOT { returnPct })
- costNet(trades[]), slip(trades[])
- calibration(rows[]) - expects { predicted: number (0-1), win: boolean } (NOT { score, won })
- Returns Metric { value, n, insufficient } - abstains correctly when n < min

API (app/api/agents/performance/metrics/route.ts)

Reads paper_performance + paper_trades, returns full metric set per market. Additive change: accept ?mandateId= for trade-level filtering. NAV metrics remain whole-book (paper_performance has no mandate column); label them clearly as book-level, not mandate-specific. Mandate-specific Sharpe comes from opportunity-level evaluation in strategy_evaluations.

UI (components/dashboard/PerformanceTruth.tsx)

Full metric tiles already render. Two additive changes only: (1) mandate selector dropdown, (2) evaluation history table below existing tiles. Existing tiles untouched.

Tables (confirmed columns from migrations audit)

- paper_trades - closed via closed_at IS NOT NULL (no status column)
- Execution quality: expected_price, realized_slip_pct, fill_status, spread_applied, data_confidence, tainted, taint_reason, excluded_from_learning
- paper_performance - canonical NAV plus seed-based total/daily P&L, cumulative

outcome counts/win rate, alpha_pct, and bench_return_pct per (date, market); NOT per mandate. Both runtime writers use the same derivation helper so a later mark-to-market cannot leave stale/default analytics behind.

- decision_observations - immutable append-only; full feature scores, weights_used, signal_id
- observation_labels - matured forward returns per observation (used by learning dataset)
- lib/learning/dataset.ts - loadLabeledDataset() already joins decision_observations x observation_labels
- edge_universe_members - NOT point-in-time; current-liquid/survivorship-biased.

Do NOT use for promotion-grade PIT universe evaluation. Flag all evaluations that use this as universe_pit_safe: false in strategy_evaluations.

Validation engine (lib/validation/engine.ts)

Walk-forward replay, block bootstrap, p-value. Already persists to validation_experiments table and updates strategy_versions.validation_experiment_id. This is the canonical challenger evidence ledger. The new strategy_evaluations table is a summary layer that REFERENCES validation_experiments via FK - it does NOT replace it.

Auth helper

Use requireOwner() from lib/auth/require-owner.ts - not requireOwnerSession().

Net-new work

Migration A - investment_mandates

```
create table public.investment_mandates (  
  id                uuid primary key default gen_random_uuid(),  
  name              text not null,  
  market            text not null check (market in ('us', 'india')),  
  horizon            text not null check (horizon in (  
    'swing_2_20d', 'position_1_6m',  
    'long_term_1y_plus', 'income_dividend'  
  )),  
  -- 'intraday' excluded: no intraday data/execution model exists  
  benchmark_symbol   text not null,  
  min_holding_days   int,  
  max_holding_days   int,  
  evaluation_horizon_days int[] not null default array[5, 10, 20],  
  max_position_pct    numeric not null default 10, -- advisory/evaluation only; NOT wired to broker  
  gateway            boolean,  
  turnover_budget_monthly numeric,  
  allowed_asset_types text[] not null default array['equity', 'etf'],  
  allowed_signal_families text[] not null default array['momentum', 'quality', 'technical', 'sentiment',  
  'macro'],  
  tax_sensitivity     text not null default 'medium'  
    check (tax_sensitivity in ('low', 'medium', 'high')),  
  income_preference   text not null default 'none'  
    check (income_preference in ('none', 'dividend', 'growth')),  
  execution_model      text not null default 'conservative_close'  
    check (execution_model in ('conservative_close', 'optimistic_close')),  
  -- NOT a live-trading authorization flag. Advisory/review eligibility only.  
  -- Broker gateway NEVER reads this. Live orders always require owner-click gate.  
  eligible_for_live_review boolean not null default false,  
  mandate_version     int not null default 1,  
  active              boolean not null default true,  
  archived_at         timestamptz,  
  created_at          timestamptz not null default now()  
);  
  
alter table public.investment_mandates enable row level security;  
-- Deny by default; service-role routes bypass RLS. Add owner policies only when direct client reads needed.  
  
-- Seed default mandates (maps to existing swing behavior - all existing signals default to these)  
insert into public.investment_mandates  
  (name, market, horizon, benchmark_symbol, min_holding_days, max_holding_days)  
values  
  ('Swing US 2-20d', 'us', 'swing_2_20d', 'V00', 2, 20),  
  ('Swing India 2-20d', 'india', 'swing_2_20d', 'NIFTY50.NS', 2, 20);
```

Migration B - strategy_evaluations (append-only, trigger-guarded)

```
create table public.strategy_evaluations (  
  id                uuid primary key default gen_random_uuid(),  
  mandate_id        uuid not null references public.investment_mandates(id),  
  -- Snapshot of mandate AT EVALUATION TIME. Mandate rows are mutable;  
  -- evaluations must be reproducible even if mandate config later changes.  
  mandate_snapshot  jsonb not null,  
  market            text not null,  
  evaluated_at       timestampz not null default now(),  
  evaluator_version  text not null,          -- VERCEL_GIT_COMMIT_SHA or 'local'  
  
  -- Input dataset bounds (for idempotency detection)  
  dataset_hash       text,                  -- sha256 of sorted trade IDs  
  window_start       date,  
  window_end         date,  
  
  -- Trade counts (always filled)  
  n_trades_total     int not null default 0,  
  n_trades_evaluable int not null default 0, -- excludes tainted + excluded_from_learning  
  tainted_count       int not null default 0,  
  excluded_count      int not null default 0,  
  n_observations      int,                  -- decision_observations count in window  
  
  -- Book metrics (from closed paper trades + paper NAV; whole-book Sharpe)  
  -- Null when n_trades_evaluable < 20  
  book_sharpe        numeric,  
  book_sortino        numeric,  
  book_max_drawdown   numeric,  
  book_win_rate       numeric,  
  book_expectancy_pct numeric,  
  book_alpha_pct      numeric,  
  book_benchmark_symbol text,  
  book_cost_adjusted_return_pct numeric,  
  book_slip_vs_modeled_bps numeric,  
  
  -- Opportunity-level metrics (from decision_observations x observation_labels)  
  -- Null until observation_labels for this window have matured  
  opp_n_labeled       int,  
  opp_hit_rate         numeric,            -- fraction of scored signals where actual beat benchmark  
  opp_benchmark_neutral_expectancy numeric, -- avg(label - benchmark_label) across observations  
  opp_ic               numeric,            -- information coefficient (score vs label rank)  
  opp_t_stat          numeric,  
  universe_pit_safe    boolean not null default false, -- true only when PIT universe membership exists  
  
  -- Walk-forward validation reference  
  validation_experiment_id bigint references validation_experiments(id),  
  walk_forward_folds      jsonb, -- [{fold, start_date, end_date, n_effective, p_improvement, passed}]  
  
  -- Display-only health flag (NOT a live-trading promotion gate)  
  -- 'insufficient_sample' | 'negative_or_zero_edge' | 'promising_but_unvalidated' | 'validation_required'  
  health_label          text not null default 'insufficient_sample',  
  health_reason         text,  
  
  -- FULL promotion gate (separate from health_label):  
  -- Requires validation_experiments.passed=true, p_improvement, fold wins, drawdown cap,  
  -- benchmark-neutral expectancy, cost-adjustment, taint ratio check.  
  -- NOT implemented in P0 display layer. Wired at Learner promotion gate in P1.  
  promotion_eligible    boolean not null default false,  
  
  created_at            timestampz not null default now()  
  -- APPEND-ONLY: trigger below blocks updates/deletes  
);  
  
alter table public.strategy_evaluations enable row level security;  
  
create index se_mandate_market_idx on public.strategy_evaluations(mandate_id, market, evaluated_at desc);  
  
-- Append-only enforcement (same pattern as decision_observations)  
create or replace function se_no_update() returns trigger language plpgsql as $$  
begin raise exception 'strategy_evaluations is append-only'; end; $$;  
create trigger se_no_update_trigger  
  before update or delete on public.strategy_evaluations  
  for each row execute function se_no_update();
```

Migration C - mandate_id FK on existing tables

```
-- Nullable first - backfill after seed inserts before making NOT NULL
alter table public.agent_signals      add column mandate_id uuid references public.investment_mandates(
id);
alter table public.paper_trades      add column mandate_id uuid references public.investment_mandates(
id);
alter table public.decision_observations add column mandate_id uuid references public.investment_mandates(
id);
```

Backfill SQL (run after seed, before any NOT NULL constraint):

```
-- Assign all existing rows to the matching default mandate by market
update public.agent_signals set mandate_id = (
  select id from public.investment_mandates
  where name = case when market = 'india' then 'Swing India 2-20d' else 'Swing US 2-20d' end
  and active = true limit 1
) where mandate_id is null;

-- Same for paper_trades
update public.paper_trades set mandate_id = (
  select id from public.investment_mandates
  where name = case when market = 'india' then 'Swing India 2-20d' else 'Swing US 2-20d' end
  and active = true limit 1
) where mandate_id is null;

-- Same for decision_observations (market column exists since mig 059)
update public.decision_observations set mandate_id = (
  select id from public.investment_mandates
  where name = case when market = 'india' then 'Swing India 2-20d' else 'Swing US 2-20d' end
  and active = true limit 1
) where mandate_id is null;
```

File wiring - mandate_id propagation

Every boundary that creates a signal, observation, paper fill, or validation run must read the default mandate and attach mandate_id. Files to update:

```
| File | What to change |
| `lib/research-agent.ts` | When inserting `agent_signals`, resolve default mandate from `investment_mandates` by market; pass `mandate_id` |
| `lib/deepseek-agent.ts` | Same - if it also inserts agent_signals or decision_observations |
| `app/api/agents/paper-trade/route.ts` | When calling paper-fill: pass `mandate_id` from linked agent_signal |
| `execute_paper_fill` RPC (migration) | Accept `mandate_id` param; write to paper_trades |
| `lib/learning/dataset.ts` -> `loadLabeledDataset()` | Add optional `mandate_id` filter to JOIN |
| `lib/validation/engine.ts` | Accept optional `mandate_id`; filters observations to mandate before walk-forward |
```

New file: lib/evaluation/run-evaluation.ts

Reuses all math from lib/analytics/performance-metrics.ts. No LLM. No weight mutation.

```
import {
  navToReturns, sharpe, sortino, maxDrawdown,
  expectancy, costNet, slip, calibration
} from "@lib/analytics/performance-metrics";
import type { SupabaseClient } from "@supabase/supabase-js";
import { createHash } from "crypto";

export async function runEvaluation(mandateId: string, market: string, supabase: SupabaseClient) {
  // 1. Fetch mandate (fail closed on error)
  const { data: mandate, error: mandateErr } = await supabase
    .from("investment_mandates").select("").eq("id", mandateId).single();
  if (mandateErr || !mandate) return { ok: false, error: mandateErr?.message ?? "mandate not found" };

  // 2. Closed, non-tainted, evaluable trades for this mandate
  // paper_trades closes via closed_at (no status column)
```

```

const { data: allTrades, error: tradesErr } = await supabase
  .from("paper_trades")
  .select("pnl_pct, spread_applied, realized_slip_pct, expected_price, fill_price, analyst_score,
closed_at, tainted, excluded_from_learning")
  .eq("mandate_id", mandateId)
  .eq("market", market)
  .not("closed_at", "is", null);
if (tradesErr) return { ok: false, error: tradesErr.message };

const trades = allTrades ?? [];
const taintedCount = trades.filter(t => t.tainted).length;
const excludedCount = trades.filter(t => t.excluded_from_learning && !t.tainted).length;
const evaluable = trades.filter(t => !t.tainted && !t.excluded_from_learning);

// Dataset hash for idempotency detection (sort by closed_at so order doesn't matter)
const tradeIds = [...trades].sort((a, b) => a.closed_at < b.closed_at ? -1 : 1).map(t => t.closed_at);
const datasetHash = createHash("sha256").update(JSON.stringify(tradeIds)).digest("hex").slice(0, 16);
const windowStart = trades.length ? trades.reduce((a, b) => a.closed_at < b.closed_at ? a : b).closed_at?.
slice(0, 10) : null;
const windowEnd = trades.length ? trades.reduce((a, b) => a.closed_at > b.closed_at ? a : b).closed_at?.
slice(0, 10) : null;

// 3. NAV series for whole-book metrics (paper_performance has no mandate column)
const { data: navRows, error: navErr } = await supabase
  .from("paper_performance")
  .select("date, nav, alpha_pct, bench_return_pct")
  .eq("market", market).order("date");
if (navErr) return { ok: false, error: navErr.message };

// 4. Compute book metrics (reuse existing math - do NOT reimplement)
const returns = navToReturns((navRows ?? []).map(r => r.nav));
// expectancy() expects rows with pnl_pct field directly
const expM = expectancy(evaluable);
const sharpeM = sharpe(returns);
const sortM = sortino(returns);
const maxDDM = maxDrawdown(returns);
const costM = costNet(evaluable);
const slipM = slip(evaluable);
// calibration() expects { predicted: 0..1, win: boolean }
const calibM = calibration(
  evaluable.map(t => ({ predicted: Number(t.analyst_score ?? 50) / 100, win: (t.pnl_pct ?? 0) > 0 }));
);
const alphaPct = (navRows ?? []).length
  ? (navRows![navRows!.length - 1].alpha_pct ?? null)
  : null;

// 5. P0 display health label (NOT a promotion gate)
const n = evaluable.length;
const MIN_EVAL = 20;
let healthLabel = "insufficient_sample";
let healthReason = `Need ${MIN_EVAL} evaluable trades, have ${n}`;
if (n >= MIN_EVAL) {
  if (sharpeM.insufficient || sharpeM.value <= 0) {
    healthLabel = "negative_or_zero_edge";
    healthReason = `Sharpe ${sharpeM.insufficient ? "N/A" : sharpeM.value.toFixed(2)} <= 0`;
  } else if (sharpeM.value < 0.5) {
    healthLabel = "promising_but_unvalidated";
    healthReason = `Sharpe ${sharpeM.value.toFixed(2)} < 0.5; run walk-forward validation`;
  } else {
    healthLabel = "validation_required";
    healthReason = `Sharpe ${sharpeM.value.toFixed(2)} >= 0.5; confirm with walk-forward`;
  }
}
// promotion_eligible requires validation_experiments.passed=true - NOT set here in P0

// 6. Persist (append-only - never upsert)
const { error: insertErr } = await supabase.from("strategy_evaluations").insert({
  mandate_id: mandateId,
  mandate_snapshot: mandate, // snapshot mandate at evaluation time
  market,
  evaluator_version: process.env.VERCEL_GIT_COMMIT_SHA ?? "local",
  dataset_hash: datasetHash,
  window_start: windowStart,
  window_end: windowEnd,
  n_trades_total: trades.length,

```

```

n_trades_evaluable:      n,
tainted_count:           taintedCount,
excluded_count:          excludedCount,
// Book metrics (whole-book NAV - not mandate-scoped Sharpe, document this)
book_sharpe:             sharpeM.insufficient ? null : sharpeM.value,
book_sortino:            sortM.insufficient ? null : sortM.value,
book_max_drawdown:       maxDDM.insufficient ? null : maxDDM.value,
book_win_rate:           expM.insufficient ? null : expM.winRate, // use winRate, not value
book_expectancy_pct:     expM.insufficient ? null : expM.value,
book_alpha_pct:          alphaPct,
book_benchmark_symbol:   mandate.benchmark_symbol,
book_cost_adjusted_return_pct: costM.insufficient ? null : costM.value,
book_slip_vs_modeled_bps: slipM.insufficient ? null : slipM.value * 10000,
// Opportunity metrics: null until observation_labels mature (P1)
opp_n_labeled:           null,
opp_hit_rate:            null,
opp_benchmark_neutral_expectancy: null,
opp_ic:                  null,
opp_t_stat:              null,
universe_pit_safe:       false, // edge_universe_members is NOT PIT-safe
// Walk-forward: null until validation_experiments run referencing this mandate
validation_experiment_id: null,
walk_forward_folds:      null,
health_label:             healthLabel,
health_reason:            healthReason,
promotion_eligible:       false, // P0: never set to true here
});
if (insertErr) return { ok: false, error: insertErr.message };

return {
  ok: true,
  n: trades.length,
  n_evaluable: n,
  health_label: healthLabel,
  health_reason: healthReason,
  sharpe: sharpeM,
};
}

```

New routes

POST /api/agents/evaluation/run (owner-gated, no cron access)

```

// Body: { mandateId: string, market: 'us' | 'india' }
// Auth: requireOwner() - no cron secret accepted
// Calls: runEvaluation() -> returns { ok, health_label, n, n_evaluable, sharpe }
// No live trading data exposed or mutated

```

GET /api/agents/evaluation/results (owner-gated)

```

// Query: ?mandateId=&market=&limit=20
// Returns: strategy_evaluations rows, latest first
// Does NOT expose: mandate_snapshot (strip to keep response lean)

```

UI additions to PerformanceTruth.tsx (additive only)

- Mandate selector (top of card)
- Dropdown from investment_mandates where market = currentMarket AND active = true
- Default: Swing US 2-20d / Swing India 2-20d
- Drives ?mandateId= on existing /api/agents/performance/metrics call for trade metrics
- Note label: "NAV/Sharpe is whole-book - trade metrics below are mandate-filtered"
- Evaluation history table (below existing tiles)

- Reads GET /api/agents/evaluation/results?mandateId=&market=
- Columns: Date | Trades(eval/total) | Sharpe | MaxDD | Alpha | Health | WF Pass
- "Run Evaluation" button -> POST -> spinner -> refresh
- Table is read-only. No edit/delete.
- Show universe_pit_safe as a warning chip when false.

Pass/fail gate - P0 vs promotion

P0 (this feature): health_label display only. Four states:

- insufficient_sample - n_evaluable < 20
- negative_or_zero_edge - Sharpe <= 0
- promising_but_unvalidated - Sharpe 0-0.5
- validation_required - Sharpe >= 0.5 (requires walk-forward to become promotion-eligible)

Promotion gate (P1, wired at LearnerAgent): all of:

- validation_experiments.passed = true
- p_improvement threshold met
- >= 3/5 folds won
- n_effective >= min by horizon
- Benchmark-neutral expectancy > 0
- Max drawdown within mandate limit
- Tainted/evaluable ratio below threshold
- No unresolved data-quality warnings

promotion_eligible in strategy_evaluations is always false in P0. Only the LearnerAgent promotion gate (after walk-forward) sets it via a new row insert.

Opportunity-level evaluation (current design gap - add to P1 scope)

P0 evaluates only closed paper trades (book truth). This is insufficient for signal quality proof because trades are policy-selected.

The repo already has the infrastructure:

- decision_observations - all scored candidates (not just traded ones)
- observation_labels - matured forward returns
- lib/learning/dataset.ts - loadLabeledDataset() joins these
- lib/validation/engine.ts - walk-forward replay

P1 addition: populate opp_* columns in strategy_evaluations:

- Filter decision_observations by mandate_id and horizon window
- Join to observation_labels (matured only)
- Compute opp_ic, opp_t_stat, opp_hit_rate, opp_benchmark_neutral_expectancy

- Mark `universe_pit_safe`: `true` only when a proper PIT universe snapshot exists

What NOT to do (hard constraints)

- No LLM in evaluation path - deterministic only
- Do NOT mutate `strategy_config`, money limits, or strategy weights from evaluation
- Do NOT make `mandate_id` NOT NULL until backfill verified in production
- Do NOT add intraday mandate - no intraday data or execution model exists
- Do NOT use `eligible_for_live_review` as a broker gateway signal - advisory label only
- Do NOT replace `validation_experiments` with `strategy_evaluations` - they serve different roles
- Do NOT call `expectancy()` with `{ returnPct }` - it expects `{ pnl_pct }`
- Do NOT call `calibration()` with `{ score, won }` - it expects `{ predicted (0-1), win }`
- Do NOT query `paper_trades` with `.eq("status", "closed")` - use `.not("closed_at", "is", null)`
- Do NOT call `requireOwnerSession()` - use `requireOwner()` from `lib/auth/require-owner.ts`
- Do NOT set `promotion_eligible = true` in P0 - reserved for P1 LearnerAgent gate

P1 follow-ups (separate feature docs)

- Opportunity-level evaluation - populate `opp_*` columns using `decision_observations x observation_labels + lib/validation/engine.ts` with mandate filter
- Data quality ledger - `data_quality_log` table per (symbol, date, dimension) with source/freshness/confidence; feed into `data_confidence` on `paper_trades/decision_observations`
- Conservative paper execution model - stale quote guard, partial fills, "would_not_fill" outcomes, market-hours check per market
- ResearchDecision output shape - extend `agent_signals` to include `expected_return_bps`, `evidence_quality`, `missing_evidence[]`, `positive_drivers[]`, `negative_drivers[]`, `abstention_reason`

Acceptance criteria

- [] `investment_mandates` seeded with Swing US + Swing India defaults
- [] All new `paper_trades` inserts get `mandate_id` from linked `agent_signals`
- [] All new `agent_signals` get `mandate_id` set in `lib/research-agent.ts`
- [] All new `decision_observations` get `mandate_id` before insert
- [] `runEvaluation()` uses `.not("closed_at", "is", null)` - not status
- [] `expectancy()` called with `pnl_pct` field; `book_win_rate` stored from `expM.winRate`
- [] `calibration()` called with `{ predicted: score/100, win: pnl_pct > 0 }`
- [] All Supabase errors return `{ ok: false, error: message }` - no silent failures
- [] `strategy_evaluations` rows never updated/deleted (trigger enforced)
- [] `promotion_eligible` is always false in P0

- [] mandate_snapshot populated with mandate row JSON at evaluation time
- [] dataset_hash populated so UI can detect duplicate/same-dataset reruns
- [] RLS enabled on investment_mandates and strategy_evaluations (deny by default)
- [] eligible_for_live_review NOT read by any broker gateway or order placement code
- [] /api/agents/evaluation/run uses requireOwner() and returns 401 without session
- [] PerformanceTruth.tsx shows "NAV/Sharpe is whole-book" label + mandate selector + history table
- [] Existing metric tiles (Sharpe/Sortino/MaxDD/Calibration) continue working unchanged
- [] lib/validation/engine.ts - NOT changed in P0; validation_experiments remains canonical

Migration summary

#	Table	Change	Risk
A	investment_mandates	NEW + RLS	Low - no FK deps initially
B	strategy_evaluations	NEW + RLS + append-only trigger	Low - no existing code reads it
C	agent_signals	ADD mandate_id (nullable)	Low - nullable, backfill after seed
C	paper_trades	ADD mandate_id (nullable)	Low - nullable
C	decision_observations	ADD mandate_id (nullable)	Low - trigger already blocks non-inserts

All migrations additive. No existing columns modified. No existing behavior changed.

pipeline data and timing - Feature Architecture

Source: `features\pipeline-data-and-timing\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Pipeline Data and Timing Remediation

Status: SHIPPED Owner: ResearchAgent and scheduling control plane Decision: `PROJECT_DECISIONS.md` Decision 61
Markets: US and India, always isolated

Implementation verification

- Completed-session filtering is applied after provider normalization and before scoring, return capture, evidence persistence, or trade-plan generation.
- US paired seasonal crons are active in production and every command carries the expected market-local slot. Duplicate seasonal invocations exit before provider or database work.
- India GDELT sentiment is unsupported by the active scorer. The replacement India news/event collector writes shadow-only intents that no decision or money path reads.
- Production migration `20260731210000_market_local_crons_and_india_news_shadow.sql` is applied.
- Verification passed on 2026-07-31: 168 test files passed (1 skipped), 1,466 tests passed (7 skipped), TypeScript passed, and the Next.js production build completed.
- The first production India shadow canary processed 12 symbols, found relevant RSS headlines for 9, wrote 13 append-only provider-call rows and 24 canonical cache rows, and created zero signals and zero paper positions. The run recorded `behavior changed=false`.

Problem

Three production facts require corrective work:

- Intraday research can receive a provider's still-forming daily candle. That can make a daily technical score depend on an incomplete session.
- Fixed UTC US crons preserve UTC, not New York wall-clock time. After a daylight-saving transition, the close monitor can run before the US close and research/trader slots shift by one hour.
- India news sentiment is nominally applicable but produced 0 available observations in the latest 310 production decisions. GDELT rejects the per-symbol access pattern under its traffic policy, so repeated calls consume time without producing evidence.

Historical replay remains diagnostic-only. Its accepted India run (111 dates, mean IC 0.0125, HAC t 1.32, unstable fold signs) does not authorize a scoring or promotion change.

Decisions

1. Daily scoring uses completed sessions only

ResearchAgent filters normalized daily candles immediately before any score, return capture, trade-plan, or evidence calculation. A candle dated today in the market's local timezone is eligible only after the regular close: 16:00 America/New_York for US and 15:30 Asia/Kolkata for India. Older valid dates remain eligible. Future and malformed dates are rejected.

This boundary belongs after provider normalization, not inside a provider adapter, because quote/chart consumers may legitimately need an intraday bar. It applies identically to every candle provider.

2. US schedules are market-local contracts

Each intended US local slot is scheduled at both possible UTC hours. The route admits only the invocation whose New York local HH:mm equals the declared slot. The other invocation exits before provider or database work. India remains on fixed UTC because IST has no daylight-saving transition.

Only cron requests carrying `local_slot` are checked. Owner/manual and closed-day catch-up invocations without the parameter preserve their existing behavior. Invalid slot syntax fails closed.

```
| Flow | Local contract | EDT UTC | EST UTC |
| US research AM | 09:00 ET | 13:00 | 14:00 |
| US paper entry AM | 11:15 ET | 15:15 | 16:15 |
| US research PM | 14:00 ET | 18:00 | 19:00 |
| US paper entry PM | 15:15 ET | 19:15 | 20:15 |
| US position monitor | 16:15 ET | 20:15 | 21:15 |
```

3. India sentiment is removed from active applicability

Until a replacement is validated, India news sentiment is structurally unavailable rather than repeatedly requested and silently renormalized. India scores continue using only measured dimensions. This changes no weight, threshold, order, or historical signal.

4. Replacement evidence is shadow-only

A daily post-close India shadow collects official NSE corporate announcements when its cookie-gated public endpoint is reachable and bounded Google News RSS headlines as an unofficial coverage fallback.

It stores sanitized, source-labelled headline/event evidence in the existing `evidence_cache_v2` and provider-call lineage in `provider_call_ledger`. It introduces no parallel truth table. It computes coverage and provenance only: no directional sentiment score and no LLM.

The shadow prioritizes open paper holdings, then enabled watchlist symbols, and rotates the remaining bounded workload by least-recently-observed symbol. Provider failure is recorded and fail-soft. Text is untrusted data, bounded in length, and never interpolated into a money-path prompt.

The program can be promoted only through a separate decision after enough distinct market sessions demonstrate symbol relevance, freshness, stable coverage, and a deterministic, historically evaluated classifier. Promotion may not reuse the GDELT contract implicitly.

Data Contracts

Shadow evidence uses these canonical intents:

- `event.corporate_announcement_shadow`, provider `nse_corporate_announcements`
- `sentiment.news_headlines_shadow`, provider `google_news_rss`

Each `evidence_cache_v2` row binds market, symbol, provider, request fingerprint, payload hash, observed/as-of timestamps, expiry, quality state, INR currency, source URL metadata, and bounded provenance. `provider_call_ledger.run_id` begins with `india-news-shadow`: for exact Upgrade Path accounting.

No scorer, ResearchAgent, PaperTrader, PositionMonitor, TraderAgent, LearnerAgent, or broker route may read either shadow intent.

Flow

```
flowchart LR
    Cron["Cron[\"Market-local cron slot\"] --> Guard[\"DST/local-slot guard\"]"]
    Guard -->|admitted| Research["Research[\"ResearchAgent\"]"]
    Providers["Providers[\"Daily candle providers\"] --> Complete[\"Completed-session filter\"]"]
    Complete --> Research
    Research --> Signals["Signals[\"Market-local signals\"]"]
    Signals --> Paper["Paper[\"PaperTrader\"]"]
    Signals --> Monitor["Monitor[\"PositionMonitor\"]"]

    IndiaCron["IndiaCron[\"India post-close shadow\"] --> NSE[\"NSE announcements\"]"]
    IndiaCron --> RSS["RSS[\"Google News RSS\"]"]
    NSE --> Cache["Cache[\"evidence_cache_v2\"]"]
    RSS --> Cache
    Cache -. "no scoring reader" .-> Review["Review[\"Upgrade Path review\"]"]
```

Safety Invariants

- US and India data are never cross-summed or cross-used.
- Partial current-session bars cannot reach daily scoring.
- A duplicate seasonal UTC invocation performs no provider, scoring, position, or order work.
- India shadow evidence cannot affect a score, threshold, signal, trade plan, position, cash balance, proposal, or broker order.
- Unknown news coverage remains unavailable; it is never converted to neutral sentiment.
- Provider text is untrusted, bounded, and non-executable.
- Historical evidence stays offline/diagnostic until a separately approved promotion gate passes.

Acceptance Gates

- Unit tests cover US summer/winter and India local dates around regular close.
- Unit tests prove current-session candles are stripped before close and retained after close.
- Route tests prove the wrong seasonal UTC invocation exits before its worker runs.
- India applicableDimensions excludes sentiment and no active India research path calls GDELT.
- Shadow parser/relevance tests reject malformed, irrelevant, or oversized feed items.
- Upgrade Path shows India-only schedule, coverage, calls, blockers, and next action.
- Repository-wide tests, TypeScript, production build, migration verification, and production cron inspection pass.
- Architecture chapters, schedule reference, agent diagrams, system map, and work log are updated in the same change.

Non-Goals

- No signal weight or threshold change.
- No automatic historical-data ingestion into scoring.
- No LLM sentiment classifier.
- No options, broker, paper/live order, or position-management change.
- No claim that headline volume predicts direction.

pit fundamentals - Feature Architecture

Source: features\pit-fundamentals\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Point-in-Time (PIT) Fundamentals - Feature Architecture

STATUS: DRAFT - awaiting owner approval Owner: - Author: architecture proposal (deep-audit P1 remediation) Last updated: 2026-07-10

1. Problem statement

The research/scoring pipeline must use financial data as it was known on the decision date. Today it does not. A revenue/EPS figure that is later restated can retroactively change what a past decision "should" have seen, which corrupts both backtests and the learning loop. This is a named P1 gap in the deep-audit remediation.

Two distinct leaks exist, and the fix must address both:

- Leak A - vintage leak on any re-fetch/backfill. All fundamentals today are fetched as **current TTM snapshots with no report date or filing date attached**. If we ever re-score a past date, run a fundamentals-based backtest over history, or backfill, we pull today's (possibly restated) numbers and stamp them onto a past decision. There is no stored vintage to prevent this.

- Leak B - unversioned evidence behind a frozen score. The live score **is** frozen at decision time (see ?3), but the fundamental inputs that produced it are stored as a loose JSON blob with no report-date / filing-date / vintage key. We cannot prove a stored score used as-known data, cannot detect when a later restatement changed the underlying TTM window, and cannot reconstruct the exact inputs for audit.

2. How fundamentals are fetched today (grounded in code)

2.1 Fetch path

- `lib/research-agent.ts` (`processSymbol`, ~L933-948) fetches all data in parallel. US fundamentals go through `fetchUsOverview(symbol, avFallback)` in `lib/data/fundamentals.ts`.

- `fetchUsOverview` calls `fetchFmpOverview` -> FMP ratios-ttm + key-metrics-ttm (day-cached, own 250/day budget), falling back to Alpha Vantage OVERVIEW (`fetchAVOverview`, `lib/research-agent.ts` ~L734).

- India uses `fetchIndiaOverview` (Yahoo quoteSummary), see `lib/india-data.ts`.

- All three map into the same AV-OVERVIEW-shaped `Record<string, string>`:

`PERatio`, `ProfitMargin`, `ReturnOnEquityTTM`, `EPS`, `QuarterlyRevenueGrowthYOY`, `AnalystTargetPrice`, `Sector`, etc.

Key fact: none of these fields carry a report period or filing date. `ratios-ttm` / `key-metrics-ttm` / AV OVERVIEW are all **trailing-twelve-month, as-of-now** rollups. AV OVERVIEW exposes only `LatestQuarter` (a single date on the whole object, not per-field). So the current fetch layer is structurally incapable of PIT - the date dimension is simply absent.

2.2 Scoring path

- `lib/data/scores.ts` -> `scoreFundamentals(overview, isEtf, currentPrice)`

(~L73-158) reads the OVERVIEW fields and produces a deterministic `fundamental_score` + an evidence object (`pe_ratio, profit_margin, roe, eps, revenue_growth_yoy, analyst_upside_pct, sector, ...`).

- `computeScores` (~L296) assembles the 5 sub-scores and fire-and-forget writes `evidence_records` rows via `writeBatchEvidence` (fundamental payload, source `alpha_vantage`, `quality_state`). No date-of-record on the fundamental payload.

2.3 Persistence path (where evidence lands)

- `agent_signals.signal_breakdown` (jsonb) - per-dimension evidence for "today".
- `decision_observations.features` (jsonb) - the learning fuel. Written at `lib/research-agent.ts` ~L1370-1427. Already stamps `schemaVersion: "v1"` and `decisionTs` (ISO timestamp), and spreads `scores.evidence` (incl. the fundamental evidence). Table is append-only, immutable (trigger blocks UPDATE/DELETE - see `docs/arch/04-database-schema.md`).
- `evidence_records` - immutable, `payload_hash` dedup.
- `signal_score_history` - append-only per-symbol scores.

2.4 Learning / backtest consumers

- `lib/learning/dataset.ts` -> `loadLabeledDataset` joins `decision_observations x observation_labels` and returns the stored scores
- `availability_mask` + forward-return labels. `walkForwardFolds` does purge/embargo splitting. It replays stored scores, it does not re-fetch fundamentals - so today's learner is accidentally PIT-safe *for the score number*, but blind to whether that number's inputs were later restated (Leak B).
- `lib/validators/backtest.ts` (Validation Engine) similarly replays stored observations. Any *new* fundamentals-based factor or a re-score/backfill that re-hydrates fundamentals from a live provider would introduce Leak A.

Net finding: the frozen-score design already prevents the crudest leak for the *existing* 5-dim replay. The exposure is (1) any future backfill / re-score / fundamentals-driven backtest, and (2) the absence of a vintage key that lets us audit or detect restatements. Both require capturing report-date + filing-date and versioning restatements - the subject of this proposal.

3. Design

3.1 (a) Capture fundamentals with report-date AND as-of/filing-date

Every fundamental fact gets three dates:

- `report_period` (a.k.a. fiscal period end / report date) - *what period the number describes* (e.g. 2026-03-31, Q1 FY26).
- `filing_date` / `accepted_date` - *when that number became public* (the SEC acceptance timestamp, or provider's `acceptedDate/filingDate`).

- captured_at - *when Kairos first observed this value* (our own vintage clock).

The PIT rule for reads: a fact is "known on date D" iff filing_date <= D (fall back to captured_at <= D when the provider gives no filing date). Never report_period <= D alone - a period can end weeks before it is filed, and using period-end would leak the not-yet-published number.

3.2 (b) Storage model that versions restatements (describe only - no DDL here)

Proposed new tables (append-only, following the existing ledger convention):

fundamental_facts - one immutable row per (symbol, metric window, vintage).

```
| Column | Type | Notes |
| `id` | uuid PK | |
| `symbol` | text | |
| `market` | text | `us` \| `india` |
| `metric_set` | text | `ttm_overview` (matches today's OVERVIEW rollup) \| `quarterly` \| `annual` |
| `report_period` | date | Fiscal period end the values describe |
| `fiscal_period` | text | `Q1`/`Q2`/`Q3`/`Q4`/`FY` (nullable) |
| `filing_date` | date | When it became public (nullable if provider omits) |
| `values` | jsonb | The OVERVIEW-shaped field map (`PERatio`, `ProfitMargin`, ...) |
| `source` | text | `fmp` \| `alpha_vantage` \| `yahoo` \| `financialdatasets` |
| `restatement_seq` | int | 0 = as-first-observed; 1,2... = later restatements of the SAME `report_period` |
| `is_latest` | bool | Convenience flag: newest vintage for (symbol, report_period) |
| `payload_hash` | text | UNIQUE per (symbol, report_period, source, values-hash) - dedups identical re-fetches |
| `captured_at` | timestampz | Our vintage clock |
```

Restatement handling: a new fetch for an existing (symbol, report_period) whose values differ from the newest stored vintage inserts a new row with restatement_seq = prev+1 and flips the old row's is_latest=false. Nothing is ever mutated in place - the original as-first-reported row survives forever. This is what makes "a later-restated revenue must not retroactively change a past score" enforceable.

fundamental_fetch_log (optional, ops) - every provider hit with throttle/shape outcome, so a restatement can be traced to a specific fetch.

Also add, on decision_observations.features (no migration - it's jsonb):

- fundamentals_vintage: { fact_id, report_period, filing_date, restatement_seq, captured_at }

so each stored decision points at the exact immutable vintage it consumed.

3.3 (c) Read API - "fundamentals as known on date D"

New module lib/data/fundamentals-pit.ts:

```
getFundamentalsAsOf(symbol, market, asOf: Date, opts?): Promise<{
  overview: Record<string,string>; // same shape scoreFundamentals reads today
  vintage: { fact_id, report_period, filing_date, restatement_seq, captured_at } | null;
  source: string;
}>
```

Query: newest fundamental_facts row for symbol where COALESCE(filing_date, captured_at) <= asOf, ordered by that date desc then restatement_seq asc-at-or-before-asOf (i.e. pick the vintage that was the latest *known* one on asOf, not a future restatement). Returns the OVERVIEW map untouched, so scoreFundamentals needs zero changes - it keeps reading a Record<string, string>.

- Live path (asOf = now): getFundamentalsAsOf(symbol, market, new Date())

simply returns the newest known vintage - behaviourally identical to today, but now it also writes/reads the vintage record.

- The existing fetchUsOverview becomes the capture writer: on each live

fetch it upserts into fundamental_facts (append vintage on change), then hands the same overview to the caller. This is the single ingestion point.

3.4 (d) ResearchAgent + learning dataset consume it consistently

- ResearchAgent (live): replace the raw `fetchUsOverview(...).overview` at `lib/research-agent.ts` ~L948 with `getFundamentalsAsOf(symbol, market, now)`, and record `features.fundamentals_vintage`. Capture-on-fetch means live scoring builds the PIT archive as a side effect.

- Learning dataset (`lib/learning/dataset.ts`): unchanged for the *existing*

5-dim replay (it already reads frozen scores - keep it that way). It gains one capability: when a future factor or the Validation Engine needs to *recompute* a fundamental at a historical decision, it **MUST** call `getFundamentalsAsOf(symbol, market, observation.ts)` - never a live fetch. Add a lint/guard note in `lib/validators/backtest.ts` that fundamentals in replay come only through the PIT read API. This is what makes backtest and live scoring agree: both resolve fundamentals through one date-parameterised function.

- Restatement-drift detection (Leak B): a periodic job can compare

`decision_observations.features.fundamentals_vintage.fact_id` against the current `is_latest` vintage for that `report_period`; a mismatch flags the observation as *restated-after-decision* (surfaced to the learner as a taint signal, not a mutation). The stored score is never changed.

3.5 (e) Backfill strategy for existing history

We cannot reconstruct as-first-reported values for the past - no current provider serves original-vs-restated history (see ?4). Honest plan:

- Forward-only PIT from go-live. Start capturing vintages now; the archive

becomes correct from day one and compounds. Mark all pre-go-live decisions `pit_status = "pre_archive"` in features (they keep their frozen scores).

- Best-effort seed for actively-traded symbols: one snapshot per current

`report_period` using FMP `income-statement / financial-reports-dates` which *do* expose `date/filingDate/acceptedDate` - stamped `restatement_seq = 0`, `source_note = "seed_latest_restated"` so it is never mistaken for a true first-report vintage.

- Never rewrite existing `decision_observations` (append-only trigger forbids it anyway). Backfill only populates the new `fundamental_facts` table.

4. Data-source capability (does the provider expose filing dates / restated?)

Provider (role today)	Report date	Filing date	Original-vs-restated
FMP <code>`ratios-ttm` / `key-metrics-ttm`</code> (**current US fundamentals**, <code>lib/data/fundamentals.ts</code>)	? (TTM rollop)	? ?	
FMP <code>`income-statement` / `financial-reports-dates`</code> (not used yet)	? <code>`date` / `calendarYear`+`period`</code>	? <code>`filingDate`, `acceptedDate`</code> ? (serves latest restated)	
Alpha Vantage <code>`OVERVIEW`</code> (fallback)	~ <code>`LatestQuarter`</code> only	? ?	
FinancialDatasets <code>`get_financial_metrics` / `get_income_statement`</code> (MCP, referenced in a doctrine prompt but not wired)	? <code>`report_period`</code> partial	? (latest restated)	
Yahoo <code>`quoteSummary`</code> (India)	partial	? ?	
Specialized PIT vendors (Compustat Point-in-Time, S&P)	? ? ?	(out of budget/scope)	

Conclusion: no in-budget provider gives true as-first-reported history. The only way Kairos gets real PIT is to become its own point-in-time archive by snapshotting filing-date-stamped fundamentals on every fetch going forward, and appending a new vintage row whenever values for a `report_period` change. Filing dates ARE obtainable (FMP `income-statement.acceptedDate`, FinancialDatasets `report_period`), so the capture side is feasible without a new paid vendor.

5. Phased build plan + effort

```
| Phase | Scope | Effort |
| **P0 - Schema** | `fundamental_facts` (+ optional `fundamental_fetch_log`) migration; verify applied via `list_migrations` before code ships (per CLAUDE.md schema rule). Update `docs/arch/04-database-schema.md`. | ~0.5 day |
| **P1 - Capture** | Make `fetchUsOverview` / `fetchIndiaOverview` upsert vintages (append-on-change, filing-date from FMP `income-statement` supplemental call, day-cached). Dedup via `payload_hash`. | ~1.5 days |
| **P2 - Read API** | `lib/data/fundamentals-pit.ts` `getFundamentalsAsOf`; unit tests against a hand-built multi-vintage fixture (mirrors `walkForwardFolds` test style). | ~1 day |
| **P3 - Wire live** | ResearchAgent consumes `getFundamentalsAsOf(now)`; stamp `features.fundamentals_vintage`. No change to `scoreFundamentals`. | ~0.5 day |
| **P4 - Replay guard** | Route all replay/backtest fundamentals through the PIT API; add restatement-drift detector + a `pit_restated_after_decision` taint signal for the learner. Update `docs/arch/09-learning-loop.md`. | ~1.5 days |
| **P5 - Backfill** | Forward-only default + best-effort seed for the active universe; `pre_archive` tagging. | ~1 day |
```

Total ~6 engineer-days, shippable phase-by-phase behind the existing append-only-ledger conventions. Docs to touch on ship: docs/arch/04-database-schema.md (P0), docs/arch/02-tech-stack.md (new FMP endpoint / capture note), docs/arch/09-learning-loop.md (P4), and public/agent-diagrams/system-map.json if the ResearchAgent->learner data flow node descriptions change.

6. Effect on existing scores

- No existing score changes. decision_observations is append-only; frozen
- scores stay frozen. Live scoring output is numerically identical on go-live (same provider, same fields) - we only *add* a vintage record alongside.
- The learner gains a new *observed* taint feature (restated-after-decision); per
- CLAUDE.md's pushback mandate it is logged evidence, not a new weighted dimension until it earns an IC track record.

7. Risks / open questions

- Filing date availability & cost. FMP income-statement gives acceptedDate, but it is a separate call - does the 250/day FMP budget absorb one more day-cached call per symbol? (Likely yes given day-cache; verify.)
- India filing dates. Yahoo quoteSummary does not expose filing dates cleanly; India may have to fall back to captured_at as the as-of clock (still correct forward, just coarser). Acceptable?
- TTM windows don't have a single filing date. A TTM rollup blends 4 quarters; the honest filing_date for a TTM metric_set is the filing date of its *most recent* constituent quarter. Confirm that convention.
- Restatement detection sensitivity. Provider re-computations / rounding can flip a value trivially. Need a tolerance threshold before appending a new restatement_seq vintage to avoid vintage churn.
- Should the seed backfill exist at all? It stamps latest-restated values as restatement_seq=0, which is *not* truly first-reported - risk of it being mistaken for real PIT. Option: forward-only, skip seed entirely.
- get_financial_metrics (FinancialDatasets MCP) wiring. It exposes

report_period cleanly and could become the canonical PIT fundamentals source instead of FMP TTM. Bigger change; out of this proposal's default scope but noted as the strongest long-term option.

8. What this does NOT do

- Does not reconstruct as-first-reported fundamentals for history predating go-live - no in-budget provider offers that.
- Does not change scoreFundamentals math, the 5-dim weights, or any score value already stored.
- Does not add a new weighted scoring dimension (restatement taint is logged evidence only, per pushback mandate).
- Does not re-fetch or mutate decision_observations / evidence_records (append-only ledgers stay immutable).
- Does not purchase a specialized PIT data vendor (Compustat/S&P) - explicitly out of scope; flagged as the only path to true pre-history original-vs-restated.
- Does not cover point-in-time *prices* or *index membership* survivorship - separate audit items.

policy event ledger - Feature Architecture

Source: `features\policy-event-ledger\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

US Policy Event Ledger

Decision

Build a US-only, record-only FOMC ledger so Kairos can distinguish a scheduled Fed decision, the pre-decision market expectation, the realized target range, and later observed symbol/benchmark returns. The existing MacroSentinel remains a slow US macro regime. This feature is not a score, trade, sizing, or exit input.

Sources And Boundaries

- Official outcome: FRED DFEDTARL / DFEDTARU, whose source is the Federal

Reserve and whose observations are the effective target range. A scheduled event is marked decided only after both ranges are observed after its date.

- Expectation: immutable snapshots are supported, but no source is enabled.

CME's supported FedWatch API is paid; do not scrape the public webpage or fabricate probabilities. A future licensed adapter writes only pre-decision snapshots with source URL, source name, and capture time.

- Impact: compound only rows already present in `symbol_daily_returns`.

No event route fetches quotes or candles, so it cannot consume research API quota or cause a late provider revision to masquerade as a decision-time fact.

- Scope: US FOMC only. India receives no Fed score, event, or inferred RBI substitute. A separate RBI design is required later.

Data Model

- `policy_rate_events`: one mutable schedule/outcome row per FOMC date.
- `policy_rate_expectation_snapshots`: append-only pre-event expected ranges.
- `policy_event_impacts`: append-only 1D/5D raw and SPY-relative returns.

An impact row includes an input fingerprint and its available time. It can be recomputed when more frozen daily-return data exists, but it never overwrites a previous calculation. Raw symbol returns may be recorded without a benchmark; SPY-relative excess stays null until identical frozen benchmark sessions exist.

Operation

`POST /api/agents/policy-events` is a daily server-only job. It upserts the official calendar, reads FRED target ranges, resolves past scheduled outcomes, and derives impacts from existing US daily-return evidence. It does not invoke ResearchAgent or an LLM. The read API powers the Markets card.

Acceptance Criteria

- A public/anonymous client cannot read or write ledger tables.
- An expectation after the decision is rejected; a missing expectation renders

as unavailable, never zero or "hold expected."

- FRED failures preserve existing records and produce no invented decision.
- Event impacts require complete 1D/5D sessions and calculate excess only when both symbol and SPY returns are comparably available.
- No scorer, trader, PaperTrader, PositionMonitor, or broker adapter imports the ledger.

relationship graph - Feature Architecture

Source: features\relationship-graph\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Relationship Graph and Cross-Company Propagation - Feature Architecture

Status: P0 feasibility study RUN AND FAILED. Recommendation: do not build. Design retained for the record.

Last updated: 2026-07-16 - P0 produced a provisional KILL; enhanced reproducibility rerun is required before final sign-off. See P0_COVERAGE_STUDY.md (./P0_COVERAGE_STUDY.md).

Previous revision: 2026-07-15 by Codex after primary-source and Kairos architecture review.

Scope: design only. No migration, provider activation, scoring change, candidate insertion, or order path is authorized.

Update this file when: source contracts, identity resolution, relationship schema, EdgeScout integration, validation gates, or money-path boundaries change.

0. P0 outcome (2026-07-16) - measured, not estimated

The P0 study ?11 authorized was run against live SEC EDGAR over Kairos's real US universe (92 US non-ETF symbols -> 79 domestic 10-K filers). Full report and method: P0_COVERAGE_STUDY.md (./P0_COVERAGE_STUDY.md). Probe: scripts/sec-customer-coverage-probe.mjs.

```
| P0 exit question (?11) | Measured result |
| Named-customer coverage **with a revenue %** | **5 / 79 = 6.3%** (CHD, SWK, AMT, CELH, CRWV - 10 links total) |
| Named, but no usable exposure % | 2 / 79 (QCOM >=10% floor only; BX $-denominated, non-traded funds) |
| Identity resolution on those 10 strings | **60% resolved**, 20% ambiguous, 20% unresolved; a silent false positive (BCRED->BX) appeared immediately |
| Point-in-time `available_at` | **Works.** `acceptanceDateTime` on 79/79, intraday-precise; amendments carry their own timestamps, so as-of is deterministic |
| Freshness | exposure weight is **~6.5 months stale at median**, up to 381 days; refreshes annually |
| Cost | **$0**, 167 requests, ~3 min, ~169 MB/year raw |
```

Answer to ?16 open decision 1 ("Is free disclosed-customer coverage high enough to justify P1?"): No.

The single riskiest assumption named in ?17 - whether Kairos can obtain enough point-in-time, correctly resolved, economically weighted public relationships for its actual US universe - failed on measurement. US GAAP requires the *amount* of a >=10% customer, not the *name*, and issuers overwhelmingly take the anonymity option on exactly the highest-exposure links (NVDA 22%, ENPH 39%, AMAT 19%, MU 17%, INTC 43%; FFIV/FSLR/RGTI literally letter or number their customers). ?5.3's caution was correct and is now evidenced.

Recommendation: do not build P1-P5. n=5 cannot support the ?9.1 cross-sectional experiment or the ?9.2 evaluation battery; it cannot even reach EdgeIC's n0bs >= 12 preliminary diagnostic. Per ?17, retain the peer-move display and stop. Revisit only if the covered universe reaches ~500+ US issuers (the falsifiable precondition derived in the study); re-running the probe then costs three minutes.

1. Review verdict

The product idea is valid: economically linked firms can transmit information slowly, and customer returns have historically predicted supplier returns. The previous V3 document was not build-ready, however. It made five unsafe leaps:

- It stored a bullish/bearish direction on a relationship. A supplier/customer/competitor relationship is not intrinsically bullish or bearish; an event has a sign and a relation determines how that event may transmit.
- It proposed a second provenance and learning system instead of reusing Kairos's Canonical

Evidence Router, evidence_records, edge_* lab, research queue, and decision ledger.

- It treated Finnhub peers as economic links. Similar-company peers are display context, not verified customer, supplier, or competitor relationships.

- It overstated free data coverage. Public filings do not guarantee that every major customer is named, and FinancialDatasets segmented financials describe product/business segments, not a complete customer-supplier graph.

- It elevated two very recent arXiv preprints into a production recommendation. They are useful hypotheses, not sufficient evidence to ship embedding propagation into Kairos.

Correct product decision: do not build a general LLM relationship graph first. Start with a narrow, falsifiable, US-only, measure-only disclosed customer momentum experiment inside the existing Edge/Factor lab. Add graph extraction, event propagation, candidate discovery, or model embeddings only after each earlier layer proves coverage and out-of-sample incremental value.

2. Product purpose

The feature should eventually answer four distinct questions without presenting speculation as fact:

- Relationship: What verified economic relationship exists between two issuers?
- Exposure: How economically important is that relationship, and to which party?
- Event: What point-in-time event or return shock occurred at the leading issuer?
- Measured implication: Has Kairos observed that this relation/event type predicts the linked issuer in the target market and horizon after costs?

The user-facing output must keep those layers separate. A relationship card may say:

Supplier A disclosed Customer B as 18% of revenue in its 10-K filed on date D.

Customer B rose 7% over the last 20 sessions. Kairos is measuring whether similar links predict Supplier A; this is not a recommendation.

It must not say "Supplier A will rise" merely because an LLM extracted a link.

3. Non-negotiable boundaries

- No LLM on the money path. An LLM may propose an entity/link assertion from bounded, sanitized filing text. It cannot set a score, direction, threshold, position size, order, long suppression, or exit.
- No relationship output affects trading in P0-P3. It is display-only or measure-only.
- No direct hedge integration. The governed downside hedge is a portfolio-level US market overlay. A bearish company relationship must never activate or size SH/PSQ.
- No automatic shorting. Negative observations remain research evidence. Kairos remains long-only for new single-name positions.
- No competitor or peer inference from a provider list. symbol_profiles.peers and the shipped peer-move strip remain context-only and outside this graph.
- No parallel provenance store. Provider acquisition uses the Canonical Evidence Router;

decision evidence links to existing evidence and provider ledgers.

- No auto-promotion. Passing an EdgeIC measurement does not alter the Champion, scorer, P(win), sizing, candidate priority, or live eligibility. Any use requires a separately approved, validated Challenger or feature-registry change.
- Per-target-market activation. US and India measurements, validation, configuration, and promotion remain independent. Currency amounts are never cross-summed.
- Fail closed on ambiguity. Unresolved issuer identity, missing source availability time, stale relationships, missing exposure basis, or conflicting assertions produce no signal.

4. Existing Kairos systems to reuse

This feature extends existing architecture; it must not replace it.

```
| Concern | Canonical Kairos owner | Required integration |
| Provider choice, pacing, caching, source provenance | `lib/evidence/*`, `EvidenceEnvelope`, evidence
policy/cache/call ledger | Add relationship/filing intents only through a reviewed Router contract; no
direct provider calls in graph logic |
| Decision evidence | `evidence_records`, append-only `decision_observations` | Store IDs/hasches that
identify the exact inputs known at decision time; never copy a second ungoverned evidence blob |
| Factor measurement | `edge_catalog`, `edge_signals`, `edge_signal_inputs`, `edge_ic_history` | Register
graph-derived measurements as versioned edges and use EdgeScout/EdgeIC lifecycle |
| Candidate backlog | `research_queue` and ResearchAgent discovery attribution | Only a later approved
phase may enqueue a linked issuer; no private queue |
| Outcome labels | `decision_observations` x `observation_labels` | Use for candidate-source evaluation
only after candidate discovery exists; do not invent another outcome table |
| Strategy governance | Feature Registry, Validation Engine, Shadow, Champion/Challenger | Required before
any measured edge can influence a score or eligibility |
| Display-only peers | `symbol_profiles.peers`, peer-move MVP | Remains separate and cannot seed verified
graph edges |
```

The current DiscoverySource union does not include relationship_graph. That is intentional. It is added only when P3 candidate-shadow architecture is separately approved.

5. What the research supports - and does not support

5.1 Peer-reviewed prior: customer momentum

Cohen and Frazzini (2008), **Economic Links and Predictable Returns**, supports a narrow claim: publicly disclosed customer returns predicted subsequent supplier returns in their historical sample. Their customer return was sales-weighted, their portfolios were formed monthly, and the reported drift persisted beyond the initial customer shock.

This supports testing a sales-weighted customer-return factor. It does not prove that:

- every partnership, competitor, or co-mention edge predicts returns;
- an arbitrary 8-K has a mechanically knowable sign for every neighbor;
- a 1-day or 5-day Kairos implementation retains the historical edge after costs;
- the effect works unchanged in today's large-cap universe or in India; or
- an LLM-extracted graph is superior to disclosed customer links.

Primary source: Cohen and Frazzini, Journal of Finance (2008) (<https://pages.stern.nyu.edu/~afrazzin/pdf/Economic%20Links%20and%20Predictable%20Returns%20-%20Cohen%20and%20Frazzini.pdf>).

5.2 Recent preprints: hypothesis grade only

- Supply Chain Propagation of Textual Signals (arXiv:2606.29290) (<https://arxiv.org/abs/2606.29290>)

reports promising results for FinBERT/graph-propagated 10-K features, but it is a June 2026 single-author preprint using 255 S&P 500 firms over 2011-2025. It is not an independently replicated production standard.

- Cross-Stock Predictability via LLM-Augmented Semantic Networks (arXiv:2604.19476) (<https://arxiv.org/abs/2604.19476>)

reports that LLM edge filtering improved a historical long-short result on S&P 500 constituents from 2011-2019. The abstract does not establish point-in-time constituent handling, implementation costs, or suitability for Kairos's long-only books.

Therefore embedding propagation is an offline research candidate, not the P1 implementation. It must prove incremental out-of-sample value over the simple disclosed-customer baseline after costs and multiple-testing correction.

5.3 Disclosure reality

SEC EDGAR is free and official, and current filing metadata/XBRL APIs require no API key. SEC fair-access guidance currently caps automated access at 10 requests/second and requires a declared user agent. That is a ceiling, not a service-level guarantee or an unlimited quota.

Major-customer disclosures are incomplete for graph construction:

- US GAAP can require disclosure that a customer contributes 10% or more of revenue and the amount/segment, but customer identity is not always required.
- Regulation S-K was modernized in 2020 into a more principles-based business-description rule.

Kairos must not assume every 10% customer is named.

- FinancialDatasets `segmented_financials` exposes product/business segment revenue. It is not proof of customer identity or customer-level revenue exposure.

Source contracts are therefore capability-probed and coverage-measured before architecture approval. Missing identity/exposure is unavailable, never guessed.

Primary sources: SEC EDGAR APIs (<https://www.sec.gov/search-filings/edgar-application-programming-interfaces>), SEC developer fair-access guidance (<https://www.sec.gov/about/developer-resources>), and SEC 2020 Regulation S-K modernization (<https://www.sec.gov/rules-regulations/2020/08/modernization-regulation-s-k-items-101-103-105>).

6. Correct conceptual model

6.1 Issuers are not ticker symbols

Graph nodes represent legal issuers, not ticker text. A separate point-in-time mapping connects an issuer to one or more tradable instruments. This prevents symbol changes, share classes, ADRs, cross-listings, delistings, and duplicate economic exposure from corrupting the graph.

Minimum identity keys:

- US issuer: CIK plus normalized legal name; LEI when available.
- India issuer: reviewed exchange/company identifier from an approved NSE/BSE/SEBI source.
- Tradable instrument: (market, exchange, symbol, currency, valid_from, valid_to, is_primary).

An LLM may return a name mention. A deterministic resolver must map it to exactly one issuer and instrument. Zero or multiple matches are quarantined.

6.2 Relationships are neutral assertions

A durable assertion describes an economic fact:

- supplier supplies customer;
- customer buys from supplier;
- parties have a material partnership/JV; or
- an issuer disclosed material dependence on another issuer.

The assertion stores no bullish/bearish direction. It separately records:

- relation type and orientation;
- exposure percentage and whose revenue/cost basis it describes;
- extraction quality and verification state;
- economic-validity interval (valid_from, valid_to); and
- knowledge-time fields (source_published_at, available_at, retrieved_at).

6.3 Events and propagation observations are separate

An event or leading-company return shock is a time-stamped observation. A propagation measurement joins an event/return to relationships that were known and economically valid before the target decision timestamp.

Conceptually:

```
leading input (customer return or validated event)
  x economic exposure
  x relationship verification quality
  x age/validity rule
  = measure-only propagation raw value
```

This is not an analyst score. Cross-sectional normalization and forward-return evaluation remain in the Edge/Factor lab.

7. Proposed data architecture

These tables are proposals only. A future implementation requires an approved migration plan and live-schema verification.

7.1 issuer_entities

Stable legal entities.

```
| Column | Purpose |
| `issuer_id` | UUID primary key |
| `legal_name`, `country` | Reviewed identity |
| `cik`, `lei`, `india_company_id` | Nullable unique external identifiers |
| `identity_status` | `verified`, `ambiguous`, `retired` |
| `created_at` | System time |
```

7.2 issuer_instruments

Bitemporal issuer-to-security map.

```
| Column | Purpose |
| `issuer_id` | FK to issuer |
| `market`, `exchange`, `symbol`, `currency` | Tradable identity; market and currency always explicit |
| `valid_from`, `valid_to` | When the mapping was economically true |
| `available_at`, `retrieved_at` | When Kairos could know it |
| `is_primary` | Prevent duplicate ADR/share-class candidate emission |
```

7.3 relationship_assertions

Append-only facts, not mutable current-state rows.

```
| Column | Purpose |
| `assertion_id` | UUID primary key |
| `subject_issuer_id`, `object_issuer_id` | Oriented issuer pair |
| `relation_type` | Narrow grammatical enum; start with `supplier_to_customer` (subject is supplier, object is customer) |
| `exposure_pct`, `exposure_basis`, `exposure_period_end` | Nullable; never mix revenue and cost exposure |
| `valid_from`, `valid_to` | Economic validity interval |
| `source_published_at`, `available_at`, `retrieved_at` | Point-in-time availability |
| `evidence_record_id`, `router_policy_version_id`, `payload_hash` | Links canonical Kairos provenance; no ad hoc `evidence_ref` blob |
| `source_locator`, `source_span_hash` | Accession/document/section and hash of the cited bounded span |
| `extractor_version`, `schema_version` | Reproducibility |
| `assertion_action` | `assert` or `retract`; an accepted retraction ends a prior assertion without mutating it |
| `verification_status` | `proposed`, `accepted`, `rejected` |
| `supersedes_assertion_id` | Corrections add rows; UPDATE/DELETE remains blocked |
| `created_at` | System time |
```

A deterministic current/as-of view resolves the accepted assert/retract supersession chain. An old row remains historically accepted; the later accepted row determines current state without rewriting history. Raw LLM confidence is not stored as economic truth. Verification confidence is derived from objective checks such as identity uniqueness, source class, citation-span match, and numeric exposure extraction.

7.4 Reuse edge_* tables for measurements

Do not add a propagation_signals truth layer. Register, for example:

```
edge_id: customer_momentum_sales_weighted_v1
input: trailing customer return known at t + accepted customer relationship known at t
raw_value: sum(exposure_weight_j * customer_return_j)
expected_sign: +1
default lookback: 20 trading sessions
evaluation horizons: 5, 10, 20 trading sessions
```

Write values to edge_signals; write relationship assertion IDs, price evidence references, availability timestamps, and adjustment policy to edge_signal_inputs; write evaluation to edge_ic_history. A missing exposure produces unavailable for the sales-weighted edge. An unweighted variant, if tested, is a separate edge ID and therefore a separate statistical trial.

8. Acquisition and extraction pipeline

8.1 Router-owned acquisition

Before build, define and review narrow Router intents such as:

- filings.document
- relationships.disclosed

The Router owns provider policy, cache, pacing, freshness, payload hashes, capability status, and call ledger. SEC should be the initial authoritative US source. FinancialDatasets may be a policy option only after a live capability/entitlement probe proves the required customer-level contract.

8.2 Untrusted-document isolation

Filings and news are untrusted text even when hosted by an official source.

- Strip scripts, active content, hidden markup, and irrelevant exhibits.

- Bound bytes, tokens, nesting, entities, and output tuples.
- The extraction model gets no tools, network, shell, credentials, memory writes, or order APIs.
- Treat instructions inside documents as data, never system instructions.
- Require strict schema validation, an exact source locator, and a supporting span hash.
- Reject unknown relation enums, self-links, unresolved entities, future timestamps, invalid percentages, and output not grounded in the supplied span.
- Store model/prompt/schema versions for reproducibility.

8.3 Verification state machine

```
raw document
-> proposed assertion (untrusted extraction)
-> deterministic identity/source/numeric checks
-> accepted OR rejected
-> later filing may append a superseding assertion
```

Only accepted assertions enter display or EdgeScout. P0 should include owner review because the covered universe is small and the cost of a false link is high. Automation can be reconsidered after precision is measured against a labeled fixture set.

9. Statistical and trading correctness

9.1 Pre-register the first experiment

The first trial is only customer_momentum_sales_weighted_v1:

- US primary common equities only;
- accepted disclosed customer relationships only;
- relationship and instrument mapping available before the as-of timestamp;
- 20-session customer return, evaluated at 5/10/20-session supplier horizons;
- benchmark-neutral and raw target returns both retained;
- split/dividend-adjusted prices with explicit availability and adjustment policy;
- no parameter search selecting the best lookback after seeing results.

Other relation types, unweighted links, event semantics, embeddings, neutralization schemes, and horizons are separate registered trials. They increase the trial-family count used by DSR/PBO/FDR governance; they are not silent variants of one factor.

9.2 Long-only decision metric

Cohen-Frazzini and both recent preprints emphasize long-short portfolios. Kairos is long-only for new names, so a long-short Sharpe cannot authorize adoption. Required evaluation includes:

- top-bucket long-only benchmark-relative return after modeled costs;
- hit rate, drawdown, turnover, capacity, and sector concentration;
- rank IC/ICIR with overlapping-horizon correction;
- stability across time, sectors, issuer size, and relationship age;
- comparison with ordinary momentum to prove incremental value; and

- explicit unavailable/coverage rates so sparse disclosure is not mistaken for selectivity.

The existing EdgeIC nobs ≥ 12 classifier is a preliminary measure-only diagnostic, not a production promotion gate for this new sparse/data-mined family. Promotion must use the canonical Edge/Factor plus Advanced Learning governance and cannot be weaker than its multiple-testing, walk-forward, sample-floor, cost, and owner-approval rules.

9.3 No immediate candidate or score effect

Even a promising measurement remains in edge_*. To affect ResearchAgent later, a separate design must choose exactly one governed route:

- Discovery route: enqueue a linked issuer for normal deterministic scoring, with no score

bonus; or

- Feature route: log a normalized relationship feature in the point-in-time decision snapshot,

then promote it only through Feature Registry + Challenger validation.

It must not do both in one experiment, because selection and score effects would become inseparable. Long-candidate suppression is also a money-path change and requires the same gates.

10. Market and session isolation

Initial scope

- P0-P2 are US-only. India is not_applicable, not silently empty.
- India requires a separately validated official filing/entity source and its own coverage study.
- Yahoo/Finnhub peer lists cannot substitute for Indian economic relationships.

Cross-market relationships

The graph may eventually represent a real US-customer/India-supplier relationship, but it does not cross-sum books or currencies. The propagation observation belongs to the target instrument's market, currency, mandate, benchmark, calendar, and owner controls.

Cross-market evaluation must prove available_at $\leq$ target_decision_ts using both exchange calendars and time zones. An event released after the target market closed cannot be treated as known for that session. ADR and local shares are one issuer with multiple instruments; candidate deduplication selects the configured primary instrument per market.

11. Phased build order and rollback

P0 - source and identity feasibility, display fixtures only

- Live-probe SEC and any optional provider contract on 25-50 representative US issuers.
- Measure named-customer coverage, exposure coverage, entity-match precision, and conflicting disclosure rate.
- Build a reviewed fixture set with positive, absent, ambiguous, superseded, and prompt-injection cases.
- Exit criterion: documented coverage plus high-precision entity/assertion verification. If the free data is too sparse, stop. Sparse is an acceptable result.

No graph table, candidate, score, or LLM cron is required merely to run the feasibility probe.

P1 - accepted relationship ledger and read-only display

Add issuer/instrument/assertion storage behind `relationship_graph_enabled=false`. Display only accepted disclosed customer links with source, exposure basis, age, and "not a recommendation". No competitor seeding and no propagation.

P2 - customer momentum in the existing Edge lab

Compute `customer_momentum_sales_weighted_v1` into `edge_*`, measure-only. The Edge dashboard shows coverage and caveats. It cannot write `agent_signals`, `research_queue`, `decision_observations`, paper positions, proposals, or orders.

P3 - candidate shadow, only after P2 evidence

If P2 passes its pre-registered tests, design a separate candidate-discovery shadow. It records which target names would have been added, but does not add them to ResearchAgent. Compare their normal deterministic outcomes with the existing universe.

P4 - separately approved paper integration

Only after P3 prospective evidence and owner approval may one route from Section 9.3 become a validated Challenger. Start US paper-only and OFF. Live remains a later, separate decision.

P5 - optional embedding/LLM network experiment

Run only as an isolated offline experiment after the simple baseline exists. It must beat the baseline out of sample after costs and trial correction. Model artifacts are pinned and hashed; provider/model updates create a new trial, never silently replace the old one.

Disable/rollback

Each phase has an independent per-market flag. Disabling stops new acquisition, extraction, measurement, or candidate shadow while preserving immutable evidence and results. A rollback never deletes or rewrites history.

12. Security, operational, and cost controls

- RLS on every new public table; no anon access; owner read only where UI needs it; service-role writes; fixed `search_path` and service-role-only grants for any SECURITY DEFINER RPC.
- Append-only assertion and evaluation ledgers reject UPDATE/DELETE. Corrections append a superseding row.
- Bounded extraction batch, wall-clock limit, per-document size cap, retry cap, dead-letter state, and health alert aggregation.
- SEC requests use a declared contact user agent, shared pacing, conditional caching, and the Router ledger. Stay materially below the SEC's maximum access rate.
- No source is described as unlimited. Unknown quota/entitlement remains unknown.
- No raw filing body, model prompt, provider error, URL query credential, token, or service key is

written to logs.

- Data retention distinguishes small immutable source-span hashes from large cached filing bodies; large bodies receive an explicit TTL and storage budget.
- Provider, parser, prompt, model, schema, and policy versions are pinned in every extraction run.

13. Product UX requirements

The eventual UI is evidence-first and novice-readable:

- Verified relationship: issuer names, relation orientation, exposure and basis if known.
- Source and age: filing type, accession/link, filed date, relationship validity.
- What happened: leading return/event and the exact as-of time.
- System posture: display only, measuring, shadow candidate, or later governed state.
- Confidence honesty: use verified, ambiguous, stale, conflicting, or unavailable; never render an LLM's self-assigned confidence percentage.
- No recommendation language before an approved integration phase.

The UI must not collapse "peer", "supplier", "customer", "partner", and "competitor" into one generic related-stock list.

14. Required tests for any future build

Identity and temporal correctness

- Symbol rename/share-class/ADR mappings resolve to the correct issuer as of date.
- Ambiguous company names and private/unlisted entities quarantine rather than guess.
- Assertions unavailable at the target decision time cannot enter a signal.
- Superseded relationships remain historically queryable but disappear from later as-of views.
- Cross-market session/time-zone tests prevent look-ahead.

Extraction and security

- Prompt instructions embedded in filing text cannot alter schema or invoke tools.
- Output without an exact source span, valid locator, and unique entity match is rejected.
- Percentages above 100, mixed exposure bases, self-links, unknown enums, and future dates reject.
- Parser/model/version changes create distinct artifacts and do not rewrite assertions.

Statistical honesty

- Sales-weighted formula matches a fixed fixture exactly.
- Missing exposure yields unavailable rather than zero/equal-weight substitution.
- Each alternate parameter/model is counted as a distinct trial.
- Point-in-time universe, price adjustment, costs, and benchmark are reproducible.
- Long-only results are reported alongside IC/long-short diagnostics.

- Ordinary momentum comparison identifies whether the graph adds incremental information.

Boundary tests

- P0-P2 imports cannot reach agent_signals, research_queue, PaperTrader, PositionMonitor, TraderAgent, broker adapters, hedge controller, or execution gateway.
- Relationship flags default OFF independently by market and phase.
- A disabled or unavailable provider produces no fabricated relationship or signal.
- No graph value changes scoring, entry eligibility, P(win), sizing, exit, or orders without a separately approved and validated strategy version.

15. Acceptance criteria for architecture approval

Before implementation is approved, the owner/reviewer should require:

- P0 coverage evidence from live free sources, not provider marketing or memory.
- A canonical Router intent/adaptor contract and point-in-time provenance mapping.
- A reviewed issuer/entity-resolution contract.
- A fixed initial edge specification and trial-family registration.
- Explicit proof that the build extends edge_* and existing evidence ledgers.
- Separate US and India applicability; India may remain unsupported.
- A no-money-path import test and flags OFF by default.
- A cost/storage/cadence estimate that fits current free-cloud budgets.

16. Open decisions

- ~Is free disclosed-customer coverage high enough to justify P1?~ ANSWERED 2026-07-16: No - 5/79 filers (6.3%) yield a named customer with a revenue share. See ?0 and P0_COVERAGE_STUDY .md. Decisions 2-6 below are now moot unless the revisit precondition (~500+ covered US issuers) is met.
- Should accepted assertions require owner review indefinitely, or only until fixture precision clears a pre-registered threshold?
- Should the first later integration be candidate discovery or a logged feature? Do not combine both.
- What minimum prospective calendar span and independent cross-sections are required beyond the current preliminary EdgeIC classifier before P3?
- Which official India filing/entity source can provide a lawful, stable, point-in-time contract?
- Where should bounded extraction run? Prefer the existing trusted research cron for reviewed filing extraction; use an isolated worker only if untrusted code/models are introduced.

17. Recommended decision now

Approve only P0 feasibility research, not implementation of the graph. Do not prioritize this ahead of the Canonical Evidence Router cutover and first trustworthy learning-loop run. If P0 later shows sparse named-customer coverage, retain the peer-move display and stop; Kairos should not build a large graph whose apparent precision comes from guessed relationships.

The single riskiest assumption is not whether customer momentum once existed. It is whether Kairos can obtain enough point-in-time, correctly resolved, economically weighted public relationships for its actual US universe to measure that anomaly honestly and cheaply today.

research journal - Feature Architecture

Source: features\research-journal\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature: Research Journal (daily funnel report + learning evolution)

Status: BUILT (v1 shipped 2026-07-06; v2 novice-first upgrade approved 2026-07-12) ? Owner: Vaibhav ? Started 2026-07-06

Motivation

Today, "why did the agent do/not do X" requires manually querying 4+ tables (decision_observations, trade_proposals, paper_trades, learner_runs) and cross-referencing by hand - no single place shows the full chain for a symbol on a given day, and no single place shows whether the learning loop is actually improving over weeks/months. app/dashboard/agents/history exists but never reads decision_observations - it's unrelated to this.

The ask: a daily report showing, per symbol scored that day - which screener bucket/criteria flagged it, its full score breakdown, why it passed or failed at each pipeline stage (research threshold -> portfolio constructor -> trade execution), and a separate longer-horizon view of how the screener, LearnerAgent, and feature registry are evolving.

What already exists (verified against live schema, 2026-07-06)

decision_observations (Phase 1 ledger) already logs, per symbol per research run: analyst_score, score_threshold, entry_eligible, sub-scores (fundamental/technical/sentiment/macro/insider), and a features jsonb blob that already contains honest per-dimension notes (e.g. "no macro data", "ETF - no company fundamentals"). This is real, live data - verified via a sample query (IAU/GDX/SLV, 2026-07-06: score 52 < threshold 60 -> correctly rejected, with the exact reason visible in features).

Gaps (why this can't be built as a pure read-only aggregation today)

- No screener-stage tag. decision_observations.features has no bucket (momentum|value) or criteria_matched field. ResearchAgent's dual-bucket scoring runs today but doesn't record which bucket/rule flagged a candidate.

- No per-symbol downstream trail. Once a candidate clears the research threshold, Portfolio Constructor's shrink/reject decision and the trade-proposal/fill outcome exist in separate tables with no shared key back to the originating decision_observations row beyond signal_id (which exists on decision_observations but isn't written-to / consistently joined by the downstream stages today).

- No feature-registry status-history log. feature_registry has a current status (proposed/quarantined/active/retired) but no history of *when* it changed - the Evolution tab needs a timeline, not a snapshot.

Proposed architecture

Principle: instrument at the source, don't reconstruct after the fact

Reconstructing "why" from final-state tables is lossy (a proposal that got rejected by Portfolio Constructor before ever becoming a trade_proposals row leaves no trace today). Instead, write one small journal row at each stage transition, all keyed by signal_id, then join for display.

Schema changes

A. `decision_observations.features` - add two fields at write time (`lib/research-agent.ts`, no migration needed - same jsonb column):

```
features.screener = { bucket: "momentum" | "value", criteria_matched: string[] }
```

Populated from the existing dual-bucket scoring logic in `lib/research-agent.ts` (the bucket assignment already happens in code - this just records which one won and why, instead of discarding it).

B. New table `pipeline_stage_events` (append-only, mirrors `decision_journal`'s generic shape but scoped to this funnel so queries stay cheap and don't compete with the general journal):

```
create table pipeline_stage_events (  
  id bigserial primary key,  
  signal_id uuid not null,  
  symbol text not null,  
  market text not null default 'us',  
  stage text not null, -- 'research' | 'portfolio_constructor' | 'proposal' | 'execution'  
  outcome text not null, -- 'passed' | 'rejected' | 'shrunk' | 'filled' | 'expired'  
  reason text, -- human-readable, e.g. "sector cap: 3/3 Tech positions already held"  
  detail jsonb, -- stage-specific numbers (e.g. shrink %, correlation haircut applied)  
  created_at timestamptz not null default now()  
);
```

Written by: `lib/research-agent.ts` (stage='research'), the Portfolio Constructor call inside `app/api/agents/paper-trade/route.ts` (stage='portfolio_constructor'), the trade-proposal/approval flow (stage='proposal'), and `execute_paper_fill/broker-order` paths (stage='execution'). Each write is one extra insert at a point where the decision is already being made - no new decision logic, just recording the existing one.

C. `feature_registry_history` (small, append-only): `id`, `feature_id`, `from_status`, `to_status`, `reason`, `created_at` - written wherever `feature_registry.status` is currently updated (feature-check route, learner's `propose_feature` tool).

API

- GET `/api/agents/research-journal?date=YYYY-MM-DD&market=us` - for each

symbol with a `decision_observations` row that day, join `pipeline_stage_events` (by `signal_id`) into an ordered stage list, plus the terminal state (rejected-at-research / rejected-at-constructor / proposed-pending / filled / expired).

- GET `/api/agents/research-journal/evolution?market=us&days=90` - time

series: LearnerAgent weight deltas (`learner_runs`), feature-registry status transitions (`feature_registry_history`), calibration drift (`model_artifacts`), shadow-decision agreement rate (`shadow_decisions` vs realized champion decisions).

UI - new page /dashboard/research-journal

Two tabs, matching the existing dark-theme card/pill conventions:

- Daily Funnel - date picker (default today), per-symbol expandable

rows: score breakdown -> screener bucket/criteria -> stage-by-stage pass/reject with reason -> terminal badge. Empty state: "No candidates scored yet - research runs at 9 AM ET" (mirrors existing empty-state language elsewhere in the app).

- Evolution - line/step charts: weight deltas over time per dimension,

feature-registry promotion/retirement timeline, calibration Brier-score trend, shadow-decision agreement rate. Each metric card states plainly when there isn't enough history yet (no false precision on thin data - matches the Goal tracker's feasibility-sentence convention).

Docs required in the same change (per CLAUDE.md)

- public/agent-diagrams/system-map.json - this doesn't add a new agent-to-agent data flow, but it DOES add new observability into existing flows (research -> constructor -> proposal -> execution). Add a note to each touched node's description (RESEARCH, TRADER/PROMOTE area) rather than a new node, plus a history entry.
- PROJECT_DECISIONS.md - one entry once approved and built.
- WORK_LOG.md - session entry.

Non-goals (v1)

- Real-time streaming of the funnel as it happens (this is a post-hoc daily report, not a live pipeline visualizer).
- Auto-alerting on funnel anomalies (e.g. "0 signals passed today") - that's already partially covered by the existing stale-run / 0-signal alert in research/cron; this feature is for understanding **why**, not re-alerting.
- Multi-market side-by-side view in v1 - one market at a time. India is now supported through the same market-scoped picker and local-calendar rules.

Acceptance

- pipeline_stage_events inserts are additive and fail-soft (a logging failure must never block or alter the actual trading decision it's describing - same convention as decision_journal writes elsewhere).
- Daily Funnel renders correctly with zero events (first day after ship).
- Evolution tab is honest about thin history ("not enough learner runs yet to show a trend") rather than drawing a misleading chart from 1-2 points.
- tsc + build pass; migrations handed over as clickable links + full paths.

V2 - novice-first evidence and context (approved 2026-07-12)

Product question

The journal must answer, in order: what is this security; why did it appear; what changed; what does Kairos currently think; how reliable is that view; what could invalidate it; and what happens next? Exact scores and raw indicators remain available, but are the audit layer rather than the headline.

Three-layer information architecture

- Decision summary (default/collapsed): asset identity/type, discovery state (new, recurring, holding, re-entry when recorded), action (research, watch, wait, paper candidate, hold, exit review, avoid), score, evidence coverage, decision confidence, why-now, strongest positive, largest risk, and next step.
- Evidence and context (expanded): grounded thesis, technical translation,

material catalysts/news context when stored, theme/peer context when supported, counter-evidence, invalidation conditions, history/score change, and source-safe links for independent verification.

- Quant audit (expanded): dimension values, structural/applied weights, point contributions, availability/freshness, raw evidence, and pipeline events.

Truth and safety invariants

- Keep available-evidence score, structural evidence coverage, and decision confidence separate. A 98 computed from two dimensions is not displayed as five-dimension/high-confidence evidence.

- Missing/inapplicable dimensions display Not used, never a neutral-looking numeric score. They contribute zero points and lower coverage only when the dimension is structurally applicable.

- The LLM may summarize already-recorded evidence. It cannot invent prices, financials, news, peers, confidence, action, targets, or invalidation levels. Future observations may show the deterministic mandate-based indicative plan defined in `features/research-trade-plan/FEATURE_ARCHITECTURE.md`; it is explicitly not LLM-authored and is re-priced from the actual fill downstream.

- News/theme/social context does not authorize a trade. It may explain, confirm, reduce confidence, or veto through a separately validated deterministic rule.

- No per-card provider fan-out on initial page load. The immutable stored decision renders first. Optional current enrichment is on-demand, cached, bounded, and explicitly newer than the historical decision.

- Point-in-time history is never rewritten. V2-derived presentation fields are computed at read time or written additively for future observations.

Asset-aware context

- Company: business/sector, growth, profitability, cash flow, leverage, valuation, estimates/earnings when available.

- ETF/fund: exposure, holdings/concentration, liquidity, expense/flows, geography/currency hedge and structural risks. Company fundamentals and insider activity are inapplicable.

- India company: NSE symbol normalization and NSE/BSE/company-announcement links; India-specific provider freshness.

- Asset type comes from recorded `agent_signals.asset_class` plus the canonical symbol classifier. Never infer ETF/company status from missing fundamentals.

History and novelty

For all symbols in the selected daily run, one bounded history query computes first-seen date, observation count, previous score/direction and change. Existing holdings are always labeled `holding`; otherwise first observation is new and later observations are recurring. re-entry requires a recorded re-entry event and is never guessed from elapsed time.

Technical translation

Translate deterministic evidence into novice language: trend, momentum, moving- average posture, volume confirmation and extension/breakdown risk. RSI/MACD are diagnostics, not independent votes; correlated indicators must not be double-counted. Never call RSI near 50 "strong" or low volume "confirmation."

External validation links

Build links deterministically from validated symbols: TradingView, Yahoo Finance, SEC/company research for US, NSE/BSE for India, and issuer pages only when a verified issuer URL exists. Links are labeled external and are never scraped as application data or treated as scoring evidence.

V2 phased delivery

- V2.0 (this build): decision/coverage/confidence separation, asset type, novelty/history, novice action/technical translation, missing-dimension honesty, score change, and deterministic TradingView/Yahoo/SEC/NSE/BSE links. Uses only stored data; no scoring or order-path change.
- V2.1 (built 2026-07-12): on-demand cached provider-backed news panel with source, publication/fetch timestamps, title deduplication, safe HTTPS links and an explicit warning that current context post-dates the historical decision. India fails honestly until a validated free NSE-news adapter is available.
- V2.2: point-in-time peer/theme comparison after comparable-group and provider coverage validation. Measure-only until incremental value is proven.

V2 acceptance

- A novice can identify the security state, current action, confidence, strongest evidence, largest gap and next step without expanding raw score construction.
- ETF examples (including EUAD) never display company-fundamental/insider evidence as applicable; India links and symbols resolve correctly.
- A high score with 2/5 applicable dimensions visibly reports limited coverage and cannot be labeled high confidence.
- Historical and on-demand current context are visually/time separated.
- API reads are bounded; partial context failure does not hide the stored decision.
- Current-news enrichment is owner-gated, opt-in per symbol, cache/budget guarded, deduplicated and never changes the recorded score/action.
- TypeScript, unit tests, production build, and US/India browser checks pass.

research journal controls - Feature Architecture

Source: features\research-journal-controls\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature: Research Journal Controls and Market-Local Clocks

Status: SHIPPED Owner: Vaibhav Date: 2026-07-21

Intent

- Score Tracker must support clearing all selected symbols.
- Its existing point filters must also identify matching symbols across the bounded candidate universe, with explicit commands to select or add those matches to the chart.
- Daily Funnel must support one-date and all-date views plus a symbol filter.
- The persistent US and India status pills must show each market's current local time in addition to the existing session state and open/close schedule.

Contracts

Score Tracker

- Candidate symbols remain the current market's watchlist/holdings/custom set.
- Candidate discovery and persisted chart selections are market-scoped. The US live-portfolio endpoint is never used to populate India's picker.
- Filter matching queries `signal_score_history` through the existing owner-only endpoint, in chunks of at most 50 symbols. It does not call a provider.
- A symbol matches when at least one stored score point satisfies the current period, score-band, direction, source, and date filters.
- Active filters narrow the displayed symbol chips. They never silently change chart selection. `Select filtered` replaces selection, `Add filtered` unions matches into selection, and `Clear selection` removes every chart line.
- The global market switcher remains the sole US/India authority.

Daily Funnel

- Default remains one market-local date and the latest decision per symbol for that date.
- `All dates` returns at most the 250 newest immutable observations for the selected market, or for one validated symbol when filtered. The response marks truncation honestly; it never claims unbounded completeness.
- Symbol filtering is server-side and exact after trim/uppercase normalization.
- Each all-date card is keyed by observation ID and shows its market-local date,

so repeated observations for one symbol remain distinct and expandable.

- Existing stage, paper/live position, outcome, evidence, and terminal-state joins remain display-only.

Market Header

- US displays current ET; India displays current IST.
- Existing status label and schedule detail remain visible.
- Time updates every 30 seconds and uses IANA time zones; no fixed UTC offsets.

Non-Goals

- No score recalculation, signal selection, provider call, order, portfolio, or risk behavior change.
- No cross-market chart or journal result.
- No unbounded historical read or infinite-scroll build in this phase.
- No new database table, migration, package, or scheduled job.

Acceptance

- Clear selection produces an empty chart state.
- Filter matching is computed over candidates that were not previously selected.
- Select/add filtered affects only the current market.
- All-date Daily Funnel can show multiple observations for one symbol and states the 250-row bound.
- Desktop and 390px mobile layouts contain all controls without overlap.

research trade plan - Feature Architecture

Source: features\research-trade-plan\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature: Research-Time Indicative Trade Plan

Status: APPROVED / BUILDING (2026-07-20) ? Owner: Vaibhav ? Builder: Codex

Problem

Research Journal explains why a symbol passed or failed, but does not show the native-currency price used by research or translate the active mandate into an approximate risk/return plan. Downstream execution already creates absolute stops and targets at fill time, so adding independent LLM-generated prices would create two conflicting truths on the money path.

Decision

Record a deterministic, point-in-time indicative plan from the candle already used by ResearchAgent. Show it in Research Journal as:

- research reference price and market date/source;
- for an eligible new long: approximate initial risk floor, profit objective, percentages, and target horizon;
- for a rejected/neutral symbol: reference price plus No entry plan;
- for a held symbol: reference price plus `PositionMonitor owns the executable exit plan` so a newly computed scenario cannot overwrite the position's fill-bound stop/target.

This is not a predicted fair value, limit order, broker stop, guaranteed exit, or new score dimension. FINRA warns that a stop price is not a guaranteed execution price and can execute materially differently in volatile markets; the product must not label an indicative level as an execution promise.

Contracts

Shared deterministic module

lib/trading/trade-plan.ts owns two pure contracts:

- buildIndicativeTradePlan: binds the latest validated research candle to the current per-market mandate. It performs no fetch and no LLM call.
- resolveExecutionRiskReward: selects fill-time stop/target percentages. A

valid ledger MAE/MFE distribution with at least 60 eligible-long outcomes for the same market/horizon uses the approved clamps (maximum 10% MAE stop and 40% MFE target); malformed/thin learned data falls back to the current mandate.

Both contracts reject non-finite/non-positive prices and preserve native market currency (USD for US, INR for India).

Persistence

No migration is required. Use the existing immutable ledger:

- decision_observations.price_at_decision: latest candle close used by scoring;

- `decision_observations.currency`: existing native-currency field;
- `decision_observations.features.trade_plan`: versioned indicative plan;
- `agent_signals.stop_loss_pct / take_profit_pct`: research-time mandate

snapshot for downstream audit/comparison, never an executable absolute price.

Historical null rows remain unchanged. The Journal must report them as unavailable rather than backfilling with current prices.

Agent Ownership

- ResearchAgent: records the point-in-time reference and indicative plan.
- PaperTrader: fetches a fresh market-local quote, re-resolves current

mandate/validated MAE-MFE policy, anchors absolute levels to the actual fill, and logs planned-versus-bound values. Research prices never authorize a fill.

- PositionMonitor: remains the sole owner of paper stop, target, trailing, score-exit, and time-exit decisions from `paper_positions`. Research cannot loosen or replace those levels.

- Live Trader/broker adapters: unchanged. Existing quote freshness, approval, pause, kill-switch, account, and order gates remain authoritative.

- LearnerAgent: may evaluate eventual MAE/MFE outcomes through the existing observation ledger. It cannot invent or directly mutate plan levels.

Safety Invariants

- No LLM-generated number enters the plan or money path.
- US and India plans never share prices, currencies, mandates, or learned samples.
- A missing/stale/non-positive research price produces `unavailable`, never a fabricated level.
- Research levels cannot bypass signal/session freshness, portfolio, cash, pause, kill-switch, approval, broker, or execution-price gates.
- Fill-time levels are anchored to the actual fill, not the research close.
- Held-position exits always use the stored position plan; the Journal labels research levels as context only.
- Stop/target prices are estimates, not guaranteed executions.

UI

Add an unframed Indicative plan row in each Research Journal symbol detail:

- Reference, Initial risk floor, Profit objective, Horizon;
- market-native currency formatting;
- source/as-of metadata;
- concise disclosure: Repriced at fill; not an order or guaranteed execution.

Do not show red/green prices for rejected/neutral symbols as though a trade is planned. Do not add controls that place orders from the Journal.

Acceptance Criteria

- A new US and India observation stores the actual latest scoring-candle close, native currency, source/date, and a versioned plan without extra provider calls.
- Eligible new longs show approximate entry/risk/target; rejected and held names show truthful non-entry/position-owned states.
- PaperTrader uses valid ledger percentiles when available, applies documented bounds, falls back on malformed/thin data, and anchors levels to fill price.
- Journal API/UI render old null-price rows honestly.
- Focused unit tests cover invalid values, both currencies, held/rejected states, learned-policy bounds/fallback, and fill anchoring.
- TypeScript, full unit suite, production build, and diff checks pass.

Reversal

Remove the Journal rendering and stop writing `features.trade_plan`. Existing JSONB observations remain harmless audit history. The fill-time resolver can be returned to mandate-only behavior independently; no position or order migration is required.

V2: Position And Outcome Truth (Approved 2026-07-20)

The Journal must not make the research-time scenario look like the currently managed trade. It adds three independent, explicitly labelled overlays:

- Current paper position: read from `paper_positions` for the same market and symbol. Show actual average cost, current price, stored initial/current stop, target, quantity, and update time. These stored fields remain owned by PaperTrader and PositionMonitor.

- Current live holding: read only from the latest

`live_account_snapshots` row for the configured active account. Never make a broker request from the Journal and never merge accounts or markets. A live holding is managed only when a filled Kairos BUY proposal for that exact active account carries a valid deterministic execution policy; otherwise it is truthfully labelled `unmanaged_by_kairos` and no stop/target is invented.

- Realized paper outcome: join `paper_trades` by the exact `signal_id` and show fill, exit, P&L, close time, and exit reason. Symbol-only matching is forbidden because it can attach a later trade to an older decision.

The API also reports exit-policy learning readiness as eligible same-market, same-horizon long labels `n / 60`. Thin data continues to use the mandate. This is observability only and cannot lower the 60-sample gate.

Dormant Live-Exit Contract

Before live autonomy can ever be enabled, every autonomous BUY proposal records the deterministic fill-time percentage policy, horizon, mandate version, and policy source in `trade_proposals.policy_snapshot`. Absolute levels are rebound to the broker's actual average fill.

The dormant live exit monitor must:

- include only filled orders whose `proposal_id` resolves to the configured active account; missing proposal/account lineage fails closed;
- reconstruct remaining lots FIFO so partial sells remove quantity and cost basis together;
- use the remaining lots' recorded policies, with the current per-market mandate only as a clearly marked fallback for legacy Kairos lots;
- use the fill-recorded resolved horizon and count market sessions exactly as Paper PositionMonitor does; legacy Kairos lots fall back to the mandate;
- never auto-manage broker holdings that predate Kairos order lineage;
- surface database read failures rather than silently treating them as no positions.

These changes do not activate `AUTONOMOUS_LIVE_ENABLED`, `live_auto_enabled`, broker-hosted protection, Webull trading, or any order canary. Owner approval, account capability proof, and a small manually approved canary remain separate deployment gates.

research universe fix - Feature Architecture

Source: features\research-universe-fix\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Research Universe Fix - Feature Architecture

Status: APPROVED -> IMPLEMENTING Date: 2026-07-09 Author: Claude / Opus 4.8

Problem

US paper fills = 0 individual stocks. Root cause confirmed (see P0_BUILD_RESULT.md in edge-factor-discovery):

gatherSymbols() in lib/research-agent.ts inserts all symbols into a single Map, with holdings first. slice(0, RESEARCH_MAX_SYMBOLS=10) takes the first 10 = exactly the 10 Robinhood ETF holdings (DXJ, FEZ, GLD, IBIT, IVV, SGOV, SPMO, VONG, VTV, XAR). Watchlist stocks appear at positions 11+; screener stocks at positions 11-13. All silently evicted by the cap. Metals and region ETFs survive only because they're appended after the cap, not before.

Evidence: 4 days of agent_signals decompose exactly as 10 holdings + 3 metals + 3 region ETFs = 16. Zero individual stocks ever enter the research batch, so zero individual stocks ever get signals, so fills stay 0.

Root Cause (single line)

```
const nonMetals = Array.from(result.values()).slice(0, cap); // cap=10 = all holdings
```

Holdings fill the entire cap before watchlist or screener stocks even enter the slice.

Fix

Separate the holdings-monitor budget from the new-buy candidate budget.

Holdings are already owned. They need to be scored always (SELL signals, exit timing, position monitor). They should NEVER compete with or consume the budget for new-buy candidates.

New-buy candidates (watchlist stocks from Theme Scout + manual adds + screener stocks) need their own budget cap.

Changed invariants

Before	After
One shared cap (`RESEARCH_MAX_SYMBOLS=10`) applies to holdings+watchlist+screener together	Holdings uncapped (always all). Candidates use separate `RESEARCH_CANDIDATE_CAP`
Watchlist/screener stocks evicted when holdings fill cap	Watchlist stocks first, then screener stocks, up to `RESEARCH_CANDIDATE_CAP`
Screener max 3 stocks	Screener max `RESEARCH_SCREENER_MAX` (default 6)

Preserved invariants (no changes)

- isHeld: true on holdings -> LLM prompt includes heldNote -> SELL signals possible
- isHeld: false on new candidates -> long-only new-position enforcement in PaperTrader
- Metals basket appended after cap (unchanged)
- Region ETFs appended after cap (unchanged)
- India holdings/candidates handled independently (unchanged)

- No schema changes, no migrations

Code Changes

One file: `lib/research-agent.ts`

Replace the `result = new Map()` block (lines ~419-440) with two separate structures:

```
holdingEntries[] - uncapped, always all holdings
candidateMap     - watchlist first, then screener, capped at RESEARCH_CANDIDATE_CAP
nonMetals        - [...holdingEntries, ...candidateMap.slice(0, candidateCap)]
```

Environment variables

Var	Old	New default	Notes
<code>`RESEARCH_MAX_SYMBOLS`</code>	10	deprecated	No longer read; remove from Vercel env to clean up
<code>`RESEARCH_CANDIDATE_CAP`</code>	-	10	Max new-buy candidates (watchlist+screener) per run
<code>`RESEARCH_SCREENER_MAX`</code>	-	6	Max screener-sourced stocks in candidates (raised from 3)

Expected batch after fix (US)

- Holdings: 10 ETFs (always)
- Candidates: up to 10 (watchlist stocks first, then screener fill up to 6 of the 10 slots)
- Metals: 4 (unchanged)
- India region ETFs: 3 (unchanged)
- Total: ~27 max (vs. 16 previously, which were 100% ETFs)

Safety Analysis

- Holdings still scored -> SELL signals preserved
- `isHeld` flag correct per entry -> PaperTrader long-only gate unaffected
- No change to scoring logic, sizing, order path, live trading
- No DB schema change
- No migration
- Additive: batch gets larger (more API calls), but within existing rate limits and budgets

Second Potential Blocker (to verify post-deploy)

`runScreener()` calls `FinancialDatasets screen_stocks`. FD key confirmed in vault. But screener output needs to be non-empty for any candidate stocks to appear. After fixing the cap eviction, verify via `agent_signals` that symbols with `source='screener_momentum'` or `source='screener_value'` appear. If screener returns 0 results, that's a separate issue (screener API health check needed).

Diagram update

System Map and `SYSTEM_OVERVIEW` do not need updating for this fix - the agent-to-agent flow is unchanged; only the internal symbol prioritization within `ResearchAgent` changes.

risk research visibility - Feature Architecture

Source: `features\risk-research-visibility\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Risk Analytics - Research Visibility

Status: *APPROVED* - built. Owner approved 2026-07-17 with ?10 decided.

Author: Claude / Opus 4.8. Date: 2026-07-17.

Supersedes the approach in `features/risk-research-integration/FEATURE_ARCHITECTURE.md`

(branch `worktree-agent-abaf0b16ef6af4175`, *unmerged*) - see ?9.

Implementation: `lib/research/risk-annotation.ts` (*pure*), the research block on

`GET /api/portfolio/risk-daily`, `PortfolioRiskPage.tsx`, and the

`?symbol=&market=` deep link on the research journal. Tests: `tests/risk-research-annotation.test.ts`.

1. What this is

Show, per holding on Risk Analytics: the research score, its direction, and how old it is - with a click through to that symbol's research-journal entry. All books: US + India, paper + live.

2. What this is NOT - the load-bearing constraint

Research does NOT enter the risk computation. Not the score, not the posture, not the action, not the trim allocation. This is a display join, nothing more.

Why, restated so a future reader does not "improve" it:

- `lib/risk/holding-risk.ts` has zero research coupling by design. `sba-v1` (the water-fill sector-breach allocator, shipped 2026-07-17) is deterministic and research-free.
- A sector is over-cap **because** research liked that sector. Letting `analyst_score` also veto the cap double-counts the same signal. The risk layer exists to be the one thing not persuaded by the score.
- The owner integrates the two views. That is where judgment belongs.

Invariant R1: no field introduced here may be read by `computeHoldingRisk`, `sba-v1`, `constructPortfolio`, the execution kernel, or any gate. Enforced by test (?7).

3. Why the age matters more than the score

This is the actual feature. On 2026-07-16, AVGO was 6 days unscored while Risk Analytics told the owner to trim it - and nothing on screen said so. A score with no age is how that hid. A 6-day-old 91 and a fresh 91 are not the same claim; today's failure mode was treating them as identical.

Per `CLAUDE.md` "Detail Over Cryptic": the cell must say what/why/next, never a bare number.

4. Staleness - measured in SESSIONS, displayed in DAYS

Do not measure staleness in calendar days. Research runs per market on weekdays. A Friday score read on Monday is 3 calendar days old but 1 session old - measuring in days would paint the whole book stale every Monday and train the owner to ignore the warning.

- Measure: market-local *sessions* elapsed since scored_at (US and India have different calendars/holidays - market_controls / the existing market-local session-date helper is authoritative; do NOT reimplement a weekday count).
- Display: "N days ago" (owner's call - plain and readable) plus the session-based state.

States:

```
| State | Rule | Render |
| `fresh` | <= `STALE_AFTER_SESSIONS` | score + direction, normal |
| `stale` | > `STALE_AFTER_SESSIONS` | **warning** styling + "not scored in N days" |
| `never` | no signal for (symbol, market) | "never scored" - a distinct state, NOT stale |
| `unavailable` | research ran but abstained (thin evidence) | "abstained - thin evidence", NOT a score |
```

never ? stale ? unavailable. Collapsing them is the exact bug class this batch removed.

5. Data

Source: latest agent_signals per (symbol, market) - analyst_score, direction, conviction, created_at (-> scored_at), is_holding, market.

Join key: (symbol, market). Never symbol alone - market is authoritative; per-market/per-currency never cross-summed.

Delivery: extend GET /api/portfolio/risk-daily (already returns per-holding rows and is owner-gated) with a nullable research block per holding. Additive only - no schema change, no migration.

```
research: {
  score: number | null,
  direction: 'long' | 'neutral' | 'short' | null,
  scored_at: string | null, // ISO
  sessions_since: number | null, // market-local
  days_since: number | null, // display
  state: 'fresh' | 'stale' | 'never' | 'unavailable',
  scored_as_holding: boolean | null, // see ?6
} | null
```

Two known traps:

- insider_score = 50 is a default-fill, not evidence. The honest record is decision_observations.availability_mask. Do not surface a dimension as evidence off agent_signals alone.
- is_holding matters. Before the starvation fix, every AVGO row was is_holding: false - scored as a *screener candidate*, not as a holding. That changes meaning: the direction gate can only emit short (a deterministic exit) when isHeld is true. A neutral from a screener-path row does not mean "no exit signal". Surface scored_as_holding and label it; do not silently present a candidate score as a holding verdict.

6. Click-through

Target: /dashboard/research-journal?symbol=<SYM>&market=<us|india> -> auto-expand that symbol's entry.

Gap: app/dashboard/research-journal/page.tsx currently expands via local toggle(symbol) state and reads no searchParams. Deep-linking must be added there. Small, but it is net-new surface, not a wire-up.

Rules: never-state symbols are not links (a link to nothing is a lie). The link must carry market - .NS symbols are unambiguous but US/India books must not cross-link.

7. Acceptance tests - each must be able to FAIL

```
| # | Test |
| T1 | **R1**: `computeHoldingRisk` / `sba-v1` output is byte-identical with the `research` block present vs absent. (Falsifiable: wire the score in -> T1 fails.) |
| T2 | A score older than `STALE_AFTER_SESSIONS` renders `stale` + day count; one inside renders `fresh`. |
| T3 | **Friday score, read Monday, threshold = 1 session -> `fresh`.** (Falsifiable: a calendar-day implementation fails this.) |
| T4 | No signal -> `never`, not `stale`, not score 0, and **not a link**. |
| T5 | An abstained (thin-evidence) signal -> `unavailable`, never rendered as a number. |
| T6 | US and India books never cross-join: an India `.NS` symbol resolves only against `market='india'` rows. |
| T7 | A `is_holding:false` row is labelled as a candidate score, not a holding verdict. |
| T8 | The risk API still returns holdings when `agent_signals` is empty/errors - research absence must never blank the risk table (fail-soft, per ?8). |
```

8. Failure modes

```
| Mode | Behavior |
| `agent_signals` read fails | Risk table still renders; research column shows an explicit "research unavailable" - **never** silently blank, never a fake score. Fail-soft: risk is the primary product here, research is the annotation. |
| Research hasn't run today | Correct and expected -> `stale` with the day count. This is the feature working. |
| Symbol held in 2 accounts | One research row per (symbol, market); it is a property of the symbol, not the account. Same annotation both places. |
| India research abstains (sentiment 0% available) | `unavailable`, not a score. |
```

9. Relationship to the earlier proposal

features/risk-research-integration/FEATURE_ARCHITECTURE.md proposed research ordering which names absorb a sector trim. Not pursued. Its central premise - that AVGO's 80 was thinEvidence - is factually wrong (AVGO carries all 5 dimensions; the neutral came from a since-changed thesis-parse-abstain path). It also could not resolve whether conviction-ordering smuggles the score back into a risk decision, and the only-diversifier objection stands. This spec is the cheaper, safer half: show both, couple neither.

10. Owner decisions - DECIDED 2026-07-17

All four are settled. This section is a record, not an open question.

```
| # | Decision | **DECIDED** | Rationale as approved |
| Q1 | `STALE_AFTER_SESSIONS` | **2** | The merged holdings rotation bounds worst-case staleness at ~2 runs, so `1` would flag ordinary rotation lag as a warning (noise) and train the owner to ignore it. `2` flags only genuine starvation - which is what AVGO actually was. |
| Q2 | Does `stale` also mark the row's **risk action** as lower-confidence? | **NO** | `stale` annotates the SCORE only. It must NOT dim or alter the risk action or verdict - that edges toward the coupling this spec exists to refuse. |
| Q3 | Show `conviction` alongside `analyst_score`? | **OMIT** | Identical to `analyst_score` in every prod row inspected. Revisit only if they diverge. |
| Q4 | Mobile (375px) columns | **score + age badge only** | Direction and `scored_as_holding` go behind the existing row expander. Mobile-first is a standing rule. |
```

Verified against prod after the decision (2026-07-17, project dionkikgdmLaotvtbnfr):

- Q3 is right, and for a better reason than stated. `analyst_score = conviction` in

458 of 463 rows - not all of them. The 5 exceptions (AAPL 58/62, NVDA 55/48, TSLA 36/58, MSFT 53/52, META 69/68) are all from 2026-06-28 and all carry `score_source = null` - the pre-deterministic_v1 LLM-scored era. Since deterministic_v1 shipped, the two have never diverged. So conviction is redundant *by construction* under current scoring, not merely *empirically*. Revisit if a future scorer decouples them.

- Q1's threshold does what it was chosen to do. AVGO's latest row (scored

2026-07-13, read 2026-07-17) is 4 sessions old -> stale. The India book (scored 2026-07-17 04:00 UTC) is fresh. The threshold separates the two.

10a. What prod actually looks like (measured, not assumed - 2026-07-17)

Three findings that shape the implementation:

- `is_holding` is false in 463 of 463 rows - 100%. Not an AVGO quirk: no holding-path score has ever been written. Every score on the Risk page is a screener-candidate score. ?5 trap 2 is therefore the **common** case, and `scored_as_holding` is labelled on every row rather than as an edge case.
- There is no abstain representation in `agent_signals`. `analyst_score` is non-null in all 463 rows; no column encodes "ran but abstained". The unavailable state is therefore defensive, and currently unreachable in prod - it fires only if research ever writes a null/NaN score, so that an abstain degrades honestly instead of rendering as 0. T5 pins it against a fixture. This is the one state with no production witness.
- `never` currently has no production witness either. Every holding in every account's latest run resolves to a signal. The state still exists and is tested - a book with a new position would hit it immediately - but it is not observable today.

10b. Spec corrections found while building

- ?6's premise was wrong. `app/dashboard/research-journal/page.tsx` **did** read `searchParams` (for `tab`), and the `toggle(symbol)` expander described there lived in a local `FunnelTab` component that was never rendered - dead the day it was written. The page has always delegated to `components/dashboard/ResearchFunnel.tsx`. Deep-link support therefore went into `ResearchFunnel.tsx`; the dead `FunnelTab` was deleted rather than modified, since it was the source of the misreading.
- The session helper already existed. `marketSessionsSince(createdAt, now, market)` in `lib/trading/paper-exit-policy.ts` (built on `isMarketHoliday` in `lib/trading/market-calendar.ts`) is the market-local session authority. Reused, not reimplemented, per ?4.

11. Out of scope

Live-trading accounts the app cannot trade (e.g. 965848641) get the same annotation - it is informational there, as the strategy note already is. No new cron, no migration, no provider call, no LLM.

risk sector breach allocation - Feature Architecture

Source: `features\risk-sector-breach-allocation\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Feature: Sector-Cap Breach Allocation (risk-internal, deterministic)

Status: SHIPPED - allocator `sba-v1`, posture consumer `hr-v3` Last updated: 2026-07-21 Owner: Vaibhav Update this file when: the allocation rule, its denominator basis, the breach arithmetic, the roles (`absorb` / `not_selected` / `no_breach` / `sector_unknown`), or the read-only advisory labelling changes.

Parent feature: `features/holding-risk-daily/FEATURE_ARCHITECTURE.md` - this file is the authority for the allocator; that file remains the authority for the score, the cron, the snapshot schema, and the LLM prose boundary.

Current posture boundary (hr-v3, 2026-07-21): this allocator remains the

correct simulation for a genuine equity-sector breach, but its `absorb` output

is internal diagnostic evidence only. Every live account displays measured

concentration under review, with no trim recommendation, because the current

limits are global references rather than account-specific mandates. An order-

permitted account is not an exception. Asset-class buckets

such as Diversified Equity, Fixed Income, Commodities, and Digital Assets never

enter this sector allocator.

1. The defect this exists to fix

Risk Analytics shipped this for AVGO, a live holding in the read-only Robinhood account 965848641:

"Strategy note (advisory): Trim your position because Technology holdings exceed

the 30% sector cap (at 65.6%), with a risk score of 63..."

A sector-cap breach is a property of the SECTOR, not of any one name. In `lib/risk/holding-risk.ts` (`hr-v1`) the posture branch read:

```
const sectorBreach = sectorUtil !== null && sectorUtil >= 1;
if (nameBreach || sectorBreach || clusterBreach) { riskPosture = "trim"; ... }
```

`sectorUtil` is the same number for every holding in the sector. So every Technology name received the identical `trim` verdict, with identical prose, and the engine never decided which names should absorb the breach or how much each gives up. The advice is arbitrary (no name is distinguished) and unactionable (no quantity). Trimming all six Tech names by an unstated amount is not a plan.

Note the same run also emitted `trim` for AVGO while ResearchAgent had not scored AVGO since 07-13 (see `WORK_LOG`, `holdings-starvation` entry). That is a separate bug; it is not evidence that risk should read research scores. See ??.

2. What this builds

`lib/risk/sector-breach.ts` - a pure, versioned (`sba-v1`) allocator:

Given a sector that is over its cap and the reduction required to reach the cap,

decide which holdings absorb it and how much each gives up.

Deterministic. No LLM. No research score. No I/O, no clock, no randomness - same inputs always yield the same allocation. It is called by the holding-risk cron before computeHoldingRisk, and its per-name result is threaded into HoldingRiskContext.sectorBreachAllocation.

3. Denominator: NAV, and why it decides the arithmetic

The breach amount is not basis-independent. Let S = sector market value, D = the denominator the cap is measured against, c = cap fraction:

```
| Denominator | Does a sale change `D`? | Value that must be sold |  
| **NAV** (cash-inclusive) | No - proceeds become cash, still inside `D` | `X = S - c*D` |  
| **Invested** (cash-excluded) | Yes - proceeds leave `D` | `X = (S - c*D) / (1 - c)` |
```

At Tech 65.6% / cap 30%: NAV basis -> sell 35.6pp. Invested basis -> sell 50.9pp. Picking the wrong basis makes the advice wrong by ~43%.

Kairos' sector cap is NAV-relative. lib/risk/live-portfolio-gate.ts - the code that actually enforces max_sector_exposure_pct on the live money path - builds the book as valuePct = position value / NAV x 100. That is the authoritative reading of the owner's limit. lib/risk/holding-risk.ts already scores name concentration against accountTotalValue (NAV) for the same reason.

So the allocator takes navValue (not a generic "denominator"), and the required reduction is the simple $X = S - cD$. The field is named navValue precisely so a future caller cannot silently feed an invested-total and get 43%-wrong advice.

3.1 Known inconsistency this fixes (and one it does not)

hr-v1 fed sectorWeightPct from computeRiskMetrics().sectorBreakdown, whose weightPct is invested-relative (lib/portfolio-risk.ts L307-318: totalValue = holdings.reduce(...marketValue)). So hr-v1 compared name concentration against NAV and sector concentration against invested - two different denominators against two caps the owner set on one basis. The cron now derives sector weights on the NAV basis from the allocator, so the sector driver and the allocation agree, and both agree with the name driver.

Not fixed here (deliberate): computeRiskMetrics().sectorBreakdown remains invested-relative for the account roll-up display. Changing it touches the paper path, constructPortfolio inputs, and the Risk page's sector bars - out of scope for a risk-verdict fix. The two numbers can differ for a cash-heavy account. Recorded as a follow-up.

4. The allocation rule - WATER-FILL (level-down, largest-first)

Rule. Trim the largest positions in the breached sector down to a single common level L , chosen so the sector lands exactly on the cap:

find L such that $\sum \min(w_i, L) = c$ over the sector's names (weights as

fractions of NAV), then trim $w_i = \max(0, w_i - L)$.

Names with $w_i \leq L$ are not touched. Names with $w_i > L$ absorb the breach, largest first, and all end at the same weight L .

Solved in closed form: sort weights ascending, and for $k = 0 \dots n-1$ test the candidate $L = (c - \text{prefix}[k]) / (n - k)$, accepting the first k where $L \geq w[k-1]$ and $L \leq w[k]$. $\sum \min(w_i, L)$ is continuous and non-decreasing in L , from 0 to the sector weight, so exactly one L exists whenever $0 < c < \text{sectorWeight}$.

4.1 Justification

Why not pro-rata? Scale every name in the sector by $c / \text{sectorWeight}$.

- *For:* expresses no view on which name is worse - appropriate, since a risk engine with no conviction input has no such view.

- *Against, decisive:* every name is trimmed, so every name still gets the same blanket "trim" verdict - the exact defect in ?1, re-shipped with numbers attached. And it preserves the sector's concentration profile: after a pro-rata cut the book is at the cap but just as lopsided, so the *name* cap - a second limit the owner set - is left as breached as it was. Rejected.

Why not "highest marginal contribution to the breach" first? For a *sector weight* cap, a name's marginal contribution to the sector weight is its weight - $\frac{w}{\sum w} = 1$. So this rule is identical to largest-first with an extra abstraction that invites smuggling correlation or beta into the ordering. Those are separate risk components with their own caps and their own missing-data paths; folding them into the sector allocation would make one number's absence silently reshuffle who sells. Rejected as a distinct rule.

Why water-fill wins. Among all allocations that reach the cap by reducing only ($0 \leq t \leq w$), the water-fill is the one that minimises the largest residual name weight $\max w$ - and, more generally, minimises $\sum (w^2)$ (the sector's Herfindahl). Sketch: any feasible allocation with two names at $a > b \geq L$ where the water-fill would have levelled both must have $\max w \geq a > L$; levelling weakly reduces both the max and the sum of squares while holding the total fixed. So the water-fill discharges the sector breach and does the most it can for the name-concentration cap at the same time - the two limits the owner actually set point the same way. It also yields real "hold" verdicts for small names, which is what makes the output actionable rather than a blanket.

Honest cost. Water-fill sells your biggest name first, which is often your best name. The risk engine cannot know that - it has no conviction input by design (?7). Absent a view, a same-sector shock hits every name in the sector and expected loss scales with weight, so reducing the largest exposures is the risk-correct action. If the owner wants conviction to reorder who sells, that is a Research<->Risk coupling decision, not a change to this allocator.

4.2 Determinism and tiebreaks

- Ties in weight need no arbitrary winner. Water-fill is a continuous

function of the weight vector: two names at identical weight receive identical trims by construction. There is no "first one wins" step that a tie could turn arbitrary. *(Test: equal weights receive equal trims.)*

- Comparison tolerance. The fill runs in weight-fraction space (values ~0-1),

not currency, so a fixed $\text{EPS} = 1e-12$ is meaningful at every account size.

- Output ordering (the only place a tiebreak is needed): absorbers are

ordered by trimPct descending, then symbol ascending (lexicographic). rank follows that order. Sector summaries are ordered by sectorWeightPct descending, then sector ascending.

- No materiality floor. A name is absorb iff its trim is > 0 . A floor

would have to redistribute the dropped remainder, which reintroduces an arbitrary choice for a cosmetic gain.

4.3 Degenerate inputs

```
| Input | Behavior |
| `navValue` <= 0 / non-finite | Empty result; every symbol `sector_unknown`-style honest reason is **not** used - the caller's structural gate already returns `insufficient_data` first. |
| `maxSectorExposurePct` <= 0 or >= 100 | Cap unusable -> sector reported `no_breach` with the cap echoed; never a fabricated breach. |
| Sector weight <= cap | `no_breach` for every name in it; `requiredReductionPct = 0`. |
| Non-finite / negative `marketValue` | That position is excluded from its sector's total and reported as `sector_unknown` (unvalued), never counted as 0. |
```

5. Per-name output (Detail Over Cryptic)

SectorBreachAllocation per symbol, with a role:

```
| Role | When | `action_reason` says |
| `absorb` | breached sector, `w? > L` | *what*: trim N pp of NAV, to X% (? value in account currency).
*why*: sector is P% vs the C% cap -> Rpp must come out; this is the #k of n largest names, allocated
largest-first down to a common L% level. *next*: advisory suffix. |
| `not_selected` | breached sector, `w? <= L` | *what*: hold. *why*: "***Technology is over its 30% cap (65.
6% of NAV) and 35.6pp must come out, but AVGO is not among the names selected to absorb it** - at 3.6% of
NAV it already sits at or below the 5.4% level the 4 larger Technology positions are being trimmed to."
*next*: what would change the verdict. |
| `no_breach` | sector within cap | sector weight vs cap, no reduction required. |
| `sector_unknown` | sector null/empty/"Other", or unvalued | "sector unknown - excluded from sector-cap
allocation: neither counted toward a breach nor asked to absorb one." Never treated as its own sector,
never as cap-compliant. |
```

6. Posture wiring (lib/risk/holding-risk.ts, current hr-v3)

Two new optional HoldingRiskContext fields: sectorBreachAllocation?: SectorBreachAllocation | null, readOnlyAccount?: boolean. Everything else - the score, the six components, the caps, the confidence weights, add_capacity - is unchanged. Scores do not move except through the NAV-basis denominator correction in ?3.1.

Precedence (the exit branch is untouched and stays first):

- verified protective-stop / thesis-break -> exit_review - unconditionally,

regardless of any allocation. Drawdown alone still never triggers it.

- nameBreach || sectorSelected || clusterBreach -> review, where

sectorSelected = sectorBreach && allocation.role === "absorb". The allocator remains internal evidence; its simulated reduction is not exposed as advice. No concentration condition creates a sell instruction.

- sectorBreach and no usable allocation -> review +

missing_inputs: ["sector_breach_allocation"]. Reason: the sector is over cap but the engine cannot say whether *this* name should absorb it.

- dataConfidence < 0.5 -> review (unchanged).

- otherwise -> hold; when role === "not_selected" the hold carries the

allocator's why-not sentence instead of "within owner-approved risk limits" (which would be a lie while the sector is over cap).

Default-to-honest, never default-to-confident. Case 3 is the degradation-guard discipline: a legacy caller that supplies no allocation gets review - the *old* blanket-trim behavior is not preserved as a fallback, because a fallback to the bug is the bug.

formula_version is now hr-v3: the posture semantics changed again, and risk-daily must not diff a v3 posture against a prior formula. No migration - formula_version is text, risk_posture keeps its existing CHECK values, and the allocation surfaces in the existing action_reason (text) and the sector driver's detail (jsonb).

7. Explicitly out of scope

- No analyst_score / agent_signals / conviction coupling. The entire point

is that this needs zero Risk<->Research coupling: the allocator is a function of weights and one owner-set cap. The competing proposal (features/risk-research-integration/FEATURE_ARCHITECTURE.md, branch worktree-agent-abaf0b16ef6af4175, unmerged) is untouched.

- No order path. The route places, previews, and cancels nothing. Only

605420660 may ever place an order, and only via the owner-click Execution Gateway, which this feature does not call.

- No LLM on any number. The LLM still writes strategy_note prose only, and

is now handed the allocation as read-only context it must explain, never alter. lib/risk/strategy-notes.ts extracts the parse step so this is provable: parseStrategyNotes() returns Map<symbol, string> and cannot express a score, a posture, a trim, or a symbol that was not asked for.

- No migration. No schema change.

8. Advisory labelling

readOnlyAccount is set by the cron: false only for 605420660 (the sole order-permitted account per CLAUDE.md), true for every other account including 965848641 - where AVGO sits. readOnlyAccount changes advisory transport wording only; concentration is review for both account classes. exit_review reasons carry:

- read-only -> **"Advisory only - this account is read-only in Kairos; the app cannot trade it."**

- 605420660 -> **"Advisory only - this feature places no order; any action requires owner approval in the Execution Gateway."**

Both are advisory. The distinction tells the owner whether an order path exists at all, rather than implying one does.

9. Acceptance tests (falsifiable - each states the implementation that fails it)

tests/sector-breach.test.ts, tests/holding-risk.test.ts, tests/strategy-notes.test.ts.

```
| # | Test | Fails when |
| 1 | Tech 65.6% of NAV vs 30% cap -> `requiredReductionPct ? 35.6`, and `? trimPct ? 35.6` | the invested-
basis formula (would give 50.9), or an off-by-cap error |
| 2 | Post-trim sector weight equals the cap exactly; every `targetWeightPct = min(w?, L)` | the fill level
is mis-solved |
| 3 | Reproducible: two runs over the same input deep-equal; input array shuffled -> identical `bySymbol` |
any iteration-order or `Map`-order dependence |
| 4 | Two names at identical weight get identical `trimPct`; ordering ties break `symbol` ascending | any
"first/last one wins" branch |
| 5 | A small name below `L` gets `role: "not_selected"`, `trimPct === 0`, and a reason naming its sector,
the cap, the breach size, and why it was not selected | pro-rata (nothing would be `not_selected`), or a
bare "hold" string |
| 6 | Every name in a breached sector gets its own `trimPct`, and **not all are equal** | blanket-trim (`hr-
vi`) - all names identical |
| 7 | `protectiveStopHit` + `role: "not_selected"` -> still `exit_review` | the allocator being consulted
before the exit branch |
| 8 | `thesisBreak` + `role: "not_selected"` -> still `exit_review` | same |
| 9 | Sector breached, **no** allocation supplied -> `review`, `missing_inputs` contains
`sector_breach_allocation`, posture is **not** `trim` | a fallback to blanket-trim |
| 10 | Sector breached, `role: "absorb"` -> `review`, no simulated sell quantity in the action reason |
diagnostic allocation leaking as advice |
| 11 | Sector breached, `role: "not_selected"`, nothing else breached -> `hold` **and** reason is not
"within owner-approved risk limits" | the generic hold string leaking while the sector is over cap |
| 12 | Name breach + `role: "not_selected"` -> `review`, with no trim recommendation | global reference
leaking as a sell mandate |
| 13 | Cluster breach + `role: "not_selected"` -> `review`, with no sell quantity | unallocated overlap
leaking as advice |
| 14 | India: INR account, `.NS` symbols, India sector labels -> identical allocation shape; `no_breach`
when within cap | any US-GICS or USD assumption |
| 15 | US and India inputs with equal weights produce equal `trimPct` and never a summed/cross-market total
| cross-market contamination |
| 16 | `sector: null` / `""` / `""` -> `role: "sector_unknown"`, excluded from every sector total,
reason says "sector unknown"; the other names' allocation is unchanged by its presence | bucketing unknowns
into a synthetic sector (what `constructPortfolio` does), or treating them as cap-compliant |
| 17 | A sector that is entirely unknown-sector names produces **no** `no_breach` claim for them | silent
cap-compliance |
| 18 | `parseStrategyNotes` given { "AVGO": { "trim_pct": 99, "risk_posture": "hold", "ZZZZ": "x" } } returns
**no** AVGO note, no `risk_posture` key, no `ZZZZ` | any parser that passes non-strings or unrequested keys
through |
| 19 | `parseStrategyNotes` over unparseable/absent LLM text -> empty map, never throws | prose failure
blocking a deterministic row |
| 20 | Both read-only and order-permitted accounts keep concentration at `review`; transport wording
remains accurate | order permission being confused with a concentration mandate |
```



```
| 21 | `navValue` unchanged, cap 0/100/NaN -> `no_breach`, never a fabricated breach | an unguarded cap  
divide |
```

Plus the parent feature's existing hr suite (structural gate, component caps, loss-only-never-exit, purity) must stay green.

10. Cross-doc updates shipped with this

- docs/arch/08-risk-and-safety.md - "Daily Per-Holding Risk Analytics" section.
- features/holding-risk-daily/FEATURE_ARCHITECTURE.md - pointer to this file.
- WORK_LOG.md - entry.
- public/agent-diagrams/system-map.json - the diagram is unchanged: no

agent-to-agent flow, handoff, table dependency, schedule, or learning-loop edge changed; this is internal to the HoldingRisk node's own compute. But the HOLDINGRISK node's own description asserted hr-v1 and the old "hard concentration/cluster breach -> trim" rule, which is now false - so the node description is corrected and a history entry appended.

- No migration.

robinhood agentic evidence - Feature Architecture

Source: features\robinhood-agentic-evidence\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Robinhood Agentic Evidence - Feature Architecture

Status: P0 approved and implemented. Later phases require a separate owner-approved design and evidence gate.

Scope: US only. Robinhood's MCP capability surface is observed, not trusted as a data provider or trading policy.

P0: Capability Snapshot

The weekly route opens an authenticated MCP session and invokes only `initialize`, `notifications/initialized`, and `tools/list`. It stores the sorted tool names, tool count, and a SHA-256 fingerprint of the name plus input-schema contract in the append-only `broker_mcp_capability_snapshots` ledger. Raw descriptions and schemas are intentionally not persisted because they are untrusted remote input.

It does not invoke `tools/call`; therefore it cannot consume data-provider quota, read accounts or positions, create research evidence, change scores, change eligibility, create paper orders, or create live orders. An unavailable connection is an observation, not a System Health incident or a trading gate.

Deferred Phases

- Owner-run, allowlisted contract probes for `get_financials`, `earnings`, and technical tools. Each probe is bounded and records no raw payload in an LLM context.
 - A robinhood adapter inside the existing Canonical Evidence Router, using its cache, pacing, provider-call ledger, provenance, health, and schema validation. No parallel evidence truth layer.
 - Shadow-only field-level comparison and point-in-time walk-forward validation.
- A broker field cannot alter a score merely because it is available.
- Separately, retain `review_equity_order` as a fail-closed pre-trade safety gate. Capture its contract only after an approved execution-specific design; it is not part of research evidence.
- No scanner, Level 2, tax lots, options, or Robinhood data call is enabled by P0.

robinhood mcp integration - Feature Architecture

Source: features\robinhood-mcp-integration\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature Architecture: In-App Robinhood MCP Client - read-only snapshot + human-gated live order execution + allowlist-backed account selector

Status

Architecture status: Implemented (OAuth + deterministic MCP path), 2026-07-07 Architecture approved: Yes (user, 2026-07-07) Approved scope: OAuth 2.1 flow + deterministic MCP client + Gateway hardening + account allowlist. Live order execution stays behind robinhood_mcp_enabled (default OFF) + all Gateway gates + human Approve/Send. Approved date: 2026-07-07 Implementation allowed: Yes

Verified OAuth metadata (Robinhood live well-known, 2026-07-07)

Confirmed by fetching the endpoints directly - NOT guessed:

- authorization_endpoint = https://robinhood.com/oauth
- token_endpoint = https://api.robinhood.com/oauth2/token/
- registration_endpoint = https://agent.robinhood.com/oauth/trading/register
- scope = internal ? PKCE S256 ? token_endpoint_auth_method = none (public client)
- resource = https://agent.robinhood.com/mcp/trading
- No revocation_endpoint is published -> Robinhood's own Agentic Trading

dashboard is the authoritative remote revoke; in-app Disconnect wipes local tokens.

Remaining runtime unknown (fail-closed, not guessed)

The exact place_equity_order / review_equity_order argument *schema* over MCP is discovered at runtime via tools/list; the adapter maps canonical order fields onto the returned inputSchema and ABORTS if a required field can't be confidently filled - it never sends a guessed real-money payload. There is no Robinhood sandbox, so the FIRST live order should be a tiny qty, watched, to confirm the field mapping before trusting it.

Revision history

- 2026-07-06 - original draft: read-only snapshot via DeepSeek+MCP, no order execution.
- 2026-07-07 (a) - scope expanded at user request: live order execution via the existing broker-adapter/Gateway pattern; live_account_source switch; broker_accounts allowlist + per-market active-account selector.
- 2026-07-07 (b) - this rewrite: full security review (18 findings) folded in as binding rules. Biggest changes: no LLM anywhere in the order write path (direct typed client.callTool() instead of a "DeepSeek tool-calling loop"), OAuth hardening (state+PKCE+authenticated callback - the Kite route-pair is NOT a safe template for OAuth 2.1), the Execution Gateway is explicitly MODIFIED (not "unchanged") to add kill-switch/guard/env/notional checks it does not have today, robinhood_mcp_enabled defaults OFF, fail-closed account resolution, and reconcile-before-resubmit semantics for ambiguous order timeouts. Prerequisite vault lockdown (migration 089) was applied live 2026-07-07: api_key_vault had a USING (true) policy for all roles with client write grants, and broker_orders had RLS disabled - both now service-role-only.

Feature Purpose

Today, refreshing the live Robinhood account snapshot (equity, buying power, positions for the read-only Trading account 965848641) depends on `execClaude` (`lib/claude-exec.ts`) - Windows-desktop-only, fails on Vercel. Robinhood order execution (account 605420660) is fully manual: approving a proposal generates a natural-language command the user pastes into their own Claude Code session with Robinhood MCP access.

This feature adds an in-app path for both concerns, using Robinhood's real remote MCP server (`https://agent.robinhood.com/mcp/trading`, confirmed via the user's `.claude.json`; it exposes `get_accounts`, `get_equity_positions`, `place_equity_order`, `review_equity_order`, `cancel_equity_order`, `get_equity_orders`, among others):

- Read path - a deterministic MCP client fetches the account snapshot

server-side (Vercel or local), replacing/coexisting with `execClaude`, selected by a new `strategy_config.live_account_source` switch (`'claude_exec'` | `'robinhood_mcp'`, default `'claude_exec'`).

- Write path - Robinhood becomes a third broker adapter in

`lib/brokers/registry.ts` (alongside Alpaca and Kite), flowing through the Execution Gateway with the same two-human-click gate (Approve -> Send to broker). The only change vs today is transport: a typed MCP call instead of a manual paste. Who decides never changes.

- Account selector - a `broker_accounts` allowlist table +

`active_account_us/active_account_india` so future additional accounts are user-selectable but never implicitly tradable.

BINDING IMPLEMENTATION RULES (non-negotiable - deviation = reject the PR)

These rules exist because a security review found the original draft ambiguous enough to permit dangerous implementations. Each rule is stated so compliance is mechanically checkable.

R1. No LLM in the write path - ever.

`submitOrder()` MUST be a direct, deterministic `client.callTool({ name: "place_equity_order", arguments: {...} })` using `@modelcontextprotocol/sdk's Client` over `StreamableHTTPClientTransport`. Arguments are built exclusively from the validated `trade_proposals` row and `broker_accounts` - never from, through, or "confirmed by" any LLM. No LLM output is ever parsed to derive order parameters, order status, or `ok:true`. (Rationale: `trader/route.ts` already documents the hallucinated-fill failure mode this prevents.)

R2. The read path is ALSO deterministic by default.

The snapshot fetch calls `get_accounts` + `get_equity_positions` directly via `callTool` and parses the structured results with a strict zod schema: numbers must be finite, `symbol` must match `^[A-Z.\-]{1,10}$`, arrays bounded (`<=500` positions). No LLM is required to call two read tools. An LLM summarizer MAY be layered on top for display text only, subject to R3.

R3. LLM data-hygiene rules (applies to any optional summarizer).

- Vault tokens (`ROBINHOOD_MCP_*`), `CRON_SECRET`, and raw account numbers

NEVER appear in any LLM prompt or LLM-visible tool argument. Account numbers are replaced with opaque labels ("Trading account") before prompting.

- MCP tool results are data, never instructions: the system prompt states

this, and no numeric/actionable value from LLM output is written to any table. Loop iterations hard-capped.

R4. OAuth 2.1 - do NOT copy the Kite route-pair internals.

Kite's callback runs unauthenticated with no state (fine for Kite's checksum-signed flow, unsafe for OAuth). The Robinhood flow MUST have:

- state: $\geq$ 128-bit random, stored in an HttpOnly, Secure, SameSite=Lax, short-TTL (10 min) signed cookie; verified and single-use in the callback.
- PKCE S256; the verifier stored in the same signed cookie (serverless-safe)
- in-memory storage does not survive between /login and /callback invocations on Vercel).
- The callback REQUIRES an authenticated owner session

(createClient().auth.getUser() + the middleware owner gate) before exchanging the code - prevents login-CSRF/token-injection.

- Exact registered redirect_uri, strict string compare. Post-auth redirect is HARDCODED to /dashboard/settings?rhmcpc=connected - no next param.

- Request the MINIMUM scope set Robinhood offers. If read-only scopes exist, the snapshot connection uses only those; if only a bundled trading scope exists, that fact is surfaced in the Settings UI text.

- Related fix shipped alongside: app/auth/callback/route.ts must validate its next param (single leading /, no //, \, or @) - current code allows ?next=@evil.com open redirect.

R5. Token handling.

- Tokens live only in api_key_vault (service-role-only as of migration

089) and are sent only in the Authorization header - never in URLs, never logged, never included in broker_orders.raw_last_state (redact before persisting any raw payload; also redact account numbers).

- Refresh is single-writer: compare-and-swap on the vault row

(UPDATE ... WHERE updated_at = <value read>), losers re-read instead of overwriting - prevents the rotating-refresh-token race between concurrent cron + user invocations from bricking the connection.

R6. Gateway modifications (the Gateway is NOT "unchanged").

app/api/broker/orders/route.ts today lacks several gates the original draft wrongly claimed it had. This feature MUST add, in order, before submitOrder:

- guardOrderRequest(req) (Host/Origin check) - extended to read the allowed host from an APP_BASE_URL env var so it works on Vercel (current guard allowlists localhost only; do NOT delete the guard to "fix" Vercel).

- env MUST be explicitly present in the request body; reject if absent.

No silent ?? "paper" default on an order-placing route.

- broker.envs.includes(orderEnv) checked in BOTH directions (a live-only broker rejects paper; a paper-only broker rejects live). The adapter additionally self-rejects wrong env (defense in depth).

- checkKillSwitches(supabase) - kill switches currently run only at

proposal build/approve, not at submission. They MUST also run here.

- Existing checks retained: `proposal status='approved'`,

`approval_expires_at`, `global trading_enabled` + `per-market trading_enabled_us/trading_enabled_india` (live env).

- Submit-time re-validation (live env): `qty` positive integer $\leq$ hard cap;

fresh quote fetched and `qty * quote` $\leq$ `max_order_notional` (new `strategy_config` column, default $\leq 15\%$ of latest `live_account_snapshots.equity`); price drift vs `proposal.price_at_proposal` $\leq$ the same threshold `trader/route.ts` uses at approval; symbol matches `^[A-Z.\-]{1,10}$`; `side='sell'` allowed only if the symbol exists in the current holdings snapshot (long-only for new positions per CLAUDE.md).

- Duplicate-submit hardening: partial unique index

`broker_orders(proposal_id) WHERE status IN ('pending_submit', 'submitted', 'partially_filled')` - the existing check-then-insert race allows double-submit on concurrent clicks.

R7. Ambiguous-failure semantics (the case that duplicates real orders).

If `place_equity_order` times out or errors AFTER possibly succeeding, the adapter MUST NOT resubmit. It marks the order row `status='unknown_needs_reconcile'` and the sync loop (or a manual button) calls `get_equity_orders` to reconcile before any human is allowed to retry. Where Robinhood's MCP supports it, use `review_equity_order` -> place sequence and/or a client-order-id for idempotency. No automatic retry of any write call, ever.

R8. Account allowlist - fail closed, hardcode stays.

- Validation lives in BOTH the Gateway (before `submitOrder`) and inside

`submitRobinhoodOrder()` (last code before the wire): resolved account must exist in `broker_accounts` with `role='trading'` and matching market.

- ANY error reading `broker_accounts/active_account_us` ABORTS the order.

The silent fallback-to-default pattern used by `getActiveBroker()` for broker selection is explicitly FORBIDDEN for account selection.

- The existing hardcode (`AGENTIC_ACCOUNT = "605420660"` in

`trader/route.ts`) is NOT removed in this feature. Removal is a separate, later change gated on the allowlist being verified live. Until then both checks run.

- New accounts default `role='view_only'`. Becoming a trading target

requires BOTH explicitly setting `role='trading'` AND selecting it as `active_account_us/active_account_india`. No auto-discovery from broker APIs marks anything tradable.

- `broker_accounts` ships service-role-only (no anon/authenticated grants,

no permissive policy) - a client-side writer to this table would defeat the entire allowlist. Migration must be verified applied to the live DB before the validation code merges (global schema rule).

R9. Kill switches and flags (contradiction in prior draft resolved).

- `robinhood_mcp_enabled` (new, default FALSE - a live-order

integration's kill switch must not ship pre-armed ON; user flips it on in Settings after connecting): when false, BOTH the MCP snapshot fetch and the MCP order path are blocked. The last cached snapshot in `live_account_snapshots` remains viewable - "blocked" means no new MCP calls, not hidden data.

- `trading_enabled_us` off: blocks live orders (all US brokers), never

blocks any read.

- In-app Disconnect: deletes vault tokens even if Robinhood's revocation

endpoint is unreachable (same contract as `disconnectKite()`), and the Settings UI states that Robinhood's own Agentic Trading dashboard is the authoritative, app-independent kill switch.

R10. Cron/auth surface fixes shipped with this feature.

- New shared `verifyCronSecret(req)` helper using `crypto.timingSafeEqual`

and rejecting when `CRON_SECRET` is unset/empty; used by the new routes (and the snapshot routes it touches). Repo-wide migration of the other ~30 `===` comparisons can follow separately.

- `GET /api/live-account/snapshot` currently returns live equity/positions

with NO auth - it MUST require an authenticated owner session before this feature writes fresher data into it.

- The `robinhood_mcp` snapshot branch writes to `live_account_snapshots`

directly via the service client - NOT via the existing loopback HTTP POST that ships `CRON_SECRET` to `NEXT_PUBLIC_APP_URL`.

Scope

- OAuth 2.1 client per R4: `/api/robinhood-mcp/login` + `/api/robinhood-mcp/callback`.

- Token storage per R5 in `api_key_vault`.

- `lib/robinhood-mcp.ts`: OAuth helpers, vault-backed token storage/refresh

(CAS), `queryRobinhoodAccount()` (read, R2), `submitRobinhoodOrder()` (write, R1/R7/R8 - called only by the broker adapter).

- `lib/brokers/robinhood-mcp.ts`: adapter implementing the standard

interface (`id: 'robinhood_mcp'`, `envs: ['live']`, `isConfigured()`, `submitOrder()`), registered as a second US-market option.

- Gateway modifications per R6.

- `strategy_config.live_account_source` + Settings selector ("Local -

Windows Scheduler + Claude Code" vs "Cloud - Robinhood MCP").

- `broker_accounts` table + `active_account_us/active_account_india` +

Settings UI per R8 (manual add form: user types the account number and picks the role explicitly; no bulk import).

- Settings card "Robinhood MCP (Live Account + Orders)": connection status

(`connected: boolean` + `updated_at` only - never token material), Connect/Re-authorize, Disconnect, `robinhood_mcp_enabled` toggle, `live_account_source` selector, authoritative-kill-switch note.

Non-Goals

- Any autonomous order placement. Every order still requires (1) human

Approve on the proposal, then (2) human "Send to broker" click. This feature changes transport for step 2 only.

- Removing the manual Claude-Code-paste flow (stays available: simply don't

select robinhood_mcp as active_broker_us).

- Removing the 605420660 hardcoded (separate later change, R8).
- Options trading: place_option_order is never called; the adapter

supports equities only.

- Auto-discovery of accounts from broker APIs.
- Building the OAuth routes before the user has confirmed Robinhood's actual endpoints/scopes against the Agentic Trading dashboard.

Data Models

- api_key_vault: ROBINHOOD_MCP_ACCESS_TOKEN, ROBINHOOD_MCP_REFRESH_TOKEN (service-role-only per migration 089 - already applied live).
- strategy_config.live_account_source - text, 'claude_exec' | 'robinhood_mcp', default 'claude_exec'.
- strategy_config.robinhood_mcp_enabled - boolean, default false (R9).
- strategy_config.max_order_notional - numeric, default null -> computed as 15% of latest live equity (R6.6).
- strategy_config.active_account_us / active_account_india - text, default today's hardcoded values.
- broker_accounts - id, broker, market, account_number, label, role ('trading' | 'view_only'), created_at; service-role-only; seeded 605420660->trading/us, 965848641->view_only/us, current Kite account->india.
- broker_orders - new partial unique index per R6.7; new status value unknown_needs_reconcile (R7). RLS enabled + client grants revoked (migration 089 - already applied live).

Error Handling

- Read-path MCP/OAuth failure -> snapshot stays stale, {ok:false, error} logged (Decision 45 contract). Never throws away the cached snapshot.
- Write-path failure before the wire -> order row status='error' with the real message. Possible-success ambiguity -> R7 reconcile flow. No silent retry anywhere.
- Account/allowlist resolution failure -> abort (fail closed, R8).

Files / Behavior That Must Not Change

- Phase 0: proposals always pending_approval; auto-approve never permitted.
- Long-only enforcement for new positions; SELL only on held positions - now also enforced at Gateway submit time (R6.6).
- The existing execClaude-based mentor/portfolio/chart-data features.
- trader/route.ts's manual-paste flow remains available and unchanged except for adding allowlist validation alongside (not replacing) the account hardcoded.

Acceptance Criteria (each is a hard review gate)

- No code path derives order parameters or order status from LLM output

(R1). Grep-level check: `place_equity_order` appears only inside `submitRobinhoodOrder()`, and that function contains no LLM call.

- Snapshot fetch works with zero LLM calls (R2); any summarizer failure cannot block or alter stored numbers.

- OAuth callback rejects: missing/mismatched state, replayed state, absent owner session (R4).

- Tokens never appear in URLs, logs, LLM prompts, or persisted raw payloads (R5) - verified by grep + a redaction unit test.

- Gateway rejects: absent env, env not in `broker.envs`, kill-switch tripped, notional over cap, price drift over threshold, sell of unheld symbol, account not in allowlist (R6, R8) - each with a unit test.

- Ambiguous submit timeout produces `unknown_needs_reconcile`, never a resubmit (R7) - unit test with a mocked timeout.

- `robinhood_mcp_enabled=false` blocks new MCP calls (read+write) while cached snapshot stays viewable; default is false (R9).

- Disconnect wipes local tokens even when Robinhood is unreachable; Settings names Robinhood's own dashboard as the authoritative kill switch (R9).

- `GET /api/live-account/snapshot` requires owner auth (R10).

- Adding a `broker_accounts` row never makes it tradable without explicit `role='trading'` + active-account selection (R8).

- All new tables/columns verified applied to the live DB before dependent code merges (global schema rule).

Approval

Architecture approved: No Approved scope: None Implementation allowed: No

router cutover - Feature Architecture

Source: features\router-cutover\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Canonical Evidence Router Controlled Cutover

Status: REVIEWED DESIGN DRAFT. NOT APPROVED FOR IMPLEMENTATION OR CUTOVER.

Last reviewed: 2026-07-15 by Codex.

Companion authority: features/data-source-policy/FEATURE_ARCHITECTURE.md.

Scope: score-affecting Router activation and rollback, independently per market.

1. Current State And Correction

The Router is shadow-only. `resolveEvidence` rejects ordinary callers while the active immutable policy has `router_enabled=false`; only an explicit shadow caller may bypass that gate. No scorer currently consumes `EvidenceEnvelope`.

Cutover is not "flip a boolean." Policy versions are immutable. Enabling or disabling the Router means creating and atomically activating a new complete market-local policy version. The policy ID is frozen at run start.

The earlier proposed runtime check ("score with and without the changed dimension") is insufficient. It cannot reliably distinguish a genuine new fact from provider availability, and one-symbol recomputation misses cohort-rank changes. Eligibility safety therefore has two separate controls: a same-input policy activation gate and a runtime evidence-degradation gate.

2. Invariants

- US and India policies, universes, caches, evaluations, signals, and rollback are independent. Activate one market and one score-affecting intent family at a time.
- A run freezes policy version, strategy/genome, universe, as-of, and evidence snapshot. Mid-run policy activation affects only the next run.
- Source changes must preserve field semantics: period, basis, unit, currency, adjustment, observation time, and knowledge time. "Same or better" coverage is not semantic parity.
- No missing, stale, conflicting, quarantined, or schema-invalid evidence is converted to zero/neutral.
- Provider/routing change alone cannot create a newly eligible long.
- Existing trading, market-control, kill-switch, drawdown, mandate, rank, approval, and execution gates remain unchanged and are rechecked at their existing sites.
- Cutover changes research evidence acquisition only; it grants no live autonomy.

3. Intent Classification

Maintain a code-owned classification:

- `score_affecting`: canonical fields consumed by deterministic dimensions;

- `eligibility_affecting`: prices/events used by filters, freshness, ranks, or gates;
- `narrative_only`: display/context that cannot change score or eligibility; and
- `unsupported`: no valid market capability.

Do not require `analyst_consensus` parity for India or make US narrative analyst data a scoring cutover blocker. Each scorer field declares its required intent, minimum field contract, acceptable quality/freshness, and legacy mapping.

4. Frozen Dual-Run Evaluation

For each candidate policy version, build one immutable evaluation cohort per market:

- freeze universe, as-of, active baseline policy, candidate policy, strategy/genome, threshold, ranks, and price basis;
- resolve/cache raw provider observations once where contracts permit;
- construct legacy and candidate canonical inputs from the same knowledge cutoff;
- run the full deterministic cohort scorer and all pre-trade eligibility/rank gates;
- persist field, quality, availability, score, rank, direction, and eligibility deltas with evidence fingerprints; and
- classify every divergence with a bounded code and reviewer note.

The comparison must include symbols where either path abstains or fails. It cannot compare only jointly successful rows. A reverse-shadow run after cutover reuses the same frozen observations/cache; it must not double provider bursts.

Semantic parity

Each canonical field has an explicit comparator:

- exact match for enums, dates, currency, basis, period, and nullability;
- documented numeric tolerance only after unit/period normalization;
- no tolerance across TTM/quarterly/annual/forward or adjusted/unadjusted bases;
- field-level provenance and conflict state retained; and
- unexplained disagreement fails the candidate, even if aggregate score is close.

5. Activation-Time Eligibility-Flip Gate

Before a score/eligibility-affecting policy may activate, its immutable evaluation must prove on the full cohort:

- zero unexplained ineligible/abstain -> eligible transitions;
- zero newly eligible transitions caused only by source availability, stale fallback, field omission, conflict resolution, basis mapping, or weight renormalization;
- no material adverse rank displacement attributable only to missingness;
- no lower required-field coverage versus baseline beyond a pre-approved non-inferiority margin; and
- no schema, currency, market, provenance, or secret-redaction failures.

A genuine evidence-value change may create eligibility only when field semantics are equivalent, the observation is valid at the shared as-of, and the divergence is explicitly classified and owner-approved. Added provider coverage is first measured; it is not silently treated as approval to trade more names.

The activation API/RPC must bind approval to the exact candidate version, baseline version, evaluation ID, evaluation code version, strategy version, market, and expiry. A stale evaluation cannot authorize a new policy or changed scorer.

6. Runtime Evidence-Degradation Guard

Activation parity cannot prevent later outages. At each research run:

- evaluate each symbol against the code-owned minimum field/quality contract;
- compare the current availability/quality mask with the last accepted market-local

baseline and record transition reasons;

- if a required field degrades (fresh -> stale beyond ceiling, available ->

missing, valid -> conflict/quarantined), abstain from any new long whose eligibility depends on renormalizing around that degradation;

- apply the existing thin-evidence floor, but do not treat two dimensions as a universal assurance when a required gate field is absent;

- existing holdings continue through PositionMonitor's normal risk/exit logic;

evidence outage cannot suppress stops or mandatory exits; and

- emit an aggregated health event, not one alert per symbol.

This guard is deterministic and records baseline/current masks, score/rank counterfactuals, reason codes, and policy/evidence IDs. It defaults to abstain for new entries, never to a more permissive score.

7. Release Evidence Floors

Calendar days alone are insufficient. For each market and intent family require:

- at least ten distinct trading sessions within the selected session's prior 45 calendar days and a representative liquid universe;
- symbol-intent sample counts and sector/industry coverage declared in the report;
- coverage non-inferiority with uncertainty, not only a point estimate;
- organic cache/live/fallback/no-data cases plus forced timeout, quota exhaustion, schema drift, provider disagreement, and complete-primary-provider outage drills;
- zero unexplained entry flips and zero market/currency/provenance failures; and
- owner review of every material semantic divergence.

Exact numeric sample floors and tolerances are approved from observed traffic before release and then versioned. They cannot be weakened in Settings. India must satisfy its own floors; US results provide no evidence for India.

8. Cutover Sequence

- Complete score-field ownership and legacy-to-canonical compatibility mapping.
- Add frozen dual-run evaluation and activation binding.

- Add runtime degradation guard and cohort-rank tests.
- Keep legacy acquisition warm; run and document a rollback drill.
- Pilot a narrative-only intent if useful, then one score-affecting intent family in one market behind a code-owned compatibility adapter.
- Create and activate a new `router_enabled=true` policy only after all gates pass.
- Run candidate primary plus legacy reverse-shadow from shared frozen inputs.
- Expand intent families, then the second market, through separate evaluations.
- Remove legacy code only after a separately approved observation period and a release proving no rollback dependency remains.

Do not flip all intents or both markets together.

9. Rollback And Circuit Breakers

Rollback creates/activates a complete disabled/legacy policy version; it does not mutate history. Before cutover, prove that legacy credentials, pacing, cache, schemas, and code still work under realistic load. Define rollback SLO and owner/automatic triggers for:

- unexplained new eligibility or rank displacement;
- required-field coverage below the approved floor;
- cross-market/currency/basis/provenance failure;
- sustained schema/contract errors, circuit open, or quota exhaustion; and
- evidence-record persistence failure.

Automatic rollback may switch acquisition policy but cannot alter signals/orders already created. New-entry research pauses during ambiguous partial rollback; the normal position/execution safety system continues for holdings and exits.

10. Data And Audit Changes Required Before Build

Extend `evidence_policy_evaluations` or a linked append-only detail table to retain:

- frozen cohort/snapshot and strategy/genome fingerprints;
- candidate/baseline policy and evaluation-code versions;
- per-field semantic/quality/availability deltas;
- score, rank, direction, and eligibility before/after;
- classified flip cause and reviewer disposition;
- coverage denominators, outage-test results, call usage, and expiry; and
- activation/rollback proof.

Do not store only aggregate `eligibility_flips`. RLS is owner-read, writes are service-role-only, RPCs use fixed `search_path`, grants exclude anon/authenticated, and evaluation rows are append-only.

11. Required Tests

- policy and strategy freeze across mid-run activation;
- full-cohort ranking under one added/removed/degraded dimension;

- false-to-true eligibility caused by renormalization is blocked;
- genuine same-contract value change is classified, not confused with availability;
- added coverage cannot bypass activation review;
- stale/conflict/quarantined/missing and zero are distinct;
- required-field gate overrides the generic two-dimension floor;
- reverse shadow makes no duplicate live provider burst;
- unsupported India intents abstain without reducing US coverage;
- policy activation rejects stale/wrong-market/wrong-baseline evaluation;
- rollback under load restores the proven legacy path within the SLO;
- position exits and all execution brakes remain active during Router outage; and
- no Router/provider configuration can call an order adapter.

Release still requires typecheck, full tests, production build, schema/RLS/grant verification, read-only production probes, and per-market shadow/rollback reports.

12. Acceptance

Cutover is acceptable only when a future run can reproduce exactly which policy, facts, availability transitions, score, rank, and gates produced each decision; a provider outage cannot create a new long; both markets can roll back independently; and the owner sees an explicit evaluation rather than a generic "parity passed."

13. Owner Decisions Before Build

- Approve the intent/field ownership matrix.
 - Approve observed sample floors, tolerances, evaluation expiry, and rollback SLO.
 - Choose the first market and first score-affecting intent family after shadow data
- is sufficient.
- Approve the exact legacy-retirement observation period separately from cutover.

14. Implementation Notes - Prerequisites Built (2026-07-16)

NOT A CUTOVER. router_enabled remains false for BOTH markets; no enabled

policy version exists; no scorer consumes EvidenceEnvelope. Verified in

production after this change: both markets' active policy version is v1,

router_enabled=false, and zero evaluations exist. This section records the

*machinery that must exist *before* a future owner-approved cutover.*

?3 - Intent classification (lib/evidence/intent-classification.ts)

Code-owned. `classifyIntent(intent, market)` returns `score_affecting | eligibility_affecting | narrative_only | unsupported`. Per ?3: `analyst.consensus` is `narrative_only` in US, `unsupported` in India, and `gatedIntents()` excludes it in both - it can never block a scoring cutover, and India parity for it is not required. `SCORER_FIELD_CONTRACTS` declares, per field: required intent, minimum field contract (`minFields/minCount`), acceptable quality + freshness ceiling, allowed bases, structural applicability, and the legacy module/symbol it maps to today. `contractClassViolations()` is asserted by tests, so a narrative intent cannot quietly acquire a scoring dimension.

Required (gating) fields: `technical.daily_bars` (all shapes) and `fundamental.reported_core` (equity/ADR only). Everything else is optional and renormalizes exactly as today. `sentiment/insider` are deliberately NOT required - they are genuinely sparse (india:sentiment, us:insider), and requiring them would abstain on healthy runs.

?6 - Runtime degradation guard - SHIPPED IN MEASURE-ONLY MODE

- Pure core: `lib/evidence/degradation-guard.ts`. Impure shell:

`lib/evidence/degradation-runtime.ts`. Wired into `lib/research-agent.ts` immediately after `resolveSignalDirection`.

- Mode: `EVIDENCE_DEGRADATION_GUARD_MODE ? off | measure_only | enforce`,

defaulting to `measure_only` - it records what it *would* abstain and changes no direction. An unparseable value falls back to `measure_only`: a broken config can never silently enforce, nor silently stop recording.

- Strictly subtractive. `applyDegradationGuard` has exactly one

transformation: `long -> neutral`. `short` (deterministic exit on a holding) and `neutral` return untouched in *every* mode

- checked before the mode branch, so no configuration can make the guard interfere with a sell. This is also enforced in the schema (`degradation_guard_is_subtractive CHECK`), so no future code path can persist a guard event that created or upsized an entry.

- Trigger: a *required*, *applicable* field is unusable and the scorer

renormalized around it (?6.3 - "eligibility depends on renormalizing around that degradation"). A required field unusable but *not* renormalized around did not shape the decision, so it does not abstain. ?6.4 holds: the required-field gate fires even when ≥ 2 other dimensions are present.

- Defaults to abstain: no baseline + unusable required field ->

`no_baseline_required_field` -> abstain. A persistent outage keeps abstaining; it never normalizes, because only a clean run is promoted to baseline.

- One current-condition health event per market (`evidence-degradation:<market>`),

never one per symbol or historical run; the detail names the current run and a clean run resolves the condition. Run-level history belongs in the append-only degradation-event ledger, not System Health.

- Known limitation (honest): the legacy path supplies availability and contract

satisfaction but *not* observation age, so `ageSeconds` is reported as `null` rather than an invented zero. In practice the legacy path therefore exercises `available->missing` and `contract_broken`, not `fresh->stale`. The core handles age/quality transitions and they are unit-tested; real ages arrive with the Router once an intent family is cut over.

?4/?5 - Frozen dual-run evaluation + activation binding

- `lib/evidence/evaluation/parity.ts` - semantic comparator. Semantic axes

(nullability -> provenance -> basis -> period -> currency -> unit -> adjustment -> conflict) are all checked before any value comparison, so a numeric tolerance is unreachable while the two values describe different facts. Cross-family bases (TTM/quarterly/annual/forward) and adjusted/unadjusted are hard mismatches even when the numbers are identical. Unknown fields default to exact-match.

- `lib/evidence/evaluation/cohort.ts` - freezes universe/as-of/baseline/candidate/

strategy/threshold/ranks/price basis; scores both paths with the production scorer and production direction gate (never a re-implementation); ranks cohort-wide; classifies every flip with a bounded cause. Rows where either path abstains or fails are first-class. Artifact causes (availability, stale fallback, field omission, conflict resolution, basis mapping, renormalization) can never create eligibility; a `genuine_value_change` is classified as such but cannot self-approve. Becoming **more** conservative never blocks.

- `lib/evidence/evaluation/persist.ts` + `activate_evidence_policy_bound()` - the

RPC binds approval to candidate + baseline + evaluation ID + evaluation code version + strategy version + market + expiry, and additionally requires the evaluation's baseline to still be the **active** policy and every flagged divergence to carry an approving review row. Production-probed: unknown/stale evaluation and invalid market are both refused.

?8 - Massive candle adapter split (Codex pre-cutover finding)

`price.daily_bars` was one massive adapter wrapping `fetchUsCandles()`, which silently fell back Massive->EODHD->TwelveData while reporting `providerId: "massive"`. Three harms: provenance named a nominal source; mode: "only" could not actually pin a provider; three providers' pacing/budget accounted as one. Now `lib/evidence/adapters/bars.ts` builds one adapter per source (`massive-bars-v1`, `eodhd-bars-v1`, `twelvedata-bars-v1`), each hitting exactly one provider and refusing a payload that claims another source. The router owns the fallback via the registry chain. Contract version bumped from `us-bars-v2` so multi-source cache rows are not read back under the new single-source contract. Note: the resolver's `MAX_SYNC_ATTEMPTS=2` means the third source is reached via the refresh queue, not synchronously - a deliberate Vercel wall-clock bound.

Migration

`supabase/migrations/20260716210000_router_cutover_prerequisites.sql` -> `evidence_field_baselines` (mutable - it is a moving reference point, not evidence), `evidence_degradation_events`, `evidence_evaluation_details`, `evidence_evaluation_reviews` (all append-only via `no_mutate`), plus frozen-cohort and binding columns on `evidence_policy_evaluations`. RLS on + owner-read policy on all four; anon has no grants; writes are service-role only; the RPC is security definer with `search_path=public` and executable by `service_role` only.

Deferred after the 2026-07-21 hardening build

- Rollback SLO and any future weakening of approved floors remain owner decisions.

Section 16 locks the current session, coverage, score, rank, and TTL values.

- The outage drills (?) - forced timeout, quota exhaustion, schema drift, provider

disagreement, and complete-primary-provider outage remain unbuilt.

- ?9 rollback drill + circuit-breaker triggers.

- The owner-only activation API exists, but no general evaluation-management UI

is built and both active policies remain disabled.

15. Implementation Notes - Cohort Builder Built (2026-07-18)

Historical v1 record. Section 16 supersedes its provider-burst allowance,

availability-only scoring, placeholder thresholds, and manual-only schedule.

STILL NOT A CUTOVER. Verified in production immediately before and after this

change: both markets' active policy is v1 with router_enabled=false, and no

scorer consumes EvidenceEnvelope. This step only makes the FIRST parity

evaluations exist. It never activates anything.

What was built

lib/evidence/evaluation/cohort-builder.ts - the piece ?14 deliberately deferred because "a builder that fetches is the step that risks provider bursts." Exposed as GET/POST /api/agents/evidence-cohort?market=us|india&limit=N[&dryRun=1] (owner- or cron-gated, mirroring evidence-shadow).

?4.2 - ONE frozen observation set, and why the reuse is structural

The governing constraint is that a dual-run must not double provider calls. The build resolves one market-local frozen observation set via the EXISTING resolveEvidence (allowDisabledPolicy: true), which shares evidence_cache_v2 with the evidence-shadow harness - so a warm cache short-circuits at zero cost.

assembleCohort is pure and takes the snapshot as an argument. It has no resolver and no network. The reverse-shadow leg therefore *cannot* re-fetch: its only path to evidence is the frozen Map. The reuse is a structural property, not a caching optimisation that a later refactor could quietly undo.

An earlier draft resolved twice (primary + reverse) and was caught by its own ledger proof - the reverse pass made 12 real bursts. That is precisely the defect the constraint exists to prevent, and it is why verifyReverseShadowReuse now confirms every (symbol, intent) is served from the frozen set rather than re-resolved.

The two legs - and what v1 honestly compares

- legacy = a real recorded production decision. agent_signals joined to its

research_packets.raw_data supplies the exact persisted availability mask (_data_quality) and the weights that actually drove the score (_profile_weights). The mask is rebuilt with research-agent's own rule, so the leg reproduces the recorded analyst_score exactly - asserted per symbol and reported as legacyReproduction (27/27 matched across the first two runs). A reconstruction that did not reproduce production would be a guess, not a baseline.

- candidate = the SAME frozen dimension scores, with availability decided by the router snapshot.

v1 scope is AVAILABILITY + eligibility-flip parity, and says so. Legacy never persisted raw field values or per-field provenance, so a value/basis/period comparison against it would be fabricated. Both legs therefore carry the field's canonical contract semantics, value/periodEnd are left null (not compared), and the real serving provider is recorded for audit. Note the consequence, stated plainly: because both values are null, compareField short-circuits at the nullability axis, so basis/period/currency/unit are not exercised on the production path in v1 - they are exercised in tests, and become live when the legacy path is itself routed. This mirrors the honesty already shipped in the degradation guard (ageSeconds: null).

Dimension scores are not re-derived from router evidence - no scorer consumes EvidenceEnvelope yet, and inventing one here would mean the evaluation stopped predicting the thing it claims to predict. The production scorer and production

direction gate are reused unchanged, via evaluateCohort.

First evaluations (real, persisted)

```
| Market | Symbols | Passed | Flips | Finding |
| US | 15 | **yes** | 0 new-eligible / 0 new-ineligible | Required fields at parity: `technical.daily_bars`
and `fundamental.reported_core` both 15/15 vs 15/15. Router **gained** insider coverage (7->10, +20%) -
recorded as `availability_gain`, measured not approved. Lost `macro.regime` (-100%) and `sentiment.
news_tone` (-86.7%): neither intent has a registered adapter yet, and neither is a required field. |
| India | 12 | **no** | 0 new-eligible / 10 new-ineligible | `coverage_below_non_inferiority_margin` on
`technical.daily_bars`: candidate 0.0% vs legacy 100.0%. Correct - every `price.daily_bars` adapter is
`markets: ["us"]`, so the router cannot serve India's required field at all. |
```

The India failure is the gate working, not a defect: US results provide no evidence for India (?7), and the two were evaluated independently. Scores moved materially in the US cohort (deltas -28 to +5) without a single threshold crossing - verified per symbol rather than inferred from the zero-flip count.

Ledger proof (?4.2), from provider_call_ledger

```
| Run | Ledger rows | Fresh cache | Real bursts |
| US primary | 50 | 40 | 10 |
| US reverse-shadow | **0** | 0 | **0** |
| India primary | 12 | 6 | 6 |
| India reverse-shadow | **0** | 0 | **0** |
```

The reverse leg wrote zero ledger rows while serving all 75 (US) and 60 (India) (symbol, intent) pairs from the frozen set. Dual-run provider cost therefore equals the single primary pass - the primary's bursts are the explicit cache misses the constraint allows. A denied lease is deliberately not counted as a burst: it is the pacing system refusing a call, i.e. the protection working.

Deliberately NOT done

- No migration. Every column persistEvaluation writes already existed.
- No activation, and no change to router_enabled (still false, both markets).
- ?7 numeric floors, tolerances, and evaluation TTL remain owner decisions (?13.2).

The builder's coverageNonInferiorityMargin (0.05) and maxAdverseRankDisplacement (5) are placeholders pending that approval, not approved thresholds.

- ADR shaping: shapeOf maps every non-ETF/non-metal symbol to equity, so a US ADR

over-includes the insider field. That is conservative (a coverage miss, never a fabricated eligibility) but it is an approximation, not a modelled distinction.

16. Approved Evidence-Hardening Build (2026-07-21)

Decision record

- Why: the daily Router shadow is fresh, but the cutover cohort ran only once,

could spend provider quota on cache misses, and its single passed bit did not distinguish entry safety from loss of useful score evidence.

- Who: the single Kairos owner operating separate US/USD and India/INR books.

- ROI: make cutover evidence repeatable and quota-neutral while preventing a

technically conservative but materially blinder candidate from being described as full parity.

- Shipped at: 866548b3 + cac4cffa; production verified 2026-07-21.

Scope and invariants

- Cohorts are cache-only. A cohort may read fresh or policy-allowed stale

evidence_cache_v2 rows. It may not acquire a provider lease, enqueue a refresh, or call an adapter. A cold field remains unavailable and the evaluation records the miss. primary_provider_calls must equal zero or the evaluation fails.

- Research-output compatibility bridge. During the existing ResearchAgent

fetch, Kairos writes canonical cache rows from the data already in memory: fundamentals, bars, sentiment, macro, and insider. This is one bounded database upsert, not another provider request. Provenance names the compatibility bridge and retains the actual serving source in the canonical payload. It is a transition adapter, not a new source; future provider-native policies must be evaluated separately before removing legacy acquisition.

- Market-wide evidence is market-wide. macro.regime_inputs uses the reserved

__MARKET__ cache key and is resolved once per market/run. It is never fanned out into one provider/cache operation per symbol. India macro remains unavailable: the US MacroSentinel cannot be relabelled as India evidence.

- Exact score-input replay. Bridge payloads carry the deterministic dimension

score and the score input/evidence that produced it. The candidate leg uses that score rather than reusing the legacy score by assumption. Provider-native canonical payloads must derive the dimension score through the same production scoring functions before they can satisfy this gate.

- Two machine verdicts. safety_pass covers required-field floors, semantic

failures, and artifact-created new eligibility. quality_pass covers all score-affecting coverage loss, material total-score drift, and adverse rank displacement. passed = safety_pass AND quality_pass; activation requires all three fields true.

- Rolling proof. An enabled policy cannot activate from one snapshot. The

bound activation RPC requires ten distinct market-session evaluations for the same market/candidate/baseline/evaluation-code/strategy tuple, all safety- and quality-passing within the selected session's prior 45 calendar days, with the selected evaluation still unexpired. The session date comes from executable ResearchAgent rows (session_validated=true, as_of_session), so weekend/holiday staged rows cannot inflate the count. US and India histories never satisfy one another.

- Schedule only after hardening. Run one cohort after each market's final daily

shadow tick. The route remains shadow-only and both active policy versions keep router_enabled=false.

- Evidence binds the real candidate. Seed one immutable, inactive

router_enabled=true candidate per market by copying that market's active rules. Cohorts resolve cache order under that exact candidate ID. The active pointer is not changed, so production remains on the disabled baseline until the bound activation RPC eventually clears every gate.

Approved numeric floors

- Rolling sessions: 10 distinct market-local sessions.

- Required-field coverage non-inferiority margin: 5 percentage points.

- Any loss of an optional score-affecting dimension is a quality failure when the cohort loss exceeds 5 percentage points; structurally inapplicable fields are excluded from the denominator.

- Material aggregate score drift: absolute candidate-vs-legacy difference greater

than 2 points for any symbol is a quality failure until owner-reviewed under a future provider-change policy.

- Missingness-caused adverse rank displacement: 3 or more places is a quality failure.

- Evaluation validity remains 72 hours; this is freshness for the selected evaluation, not a substitute for the ten-session history.

Data changes

Add immutable evaluation columns `safety_pass` `boolean not null`, `quality_pass` `boolean not null`, and nullable `market_session_date` `date` sourced from validated ResearchAgent sessions (old rows have no valid session proof). Existing evaluations default both verdicts false, so an old availability-only run can never authorize cutover. Harden `activate_evidence_policy_bound()` to require the selected row and the rolling ten-session history. No new provenance table is introduced: bridge payloads live in the existing Router cache and immutable decision detail remains in `evidence_policy_evaluations / evidence_evaluation_details`.

Acceptance

- A cohort test fails if its resolver attempts a provider on a cache miss.
- A warm compatibility row reproduces its recorded dimension and aggregate score.
- Losing macro or sentiment can remain entry-safe but cannot be `quality_pass=true`.
- Macro produces one market-key lookup regardless of cohort size.
- India bars become observable from the Upstox/Yahoo candles already fetched by ResearchAgent; no extra Upstox/Yahoo request is made.
- The recurring jobs persist evaluations but cannot activate a policy.
- Every evaluation binds the inactive enabled candidate and the currently active disabled baseline; a baseline/candidate change starts a new ten-session series.
- The activation RPC refuses old evaluations, fewer than ten passing sessions, wrong-market history, or any safety/quality failure.
- Both production policies remain `router_enabled=false` after deployment.

Production verification (2026-07-21)

- Applied migrations `router_evidence_hardening` and `router_cohort_session_source` in the FinanceOS Supabase project.
- Vercel production deployments for both implementation commits reached Ready.
- US evaluation `a2226bb7-e114-464d-96b8-01ff3684a85a` bound candidate v2 to baseline v1 for session 2026-07-20. India evaluation `78770c01-4583-49ac-b605-2e82f1730458` did the same for session 2026-07-21.
- Both evaluations recorded zero primary and reverse provider calls with ledger proof holding. Both failed safety and quality on current cache coverage, which is the required fail-closed startup behavior rather than permission to cut over.
- Active US and India policies remain v1 with `router_enabled=false`. The bound

activation RPC is executable by service_role only.

- Gates: TypeScript clean; 1,162 tests passed / 6 skipped; production build clean.

scoring data truth - Feature Architecture

Source: `features\scoring-data-truth\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Scoring Data-Truth Audit

Status: Corrective fixes implemented; semantic redesigns remain proposed Date: 2026-07-31 Markets: US and India, always evaluated separately Money path: Yes. Research scores can authorize paper and future live entries.

1. Problem

The scorer's formulas were individually plausible, but provider payloads did not always satisfy the formula's assumed contract. Static review and synthetic tests therefore passed while production data produced systematically distorted scores. The critical example was treating Finnhub `finnhubIndustry` as a GICS sector and silently applying a P/E norm of 20 whenever its value did not match a GICS key.

Finnhub documents Profile 2 as returning its own `finnhubIndustry` classification; the free response does not provide the paid profile's GICS `gsector` field: <https://www.finnhub.io/docs/api/crypto-profile>

2. Production Findings Reproduced

Snapshot queried on 2026-07-31.

```
| Finding | Production result |
| Finnhub fundamental rows, last 7d | 425 |
| Rows silently using unmatched/default P/E 20 | 291 (68%) |
| Finnhub P/E median / p90 / max | 30.56 / 153.84 / 8,827.05 |
| Finnhub P/E > 200 | 31 / 425 |
| US technical score exactly 20, last 45d | 320 / 1,635 (19.6%) |
| India technical score exactly 20 | 55 / 405 (13.6%) |
| US technical score exactly 100 | 315 / 1,635 (19.3%) |
| India technical score exactly 100 | 163 / 405 (40.2%) |
| Usable dimensions, last 7d US | technical 687; sentiment 664; macro 687; insider 288 |
| Usable dimensions, last 7d India | fundamental 200; technical 200; sentiment/macro/insider 0 |
```

Availability is read from `decision_observations.features.weighting.included_dims`, not from non-null score columns. Score columns retain neutral placeholders even when a dimension is excluded.

India sentiment is wired, but GDELT supplied zero usable observations in the measured window. The defect is availability, not missing source code. Core learner, validation, champion, and performance records are market-scoped; the unresolved issue is score comparability, not direct US/India pooling on the money path.

3. Corrective Build

3.1 Fundamental taxonomy and P/E

- Provider mappers stamp `SectorTaxonomy`.
- Known Finnhub industries use an explicit, reviewable industry-to-sector crosswalk.
- Ambiguous or unknown industries do not receive a fabricated sector norm.
- P/E values outside $(0, 200]$ are unpriceable for this component. They are omitted, not winsorized and not assigned a maximum bearish penalty.
- Other valid fundamental fields continue scoring when P/E is omitted.
- Evidence records expose mapping and omission status.

MAX_SCORABLE_PE=200 is a data-sanity bound, not an alpha threshold. The valuation ladder already saturates long before 200, so larger values add no ranking information and are commonly caused by near-zero trailing earnings.

3.2 Breakdown veto

Only either of these can hard-veto:

- A down move of at least 2.5 ATR.
- A decline of at least 7% on at least 1.5x average volume.

A bottom-quartile close on an ordinary down day remains evidence as a warning but no longer independently forces technical score 20. Matured labels did not support that condition as a hard classifier: US h2 was positive on average with a 61.8% win rate, and India h5 was positive with an 80% win rate in the available small cohort.

3.3 Availability and one scorer

- Fundamental, sentiment, macro, and insider inclusion require explicit `dataQuality.*Available === true`; missing fields fail closed.
 - A macro row with missing/out-of-range `danger_score` is unavailable.
 - `available=true` insider evidence without a finite 0..100 score is unavailable.
 - The Supabase research-agent Edge Function is retired with HTTP 410.
- `/api/agents/research/cron plus lib/research-agent.ts` is the sole scorer.
- The US scanner fallback is market-scoped. Deep Dive reads same-market signals and holdings and never attaches the US macro regime to India.

4. Counterfactual Impact

A read-only SQL reconstruction over 21 days applied the new P/E mapping, P/E omission, and technical-veto rule to frozen observations and their recorded effective weights.

Market	n	Old pass	Estimated new pass	Up flips	Down flips	Old avg	Est. new avg
US	1,461	640 741	106 5	56.97	58.49		
India	368	274 284	17 7	73.62	75.49		

This is diagnostic only. It infers historical provider taxonomy where old evidence did not stamp it and does not replace a point-in-time replay. Historical signals, positions, and trades are never rewritten. New behavior begins with future research.

5. Still Proposed, Not Implemented

These change strategy semantics and require owner approval plus shadow evidence:

- Insider redesign. Shadow current versus `exclude-insider` and a purchase-only event feature. Do not invent a new baseline from the same small sample.
- Per-market threshold calibration. The mandate already stores separate thresholds.

Calibrate precision, return, pass rate, and turnover per market; do not force numeric score equivalence across different available dimensions.

- True PEAD. Add point-in-time actual-versus-consensus earnings surprise and drift as a shadow archetype. Options-implied move remains risk, not surprise direction.
- India exit policy. Diagnose time-stop concentration against MFE/MAE and fresh

score paths before changing hold periods or exit thresholds.

- Technical ceiling. Scores at 100 are saturated (US 19.3%, India 40.2%). Evaluate a versioned shadow formula; do not tune the live formula from this audit.

6. Acceptance Invariants

- No unknown provider taxonomy silently receives a valuation benchmark.
- No invalid P/E can become confident bearish evidence.
- A weak close alone cannot hard-veto.
- Missing availability metadata excludes a dimension.
- Exactly one production component can write authoritative ResearchAgent scores.
- No corrective deployment writes an order or mutates historical trades.

scoring methodology - Feature Architecture

Source: features\scoring-methodology\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Scoring Methodology Upgrade - Feature Architecture

Last updated: 2026-07-10 Status: REVIEWED / IMPLEMENTATION-READY, but every rollout phase still requires Vaibhav approval Authors: Claude Sonnet 4.6; reviewed and materially corrected by ChatGPT/Codex, 2026-07-10 Scope: US equities, US ETFs, and India NSE equities; long-only new positions; 2-20 trading-day swing horizon.

1. Objective and honest benchmark

Kairos must rank opportunities reproducibly, abstain when evidence is weak, learn only from point-in-time outcomes, and promote a scoring version only after net-of-cost out-of-sample evidence. No public document describes the proprietary "best scoring system in the world," and no architecture can guarantee profit. The defensible target is the process used by serious systematic research teams:

- point-in-time universe and data;
- compact, economically motivated features;
- cross-sectional expected-return ranking with uncertainty;
- independent portfolio/risk construction;
- purged walk-forward validation, costs, and multiple-testing controls;
- shadow -> paper -> small-live promotion with rollback.

Research supports starting with momentum/trend, liquidity, volatility, industry-relative strength, and carefully lagged fundamentals. Nonlinear models can help when there is enough broad historical data, but shallow/regularized models are the correct first production model for Kairos's current sample. References are in ?14.

Current assessment

```
| Layer | Current state | Target |
| Candidate discovery | Mixed static/watchlist/screener sources | PIT universe snapshot + discovery-source attribution |
| Features | Five coarse 0-100 dimensions | Compact raw features with source/as-of/quality metadata |
| Ranking | Renormalized linear heuristic | Setup-specific alpha score + comparable-universe percentile |
| Direction | LLM can return long/neutral/short | Deterministic entry/exit decision; LLM explanation only |
| Confidence | Count/availability semantics | Structural applicable-weight coverage + model uncertainty |
| Learning | Five-weight challenger | Versioned feature/model/setup genome; statistics proposes, human promotes |
| Validation | Existing walk-forward and IC pieces | Purged nested walk-forward, costs, trial accounting, OOF calibration |
```

The existing five-dimension scorer remains a transparent baseline (deterministic_v1). It must not be relabeled deterministic_v2.

2. Non-negotiable invariants

- An LLM may not generate analyst_score, rank_score, expected return, probability, direction, weights, size, or eligibility.
- DeepSeek comparison signals are llm_advisory, never pending, and never consumable by PaperTrader or TraderAgent.

- New positions are long-only. A held-position exit is `exit_candidate/SELL`, never a synthetic "short" forecast.
- Missing, stale, failed, degraded, and structurally inapplicable data are distinct.
- The denominator of evidence confidence is structurally applicable base weight, never post-renormalization `applied_weights`. The numerator includes only fresh, valid data; degraded dimensions are excluded.
- No model is trained or evaluated on information unavailable at `decision_ts`.
- Cross-sectional transformations use a recorded reference universe for the same market/date. The three daily candidates are not a valid reference universe.
- A scoring version cannot become paper-active or live-eligible merely by changing a string or Settings value.
- LLM explanations can veto/reduce a candidate only through a bounded, evidence-citing schema; they can never rescue a failed deterministic gate.
- Scoring chooses opportunities. Portfolio construction and Execution Gateway independently decide whether and how much may be traded.

3. Target pipeline

```
PIT universe snapshot
-> provider observations (value, source, observed_at, available_at, retrieved_at)
-> feature snapshot + per-feature quality
-> deterministic eligibility/liquidity/event gates
-> asset/setup router
-> setup raw-alpha models
-> comparable-universe rank + uncertainty/evidence adjustment
-> deterministic long_candidate | watch | abstain | exit_candidate
-> LLM explanation / bounded veto (optional; never rescue)
-> agent_signals summary + append-only decision_observations canonical snapshot
-> matured labels at 2/5/10/20 trading days
-> IC, precision@K, calibration, net return, turnover, drawdown
-> purged walk-forward challenger comparison
-> shadow -> paper -> live-review lifecycle
```

Separation of concerns

- Universe layer: defines what could have been selected at the time.
- Alpha layer: estimates relative expected return and uncertainty.
- Decision layer: applies evidence, event, contradiction, and mandate gates.
- Portfolio layer: sizes from calibrated edge, volatility, correlation, current book, and caps.
- Execution layer: validates account, quote, limits, budget, idempotency, and broker state.

No layer may silently absorb another layer's responsibility.

4. Canonical contracts

```
type Market = "us" | "india";
type AssetType = "equity" | "etf" | "adr";
type DataState =
  | "ok"
  | "inapplicable"
  | "missing"
  | "stale"
  | "provider_failed"
  | "degraded";

interface FeatureValue<T = number> {
  value: T | null;
  state: DataState;
  source: string;
  observedAt: string | null; // market timestamp of the observation
  availableAt: string | null; // first timestamp Kairos could legally know it
  retrievedAt: string;
  staleAfter: string | null;
}

type SetupType =
  | "quality_momentum"
  | "value_inflection"
  | "post_earnings_drift"
  | "etf_trend"
  | "india_quality_momentum"
  | "india_sector_rotation";

interface ScoringFeatureSnapshot {
  schemaVersion: "1";
  scoringVersion: string;
  symbol: string;
  market: Market;
  assetType: AssetType;
  decisionTs: string;
  marketSessionDate: string;
  universeSnapshotId: number;
  benchmarkSymbol: string;
  sectorId: string | null;
  features: Record<string, FeatureValue<number | boolean | string>>;
  sourcePayloadHashes: string[];
}

interface SetupScore {
  setupType: SetupType;
  rawAlphaScore: number; // stable deterministic composite; not a probability
  rankScore: number; // 0..100 percentile within recorded comparable universe
  evidenceConfidence: number; // 0..1 structural coverage, ?7
  contradictionPenalty: number; // 0..1 direct penalty, ?8
  regimeScaler: number; // bounded [0.75,1.00] in initial release
  finalScore: number; // rankScore x confidence x (1-penalty) x scaler
  pWin: number | null; // calibrated OOF model only
  expectedReturnBps: number | null; // calibrated OOF model only, net estimate separate
  predictionIntervalBps: [number, number] | null;
  action: "long_candidate" | "watch" | "abstain" | "exit_candidate";
  reasonCodes: string[];
  scoreSource: "deterministic_v1" | "deterministic_v2" | "llm_advisory";
  scoringVersion: string;
}
```

`analyst_score` remains a compatibility/display field during migration. For v2 it equals rounded `finalScore`; consumers must use `score_source`, `scoring_version`, and lifecycle eligibility rather than treating every 0-100 number as equivalent.

5. Universe and comparable groups

Every run persists the eligible universe before scoring, including inclusion reason, market, asset type, sector, liquidity fields, and effective timestamps. Historical validation must not apply today's constituents to the past.

Comparable groups:

- US equity: market/date/sector where sample ≥ 20 ; otherwise market/date/asset type.
- US ETF: market/date/ETF category (broad, sector, thematic, fixed income where supported); never rank ETFs against companies.
- India equity: market/date/NSE sector where sample ≥ 15 ; otherwise market/date/large/mid-cap liquidity bucket.

Winsorization and z-scores are fitted within the training/reference group only. If the group is too small, use pre-registered fixed transforms, mark `rank_quality=degraded`, and do not invent a percentile from the three finalists.

Universe membership must record survivorship limitations. The current `edge_universe_members` list is useful for measure-only work but is not PIT-safe proof.

6. Feature set and setup experts

Shared price/liquidity features

- 5/20/60/120 trading-day total return (120d may be absent in early phases)
- 12-1 momentum when enough history exists
- benchmark- and sector-relative 20/60d momentum
- realized volatility and downside volatility
- distance to 52-week high
- volume surprise using prior 20 sessions (exclude current bar from denominator)
- average traded value and, where available, spread/price-impact proxy
- EMA20/50/200 state; EMA is a state feature, not four duplicate indicators

Corporate actions must be adjusted consistently. A split/dividend discontinuity cannot become momentum or a fake drawdown.

Fundamental/event features

- sector-relative value and quality composites using only fields with `PIT available_at`
- earnings surprise/revision and post-event age
- next earnings/ex-dividend dates as event/risk metadata
- stale filing age and provider quality

If PIT availability cannot be proven, the feature is excluded from training and trading, not backfilled with today's value.

India-specific context

- NIFTY 50 and sector-relative strength
- India VIX state
- INR/USD and crude-oil change as bounded context features

- FII/DII flow only after a reliable timestamped source exists

US macro_regime must not be reused as India macro evidence. India context begins as a small scaler/feature and earns influence through validation.

Setup experts

The six existing archetypes remain, but formulas are priors to validate, not truths:

```
| Setup | Asset | Load-bearing evidence | Initial role |
| `quality_momentum` | US equity/ADR | positive 20/60d trend, liquidity, quality if PIT | primary |
| `value_inflection` | US equity/ADR | PIT relative value/quality + technical stabilization | shadow until
PIT fundamentals proven |
| `post_earnings_drift` | US equity/ADR | timestamped event + surprise/revision + volume/trend | shadow
until event feed proven |
| `etf_trend` | US ETF | category-relative trend, liquidity, vol | primary |
| `india_quality_momentum` | India equity | NIFTY/sector-relative trend + liquidity + PIT quality | primary
price-only first |
| `india_sector_rotation` | India equity | stable timestamped sector-index data | shadow until source
reliability proven |
```

Router rules are deterministic and pre-registered. An event-qualified symbol can be evaluated by the event expert; otherwise the asset-specific priority applies. If multiple experts qualify, persist all shadow scores but only the pre-declared champion expert may create the actionable signal. This prevents after-the-fact selection of whichever model looked best.

Initial raw-alpha models are monotonic, regularized linear composites over transformed features. Do not hardcode arbitrary $z(\dots)$ formulas until the comparable universe and transformation definition exist. Gradient-boosted trees or shallow neural nets are P5 research candidates only after a broad PIT dataset, nested validation, and a linear baseline comparison.

7. Evidence confidence - exact math

For setup s , let w_f be the pre-registered structural weight of feature group f . Let A_s contain only groups structurally applicable to the symbol/setup.

```
denominator = ?  $w_f$  for  $f \in A_s$ 
numerator   = ?  $w_f$  for  $f \in A_s$  where state( $f$ ) = ok and freshness passes
data_confidence = numerator / denominator
```

- inapplicable is omitted from both numerator and denominator.
- degraded, missing, stale, and provider_failed remain in the denominator and contribute zero to the numerator.
- Never use post-renormalization applied_weights in either term.
- A missing hard-required feature causes abstain regardless of aggregate confidence.
- A malformed/zero denominator produces quality_status=unknown and abstain for actionable paths.

This must align with v_decision_quality and features.weighting.base_weights; the v2 writer must preserve the structural weights, included feature groups, and quality states in decision_observations.features.

Initial policy:

- < 0.60 : abstain;
- $0.60 - 0.74$: shadow/watch only;
- ≥ 0.75 : eligible for the remaining gates.

These are policy defaults, not learned alpha thresholds. Auto-live may require a stricter threshold and never accepts a quality override.

8. Contradictions and direction

Contradiction penalties are direct additive values, capped at 1.0. Do not multiply a penalty by a second copy of its own weight.

```
let penalty = 0;
if (bullishEvent && relMomentum20d < -0.03) penalty += 0.30;
if (valueRank > 70 && momentum60d < -0.10) penalty += 0.25;
if (highAttention && volumeSurge < 0.80) penalty += 0.20;
if (hostileRegime && fragileLiquidity) penalty += 0.25;
return Math.min(1, penalty);
```

- ≤ 0.20 : clean;
- $0.20 - 0.40$: watch/shadow or deterministic size haircut in the portfolio layer;
- > 0.40 : abstain.

Machine-readable hard vetoes - illiquidity, upcoming earnings policy, missing required evidence, stale quote, corporate-action anomaly - run before the LLM.

Direction/action order:

- Determine whether the symbol is held in the relevant paper/live book.
- Evaluate held-position exit rules independently; output `exit_candidate` only for held quantity.
- For new positions, apply deterministic setup/evidence/contradiction/event gates.
- Output `long_candidate`, `watch`, or `abstain`.
- Ask the LLM for explanation and optional bounded veto.
- A valid veto can only downgrade `long_candidate` -> `watch`; it cannot upgrade any result.

The LLM schema may output summary, risks, catalysts, and `{vetoed, category, citedEvidenceIds}`. It may not output any numeric trading field. Invalid JSON or unavailable LLM leaves the deterministic decision unchanged and records `explanation_status=unavailable`; no LLM is required for correctness.

9. Persistence and migrations

All migrations are additive and must be applied before schema-coupled code ships. Use the next real migration number discovered at build time; do not copy `N+1` literally.

agent_signals summary columns

Add nullable columns with constraints: `score_source`, `scoring_version`, `setup_type`, `rank_score`, `final_score`, `evidence_confidence`, `contradiction_penalty`, `p_win`, `expected_return_bps`, `abstain_reason`, `reason_codes` `text[]`, `llm_veto` `jsonb`, `universe_snapshot_id`.

Do not add a second full features blob to `agent_signals`; `decision_observations.features` is the canonical PIT snapshot. Duplication would drift. A signal links to it through `signal_id`.

Add numeric range checks as NOT VALID, backfill/inspect legacy rows, then validate in a later step. Add a DB constraint so new `status='pending'` rows require an approved deterministic source; legacy rows must be explicitly backfilled, not silently accepted. RLS remains enabled and service-only. Verify table grants because current Supabase projects may not expose new tables/columns automatically.

decision_observations

This table already has features `jsonb`, `availability_mask`, and UUID `signal_id`, and is append-only. Add only missing summary columns: `score_source`, `scoring_version`, `setup_type`, `rank_score`, `final_score`, `evidence_confidence`, `contradiction_penalty`, `p_win`, `expected_return_bps`, `universe_snapshot_id`.

Never update or delete historical observations to "upgrade" them. New logic writes new rows/versioned snapshots.

Version lifecycle

Extend the existing `strategy_versions` lifecycle rather than creating an unrelated activation switch. Each scoring challenger records `scoring_version`, code/config hash, feature schema, universe definition, training cutoff, trial-family ID, and validation experiment. Lifecycle:

```
draft -> measure_only -> shadow_paper -> paper_active -> live_review_eligible -> live_approved -> retired
```

Only owner promotion changes lifecycle. Learner/LLM may propose a challenger; it cannot activate one. PaperTrader consumes `paper_active`; TraderAgent/live auto consumes only `live_approved` and rechecks the linked validation evidence.

10. Validation, calibration, and learning

Labels

- Forward returns at 2/5/10/20 trading-day horizons.
- Benchmark-neutral and sector-neutral returns.
- Corporate-action-adjusted prices.
- Labels mature only after the horizon; late/provider-revised labels are versioned, never overwritten silently.

Required evaluation

- Spearman rank IC and Newey-West uncertainty by market/setup/horizon.
- Top-K precision/hit rate and top-minus-median spread for this long-only system.
- Net-of-spread/slippage/fees/tax-assumption expectancy and turnover.
- Brier score, log loss, and expected calibration error for `pwin`.
- Sharpe/Sortino, max drawdown, benchmark alpha, concentration, and regime/year stability.
- Champion-vs-challenger paired results on the same opportunity set.
- Trial count/effective number of variants, Deflated Sharpe or a documented multiple-testing correction.

Walk-forward rules

- Purge samples whose forward-label windows overlap the test window; apply an embargo.
- Fit transforms, imputers, coefficients, and calibrators on each training fold only.
- Produce out-of-fold predictions for every validation metric.
- The current `lib/validation/calibration.ts` fits coefficients on the full dataset and then scores fold test rows with those same coefficients; that is leakage. P3 must replace this with per-fold fits and a final production fit only after OOF metrics pass.

- Do not use 60 observations as a universal green light. Require enough effective, non-overlapping observations for model complexity and both outcome classes. Initial default: at least 250 labeled rows, at least 50 effective horizon blocks, and at least 20 positive and 20 negative outcomes per fitted parameter family; otherwise keep `pwin/expected return` null.

Learning boundary

The LearnerAgent may:

- discover hypotheses/features;
- request deterministic experiments;
- propose bounded challenger configs;
- summarize failures.

It may not directly change active weights, thresholds, setup routing, code, money limits, or lifecycle state. Numeric optimizers/regularized fits produce candidate parameters; the LLM does not.

11. Data-source policy

Provider tiers and quotas change; `docs/DATA_PROVIDER_MATRIX.md` (or the existing canonical provider review) must record current plan, quota, freshness, market coverage, legal/ToS status, and fallback behavior. Do not label a source "free" in code architecture without verifying the configured account tier.

- Massive/Polygon, Alpha Vantage, FMP, Finnhub, Upstox, Robinhood, and Kite are official/API sources where configured.
- Yahoo endpoints are an unofficial, no-SLA fallback and cannot be the sole live-auto dependency.
- TradingView paid UI access does not grant an API license; CSV/manual validation is allowed, scraping/auth automation is not an architectural dependency without explicit ToS review.
- Fallbacks may supply research/shadow data only when source equivalence is validated. Provider substitution is recorded in the snapshot and can reduce confidence.
- API exhaustion results in cached-with-freshness, alternate approved provider, or abstain. Never fabricate or ask an LLM for market data.

12. Build phases and exact gates

P0 - provenance safety (no formula change)

- Migration: source/version/summary columns and safe constraints.
- Tag current ResearchAgent rows `deterministic_v1`, version `v1.0`; DeepSeek rows `llm_advisory`.
- PaperTrader explicitly requires the current `paper_active` deterministic version.
- TraderAgent requires `live_approved` scoring lifecycle for live BUY proposals.
- Replace LLM direction for new entries with a deterministic `v1` threshold/evidence gate; held exits remain separate.

P1 - measure-only feature/universe snapshots

- Add PIT universe snapshot and feature computation.
- Persist canonical snapshots in `decision_observations.features`.
- Verify corporate-action adjustment, source timestamps, reference-universe ranks, and US/India market calendars.

- No change to paper/live selection.

P2 - shadow setup experts

- Implement pure scorers and deterministic router.
- Persist all expert scores as shadow evidence; only v1 remains actionable.
- Run IC/stability/cost analysis over broad PIT opportunities.

P3 - OOF calibration and expected return

- Correct the calibration leakage.
- Fit regularized baseline models per setup/market only when sample gates pass.
- Store OOF predictions, model artifact/hash, transformation parameters, and trial-family count.

P4 - paper champion/challenger

- Owner promotes v2 to `paper_active` after validation.
- Shadow v1 and v2 on the same opportunity set.
- UI explains features, data quality, comparable group, abstentions, and outcome history.

P5 - live review

Require statistically and economically positive OOF/paper evidence, stable performance across time slices, acceptable drawdown/turnover, no unresolved data-integrity alerts, and owner promotion to `live_approved`. This only makes the signal eligible; all risk and execution gates still apply.

13. Acceptance tests

- Same snapshot + same version produces byte-stable score/reason codes.
- DeepSeek/LLM signals cannot be claimed by PaperTrader or TraderAgent even if status is accidentally set pending.
- ETF cannot enter equity setup or fundamental denominator.
- India cannot consume US macro as valid India evidence.
- Degraded/stale/failed required data remains in confidence denominator and contributes zero numerator.
- `applied_weights` are never used for confidence math.
- Contradiction penalty can reach abstain threshold; tests cover each combination.
- New unheld short/SELL is impossible; held exit quantity cannot exceed verified holdings.
- Rank computation fails to watch/abstain when the comparable universe is absent or too small.
- Current-bar volume is excluded from its own baseline; all N-day features have no off-by-one/look-ahead.
- Split/dividend-adjusted return tests for US and India.
- Walk-forward preprocessing and calibration fit only on fold train data.
- Pending actionable signal requires deterministic source and active version.
- Auto/live eligibility requires `live_approved`; score threshold alone is insufficient.

- Missing provider data produces abstain or approved fallback with provenance, never neutral 50 as real evidence.

14. Evidence basis

- Gu, Kelly & Xiu, *Empirical Asset Pricing via Machine Learning* (RFS 2020) (<https://academic.oup.com/rfs/article/33/5/2223/5758276>): nonlinear interactions can help with broad data; price trends, liquidity, and volatility are dominant predictors; shallow learning can outperform deeper models in low-signal finance.
- Harvey, Liu & Zhu, *...and the Cross-Section of Expected Returns* (https://www.nber.org/system/files/working_papers/w20592/w20592.pdf): factor discovery requires multiple-testing discipline.
- Bailey & Lopez de Prado, *The Deflated Sharpe Ratio* (<https://www.davidhbailey.com/dhbpapers/deflated-sharpe.pdf>): correct selection bias, non-normality, and backtest overfitting.
- Bailey et al., *The Probability of Backtest Overfitting* (https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2326253): ordinary holdouts are insufficient when many strategies are tried.
- Jegadeesh/Titman momentum evidence (<https://www.nber.org/papers/w7159>): momentum is a reasonable prior, not permission to skip contemporary out-of-sample validation and trading costs.

15. What Claude must not improvise

- Do not implement all phases in one PR.
- Do not invent migration numbers; inspect the repo first.
- Do not create a second feature truth store.
- Do not hardcode thresholds as "validated"; mark priors and lifecycle state.
- Do not make Yahoo/TradingView scraping a live dependency.
- Do not train a complex model on the current handful of paper trades.
- Do not make any scoring change affect live trading until its migration, tests, shadow evidence, and owner promotion exist.

16. Provider Data-Truth Amendment (2026-07-31)

Formula review is not sufficient acceptance for a scoring feature. Every dimension must publish provider taxonomy/units, availability semantics, silent-default hit rate, production distribution including floor/ceiling mass, and a market-local frozen counterfactual. The corrective baseline and remaining semantic proposals are recorded in `features/scoring-data-truth/FEATURE_ARCHITECTURE.md`.

self healing agent - Feature Architecture

Source: features\self-healing-agent\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Live-Order Caps + Self-Healing / Health-Monitoring Agent

Status: DRAFT - awaiting approval. No implementation code until approved. Author: Claude (Opus 4.8), reviewed/updated by ChatGPT, 2026-07-08. Companion to: features/learning-integrity/FEATURE_ARCHITECTURE.md (the health agent surfaces `v_decision_quality` taint rates; the notional caps are an execution guardrail the agent monitors).

Decision inputs (user, 2026-07-08):

- Per-order live \$ cap must be controllable from the app, per market (US/India).
- Self-healing agent: LLM model selectable from Settings (like other agents); its

health shown on the dashboard alongside the existing agent-LLM list.

- Hard boundary (Claude pushback, accepted): an LLM may diagnose and suggest, and may auto-apply only deterministic whitelisted actions. It must never autonomously change weights, config money-limits, code, or place/cancel orders.

PART A - Live per-order notional caps (per market)

A1. Current state (verified 2026-07-08)

- `strategy_config.max_order_notional = 50` (single numeric, no currency).
- Enforced only in the US Execution Gateway `app/api/broker/orders/route.ts`

(`qty * fresh_price > cap` -> 403; fail-closed if no cap AND no equity snapshot; default fallback = 15% of live equity when the column is null).

- Gaps:
- No Settings UI to set it - DB-column only.
- The Kite route `app/api/kite/order/route.ts` enforces owner-gate + rate-limit + long-only + `confirm:true`, but has no notional cap. India live orders are not \$-capped.
- Single value is USD-shaped. ?50 would block every India order; \$50 is fine for US.

One column can't serve both currencies.

A2. Design

- Schema (additive): add `max_order_notional_usd` numeric, `max_order_notional_inr` numeric to `strategy_config`. Backfill `usd` from the existing 50; set `inr` to an approved default (proposal: ?4000 ? \$48, US-parity). Keep `max_order_notional` as a deprecated read-through (`usd`) for one release, then drop.

- Enforcement:
- US Gateway reads `max_order_notional_usd`.
- Kite route adds the same fresh-quote notional check against `max_order_notional_inr`, same fail-closed semantics (no cap resolvable -> refuse).

- Preserve the existing 15%-of-equity fallback per market when the column is null.
- Settings UI: owner-gated "Live Order Limits" section with a USD field and an INR

field, writing `strategy_config` via an owner-gated route (extend the existing risk-profile/settings route; never cron-callable).

- Value shown on the trade-approval screen so the human sees the ceiling before clicking submit.

Reviewer corrections to A2 (binding):

- Keep `max_order_notional` as a deprecated USD read-through for at least one release;

do not drop it in this feature. Dropping is a separate approved cleanup after all deployed code reads `max_order_notional_usd`.

- `max_order_notional_usd` / `max_order_notional_inr` readers must ship only after the additive migration is applied. A live-money route must never treat a missing cap column as uncapped.

- The current US 15%-of-equity fallback may remain only when a fresh live equity snapshot exists. Kite/India must fail closed if `max_order_notional_inr` is null because no trusted Kite equity fallback is defined here.

- Settings writes are owner-human actions only; the health agent/LLM cannot change `strategy_config` money limits.

A3. Acceptance criteria

- A live US order > USD cap -> 403; a live India order > INR cap -> 403.
- Neither path can place an uncapped live order (fail-closed retained on both).
- Cap editable in-app by owner only; change is audit-logged.
- No cron path can place or uncap an order.
- New positions remain long-only; SELL remains allowed only for held positions. This feature must not weaken either order-side guard.

PART B - Self-healing / health-monitoring agent

B1. Principle: three tiers, one hard boundary

Tier	Actor	Allowed	Risk
1.	Deterministic auto-remediation	Rule-based code (no LLM)	Bounded whitelist actions (below) Low - reversible, no judgment
2.	LLM triage	LLM (model from Settings), **read-only**	Diagnose, rank, suggest a fix, mark whether a Tier-1 action can handle it Low - writes advice only
3.	Apply	Human click, or Tier-1 auto for whitelisted actions only	Execute a suggested fix Gated

Hard boundary (non-negotiable): the LLM never autonomously mutates weights, `strategy_config` money-limits, code, or places/cancels orders. Those remain human-gated regardless of LLM confidence. The LLM's output is advice + a whitelisted-action tag. The deterministic apply route must also reject any action id that would touch orders, code, strategy weights, model selection, broker credentials, secrets, or money limits.

B2. Tier 1 - deterministic auto-remediation (rules)

Whitelisted, bounded, already partly present (kill switches, needs-reconcile, AV->FRED fallback). Candidate actions, each individually feature-flagged:

- Re-run research for the N symbols that failed a cron batch.
- Post-enablement: auto-exclude a trade whose linked `v_decision_quality` is tainted

(sets `excluded_from_learning`, never deletes).

- Trigger a one-shot retry using an already-coded fallback path when the primary provider is rate-limited. Do not persistently change provider config, API keys, provider priority, or budgets.

- Auto-resolve a stale `agent_alerts` row when its condition has cleared.

Each action: idempotent, logged to `agent_alerts/decision_journal`, and reversible. None touch orders, weights, money-limits, or code.

B3. Tier 2 - health-triage LLM agent (read-only)

- New agent `health-triage`. Runs on schedule (and/or on a new CRITICAL alert).
- Model selectable from Settings: add a row to `agent_config`

(`agent_name='health-triage'`, default `deepseek-v4-flash - cheap` advisory). It then appears in the existing Settings -> LLM Config selector automatically, same pattern as `macro-read/mentor`. `getConfiguredModel('health-triage')` with a safe fallback.

- Inputs (read-only): open `agent_alerts`, recent `agent_runs` errors, `v_decision_quality` daily taint/unknown rate, provider budgets (`provider_budget_7d`), broker-token expiry.
- Output: one triage record per open issue - `root_cause`, `blast_radius`, `suggested_fix` (plain English), `auto_remediable` (bool -> maps to a Tier-1 action id or null), `severity`, `model_used`, tokens. Written to a new table `health_triage` keyed to `agent_alerts.issue_key`.
- Never writes to `money/code/order/weight` paths.
- The LLM prompt/tool contract is read-only. It may receive issue context and emit

JSON advice, but it must not receive tools or route access that can write `strategy_config`, `strategy_versions`, `agent_config`, `broker/order` tables, code, or secrets.

B4. Tier 3 - apply

- Dashboard renders each suggestion in the System Health card:

"? Suggested fix: ... [Apply]" - the Apply button is enabled only when `auto_remediable` maps to a whitelisted Tier-1 action; otherwise it's advice-only text.

- Owner click triggers the deterministic action through an owner-gated, CSRF/Origin protected route. It must not accept cron secrets. Every apply is audit-logged with the triggering triage id.

- Auto-apply without a click is allowed only for explicitly feature-flagged Tier-1 actions that are non-money, non-order, non-code, non-weight, and reversible. It must never call live order routes.

B5. Dashboard health of the agent itself (user requirement)

- health-triage logs to agent_runs (agent_type='health_triage') -> last-run, errors, tokens show in the existing agents list exactly like the other agents.
- The model-freshness checker covers health-triage too (deprecated-model -> CRITICAL).
- The agent's model + enabled toggle appear in Settings -> LLM Config with the rest.
- So it's a first-class citizen: same health surface, same model-config surface as every other agent LLM already on the dashboard.

B6. Schema

- New table `health_triage(id, ts, issue_key text, root_cause text, blast_radius text, suggested_fix text, auto_remediable boolean, remediation_action text, severity text, applied boolean default false, applied_at, applied_by, model_used text, tokens_input int, tokens_output int). Links to agent_alerts via issue_key`.
- agent_config: one new row health-triage.
- Notional-cap columns are the only money-limit schema in Part A, and they are edited only by owner Settings. Part B adds no money/order columns and must not mutate order tables except read-only diagnostics.

B7. Acceptance criteria

- Changing the health-triage model in Settings changes the model used on the next run.
- The agent's health (last run, errors) shows in the dashboard agents list.
- Triage suggestions render on the health card; Apply is live only for whitelisted deterministic actions.
- No code path lets this agent place/cancel an order, change a money-limit, mutate weights, or edit code. (Test: attempt each -> blocked.)
- If the LLM is unreachable, Tier-1 remediation and the health card still function (LLM is additive, not load-bearing).
- app/api/kite/order/route.ts refuses live India orders over max_order_notional_inr and refuses if the INR cap is null/unavailable.
- app/api/broker/orders/route.ts reads max_order_notional_usd, with the deprecated max_order_notional only as a one-release compatibility fallback.

B8. Rollout

- A first (small, high-value guardrail): per-market notional caps + Settings UI + Kite enforcement. Independent of the agent.
- B-Tier 1: deterministic remediation whitelist (feature-flagged, off by default).
- B-Tier 2: health-triage agent + agent_config row + health_triage table + dashboard surfacing (read-only advice).

- B-Tier 3: one-click apply for whitelisted actions only.

B9. Open questions

- Triage cadence: every N hours vs only on new CRITICAL? (proposal: on new CRITICAL + a 6h heartbeat.)
- Which Tier-1 actions ship in the first whitelist.
- Default INR cap value (proposal ?4000).
- Does health-triage also summarize into the daily briefing email?

Reviewer correction to B9: Default INR cap remains an open decision until Vaibhav explicitly approves it; proposal is INR 4000.

B10. Diagram / system-map impact

Adds a health-triage node reading `agent_alerts/agent_runs/v_decision_quality` and writing `health_triage`. On build, update `public/agent-diagrams/system-map.json` + add the per-agent diagram, and append a history entry per project convention.

Reviewer changelog (ChatGPT)

- Header: Updated author line to "reviewed/updated by ChatGPT, 2026-07-08" as requested.
- Section A2: Added binding reviewer corrections that keep `max_order_notional` as a deprecated USD read-through for at least one release, require additive migration application before route code reads new cap columns, and forbid uncapped live-money fallback on missing schema.
- Section A2: Corrected cap fallback semantics: US may keep the current fresh-equity 15% fallback; Kite/India must fail closed if `max_order_notional_inr` is null because no trusted Kite equity fallback is defined.
- Section A2/A3: Added explicit owner-human gating for money-limit edits and preserved long-only / sell-only-if-held order-side rules.
- Section B1: Strengthened the hard LLM boundary so the deterministic apply route also rejects action ids touching orders, code, weights, model selection, broker credentials, secrets, or money limits.
- Section B2: Replaced persistent provider "auto-flip" behavior with one-shot fallback retry only; no automated provider config/API-key/priority/budget mutation.
- Section B3: Added a read-only prompt/tool contract for the LLM; it must not receive tools or route access that can write `money/config/code/order/secret` state.
- Section B4: Required owner-click routes to be CSRF/Origin protected and not cron-callable; auto-apply is limited to reversible non-money/non-order/non-code/non-weight Tier-1 actions.
- Section B6: Clarified that Part B adds no money/order schema and must not mutate order tables except read-only diagnostics.
- Section B7: Added acceptance tests for Kite INR notional-cap enforcement and US Gateway use of `max_order_notional_usd`.
- Section B9: Default INR cap is still an open question until Vaibhav explicitly approves it; proposal remains INR 4000.
- Sections reviewed with no additional change: B5, B8, and B10 were consistent after the safety edits above.

settings llm control - Feature Architecture

Source: `features/settings-llm-control/FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Settings-driven LLM + API-key control for every flow

Status: SHIPPED (both parts). 2026-07-08. Part 1 - every flow honors `agent_config`: migration 129 reconciled theme-scout/ briefing/mentor rows to their real current model, then wired those callsites to `getConfiguredModel` (learner + markets-thesis/synthesis + macro-read + backtest-optimize + health-triage + deep-dive already honored it; research keeps adaptive routing; deepseek A/B stays deepseek). Part 2 - `lib/llm-keys.ts` (`getProviderKey` vault-first/env-fallback + status/set/clear), owner-gated `app/api/agents/provider-keys`, router + guards + model-check made vault-aware, Providers card in the LLM Config tab (write-only, masked, clear-to-revert-to-env). Last updated: 2026-07-08

Goal (user ask)

"I should be able to choose the LLM and its API through Settings, so the app can use them for any flow/agent." One place to pick, per agent/flow: (a) which model, (b) which provider API key - and have EVERY flow actually obey it.

Current state (what exists, what's broken)

- `agent_config(agent_name, model, enabled)` table + `getConfiguredModel(svc, name, fallback="deepseek-v4-pro")` ([lib/agent-model-config.ts])` - the intended control point.

- Settings -> Agents -> LLM Config tab + the Model Freshness panel

(`app/api/models/check`) read `agent_config`.

- Gap 1 - not every flow honors it. Only callsites using `getConfiguredModel`

obey the setting (markets-thesis, markets-synthesis, macro-read, backtest-optimize, health-triage). Others HARDCODE and ignore the config:

- Theme Scout -> `llama-3.3-70b-versatile` (but freshness shows it as deepseek - misleading)
- Briefing -> `deepseek-v4-flash` (x2)
- Mentor-coach -> `deepseek-v4-pro`
- DeepSeek A/B agent -> `deepseek-v4-flash`
- ResearchAgent -> computed `screenModel` (claude-haiku on structured path, else deepseek)

So the panel implies control the code doesn't actually grant for those flows.

- Gap 2 - API keys are env-only. `ANTHROPIC_API_KEY` / `DEEPSEEK_API_KEY` /

`GROQ_API_KEY` are read from `process.env` inside the router. A user cannot set or rotate a provider key from Settings; adding a provider means a Vercel env change + redeploy.

Proposed design

Part 1 - every flow honors `agent_config` (single source of truth)

- Enumerate every LLM-using flow and give each a stable `agent_name`

(research, screen, briefing-editor, briefing-outlook, theme-scout, mentor, learner, deep-dive, markets-thesis, markets-synthesis, backtest-optimize, macro-read, health-triage, deepseek-ab). Seed one `agent_config` row per flow.

- Replace each `hardcoded model:` with `await getConfiguredModel(svc, "<agent_name>")`,

keeping the CURRENT model as that row's default (behavior-preserving: nothing changes until the user edits a row). ResearchAgent's structured-vs-thin branch stays, but both branches read their configured model instead of literals.

- The Model Freshness panel then reflects TRUE wiring - every listed agent is one the code actually obeys. (Fixes the Theme-Scout=llama-shown-as-deepseek lie.)

Part 2 - provider API keys settable from Settings (vault-backed)

- Add a Providers section to the LLM Config tab: one row per provider

(Anthropic, DeepSeek, Groq, ...) with a masked key field + "Test" button.

- Store keys in the existing Supabase vault (same pattern as the Kite/Robinhood tokens + kairos_cron_secret), NOT in a plain table. Owner-gated write.

- Router key resolution becomes vault-first, env-fallback: a small

`getProviderKey(provider)` reads the vault secret, falling back to `process.env.*` so existing env keys keep working with zero migration. `callClaude/callDeepSeek/ callGroq` call it instead of reading `process.env` directly.

- Security constraints (hard): keys are write-only from the UI (never rendered

back - show only "last4" + set/rotate), owner-gated, stored encrypted in vault, never logged, never returned by any GET. Adding/rotating a key is an "account-settings"-class action -> owner click only, never agent-triggered.

Part 3 - model dropdown is provider-aware

- The per-agent model dropdown offers the known model ids per provider (from the same

list Model Freshness already fetches), plus the tier aliases ("fast"/"reasoning"). Picking a model whose provider has no key surfaces a warning (uses env fallback or fails loudly via the existing System Health funnel).

Touch list (for approval)

- DB: seed `agent_config` rows for all flows (data, not schema - additive upsert).

Vault secrets for provider keys (no new table).

- `lib/llm-router.ts: getProviderKey()` vault-first resolution in the three

`call* fns`; no routing-logic change.

- ~8 route/agent files: swap `hardcoded model:` for `getConfiguredModel(...)`.

- Settings LLM Config UI: add Providers (key set/rotate/test) + provider-aware model dropdown; keep per-agent enable toggle.

- Owner gating + vault read/write helper (reuse existing vault util).

- Docs: `SYSTEM_OVERVIEW.md` LLM section + `system-map.json` only if a flow's provider changes materially (default-preserving, so likely just `SYSTEM_OVERVIEW`).

Non-goals / guardrails

- No autonomous key creation or provider signup - user pastes their own key; the app only stores/uses it. (Account-creation + credential-entry stay owner actions.)
- Default-preserving: seeding rows with today's models means nothing changes until the user edits. No behavior drift on deploy.
- Does not touch money limits, order paths, or the champion-genome live controls.

Open decisions for you

- Scope now: do BOTH parts (full: every-flow-honors-config + vault key management), or Part 1 only first (make every flow obey config; keep keys in env for now)?
 - Provider key storage: Supabase vault (recommended, matches existing secrets) - confirm, or prefer a different store?
 - Any flow you want to DELIBERATELY pin (e.g. keep Theme Scout on free Groq/llama regardless) rather than expose as user-editable?
- No implementation until you approve + answer 1-3.

shadow registry - Feature Architecture

Source: features\shadow-registry\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Shadow Registry and Upgrade Path

Status: APPROVED (owner instruction, 2026-07-29) Owner: Vaibhav Implementation role: Codex

Problem

Kairos has several measure-only, counterfactual, and paper-validation programs. Their evidence is spread across purpose-built ledgers and cron definitions, so the owner cannot answer five basic questions from the app:

- What is being measured?
- What does it cost in provider calls?
- Has it helped, failed, or merely remained inconclusive?
- What exact gate remains before it may influence paper or live behavior?
- How long might that gate take at the observed collection rate?

The absence of one inventory also creates governance drift: an old cron can keep running after its program becomes idle, and a program may be described as "shadow" after paper execution has already been enabled.

Decision

Build an owner-only /dashboard/upgrade-path page backed by a typed, version-controlled registry and live adapters over existing truth ledgers.

Do not create a parallel evidence table. The page reads:

- provider_call_ledger and evidence_policy_evaluations for Router evidence;
- shadow_decisions for setup-expert comparisons;
- edge_signals, edge_ic_history, and edge_readiness_status for technical calibration;
- rotation_config and rotation_events for capital rotation;
- earnings_risk_observations for earnings-event risk;
- international_allocation_* for allocation evidence;
- trade_proposals and strategy_config for autonomous-live shadow;
- strategy_validation_automation and strategy_versions for challenger routing;
- a service-only cron-status RPC for schedule truth.

Product Shape

The left navigation gains Upgrade Path under Research.

The shared DashboardShell US/India switch is the sole market authority. The page must not add a second market picker. Every evidence query, count, verdict, blocker, and ETA is scoped to the selected market before aggregation. Programs that

do not support the selected market remain visible as `not_applicable`.

The page contains:

- a compact status bar: total programs, actively collecting, review-ready, blocked/idle, and provider calls recorded over seven days;
- lifecycle filters;
- one un-nested program row/panel per registry entry;
- purpose and concrete value to Kairos;
- market scope and safety boundary;
- actual schedule state;
- evidence completed, target and progress;
- seven-day collection rate;
- tracked provider calls, cache hits and network attempts when authoritative;
- an honest benefit verdict;
- exact blockers and next owner/engineering action;
- estimated days only when a measurable target and non-zero collection rate exist. Otherwise the UI says No defensible ETA.

Registry Contract

`lib/shadows/registry.ts` is the authoritative descriptive catalog. Every entry must include:

- stable id;
- display name and category;
- supported markets;
- one-sentence purpose;
- product and trader benefit;
- evidence source;
- current and maximum permitted influence;
- activation gate;
- cron job names, if any;
- provider-call accounting mode;
- owner and architecture reference.

`lib/shadows/status.ts` owns aggregation and derives a normalized `ShadowProgramStatus`. It may reference existing ledgers but cannot write them.

A coverage test pins the known producers/cron names to registry entries. Future shadow work is incomplete until this registry is extended.

Lifecycle Vocabulary

- `collecting`: schedule/trigger is active and evidence is arriving.

- `ready_for_review`: declared evidence gate is met; still no automatic enable.
- `blocked`: evidence or quality gate failed.
- `armed`: automation exists but has no active candidate.
- `paper_active`: paper behavior is enabled; live remains gated.
- `idle`: scheduled but no evidence has arrived in the observation window.
- `off`: schedule/config is disabled.

These labels describe operational state, not expected profitability.

Benefit Vocabulary

- `benefited`: realized app-specific evidence supports the change.
- `promising`: directional evidence is positive but the activation gate is not met.
- `mixed`: evidence differs by market or gate.
- `not_beneficial`: a declared test failed with enough evidence.
- `insufficient`: there is not enough app-specific evidence.
- `operational_only`: the program improves safety/observability rather than predicting returns.

The API must not infer `benefited` merely from row count.

Call Accounting

Provider calls are reported only from authoritative instrumentation:

- Router: `provider_call_ledger`, separating cache hits from leased network attempts.
- Reused-input programs: explicitly `incremental` when code proves no extra provider fetch occurs.
- Uninstrumented programs: not `metered`; never display zero.

Future provider-consuming shadows should write to the existing `provider_call_ledger` with a stable run prefix instead of creating a new meter.

Read API

GET `/api/upgrade-path`

- confirmed-owner only via `requireOwner()`;
- uses the service client after the owner gate;
- returns normalized aggregates, no raw provider payloads, credentials, cron commands, option chains, account IDs, or source bodies;
- fail-soft per adapter: one unavailable ledger does not erase other programs;
- top-level `generatedAt` makes freshness explicit.

Approved Cleanup in the Same Change

- Owner-gate /api/options/signal; it spends an external request.
- Remove dead ResearchAgent prompt text claiming unusual calls should alter conviction. Options are not fetched or scored there.
- Unschedule kairos-shadow-us and kairos-shadow-india. They produced no shadow proposals in seven days while live-auto is disabled. The routes remain available for an explicitly approved future campaign.
- Do not change scoring, options weights, live-auto settings, or broker/order behavior.

Safety

- Page and API are read-only.
- No LLM.
- No scoring or trading consumer.
- US and India metrics stay separate; no currency values are summed.
- A readiness label cannot activate anything.
- Estimates are observational and suppressed when blocked or underidentified.
- Capital rotation must be labelled paper_active because paper execution is currently enabled; calling it shadow-only would be false.

Acceptance Criteria

- Unauthorized requests receive 401/403.
- Every registry entry has purpose, benefit, evidence source, safety boundary, gate, owner, and architecture reference.
- The API reports live production counts without storing duplicate evidence.
- Provider calls distinguish tracked network attempts, cache hits, zero incremental calls, and unmetered calls.
- Router remains disabled in both markets.
- Earnings risk remains policy_mode='shadow' and behavior_changed=false.
- Live rotation proposals remain disabled.
- Autonomous-shadow cron jobs are absent after migration and are not expected by stale-check.
- Options remain absent from analyst_score.
- Desktop and 375px mobile views have no overlap or horizontal page overflow; wide detail tables may scroll within their own region.
- Typecheck, focused tests, full tests, production build, schema verification,

browser verification, and production deployment pass.

Reversal

- Remove the nav/page/API/registry with no evidence loss.
- Reschedule autonomous-shadow jobs from migration 164 only after a new owner decision.
- Existing source ledgers remain untouched.

stock context - Feature Architecture

Source: `features\stock-context\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Feature Architecture: Stock Context

Status

Architecture status: Draft Architecture approved: Yes (owner: "build it"; Codex endorsed peers-context-only) Approved scope: Read-only "what is this stock" context layer, off the money path Approved date: 2026-07-15 Implementation allowed: Yes

Feature Purpose

Research and watchlist screens currently show bare tickers (RELIANCE.NS, NVDA) with no plain-language answer to "what is this company and why am I looking at it?". Stock Context adds a lightweight, cached, display-only profile layer: company name, a one-sentence "what it does", sector/industry, exchange, market-cap tier, next earnings date, and up to a few peer tickers shown as context chips (never clickable trades). It also surfaces a short "why revisit + when" reason derived from data the app already holds.

This is orientation only. It never scores, sizes, gates, or orders anything.

User/System Questions This Feature Answers

- What is this company, in one sentence?
- What sector / industry / exchange is it, and roughly how big (mega/large/mid/small/micro)?
- When does it next report earnings?
- Who are a few peers (for context - NOT trade suggestions)?
- Why is this symbol in my queue/watchlist, and when should I look again?

Scope

This feature includes:

- A `symbol_profiles` cache table (per (symbol, market)), RLS-locked.
- `lib/data/symbol-profile.ts` - fetch + cache one symbol's profile (FREE providers only).
- `app/api/agents/symbol-profiles/backfill/route.ts` - owner/cron-gated, bounded backfill for watchlist symbols missing a fresh (<30d) profile; also backfills `watchlist.company_name` where null.
- `lib/revisit-reason.ts` - pure function: short "why revisit + when" string from existing row data.
- `app/api/symbol-profile/route.ts` - owner-gated GET returning the cached profile + revisit reason.
- `components/dashboard/StockContextStrip.tsx` - mobile-first display strip.
- Wiring the strip into the symbol detail page.

Non-Goals

This feature does not include:

- Any LLM call anywhere (the one-liner is composed deterministically).

- Any import by scoring / sizing / gating / order code (off the money path - hard invariant).
- Clickable peers or peer-driven navigation into trades. Peers are inert context chips.
- Live price/quote (that already exists elsewhere on the detail page).
- Paid data providers. FREE tier only (Finnhub free, Yahoo auth-free/crumb).
- Mixing per-market data or per-currency values.

Current Behavior

Research/watchlist rows render the ticker string and, where present, `watchlist.company_name` (often null). No sector/industry/earnings/peers context.

Proposed Behavior

- A backfill job (owner-triggered or cron) fills `symbol_profiles` for watchlist

symbols that lack a profile fresher than 30 days, bounded by a wall-clock budget so it never blows Vercel `maxDuration`. It also copies the fetched company name into `watchlist.company_name` when that column is null.

- The symbol detail page reads the cached profile + a computed revisit reason via

an owner-gated GET route and renders `StockContextStrip`.

- If no profile is cached yet, the strip renders nothing (graceful empty state) -

the page is fully functional without it.

User Journey / System Flow

- Owner opens `/dashboard/symbol/NVDA`.
- Page server-fetches signals/trades (existing) and passes `symbol` + `market` to the client.
- `StockContextStrip` calls `GET /api/symbol-profile?symbol=NVDA&market=us`.
- Route (owner-gated) reads `symbol_profiles` cache + computes revisit reason, returns JSON.
- Strip renders: name ? one-liner ? sector ? exchange ? mkt-cap tier ? next earnings ? peer chips ? revisit reason.

Screen / Page / Module Inventory

- Table: `symbol_profiles` (cache).
- Module: `lib/data/symbol-profile.ts` (fetch+cache; US Finnhub, India Yahoo).
- Module: `lib/revisit-reason.ts` (pure).
- Route: `POST /api/agents/symbol-profiles/backfill` (owner/cron-gated, bounded).
- Route: `GET /api/symbol-profile` (owner-gated read).
- Component: `components/dashboard/StockContextStrip.tsx`.
- Wiring: `app/dashboard/symbol/[symbol]/page.tsx`.

UI Architecture

Layout

A single horizontal strip above the tab bar on the symbol detail page. On mobile it wraps to multiple rows; each datum is a labeled inline chip. Peer tickers render as a small chip row labeled "Peers (context)".

Components

StockContextStrip (client). Fetches its own data; renders nothing while loading or when no profile exists. Uses the existing T token palette + `clamp()` responsive sizing.

Sheets / Modals / Drawers

None.

Navigation

None changed. Peer chips are NOT links and do not navigate.

Empty State

Renders nothing (null) - the rest of the page is unaffected.

Loading State

Renders nothing (null) until the fetch resolves - avoids layout jitter for a secondary, non-blocking strip.

Error State

On fetch error the strip renders nothing (fail-soft; this is context, not critical).

Success State

Full strip with all available fields; missing fields are omitted, not shown as " - ".

System Architecture

Modules

- `symbol-profile.ts` - `getSymbolProfile(symbol, market, svc)` returns the cached row if fresh (<30d), else fetches + upserts + returns. `fetchSymbolProfile()` composes the profile from free providers. `oneLinerFrom()` builds the deterministic sentence. `capTierFrom()` maps market-cap (USD millions) to a tier.
- `revisit-reason.ts` - `computeRevisitReason(input)` pure; no I/O.

API Contracts

- GET `/api/symbol-profile?symbol=<sym>&market=us|india` -> { profile: SymbolProfile | null, revisit: { reason, urgency } | null }. Owner-gated (401/403 otherwise).

- POST /api/agents/symbol-profiles/backfill?market=us|india&limit=<n> (owner OR x-cron-secret)
-> { processed, filled, watchlistNamesBackfilled, skippedFresh, deferred }.

Data Models

symbol_profiles:

- symbol text, market text (us|india), company_name text, one_liner text,
sector text, industry text, exchange text, market_cap_tier text (mega|large|mid|small|micro),
next_earnings_date date, peers text[], source text, updated_at timestampz.PK (symbol, market).

Auth / Permissions

- Table: RLS ENABLED. Policy: for select to authenticated using (true) - anon denied,
owner (authenticated) can read, service_role writes via bypass. (New public tables with RLS off are a Security Advisor
ERROR - this table ships RLS-on from migration 1.)

- Read route: requireOwner().
- Backfill route: requireOwner() OR verifyCronSecret() (service-to-service).

Error Handling

Every external fetch is wrapped and fail-soft: a provider miss yields a partial or null profile, never a thrown error. India
peers/earnings are best-effort (often null).

Data Architecture

- Required data: symbol, market.
- Optional data: everything else (all nullable; store what the provider returns).
- Mock vs real vs derived: real (Finnhub/Yahoo) + derived (one_liner, market_cap_tier).
- Persistence: symbol_profiles, refreshed when older than 30 days.
- Validation: market constrained to us|india; market_cap_tier constrained to the 5 tiers.

Providers (FREE only)

- US: Finnhub /stock/profile2 (name, finnhubIndustry->sector, exchange,
marketCapitalization->tier), /stock/peers (peers), /calendar/earnings (next earnings date). Finnhub free = 60/min,
no daily cap.

- India (.NS/.BO): Yahoo (auth-free chart + crumbed quoteSummary, reused from
lib/india-data.ts) for name/sector/industry/market-cap/exchange, and fetchIndiaEarningsDate for next earnings.
Peers are unavailable for free on India - stored as []/null (best-effort, acceptable).

Off-Money-Path Guarantee

lib/data/symbol-profile.ts, lib/revisit-reason.ts, StockContextStrip.tsx, and both routes are
display-only. They are NEVER imported by scoring/sizing/gate/order code (lib/research-agent, lib/scoring/*,
lib/data/scores.ts, paper-trade/trader routes). Import direction is one-way: this feature imports existing helpers;
nothing on the money path imports this feature. Deterministic - no LLM is invoked anywhere in this feature.

Per-Market Handling

market is a first-class key ((symbol, market) PK). US and India never share a row. India profiles are Yahoo-sourced with best-effort peers/earnings; US profiles are Finnhub-sourced. No per-currency numeric values are stored (tier is a categorical label), so no currency mixing is possible.

Peers-Context-Only Rule

Peers are stored as text[] and rendered as inert chips under a "Peers (context)" label. They are NEVER clickable, NEVER navigated to, and NEVER fed into scoring, screening, candidate generation, or any candidate list. This is a hard boundary Codex endorsed.

Files Likely To Change

- features/stock-context/FEATURE_ARCHITECTURE.md (this file)
- supabase/migrations/20260715150000_symbol_profiles.sql (new)
- lib/data/symbol-profile.ts (new)
- lib/revisit-reason.ts (new)
- tests/revisit-reason.test.ts (new)
- app/api/agents/symbol-profiles/backfill/route.ts (new)
- app/api/symbol-profile/route.ts (new)
- components/dashboard/StockContextStrip.tsx (new)
- app/dashboard/symbol/[symbol]/page.tsx (wire strip)
- lib/india-data.ts (additive: expose name/market-cap/exchange from the Yahoo overview)

Files / Behavior That Must Not Change

- Scoring / sizing / gate / order code - must not import this feature.
- computeScores, research-agent, paper-trade, trader - untouched.
- Peer navigation - must remain absent.

Acceptance Criteria

- symbol_profiles exists in prod with RLS on and an authenticated SELECT policy (verified).
- getSymbolProfile returns a cached row and refreshes when >30d old.
- No LLM call in any file of this feature.
- No scoring/order module imports this feature.
- Backfill is idempotent, bounded by a wall-clock budget, and backfills watchlist.company_name.
- Strip is responsive (mobile-first) and renders peers as non-clickable chips.
- tsc --noEmit, npm run build, vitest run all pass.

Approval

Architecture approved: Yes Approved scope: As above (peers-context-only, off money path) Implementation allowed: Yes

system health model resilience - Feature Architecture

Source: features\system-health-model-resilience\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature Architecture: System Health funnel + Model Resilience

Status

Architecture status: Implemented (P1 + P2 + P3) Architecture approved: Yes ("build", 2026-07-07) Approved scope: All three phases Approved date: 2026-07-07 Implementation allowed: Yes

Implementation notes (2026-07-07)

- Migration 099: agent_alerts.issue_key + partial unique index (one open row per key).
- lib/system-health.ts: reportIssue / resolveIssue / reconcileIssues.
- Reporters wired: model-check (deprecated -> critical, reachable-provider-aware auto-resolve; newer-available -> info), AV budget exhaustion (self-expiring), kill-switch trips (per market), unreconciled orders (Gateway), Robinhood + Kite token expiry.
- Surfaces: SystemHealthCard on the dashboard home (persistent, severity-ranked, deep-link fix hints; green when clean) + "Open Issues" band in every daily brief. DashboardShell severity vocabulary extended with critical (top rank).
- Model resilience (lib/llm-router.ts): TIER_MODELS aliases (fast/reasoning/claude-fast/claude-smart), SAME_TIER_FALLBACK graceful fallback on an unavailable model (loud model-fallback: alert, never blind-latest), and priceFor pricing fallback so cost never silently logs \$0.

Why this feature exists

Tonight's session exposed the core risk of an autonomous-ish trading system: silent failures. Every real problem was invisible until stumbled upon - DeepSeek models deprecated (agents would've errored on next call), the vault display_name/provider NOT-NULL break, the trade_queue vs trade_proposals split, the AV budget starving the scorer, the snapshot parse returning nulls. None surfaced anywhere the user would see them.

Two related gaps:

- No unified "open issues until fixed" surface. Issues are scattered: model-check and a couple of others post to agent_alerts, but most subsystems report nothing, there's no dedup/auto-resolve, and there's no persistent dashboard panel or brief section that shows OPEN issues until they clear.
- Model config is brittle. Concrete model names are hardcoded/stored per agent; when a provider renames or deprecates a model, every agent pointed at it breaks and must be hand-updated (as just happened). There is no tier indirection and no graceful fallback - a deprecated model is a hard error, not a flagged degradation.

This feature makes failures loud and persistent and makes model config break-proof but never silently drifting. It changes no trading logic.

Existing infra to BUILD ON (do not duplicate):

- agent_alerts table: id, created_at, severity, category, title, detail,

resolved, resolved_at, auto_expire_at.

- /api/alerts: GET (open alerts), POST (create), PATCH (resolve).
- Current reporters: models/check, research/cron, broker/orders/sync only.

Phase 1 - Issue funnel: standardized reporting + dedup + auto-resolve

Problem

agent_alerts exists but: (a) most subsystems don't report into it; (b) no stable dedup key, so a recurring condition would spam duplicate rows (or, as today, never report at all); (c) no auto-resolve when the condition clears.

Proposed behavior

- Add agent_alerts.issue_key text (a stable identifier for a condition, e.g. model-deprecated:deepseek-reasoner, av-budget-exhausted, kite-token-expired, order-needs-reconcile:<orderId>, cron-failed:kairos-learner). Migration adds the column + a partial unique index on (issue_key) WHERE resolved = false so at most one OPEN row per condition.
- A shared helper reportIssue({ issueKey, severity, category, title, detail, autoExpireAt? }) - upserts by issue_key on open rows (refreshes detail / keeps the existing open row rather than duplicating), and resolveIssue(issueKey) - marks the matching open row resolved. Both service-side, used by any subsystem.
- Wire reporters across the app (each reports AND auto-resolves):
- Model deprecation (models/check): open an issue per deprecated assignment; resolve when the model reappears / is reassigned.
- API budget (av-cache / a daily check): open when AV daily budget is exhausted; auto-expire next day.
- Broker/connection health: Kite token expired/near-expiry, Robinhood MCP token missing/refresh-failing.
- Kill switches: open an issue while a kill switch is tripped; resolve when cleared.
- Unreconciled orders: broker_orders.status = unknown_needs_reconcile opens an issue until the order reconciles.
- Cron failures: a cron that errors or writes 0 where it shouldn't.
- Schema drift: a migration named in the repo not applied to the live DB (optional, lower priority).
- Auto-resolve: reporters that run on a schedule resolve their own issue when the condition is gone; time-bounded ones use auto_expire_at.

Data model

- agent_alerts.issue_key text + create unique index ... (issue_key) where

resolved = false`. (Verify current indexes first.)

Acceptance criteria

- A recurring condition produces exactly ONE open agent_alerts row (dedup by issue_key), not duplicates.
- Resolving the underlying condition auto-clears the issue.
- At least the model-deprecation, budget, broker-token, kill-switch, and needs-reconcile conditions report into the funnel.

Phase 2 - Persistent surfaces: dashboard System Health card + brief section

Problem

Even the issues that ARE in agent_alerts aren't shown anywhere persistent - only reachable by opening a specific card.

Proposed behavior

- Dashboard "System Health" card (top of /dashboard, or the status bar):
reads OPEN agent_alerts, shows a count badge (red when any CRITICAL/HIGH), grouped by severity + category, each with title/detail and (where possible) a one-click "how to fix" hint or deep link (e.g. model-deprecation -> Settings -> LLM Config). Stays visible until issues resolve. Collapsed/green when clean.
- Daily brief "Open Issues" section (morning + evening, both markets): lists
OPEN issues (severity-ranked) so they're in the user's face every day until fixed - the thing that would have surfaced tonight's silent breakages on day one.
- Owner-only; reads via /api/alerts (already exists) or a small
/api/system-health aggregator.

Acceptance criteria

- Open issues appear on the dashboard persistently and in every daily brief until resolved.
- Zero open issues -> a clean/green state, no noise.
- Severity ordering: CRITICAL/HIGH first; live-trading-affecting issues (broker token, kill switch, needs-reconcile) surfaced most prominently.

Operational taxonomy correction (2026-07-20)

agent_alerts remains the transport for both actionable conditions and bounded operational telemetry, but the product must not call both "open issues":

- critical, error, and warn are actions required, shown expanded and counted in the System Health headline;
- info is an operational notice, collapsed by default and counted

separately;

- successful agent activity (for example, Theme Scout adding symbols) belongs in the agent/watchlist history, not System Health, and must not write an alert;

- expected sparse evidence and free-tier quota exhaustion may remain notices

because they explain score coverage, but their wording must not imply a broken provider or more real calls than the enforced budget allowed.

This changes presentation and activity routing only. It does not suppress real broker, cron, data-starvation, kill-switch, or reconciliation faults.

Phase 3 - Model resilience: tier aliases + graceful fallback

Problem

Model names are concrete and per-agent. A provider rename/deprecation hard- breaks every agent pointed at it (just happened: deepseek-chat/reasoner -> V4). "Always use latest" is NOT the answer for a trading system - a new model silently changes reasoning quality, output format, and cost, which must never drift under live financial decisions without review (the model-check's "never auto-switched, upgrades come from reviews" stance is correct).

Proposed behavior

- Tier aliases. Introduce roles (fast, reasoning) resolved by ONE small

mapping (config table or `lib/llm-router.ts` constant) -> the current concrete model per provider (e.g. fast->deepseek-v4-flash, reasoning->deepseek-v4-pro). `agent_config` can reference a tier OR a concrete model. Next provider rename = update ONE mapping line, not every agent row.

- Graceful fallback (break-proof, never silent). When `callLLM` gets a

model-not-found / deprecated error (or the model-check has flagged the model), the router falls back to a currently-available model of the SAME tier and `reportIssue({issueKey: 'model-fallback:<agent>', severity: 'high', ...})` so the run completes but the swap is loudly, persistently flagged for review. It does NOT silently ride "latest" - the fallback is same-tier and surfaced.

- Pricing safety. When a new model is used, if it has no PRICING entry,

cost logging must not silently zero it - fall back to the tier's price and flag "pricing unverified for <model>" so `llm_call_log` cost stays meaningful.

Non-Goals

- No auto-upgrade to the newest model. Model *upgrades* remain a reviewed decision (the checker proposes; a human approves), per existing doctrine.

Acceptance criteria

- A deprecated configured model does NOT hard-break the agent - it falls back to a same-tier available model and raises a persistent issue.

- Changing which concrete model a tier maps to is a one-line change.

- No LLM cost is silently logged as \$0 due to a missing pricing entry.

Cross-cutting

- Read-only/advisory surfaces; nothing here trades, sizes, or changes agent reasoning beyond swapping a broken model for a same-tier working one.
- Owner-gated APIs; no secrets in alert detail.

Files (indicative, verify before building)

- supabase/migrations/0NN_agent_alerts_issue_key.sql (issue_key + partial unique index)
- lib/system-health.ts (new - reportIssue/resolveIssue helpers)
- reporters: app/api/models/check, lib/av-cache.ts, lib/kite.ts / lib/robinhood-mcp.ts (token health), lib/kill-switches.ts, app/api/broker/orders(/sync), cron routes.
- components/dashboard/SystemHealthCard.tsx (new) + dashboard/StatusBar wiring
- app/api/briefing/generate/route.ts (Open Issues section)
- lib/llm-router.ts (tier aliases + graceful fallback + pricing fallback)
- app/api/system-health/route.ts (optional aggregator)

Approval

Architecture approved: No Approved scope: None Implementation allowed: No

system reference - Feature Architecture

Source: features\system-reference\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

System Reference Architecture

Status: *Approved*

Implementation allowed: *Yes*

Last updated: 2026-07-30

Goal

Replace the stale, hardcoded Architecture tab on Dashboard -> Agents with a maintainable owner-only reference surface. Preserve the live agent diagram as the topology authority and do not add a new main-navigation documentation area.

Scope

- Curated documents grouped by orientation, architecture chapters, decisions, and selected feature designs.
- Owner-gated read/download endpoint backed by a fixed document registry.
- Architecture portal, concise system overview, and chapter-index governance rules.
- Existing Agent & Flow Architecture visualization remains in place below the tabs.

Contract

GET /api/system-reference/:documentId

- Requires the confirmed owner through `requireOwner()`.
- `documentId` must resolve from `SYSTEM_REFERENCE_DOCUMENTS`; unknown IDs return 404.
- The server reads only the fixed registry path and returns Markdown with ``private, no-store caching and nosniff``.
- `?download=1` changes disposition to attachment. No request value is used to form a filesystem path.

Non-goals

- No repository/file browser, editor, search index, public docs portal, or markdown authoring workflow.
- No architecture state in Supabase and no user-provided documents.
- No provider, data, scoring, paper, live execution, broker, or schema change.
- No duplicate static agent topology; the JSON map remains canonical.

Governance

The portal explains ownership. docs/arch/ is the detailed operational architecture; feature documents are micro-architecture; implementation result files record delivery; project decisions capture approval and rationale. An agent-flow change updates the diagram source and history, then its relevant chapter. A drift test guards the diagram contract.

Acceptance criteria

- The old hardcoded architecture content, including stale schedules and account fragments, is removed from the client bundle.
- The System Reference tab supports opening and downloading only allowlisted files.
- Anonymous/non-owner calls receive the usual owner-gate response; traversal-like identifiers cannot return a repository file.
- The live topology remains visible and linked from the reference panel.
- TypeScript, focused tests, and production build pass.

technical factor calibration - Feature Architecture

Source: `features\technical-factor-calibration\FEATURE_ARCHITECTURE.md` | Feature micro-architecture: verify lifecycle status within the document and any `IMPLEMENTATION_RESULT.md`

Technical Factor Calibration

Status: *APPROVED FOR MEASURE-ONLY IMPLEMENTATION*

Approved by owner: 2026-07-21

Decision authority: `PROJECT_DECISIONS.md` Decision 51

1. Decision

Measure whether Kairos's existing technical score and a deliberately small set of candidate technical factors predict forward returns in the existing Edge evidence lab. Do not change production scoring from this feature.

The first trial family is frozen to:

- `kairos_technical_score_v1`: the exact current `computeTechnicals` plus `scoreTechnicals` composite, including its breakdown veto;
- `rel_strength_6m`: the existing stock-minus-market six-month return;
- `macd_atr_12_26_9`: MACD(12,26,9) histogram divided by Wilder ATR(14);
- `signed_adx_14`: ADX(14) strength signed by +DI versus -DI; and
- the existing price/volume factors already in EdgeScout.

No Fibonacci, oscillator zoo, per-stock optimization, or automatic sector formula is added. MACD and ADX are challengers, not new score weights.

2. Why

The current technical formula is a hand-tuned prior. Kairos needs a repeatable way to determine whether the composite, its existing components, or a small challenger set adds market-local forward-return information. The same path must be reusable when parameters are reconsidered later and must retain losing trials to prevent selection-history erasure.

3. Existing Substrate

Reuse the existing measure-only Edge lab:

- `lib/edges/registry.ts`: pure factor definitions over candles frozen at `asOf`;
- `lib/edges/ic.ts`: market-local forward-return rank IC with sampled dates and

Newey-West standard errors;

- `edge_signals` and `edge_signal_inputs`: immutable prospective observations and input lineage;
- `edge_ic_history`: historical diagnostics; and
- `edge_market_status`: advisory market-level lifecycle only.

This feature creates no parallel truth layer and makes no provider call per added factor. EdgeScout resolves a symbol's candle history once per run and evaluates every factor over the same logical `asOf` slice.

Current PIT limit: the run stores a dataset fingerprint, but its candle inputs are retrieved from current providers. Slicing them to asOf prevents direct future-bar look-ahead; it does not prove that historical bars were not restated after asOf. Therefore this evidence is retrospective/measure-only today and must not be labeled PIT-safe or satisfy a future promotion gate until a Kairos-owned immutable daily candle-vintage archive is available.

4. Recalibration And Memory

An edge ID is a versioned formula identity. Changing any period, threshold, anchor, normalization, unavailable-data rule, or veto creates a new edge ID; it never edits an old ID in place. Every run records market, horizon, window end, segment, history depth, sampling step, universe size, evidence quality, and provider report.

edge_ic_history stores separate rows for:

- segment_type = market, segment_value = all; and
- segment_type = sector, segment_value = <normalized sector>.

Old market-wide rows are migrated to the explicit market/all identity. A rerun of the same edge/market/window/horizon/segment against the exact same frozen dataset and configuration is idempotent. Provider corrections or newly available symbols change the dataset fingerprint and append a distinct snapshot. Each report records the benchmark source and per-provider symbol counts so the change is explainable. A changed formula requires a new edge ID and therefore appends a distinct trial history.

The Vibe comparator is pinned separately to commit a5eb30fd00d6ee71cd5099d15f57de5ae47010ff. Its full package is not admitted: the audited package includes LLM, MCP, broker, web, dynamic-import, and network dependencies unnecessary for calibration. A later sandbox may compare a narrowly vendored MIT validation subset against the same frozen inputs. It cannot replace Kairos evidence or influence scoring.

5. Sector Policy

Sector is an evaluation slice, not initially a separate strategy:

- use symbol_profiles.sector when present;
- fall back only to Kairos's deterministic known-symbol map;
- treat missing, Other, diversified funds, fixed income, commodities, and digital

assets as unclassified and exclude them from equity-sector conclusions;

- require at least five symbols in a sector on an as-of date before calculating its

cross-sectional IC; and

- require the existing minimum of 36 independent sampled IC observations before a

sector result can be anything other than measure_only.

Sector-specific parameters require a separate future decision. They must improve out of sample across multiple windows and neighboring parameter values after all tested sectors/parameters are included in the trial family. A single strong sector backtest is not sufficient.

6. Market Isolation

US and India are evaluated independently. They never share symbols, benchmarks, returns, sectors, observations, IC rows, status, currency, or promotion evidence. The same formula may be useful in one market and benched in the other.

7. Safety Boundary

This feature may write only Edge analytics, run metadata, and System Health state. It may not read or write production score weights, agent_signals, entry eligibility, mandates, positions, cash, proposals, broker orders, or exits. No LLM participates.

shadow_eligible remains advisory. No result changes production behavior without a separate architecture, locked holdout, trial-family correction, shadow observation, paper validation, and owner approval.

8. Acceptance Criteria

- Existing production scoreTechnicals() is represented exactly as a versioned edge.
- MACD is normalized by ATR and returns null on insufficient/invalid data.
- ADX uses Wilder smoothing, is direction-signed, and returns null on insufficient data.
- Market-wide and sector rows cannot collide in persistence.
- Sector rows never change edge_market_status; only market/all evidence can do so.
- Unknown/thin sectors abstain rather than report zero or fabricate confidence.
- Added factors trigger no additional provider request within a run.
- Formula, as-of slicing, market isolation, segment isolation, and thin-sample tests pass.
- Every persisted report names the benchmark source and per-provider symbol counts.
- Current-provider historical-candle evidence is labeled retrospective, not PIT-safe.
- TypeScript, Vitest, production build, migration verification, and security checks pass.

9. Build Order

- Version and add the three bounded factor definitions.
- Extend IC evaluation with optional sector maps and explicit segments.
- Migrate historical IC rows to market/all and update the unique key.
- Load display/profile sector metadata at the measure-only route boundary.
- Run US and India diagnostics; allow the existing evidence floor to abstain.
- After enough evidence, compare parameter plateaus and decide whether any native score change deserves a separate proposal.

10. Explicit Non-Goals

- No score/weight/threshold change.
- No Vibe app, agent, MCP, provider, or broker installation.
- No per-sector production parameters.
- No automatic selection of the best in-sample trial.
- No claim that retrospective current-universe history is point-in-time or free of survivorship bias.

11. Data-Truth Finding (2026-07-31)

Production distributions show the live technical score is heavily saturated: exactly 100 on 19.3% of US observations and 40.2% of India observations over 45 days. Exactly 20 occurred on 19.6% US and 13.6% India. The standalone bottom-quartile weak-close veto was removed after matured cohorts failed to support it as a hard breakdown classifier; ATR-scaled and high-volume breakdown conditions remain.

The 100 ceiling is not changed here. Add it to the versioned measure-only comparison and require market-local out-of-sample evidence before proposing a new live formula. Full production audit: `features/scoring-data-truth/FEATURE_ARCHITECTURE.md`.

trade behavior mirror - Feature Architecture

Source: features\trade-behavior-mirror\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Trade Behavior Mirror - Past-Trade Analysis - Feature Architecture

STATUS: DRAFT - NOT APPROVED. Proposal for review. No code until explicit sign-off.

Owner decision pending. Created 2026-07-10.

1. Intent (owner's ask, restated)

Import all past Robinhood transactions. For each historical buy/sell, reconstruct the fundamental + technical values *at the time* of the trade, compare to those same stocks now, and have the Mentor characterize the owner's behavioral / mental pattern - do you buy strength or dips, sell winners too early, average down on losers, panic in drawdowns - grounded in the actual indicator values at each entry/exit, not vibes.

2. What already exists (partial build)

- Import: Robinhood CSV -> trade_decisions (symbol, action, qty, exec_price, exec_date).
app/api/live-portfolio/import-csv/route.ts.
- Enrich: per trade -> macro regime at trade date, forward price 1d/1w/1m/3m, outcome_score (was the buy/sell right), pattern_tags (win/loss/breakeven + regime). app/api/live-portfolio/enrich/route.ts.
- Mentor reads trade_decisions (thesis/evaluate/ask) - comments on outcomes.
- Trades embedded into RAG (trade_memories, migrations 043/045).

3. The gap (the core of the ask - NOT built)

- ? Point-in-time fundamental + technical values at each trade. Enrich pulls forward *price* only - never "RSI/P-E/ROE/sentiment when you bought." So it cannot say *"you bought when RSI was 82 and P/E was 60."*
- ? Then-vs-now comparison per stock.
- ? Behavioral fingerprint synthesis - the mental-pattern mirror itself.

4. Why it's non-trivial

- Technical PIT (RSI/EMA/RS/52w-distance at exec_date): CHEAP - computable from the daily series enrich already fetches (TIME_SERIES_DAILY_ADJUSTED). No new data cost.
- Fundamental PIT (P/E in 2021): NOT free - AV OVERVIEW is a *current* snapshot only. Must reconstruct: historical price x historical EPS from FinancialDatasets historical income statements (MCP available). Higher cost, per-symbol historical fetch.

5. Proposed design - 3 layers

Layer A - Technical PIT fingerprint (cheap, high value). For each `trade_decisions` row, compute at `exec_date` from the daily series: RSI(14), EMA20/50 position, 20d trend, relative strength vs SPY/^NSEI, 52w-high distance, volume state. Store on the row (new columns, additive migration). Also snapshot each symbol's values now -> then-vs-now delta. Reuses `computeTechnicals` (`lib/data/technicals.ts`) fed a PIT-sliced series.

Layer B - Fundamental PIT. For each trade, fetch historical income statement / EPS at/around `exec_date` from `FinancialDatasets`; derive P/E (`exec_price` x shares / historical earnings), margin, growth. Store PIT + now. Day-cached

- budget-guarded. Degrades gracefully when history is missing (Layer A still stands alone).

Layer C - Mentor behavioral fingerprint. A deterministic pass clusters entries/exits by their PIT signatures and computes behavioral metrics: entry-RSI distribution (chase vs dip), win-hold vs loss-hold duration (cut winners early?), add-to-loser rate (averaging down), drawdown-exit timing (panic sells), regime performance. The Mentor LLM then **narrates** these deterministic findings (never invents the numbers) -> `mentor_insights`. This is the mirror.

6. Data model (additive, draft)

`trade_decisions` new columns (nullable, backfilled by re-enrich): `rsi_at_trade`, `ema20_pos_at_trade`, `ema50_pos_at_trade`, `rs_vs_bench_at_trade`, `dist_52w_high_at_trade`, `pe_at_trade`, `roe_at_trade`, `rev_growth_at_trade`, `tech_now_json`, `fund_now_json`, `fingerprint_json`.

New: `behavior_fingerprint` (one row per re-analysis run: entry-style, exit-discipline, averaging, regime-skew metrics + Mentor narrative).

7. Guardrails

- Owner-only (all `live-portfolio/*` already `requireOwner`).
- Read/analysis only - never places or alters live/paper orders.
- PIT reconstruction must not leak future data into the "at-trade" snapshot.
- Mentor narrates deterministic metrics; it does not fabricate indicator values.
- `FinancialDatasets` fetches go through the day-cache + budget guard.

8. Phasing

- P0 - Layer A (technical PIT + then-vs-now). Additive migration; re-enrich backfills.
- P1 - Layer C on Layer-A signatures (behavioral fingerprint + Mentor narration).
- P2 - Layer B (fundamental PIT) once A+C prove useful.

9. Open questions for review

- Start with Layer A only (cheap, fast) or commit to A+B+C up front?
- Benchmark for at-trade relative strength: SPY only, or SPY + sector?
- How far back does the Robinhood history go (bounds the `FinancialDatasets` historical cost)?
- Behavioral metrics to prioritize - which patterns matter most to you to see first?

trading mandate - Feature Architecture

Source: features\trading-mandate\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature Architecture: Cross-Market Trading Mandates

Status

Architecture status: Approved by owner Approved date: 2026-07-12 Roles: Codex Architect + Builder Implementation allowed: Yes Implementation status: Complete and migration 168 applied to FinanceOS production

Purpose

Give the user an honest, enforceable description of how agents trade without pretending that a few exit percentages constitute a different investment strategy. A mandate is configured independently for US and India and separates:

- investment horizon: short_swing, swing, or position;
- strategy preference: adaptive, momentum, balanced, or value_quality;
- horizon governance: user or agent;
- existing-position policy: grandfather or apply.

Day trading and genuine long-term investing are out of scope. The approved platform remains long-only and uses once-daily decisions over 2-20 market days.

Product Contract

Horizon	Market-day range	Default	Entry	Stop	Target
Short swing	2-5	5	65	5%	12%
Swing	5-15	10	60	7%	20%
Position	10-20	20	58	10%	30%

Risk profile remains orthogonal: it controls sizing, concentration, and kill switches. A mandate controls horizon, evidence emphasis, and exit geometry.

Strategy preferences use bounded deterministic multipliers before weight renormalization. They do not invent evidence and cannot make an unavailable dimension available:

- adaptive: champion/default weights unchanged;
- momentum: technical 1.25, sentiment 1.10, fundamental 0.85;
- balanced: all 1.00;
- value_quality: fundamental 1.30, insider 1.10, technical 0.85, sentiment 0.85.

The exact effective weights, mandate version, and governance source are stored with every decision observation and signal.

Authority And Precedence

- Hard safety, liquidity, account, currency, and kill-switch controls always win.
- The user mandate selects horizon and strategy preference per market.
- With horizon_governance=user (default), the mandate horizon overrides a champion genome's horizon. The champion still controls its other validated fields.

- With `horizon_governance=agent`, a valid promoted champion may select horizon within the mandate's min/max range; out-of-range values are clamped.
- Changing a mandate never promotes or mutates a champion.
- `LearnerAgent` may propose challengers but cannot change the mandate.

Existing Positions

Default grandfather: each new paper fill stores a mandate snapshot and resolved horizon. `PositionMonitor` uses the stored horizon for that position, so changing Settings does not cause surprise exits. `apply` deliberately applies the current mandate to open positions on the next monitor run and is visibly labeled.

Cross-Agent Flow

- Settings: owner-only read/write for US and India mandates.
- `ResearchAgent`: resolve mandate by symbol market; tilt only applicable+available dimension weights; use the mandate entry threshold; persist provenance.
- `PaperTrader`: require signal/market match; resolve the same mandate; use mandate stop, target, and horizon; persist snapshot on position/trade/order event.
- `PositionMonitor`: count market weekdays, not calendar days; use the fill snapshot under grandfather policy; otherwise resolve current mandate/champion precedence.
- `TraderAgent/live` proposals: include mandate snapshot and horizon in proposal rationale/audit. Existing approval, account, LTCM, and notional gates remain.
- `LearnerAgent`: receives mandate as immutable context. Challengers outside user horizon bounds are ineligible; it cannot write mandate rows.
- `Backtest/validation`: run and report results under the same resolved mandate, market calendar, factor tilts, threshold, stop, target, and horizon.
- `Explainer/journal`: show selected mandate, effective weights, resolved horizon, governance source, and whether an existing position was grandfathered.

Data Contract

New `trading_mandates` table, one row per market:

- market primary key (us|india)
- `horizon_style`, `strategy_preference`, `horizon_governance`
- `min_hold_days`, `target_hold_days`, `max_hold_days`
- `score_threshold`, `stop_loss_pct`, `target_pct`
- `existing_positions_policy`
- `version`, `timestamps`, `updated_by`

New nullable provenance columns on paper positions and trades:

- `mandate_version`

- mandate_snapshot jsonb
- resolved_horizon_days

All writes are owner/service-only. Reads fail closed for trade-affecting paths. Pre-migration code falls back to the behavior-preserving Position/Balanced mandate and reports the source as default.

Market Rules

- US and India mandates are independent; no currency or configuration blending.
- Holding age uses each market's weekdays. Exchange-holiday support remains a calendar-service follow-up; weekends are never counted.
- US and India can use different strategies because evidence coverage differs.

Missing dimensions are still excluded before renormalization.

Non-Goals

- Intraday/day trading, leverage, shorting, options, or crypto.
- A cosmetic long_term label for a 20-day strategy.
- Automatic live enablement or order submission.
- Agent mutation of user mandates.
- Retroactive rewriting of historical signals, fills, or outcomes.

Acceptance Gates

- Per-market settings round-trip independently.
- Research effective weights and thresholds match the selected mandate.
- Unavailable evidence remains unavailable after tilting.
- Paper fills persist a mandate snapshot and resolved horizon.
- Grandfathered positions retain their entry mandate after Settings changes.
- User governance overrides champion horizon; agent governance clamps it.
- US and India tests prove no mandate/currency leakage.
- No live order path is enabled or invoked.
- Tests, typecheck, production build, and dependency audit pass.

Implementation Result

Implemented 2026-07-12 across Settings, ResearchAgent, PaperTrader, PositionMonitor, LearnerAgent, and the US proposal-building TraderAgent. Migration 168 was applied directly as one reviewed SQL file to the verified linked FinanceOS project (dionkikgdm1aotvtbnfr) because its timestamped migration ledger does not align with the repository's numbered local ledger and a broad CLI push could replay unrelated migrations. The table, US/India seeds, provenance columns, and owner-read RLS policy were verified from the production schema after application.

Paper Entry Cadence Correction (2026-07-21)

- Research produces signals only; it never invokes PaperTrader inline.

- Exactly one `pg_cron` job per market attempts paper entries each regular session.
- PaperTrader independently verifies the market-local session and holiday calendar, so UTC/DST drift or a manual invocation cannot create an off-session fill.
- US runs at 15:15 UTC (inside both EDT and EST sessions); India runs at 04:10 UTC (09:40 IST). Both retain same-session signal freshness and all existing gates.
- A rejected fill RPC is written to `pipeline_stage_events` and summarized by reason in `agent_runs.workload_metrics`; observability never changes eligibility.
- The per-market 10-name default remains a concentration gate, not a cash-use target. Idle cash does not authorize an eleventh name or averaging down.

us paper fractional shares - Feature Architecture

Source: features\us-paper-fractional-shares\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

US Paper Fractional Shares

Status: APPROVED FOR IMPLEMENTATION

Owner approval: 2026-07-26

Scope: US paper book only

Problem

The US paper pool holds cash that can fund qualified positions, but PaperTrader rounds all quantity down to a whole share. A valid US allocation below one share becomes an `insufficient_cash_for_1_share` rejection even though US fractional execution is available. Production on 2026-07-26 had \$5,178.58 cash and multiple such skips.

Decision

Allow US paper entries, rotations, and target partials to use quantities rounded down to six decimal places. India remains whole-share because this app has not established fractional NSE execution semantics. `qty` and all relevant paper RPC arguments are already numeric, so no schema migration is needed.

The US paper mandate increases from 10 to 15 alpha names. It is a per-market setting; it does not modify India, existing positions, stops, targets, score thresholds, or live-order limits.

Rules

- US entry quantity: `floor(maxSpend / price, 6 decimals)`; must be finite and > 0 .
- India entry quantity: unchanged `floor(maxSpend / price)`; must be ≥ 1 .
- US partial target: sell half rounded down to six decimals only when both the sell and remaining legs are positive. Otherwise close the full position.
- All cash, notional, caps, name/sector limits, rotation gates, and transactional RPC checks run unchanged and continue to receive the exact numeric quantity.
- No live broker call or live quantity behavior changes.

Acceptance

- A \$100 US allocation can buy a fractional share priced above \$100.
- An equivalent India allocation remains rejected if it cannot buy one share.
- US partial profit-taking preserves quantity and cash conservation.
- Decimal quantities pass the existing fill/exit/rotation RPC contracts.
- Tests cover rounding, no dust/NaN, US-vs-India isolation, and partial exits.

walk forward ic folds - Feature Architecture

Source: features\walk-forward-ic-folds\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature Architecture: Purged Out-of-Sample IC Validation

Status

Architecture status: MEASURE-ONLY - promotion dormant Architecture approved: Steps 2-3 shipped; US step 4 shipped; India step 4 blocked Approved scope: diagnostics and evidence hardening only Approved date: 2026-07-28 Implementation allowed: Measurement work may continue; policy promotion may not.

Outcome. The stack produced useful negative evidence: mom_12_1 has no useful signal at h5 in this sample, and daily IC is noisy. The study did not prove an effective breadth of 17 or that universe size and sector neutralization cannot help. Temporal IC variance combines sampling noise, genuine factor variation, changing membership, and coverage; the current artifact does not identify those components.

POST /api/agents/backtest/promote remains disabled. EdgeScout/EdgeIC continue as advisory diagnostics and must never be cited as promotion evidence.

Next measure-only work must record usable cross-section per date, compare universe sizes on matched dates, measure sector-neutral IC with point-in-time sector provenance, and obtain matched-horizon evidence. None of those results may change scoring or trade authority directly.

2026-07-28 correction build: OOS report schema v2 now retains the usable cross-section for every successful date plus the resolved universe fingerprint, members, provenance, and dates to which each snapshot was applied. OOS Spearman IC now ranks raw finite edge values; it no longer winsorizes first and creates artificial tail ties. This improves future diagnostics only.

Remaining order:

- Use per-date membership plus required snapshot persistence for every promotion-grade run; India remains blocked on a PIT membership source.
- Bind experiments to edge id, formula version, market, horizon, and immutable

OOS run fingerprint.

- Run matched-date n=200/n=400 and point-in-time sector-neutral diagnostics.
- Run a separately predeclared h20 design with HAC-lag sensitivity and costs.
- Only then decide whether a promotion statistic and floor are supportable.

Items 1-4 are not interchangeable. Running experiments without per-date membership and required persistence would spend provider quota on another non-promotion-grade artifact.

US step 4 shipped and verified: us_pit_adv20_top400_v2 computes average dollar volume over exactly 20 market sessions ending at each as-of date, shares session fetches across overlapping windows, fails closed on an incomplete window, and persists a top-400 superset whose deterministic prefixes support matched n=200/n=400 tests.

persist_edge_pit_snapshot () writes through a service-role-only, advisory-locked RPC; exact reruns are idempotent, differing reruns conflict, and PIT rows are append-only. The orchestrator must set persistSnapshots=true and membershipCadence="per_date" for promotion-grade output. Defaults remain diagnostic-only.

Implementation Status (2026-07-28)

Steps 2 and 3 are SHIPPED. Migration 20260728090000_promotion_schema_repair_and_atomic_rpc.sql, applied and verified against production while strategy_policies and backtest_experiments were both empty:

```
| Repair | State |
| `dsr` renamed `t_margin_vs_trials` (it was never a Deflated Sharpe Ratio) | done |
| `walk_forward_pass` renamed `ic_stability_pass` | done |
| `validation_mode` NOT NULL, CHECK in (`purged_temporal_oos`, `walk_forward`) | done - a legacy rolling-
window result has no representable value, so it cannot be promoted |
| `experiment_id` NOT NULL FK to `backtest_experiments` | done |
| Mutation trigger compares whole rows minus `superseded_at` | done - the hand-listed column set left 7
fields silently mutable |
| `superseded_at` and experiment `policy_id` are write-once | done |
| `promote_strategy_policy()` RPC, SECURITY DEFINER, service-role only | done |
| Promote route calls the RPC instead of supersede-then-insert | done |
| `experiment_id` required; `variants_run` must be recorded | done - the old fallback chain substituted a
flattering trial count when the run count was missing |
```

P0 proven in production 2026-07-28: inside one transaction, two promotions produced base line then variant with exactly one active row; a third promotion was then forced to fail at insert AFTER its supersede, and the incumbent survived unchanged. The whole block was rolled back, leaving both tables at zero rows.

Promotion remains DORMANT at the route (promotion_evidence_not_oos, 503). Steps 2-3 fixed how a policy would be written. They did nothing about whether the evidence deserves to be written, which is steps 4-10.

Decision

Build this before any strategy_policies promotion is allowed.

The current edge_ic_history rows are useful retrospective diagnostics, but they are not promotion evidence. Splitting the same retrospective current-name dataset into date ranges would not repair point-in-time universe bias, experiment-selection bias, or the absence of a frozen train/test protocol.

Until this feature and the atomic promotion RPC are approved and shipped, POST /api/agents/backtest/promote must remain operationally dormant. A later approved implementation should make the route fail closed with promotion_evidence_not_oos rather than treating three rolling windows as an evidence minimum.

Purpose

Create reproducible, market-local, point-in-time out-of-sample evidence for an edge before it can become a policy. The system must answer:

- What formula and trial family were fixed before evaluation?
- What symbols and inputs were knowable on each as-of date?
- Which observations trained or calibrated the formula?
- Which later observations tested it without label overlap?
- Does the combined out-of-sample evidence survive dependence, costs, and multiple testing?
- Can promotion append a replacement policy atomically?

Measured Current State

As of 2026-07-27:

```
| Fact | Production value |
| Rolling history per IC row | 1,000 calendar days |
| US rows per edge/horizon vs distinct end dates | 6 vs 4 |
```



```
| Span from first to last US end date | 16 calendar days |
| Approximate overlap | 98.4% |
| Best latest US market-wide t-stat | 1.73 |
| Best latest India observation | 1.57, Financials sector, only 2 windows |
| Active `strategy_policies` | 0 |
| `backtest_experiments` | 0 |
| Promotion-grade PIT universe | US infrastructure shipped; India unavailable; no promotion run accepted yet |
| Regime rows allowed by `edge_ic_history` schema | no |
```

The current universe is documented in migration 132 and the EdgeIC response as a current-liquid snapshot. Replaying today's survivors through old dates is not point-in-time validation.

Corrections To The Previous Draft

- Disjoint test slices are not automatically walk-forward. For a fixed, never-refit formula they are purged temporal out-of-sample folds. The term walk-forward is reserved for a protocol that freezes each fold's formula or parameters using only its earlier training/calibration interval and then evaluates a later test interval.
- Purge is measured in market sessions, not calendar days. For horizon H, no test origin may have a forward-return label extending beyond its test interval. Training observations whose labels touch the test interval are purged; an embargo is added only when the feature construction requires it.
- A boolean `is_disjoint` is not proof. Boundaries, label end dates, fingerprints, and non-overlap validation are durable facts. The gate recomputes their consistency.
- Fold index is not a global identity. It is unique only inside a frozen experiment/validation plan.
- Current-universe history cannot promote. Point-in-time membership, delistings, symbol changes, corporate-action adjustment policy, and input availability are required.
- Fold-over-fold endpoint decay is not the primary statistic. The primary evidence is the predeclared aggregate out-of-sample date-level rank-IC series with a dependence-robust uncertainty estimate. Fold sign consistency and a worst-fold guard are secondary diagnostics.
- The current `t - expectedMaxT` value is not the Bailey/Lopez de Prado Deflated Sharpe Ratio. It is a trial-count-adjusted t margin. DSR also accounts for sample length and non-normality of strategy returns. IC discovery should use a declared multiple-testing procedure; DSR belongs on cost-adjusted strategy-return evidence.

Statistical Contract

Unit of observation

- Primary series: one cross-sectional rank IC per eligible market session/as-of date.
- The same-market cross section must meet a predeclared minimum symbol count.
- US and India observations, calendars, universes, currencies, benchmarks, and results are never pooled.
- Sector evidence is a separate plan and must meet stricter sample floors.

- Regime evidence is unavailable until a separate approved, PIT-safe regime label exists. It must not silently fall back to market-wide evidence.

Frozen plan

Before any result is computed, an immutable experiment records:

- `edge_id` and `exact formula_version`
- market, horizon, and exactly one segment dimension
- trial-family identifier and total challengers considered since the current champion was promoted
- point-in-time universe policy/version and snapshot fingerprint
- feature/input availability policy and adjustment policy
- train/test boundaries expressed as market sessions
- step size, purge sessions, optional embargo sessions, fold count
- minimum as-of dates and minimum cross-section size
- primary statistic, secondary diagnostics, and pass thresholds
- data cutoff and code/config fingerprint

Changing any field creates a new experiment. It never edits a completed one.

Fold construction

For fold `k`:

- Build or calibrate only from sessions at or before `train_end[k]`.
- Purge training origins whose H-session labels overlap the test interval.
- Freeze the selected formula/parameters.
- Evaluate consecutive later sessions from `test_start[k]` through `test_end[k]`.
- Require every label to mature by `label_end[k] <= data_cutoff`.
- Append immutable date-level IC observations and one fold summary.

Test intervals do not overlap. Their label intervals do not cross fold boundaries. Folds are generated from an anchored plan, never from the cron's wall-clock execution date.

If there is no training or calibration step because the formula was fixed externally before all folds, label the mode `purged_temporal_oos`, not `walk_forward`.

Aggregation and pass criteria

The gate consumes all matured OOS date-level IC observations for one frozen plan:

- minimum 3 completed test folds
- minimum 24 eligible as-of dates per fold, subject to power analysis before approval

- minimum cross-section size per as-of date; sparse dates are excluded and counted, never converted to zero IC
- aggregate mean rank IC above the approved floor
- Newey-West/HAC t-stat on the concatenated chronological OOS IC series above the approved hurdle
- positive fold sign in a predeclared majority and no catastrophic worst fold
- multiple-testing control against the complete frozen trial family
- cost-adjusted long-only bucket test and benchmark comparison before policy promotion

- no unresolved PIT, provider, adjustment, or provenance degradation

No single latest fold, maximum fold t-stat, or pooled standard error from overlapping windows may decide promotion.

For factor discovery, use a predeclared false-discovery method appropriate to the trial dependence. Do not label the current expected-max-t subtraction as DSR. If DSR is later used, compute it from the strategy-return series with the paper's sample-length, skewness, kurtosis, and trial-selection inputs.

Data Architecture

Reuse the existing lineage

Do not create a third experiment/provenance truth layer.

Extend `backtest_experiments` to be the immutable plan header with:

- `edge_id`, `formula_version`, `horizon`
- `validation_mode`
- `trial_family_id`, `trials_considered`
- `universe_policy_version`, `universe_fingerprint`
- `dataset_fingerprint`, `code_version`
- `validation_spec` JSONB with a schema/version CHECK
- `data_cutoff`

The plan references existing `edge_signal_inputs`, `edge_universe_members`, and `evidence/provider` fingerprints. It does not copy or reinterpret their provenance.

New result tables

`edge_ic_fold_results`:

- `experiment_id`, `fold_index`
- train, purge, test, and label-end session boundaries
- formula and dataset fingerprints
- `as_of_dates`, `min_cross_section_n`, `excluded_dates`
- mean IC, IC IR, HAC t-stat, confidence interval
- status and machine-readable refusal reasons

- unique (experiment_id, fold_index)
- service-role only, append-only

edge_ic_oos_observations:

- experiment_id, fold_index, as_of_date
- rank IC, cross-section count, input/universe fingerprint
- unique (experiment_id, as_of_date)
- service-role only, append-only

edge_ic_history remains the retrospective diagnostic ledger. Do not add is_disjoint to it and then treat the same current-universe reconstruction as promotion-grade.

Strategy-policy schema repair

The promotion migration must:

- add the correctly named ic_stability_pass and explicit

validation_mode; deprecate walk_forward_pass

- stop describing the trial-adjusted t margin as dsr
- harden the mutation trigger so every field except a one-way

superseded_at: null -> timestamp transition is immutable

- bind each policy to its exact experiment and fold result set
- make experiment policy_id a one-way null-to-value transition

Production currently has zero policy rows, so this is the lowest-risk time to repair the schema semantics.

Atomic Promotion Boundary

Supersede and insert must occur in one database transaction. A partial unique index prevents two active rows but cannot prevent zero active rows after supersede succeeds and insert fails.

Add a service-role-only database RPC modeled on promote_strategy_champion:

- validate the experiment, segment, formula, market, horizon, trial family,

matured folds, and gate result inside the transaction

- acquire a transaction advisory lock for the exact policy segment
- lock the incumbent row
- append the new policy
- supersede the incumbent
- bind the experiment to the new policy
- return both IDs

Revoke execution from public, anon, and authenticated; grant only the service role. The HTTP route remains cron-or-confirmed-owner gated and calls only this RPC.

Module Inventory

```
| Module | Approved future change |
| `lib/edges/ic.ts` | Pure fold evaluator over frozen observations |
| `lib/edges/validation-plan.ts` | Session-aware plan validation and fingerprints |
| `app/api/agents/edge-ic/route.ts` | Keep retrospective diagnostics; do not overload it |
| new validation route/worker | Execute approved frozen plans and append OOS results |
| `lib/gates/promotion-gate.ts` | Consume aggregate OOS result, not rolling windows |
| `app/api/agents/backtest/promote/route.ts` | Validate request, require bound experiment, call atomic RPC |
| `supabase/migrations/*` | Extend lineage, add immutable results, repair policy semantics, add RPC |
| `docs/arch/04-database-schema.md` | Canonical schema and immutability contract |
| `docs/arch/09-learning-loop.md` | Discovery -> OOS validation -> promotion sequence |
| `public/agent-diagrams/system-map.json` | Remove all current walk-forward/DSR overclaims |
```

Failure Modes And Required Refusals

```
| Failure | Required outcome |
| Current-day universe replayed historically | refuse `universe_not_point_in_time` |
| Feature observed/revised after as-of date | refuse `input_not_point_in_time` |
| Formula/config changed after plan creation | refuse `plan_fingerprint_mismatch` |
| Label crosses test boundary or is immature | refuse `label_overlap_or_immature` |
| Fold/test range overlaps another result | refuse `test_fold_overlap` |
| Too few dates or symbols | refuse `sample_floor_not_met` |
| Experiment segment differs from request | refuse `experiment_scope_mismatch` |
| Trial count absent or lower than family ledger | refuse `trial_family_incomplete` |
| Any India/US mixing | refuse `market_scope_mismatch` |
| Promotion insert/bind fails | transaction rolls back; incumbent remains active |
```

Validation Plan

Pure tests

- US and India session calendars, including holidays
- horizon-label purge at every fold boundary
- no overlap in test origins or label intervals
- immature labels rejected
- exact rerun idempotency
- changed formula/universe/config creates a different plan
- sparse fold rejection
- aggregate HAC statistic uses chronological OOS observations
- no max-t or latest-fold cherry-pick
- malformed/mismatched trial family fails closed

Database tests

- anon/authenticated cannot read or mutate plan/results
- result rows cannot update/delete
- experiment and policy bindings are one-way
- concurrent promotion leaves exactly one active policy
- forced insert failure preserves the incumbent
- null/value segment branches match the unique-index key exactly

Acceptance

- A synthetic known-positive factor passes only under clean PIT OOS data.
- The same factor fails when its universe or labels are contaminated.
- A known-null factor does not pass under repeated trial families.
- No policy can be created from legacy `edge_ic_history` alone.
- Full typecheck, tests, build, schema verification, RLS checks, and production dry-run pass before activation.

Build Order

- Approve this architecture and thresholds/power analysis.
- Disable legacy-window promotion explicitly while the policy table is empty.
- Repair policy/experiment schema semantics and add the atomic RPC.
- Establish PIT universe and input eligibility for US and India separately.
- Extend the existing experiment lineage and add immutable OOS results.
- Build the session-aware fold engine and synthetic leakage tests.
- Backfill diagnostics only; never relabel legacy rows as OOS.
- Run paper-only validation plans and review at least one complete trial family.
- Wire the promotion route to the atomic RPC.
- Consider policy consumption only under a separate approved architecture.

Open Decisions For Approval

- Minimum dates and symbols per fold, determined by power analysis rather than an arbitrary `fold_days`.
- Expanding-window calibration versus externally fixed formula mode.
- False-discovery procedure for correlated edge families.
- Required cost-adjusted portfolio test and benchmark.
- PIT-universe source and delisting/corporate-action policy for each market.

References

- Bailey and Lopez de Prado, *The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting and Non-Normality*: <https://ssrn.com/abstract=2460551>
- Bailey, Borwein, Lopez de Prado, and Zhu, *The Probability of Backtest Overfitting*: <https://ssrn.com/abstract=2326253>
- Harvey, Liu, and Zhu, *... and the Cross-Section of Expected Returns*: <https://www.nber.org/papers/w20592>

These sources support multiple-testing and backtest-overfitting controls. The specific Kairos schema, market isolation, PIT requirements, and fail-closed design above are architectural decisions derived from the current code and production schema.

Annex A - Open Decisions #1 and #5: options with arithmetic (2026-07-28)

Prepared for owner decision. Nothing here is approved or implemented.

Back-derived diagnostics (not direct sigma measurements)

An effective dispersion proxy backed out of Kairos's own rows, using $\sigma = \text{mean_ic} * \sqrt{\text{as_of_dates}} / t_stat$ on the latest window of each market-wide edge/horizon. Because t_stat uses a HAC standard error, this proxy mixes sample dispersion with serial covariance and is not a direct sample sd:

Market	Horizon	Universe	As-of dates	Implied sigma_IC	Mean abs IC	Best abs IC
US	5	40	96	0.318	0.0192	0.0403
US	10	40	96	0.364	0.0279	0.0625
US	20	40	96	0.438	0.0308	0.0612
India	5	39	120	0.222	0.0133	0.0325
India	10	39	120	0.264	0.0127	0.0461
India	20	39	120	0.307	0.0203	0.0561

The theoretical cross-sectional rank-IC standard deviation for n independent names is approximately $1/\sqrt{n-3}$; at $n=40$ that is 0.164. US h20 shows 0.438, an effective inflation of ~2.7x. Label overlap is a plausible contributor, but the proxy does not identify how much comes from overlap, serial covariance, factor structure, universe construction, or regime concentration.

Decision #1 - sample floors

Required as-of dates for a target IC at hurdle T :

$$N = (T * \sigma_{IC} / IC_{target})^2$$

Two independent levers, and they are not equally priced.

Lever 1 - remove label overlap ($\text{step_days} \geq \text{horizon}$). If σ fell from the 0.438 HAC-implied proxy to ~0.164, N would fall by ~7.1x. But available dates/year falls from ~50 to ~12.5, costing 4x. This is a sensitivity scenario, not a measured 1.8x net gain.

Lever 2 - enlarge the universe. $\sigma \sim 1/\sqrt{n-3}$, and N scales with σ^2 , so N falls *linearly* in n . This costs no calendar time at all.

n	sigma_IC	vs n=40
40	0.164	1.0x
100	0.101	2.6x fewer dates
200	0.071	5.3x fewer dates
400	0.050	10.7x fewer dates

Years of history required, non-overlapping ($\text{step}=\text{horizon}=20$, ~12.5 dates/year), at $T=2.0$:

True IC	n=40	n=100	n=200	n=400
0.03	9.6 yrs	3.6 yrs	1.8 yrs	0.9 yrs
0.05	3.4 yrs	1.3 yrs	0.7 yrs	0.3 yrs
0.06 (best observed)	2.4 yrs	0.9 yrs	0.5 yrs	0.2 yrs

At $T=3.0$ (Harvey/Liu/Zhu for newly proposed factors) multiply every cell by 2.25.

The conclusion is blunt: at $n=40$ nothing in the current registry is provable this decade. Universe size is the dominant lever and it is free.

Options

```
| Option | Floors | Consequence |
| **1A Keep n?40** | any | Rejected. Even IC=0.06 needs 2.4 yrs at T=2.0 and 5.4 yrs at T=3.0. Weak edges are permanently unprovable. |
| **1B n>=100, 12 dates/fold, 3 folds** | 36 dates, ~2.9 yrs | Detects IC>=0.05 at T=2.0. Marginal at T=3.0. |
|
| **1C n>=200 US / n>=100 India, 12 dates/fold, 3 folds (RECOMMENDED)** | 36 dates, ~2.9 yrs | Detects IC>=0.035 at T=2.0 and IC>=0.05 at T=3.0. Reachable with existing free data. |
| **1D n>=400, 8 dates/fold, 3 folds** | 24 dates, ~1.9 yrs | Strongest, but 400 liquid US names stretches "liquid" and multiplies fetch cost. |
```

Recommended floors under 1C: min_symbols_per_as_of_date 200 US / 100 India; min_as_of_dates_per_fold 12; min_folds 3; step_days = horizon; declared IC floor 0.04, replacing the current 0.02 - which the table shows is below the detection limit at any realistic sample.

Verify before adopting: the system map records Yahoo at "251 bars" while edge_ic_history.history_days is 1000. 1C needs ~3 years of daily candles for 200 US names. If the 251-bar cap is real, deeper history must be sourced first and 1C is blocked until then.

Decision #5 - point-in-time universe

The current list is a hand-picked, current-liquid snapshot (lib/edges/universe.ts says so in its own header). Replaying today's survivors through past dates is survivorship bias, and it cannot be fixed by waiting.

```
| Option | Survivorship fixed? | Cost | Notes |
| **5A Massive PIT tickers (RECOMMENDED, US)** | Yes | MCP already connected; plan limits unverified | `GET /v3/reference/tickers` takes `date` (membership as of that date) and `active=false` (delisted names). `vX/reference/tickers/{id}/events` covers renames. This is a real PIT source already wired in. |
| **5B Reconstructed liquidity screen** | **No** | Free | Rank by trailing dollar volume using only data <= as-of date, take top N. Removes *selection* hindsight but NOT delisting survivorship - the dead names are absent from the symbol list to begin with. **Only sound layered on top of 5A.** |
| **5C Freeze-forward (run in parallel)** | Yes, by construction | Free, already running | `edge_universe_members` has snapshotted since 2026-07-09 (805 rows, 9 dates). Validate only on data at/ after the first snapshot. Zero bias, but ~19 days of history today, so first promotable evidence is years away. |
| **5D Index-membership history** | Yes | Free-ish for NIFTY via NSE archives; S&P harder | Manual/scraped, ongoing maintenance. Fallback for India if 5A lacks NSE coverage. |
```

Recommended: 5A + 5B for US (PIT membership, then a PIT liquidity rank within it), 5D for India if Massive's NSE coverage is thin, and 5C running in parallel indefinitely as the unimpeachable ground truth that eventually supersedes the reconstruction. Any disagreement between 5C and 5A/5B once they overlap is a bug in the reconstruction and must fail closed.

Verify before adopting 5A: Massive plan limits for /v3/reference/tickers with active=false, and whether NSE/India is covered at all. Both are unknown.

What these two decisions do NOT settle

Even at 1C + 5A, the binding constraint stays the t-hurdle. Best current latest-window t is 1.73 (US) / 1.04 (India). A larger universe and clean PIT data change the *precision* of the estimate, not its *sign* - they make a real edge provable and a fake edge refutable. Neither manufactures an edge that is not there.

Annex B - Verification of the two unknowns (2026-07-28)

Both open questions from Annex A were tested against the live providers. One blocker is real, one dissolved, and a third problem surfaced that Annex A missed.

B1. Candle depth - RESOLVED, and the fix already exists

Massive (Polygon-backed) has a hard 2-year lookback wall on the current plan.

```
- /v2/aggs/ticker/AAPL/range/1/day/2022-07-01/2026-07-27 with limit=5000
```

returned exactly 500 bars, 2024-07-29 -> 2026-07-27.

```
- Requesting only the older window 2022-07-01 -> 2023-07-01 returned
```

HTTP 403 NOT_AUTHORIZED - "Your plan doesn't include this data timeframe."

This matters because `resolveCandles()` routes US through Massive first (`lib/edges/data.ts`), so US IC history is capped at ~2 years. Net of a 252-day feature lookback (12-1 momentum), that leaves only ~12 usable non-overlapping 20-day as-of dates. 1C would have been unbuildable on US.

However - Yahoo's auth-free chart endpoint serves 5 years for BOTH markets, free, and Kairos already has the code. Measured:

```
| Symbol | range=5y | range=3y |
| AAPL | 1254 bars, from 2021-07-28 | 751 bars, from 2023-07-28 |
| RELIANCE.NS | 1239 bars, from 2021-07-27 | 742 bars, from 2023-07-27 |
```

`fetchIndiaCandles(symbol, range)` in `lib/india-data.ts` is a generic Yahoo chart fetcher. It is India-only by naming accident, not by capability - it takes any symbol and any range, and `indiaRange()` already maps >900d -> 3y and 1400d -> 5y. The 251-bar figure in the system map is simply the 1y default.

Proposed (NOT approved, step 4 scope): rename it to `fetchYahooCandles` and use it as the DEEP-HISTORY source for both markets in the validation path, while Massive stays primary for recent/live US data. No new provider, no new key, no cost.

Recomputed 1C feasibility with 5 years (~1254 sessions, minus a 252-day feature lookback, non-overlapping at h20 -> ~50 as-of dates):

```
| n | Detectable IC at T=2.0 | at T=3.0 |
| 100 | 0.029 | 0.043 |
| 200 | 0.020 | 0.030 |
```

1C becomes fully buildable: 3 folds x 16 dates, comfortably above the 12/fold floor, with the proposed 0.04 IC floor safely above the detection limit.

B2. Massive PIT coverage - CONFIRMED for US, ABSENT for India

US: works. GET `/v3/reference/tickers` with `date=2023-06-30` and `active=false` returned genuine delisted names with `delisted_utc` timestamps (e.g. Altaba, delisted 2019-10-07; Advanced Accelerator Applications, 2018-02-12). This is a real point-in-time membership source with survivorship intact, already connected, no new vendor.

India: not covered. Searching RELIANCE returned only US-listed and OTC instruments - Reliance Global Group (NASDAQ), Reliance Inc (NYSE), and OTC GDRs. No NSE listings. Massive is US-equities-only for this purpose.

B3. Consequence Annex A did not anticipate

The two markets have opposite gaps:

```
| | PIT universe source | Deep candle history |
| US | 5A Massive - **yes** | Massive capped at 2y; **Yahoo 5y fixes it** |
| India | Massive - **no**, needs 5D | Yahoo - **yes, 5y** |
```

Neither market has both from one provider. The recommended pairing is therefore asymmetric by necessity, not by preference:

```
- US: 5A (Massive PIT tickers) for membership + Yahoo 5y for candles + 5B
```

liquidity rank within the PIT set.

- India: 5D (NSE index-membership archives) for membership + Yahoo 5y for

candles. Until 5D exists, India has no PIT universe and therefore cannot produce promotion-grade evidence at all - it stays diagnostic-only.

- Both: 5C freeze-forward continues in parallel as ground truth.

This means US reaches promotable evidence materially before India. That is an honest consequence of the data available, not a choice - and it should be recorded rather than papered over by letting India promote on a survivorship- biased universe.

Revised recommendation

Adopt 1C + 5A/5B (US) + 5D (India, blocked until built), with the Yahoo deep-history change as a prerequisite for both. Do not relax the fold floors to accommodate the 2-year Massive wall - the wall is removable for free, and lowering statistical floors to fit a provider limit would be exactly the kind of accommodation this whole feature exists to prevent.

B4. Why the brokers (Robinhood / Webull / Kite) do not solve either problem

Asked during review: can the broker APIs supply what is missing?

For #5 (PIT universe) - structurally impossible, not a plan limitation. A broker API lists instruments you can trade *today*. That is the literal definition of a survivorship-biased universe. Robinhood has no reason to serve a name that delisted in 2019, and does not. `active=false` on the Massive reference endpoint exists precisely because a *market-data* vendor has an incentive to retain dead tickers, while a *broker* has none. No broker integration can fix survivorship bias, regardless of entitlement or plan.

For #1 (candle depth) - possible in principle, unnecessary in practice, and poorly shaped for it.

- Broker historicals are entitled and rate-limited for a personal account, not

for a 200-symbol nightly backfill across 5 years.

- Robinhood MCP requires an interactive OAuth session; the token is not available to unattended validation jobs on the same terms as an open HTTP endpoint.

- Webull's data tools are recorded in this repo as requiring `category:US_STOCK`, returning quarterly fractions, being cron-entitled, and the adapter is currently dead.

- Kite is India-only and account-scoped.

- Yahoo already supplies 5 years for both markets, free, keyless, with existing code (B1). Adding a broker dependency to solve a solved problem would trade a keyless endpoint for an entitled, rate-limited, auth-bound one.

Conclusion: the brokers stay what they are - execution and live account state. They are not a research-data source, and routing validation evidence through them would couple the evidence layer to trading entitlements. The existing separation is correct and should not be relaxed.

Annex C - Correction to Annex B, and the step-4 scope boundary (2026-07-28)

C1. Annex B overstated the Massive dependency - corrected

Annex B said "`resolveCandles()` routes US through Massive first, so US IC history is capped at ~2 years." That is true only of the edge/IC path. It is not true of US candles generally, and the difference matters.

There are two separate US candle resolvers:

```
| Resolver | Used by | US order | Depth |  
| `resolveCandles()` in `lib/edges/data.ts` | edge lab / IC | Massive -> EODHD -> TwelveData | **capped at  
2y by the Massive plan** |  
| `fetchUsCandles()` in `lib/data/candles.ts` | main research path | **Yahoo first**, then Massive -> EODHD  
-> TwelveData -> AV | governed by the range passed |
```

So the main US path was already Yahoo-first. Only the edge/IC path - the one this feature depends on - goes to Massive first and hits the 2-year wall. The Annex B conclusion for step 4 stands; the general claim about "US candles" did not, and is withdrawn.

C2. Correction: the live US candle path is not under-served

The earlier text described a stale API shape. `lib/data/candles.ts` owns a local `fetchYahooCandles(symbol)` that explicitly requests `range=1y`, returning roughly 251 sessions. It is not the market-agnostic deep-history function with a `range` argument.

The production technical score uses RSI14, EMA20/50, ATR14, 20-day trend, and 20-day volume. One year is sufficient for those inputs. `mom_12_1` is an edge lab formula and uses the separate OOS candle resolver with explicitly requested deep history; it is not computed by the live ResearchAgent technical score.

Therefore there is no pending live candle-depth fix and no reason to change production scoring inputs. The system-map YAHOO node now records the actual one-year path and the separate deep-history role.

C3. Step 4 is NOT unblocked by the 1C + 5A/5B approval alone

Owner approved 1C (sample floors) and 5A/5B (PIT universe source) on 2026-07-28. Those answer Open Decisions #1 and #5.

Three remain open, and the approved scope line at the top of this document says steps 4-10 are blocked on all five:

```
| # | Decision | Blocks |  
| 2 | Expanding-window calibration vs externally fixed formula | Fold construction - whether a fold has a  
train segment at all, and therefore whether the mode is `walk_forward` or `purged_temporal_oos` |  
| 3 | False-discovery procedure for correlated edge families | Aggregate pass criteria (step 8) |  
| 4 | Required cost-adjusted portfolio test and benchmark | Final promotion criteria (step 9) |
```

#2 genuinely blocks step 4. Whether folds carry a training segment determines the fold boundary layout, the purge rule, and which `validation_mode` value the schema will accept - all of which step 4 must fix before any universe snapshot is keyed to a plan.

#3 and #4 do not block step 4. They gate steps 8-9. Step 4 could proceed once #2 is answered.

Recommendation: answer #2 before step 4 starts; #3 and #4 can be decided while steps 4-7 are built.

Annex D - Open Decision #2: calibration mode (2026-07-28)

Recommendation: `purged_temporal_oos`. Not a preference - the codebase already determines it.

Evidence

Nothing in the edge pipeline is fitted from data.

```
| Check | Result |  
| `grep -c 'fit\|train\|calibrat\|regress\|optimi[sz]e\|learned'` over `lib/edges/registry.ts` and `lib/  
edges/compute.ts` | **0 matches in both** |  
| Edge formulas | Closed-form with hardcoded constants - `mom_12_1` reads `c[n-22] / c[n-253] - 1`; MACD  
is fixed 12/26/9; ADX is fixed 14; 52w proximity is a fixed lookback |  
| `crossSectionalZ` / `winsorize` (`lib/edges/standardize.ts`) | Computed **per (date, market, edge) across  
symbols** - purely cross-sectional, no time dimension, so no temporal parameter to leak |
```

```
| `kairos_technical_score_v1` -> `scoreTechnicals` | Hand-set integer constants (+15 EMA50, +10 EMA20, ?25 RSI anchors), authored by a human. Not estimated |
```

A fold cannot have a training segment when there is nothing to train. Declaring `walk_forward` would mean building train/test boundaries, a purge rule against the train edge, and a refit step that would all be inert - machinery that implies a protocol the system does not actually run. That is the same class of overclaim as the `walk_forward_pass` field this feature already renamed.

What this does NOT excuse

Hand-set constants are still researcher degrees of freedom. +15 for EMA50 was chosen by a person who had seen this market. That is selection bias - but it is bias in the trial family, which Open Decision #3 (false-discovery procedure) governs, not bias the fold protocol can repair. No train/test split fixes a constant a human tuned by eye before the split existed.

Two consequences to carry into the frozen plan:

- The trial family must count human tuning history, not just machine variants.

`variants_run` currently counts engine runs only.

- `kairos_technical_score_v1` is not a priored factor. `mom_12_1`, `high_52w_proximity` and `vol_adj_mom_6m` carry published citations (Jegadeesh-Titman, George-Hwang); the in-house composite does not. The `T_HURDLE = 2.0` comment justifies itself as "pried-factor standard", which is defensible for the published set and not for the composite. Harvey/Liu/ Zhu argue ~3.0 for a newly proposed factor. Recommend a per-edge hurdle: 2.0 for cited priors, 3.0 for in-house constructions.

Guard required

`purged_temporal_oos` is correct today and must not silently persist if that changes. The genome (`entry_threshold`, `entry.rank_pct_min`, the five dimension weights) is fitted - `LearnerAgent` mutates it from realized outcomes. It does not touch `edge_ic_history` today because IC is computed per raw edge and the genome acts downstream on `analyst_score` and `PaperTrader`. If a composite whose parameters come from `LearnerAgent` ever becomes a validated edge, that edge requires `walk_forward` and must not be allowed the simpler mode.

Step 4 should therefore record, per experiment, whether the formula carries any data-derived parameter, and the gate must reject `purged_temporal_oos` when it does. The `validation_mode CHECK` already accepts both values, so this is a plan-level assertion rather than a schema change.

Decision requested

Approve `purged_temporal_oos` as the mode for the current registry, with:

- the per-edge t-hurdle split (2.0 cited / 3.0 in-house), and
- the data-derived-parameter guard forcing `walk_forward` when it ever applies.

This unblocks step 4. #3 and #4 remain open and gate steps 8-9 only.

Annex E - Step 4 progress and a new constraint (2026-07-28)

Approved 2026-07-28: 1C sample floors, 5A/5B PIT universe, `purged_temporal_oos` mode, the 2.0/3.0 per-edge t-hurdle split, and the fitted-parameter guard.

Shipped

lib/edges/pit-universe.ts - resolves which symbols were tradeable and liquid ON an as-of date, using only information knowable then. 17 tests.

It fails closed on every path, with a named reason and never a fallback to the curated current-liquid list:

```
| Reason | When |
| `universe_not_point_in_time` | market ? us - Massive carries no NSE listings, so India has no PIT
source until 5D |
| `liquidity_not_available_for_date` | as-of date outside the ~2y aggregate entitlement (checked before any
network call) |
| `membership_incomplete` | the page walk truncated - see below |
| `liquidity_unavailable` | grouped aggregates returned nothing usable |
| `universe_below_min_symbols` | fewer eligible liquid names than the 1C floor |
| `provider_unconfigured` | no API key |
```

Migration 20260728120000_pit_universe_provenance.sql written, NOT APPLIED (Supabase MCP unavailable at the time). It extends edge_universe_members rather than adding a parallel table: is_point_in_time (default false, so every pre-existing row correctly declares itself non-PIT), membership_source, pit_policy_version, active_on_as_of, delisted_at, adv_value, adv_rank, snapshot_fingerprint. No shipped code reads these columns yet, so nothing is schema-coupled ahead of the migration.

Defect caught during verification

The first implementation walked pagination and broke on any failed page. A real 429 was hit while verifying against the live provider - the membership set is ~10k tickers, so a full walk is ~10 requests and the plan rate-limits.

That would have produced a smaller universe that looked entirely valid: fewer names, a different fingerprint, a different IC, and no error anywhere. A truncated walk is not a smaller universe, it is an unknown one. fetchPitMembership now returns { tickers, complete, pages } and the resolver refuses on complete: false. Two tests cover it.

NEW CONSTRAINT - 1C's fold floors are not reachable yet

Measured entitlement boundary (verified 2026-07-28):

```
| As-of date | Grouped daily aggregates |
| 2026-07-24 | OK - 12,410 tickers in one call |
| 2024-10-15 | OK - 10,704 |
| 2023-06-30 | **NOT_AUTHORIZED** |
```

Membership resolves at any date; liquidity does not. So the usable OOS window is ~2 years ? 500 sessions.

Non-overlapping at h20 that is ~25 as-of dates.

1C approved 3 folds x 12 dates = 36. Short by ~11.

Options - I am not picking one, because Annex A explicitly recommended against relaxing statistical floors to fit a provider limit:

- 3 folds x 8 dates = 24. Meets fold count, breaks the 12/fold floor.
- 2 folds x 12 = 24. Meets the per-fold floor, breaks the 3-fold minimum.
- Wait. Each further month adds ~1 as-of date at h20; reaching 36 takes ~11 months.

- Find another whole-market liquidity source with deeper history. Yahoo is

per-symbol, so ranking ~10k names per date is not viable through it; this needs a grouped/bulk endpoint.

- Rank liquidity once per fold rather than per as-of date. Liquidity is

slow-moving, so a fold-start rank is defensible and cuts the calls ~12x - but it does not extend the 2-year entitlement, so it does not add dates.

Note options 1 and 2 both land on 24 usable dates. The honest reading is that the current entitlement supports roughly one 1C-shaped experiment, not three folds at full width.

Not done

- Migration not applied; persistence of snapshots to `edge_universe_members` not written (blocked on the migration).

- Input-eligibility enforcement beyond the universe (feature availability per as-of date) not started.

- Steps 5-10 unchanged. Promotion remains dormant.

- Open Decisions #3 (false-discovery) and #4 (cost-adjusted test) still open, gating steps 8-9.

Annex F - The 25-vs-36 question was the wrong question (2026-07-28)

The framing error

Annex E presented "~25 available vs 36 required" as a shortfall needing a relaxed floor. 36 was never a statistical requirement. It was the arithmetic product of a fold partition (3 x 12) that Annex A chose without deriving.

Under the approved `purged_temporal_oos` mode there is no training segment (Annex D: nothing in the edge pipeline is fitted). So folds do not partition data for leakage control - they exist only as diagnostics: fold sign consistency and a worst-fold guard. The primary statistic is the aggregate HAC t-stat over the concatenated OOS date-level IC series.

Statistical power therefore depends on N total as-of dates, not on how they are partitioned. Asking "3x12 or 2x12 or 3x8" was asking about the wrong number.

The number that actually binds

Detectable IC = $T * \sigma / \sqrt{N}$, against the approved 0.04 IC floor:

Hurdle N needed to detect 0.04 Available (~2y) Verdict
T = 2.0 (cited priors) **12.6 dates** ~25 **2x margin - sufficient today**
T = 3.0 (in-house) **28.4 dates** ~25 short by ~4 dates (~4 months)

The approved IC floor (0.04) sits above the detection limit at N=25 (0.028 at T=2.0). The floor binds first, not the sample size. That is the correct relationship - the floor should be the constraint, since it encodes what counts as economically meaningful rather than merely measurable.

Recommendation

- Fold layout: 3 folds x 8 dates = 24. Three folds preserves the

sign-consistency diagnostic; 8 per fold is fine because per-fold t-stats are diagnostics, never the promotion statistic. No floor is being "relaxed to fit a provider limit" - the 12/fold figure was never derived.

- Cited-prior edges proceed now (`mom_12_1`, `high_52w_proximity`, `vol_adj_mom_6m`, `rel_strength_6m` - the ones carrying published citations).

- In-house edges are refused until $N \geq 29$ - `kairos_technical_score_v1`

and friends carry the 3.0 hurdle per the approved split, and 25 dates cannot detect a 0.04 IC at that hurdle. ~4 months of accumulation, or they simply never qualify. Both are acceptable outcomes; silently running them at 25 dates is not.

- Do not wait 11 months. That recommendation in Annex E was downstream of the 36-date error.

THE ASSUMPTION THIS ALL RESTS ON - measure it first

Everything above uses $\sigma = 0.071$, the *theoretical* cross-sectional rank-IC standard deviation at $n=200$ ($1/\sqrt{n-3}$). The measured sigma on the current windows was previously described as 0.438. That value is actually a HAC-implied proxy, not a sample sd, and its excess cannot be attributed to overlap alone.

The whole plan assumes that removing overlap (step = horizon) collapses sigma toward theoretical. Sensitivity:

True sigma	N needed for IC=0.04 @ T=2.0	Years @12.5/yr
0.071 (assumed)	12.6	1.0
0.090	20.2	1.6
0.100	**25.0**	**2.0 - BREAK-EVEN vs what we have**
0.150	56.2	4.5
0.200	100.0	8.0
0.438 (HAC-implied legacy proxy)	479.6	38.4

The recommendation holds only if realized sigma $\leq \sim 0.10$. Above that, $N=25$ is not enough and the answer changes materially; at anything near 0.438 the entire IC-on-40-to-200-names approach is dead regardless of entitlement.

So the first output of the fold engine must be the realized sigma of the non-overlapping OOS IC series, reported before any promotion decision is evaluated. If it lands above ~ 0.10 , stop and re-derive rather than proceeding on these floors. This is cheap to measure and it invalidates or confirms the plan in one run.

Open Decisions status

#1 answered (1C). #2 answered (Annex D). #5 answered (5A/5B, US only). #3 (false-discovery) and #4 (cost-adjusted test) remain open, gating steps 8-9.

Annex G - Fold engine shipped; migration was NOT applied (2026-07-28)

Migration verification FAILED

Reported as applied. It is not. Verified through PostgREST with the app's own service-role credentials, with a control:

<code>select=symbol</code>	<code>-> [{"symbol":"SOXL"}]</code>	(table reachable)
<code>select=is_point_in_time</code>	<code>-> 42703 "column ... does not exist"</code>	(NOT applied)

Same result for `pit_policy_version`, `snapshot_fingerprint`, `adv_rank`, `delisted_at`. Migration `20260728120000_pit_universe_provenance.sql` is in the repo and not in the database - exactly the failure mode the standing rule describes. Snapshot persistence remains blocked and no shipped code reads those columns.

Shipped - `lib/edges/folds.ts`

Pure: no network, no database, no clock. 14 tests.

`buildPurgedFolds` lays out disjoint folds over market SESSIONS. Purge is structural, not advisory: fold $k+1$ starts only after fold k 's last as-of date has had `horizonSessions` to mature, so no fold's label window can reach into the next fold's features.

Refusals, all fail-closed:


```
| Reason | Why |
| `step_below_horizon` | `step < horizon` means consecutive as-of dates share forward-return windows - the exact defect that makes the legacy windows unusable. Refused by construction rather than corrected afterwards. |
| `insufficient_sessions` | Refuses rather than emitting a short final fold, which would be a smaller sample masquerading as a peer of the others. |
| `invalid_plan` | Non-positive horizon/fold/date/step. |
```

`validateFoldDisjointness` independently re-derives non-overlap from the indices. Per the architecture's "a boolean is not proof", the gate must recompute this rather than trust a flag the builder set about itself. It is tested against a hand-built violation the builder cannot produce.

`aggregate0osIc` returns the concatenated-series statistics, reporting `sigmaIc` and `sigmaWithinPlan` alongside `tHac`.

`neweyWestSEofMean` was exported from `lib/edges/ic.ts` rather than reimplemented - one Newey-West implementation, not two.

The approved layout fits

3 folds x 8 dates at `step = horizon = 20` needs 483 sessions ? 1.9 years, inside the ~500 sessions the 2-year liquidity entitlement allows. Confirmed by test, not by arithmetic in a document.

Note `neweyWestLag(20, 20) = 1`: with non-overlapping as-of dates there is no mechanical autocorrelation left to correct. That is the point of `step = horizon`, and it is the mechanism by which `sigma` is expected to fall from the measured 0.438 toward theoretical.

The stop condition is now executable

`SIGMA_PLAN_CEILING = 0.10` is in code with `sigmaWithinPlan` computed against it. Annex F made every approved floor conditional on realized `sigma`; that condition is no longer prose. A caller must check `sigmaWithinPlan` before treating `tHac` as meaningful, and both tests for it are present - one series inside the ceiling, one above it.

Not done

- Snapshot persistence - blocked on the migration.
- The runner that fetches PIT universes and candles per as-of date and produces the per-date IC series the aggregator consumes.
- Steps 7-10. Promotion remains dormant.
- Open Decisions #3 and #4 still open, gating steps 8-9.

Annex H - Migration applied and verified; OOS runner shipped (2026-07-28)

Migration 20260728120000 - APPLIED, verified independently

Supabase MCP was unavailable, so it was applied through the Management API (POST `/v1/projects/{ref}/database/query`, HTTP 201). `urllib` was blocked by Cloudflare (403/1010); curl succeeded.

Verified afterwards through PostgREST with a control column, the same check that caught the earlier false "applied" report:

```
| Column | Result |
| `symbol` (control) | OK |
| `is_point_in_time`, `membership_source`, `pit_policy_version`, `active_on_as_of`, `delisted_at`, `adv_value`, `adv_rank`, `snapshot_fingerprint` | all OK |
```

The honesty default behaved as designed: 805 of 805 pre-existing rows are `is_point_in_time = false`, zero true. Every legacy row now declares itself a survivorship-biased snapshot rather than passing as evidence.

Shipped - lib/edges/oos-runner.ts

Turns a fold plan into the per-as-of-date IC series. 12 tests.

The ordering discipline is the entire point, and it is tested rather than asserted:

```
| Element | Sees |  
| feature | candles `<= asOf` only |  
| label | `asOf` -> `asOf + H`, and only if fully matured |  
| universe | PIT membership resolved AT `asOf` |
```

A spy edge confirms it: with monotonically rising series, the values the edge observes at day(1) are the SECOND close of each symbol, never the fifth. Had the full series leaked in, every date would have produced the same value.

Refusals, all counted rather than silently dropped:

- cross_section_below_min - a sparse date is excluded and recorded. Coercing

it to IC = 0 would assert "no predictive power" when the truth is "unmeasurable".

- ic_not_finite - a degenerate cross-section (zero label variance) is refused

for the same reason. Found by a test whose own fixture was accidentally degenerate; the guard was already correct.

- universe_unavailable - datesEvaluated + datesSkipped always equals the

planned date count, so no date can vanish unaccounted for.

Labels that have not matured by the data cutoff return null rather than a truncated return: a partially matured label is a shorter horizon wearing the same name, and mixing the two silently changes what the IC measures.

The stop condition is executable end to end

describeSigma() renders the Annex F verdict in words, including the dates that *would* be required at the realized sigma. At the legacy 0.438 it emits:

STOP: sigma=0.4380 EXCEEDS the 0.10 plan ceiling. Detecting the approved 0.04

IC floor at t=2.0 would need ~480 as-of dates, not 24.

Both branches are tested.

Not done

- Snapshot persistence to edge_universe_members (schema is now ready).

- The fetch orchestration: resolve PIT universes across the 24 as-of dates,

fetch the union of symbols' 5y candles once each, and call runOosFolds. This is the step that finally produces the real sigma number.

- Steps 8-10. Promotion remains dormant.

- Open Decisions #3 and #4 still open, gating steps 8-9.

Annex I - PRELIMINARY SIGMA MEASUREMENT (2026-07-28)

First end-to-end OOS harness run. Verdict: useful diagnostic, not promotion-grade evidence and not sufficient to void the approved plan.

Run configuration

mom_12_1 (Jegadeesh-Titman, a cited prior), US, horizon 20 sessions, step = horizon, 2 folds x 6 dates = 12 as-of dates, top-200 PIT universe, min cross-section 100, 5y Yahoo candles, as-of dates confined to the 2-year liquidity window. 255 symbols fetched, 5 missing. Zero universe errors, zero skipped dates - every planned date produced an IC.

Approximation used: per_fold membership cadence (2 walks instead of 12), so this is not fully point-in-time. The run also ranked liquidity from one session rather than a trailing ADV window and used raw Yahoo closes rather than corporate-action-adjusted prices. Those are material methodology limitations, not footnotes.

Result

```
| Statistic | Value |
| n (as-of dates) | 12 |
| mean IC | **0.0335** |
| **realized sigma** | **0.1436** |
| Newey-West SE | 0.0419 |
| **t_HAC** | **0.80** |
| NW lag | 1 (a configured HAC choice; not justified by overlap alone) |
| fold signs | [+1, +1] - consistently positive |
```

The saved artifact contains only this aggregate, not the 12 per-date IC values. The aggregate therefore cannot be independently recomputed from the artifact. The runner now emits the complete per-date series; this preliminary run must be rerun before its sigma is used.

What the run does and does not show

The legacy 0.438 value was back-derived from $\text{mean_ic} * \sqrt{n} / t_stat$. Because that t-stat used a HAC standard error, 0.438 is not a directly measured sample standard deviation and is not comparable one-for-one with 0.1436. The claim that removing overlap produced a 3.1x sigma reduction is withdrawn.

For the 12 preliminary observations, the sample standard deviation is 0.1436. A textbook chi-square interval under independent normal observations is [0.1017, 0.2437], but cross-sectional ICs are not established to be independent normal draws here. Its lower endpoint clears 0.10 by only 0.0017, so that model assumption decides the conclusion. "Confidently above 0.10" is not supported.

The difference from the iid heuristic may reflect factor structure, universe construction, corporate actions, regime concentration, or sampling error. This run does not identify the cause. The earlier factor-structure attribution is withdrawn until it is tested.

What it costs

Conditional planning arithmetic at sigma = 0.1436:

```
| IC floor | T = 2.0 | T = 3.0 |
| 0.04 (approved) | **51.5** | 115.9 |
| 0.05 | 33.0 | 74.2 |
| 0.06 | 22.9 | 51.5 |
| 0.08 | 12.9 | 29.0 |
```

These values are arithmetic conditional on a stable sigma, independent effective dates, and the stated hurdle. They are not a new sample-size contract.

The preliminary mom_12_1 point estimate does not pass the promotion hurdle: mean IC 0.0335 and t_HAC 0.80. That is a diagnostic refusal, not proof that the edge is ineffective or unreachable under a corrected, adequately sized design.

Honest reading

The fail-closed outcome is right: promotion remains dormant. The stronger claims that the machinery is fully correct, the overlap hypothesis is confirmed, or no realistic promotion path exists are not established by this run.

Options, none chosen

- Rerun with adjusted prices, full per-date output, and the latest eligible session window.
- Replace one-day dollar volume with a trailing PIT liquidity measure.
- Use per-date membership for any evidence that could reach promotion.
- Predeclare the HAC lag/sensitivity analysis and minimum number of dates.
- Only then compare universe sizes, horizons, and edge families under immutable experiment lineage.

Immediate consequence

The observed point estimate exceeds $\text{SIGMA_PLAN_CEILING} = 0.10$, so it raises a planning warning. It does not void the approved floors. Promotion stays dormant because the run is approximate, too small, not independently recomputable, and not fully PIT.

Open Decisions #3 and #4 remain open, but they are moot until the sample-floor question is resolved.

Annex L - Immutable bounded OOS runs (2026-07-29)

The first two runs under the immutable manifest contract completed in production. Both used code `cb538a77b8572ba76739faffc321cec9990ddb27`, adjusted candles bounded at 2026-07-28, persisted per-date PIT membership, and no approximations. They are evidence collection only. Promotion remains dormant.

Experiment	Variant	Dates	Mean IC	Sigma	HAC t	Universe errors
d61c4990-f03a-4b0d-919f-a3d4e604428b	h5	n=200	3	-0.1934	0.4404	-0.93 0
same dates/snapshots	n=400	3	-0.1678	0.4323	-0.82	0
b7a1eba5-70eb-4d48-8f76-6e16582b431b	h20	n=200	3	-0.0902	0.4681	-0.44 0

The matched h5 result does not support the claim that increasing the cross-section from 200 to 400 will normalize temporal IC dispersion: sigma changed by only 0.0081 on the same dates. The direct h20 result is also noisy, so h5 dispersion must not be transferred to h20. Three dates are far below a decision-grade sample and the recent negative means do not establish that the factor is permanently dead. The correct next action is to accumulate more predeclared PIT dates and test sector-neutral IC before any strategy-return or promotion work.

Annex K's corrected 80-date raw-rank rerun later measured pooled h5 sigma 0.2639. This does not conflict with the three-date 0.4404: the latter is a very wide recent-window estimate. The rerun does not support winsorization as the explanation, although the old artifact lacks enough universe provenance for a clean method-only comparison.

Plan fingerprints:

- h5 matched family:
43775a8d831304c5fc3552e3c2d3c3b134e9ad4bd6f0052bc4b4df8eb3bb64ae
- h20 family:
59d2f383e546bd2e27f718a35f0206fc83eec004100679ec843c6ef94b621ae0

Annex J - 2026-07-28 adversarial review blockers

The following are hard blockers before promotion can be enabled:

- Experiment lineage implementation (completed 2026-07-29).

`backtest_experiments` now binds `edge_id`, evidence horizon, formula version, trial family, universe policy, cutoff, code version, the canonical immutable plan hash, and write-once realized universe/dataset/run hashes.

`runBoundOosVariant()` constructs the provider run from that manifest and enforces the data cutoff. The dormant promotion boundary still needs to independently verify these identities before it may ever be enabled.

- Liquidity is not trailing ADV. `resolvePitUniverse()` currently ranks one

session's adjusted close times volume. The approved contract says trailing PIT dollar volume. An event-volume spike can therefore enter the universe and alter IC/sigma. Build a cached trailing window or rename and reapprove the selection policy; do not present the current rank as ADV.

- Promotion evidence requires per-date membership. `per_fold` is a quota

approximation and must remain diagnostic-only. No promotion writer may accept a report carrying that approximation.

- HAC lag selection is not derived. `neweyWestLag(20, 20) = 1` is a policy

choice, not a consequence of non-overlapping labels. Predeclare lag selection and report lag sensitivity before setting a statistical gate. Newey and West explicitly treat lag selection as an estimation problem, and note finite sample size distortions in their simulations.

- Fold partition is conditionally irrelevant only. Aggregate mean/sigma on

a fixed concatenated series do not depend on fold labels, but purge width, entitlement depth, fold consistency, and worst-fold gates change the usable dates and decision. The earlier unconditional wording is withdrawn.

Primary references:

- Newey, W.K. and West, K.D. (1994), "Automatic Lag Selection in Covariance

Matrix Estimation," *Review of Economic Studies* 61(4), 631-653, <https://doi.org/10.2307/2297912>

- Bailey, D.H., Borwein, J., Lopez de Prado, M., and Zhu, Q.J. (2015), "The

Probability of Backtest Overfitting," *Journal of Computational Finance**, <https://ssrn.com/abstract=2326253>

Annex K - Multi-period h5 dispersion diagnostic (corrected 2026-07-29)

The original run applied 2%-tail winsorization before Spearman. That was unnecessary because Spearman already ranks observations, and clipping creates artificial ties. The design was rerun with raw finite values under immutable diagnostic experiment 2b7cf834-b516-48bd-9533-d2511229e7f6, fixed cutoff 2026-07-28, persisted fold-anchor snapshots, and code 88c963f14ef6e8651076055b774b10628b844771.

The corrected run measured pooled sigma 0.2639, close to the old 0.2665. This does not support winsorization as the explanation for Annex L's three-date sigma of 0.4404; the shorter, recent date sample is the material design difference. This is not a clean one-variable comparison because the old artifact omitted its members and its four universe fingerprints differ from the newly persisted trailing-ADV snapshots. The permanent closure and `n_eff` ? 17 inference remain superseded in `PROJECT_DECISIONS.md`.

Design: `mom_12_1`, US, horizon 5 (step 5), 4 folds x 20 dates = 80 as-of dates, universe held at 200, `per_fold` cadence, adjusted prices. 320 symbols, 0 universe errors, 0 skipped dates.

Horizon 5 rather than 20 because the ~2-year entitlement holds ~100 as-of dates at h5 versus ~25 at h20 - four

independent periods of 20 instead of four of 6. This measures h5 temporal dispersion. It does not estimate h20 dispersion

or decompose the sources of variance.

Result 1 - h5 dispersion is high and varies across folds

Fold	Period	mean IC	sigma
0	2024-12-13 .. 2025-05-05	-0.0124	0.2286
1	2025-05-13 .. 2025-09-29	+0.0331	0.2254
2	2025-10-07 .. 2026-02-24	-0.0069	0.2818
3	2026-03-04 .. 2026-07-21	+0.0423	0.3235

Pooled n=80: mean IC 0.0140, sigma 0.2639, HAC t 0.51. Fold sigma spread: 0.2254-0.3235.

The four estimates provide a useful range, but they rise monotonically and each uses only 20 dates. They do not prove stationarity, and they cannot explain whether the movement comes from market regimes, changing cross-sections, membership cadence, or sampling noise. The earlier h20 estimates remain too small to settle h20 dispersion.

Result 2 - effective breadth is not identified

Realized pooled sigma 0.2639 against a theoretical 0.0712 at n=200.

That comparison is diagnostic only. $1/\sqrt{t(n-3)}$ is not an estimator that can turn observed time-series variance of Spearman IC into independent-name count. Observed variance includes cross-sectional sampling error, time-varying true IC, changing membership, and data attrition. The raw artifact also omitted successful-date cross-section counts, so those causes cannot be separated retrospectively.

The previous $n_{\text{eff}} \geq 17$ claim and the derived 180-date requirement are withdrawn. Test n=200 versus n=400 on the same dates and memberships before claiming how dispersion scales. This is a measure-only diagnostic, not a reason to enable promotion.

Statistical basis for the correction:

- Ornstein and Lyhagen (2016), *Asymptotic Properties of Spearman's Rank

Correlation for Variables with Finite Support*, derives dependence-specific asymptotic variance rather than treating the independence-null variance as a general effective-sample-size estimator: <https://doi.org/10.1371/journal.pone.0145595>

- Moskowitz and Grinblatt (1999), *Do Industries Explain Momentum?*, documents

a strong industry component in stock momentum. That supports measuring a sector-neutral specification; it does not justify predicting its result without running it: <https://doi.org/10.1111/0022-1082.00146>

Result 3 - mom_12_1 shows no signal at h5

Fold signs [-1, +1, -1, +1] - alternating across four independent periods. Pooled mean IC 0.0140, t_HAC 0.51.

Not weak-but-positive. No consistent sign at all. (The h20 runs were [+1, +1], so this is horizon-specific and does not condemn the factor at 20 sessions - but it does mean h5 carries nothing.)

What this settles

- mom_12_1 is not useful at h5 in this sample.
- Daily h5 IC is much noisier than an independent-name null benchmark.
- The original fixed sample floors are not validated by this run.
- Promotion must remain disabled.

What it does NOT settle

- h20 sigma or power; h5 cannot be transferred to h20.
- Effective breadth or the contribution of correlation versus factor

non-stationarity and changing coverage.

- Whether $n=400$ reduces sampling variance on matched dates.
- Whether point-in-time sector-neutral IC improves stability.
- Whether a *different* edge has signal. Only mom_12_1 was run.
- per_fold cadence remains an approximation - this is a planning measurement, not promotion evidence.

Consequence

Promotion stays disabled. Continue only bounded, predeclared measure-only experiments. Record per-date cross-section and provenance, compare matched universes, and do not derive production thresholds from this single edge and horizon.

webull broker - Feature Architecture

Source: features\webull-broker\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Feature Architecture - Webull as 3rd Broker (MCP)

Status: Phase 1 (read-only, Cloud MCP) BUILT - OAuth connected; live read capture still fail-soft if a configured tool is not offered by the connected MCP surface.

Decision: Cloud MCP (OAuth), not OpenAPI - the OpenAPI REQUIRES an IP

whitelist (no free serverless has a static egress IP; would force an Oracle

Free VM), whereas the MCP's OAuth is not IP-locked and runs from Vercel free.

Reuses the Robinhood MCP OAuth machinery.

Last updated: 2026-07-13

Refactored 2026-07-13 onto the config-driven MCP broker framework - Webull

is now a registry entry, not bespoke code (see "Config-driven framework" below).

Reviewer correction 2026-07-13: the currently connected Codex Webull MCP

surface exposes account/balance/position/order-history style read tools, but not

the quote/research/order-placement tools listed in the original architecture.

Treat quote/research/order tool names as unverified until tools/list in the

deployed app proves them. The order adapter must remain unreachable.

Config-driven framework (2026-07-13 refactor)

The bespoke lib/webull-mcp.ts was generalized so future OAuth-MCP read-only brokers (E*TRADE, other US/India) are added by a CONFIG ENTRY, not new code:

- lib/brokers/mcp-registry.ts - McpBrokerConfig type + MCP_BROKERS registry

(id/label/market/currency, mcpUrl, oauth endpoints+scopes, vaultProvider+keys, tool names, and response field-name candidates). Seeded with the webull entry (same endpoints/scopes/tools/vault keys - behaviour-neutral).

- lib/brokers/mcp-driver.ts - the generic driver: every fn takes an

McpBrokerConfig - getOrRegisterClient / buildAuthUrl / exchangeCode / getValidAccessToken (CAS refresh) / hasToken / mcpRpc / captureAccounts (driven by cfg.tools + cfg.fields -> BrokerAccount[]) / checkTokenHealth / disconnect / saveOAuthState+consumeOAuthState (keyed by broker id). Reuses makePkce/makeState/signOAuthCookie/verifyOAuthCookie/mcpToolJson from lib/robinhood-mcp.ts.

- Generic routes app/api/broker-mcp/[broker]/{login,callback,status,disconnect}

resolve the config from MCP_BROKERS[broker] (404 if unknown). Login/status/ disconnect owner-gated; callback verifies the single-use server-side state.

- Settings iterates MCP_BROKERS -> one connect card per broker (future brokers

auto-render). The legacy `app/api/webull/*` routes are thin 307 redirects to the generic ones (preserving the DCR-registered `/api/webull/callback` redirect uri).

- `lib/webull-mcp.ts` was deleted - nothing imported it after the repoint.

Phase 1 built (read-only)

- OAuth 2.1 (DCR + PKCE S256, hosted Vercel callback) + CAS token refresh (vault

keys `WEBULL_MCP_*`, provider `webull_mcp`) + JSON-RPC client + `captureAccounts()` (`get_account_list` -> `get_account_positions` + `get_account_balance` -> `get_stock_quotes`), now via the generic `lib/brokers/mcp-driver.ts` + `MCP_BROKERS.webull`. Scope `account:read market:read instrument:read` ONLY. If the connected MCP surface omits `get_stock_quotes`, capture fails soft rather than fabricating prices. No `order:write` scope is requested by the login route.

- Legacy routes `app/api/webull/{login,callback,status,disconnect}` -> 307 redirects to `app/api/broker-mcp/webull/{login,callback,status,disconnect}`.

- Settings connect card (generic, driven by the registry). `BrokerName` union + `webull`.

- Owner action: already connected once through cloud OAuth. Verify holdings

appear after the next live snapshot, and investigate only read-tool gaps.

- Phase 2 (later): order adapter (`order:write` + `preview`->`place`->`status`->`cancel`)

behind an explicit per-account allowlist + all existing gates. Code exists only as an inert adapter scaffold and is not reachable while `orderCapable=false`, `webull_orders_enabled` is absent/false, and no Webull trading allowlist row is configured.

Confirmed integration facts (from public OAuth metadata + Webull MCP docs)

Cloud MCP endpoint: `https://api.webull.com/mcp` (JSON-RPC, MCP spec). Auth = OAuth 2.1 + PKCE(S256) + dynamic client registration - identical shape to the Robinhood MCP already implemented in `lib/robinhood-mcp.ts`, so those helpers are reused with Webull's endpoints:

- authorize: `https://passport.webull.com/oauth2/ai-mcp/login`
- token: `https://u1suserauth.webullfintech.com/api/userauth/oauth/token/token`
- register: `https://u1suserauth.webullfintech.com/api/userauth/oauth/client/register`
- public client (`token_endpoint_auth_method: none`), grants `authorization_code`+`refresh_token`
- scopes: `account:read order:read order:write market:read instrument:read`

Tool schema status: partly verified, order placement unverified.

Current connected tool exposure in Codex on 2026-07-13 included: `get_account_list`, `get_account_balance`, `get_account_positions`, `get_order_detail`, `get_instruments`, `get_open_orders`, `get_order_history`. It did not expose `get_stock_quotes`, `get_stock_snapshot`, `preview_stock_order`, `place_stock_order`, `cancel_order`, or the analyst/ financial research tools. The table below is therefore a desired/previous-doc mapping, not proof that orders can be placed from this app today.

Need	Webull MCP tool
accounts	<code>`get_account_list`</code>
balance / buying power	<code>`get_account_balance`</code>
holdings	<code>`get_account_positions`</code>
quotes (pricing)	<code>`get_stock_quotes`</code> / <code>`get_stock_snapshot`</code>
review order	<code>`preview_stock_order`</code>
place order	<code>`place_stock_order`</code>
order status	<code>`get_order_detail`</code>
open orders	<code>`get_open_orders`</code>


```
| cancel | `cancel_order` |
```

Only the account/balance/position/order-read portion is currently verified in this Codex connection. Quotes and order placement must stay fail-soft/unreachable until a deployed `tools/list` proves the tools and schemas.

Goal

Add Webull as a third broker - US equities, parallel to Robinhood - using the existing broker abstraction (`BrokerAdapter` + registry). Same safety stack; the adapter only executes, every gate sits above it.

Why it fits cleanly

`lib/brokers/adapter-types.ts`: "adding an execution broker = one new file implementing this interface + one registry entry. Zero route/UI changes." Webull is US (like Robinhood), and it's an MCP - so the Robinhood-MCP integration is the template, reused almost verbatim with a different endpoint + tool names.

Build (phased - read-only first)

Phase 0 - Discovery (interactive; blocks the rest)

The exact OAuth endpoints + tool names come from the live server, not guesswork. Standard MCP auth = OAuth 2.1 + dynamic client registration via a `.well-known/oauth-authorization-server` discovery doc - the same flow the RH integration already implements (`lib/robinhood-mcp.ts`: DCR -> PKCE S256 -> token in vault -> CAS refresh). Steps:

- Connect the Webull MCP (interactive OAuth) - owner authorizes.
- Call `tools/list` to capture the real tool names (accounts / positions / quotes / review-order / place-order / order-status / cancel).
- Pin those into the client. No adapter code is written against guessed names.

Phase 1 - Read-only (dashboard)

- `lib/webull-mcp.ts` - generic `webullRpc(token, method, params, session)` (copy of RH's `mcpRpc`, endpoint `https://api.webull.com/mcp`) + `captureWebullAccounts()` -> `BrokerAccount[]`.
- `broker_accounts` row(s): `broker='webull', market='us', account_role, enabled, notional_cap_usd, agentic_allowed=false` (reads only).
- Wire into the live account book (like the 6 RH accounts): holdings, risk, the live-vs-VOO chart, `live_account_snapshots`.
- OAuth routes `/api/webull/login` + `/api/webull/callback` - reuse the RH OAuth helpers (dynamic client reg, PKCE, signed state cookie, vault token, CAS refresh). Owner-gated login; callback verifies the state nonce.
- Settings: Webull connect card + account list (mirror the RH-MCP card).

Phase 2 - Order-capable (only if opted in)

- `lib/brokers/adapters/webull.ts` is an inert adapter scaffold. It must not be enabled until live tool discovery proves `preview/place/status/cancel` schemas, Webull order scopes are intentionally requested, and the owner approves.

- An account becomes order-capable ONLY when you explicitly allowlist it
(`broker_accounts{broker='webull', market='us', role='trading'}`) and the broker registry marks Webull order-capable.
- Every existing gate applies unchanged: `isTradingEnabled(svc, 'us')`,
per-market controls, kill switch, notional caps, human click + confirm.

Safety (unchanged)

- Read-only first; orders are a separate, opt-in phase.
- One allowlisted account for orders; all others read-only.
- Credentials never in code/logs - vault only; owner enters/authorizes.
- Same fail-closed order path as RH (dup-index, durable ACK, reconcile).

Two decisions needed to start

- Scope: read-only (holdings on the dashboard) first - recommended - or go straight to order-capable?
- Discovery: how do I get its tool schema - (a) you add the Webull MCP as a connector to this app so I can `tools/list` it, or (b) you paste the Webull MCP auth + tool docs? (I can't run the interactive OAuth from here.)

webull trading api - Feature Architecture

Source: features\webull-trading-api\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Webull Trading API Adapter

Status: transport implementation approved 2026-07-19; disabled and not approved for activation. US live money path.

2026-07-19 Transport Restoration Decision

The signed HTTP sender may be implemented now, but remains unreachable from the broker registry and must not make a real request during build or test. The activation flag stays false and the adapter continues to report unconfigured.

The current `api_key_vault` schema has no environment column. Environment isolation therefore uses three distinct key names per environment under `provider='webull_trade'`: `WEBULL_TRADE_<ENV>_APP_KEY`, `WEBULL_TRADE_<ENV>_APP_SECRET`, and `WEBULL_TRADE_<ENV>_ACCESS_TOKEN`. This is the existing vault convention and avoids an unapproved schema migration.

There is one network implementation and two capabilities over it:

- A one-shot preflight permit reads and requires the false-by-default database feature flag and can call only the official token-check, account-list, or order-preview paths. It cannot place, query, or cancel an order. This resolves the unavoidable ordering dependency: live token status must be known before gate 7 can pass.

- A one-shot order permit is created only from a full `GateSnapshot` for which all nine gates pass. It permits only the fixed order endpoint allowlist and is consumed by the first request, preventing stale gate clearance from becoming a reusable transport handle.

The current hardcoded account gate names the production account only, so order permits are production-only. Sandbox token/account/preview proof is supported, but a sandbox place/cancel proof remains blocked until its exact test account has a separately approved allowlist gate; production account identity must never be reused as a proxy for a sandbox account.

Both capabilities use the same `liveWebullTransport()` sender and the same sole `fetch` call. Environment and host are pinned to the credential; signing uses the hostname without the URL scheme, as required by Webull. Every request gets a new nonce and timestamp, a 10-second default timeout, the exact signed body bytes, and `x-access-token`. HTTP and network failures return fixed, redacted `TransportResult` errors and never throw credential material.

The official token-status endpoint is `POST /openapi/auth/token/check`, not the previously proposed `GET /openapi/trade/token/query`. A successful `NORMAL` response produces the fresh `WebullTokenRecord` consumed by gate 7. No keepalive schedule is added.

Sandbox proof sequence (not yet authorized)

- Provision sandbox-only vault keys and keep the production keys absent.
- With the database feature flag enabled only for the supervised proof window, call token-check, then `GET /openapi/account/list`. Pass only if status is `NORMAL`, exactly the expected test account is returned, and no production account identifier appears.
- Call `POST /openapi/trade/order/preview` with a one-share US equity limit fixture. Pass only if the broker accepts the account/order shape and returns a non-mutating estimate.
- Add a separately approved exact sandbox-account gate before enabling any sandbox place/cancel capability. The production account constant is not valid proof of sandbox identity.

- With owner approval, submit one non-marketable one-share limit order that is inside Webull's accepted price bands, query it, cancel it, and query the terminal state. Do not use \$0.01: a broker price-band rejection proves only rejection handling, not place/query/cancel reconciliation.
- Pass only if one client order id maps to one broker order, no fill occurs, the cancel becomes terminal, the audit/reconciliation record is complete, and no secret appears in logs. Any timeout or unknown state stops the sequence for reconciliation; it is never resubmitted.

Resolved Boundary

Webull exposes two different products:

- Cloud MCP is an AI-friendly, query-only surface. Kairos may use it for account, position, balance, and quote reads. It has no configured order scopes or write tools and `orderCapable` remains false.
- Trading API is a separately entitled, signed REST/gRPC integration. It is the only candidate for future Webull order placement.

The two transports, credentials, scopes, and registries must never be combined. The existing `webull` MCP adapter remains read-only. A future trading adapter gets a distinct id such as `webull_trade`.

Verified Trading API Facts

- Individual applications require Webull approval and API credentials.
 - Access tokens are reusable; 2FA (when enabled) is verified via the Webull app
- ONCE at token acquisition, NOT a secret pasted into each order - so autonomous cron execution is technically feasible (owner Q1 RESOLVED).

- The official token lifecycle documents `INVALID` after 15 consecutive days without an API call, `EXPIRED` when initial verification is not completed within 5 minutes, and reusable `NORMAL` tokens. Kairos tracks the last confirmed authenticated call, checks token status before order activity after an idle interval, and alerts before the 15-day boundary. It must not generate meaningless keepalive traffic merely to hide an unused credential. Invalid, expired, or unknown status fails closed before submission.

- Requests use Webull's documented signed authentication protocol, including its canonical request and HMAC-SHA1 signature. Do not substitute generic OAuth or a vendor SDK assumption.

- Stock order endpoints support `market/limit/stop` variants, client order ids, order query/cancel/replace, and documented rate limits. The Trading API also documents richer types - `GTC` `stop-market/stop-limit`, `OTO/OCO/OTOCO`, attached take-profit/stop-loss, fractional, extended/overnight sessions, and `TWAP/VWAP/POV` algos. Documented ? in scope. These are pinned to fixtures only if/when a later phase needs them; the initial adapter must NOT expose them.

- Initial order-type scope (locked small): `market + limit` for manual-approved entries, and `GTC STOP_LOSS` in `CORE` as the disaster floor only (the hybrid-stop feature). `OCO/OTOCO` come only after the basic place/query/cancel/reconcile lifecycle is proven. Trailing stops, shorts, options, algos, and automatic venue routing are excluded outright.

- Trailing-stop stock orders are documented with `DAY` time-in-force only - they

cannot rest multi-day, so they are NOT a substitute for Kairos's exit policy and stay out (adopting broker-native trailing would also make exits vary by venue and contaminate the Learner - see `features/hybrid-stop`).

- Webull fills the conditional `BrokerProtectiveCapabilities` matrix from

`features/hybrid-stop`. Initial fixtures declare only US equity SELL protection using `GTC STOP_LOSS` in `CORE`. The generic presence of `ALL/NIGHT` sessions elsewhere in the API does not prove a stop order can trigger there.

- Sandbox and production are separate environments and credentials.

Exact fields and limits must be pinned from the current official documentation in contract fixtures immediately before implementation because this is a live broker surface and may change.

Recommendation

Do not enable Webull trading now. Build only after the existing paper and live decision loops, router cutover, and protective-order policy are stable. When built:

- Webull is an additional US live venue, not a new strategy.
- Deterministic Kairos logic remains authoritative.
- Native trailing stops stay out until a separately versioned strategy experiment

proves broker-dependent exit semantics are acceptable.

- Long-only is enforced in both the gateway and adapter; `SHORT` is rejected.

Credential and Transport Design

- Store production and sandbox credentials as separate encrypted vault records.

- Never expose app secret, access token, refresh material, account id, signature, or canonical request in logs, errors, UI, or model context.

- Fetch credentials through a server-only accessor with bounded in-memory caching.

- Pin endpoint environment to the credential record. A sandbox credential can never resolve a production host.

- Implement the official signing algorithm directly with golden fixtures from the

documentation. Enforce TLS, timestamp skew, and a fresh nonce per request; redact the canonical request before any telemetry boundary because it can contain account and order data even when the secret itself is absent.

- Use a bounded timeout and retry only requests proven idempotent.

Adapter Contract

Create a distinct `webull_trade` adapter implementing:

- account discovery and exact allowlist match
- position, cash, buying-power, order, and fill reconciliation
- preview when supported by the entitled surface
- place, query, modify, and cancel with a stable `client_order_id`
- normalized partial-fill and unknown-state handling
- provider rate-limit classification and health reporting

It writes only through the existing execution kernel and execution ledger. It may not create a parallel trade, cash, position, or P&L store.

The current `lib/brokers/adapters/webull.ts` is an inert scaffold for MCP-derived order tools that do not exist. During implementation it must be deleted or replaced, not enabled and not left as a second Webull order adapter. The read-only Cloud MCP registry may remain under `webull`; signed execution uses only `webull_trade`.

Mandatory Gate Ladder

Every order requires all gates in this order:

- Global trading enabled and app not paused.
- US market control enabled and kill switch clear.
- Drawdown/circuit breakers clear; exits remain possible when entries halt.
- Live autonomy/approval mode satisfied.
- Exactly one enabled Webull trading account allowlisted for US.
- Explicit `webull_trade_orders_enabled` false-by-default feature flag.
- Valid NORMAL production token when 2FA is enabled, valid signing credential, expected endpoint, and acceptable timestamp skew.
- Existing buying-power, notional, name, sector, gross, turnover, and duplicate order checks.
- Deterministic quantity no greater than the reconciled mandate allowance.

Missing schema, config, account, credential, or broker response fails closed.

Order Lifecycle

- Reserve an execution intent using the existing idempotency mechanism.
- Reconcile account and held quantity immediately before submission.
- Build a normalized order and reject short/options/unsupported sessions locally.
- Sign and submit once with a stable client order id.
- A timeout or response without an order id becomes `needs_reconcile`; never report success and never blindly resubmit.
- Reconcile order status and fills until terminal.
- Update the existing execution and position truth using actual fills.

Failure and Security Tests

- Cloud MCP configuration contains no write scope or order tool.
- Every missing gate blocks before network submission.
- Two allowlisted accounts fail closed as ambiguous.
- SHORT, options, and unsupported order/session combinations are rejected.
- Sandbox credentials cannot call production and vice versa.
- Official signature fixtures match exactly; mutated method/path/body fail.

- Secret capture assertions cover logs, errors, traces, and health events.
- Reusing a client order id cannot create a second order.
- Timeout and malformed response enter reconciliation, not retry-to-place.
- Partial fills and external cancellations preserve correct held quantity.
- US pause, kill, drawdown, approval, and notional gates are regression-tested.
- India state is unreachable from the adapter.
- A token idle for 15 days, unknown token status, timestamp replay, or nonce reuse blocks before order submission.

- GTC STOP_LOSS is rejected outside the exact session/product combinations proven by current contract fixtures.

Open Owner Decisions

- Does Webull replace Robinhood for US live execution or remain an optional second venue? Recommendation: optional manual venue first; no automatic routing.
- Which approved individual Webull account is allowlisted?
- Is the required Webull API entitlement active for that account?
- After sandbox proof, authorize one manually approved minimal live order?

Build Order After Approval

- Confirm entitlement and capture current official schemas/rates/signing fixtures.
- Add vault records and secret-hygiene tests.
- Build read/reconciliation in sandbox with no place permission.
- Build signed order operations behind all gates, still disabled.
- Run fault injection and full money-path regression tests.
- Owner reviews sandbox ledger and explicitly approves one minimal live test.
- Keep autonomous Webull routing disabled until a separate production sign-off.

Primary Sources

- Authentication and reusable 2FA token overview: <https://developer.webull.com/apis/docs/authentication/overview/>
- Token lifecycle and 15-day inactivity rule: <https://developer.webull.com/apis/docs/authentication/token/>
- Stock order lifecycle, types, TIF, and sessions: <https://developer.webull.com/apis/docs/trade-api/stock/>
- Cloud MCP query-only tool inventory: <https://developer.webull.com/apis/docs/AI-friendly-Resources/mcp/>

weekend research catchup - Feature Architecture

Source: features\weekend-research-catchup\FEATURE_ARCHITECTURE.md | Feature micro-architecture: verify lifecycle status within the document and any IMPLEMENTATION_RESULT.md

Market-Closed-Day Research Catch-up and Agent Capacity

Status: APPROVED by owner on 2026-07-19 ("do it"; holiday extension approved with "ok")

Problem

The US and India research queues can contain more symbols than one market-session ResearchAgent run can finish. Provider quotas reset on weekends and full exchange holidays, but warming alone does not finish scoring, write a safely reusable staged result, or reduce the next-session scoring workload. The app also does not show queue depth, throughput, or estimated time to clear.

Decision

Use market-closed-day quota for full deterministic research catch-up, but make every result non-executable until the same market completes a fresh session revalidation. Keep paper and live trading session-only. Display honest backlog and capacity telemetry for queue-backed and workload-backed agents.

Signal State Machine

```
stateDiagram-v2
    [*] --> weekend_staged: Closed-day catch-up scores symbol
    weekend_staged --> superseded: Newer closed-day score replaces it
    weekend_staged --> revalidated: Session ResearchAgent successfully re-scores symbol
    revalidated --> [*]
    [*] --> pending: Normal session-validated research score
    pending --> paper_traded: PaperTrader fills
    pending --> expired: Trading-day freshness expires
    pending --> rank_rejected: Cross-sectional gate rejects entry
```

weekend_staged is retained as the legacy database status name. Its semantic meaning is now any supported full market-closure day. Every such row has session_validated=false and an as_of_session equal to the last completed market session. A normal weekday score has session_validated=true. A successful weekday score marks older staged rows for that market/symbol revalidated; the newly inserted pending row is the only entry candidate.

Downstream Behavior

- PaperTrader: positive allowlist requires pending, deterministic_v1, and session_validated=true. It never fills staged rows.
- TraderAgent/live proposals: same positive allowlist. It never creates a proposal from a staged row.
- AutonomousShadow / AutonomousLive: their direct recent-signal queries also require session_validated=true; weekday scheduling and the 24-hour lookback are not treated as sufficient proof.
- PositionMonitor: mechanical price stops, targets, trailing stops, and time stops remain independent. Score/direction exits may only read session_validated=true signals; staged scores cannot force a sale.
- LearnerAgent: staged rows remain measurement evidence, but cannot become

filled-trade outcomes. Existing observation-ledger rules remain unchanged.

- ResearchAgent weekday run: holdings remain first-class and cap-exempt. A

successful re-score produces the fresh decision for that session. A failure leaves the staged row informational only.

- CapitalRotation: latest-score lookup requires session validation, so a staged score cannot change weakest-holding selection or a rotation edge.

Closed-Day Scheduling

Schedule one bounded catch-up trigger per market every day after that market's prewarm window. A shared exchange-local calendar permits work only on weekends or verified full equity-market holidays. Normal trading days self-skip. An unsupported calendar year and special sessions such as NSE Muhurat Trading both abstain. An unsupported year opens one deduplicated System Health warning until the annual calendar is installed. The route uses the existing research wall-clock budget, provider pacing, candidate cap, per-market scope, and idempotency guard. It does not chain PaperTrader or TraderAgent.

The 2026 US calendar is sourced from NYSE. The 2026 India calendar is sourced from NSE Capital Market circular CMTR71775 plus amendment CMTR72260. Settlement, currency, debt, and commodity holiday lists are not interchangeable with the equity-market calendar. Early-close days are trading days, not catch-up days.

Closed-day candidate scores are retained in `research_queue` for next-session revalidation. Held symbols do not need queue retention because holdings are always gathered. Repeated closed-day runs supersede the prior staged row rather than creating multiple active staged decisions.

Data Model

Add to `agent_signals`:

- `session_validated` boolean not null default true
- `as_of_session` date null
- `staged_at` timestamptz null
- CHECK: `status='weekend_staged'` implies `session_validated=false`
- one active `weekend_staged` row per (market, symbol)

No order, position, portfolio, or broker schema changes.

Agent Capacity View

The Agents dashboard gets a Capacity tab, scoped by the global market switcher. Each row declares its workload type so a workload is not mislabeled as a durable queue:

- ResearchAgent: `research_queue` depth, oldest age, staged count, recent median

successful throughput/day, and estimated clearing days.

- PaperTrader: fresh validated pending longs; configured per-run selection cap.
- PositionMonitor: currently open paper positions; all are expected per run.
- LearnerAgent: report unavailable when a defensible queue cannot be derived;

never fabricate a backlog.

Capacity uses the median of recent completed per-market runs where possible. Environment caps are shown as configured ceilings, not achieved throughput. `estimated_days = ceil(backlog / max(1, observed_daily_capacity))`;

null is shown when no defensible estimate exists.

Failure Rules

- Missing migration makes the new positive eligibility query return no entry candidates (fail closed).
- Closed-day scoring failure re-defers the symbol and writes the normal run error.
- Queue telemetry failure shows unavailable; it never blocks any agent.
- A staged row can never be mutated into pending; revalidation writes a fresh row, preserving point-in-time history.
- US and India remain fully separate. No queue, throughput, score, or estimate is cross-summed.

Acceptance Criteria

- Weekend and verified full-holiday catch-up writes weekend_staged signals and no trade/proposal.
- PaperTrader and TraderAgent reject staged/unvalidated signals even at score 100.
- PositionMonitor ignores staged scores for score/direction exits.
- Next-session success writes a new validated signal and closes prior staged state.
- Failed session revalidation cannot make the staged result executable.
- Closed-day candidate remains queued for session revalidation without increasing its failed-attempt count.
- Capacity UI is market-scoped and labels queue versus workload honestly.
- Existing weekday research-to-paper flow remains unchanged for validated rows.

Disable / Rollback

Unschedule the two closed-day catch-up crons. Existing staged rows remain inert and may be marked superseded. The additive columns can stay; their defaults preserve the pre-feature weekday behavior.